ML ACADEMY

Machine learning from zero

The printable workbook: Lesson 0 and all 28 lessons with maths, worked examples, code, quizzes and exercises — with space to work by hand. Answers are at the back.

HOW TO USE THIS WORKBOOK

- Read a lesson, then try its quiz and exercises in the boxes provided before checking the answers at the back.
- Interactive demos, animations and automatic answer checking live on the website (open *index.html* in the course folder); the workbook points to them.
- Every code example can be run from the lesson's Jupyter notebook in the *notebooks* folder.

Contents

1. Lesson 0: Python, NumPy & pandas for Beginners
2. Lesson 1: What Is Machine Learning?
3. Lesson 2: Vectors & Matrices
4. Lesson 3: Derivatives & Gradients
5. Lesson 4: Probability & Statistics
6. Lesson 5: Data, Features & Generalization
7. Lesson 6: Linear Regression
8. Lesson 7: Gradient Descent
9. Lesson 8: Logistic Regression
10. Lesson 9: Evaluating Models
11. Lesson 10: Overfitting, Regularization & Bias–Variance
12. Lesson 11: k-Nearest Neighbours
13. Lesson 12: Naive Bayes
14. Lesson 13: Decision Trees
15. Lesson 14: Random Forests & Boosting
16. Lesson 15: Support Vector Machines
17. Lesson 16: Clustering
18. Lesson 17: Dimensionality Reduction & PCA
19. Lesson 18: Neural Networks from Scratch
20. Lesson 19: Deep Learning Architectures
21. Lesson 20: Feature Engineering, Pipelines & Final Project
22. Lesson 21: Ethics & Fairness in Machine Learning
23. Lesson 22: SQL & Data Wrangling for ML
24. Lesson 23: Hyperparameter Tuning & Model Selection
25. Lesson 24: Interpretability & Explainable ML
26. Lesson 25: Imbalanced Data & Anomaly Detection
27. Lesson 26: Time Series Forecasting
28. Lesson 27: Recommender Systems

29. Lesson 28: Bayesian & Probabilistic ML

30. Answers to quizzes and exercises

BEFORE YOU BEGIN · OPTIONAL · ~60 MIN

Lesson 0: Python, NumPy & pandas for Beginners

Never coded? Learn just enough Python, NumPy and pandas to read and run every example in the course.

BY THE END OF THIS LESSON YOU CAN

- Run Python code in a Jupyter notebook and read its output and error messages
- Use variables, numbers, strings, lists and dictionaries, counting positions from 0
- Write if-statements, for-loops and simple functions
- Do arithmetic on whole NumPy arrays and use shape, axis and boolean filtering
- Load, select, filter and summarise a table with pandas

↺ BEFORE YOU START

No prerequisites — this is the starting point.

Where this fits: This optional lesson comes before everything else. If you can already read basic Python, NumPy and pandas, skip straight to Lesson 1.

1. Why Python — and how to use this lesson

Every lesson in this course has Python code. If you've never programmed, this lesson gives you exactly enough to read and run it: about an hour, no prior experience needed. You will learn the core of the language, then the two libraries that machine learning in Python is built on: **NumPy** for numbers in bulk and **pandas** for tables.

Python is popular for machine learning because it reads almost like English, and because the best ML libraries (scikit-learn, PyTorch, TensorFlow) use it. You don't need to memorise anything here — come back whenever a later lesson's code looks unfamiliar.

LEARN BY RUNNING

Each code box below is followed by the **output** Python actually printed. The best way to learn is to type the code yourself in the Lesson 0 notebook, run it, then change something and run it again.

2. Running Python code

Follow the Setup instructions once: they install Python and **JupyterLab**, a notebook that runs in your browser. A notebook is a list of *cells*. Type code in a cell and press **Shift+Enter**: the code runs and its output appears underneath. Download this lesson's notebook with the button at the top of the page.

The simplest program prints a message:

```
print("Hello, machine learning!")
```

Output:

```
Hello, machine learning!
```

`print` is a *function*: you give it something in brackets and it shows it. Text goes inside quotes. Anything after a `#` is a *comment* — a note for humans that Python ignores.

3. Values, variables and arithmetic

A **variable** is a name for a value. The `=` sign means "store this value under this name" (it is not the maths equals).

```
price = 72.5        # a decimal number (a float)
bedrooms = 3        # a whole number (an int)
city = "Chennai"     # text (a string)
print(price, bedrooms, city)
```

Output:

```
72.5 3 Chennai
```

Arithmetic works like a calculator. `**` means "to the power of", `/` always gives a decimal, and `//` divides and drops the remainder.

```
print(7 + 2, 7 - 2, 7 * 2, 7 / 2)
print(7 // 2, 7 % 2, 2 ** 10)
area = 12.5 * 8     # a variable can hold the result of a calculation
print(area)
```

Output:

```
9 5 14 3.5
3 1 1024
100.0
```

You can change a variable at any time; the new value replaces the old one:

```
count = 1
count = count + 1   # take the old value, add 1, store it back
count += 1          # a shorter way to write the same thing
print(count)
```

Output:

```
3
```

4. Text and printing

Text values are called **strings**. An *f-string* (a string with `f` before the quote) lets you drop variables into text with curly brackets, and `:.2f` rounds a number to 2 decimal places:

```
name = "house 7"
price = 84.1234
print(f"{name} costs {price:.2f} lakh")
print(name.upper(), len(name))   # a method, and the length of the string
```

Output:

```
house 7 costs 84.12 lakh
HOUSE 7 7
```

5. Lists and indexing

A **list** holds several values in order, inside square brackets. Positions are counted from **0**, not 1 — the first item is `[0]`. Negative positions count from the end.

```
prices = [58, 82, 55, 88, 96]
print(prices[0], prices[1], prices[-1])   # first, second, last
print(len(prices))                          # how many items
```

Output:

```
58 82 96
5
```

A **slice** `[start:stop]` takes items from `start` up to *but not including* `stop`:

```
print(prices[1:3])     # items at positions 1 and 2
print(prices[:2])      # the first two
print(prices[2:])      # from position 2 to the end
prices.append(61)      # add an item at the end
print(prices)
```

Output:

```
[82, 55]
[58, 82]
[55, 88, 96]
[58, 82, 55, 88, 96, 61]
```

> **! THE OFF-BY-ONE TRAP**
>
> `prices[1:3]` has *two* items (positions 1 and 2), and the last item of a 5-item list is `prices[4]` — `prices[5]` is an error. Almost every beginner trips on this; it soon becomes natural.

6. Dictionaries

A **dictionary** stores values under names (*keys*) instead of positions — like a row of a table:

```
house = {"area": "Central", "size_m2": 120, "price_lakh": 88}
print(house["size_m2"])
house["sold_fast"] = True     # add a new key
print(house)
```

Output:

```
120
{'area': 'Central', 'size_m2': 120, 'price_lakh': 88, 'sold_fast': True}
```

7. Making decisions and repeating: if and for

`if` runs code only when a condition is true. The indented lines (4 spaces) belong to it — in Python, **indentation is part of the syntax**.

```
price = 96
if price > 90:
    print("expensive")
elif price > 60:
    print("mid-range")
else:
    print("affordable")
```

Output:

```
expensive
```

A `for` loop repeats its indented block once for each item:

```
prices = [58, 82, 55, 88]
total = 0
for p in prices:
    total = total + p
print("total:", total, " average:", total / len(prices))
```

Output:

```
total: 283  average: 70.75
```

`range(n)` gives the numbers 0 to n−1, handy for repeating something n times. A **list comprehension** builds a new list from an old one in one line:

```
for step in range(3):
    print("step", step)
squares = [x ** 2 for x in [1, 2, 3, 4]]
print(squares)
```

Output:

```
step 0
step 1
step 2
[1, 4, 9, 16]
```

8. Functions

A **function** packages code under a name so you can reuse it. `def` defines it, the brackets list its inputs, and `return` sends back the result. This one is the mean squared error from Lesson 1:

```
def mse(y_true, y_pred):
    total = 0
    for t, p in zip(y_true, y_pred):   # zip walks through both lists together
        total += (p - t) ** 2
    return total / len(y_true)

print(mse([3, 5, 7], [2.5, 5, 8]))
```

Output:

```
0.4166666666666667
```

9. Libraries and import

Python's real power is its libraries. `import` loads one; `as` gives it a short nickname. These three lines start almost every notebook in this course:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
print("NumPy", np.__version__, "and pandas", pd.__version__, "loaded")
```

Output:

```
NumPy 2.5.3 and pandas 3.0.5 loaded
```

10. NumPy: numbers in bulk

A NumPy **array** is like a list of numbers, but arithmetic applies to *every element at once* — no loop needed. This is called **vectorisation**, and it's both shorter and much faster.

```
x = np.array([1.0, 2.0, 3.0, 4.0])
print(x * 2)          # every element doubled
print(x + 10)
print(x ** 2)
print(x.mean(), x.sum(), x.max())
```

Output:

```
[2. 4. 6. 8.]
[11. 12. 13. 14.]
[ 1.  4.  9. 16.]
2.5 10.0 4.0
```

Arrays can have two dimensions — rows and columns, like a table of examples × features. `.shape` tells you the size, and `axis=0` means "down the rows" (one result per column):

```
X = np.array([[120, 3, 4.0],
              [ 95, 2, 6.5],
              [150, 4, 12.0]])   # 3 houses × 3 features
print(X.shape)
print(X[0])           # the first row (house 1)
print(X[:, 0])        # the first column (sizes)
print(X.mean(axis=0)) # the average of each column
```

Output:

```
(3, 3)
[120.   3.   4.]
[120.  95. 150.]
[121.66666667   3.          7.5       ]
```

Comparisons give arrays of True/False, which you can use to **filter**. And `@` multiplies a matrix by a vector — the dot product of each row with the weights (Lesson 2):

```
sizes = X[:, 0]
print(sizes > 100)
print(sizes[sizes > 100])
w = np.array([0.5, 2.0, -1.0])
print(X @ w)          # one prediction per house
```

Output:

```
[ True False  True]
[120. 150.]
[62. 45. 71.]
```

11. pandas: tables of data

pandas works with **DataFrames** — tables with named columns, like a spreadsheet. Let's load the course's running example, 12 houses in Chennai (`houses.csv`, in the `data` folder):

```
df = pd.read_csv("houses.csv")
print(df.shape)               # (rows, columns)
print(df.columns.tolist())
```

Output:

```
(12, 9)
['id', 'area', 'size_m2', 'bedrooms', 'dist_km', 'age_yrs', 'description', 'price_lakh',
'sold_fast']
```

```
print(df[["area", "size_m2", "price_lakh"]].head(4))   # the first 4 rows of three
columns
```

Output:

```
      area  size_m2  price_lakh
0  Central       65          58
1  Central       90          82
2  Central      120          95
3   Suburb       80          55
```

Select one column with `df["name"]` (it behaves like a NumPy array) and filter rows with a condition in square brackets:

```
print(df["price_lakh"].mean())
big = df[df["size_m2"] > 120]
print(len(big), "houses over 120 m²")
print(big[["id", "size_m2", "price_lakh"]])
```

Output:

```
72.08333333333333
4 houses over 120 m²
    id  size_m2  price_lakh
5    6      140          79
7    8      150          84
9   10      130          50
11  12      125          96
```

`groupby` summarises by category, and you can create new columns from old ones:

```
print(df.groupby("area")["price_lakh"].mean().round(1))
df["price_per_m2"] = df["price_lakh"] / df["size_m2"]
print(df.sort_values("price_per_m2", ascending=False)[["id", "area",
"price_per_m2"]].head(3).round(3))
```

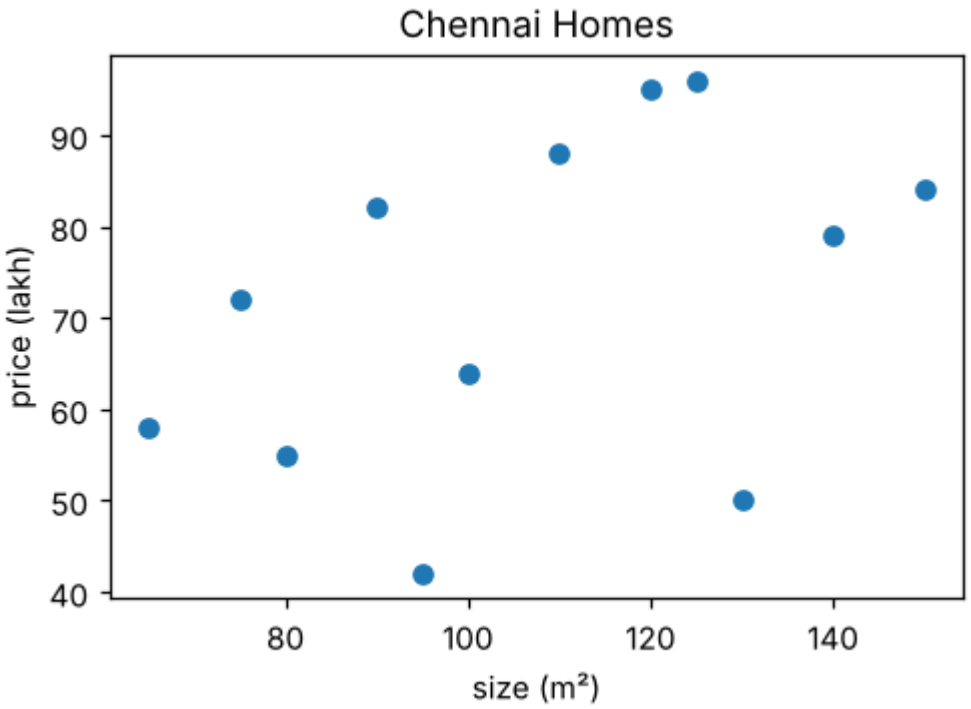
Output:

```
area
Central      76.8
Outskirts    58.7
Suburb       76.4
Name: price_lakh, dtype: float64
    id     area  price_per_m2
10  11  Central         0.960
1    2  Central         0.911
0    1  Central         0.892
```

12. A first plot

matplotlib draws charts. A scatter plot of price against size is the first thing you'd look at before fitting a model (Lesson 6):

```
plt.figure(figsize=(5, 3.2))
plt.scatter(df["size_m2"], df["price_lakh"])
plt.xlabel("size (m²)")
plt.ylabel("price (lakh)")
plt.title("Chennai Homes")
plt.show()
```



The plot this code draws: bigger houses tend to cost more, but with plenty of scatter — distance and age matter too.

13. Reading error messages

Errors are normal — professionals see them all day. Python stops and prints a *traceback*; the **last line** says what went wrong. Here are the ones you'll meet most, with the real messages:

```
print(prise)
```

Error:

```
NameError: name 'prise' is not defined
```

NameError: a typo, or a variable used before the cell that creates it was run. In notebooks, run cells from the top.

```
"Price: " + 72
```

Error:

```
TypeError: can only concatenate str (not "int") to str
```

TypeError: mixing types that don't go together. Fix: `"Price: " + str(72)`, or an f-string.

```
prices = [58, 82, 55]
prices[3]
```

Error:

```
IndexError: list index out of range
```

IndexError: counting from 1 instead of 0 — the last of 3 items is `prices[2]`.

```
np.array([1, 2, 3]) + np.array([1, 2])
```

Error:

```
ValueError: operands could not be broadcast together with shapes (3,) (2,)
```

ValueError (a shape mismatch): element-by-element arithmetic needs arrays of the same size. Print `.shape` to check — the single most useful debugging habit in machine learning.

> **WHEN YOU'RE STUCK**
>
> Read the last line of the error, then the line of your code it points to. Check spelling, brackets, quotes and indentation. Restart the notebook and run all cells from the top. Searching the exact error message online almost always finds an answer.

14. Questions beginners often ask

Q. Do I have to finish this before Lesson 1?

No. Lesson 1 has very little code. Many people read Lessons 1–5 first and come back here when the code in later lessons starts to matter.

Q. Which version of Python?

Any Python 3 from the last few years (3.9 or newer). Python 2 is obsolete; you won't meet it.

Q. Can I break my computer by running code?

Not with anything in this course. The worst that happens is an error message — then you fix the cell and run it again.

Q. Why do positions start at 0?

It's a convention most programming languages share: the position is "how many steps from the start". You'll stop noticing it within a day or two.

Q. NumPy or pandas — when do I use which?

pandas for loading and exploring tables with named columns; NumPy for maths on arrays of numbers. Models in scikit-learn accept either, and `df.to_numpy()` converts a table into an array.

! COMMON MISTAKES TO AVOID

- Forgetting that positions start at 0, so `a[3]` is the fourth item and a slice `a[1:3]` has two items.
- Mixing tabs and spaces, or forgetting to indent the lines under `if`, `for` or `def`.
- Running notebook cells out of order, so a variable doesn't exist yet (NameError). Restart and run all cells from the top.
- Using a plain list where you need a NumPy array: `[1, 2] * 2` repeats the list, `np.array([1, 2]) * 2` doubles the numbers.
- Writing `df["price"] > 80` and expecting the rows; you need `df[df["price"] > 80]`.

! INTERVIEW CORNER

Don't confuse

- **Python list vs NumPy array** — Lists are containers; arrays are vectors.
- **View vs copy** — NumPy slices share memory; lists don't.
- **`*` vs `@` in NumPy** — Star = side by side; at-sign = rows times columns.
- **`.loc` vs `.iloc`** — i for integer position.

Hidden questions — answer in your head first, then open.

▼ **You compute MSE with `np.mean((y_pred - y)**2)` and get 2.33, but by hand it's 0.33. What went wrong?**

The trap: The formula is right, so people check the maths.

A shape bug. If `y_pred` has shape (3, 1) and `y` has shape (3,), broadcasting turns `y_pred - y` into a 3×3 matrix of every prediction minus every target. For predictions 1, 2, 3 and targets 1, 2, 4 that gives 2.33 instead of 0.33. Flatten with `.ravel()` and check `.shape` before computing losses.

▼ **Why does `0.1 + 0.2 == 0.3` return False, and what should you do instead?**

The trap: Assuming it's a Python bug.

Floats are stored in binary, where 0.1 can't be represented exactly; the sum is 0.30000000000000004. Compare with a tolerance: `np.isclose(a, b)` or `abs(a - b) < 1e-9`. This matters when testing model outputs or gradients.

▼ **You slice a NumPy array, edit the slice, and your original data changes. Why?**

The trap: Expecting Python list behaviour.

NumPy slices are *views* sharing the same memory. Use `.copy()` when you need an independent array. Python list slices, by contrast, are copies.

▼ **pandas warns *SettingWithCopyWarning* after `df[df.age > 30]['score'] = 0`. Did the data change?**

The trap: Ignoring warnings.

Probably not: the first indexing may return a copy, and the assignment changes the copy. Use one `.loc` call: `df.loc[df.age > 30, 'score'] = 0`.

15. Key takeaways

- Variables name values: numbers, strings, lists (positions from 0) and dictionaries (named keys).
- `if` decides, `for` repeats, `def` packages code into functions — and indentation marks which lines belong together.
- NumPy arrays do arithmetic on every element at once; `.shape`, `axis=`, boolean filtering and `@` are the tools you'll use most.
- pandas DataFrames are tables: `read_csv`, `df["col"]`, `df[condition]`, `groupby` and new columns cover most everyday work.
- Errors are information: read the last line first.

16. Check your understanding

1. What does `[10, 20, 30, 40][1]` give?

a. 10

b. 20

c. 30

2. What does `[10, 20, 30, 40][1:3]` give?

a. [20, 30]

b. [10, 20, 30]

c. [20, 30, 40]

3. `x = np.array([1, 2, 3])`. What is `x * 2`?

a. [1, 2, 3, 1, 2, 3]

b. [2, 4, 6]

c. An error

4. A NumPy array has shape `(100, 5)`. What does `X.mean(axis=0)` return?

a. One number

b. 100 numbers (one per row)

c. 5 numbers (one per column)

5. Which line keeps only the rows of a DataFrame where the price is above 80?

a. `df["price"] > 80`

b. `df[df["price"] > 80]`

c. `df.price(80)`

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

For `a = [3, 6, 9, 12, 15]`, what are (a) `a[-2]` and (b) `a[1:4]`?

WORKING

EXERCISE 2 By hand

`X = np.array([[1, 2, 3], [4, 5, 6]])`. Give `X.shape`, `X.sum(axis=0)` and `X[:, 1]`.

WORKING

EXERCISE 3 Code

Load `houses.csv` with pandas. How many houses cost more than 80 lakh, and what is the average price of the houses that sold fast (`sold_fast == 1`)?

WORKING

EXERCISE 4 Code

Write a function `celsius_to_f(c)` that returns `c × 9/5 + 32`. What does `celsius_to_f(37)` return?

WORKING

NOTES

PART 1 · FOUNDATIONS · ~35 MIN

Lesson 1: What Is Machine Learning?

Rules vs learning, the core vocabulary, the types of ML, and how a model learns by minimising a loss.

BY THE END OF THIS LESSON YOU CAN

- Explain how machine learning differs from traditional programming
- Use the words feature, label, model, parameter, training and inference correctly
- Tell regression, classification and clustering problems apart
- Compute the mean squared error of a simple model by hand

↺ BEFORE YOU START

No prerequisites — this is the starting point. Never written code? Lesson 0 teaches just enough Python, NumPy and pandas for the whole course.

Where this fits: This is the map for the whole course: every later lesson is one part of the picture of learning from data — the data itself, the model, the loss, the optimiser and the evaluation.

1. Welcome: what you'll be able to do

By the end of this lesson you will be able to explain, in plain words, what machine learning is, how it differs from ordinary programming, and what the words *model*, *feature*, *label*, *training* and *prediction* mean. You'll also "be the algorithm" yourself in an interactive demo — which is the fastest way to understand what a computer does when it learns.

No maths or programming knowledge is needed for this lesson. Everything is built up from scratch.

2. Two ways to make a computer smart

Imagine you want a program that decides whether an email is spam.

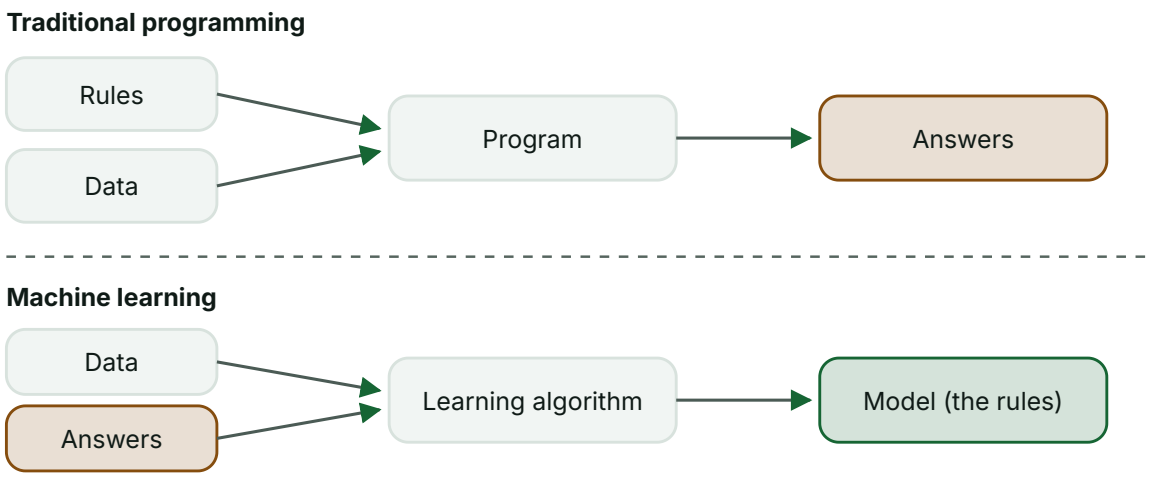


Figure 1. Traditional programming turns hand-written rules into answers. Machine learning turns examples (data plus answers) into the rules — a trained model that can then answer new cases.

Way 1 — write the rules yourself. You sit down and write instructions:

```
if "free money" in email: spam
if "winner" in subject and sender is unknown: spam
...
```

This is traditional programming. It works for simple problems, but it breaks down quickly: spammers change their wording, there are thousands of special cases, and you can never write them all.

Way 2 — show examples and let the computer find the rules. You collect 100,000 emails that people have already marked "spam" or "not spam", and give them to a learning algorithm. The algorithm looks for patterns — which words, senders and formats tend to appear in spam — and produces a program that can classify new emails it has never seen.

That second way is **machine learning**.

> **≡ DEFINITION**
>
> Google's Machine Learning course puts it like this: machine learning is "a way to train software, called a **model**, to make predictions or generate content using data."

◎ ANALOGY

Think about how a child learns what a "dog" is. Nobody gives the child a rulebook ("four legs, fur, tail, barks..."). Instead, the child sees many dogs, and many things that are not dogs, and gradually builds an internal sense of "dog-ness". If they call a cat a dog, someone corrects them, and their understanding adjusts. Machine learning works the same way: *examples + correction = learning*.

3. Where you already meet ML

- **Your phone's keyboard** predicts your next word.
- **Maps apps** predict how long a trip will take.
- **Streaming services** recommend what to watch next.
- **Banks** flag card transactions that look fraudulent.
- **Photo apps** group pictures by the faces in them.
- **Chatbots and image generators** create new text and pictures.

Each of these has the same core: a model that turns some input (your typing, a route, your viewing history) into an output (a word, a time, a recommendation).

4. The essential vocabulary

Let's use one running example: predicting the **price of a house**.

Size (m²)	Bedrooms	Distance to city (km)	Price (k USD)
50	1	2	210
80	2	5	290
120	3	12	340
150	4	3	520

- **Example** (also called a sample, row, or data point): one house — one row of the table.
- **Features** (inputs, usually written x): the facts we know about each example — size, bedrooms, distance. A house is described by a list of features, e.g. $x = (50, 1, 2)$.
- **Label** (target, output, written y): the thing we want to predict — price. For the first house, $y = 210$.
- **Dataset**: the whole table — many examples, each with features and (usually) a label.
- **Model**: a function that takes features and outputs a prediction, written $\hat{y}$ ("y-hat"). The hat means "our guess", as opposed to the true y.

- **Parameters** (or weights): the adjustable numbers inside the model. Learning means finding good values for them.
- **Training**: the process of adjusting the parameters so predictions match the labels in the dataset.
- **Inference** (prediction): using the trained model on new examples whose labels we don't know.

> **NOTATION YOU'LL SEE EVERYWHERE**
>
> x = features, y = true label, $\hat{y}$ = predicted label, n = number of examples, d = number of features. A subscript picks out one example: x_3 and y_3 are the features and label of example 3.

5. The main types of machine learning

1. Supervised learning

Every training example comes with the right answer (the label). The model learns the mapping from features to label. Two flavours:

- **Regression** — the label is a number. *"What will this house sell for?"* → 340.
- **Classification** — the label is a category. *"Is this email spam?"* → yes / no. *"Which digit is in this image?"* → 0–9.

2. Unsupervised learning

There are no labels. The goal is to discover structure in the data on its own — for example, grouping customers into segments with similar shopping habits (*clustering*), or compressing many features into a few (*dimensionality reduction*).

3. Reinforcement learning

An agent takes actions in an environment and receives rewards or penalties, learning by trial and error which actions lead to the most reward — the way a game-playing program learns to win.

4. Generative AI

Models that create new content — text, images, audio — learned from huge datasets. Chatbots and image generators are examples.

This course focuses on **supervised** and **unsupervised** learning, which are the foundation for everything else.

6. How does a model actually ”learn”?

Let's make it concrete with the simplest possible model. Suppose we predict house price from size only, using a straight line:

$$\hat{y} = w \cdot x + b$$

- x is the size of the house,
- $\hat{y}$ is the predicted price,
- w (the *weight*, or slope) says how much the price rises per extra m²,
- b (the *bias*, or intercept) is the starting price when size is 0.

w and b are the model's **parameters**. Different values give different lines. Learning means finding the w and b that make the line fit the data best. To do that we need three ingredients:

1. **A model** with adjustable parameters — here, the line $\hat{y} = wx + b$.
2. **A way to measure how wrong it is** — called the *loss* (or cost).
3. **A way to improve the parameters** — an optimisation method that changes w and b to reduce the loss.

Measuring wrongness: the loss

For each house, the **error** (or residual) is the gap between prediction and truth: $\hat{y}_i - y_i$. Some errors are positive, some negative, so we square them (making all positive, and punishing big errors more) and take the average. This is the **mean squared error (MSE)**:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Reading the symbols: $\sum_{i=1}^{n}$ means "add up, for every example i from 1 to n". So: compute the squared error for every house, add them up, divide by how many houses there are.

✎ WORKED EXAMPLE — COMPUTE MSE BY HAND

Three houses with sizes $x = 1, 2, 3$ (in 100 m²) and prices $y = 2, 4, 5$. Try the line $w = 2,\ b = 0$, i.e. $\hat{y} = 2x$.

x	y	$\hat{y} = 2x$	error $\hat{y} - y$	squared
1	2	2	0	0
2	4	4	0	0
3	5	6	1	1

$\text{MSE} = \frac{0+0+1}{3} \approx 0.33.$

Now try $w = 1.5,\ b = 0.67$: predictions are $2.17, 3.67, 5.17$; errors $0.17, -0.33, 0.17$; squared $0.028, 0.111, 0.028$; MSE ≈ 0.056. Smaller loss — **a better line**. Learning is simply the search for the parameters with the smallest loss.

? MAKE SENSE OF IT

The situation: You're an estate agent testing a quick pricing rule — "price is twice the size" — on three homes you've already sold.

What the result tells you: The MSE of 0.33 is the *average squared miss*. Its square root, about 0.58, is easier to feel: a typical prediction is off by roughly 0.58 price units. The rule nails the first two homes and misses the third by 1.

What if…?

- **One wrong price.** If the third home had sold for 8 instead of 5, its error becomes 2, its square 4, and the MSE jumps from 0.33 to **1.33** — four times bigger from one house. The average absolute error only doubles (0.33 → 0.67).
- **An error of 3** on that house (price 9) gives MSE **3.0**: squaring makes an error of 3 count nine times as much as an error of 1.
- **A better line** ($w = 1.5,\ b = 0.67$) spreads small misses across all three houses and cuts the MSE to about 0.056.

Why it matters in ML: MSE is the loss most regression models minimise. Because it squares errors, one large miss (or one bad data point) can dominate it — which is why outliers deserve a look before you train, and why some problems use the absolute error instead.

Improving the parameters

You could guess randomly, but that's slow. Instead, algorithms use **calculus** to work out which direction to nudge each parameter so the loss goes down, take a small step, and repeat. This is called **gradient descent**, and you'll build it yourself in Lesson 7 after learning derivatives in Lesson 3.

READ THE FORMULA **Mean squared error**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

- **1/n** — divide by the number of examples — turns a total into an average
- **Σ** — add up over every example i = 1 … n
- **$(\hat{y}_i - y_i)^2$** — the error on example i, squared so it's positive and big errors count more

Hover over (or tap) a part to highlight it in the formula.

7. Try it: be the learning algorithm

Below are some houses (dots). Move the sliders to change w and b and try to make the line fit the dots. Watch the MSE: your goal is to make it as small as possible. The red vertical lines are the errors. When you're done, press *Show best fit* to see what an algorithm finds.

Interactive demo — open *lesson-01.html* on the website to try it.

What you just did — adjust parameters, check the loss, adjust again — is exactly what a learning algorithm does. The only difference is that the computer does it with precise maths, millions of times faster, and with thousands of parameters instead of two.

8. The machine learning workflow

Real projects follow roughly the same steps:

1. **Define the problem.** What exactly do you want to predict, and how will you measure success?
2. **Collect data.** Gather examples with features (and labels, for supervised learning).
3. **Split the data.** Keep some examples aside as a *test set* that the model never sees during training, so you can check honestly how it does on new data (Lesson 5).

4. **Prepare the data.** Clean errors, handle missing values, convert text categories to numbers (Lesson 20).
5. **Train a model.** Start simple, then try better models.
6. **Evaluate.** Measure performance on data the model hasn't seen (Lesson 9).
7. **Deploy and monitor.** Use the model in the real world and watch that it keeps working.

> **! THE ONE BIG TRAP**
>
> A model can memorise its training data — like a student who memorises the answers to last year's exam without understanding the subject. It looks perfect on data it has seen and fails on new data. This is called **overfitting**, and avoiding it is one of the central themes of this course.

9. Your first ML code

Here is the same "fit a line" task in Python, using the popular **scikit-learn** library. Don't worry about every detail yet — notice the pattern: create a model, `fit` it to data, then `predict`.

```
from sklearn.linear_model import LinearRegression

# features: house size in 100 m² (each row is one house)
X = [[1], [2], [3], [4], [5]]
# labels: price in units of 100,000 USD
y = [2.1, 3.9, 6.2, 7.8, 10.1]

model = LinearRegression()   # 1. choose a model
model.fit(X, y)              # 2. train: find the best w and b

print(model.coef_, model.intercept_)   # the learned w and b
print(model.predict([[6]]))            # 3. predict a new 600 m² house
```

Running this prints a slope of about `1.99`, an intercept of about `0.05`, and a prediction of about `12` (i.e. 1.2 million USD) for the new house. To run it yourself, see the setup steps on the home page.

10. Running example: Chennai Homes

Meet the dataset we'll come back to in every lesson: 12 homes for sale in Chennai (made up, but realistic). Each row is one house — an **example**. The columns are **features** (area, size, bedrooms, distance to the centre, age, the listing text) and two possible **labels**: the price in lakh rupees (a regression target) and whether it sold within 30 days (a classification target).

id	area	size_m2	bedrooms	dist_km	age_yrs	description
1	Central	65	1	2	5	bright compact flat near metro
2	Central	90	2	3	12	spacious flat near metro with balcon
3	Central	120	3	4	20	large family flat needs renovation
4	Suburb	80	2	9	8	quiet flat with parking
5	Suburb	110	3	11	3	new spacious villa with garden
6	Suburb	140	3	10	25	old villa needs renovation
7	Suburb	100	2	12	15	flat with parking near school
8	Outskirts	150	4	22	6	new large villa with garden
9	Outskirts	95	2	25	10	quiet flat far from city
10	Outskirts	130	3	20	30	old house needs renovation
11	Central	75	2	5	2	new bright flat near metro
12	Suburb	125	3	8	4	new spacious villa near school

⬇ Download houses.csv Save it next to your notebook (it's also already in the course's `data` folder).

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
print(df.shape)                       # rows = houses, columns = facts about each
house
print(df[["area", "size_m2", "dist_km", "price_lakh", "sold_fast"]].head(3))
print("average price:", round(df["price_lakh"].mean(), 1), "lakh")
```

Output:

```
(12, 9)
      area  size_m2  dist_km  price_lakh  sold_fast
0  Central       65        2          58          1
1  Central       90        3          82          1
2  Central      120        4          95          0
average price: 72.1 lakh
```

⚙ WHERE THIS IS USED

- Spam filters, product recommendations, fraud alerts, voice assistants, medical image screening

Choose it when: When the rules are too complex or change too often to write by hand, and you have example data.

Think twice when: When simple fixed rules already solve the problem, or you have little or no relevant data.

× MISCONCEPTIONS TO UNLEARN

MYTH ML models understand the world like people do.

REALITY A model finds statistical patterns in its training data. It has no common sense, and it can fail badly on situations unlike its training data.

MYTH More data always fixes everything.

REALITY More data helps against overfitting, but not against wrong labels, biased samples or missing important features.

RESPONSIBLE AI · WHEN NOT TO USE MACHINE LEARNING

ML learns whatever patterns are in its data — including unfair ones. In 2018 Reuters reported that Amazon scrapped an experimental hiring model that had learned to downgrade CVs containing the word "women's", because it was trained on a decade of mostly male hires. Before reaching for ML, ask: would simple, transparent rules do? Is there good, representative data? Can mistakes be noticed and corrected? If a decision must be explained to the person it affects, or errors are unacceptable, ML may be the wrong tool.

11. Questions beginners often ask

Q. Is machine learning the same as AI?

No. Artificial intelligence is the broad goal of making computers do intelligent tasks. Machine learning is one approach to it — learning from data — and today it's the most successful one. Deep learning (Lesson 19) is a part of machine learning.

Q. How much data do I need?

It depends on how complex the pattern is. A straight line can be fitted from a handful of points; recognising objects in photos needs thousands to millions of images. Lesson 5 shows how to check whether you have enough: if the model overfits, more data usually helps.

Q. Do I need to be good at maths?

You need to be comfortable with basic algebra. Lessons 2–4 teach the rest from scratch, and every formula is explained with a worked example.

! COMMON MISTAKES TO AVOID

- Thinking a model that scores perfectly on its training data must be good — it may simply have memorised it (overfitting).
- Confusing the label (what you predict) with the features (what you know).
- Calling every yes/no problem "regression" because the algorithm is named logistic *regression*.

! INTERVIEW CORNER

Don't confuse

- **Algorithm vs model** — Algorithm = recipe; model = the cake.
- **Parameter vs hyperparameter** — If training changes it, it's a parameter. If you set it, it's a hyperparameter.
- **Loss vs metric** — Loss is for the optimiser; the metric is for the business.
- **Training vs inference** — Learn once, predict many times.
- **AI vs machine learning vs deep learning** — Nested circles: DL ⊂ ML ⊂ AI.
- **Regression (the task) vs linear regression (a model)** — Task names and model names are different things.

Hidden questions — answer in your head first, then open.

▼ **Is linear regression machine learning, or "just statistics"?**

The trap: Thinking ML means something complicated.

It's both. It learns its parameters from data to make predictions, which is exactly what ML is. Statistics and ML share many models; they differ more in goals (explaining vs predicting) than in methods.

▼ **What's the difference between a parameter and a hyperparameter? Give two examples of each.**

The trap: Calling the learning rate a parameter.

Parameters are learned from data (linear-regression weights, a tree's split thresholds). Hyperparameters are set before training and tuned on validation data (learning rate, k in kNN, maximum tree depth, regularisation strength λ).

▼ **Your model gets 100% accuracy on the training data. Is that good news?**

The trap: Celebrating.

Not by itself. It may have memorised the training data (overfitting), or a feature may leak the answer. Only a held-out validation/test score tells you how it generalises; also check the trivial baseline.

▼ **Why do we train with one function (the loss) but report another (the metric)?**

The trap: Assuming they're the same thing.

The loss must be smooth for gradient-based optimisation (log loss, MSE). The metric should reflect the real goal (F1, recall at a fixed precision, profit) and is often not

differentiable, so you can't optimise it directly.

12. Key takeaways

- Machine learning trains a **model** from **data** instead of hand-writing rules.
- Data is made of **examples**, each with **features** x and (in supervised learning) a **label** y.
- **Supervised** learning has labels (regression = numbers, classification = categories); **unsupervised** learning finds structure without labels.
- Learning = choosing **parameters** that minimise a **loss**, such as mean squared error.
- The real goal is to do well on **new** data, not just the training data.

13. Check your understanding

1. A model predicts tomorrow's temperature in °C. What kind of problem is this?

a. Classification

b. Regression

c. Clustering

2. In a dataset of emails labelled spam / not spam, what is the "label"?

a. The words in the email

b. The sender's address

c. Whether the email is spam

3. Predictions are 3, 5 and true values are 4, 5. What is the MSE?

a. 0.5

b. 1

c. 2

4. You group shoppers by buying habits without any predefined categories. This is...

a. Supervised learning

b. Unsupervised learning

c. Reinforcement learning

5. A model scores 100% on its training data but poorly on new data. This is called...

a. Underfitting

b. Regularization

c. Overfitting

14. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 Concepts

Classify each task as **regression**, **classification** or **clustering**: (a) predicting a delivery time in minutes; (b) deciding whether a photo contains a cat; (c) grouping news articles by topic when no topics are given; (d) predicting tomorrow's rainfall in mm.

WORKING

EXERCISE 2 By hand

True values are $y = 4, 7, 10$ and predictions are $\hat{y} = 5, 6, 10$. Compute the MSE.

WORKING

EXERCISE 3 By hand

Points $(1, 3), (2, 4), (3, 8)$. Which line has the lower MSE: $\hat{y} = 2x + 1$ or $\hat{y} = 2x + 0.5$?

WORKING

EXERCISE 4 Code

Change the Lesson 1 code so it predicts the price of a 350 m² house (that's $x = 3.5$). What does it print?

WORKING

NOTES

NOTES

PART 1 · FOUNDATIONS · ~45 MIN

Lesson 2: Vectors & Matrices

Lists and tables of numbers, the dot product, distances, and why predictions are matrix multiplications.

BY THE END OF THIS LESSON YOU CAN

- Add, scale and measure vectors
- Compute a dot product by hand and explain what it means geometrically
- Read a dataset as a matrix and multiply a matrix by a vector
- Use NumPy for the same operations

↺ BEFORE YOU START

This lesson builds on:

- Lesson 1: What Is Machine Learning? — A model maps features x to a prediction $\hat{y}$. Training adjusts its parameters to make a loss (such as MSE) small, and the real goal is doing well on new data.

Where this fits: Every model in the course is written in the language of vectors and matrices. Learning it now makes the formulas in Lessons 6, 8, 15, 17 and 18 readable.

1. Why vectors and matrices?

In Lesson 1, each house was described by a list of numbers: size, bedrooms, distance. A whole dataset was a table of such lists. In maths, a list of numbers is called a **vector**, and a table of numbers is called a **matrix**. Almost every ML algorithm is written in this language, because it lets us describe operations on *thousands* of examples and features in one short line — and lets computers run them very fast.

You only need a handful of ideas, and we'll build each one by hand before using code.

2. Vectors: lists of numbers with meaning

A vector is an ordered list of numbers. We write it in bold or with an arrow, and usually as a column:

$$\mathbf{x} = \begin{bmatrix} 50 \\ 1 \\ 2 \end{bmatrix}$$

This could be a house with size 50, 1 bedroom, 2 km from the city. The number of entries is the vector's **dimension**; this vector lives in 3-dimensional space, written $\mathbf{x} \in \mathbb{R}^3$ ("x is a vector of 3 real numbers"). The entries are called **components**, and we refer to them as x_1, x_2, x_3.

◎ TWO WAYS TO PICTURE A VECTOR

As a point: the vector $(3, 2)$ is the point 3 steps right and 2 steps up. **As an arrow**: an arrow from the origin $(0, 0)$ to that point — it has a *direction* and a *length*. Both pictures are useful. In ML, each example is a point in "feature space", and similar examples are points close together.

3. Adding and scaling vectors

Addition works component by component (the vectors must have the same dimension):

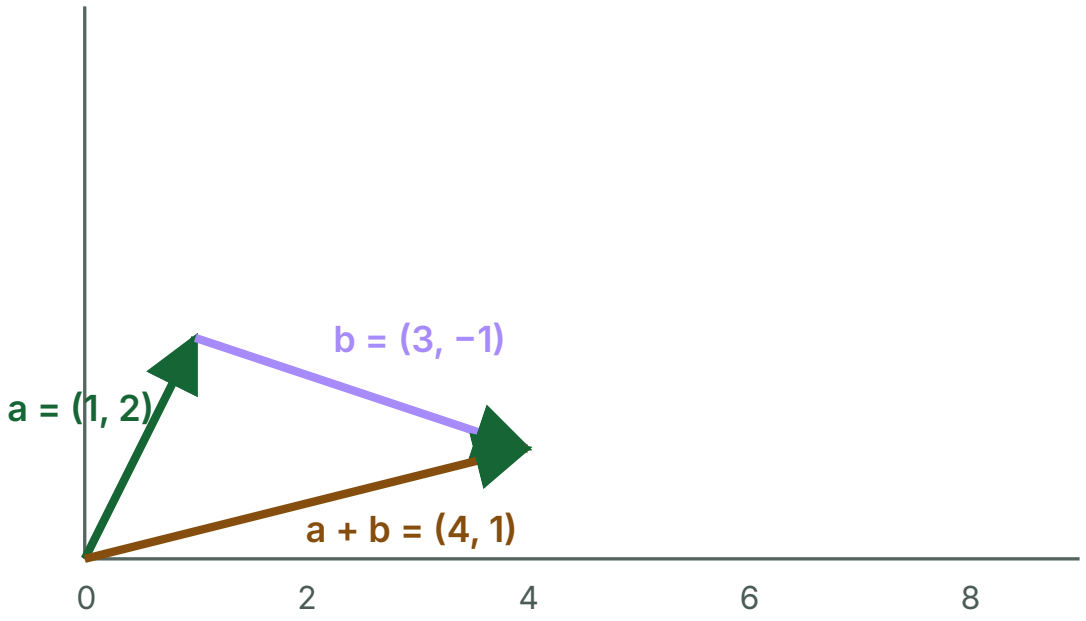


Figure 2. Adding vectors: walk along a, then along b; the sum goes straight from start to finish (axis ticks every 2 units, drawn at 40 px per unit).

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 1+3 \\ 2+(-1) \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

Geometrically: walk along the first arrow, then the second; you end at the sum.

Scalar multiplication multiplies every component by a single number (a *scalar*):

$$3\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}3\\6\end{bmatrix}$$

This stretches the arrow 3 times longer in the same direction. Multiplying by -1 flips it around.

> **WHERE YOU'LL SEE THIS**
>
> In gradient descent (Lesson 7), the update rule $\mathbf{w}_{\text{new}} = \mathbf{w} - \eta\,\nabla L$ is exactly "scale a vector, then subtract it from another vector".

4. Length (norm) and distance

The length of a vector — its **norm** — comes from Pythagoras' theorem. For $\mathbf{v} = (3, 4)$:

$$\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

In general, for a vector with d components:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_d^2} = \sqrt{\sum_{i=1}^{d} v_i^2}$$

This is the **Euclidean** (or L2) norm. The **distance** between two points is the length of their difference: $\text{dist}(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|$.

> **✎ WORKED EXAMPLE — DISTANCE BETWEEN TWO HOUSES**
>
> House A = (size 5, bedrooms 2), house B = (size 8, bedrooms 6). Difference $= (8-5,\ 6-2) = (3, 4)$. Distance $= \sqrt{9+16} = 5$.
>
> The k-nearest-neighbours algorithm (Lesson 11) classifies a new example by finding the training examples with the smallest distance to it — exactly this calculation.

? MAKE SENSE OF IT

The situation: A property app shows "homes like this one". It needs a number for how alike two homes are, and distance is that number.

What the result tells you: A distance of 5 combines a size difference of 3 and a bedroom difference of 4 into one number, using Pythagoras. Smaller means more similar.

What if...?

- **Same bedrooms:** if both homes had 2 bedrooms, the distance is just the size gap, **3**.
- **Change the units:** measure size in m² instead (500 vs 800). The difference becomes $(300, 4)$ and the distance **300.03** — bedrooms now barely count. Nothing about the houses changed, only the units.
- **Shrink the size scale** instead (5 → 0.5, 8 → 0.8): the distance is about **4.01**, almost all from bedrooms.

Why it matters in ML: kNN, k-means and SVMs all rely on distances, so the feature with the biggest numbers wins by default. That's why we standardise features first (Lessons 11 and 20).

Another norm you'll meet is the **L1 norm**, the sum of absolute values: $\|\mathbf{v}\|_1 = |v_1| + |v_2| + \ldots$. For $(3, 4)$ it is 7 — the distance if you could only walk along a city grid. It appears in Lasso regularization (Lesson 10).

5. The dot product — the most important operation in ML

PICTURE IT At a shop you buy 2 notebooks at ₹40, 3 pens at ₹10 and 1 bag at ₹500. Your bill is $2 \cdot 40 + 3 \cdot 10 + 1 \cdot 500 = 610$: multiply matching entries and add. That's a dot product — and it's exactly how a linear model turns features and weights into a prediction.

The dot product takes two vectors of the same length and returns a single number: multiply matching components and add them up.

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_d b_d = \sum_{i=1}^{d} a_i b_i$$

✎ WORKED EXAMPLE

$\mathbf{a} = (2, 3, 1)$, $\mathbf{b} = (4, -1, 5)$.

$\mathbf{a} \cdot \mathbf{b} = 2 \cdot 4 + 3 \cdot (-1) + 1 \cdot 5 = 8 - 3 + 5 = 10.$

? MAKE SENSE OF IT

The situation: Think of $\mathbf{a}$ as one applicant's three feature values and $\mathbf{b}$ as the weights a model gives those features. The dot product is the model's score.

What the result tells you: Each pair contributes separately: 8 from the first feature, -3 from the second (its weight is negative, so it pulls the score down) and 5 from the third. The total score is 10.

What if…?

- **Raise the second feature** from 3 to 6: the score *falls* to **7**, because that feature has a negative weight.
- **Flip every weight** to $(-4, 1, -5)$: the score becomes **−10** — the vectors now point in opposite directions.
- **Weights** $(3, -2, 0)$**:** the score is **0** — the vectors are perpendicular, and this applicant is "neutral" to the model.

Why it matters in ML: Every linear model, every neuron and every attention score in a Transformer is a dot product of inputs with weights. Reading it as "sum of contributions" lets you see which feature pushed a prediction up or down.

Why it matters: a prediction is a dot product

Suppose a model predicts price as $\text{price} = 3 \cdot \text{size} + 20 \cdot \text{bedrooms} - 5 \cdot \text{distance}$. Put the *weights* in one vector $\mathbf{w} = (3, 20, -5)$ and the features in another $\mathbf{x} = (50, 1, 2)$. Then:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} = 3 \cdot 50 + 20 \cdot 1 + (-5) \cdot 2 = 150 + 20 - 10 = 160$$

So a linear model's prediction *is* a dot product (plus a bias b): $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$. Each weight says how much its feature matters, and its sign says in which direction.

The geometric meaning: how aligned are two vectors?

The dot product also equals:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta$$

where θ is the angle between the arrows. Since $\cos 0^\circ = 1$, $\cos 90^\circ = 0$, $\cos 180^\circ = -1$:

- **Positive** dot product → vectors point in a similar direction.
- **Zero** → perpendicular (called *orthogonal*): unrelated directions.
- **Negative** → pointing in opposite directions.

Dividing out the lengths gives **cosine similarity**, a score from -1 to 1 widely used to compare documents, words and images:

$$\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}$$

ADVANCED · OPTIONAL **Why a·b = ‖a‖‖b‖cos θ** ▾ hide

Put $\mathbf{a}$ and $\mathbf{b}$ tail to tail; the third side of the triangle is $\mathbf{a}-\mathbf{b}$. Expanding with the dot product:

$$\|\mathbf{a}-\mathbf{b}\|^2 = (\mathbf{a}-\mathbf{b})\cdot(\mathbf{a}-\mathbf{b}) = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a}\cdot\mathbf{b}.$$

The law of cosines says the same side satisfies $\|\mathbf{a}-\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$. Comparing the two gives $\mathbf{a}\cdot\mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$. Because $|\cos\theta| \le 1$, it also proves the **Cauchy–Schwarz inequality** $|\mathbf{a}\cdot\mathbf{b}| \le \|\mathbf{a}\|\|\mathbf{b}\|$, which we use in Lesson 3.

READ THE FORMULA **Dot product**

$$\mathbf{a}\cdot\mathbf{b} = \sum_{i=1}^{d} a_i\, b_i$$

- **a · b** — the result: one single number
- **Σ** — add up over all d positions
- **a_i b_i** — multiply matching components

Hover over (or tap) a part to highlight it in the formula.

6. Try it: explore the dot product

Drag the tips of the two arrows. Watch how the dot product, the angle and the cosine similarity change. Notice the dot product is exactly zero when the arrows are at 90°.

Interactive demo — open *lesson-02.html* on the website to try it.

7. Matrices: tables of numbers

A **matrix** is a rectangular grid of numbers. A matrix with m rows and n columns has *shape* $m \times n$. Our house dataset (4 houses × 3 features) is a 4×3 matrix, conventionally called X:

$$X = \begin{bmatrix} 50 & 1 & 2 \\ 80 & 2 & 5 \\ 120 & 3 & 12 \\ 150 & 4 & 3 \end{bmatrix}$$

Each row is one example; each column is one feature. This convention is used throughout ML and in scikit-learn. The entry in row i, column j is written X_{ij}; for example, $X_{32} = 3$ (house 3 has 3 bedrooms).

8. Matrix × vector: predicting for all examples at once

Multiplying a matrix by a vector means taking the **dot product of each row** with the vector:

$$X\mathbf{w} = \begin{bmatrix} \text{row}_1 \cdot \mathbf{w} \\ \text{row}_2 \cdot \mathbf{w} \\ \vdots \end{bmatrix}$$

✎ WORKED EXAMPLE — FOUR PREDICTIONS IN ONE STEP

With $\mathbf{w} = (3, 20, -5)$:

Row 1: $3 \cdot 50 + 20 \cdot 1 - 5 \cdot 2 = 160$
Row 2: $3 \cdot 80 + 20 \cdot 2 - 5 \cdot 5 = 255$
Row 3: $3 \cdot 120 + 20 \cdot 3 - 5 \cdot 12 = 360$
Row 4: $3 \cdot 150 + 20 \cdot 4 - 5 \cdot 3 = 515$

$$X\mathbf{w} = \begin{bmatrix} 160 \\ 255 \\ 360 \\ 515 \end{bmatrix}$$

One expression, $\hat{\mathbf{y}} = X\mathbf{w}$, produced predictions for every house. That's why ML is written with matrices.

? MAKE SENSE OF IT

The situation: Each row is one house: size (m²), bedrooms and age (years). The weights say each m² adds 3, each bedroom 20, and each year of age removes 5.

What the result tells you: 160, 255, 360 and 515 are the model's predicted prices for the four houses — computed in one matrix–vector product instead of four separate sums.

What if…?

- **House 1 gains a bedroom** (1 → 2): its prediction rises by exactly the bedroom weight, 160 → **180**.
- **House 1 is 10 years older** (2 → 12): 10 × (−5) = −50, so the prediction drops to **110**.
- **Change one weight** (size 3 → 4): every prediction rises by that house's size — **210, 335, 480, 665**. One weight affects every row.

Why it matters in ML: Training adjusts the weights so that $X\mathbf{w}$ matches the real prices. Because every weight touches every prediction, changing one weight is a global change — which is exactly what gradient descent does, a little at a time.

Shape rule: an $(m \times n)$ matrix times an n-vector gives an m-vector. The inner sizes (n) must match — each row needs exactly as many numbers as the vector has.

9. Matrix × matrix

To multiply A $(m \times n)$ by B $(n \times p)$, compute every row of A dotted with every column of B. The result $C = AB$ has shape $m \times p$, with $C_{ij} = (\text{row } i \text{ of } A) \cdot (\text{column } j \text{ of } B)$.

✎ WORKED EXAMPLE

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

? MAKE SENSE OF IT

The situation: A neural network applies one transformation after another. Multiplying their matrices gives a single matrix that does both steps at once.

What the result tells you: Each entry of the result is a row of the first matrix dotted with a column of the second: $19 = 1 \cdot 5 + 2 \cdot 7$.

What if...?

- **Swap the order:** $BA = \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}$, not the same as AB. Doing step B then step A is a different overall transformation.

Why it matters in ML: Layers in a network are matrix multiplications applied in a fixed order. It's also why a stack of purely linear layers collapses into one matrix — and why networks need non-linear activations between them (Lesson 18).

! ORDER MATTERS

In general $AB \neq BA$. And the shapes must line up: $(m \times \mathbf{n})(\mathbf{n} \times p)$. If you ever see a "shapes not aligned" error in NumPy, this rule is why.

Neural networks (Lesson 18) are mostly matrix multiplications: each layer computes $W\mathbf{x} + \mathbf{b}$ for a weight matrix W.

10. Transpose and the identity matrix

The **transpose** $A^\top$ flips a matrix over its diagonal: rows become columns.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

A 2×3 matrix becomes 3×2. You'll see it constantly: the dot product can be written $\mathbf{a}^\top \mathbf{b}$, and the linear regression solution (Lesson 6) contains $X^\top X$.

The **identity matrix** I has 1s on the diagonal and 0s elsewhere. It acts like the number 1: $I\mathbf{x} = \mathbf{x}$.

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

11. Doing it in Python with NumPy

NumPy is the Python library for vectors and matrices. Everything above, in a few lines:

```
import numpy as np

a = np.array([2, 3, 1])
b = np.array([4, -1, 5])
print(a + b)             # [6 2 6]      addition
print(3 * a)             # [6 9 3]      scalar multiplication
print(a @ b)             # 10           dot product
print(np.linalg.norm(np.array([3, 4])))   # 5.0   length

X = np.array([[50, 1, 2],
              [80, 2, 5],
              [120, 3, 12],
              [150, 4, 3]])
w = np.array([3, 20, -5])
print(X.shape)           # (4, 3)  4 examples, 3 features
print(X @ w)             # [160 255 360 515]   all predictions at once
print(X.T.shape)         # (3, 4)  transpose
```

The `@` operator means matrix multiplication (or a dot product, for two vectors). Run this and compare with the hand calculations above — they match.

12. Running example: Chennai Homes

Each house becomes a vector of numbers, and the whole table becomes a matrix X with 12 rows and 4 columns. Houses 1 and 2 are 26 units apart (mostly because their sizes differ by 25 m²). Multiplying X by a weight vector gives a prediction for every house at once.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
X = df[["size_m2", "bedrooms", "dist_km", "age_yrs"]].to_numpy()
print(X.shape)                                  # 12 houses x 4 features
a, b = X[0], X[1]                               # house 1 and house 2 as vectors
print(a, b, round(np.linalg.norm(a - b), 2))    # their Euclidean distance
w = np.array([0.6, 2.0, -1.2, -0.5])            # some made-up weights
print((X @ w)[:3].round(1))                     # predictions for the first 3
houses
```

Output:

```
(12, 4)
[65  1  2  5] [90  2  3 12] 26.0
[36.1 48.4 63.2]
```

⚙ WHERE THIS IS USED

- Representing customers, images and words as vectors
- Search engines and recommenders comparing items with cosine similarity
- GPUs running matrix multiplications for neural networks

Choose it when: Always — this is the notation for nearly all ML.

× MISCONCEPTIONS TO UNLEARN

MYTH The dot product measures distance.

REALITY It measures alignment (and depends on lengths). Distance is $\|\mathbf{a} - \mathbf{b}\|$ — two very different things.

MYTH A matrix is just storage for numbers.

REALITY A matrix also represents an operation: multiplying by it transforms vectors. That's what each neural-network layer does.

RESPONSIBLE AI · TURNING PEOPLE INTO VECTORS

Representing a person as a list of numbers hides choices. A feature such as a postcode can act as a **proxy** for ethnicity or income even if those columns are removed, so a model can discriminate without ever "seeing" a protected attribute. Removing a sensitive column is not enough; check what the remaining features correlate with.

13. Questions beginners often ask

Q. Why use matrices instead of loops?

They describe operations on all examples at once in one short expression, and libraries like NumPy run them in fast compiled code — often hundreds of times faster than a Python loop.

Q. What's the difference between a vector and a list?

In code they look alike. Mathematically, a vector supports operations like addition, scaling and dot products, and has a length and a direction.

Q. Do I have to multiply matrices by hand?

No — NumPy's `@` does it. Doing a few small ones by hand builds the intuition you need to read formulas and fix shape errors.

! COMMON MISTAKES TO AVOID

- Multiplying matrices whose inner sizes don't match — check shapes first: $(m \times n)(n \times p)$.
- Assuming $AB = BA$. Matrix multiplication order matters.
- Mixing up NumPy's `*` (element-wise) with `@` (matrix multiplication / dot product).

! INTERVIEW CORNER

Don't confuse

- **Dot product vs element-wise product** — Dot product ends in a sum.
- **Distance vs similarity** — Distance: smaller is closer. Similarity: bigger is closer.
- **L1 norm vs L2 norm** — L1 walks the streets; L2 flies.

Hidden questions — answer in your head first, then open.

▼ **Is cosine similarity a distance? What does it ignore?**

The trap: Treating 1 − cosine as a proper distance.

It measures the angle between vectors and ignores their lengths, so (1, 1) and (100, 100) are identical to it. $1 - \cos$ is not a true metric: it can violate the triangle inequality. Use it when direction matters more than magnitude, as with text embeddings.

▼ **Can the nearest neighbour change if you switch from Euclidean (L2) to Manhattan (L1) distance?**

The trap: Assuming distances always agree on order.

Yes. From (0, 0): the point (3, 0) is at L1 distance 3 and L2 distance 3; the point (2, 2) is at L1 4 but L2 2.83. L1 picks (3, 0); L2 picks (2, 2).

▼ **In NumPy, what's the difference between `A * B` and `A @ B`?**

The trap: Using `*` for matrix multiplication.

`*` multiplies element by element (and broadcasts). `@` is matrix multiplication. A forward pass needs `@`; using `*` can silently give a wrong answer instead of an error if the shapes happen to broadcast.

▼ **Why do we write predictions as $X\mathbf{w}$ with examples as rows?**

The trap: Thinking the order is arbitrary.

With X of shape (n, d) (one row per example) and $\mathbf{w}$ of shape $(d,)$, $X\mathbf{w}$ gives one prediction per row. It's a convention, but it fixes every other shape — in a neural layer, inputs (n, d) times weights (d, h) give outputs (n, h).

14. Key takeaways

- A **vector** is a list of numbers — one example's features, or a model's weights.
- A **matrix** is a table — a dataset X has one row per example and one column per feature.
- The **norm** measures length; $\|\mathbf{a} - \mathbf{b}\|$ measures distance between examples.
- The **dot product** $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models make predictions, and it measures how aligned two vectors are.
- $X\mathbf{w}$ computes predictions for all examples at once. Inner dimensions must match.

15. Check your understanding

1. What is $(1, 2, 3) \cdot (4, 0, -1)$?

a. -3

b. 1

c. 7

2. What is the length of $(6, 8)$?

a. 14

b. 7

c. 10

3. A dataset has 500 examples and 20 features. What is the shape of X?

a. 500 × 20

b. 20 × 500

c. 520 × 1

4. Two vectors have a dot product of 0. What does that tell you?

a. They are identical

b. They are perpendicular

c. One of them must be zero

5. A is 3×4 and B is 4×2. What is the shape of AB?

a. 4 × 4

b. It's not allowed

c. 3 × 2

16. Going deeper: the linear algebra behind ML algorithms

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above gives you what you need to *use* vectors and matrices in ML code. This section covers the ideas that explain *why* algorithms behave as they do: why lasso and ridge give different weights, why some regressions can't be solved, how PCA and recommender systems find hidden structure, and why nearly duplicate features make a model unstable. Each idea is used again in a later lesson, which is named as you go.

You'll need: everything in this lesson, especially the dot product, matrix × matrix and the transpose.

Other ways to measure size: norms

Section 4 measured a vector's length with the familiar straight-line distance. That's one **norm**, a rule that gives every vector a size. ML uses several, and the choice changes how models behave:

- **L2 norm** $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + \cdots + v_n^2}$: the straight-line length. Big entries count a lot, because they're squared.
- **L1 norm** $\|\mathbf{v}\|_1 = |v_1| + \cdots + |v_n|$: the "city-block" distance, as if you could only walk along streets. Every entry counts in proportion to its size.
- **L∞ norm** $\|\mathbf{v}\|_\infty = \max_i |v_i|$: only the biggest entry counts.
- **Frobenius norm** of a matrix, $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$: the L2 norm of all its entries treated as one long list. It measures how far apart two matrices are, as in the SVD example below.

```
import numpy as np

v = np.array([3.0, -4.0, 1.0])
print("L1 norm (sum of sizes):    ", np.linalg.norm(v, 1))
print("L2 norm (straight line):   ", round(np.linalg.norm(v), 3))
print("L-infinity (biggest entry):", np.linalg.norm(v, np.inf))

M = np.array([[1.0, 2.0], [3.0, 4.0]])
print("Frobenius norm of M:       ", round(np.linalg.norm(M), 3))
```

Output:

```
L1 norm (sum of sizes):     8.0
L2 norm (straight line):    5.099
L-infinity (biggest entry): 4.0
Frobenius norm of M:        5.477
```

For $\mathbf{v} = (3, -4, 1)$: L1 is $3 + 4 + 1 = 8$, L2 is $\sqrt{9 + 16 + 1} = \sqrt{26} \approx 5.099$, and L∞ is 4. **Why it matters:** ridge regression (Lesson 10) penalises the L2 norm of the weights and shrinks them all a little; lasso penalises the L1 norm and pushes some exactly to zero. The difference comes from the shapes of the "unit balls" $\|\mathbf{v}\| = 1$: a circle for L2, but a diamond with sharp corners on the axes for L1. kNN (Lesson 11) can use L1 or L2 distance too.

Independence, rank and the determinant

Columns of a matrix are **linearly independent** if none of them can be built from the others by adding and scaling. The **rank** is the number of independent columns. It matters because least squares needs $X^\top X$ to be invertible, and that happens only when the columns of X are independent (full rank).

The **determinant** of a square matrix is the factor by which it scales areas (in 2D) or volumes (in 3D). If $\det A = 0$, the matrix squashes space flat, information is lost, and A has no inverse.

```
# Three features, but the third is the sum of the first two
X = np.array([[1, 2, 3],
              [2, 0, 2],
              [0, 1, 1],
              [3, 1, 4]], dtype=float)
print("columns:", X.shape[1], " rank:", np.linalg.matrix_rank(X))
print("det(XᵀX):", round(np.linalg.det(X.T @ X), 6))

A = np.array([[2.0, 1.0], [1.0, 3.0]])
print("det(A):", round(np.linalg.det(A), 3), "→ A scales areas by this factor")
```

Output:

```
columns: 3  rank: 2
det(XᵀX): 0.0
det(A): 5.0 → A scales areas by this factor
```

The third feature is exactly feature 1 + feature 2, so only 2 of the 3 columns carry new information: rank 2, and $\det(X^\top X) = 0$. This is the same problem as the dummy-variable trap in Lesson 6. For the 2×2 matrix A, $\det A = 2 \cdot 3 - 1 \cdot 1 = 5$: it stretches every shape to 5 times its area.

Eigenvalues and eigenvectors

Most vectors change direction when you multiply them by a matrix. An **eigenvector** is a special direction that the matrix only stretches or shrinks:

$$A\mathbf{v} = \lambda\mathbf{v}$$

The number λ is its **eigenvalue**: how much that direction is stretched. To find the eigenvalues by hand, solve $\det(A - \lambda I) = 0$. For $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$:

$$\det\begin{pmatrix} 2-\lambda & 1 \\ 1 & 3-\lambda \end{pmatrix} = (2-\lambda)(3-\lambda) - 1 = \lambda^2 - 5\lambda + 5 = 0 \;\Rightarrow\; \lambda = \frac{5 \pm \sqrt{5}}{2} \approx 1.382,\ 3.618$$

For $\lambda = 3.618$, the first row of $(A - \lambda I)\mathbf{v} = 0$ gives $v_2 = 1.618\, v_1$; scaled to length 1, that's $\mathbf{v} \approx (0.526, 0.851)$.

```
vals, vecs = np.linalg.eigh(A)            # eigh: for symmetric matrices
print("eigenvalues:", vals.round(3))
print("eigenvectors (columns):")
print(vecs.round(3))
v1 = vecs[:, 1]
print("A @ v:     ", (A @ v1).round(3))
print("lambda * v:", (vals[1] * v1).round(3))
print("dot product of the two eigenvectors:", abs(round(vecs[:, 0] @ vecs[:, 1], 10)))
```

Output:

```
eigenvalues: [1.382 3.618]
eigenvectors (columns):
[[-0.851  0.526]
 [ 0.526  0.851]]
A @ v:      [1.902 3.078]
lambda * v: [1.902 3.078]
dot product of the two eigenvectors: 0.0
```

The check $A\mathbf{v} = \lambda\mathbf{v}$ works, and the two eigenvectors have dot product 0: they're at right angles. That's not luck. The **spectral theorem** says every *symmetric* matrix ($A = A^\top$) has real eigenvalues and a full set of perpendicular eigenvectors, so it can be written as $A = Q\Lambda Q^\top$ (rotate, stretch along the axes, rotate back). Covariance matrices are symmetric, which is exactly why PCA (Lesson 17) works: its components are the covariance matrix's eigenvectors, and each eigenvalue is the variance along that direction. Use `np.linalg.eigh` for symmetric matrices; it's faster and more accurate than the general `eig`.

Positive definite matrices

A symmetric matrix is **positive definite** if $\mathbf{x}^\top A\mathbf{x} > 0$ for every non-zero $\mathbf{x}$, which is the same as all its eigenvalues being positive. It's **positive semi-definite** (PSD) if ≥ 0 is allowed. The matrix A above is positive definite: both eigenvalues are positive.

The matrix that appears all over ML, $X^\top X$, is always PSD, and the proof is one line:

$$\mathbf{w}^\top (X^\top X)\,\mathbf{w} = (X\mathbf{w})^\top (X\mathbf{w}) = \|X\mathbf{w}\|^2 \geq 0$$

It's positive definite exactly when the columns of X are independent. Lesson 3's "Going deeper" uses this to show that the squared-error loss is *convex*: shaped like a bowl, with one lowest point.

The singular value decomposition (SVD)

Eigenvectors need a square matrix. The **SVD** works for *any* matrix, including a data table with n rows and d columns:

$$A = U\,\Sigma\,V^\top$$

U and V have perpendicular columns (directions in row space and column space), and Σ is diagonal with the **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$, which say how important each direction is. The key result (the Eckart–Young theorem) is that keeping only the k largest singular values gives the *best possible* rank-k approximation of A.

Here's why that's useful. Below is a table of 50 users' ratings of 40 films. It was secretly built from just 3 hidden "tastes" (say, action, comedy and drama) plus noise:

```
# A ratings table: 50 users × 40 films, built from 3 hidden "tastes" plus noise
rng = np.random.default_rng(0)
R = rng.normal(size=(50, 3)) @ rng.normal(size=(3, 40)) + rng.normal(0, 0.3, size=(50,
40))
U, s, Vt = np.linalg.svd(R, full_matrices=False)
print("largest singular values:", s[:6].round(1))
for k in [1, 2, 3, 5, 10]:
    approx = U[:, :k] @ np.diag(s[:k]) @ Vt[:k]          # keep only the k largest
    err = np.linalg.norm(R - approx) / np.linalg.norm(R)
    print(f"rank {k:2d}: stores {k * (50 + 40 + 1):4d} of 2000 numbers, relative error
{err:.3f}")
```

Output:

```
largest singular values: [54.9 40.6 38.5  3.9  3.6  3.4]
rank  1: stores   91 of 2000 numbers, relative error 0.722
rank  2: stores  182 of 2000 numbers, relative error 0.510
rank  3: stores  273 of 2000 numbers, relative error 0.157
rank  5: stores  455 of 2000 numbers, relative error 0.142
rank 10: stores  910 of 2000 numbers, relative error 0.110
```

The singular values drop sharply after the third (38.5, then 3.9): the SVD has discovered that 3 hidden factors explain the table. A rank-3 approximation stores 273 numbers instead of 2,000, with a relative error of 0.157, and most of that error is the random noise. Keeping more singular values hardly helps (0.142 at rank 5), because they only describe noise. This is the idea behind recommender systems (predicting missing ratings), PCA (Lesson 17, which is the SVD of the centred data), image compression and the **pseudo-inverse** $A^{+} = V\Sigma^{+}U^{\top}$ that `np.linalg.lstsq` uses to solve least squares safely.

Condition number: when tiny changes cause big swings

The **condition number** of a matrix is the ratio of its largest to its smallest singular value (for a symmetric PSD matrix like $X^{\top}X$, its largest to smallest eigenvalue). It measures how much small errors in the input can be amplified in the answer. A value near 1 is perfect; values in the thousands or millions mean trouble. In this example, two features measure almost the same thing: a home's size, recorded twice with slightly different tape measures.

```
rng = np.random.default_rng(2)
n = 100
size = rng.uniform(50, 150, n)
size2 = size + rng.normal(0, 0.5, n)                # a second, almost identical
measurement
Xc = np.column_stack([size, size2]); Xc = Xc - Xc.mean(axis=0)
y = 0.6 * Xc[:, 0] + rng.normal(0, 5, n)
y_nudged = y + rng.normal(0, 1, n)                  # a small change in the targets

G = Xc.T @ Xc
print("condition number of XᵀX:", round(np.linalg.cond(G)))
print("weights:            ", np.linalg.solve(G, Xc.T @ y).round(2))
print("after a tiny change:", np.linalg.solve(G, Xc.T @ y_nudged).round(2))
Gr = G + 100 * np.eye(2)                            # ridge: add λI with λ = 100
print("condition number with ridge:", round(np.linalg.cond(Gr)))
print("ridge weights:      ", np.linalg.solve(Gr, Xc.T @ y).round(2))
print("after a tiny change:", np.linalg.solve(Gr, Xc.T @ y_nudged).round(2))
```

Output:

```
condition number of XᵀX: 14469
weights:             [ 0.96 -0.36]
after a tiny change: [ 1.2  -0.61]
condition number with ridge: 1462
ridge weights:       [0.37 0.23]
after a tiny change: [0.39 0.21]
```

With a condition number of about 14,000, a tiny change in y swings the weights from $(0.96, -0.36)$ to $(1.2, -0.61)$. Their *sum* stays near 0.6, so the predictions barely change, but the individual weights mean nothing. This is the linear-algebra view of multicollinearity (Lesson 6's "Going deeper"). Adding λI, which is exactly what ridge regression does, raises every eigenvalue by λ. That cuts the condition number about tenfold, to 1,462, and the weights become stable ($0.37, 0.23$ and then $0.39, 0.21$).

Matrix calculus: the four rules you'll actually use

Many ML derivations need the gradient of an expression with vectors and matrices in it. These rules cover almost all of them ($\mathbf{a}$ is a constant vector and A a constant matrix):

Function of $\mathbf{w}$	Gradient $\nabla_{\mathbf{w}}$	Like the 1-D rule
$\mathbf{a}^\top \mathbf{w}$	$\mathbf{a}$	$\frac{d}{dw}(aw) = a$
$\mathbf{w}^\top \mathbf{w} = \|\mathbf{w}\|^2$	$2\mathbf{w}$	$\frac{d}{dw} w^2 = 2w$
$\mathbf{w}^\top A\, \mathbf{w}$	$(A + A^\top)\, \mathbf{w}$, which is $2A\mathbf{w}$ if A is symmetric	$\frac{d}{dw} aw^2 = 2aw$
$\|X\mathbf{w} - \mathbf{y}\|^2$	$2X^\top (X\mathbf{w} - \mathbf{y})$	chain rule

The last rule gives the normal equation from Lesson 6 in one line: set $2X^\top(X\mathbf{w} - \mathbf{y}) = \mathbf{0}$, so $X^\top X\, \mathbf{w} = X^\top \mathbf{y}$. Always check a hand-derived gradient numerically, by nudging each weight a tiny amount (Lesson 3 shows how):

```
# Check the matrix-calculus rule  ∇_w ||Xw - y||² = 2 Xᵀ(Xw - y)  with finite
differences
rng = np.random.default_rng(3)
Xg = rng.normal(size=(5, 3)); yg = rng.normal(size=5); w = rng.normal(size=3)
f = lambda w: np.sum((Xg @ w - yg) ** 2)
formula = 2 * Xg.T @ (Xg @ w - yg)
h = 1e-6
numeric = np.array([(f(w + h * e) - f(w - h * e)) / (2 * h) for e in np.eye(3)])
print("formula:", formula.round(4))
print("numeric:", numeric.round(4))
```

Output:

```
formula: [-13.3106  19.8282   2.4238]
numeric: [-13.3106  19.8282   2.4238]
```

The formula and the numerical estimate agree to 4 decimal places, so the rule is right. Deep-learning libraries compute such gradients automatically, but knowing these rules lets you read papers and debug models.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Why does L1 regularization produce exactly-zero weights, while L2 only shrinks them?**

The trap: "Because L1 is stronger."

It's about shape, not strength. The L1 ball $|w_1| + |w_2| \le t$ is a diamond with corners on the axes. When the loss's contours first touch it, they usually touch at a corner, where some weights are exactly 0. The L2 ball is round, so the touching point is almost never on an axis. Equivalently, the L1 penalty's slope stays the same size right up to zero, so it keeps pushing until a weight reaches 0. The L2 penalty's slope $2\lambda w$ fades as w shrinks.

▼ **What is an eigenvector, and where do eigenvectors appear in ML?**

The trap: Giving only the equation, with no meaning.

A direction that a matrix only stretches, $A\mathbf{v} = \lambda\mathbf{v}$. In ML: PCA's components are the eigenvectors of the covariance matrix, with eigenvalues equal to the variance in each direction. The eigenvalues of the Hessian describe a loss surface's curvature (and so how fast gradient descent converges). Spectral clustering and Google's PageRank are also eigenvector methods.

▼ **What's the relationship between PCA and the SVD?**

The trap: Treating them as unrelated algorithms.

Centre the data matrix X and take its SVD, $X = U\Sigma V^\top$. The columns of V are the principal components (the eigenvectors of $X^\top X$), and $\sigma_i^2/(n-1)$ is the variance along each one. Libraries compute PCA through the SVD because it's more accurate than forming $X^\top X$, which squares the condition number.

▼ **Your linear regression's weights change wildly when you add a few rows of data. What's going on, and how do you fix it?**

The trap: Blaming the optimiser or the random seed.

Probably ill-conditioning: some features are nearly linear combinations of others, so $X^\top X$ has a tiny eigenvalue and a huge condition number. Small data changes then cause big swings along that weak direction. Check the VIFs or the condition number. Fix it by removing or combining redundant features, using PCA, or adding ridge regularization, which adds λ to every eigenvalue.

▼ **Why is $X^\top X$ never negative definite, and when is it invertible?**

The trap: "It's always invertible because it's square."

$\mathbf{w}^\top X^\top X\mathbf{w} = \|X\mathbf{w}\|^2 \geq 0$, so it is always positive semi-definite. It is invertible (positive definite) exactly when the columns of X are linearly independent, which requires at least as many rows as columns. With more features than examples ($d > n$) it is always singular.

Check your understanding: going deeper

1. What is the L1 norm of $(2, -3, 1)$?

a. $\sqrt{14}$

b. 6

c. 3

2. A data matrix has 5 columns but rank 4. What does that tell you?

a. One column can be built from the others, so $X^\top X$ can't be inverted

b. One row is missing

c. The model will be 80% accurate

3. $A\mathbf{v} = 3\mathbf{v}$. What does the matrix A do to the direction of $\mathbf{v}$?

a. Rotates it by 3 degrees

b. Keeps the direction and stretches it 3 times

c. Flips it

4. The singular values of a data table are 90, 60, 2, 1 and 0.5. A good low-rank summary keeps about...

a. 1 component

b. 2 components

c. All 5 components

5. Ridge regression adds λI to $X^\top X$. What does that do to its eigenvalues?

a. Multiplies each by λ

b. Adds λ to each, so none is near zero

c. Nothing

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

For $\mathbf{a} = (3, -1, 2)$ and $\mathbf{b} = (1, 4, 0)$ compute $\mathbf{a} + \mathbf{b}$, $2\mathbf{a} - \mathbf{b}$ and $\mathbf{a} \cdot \mathbf{b}$.

WORKING

EXERCISE 2 By hand

Find the length of $(2, -3, 6)$ and the cosine similarity between $(1, 0)$ and $(1, 1)$.

WORKING

EXERCISE 3 By hand

$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ and $\mathbf{w} = (2, -1)$. Compute $X\mathbf{w}$. What shape is AB if A is 2×3 and B is 3×4?

WORKING

EXERCISE 4 Code

Check all three answers above with NumPy.

WORKING

EXERCISE 5 Intermediate

For $\mathbf{v} = (1, -2, 2)$, compute the L1, L2 and L∞ norms.

WORKING

EXERCISE 6 Intermediate

Find the eigenvalues of $A = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$. Give the smaller one first.

WORKING

EXERCISE 7 Intermediate

A matrix has singular values $10, 4, 1$. Keeping only the largest, what fraction of the squared Frobenius norm $\sum \sigma_i^2$ does the rank-1 approximation capture?

WORKING

EXERCISE 8 Intermediate

A symmetric matrix has eigenvalues 200 and 0.5. What is its condition number? After adding λI with $\lambda = 9.5$, what is it?

WORKING

EXERCISE 9 Code · intermediate

Use `np.linalg.svd` on the ratings table `R` from the "Going deeper" code. What fraction of $\sum \sigma_i^2$ do the first 3 singular values capture?

WORKING

NOTES

PART 1 · FOUNDATIONS · ~50 MIN

Lesson 3: Derivatives & Gradients

Slopes, derivatives, the chain rule, partial derivatives and the gradient — the maths of learning.

BY THE END OF THIS LESSON YOU CAN

- Explain a derivative as a slope and compute it from the definition
- Apply the power, sum and chain rules
- Compute partial derivatives and assemble a gradient
- Explain why gradient descent moves against the gradient

↺ BEFORE YOU START

This lesson builds on:

- Lesson 1: What Is Machine Learning? — A model maps features x to a prediction $\hat{y}$. Training adjusts its parameters to make a loss (such as MSE) small, and the real goal is doing well on new data.
- Lesson 2: Vectors & Matrices — A vector is a list of numbers; a dataset X is a matrix with one row per example. The dot product $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models predict, and $\|\mathbf{a} - \mathbf{b}\|$ is the distance between two examples.

Where this fits: Calculus answers "which way should each parameter move?". It's the engine behind gradient descent (Lesson 7) and backpropagation (Lesson 18).

1. Why calculus?

In Lesson 1 you fitted a line by moving sliders and watching the loss go up or down. A computer can't "watch"; it needs a precise answer to one question:

> ”If I change this parameter a tiny bit, does the loss go up or down — and how fast?”

That question is exactly what a **derivative** answers. With derivatives, an algorithm always knows which way to adjust every parameter. This lesson teaches the small amount of calculus that ML needs: slopes, derivatives, a few rules, partial derivatives and the gradient.

2. Slope of a straight line

For a straight line, the **slope** is "rise over run": how much y changes when x increases by 1.

$$\text{slope} = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

(Δ, "delta", means "change in".) For the line $y = 3x + 1$, going from $x = 1$ to $x = 2$ moves y from 4 to 7, so the slope is $3/1 = 3$. A straight line has the same slope everywhere.

3. The derivative: slope of a curve at a point

PICTURE IT You're driving and glance at the speedometer: 60 km/h. That's not your average speed for the trip — it's how fast your position is changing *right now*. A derivative is the speedometer reading of any function.

A curve like $f(x) = x^2$ is steeper in some places than others. Its slope *at a single point* is the slope of the **tangent line** — the straight line that just touches the curve there.

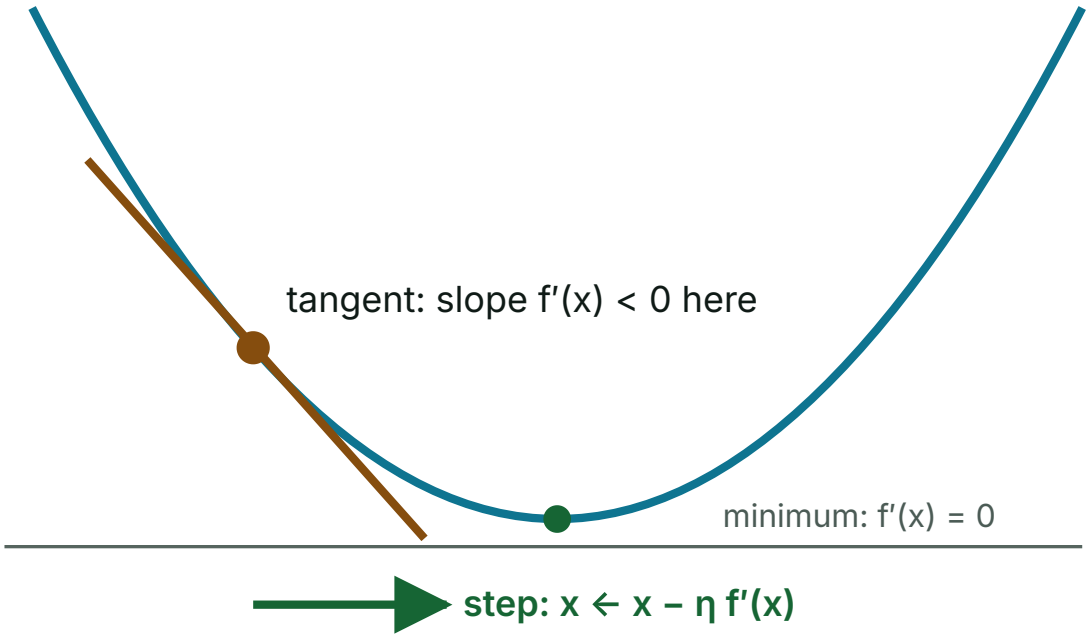


Figure 3. The derivative is the slope of the tangent line. Where the slope is negative, gradient descent moves right — downhill — until the slope is zero at the minimum.

To find it, take two points very close together, compute the slope between them, and let the gap h shrink towards zero:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$f'(x)$ (read "f prime of x"), also written $\frac{df}{dx}$, is the **derivative** of f.

✎ WORKED EXAMPLE — DERIVATIVE OF X^2 FROM SCRATCH

$f(x+h) = (x+h)^2 = x^2 + 2xh + h^2$. So

$$\frac{f(x+h) - f(x)}{h} = \frac{2xh + h^2}{h} = 2x + h$$

As $h \to 0$ this becomes $2x$. So $f'(x) = 2x$.

Check with numbers at $x = 3$: with $h = 0.001$, $\frac{(3.001)^2 - 9}{0.001} = \frac{0.006001}{0.001} = 6.001 \approx 6 = 2 \cdot 3$. ✓

? MAKE SENSE OF IT

The situation: Suppose x is how far a prediction is from the truth and $f(x) = x^2$ is its squared error. The derivative tells you how fast that error changes if you nudge the prediction.

What the result tells you: At $x = 3$ the slope is 6: move the prediction 0.01 further away and the squared error grows by about $6 \times 0.01 = 0.06$.

What if…?

- **At** $x = -3$ the slope is **−6**: same steepness, opposite direction — the error falls if you increase x.
- **At** $x = 0$ the slope is **0**: you're at the bottom, nothing to improve.
- **Bigger** h**:** the slope estimate is 7.0 with $h = 1$, 6.1 with $h = 0.1$ and 6.001 with $h = 0.001$ — the smaller the step, the closer to the true 6.

Why it matters in ML: The sign of the derivative tells a model which way to move a parameter, and its size tells how urgently. Far from the target (big x) the slope is steep; near the target it flattens — so learning naturally slows as it gets close.

◎ WHAT THE DERIVATIVE TELLS YOU

- $f'(x) > 0$: the function is **going up** at x — increasing x increases f.
- $f'(x) < 0$: the function is **going down**.
- $f'(x) = 0$: **flat** — often a minimum or maximum.
- The bigger $|f'(x)|$, the steeper the curve.

Imagine standing on a hillside in fog. You can't see the valley, but you can feel the slope under your feet. The derivative is that feeling.

STEP-BY-STEP DERIVATION **Derive the power rule for x^3 from the definition**

1 $$f'(x) = \lim_{h \to 0} \frac{(x+h)^3 - x^3}{h}$$

Start from the definition of the derivative.

2 $$(x+h)^3 = x^3 + 3x^2h + 3xh^2 + h^3$$

Expand the cube (multiply out $(x+h)(x+h)(x+h)$).

3 $$\frac{(x+h)^3 - x^3}{h} = \frac{3x^2h + 3xh^2 + h^3}{h}$$

The x^3 terms cancel.

4 $$= 3x^2 + 3xh + h^2$$

Divide every term by h.

5 $$f'(x) = 3x^2$$

Let $h \to 0$: the terms containing h vanish. This matches the power rule nx^{n-1} with $n = 3$.

READ THE FORMULA **Definition of the derivative**

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

- **lim h→0** — make the step smaller and smaller
- **f(x+h) – f(x)** — how much the output rises (the "rise")
- **h** — how far we stepped (the "run")

Hover over (or tap) a part to highlight it in the formula.

4. Try it: the tangent and its slope

Move the point along the curve $f(x) = (x-1)^2 + 0.5$. The orange line is the tangent; its slope is the derivative $f'(x) = 2(x-1)$. Notice the slope is negative on the left, positive on the right, and zero exactly at the bottom.

Interactive demo — open *lesson-03.html* on the website to try it.

5. The rules you actually need

You don't need to use limits every time. A few rules cover almost everything in ML:

Rule	Formula	Example
Constant	$\frac{d}{dx} c = 0$	$\frac{d}{dx} 7 = 0$
Power	$\frac{d}{dx} x^n = n x^{n-1}$	$\frac{d}{dx} x^3 = 3x^2$
Constant multiple	$\frac{d}{dx}[c\,f(x)] = c\,f'(x)$	$\frac{d}{dx} 5x^2 = 10x$
Sum	$\frac{d}{dx}[f+g] = f' + g'$	$\frac{d}{dx}(x^2 + 3x) = 2x + 3$
Exponential	$\frac{d}{dx} e^x = e^x$	—
Logarithm	$\frac{d}{dx} \ln x = \frac{1}{x}$	—

✎ WORKED EXAMPLE

$f(x) = 4x^3 - 2x^2 + 7x - 9$

Term by term: $4 \cdot 3x^2 - 2 \cdot 2x + 7 - 0 = 12x^2 - 4x + 7$.

? MAKE SENSE OF IT

The situation: Picture f as a cost curve. The derivative $12x^2 - 4x + 7$ gives its slope at any point, without drawing it.

What the result tells you: Each term is differentiated on its own and the results are added. The constant -9 disappears because a constant doesn't change.

What if...?

- **At** $x = 0$ the slope is **7**; at $x = 1$ it's **15**; at $x = -1$ it's **23**.
- **At** $x = 10$ it's **1,167**, and almost all of that comes from the $12x^2$ term. For large inputs the highest power dominates.

Why it matters in ML: Loss functions are built from sums of simple terms, so their gradients are too. This term-by-term rule is what automatic differentiation in PyTorch applies millions of times per second.

6. The chain rule — the engine of deep learning

Often one function is inside another: $f(g(x))$. The **chain rule** says: derivative of the outside (evaluated at the inside) × derivative of the inside.

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

Or, with $u = g(x)$: $\frac{df}{dx} = \frac{df}{du} \cdot \frac{du}{dx}$ — the "du"s look like they cancel, which is a handy memory aid.

✎ WORKED EXAMPLE

$f(x) = (3x + 1)^2$. Outside: u^2, inside: $u = 3x + 1$.

$\frac{df}{du} = 2u = 2(3x + 1)$, $\frac{du}{dx} = 3$. So $f'(x) = 2(3x + 1) \cdot 3 = 6(3x + 1)$.

Check at $x = 0$: $f'(0) = 6$. Numerically, $\frac{(3 \cdot 0.001 + 1)^2 - 1}{0.001} = \frac{1.006009 - 1}{0.001} \approx 6.009$. ✓

? MAKE SENSE OF IT

The situation: A network is a chain: the input goes through one step, then another. Here the inside step multiplies by 3 and adds 1; the outside step squares.

What the result tells you: The chain rule multiplies the slopes of the steps: the outside slope $2(3x+1)$ times the inside slope 3. At $x=0$ that's $2 \times 1 \times 3 = 6$.

What if...?

- **Make the inside steeper** $(5x+1)$: the slope at 0 becomes $2 \times 1 \times 5 =$ **10**. The inside factor is multiplied straight in.
- **At** $x=1$ the slope is $6 \times 4 =$ **24**.

Why it matters in ML: Backpropagation is the chain rule through many layers, so gradients are products of many factors. If those factors are mostly below 1 the product shrinks towards zero (vanishing gradients); if above 1 it can blow up (exploding gradients) — Lesson 19.

A neural network is a long chain of functions inside functions. **Backpropagation** (Lesson 18) is just the chain rule applied over and over, from the output back to the first layer.

7. Applying it: the derivative of a loss

Now the payoff. Take the simplest model, $\hat{y} = wx$ (no bias), and one data point $(x, y) = (2, 6)$. The squared error is:

$$L(w) = (wx - y)^2 = (2w - 6)^2$$

Using the chain rule (outside: u^2; inside: $u = 2w - 6$, whose derivative is 2):

$$\frac{dL}{dw} = 2(2w - 6) \cdot 2 = 4(2w - 6)$$

- At $w=1$: $\frac{dL}{dw} = 4(2-6) = -16$. Negative → the loss **decreases** if we increase w. So increase w.
- At $w=5$: $\frac{dL}{dw} = 4(10-6) = 16$. Positive → decrease w.
- At $w=3$: $\frac{dL}{dw} = 0$. Flat — the minimum. And indeed $3 \cdot 2 = 6$, a perfect prediction.

The rule "**move against the derivative**" always takes us towards lower loss. That single idea is gradient descent.

8. Partial derivatives: more than one parameter

Real models have many parameters, e.g. $L(w, b)$. A **partial derivative** asks how L changes when you nudge *one* parameter while holding all others fixed. It's written with a curly ∂:

$$\frac{\partial L}{\partial w}, \quad \frac{\partial L}{\partial b}$$

To compute one, treat every other variable as a constant and differentiate normally.

✎ WORKED EXAMPLE

$f(w, b) = w^2 + 3wb + b^2$

$\frac{\partial f}{\partial w} = 2w + 3b$ (treating b as a constant)
$\frac{\partial f}{\partial b} = 3w + 2b$ (treating w as a constant)

At $(w, b) = (1, 2)$: $\frac{\partial f}{\partial w} = 8$, $\frac{\partial f}{\partial b} = 7$.

? MAKE SENSE OF IT

The situation: Think of $f(w, b)$ as a model's loss with two knobs, w and b. Each partial derivative says what happens if you turn one knob and leave the other alone.

What the result tells you: At $(1, 2)$ the loss is 11. Its slope is 8 in the w direction and 7 in the b direction, so the gradient is $(8, 7)$.

What if...?

- **Turn w up by 0.1:** the loss becomes 11.81 — up by about $8 \times 0.1 = 0.8$, as predicted.
- **Turn b up by 0.1:** the loss becomes 11.71 — up by about 0.7.
- **Step against the gradient** with learning rate 0.1: $(1, 2) \to (0.2, 1.3)$ and the loss falls from 11 to **2.51**.

Why it matters in ML: This is one step of gradient descent in miniature. Real models have millions of knobs, but the recipe is the same: compute every partial derivative, then move each parameter a little against its own slope.

9. The gradient: the direction of steepest ascent

Collect all the partial derivatives into a vector and you get the **gradient**, written ∇L ("nabla L" or "grad L"):

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

The gradient has a beautiful property: it **points in the direction in which L increases fastest**. So $-\nabla L$ points downhill, towards lower loss. In the example above, $\nabla f(1, 2) = (8, 7)$.

◎ PICTURE IT

Think of the loss as a landscape: the ground's position is the parameters (w, b), the height is the loss. At any spot, the gradient is an arrow pointing straight uphill. Training a model means walking downhill until you reach a valley — a minimum of the loss.

ADVANCED · OPTIONAL **Why the gradient points in the direction of steepest ascent** ▾ hide

The slope of f in a direction given by a unit vector $\mathbf{u}$ is the *directional derivative* $D_{\mathbf{u}} f = \nabla f \cdot \mathbf{u}$ (chain rule along the line $\mathbf{x} + t\mathbf{u}$). By Cauchy–Schwarz,

$$\nabla f \cdot \mathbf{u} \le \|\nabla f\| \, \|\mathbf{u}\| = \|\nabla f\|,$$

with equality exactly when $\mathbf{u} = \nabla f / \|\nabla f\|$. So no direction climbs faster than the gradient, the steepest slope is $\|\nabla f\|$, and $-\nabla f$ is the steepest way down — the justification for gradient descent.

10. Preview: gradient descent in one line

Put it together and you get the learning rule used by almost every modern ML model:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \nabla L(\mathbf{w})$$

"Take the current parameters, and step a little bit (η, the *learning rate*) in the downhill direction." Repeat until the loss stops falling. Use the *Step* button in the demo above to watch this happen on a curve. Lesson 7 covers it in depth.

11. Checking derivatives with code

You can always check a derivative numerically with a tiny h — a habit professionals use to catch bugs:

```
def f(x):
    return (3 * x + 1) ** 2

def f_prime(x):          # our hand-derived answer
    return 6 * (3 * x + 1)

def numeric_derivative(f, x, h=1e-6):
    return (f(x + h) - f(x - h)) / (2 * h)

for x in [0, 1, -2]:
    print(x, f_prime(x), round(numeric_derivative(f, x), 4))
# 0 6 6.0
# 1 24 24.0
# -2 -30 -30.0
```

And gradient descent on the loss $L(w) = (2w - 6)^2$ from the example above:

```
w = 0.0
lr = 0.05                       # learning rate (eta)
for step in range(30):
    grad = 4 * (2 * w - 6)      # dL/dw
    w = w - lr * grad           # move downhill
print(round(w, 4))              # 3.0  → the perfect weight
```

12. Running example: Chennai Homes

Take the simplest model, $\text{price} = w \times \text{size}$. The derivative of the loss tells us which way to move w: at $w = 0.4$ the slope is strongly negative (increase w), at $w = 0.9$ it's strongly positive (decrease w), and near $w = 0.65$ it's almost flat — close to the best value (about 0.6545).

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
x, y = df["size_m2"].to_numpy(), df["price_lakh"].to_numpy()
def loss(w):            # MSE for the model price = w * size
    return np.mean((w * x - y) ** 2)
def grad(w):            # dL/dw = (2/n) * sum((w*x - y) * x)
    return 2 * np.mean((w * x - y) * x)
for w in [0.4, 0.65, 0.9]:
    print(w, round(loss(w), 1), round(grad(w), 1))
```

Output:

```
0.4 1116.2 -6130.0
0.65 336.4 -109.2
0.9 1061.7 5911.7
```

⚙ WHERE THIS IS USED

- Training every neural network and most other models
- Sensitivity analysis: how much does an output change if an input changes?

Choose it when: Whenever a model is trained by minimising a loss.

× MISCONCEPTIONS TO UNLEARN

MYTH A derivative of zero means you've found the minimum.

REALITY It could also be a maximum or a flat "saddle" point. For convex losses (one bowl) it is the minimum.

MYTH The gradient points to the minimum.

REALITY It points in the direction of steepest increase at the current spot — not at any particular destination. That's why we take many small steps.

RESPONSIBLE AI · YOU GET WHAT YOU OPTIMISE

Gradient-based learning makes the loss small — nothing else. If the measurable target is only a stand-in for what you care about, the model will optimise the stand-in (Goodhart's law: "when a measure becomes a target, it ceases to be a good measure"). Choosing the loss and the target is a value judgement, not just a technical detail.

13. Questions beginners often ask

Q. Why do we need derivatives at all?

To know which direction to move each parameter to reduce the loss. Without them you'd have to guess, which becomes hopeless with thousands of parameters.

Q. Will I have to differentiate everything by hand?

No. Libraries compute gradients automatically. Understanding what a gradient means is what lets you choose learning rates and diagnose training problems.

Q. Why does gradient descent subtract the gradient?

The gradient points uphill (towards higher loss), so stepping in the opposite direction goes downhill.

! COMMON MISTAKES TO AVOID

- Forgetting the inner derivative in the chain rule: $\frac{d}{dx}(3x+1)^2 \neq 2(3x+1)$; it's $6(3x+1)$.
- Moving *with* the gradient instead of against it — that climbs uphill and increases the loss.
- Treating the other variables as zero (instead of as constants) when taking a partial derivative.

! INTERVIEW CORNER

Don't confuse

- **Derivative vs gradient** — Gradient = all the slopes at once, as a vector.
- **Gradient vs gradient descent** — One is a measurement, the other is a method.
- **Local minimum vs global minimum vs saddle point** — Zero slope doesn't mean the bottom.

Hidden questions — answer in your head first, then open.

▼ **The gradient is zero. Have you found the minimum?**

The trap: Assuming zero slope means the bottom.

Not necessarily: it could be a maximum, a local minimum or a saddle point. In high-dimensional neural networks, saddle points are far more common than poor local minima, and noise in SGD helps escape them.

▼ **ReLU isn't differentiable at 0. How can we train networks with it?**

The trap: Thinking every function must be differentiable everywhere.

The kink is a single point that inputs almost never hit exactly. Frameworks simply use a derivative of 0 (or 1) there — a valid *subgradient* — and training works fine.

▼ **Why does the gradient point in the direction of steepest *ascent*, and why do we subtract it?**

The trap: Thinking it points downhill.

The directional slope along a unit vector $\mathbf{u}$ is $\nabla f \cdot \mathbf{u}$, which is largest when $\mathbf{u}$ points along ∇f. To decrease the loss we move the opposite way: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$.

▼ **Why do frameworks compute gradients backwards (reverse mode) rather than forwards?**

The trap: Not knowing there's a choice.

A network has one scalar loss and millions of parameters. Reverse mode gets all the partial derivatives of one output in a single backward pass, at about the cost of one forward pass. Forward mode would need one pass per parameter.

14. Key takeaways

- The **derivative** $f'(x)$ is the slope of f at x: how fast f changes as x changes.
- Rules to remember: power rule nx^{n-1}, sums and constants, and the **chain rule** $f'(g(x))\,g'(x)$.
- A **partial derivative** changes one variable while holding the others fixed.
- The **gradient** ∇L is the vector of all partial derivatives; it points uphill.
- Learning = moving parameters **against** the gradient: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$.

15. Check your understanding

1. What is the derivative of $f(x) = 5x^4$?

a. $5x^3$

b. $4x^3$

c. $20x^3$

2. At some w, $\frac{dL}{dw} = -3$. To reduce the loss you should...

a. increase w

b. decrease w

c. leave w alone

3. Using the chain rule, what is $\frac{d}{dx}(2x - 5)^2$?

a. $2(2x - 5)$

b. $4(2x - 5)$

c. $(2x - 5)$

4. For $f(w, b) = 3w + wb$, what is $\frac{\partial f}{\partial w}$?

a. $3 + w$

b. $3 + b$

c. $w + b$

5. The gradient of the loss points...

a. towards the minimum

b. sideways along a contour

c. in the direction of steepest increase

16. Going deeper: calculus for optimisation

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above gives you derivatives, the chain rule and the gradient: enough to understand gradient descent. This section adds the tools that explain *how well* optimisation works: derivatives of whole vector functions (the Jacobian, used in backpropagation), curvature (the Hessian), why some losses have one clean minimum (convexity), a much faster method that uses curvature (Newton's method), how to test your own gradients, and how to optimise with constraints (Lagrange multipliers, used by SVMs and PCA).

You'll need: partial derivatives and the gradient from this lesson, and eigenvalues from Lesson 2's "Going deeper".

The Jacobian: derivatives of vector functions

The gradient handles a function that takes many numbers and returns **one**. A layer of a neural network takes many numbers and returns **many**. Its derivative is the **Jacobian**: a matrix with one row per output and one column per input, where entry (i, j) is $\partial f_i / \partial x_j$.

For $\mathbf{f}(x, y) = \big(x^2 y,\ 5x + \sin y\big)$:

$$J = \begin{pmatrix} \dfrac{\partial f_1}{\partial x} & \dfrac{\partial f_1}{\partial y} \\ \dfrac{\partial f_2}{\partial x} & \dfrac{\partial f_2}{\partial y} \end{pmatrix} = \begin{pmatrix} 2xy & x^2 \\ 5 & \cos y \end{pmatrix}, \qquad J(1, 2) = \begin{pmatrix} 4 & 1 \\ 5 & -0.4161 \end{pmatrix}$$

```
import numpy as np

def f(v):                                  # a function from 2 numbers to 2 numbers
    x, y = v
    return np.array([x**2 * y, 5 * x + np.sin(y)])

def jacobian_formula(v):
    x, y = v
    return np.array([[2 * x * y, x**2],    # row 1: ∂f1/∂x, ∂f1/∂y
                     [5.0, np.cos(y)]])    # row 2: ∂f2/∂x, ∂f2/∂y

def jacobian_numeric(f, v, h=1e-6):
    cols = [(f(v + h * e) - f(v - h * e)) / (2 * h) for e in np.eye(len(v))]
    return np.column_stack(cols)

v = np.array([1.0, 2.0])
print(jacobian_formula(v).round(4))
print(jacobian_numeric(f, v).round(4))
```

Output:

```
[[ 4.      1.    ]
 [ 5.     -0.4161]]
[[ 4.      1.    ]
 [ 5.     -0.4161]]
```

The formula and the numerical estimate agree. **Why it matters:** the chain rule for vector functions is simply *multiplying Jacobians*. If $\mathbf{z} = \mathbf{g}(\mathbf{h}(\mathbf{x}))$, then $J_{\mathbf{z}} = J_{\mathbf{g}}\, J_{\mathbf{h}}$, with shapes that line up like any matrix product. Backpropagation (Lesson 18) multiplies these Jacobians one layer at a time, starting from the loss, and it never builds the full matrices, only their products with vectors.

The Hessian: curvature in many directions

The first derivative is the slope; the second derivative is the **curvature**: how fast the slope changes. With many parameters, the second derivatives form the **Hessian** matrix $H_{ij} = \dfrac{\partial^2 L}{\partial w_i\, \partial w_j}$. It's symmetric, so (from Lesson 2's "Going deeper") it has real eigenvalues and perpendicular eigenvectors. Each eigenvector is a direction, and its eigenvalue is the curvature in that direction.

At a point where the gradient is zero, the eigenvalues tell you what kind of point it is:

- all positive: a **minimum** (curves up in every direction);
- all negative: a **maximum**;
- some positive, some negative: a **saddle point**, like a mountain pass.

```
def hessian_numeric(g, v, h=1e-4):
    d = len(v); H = np.zeros((d, d))
    for i in range(d):
        for j in range(d):
            ei, ej = np.eye(d)[i] * h, np.eye(d)[j] * h
            H[i, j] = (g(v + ei + ej) - g(v + ei - ej) - g(v - ei + ej) + g(v - ei -
ej)) / (4 * h * h)
    return H

bowl = lambda w: w[0]**2 + 10 * w[1]**2          # a long, narrow valley
saddle = lambda w: w[0]**2 - w[1]**2             # up in one direction, down in the
other
for name, g in [("bowl", bowl), ("saddle", saddle)]:
    H = hessian_numeric(g, np.array([0.0, 0.0]))
    print(name, "Hessian eigenvalues:", np.linalg.eigvalsh(H).round(3))
```

Output:

```
bowl Hessian eigenvalues: [ 2. 20.]
saddle Hessian eigenvalues: [-2.  2.]
```

$L = w_1^2 + 10w_2^2$ has Hessian eigenvalues 2 and 20: a minimum, but in a valley that is 10 times steeper one way than the other. $w_1^2 - w_2^2$ has eigenvalues -2 and 2: a saddle. In the huge loss surfaces of neural networks, most points with zero gradient are saddles rather than bad local minima, which is one reason gradient descent works better there than you might expect.

Convexity: when there's exactly one valley

A function is **convex** if the straight line between any two points on its graph never goes below the graph, like a bowl. Formally, for any $\mathbf{a}, \mathbf{b}$ and any t between 0 and 1:

$$L\big(t\mathbf{a} + (1-t)\mathbf{b}\big) \le t\,L(\mathbf{a}) + (1-t)\,L(\mathbf{b})$$

The practical test: a function is convex when its Hessian is positive semi-definite everywhere. The big payoff is that **every local minimum of a convex function is a global minimum**, so gradient descent can't get stuck in the wrong valley.

- **Mean squared error for linear regression is convex.** Its Hessian is $\frac{2}{n}X^\top X$, which Lesson 2's "Going deeper" showed is always PSD.
- **Log loss for logistic regression is convex** (Lesson 8), and so is the SVM's hinge loss (Lesson 15).
- **Neural network losses are not convex**: there are many minima and saddles, and the starting point matters (Lesson 19).

Taylor series and Newton's method

Near a point $\mathbf{w}$, any smooth function looks like a simple bowl. The second-order **Taylor approximation** says:

$$L(\mathbf{w} + \Delta) \approx L(\mathbf{w}) + \nabla L^\top \Delta + \tfrac{1}{2}\, \Delta^\top H\, \Delta$$

Gradient descent uses only the first two terms (the slope) and takes a cautious step $\Delta = -\eta \nabla L$. **Newton's method** uses the curvature too: it jumps straight to the bottom of the approximating bowl. Setting the gradient of the right-hand side to zero gives

$$\Delta = -H^{-1} \nabla L$$

Here are both on the long, narrow valley $L = w_1^2 + 10 w_2^2$, starting from $(10, 1)$:

```
grad = lambda w: np.array([2 * w[0], 20 * w[1]])
H = np.array([[2.0, 0.0], [0.0, 20.0]])

for lr in [0.01, 0.05, 0.09, 0.11]:
    w = np.array([10.0, 1.0])
    for step in range(1, 10001):
        w = w - lr * grad(w)
        if np.linalg.norm(w) < 1e-6 or np.linalg.norm(w) > 1e6: break
    status = "diverged" if np.linalg.norm(w) > 1e6 else f"{step} steps"
    print(f"gradient descent, learning rate {lr}: {status}")

w = np.array([10.0, 1.0])
w = w - np.linalg.solve(H, grad(w))              # one Newton step
print("Newton's method: 1 step, lands at", w)
```

Output:

```
gradient descent, learning rate 0.01: 798 steps
gradient descent, learning rate 0.05: 153 steps
gradient descent, learning rate 0.09: 82 steps
gradient descent, learning rate 0.11: diverged
Newton's method: 1 step, lands at [0. 0.]
```

Gradient descent can't use a learning rate of 0.1 or more, because the steep direction (curvature 20) would overshoot and blow up. That is the stability limit $\eta < 2/20$ from Lesson 7. But with η that small, progress along the gentle direction (curvature 2) is slow: 82 steps at best. Newton's method rescales each direction by its own curvature and, on a quadratic, lands exactly on the minimum in one step.

So why isn't Newton's method used everywhere? With d parameters the Hessian has d^2 entries and solving with it costs about d^3 operations. For a model with a million weights that's impossible.

Practical methods approximate the curvature cheaply instead: **L-BFGS** (scikit-learn's default solver for logistic regression) builds up an estimate from recent gradients, and **Adam** (Lesson 7) keeps a per-weight scaling.

Checking gradients like a professional

When you derive a gradient by hand, test it against a **central difference**, which is far more accurate than the one-sided version:

$$\frac{\partial L}{\partial w_i} \approx \frac{L(\mathbf{w} + h\mathbf{e}_i) - L(\mathbf{w} - h\mathbf{e}_i)}{2h}$$

Its error shrinks like h^2 instead of h. Compare the two gradients with the **relative error** $\frac{\|\mathbf{g}_1 - \mathbf{g}_2\|}{\|\mathbf{g}_1\| + \|\mathbf{g}_2\|}$, which doesn't depend on the gradient's scale. A common guide: below 10^{-7} is right, around 10^{-4} is suspicious, and above 10^{-2} is a bug.

```
# Gradient check for a tiny model: loss(w) = mean((tanh(Xw) - y)²)
rng = np.random.default_rng(0)
X = rng.normal(size=(20, 3)); y = rng.normal(size=20); w = rng.normal(size=3)
loss = lambda w: np.mean((np.tanh(X @ w) - y) ** 2)

def grad_formula(w):
    p = np.tanh(X @ w)
    return X.T @ (2 * (p - y) * (1 - p**2)) / len(y)   # chain rule: tanh' = 1 - tanh²

def rel_error(a, b):
    return np.linalg.norm(a - b) / (np.linalg.norm(a) + np.linalg.norm(b))

h = 1e-5
numeric = np.array([(loss(w + h * e) - loss(w - h * e)) / (2 * h) for e in np.eye(3)])
print("relative error, correct formula:", f"{rel_error(grad_formula(w), numeric):.1e}")
buggy = lambda w: X.T @ (2 * (np.tanh(X @ w) - y)) / len(y)   # forgot the tanh' factor
print("relative error, buggy formula:  ", f"{rel_error(buggy(w), numeric):.1e}")
```

Output:

```
relative error, correct formula: 3.4e-11
relative error, buggy formula:   6.7e-02
```

The correct gradient matches to about 3×10^{-11}. The buggy version, which forgot the $\tanh'$ factor from the chain rule, is off by 6.7×10^{-2}, which flags it clearly. Two tips: use double precision (64-bit floats), and don't pick h too small (10^{-5} to 10^{-6} is typical), or rounding errors take over.

Optimising with a constraint: Lagrange multipliers

Sometimes the parameters must obey a rule. PCA looks for the direction of greatest variance among vectors of length 1, and an SVM finds the widest margin while classifying every point correctly. The method of **Lagrange multipliers** handles an equality constraint $g(\mathbf{x}) = 0$: at the best point, the gradient of the objective must be parallel to the gradient of the constraint,

$$\nabla f = \lambda \nabla g$$

Otherwise you could slide along the constraint and still improve f. The number λ is the **Lagrange multiplier**.

Example. Find the largest rectangle with perimeter 20: maximise $f = xy$ subject to $g = 2x + 2y - 20 = 0$. Then $\nabla f = (y, x)$ and $\nabla g = (2, 2)$, so $y = 2\lambda$ and $x = 2\lambda$, giving $x = y$. The constraint then says $4x = 20$, so $x = y = 5$ and the area is 25: the square wins.

```
from scipy.optimize import minimize

# Largest rectangle with perimeter 20: maximise x·y subject to 2x + 2y = 20
res = minimize(lambda v: -v[0] * v[1], x0=[1.0, 1.0],
               constraints=[{"type": "eq", "fun": lambda v: 2 * v[0] + 2 * v[1] - 20}])
print("numerical answer (x, y):", res.x.round(4), " area:", round(-res.fun, 4))
```

Output:

```
numerical answer (x, y): [5. 5.]  area: 25.0
```

A numerical optimiser with the constraint built in agrees. Using the same idea on PCA, maximising $\mathbf{v}^\top \Sigma \mathbf{v}$ with $\mathbf{v}^\top \mathbf{v} = 1$, gives $\Sigma \mathbf{v} = \lambda \mathbf{v}$: the best direction is an eigenvector (Lesson 17). For inequality constraints such as the SVM's, the same idea extends to the **KKT conditions** (Lesson 15).

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Why don't we train neural networks with Newton's method, if it converges so much faster?**

The trap: "Because it doesn't work on non-convex functions."

Cost first: the Hessian has d^2 entries and solving with it is about d^3 work, which is impossible for millions of weights. It also needs the full dataset's curvature, whereas we train on noisy mini-batches. And on non-convex surfaces the Hessian isn't positive definite, so a raw Newton step can head *towards* a saddle or maximum. Quasi-Newton methods (L-BFGS) and adaptive methods (Adam) capture some curvature cheaply instead.

▼ **What makes a loss function convex, and which common ML losses are convex?**

The trap: Saying "it has one minimum" without the test.

A twice-differentiable function is convex if and only if its Hessian is positive semi-definite everywhere. Equivalently, the chord between any two points lies on or above the graph. Convex: MSE for linear models, log loss for logistic regression, hinge loss for SVMs, and these plus L1 or L2 penalties. Not convex: neural network losses, k-means' objective, mixture-model likelihoods.

▼ **What's the difference between the Jacobian and the Hessian?**

The trap: Mixing up which one holds first derivatives.

The Jacobian holds the *first* derivatives of a vector-valued function (outputs × inputs). The Hessian holds the *second* derivatives of a scalar function (inputs × inputs), and it is symmetric. In fact, the Hessian is the Jacobian of the gradient. Backprop multiplies Jacobians; second-order optimisers use the Hessian.

▼ **In an SVM or in PCA, where do Lagrange multipliers come in?**

The trap: Saying they're only a trick for textbook problems.

PCA maximises the variance $\mathbf{v}^\top \Sigma \mathbf{v}$ subject to $\|\mathbf{v}\| = 1$. The Lagrange condition gives $\Sigma \mathbf{v} = \lambda \mathbf{v}$, so the components are eigenvectors and λ is the variance. The SVM maximises the margin subject to inequality constraints. Its Lagrange multipliers (via the KKT conditions) are zero for every point except the support vectors, and its dual form is what makes the kernel trick possible.

Check your understanding: going deeper

1. A function takes 4 numbers and returns 3. What shape is its Jacobian?

a. 4 × 3

b. 3 × 4

c. 3 × 3

2. At a point with zero gradient, the Hessian's eigenvalues are 5 and −1. What kind of point is it?

a. A minimum

b. A maximum

c. A saddle point

3. Why is a convex loss good news for gradient descent?

a. Every local minimum is the global minimum

b. It always converges in one step

c. It needs no learning rate

4. On a quadratic loss, how many steps does Newton's method need to reach the minimum?

a. One

b. About as many as gradient descent

c. It never reaches it exactly

5. A gradient check gives a relative error of 0.05. What should you conclude?

a. Everything is fine

b. There is probably a bug in the gradient

c. h is too big

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Differentiate $f(x) = 3x^2 - 5x + 2$ and evaluate the slope at $x = 2$.

WORKING

EXERCISE 2 By hand

Use the chain rule on $f(x) = (2x + 1)^3$. What is $f'(0)$?

WORKING

EXERCISE 3 By hand

For $f(w, b) = w^2 b + 3b$, find both partial derivatives and the gradient at $(w, b) = (2, 1)$.

WORKING

EXERCISE 4 By hand

Run 3 steps of gradient descent on $L(w) = (w - 4)^2$ from $w = 0$ with $\eta = 0.25$.

WORKING

EXERCISE 5 Intermediate

Compute the Jacobian of $\mathbf{f}(x, y) = (xy,\ x + y^2)$ at $(2, 3)$. Give the entries row by row.

WORKING

EXERCISE 6 Intermediate

For $L(w_1, w_2) = 3w_1^2 + 2w_1 w_2 + w_2^2$, find the Hessian and its eigenvalues. Is L convex?

WORKING

EXERCISE 7 Intermediate

Use one Newton step to minimise $L(w) = w^4$ in one dimension, starting from $w = 2$. Where do you land?

WORKING

EXERCISE 8 Intermediate

Maximise $f = x + y$ subject to $x^2 + y^2 = 8$ using a Lagrange multiplier. What are x, y and the maximum?

WORKING

NOTES

PART 1 · FOUNDATIONS · ~50 MIN

Lesson 4: Probability & Statistics

Probability rules, Bayes' theorem, mean and variance, z-scores and the normal distribution.

BY THE END OF THIS LESSON YOU CAN

- Apply the basic probability rules and conditional probability
- Use Bayes' theorem on a real-world problem
- Compute mean, variance, standard deviation and z-scores by hand
- Describe the normal distribution and the 68–95–99.7 rule

↺ BEFORE YOU START

This lesson builds on:

- Lesson 1: What Is Machine Learning? — A model maps features x to a prediction $\hat{y}$. Training adjusts its parameters to make a loss (such as MSE) small, and the real goal is doing well on new data.

Where this fits: Real data is noisy, so models reason with probabilities. This lesson underpins logistic regression (Lesson 8), Naive Bayes (Lesson 12) and how we summarise and standardise data.

1. Why probability and statistics?

Real data is messy. Two houses of the same size sell for different prices; the same symptoms can come from different illnesses. ML models live with this uncertainty, so they speak the language of probability: a spam filter doesn't say "spam", it says "97% likely spam". Statistics gives us tools to summarise data — averages, spread — that we'll use in almost every lesson.

2. Probability basics

A **probability** is a number between 0 and 1 that measures how likely an event is: 0 = impossible, 1 = certain, 0.5 = as likely as not. We write $P(A)$ for "the probability of event A".

When all outcomes are equally likely:

$$P(A) = \frac{\text{number of outcomes where } A \text{ happens}}{\text{total number of outcomes}}$$

✎ WORKED EXAMPLE — A FAIR DIE

$P(\text{roll a } 6) = \frac{1}{6} \approx 0.167$. $P(\text{roll an even number}) = \frac{3}{6} = 0.5$ (outcomes 2, 4, 6).

? MAKE SENSE OF IT

The situation: A board game needs a 6 to start. How often will that happen?

What the result tells you: A probability of 1/6 means that over many rolls, about one roll in six is a 6. Roll 600 times and you'd expect around 100 sixes — rarely exactly 100.

What if…?

- **Two tries:** the chance of at least one 6 is $1 - (5/6)^2 \approx$ **0.31**.
- **Six tries:** about **0.67** — not 1. Even with six rolls, one game in three still waits.

Why it matters in ML: Classifiers output probabilities. A prediction of "70% spam" means that among many emails given 70%, about 7 in 10 should really be spam — a property called calibration.

Three rules do most of the work:

- **Complement**: $P(\text{not } A) = 1 - P(A)$. Probability of *not* rolling a 6 is $5/6$.
- **Addition** (for events that can't both happen): $P(A \text{ or } B) = P(A) + P(B)$. Rolling a 1 or a 2: $1/6 + 1/6 = 1/3$.
- **Multiplication** (for *independent* events — one doesn't affect the other): $P(A \text{ and } B) = P(A)\,P(B)$. Two coin flips both heads: $0.5 \times 0.5 = 0.25$.

3. Conditional probability

Often we know something already, and that changes the odds. $P(A \mid B)$ — "probability of A **given** B" — is the probability of A when we only consider situations where B happened:

$$P(A \mid B) = \frac{P(A \text{ and } B)}{P(B)}$$

✎ WORKED EXAMPLE — EMAILS

Out of 100 emails, 30 are spam. 20 emails contain the word "free", and 15 of those are spam.

$P(\text{spam}) = 30/100 = 0.30.$
$P(\text{spam} \mid \text{"free"}) = 15/20 = 0.75.$

Knowing the email contains "free" raised the spam probability from 30% to 75%. This is the core idea behind Naive Bayes spam filters (Lesson 12): features change our beliefs about the label.

? MAKE SENSE OF IT

The situation: You run a mail service. Overall, 30% of emails are spam. Should the word "free" make you more suspicious?

What the result tells you: Among emails with "free", 75% are spam, compared with 30% overall. The word is strong evidence, because it raises the probability by a factor of 2.5.

What if...?

- **If only 6 of the 20 "free" emails were spam,** $P(\text{spam} \mid \text{free}) = 0.30$ — the same as the overall rate. Then the word tells you nothing.
- **If 19 of 20 were spam,** it would be **0.95** — almost decisive.

Why it matters in ML: A feature is useful exactly when it changes the probability of the label. Comparing $P(\text{label} \mid \text{feature})$ with $P(\text{label})$ is a quick way to find informative features.

! CAREFUL: $P(A \mid B) \neq P(B \mid A)$

$P(\text{spam} \mid \text{"free"}) = 15/20 = 0.75$, but $P(\text{"free"} \mid \text{spam}) = 15/30 = 0.5$. Mixing these up is one of the most common reasoning errors — and the next section shows how to convert between them.

4. Bayes' theorem

PICTURE IT An airport scanner beeps. Is the bag dangerous? It depends on two things: how good the scanner is, and how rare dangerous bags are in the first place. Bayes' theorem combines both.

Bayes' theorem flips a conditional probability around:

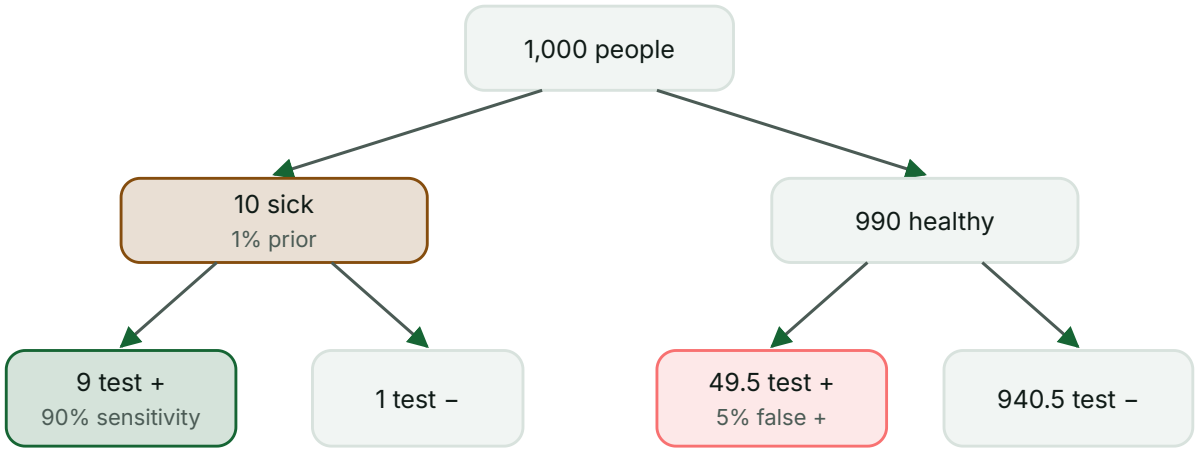


Figure 4. Of about 58.5 positive tests, only 9 come from sick people: P(sick | positive) ≈ 9 / 58.5 ≈ 15%.

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}$$

In ML language, with A = "a hypothesis" (e.g. has the disease) and B = "the evidence" (e.g. tested positive):

- $P(A)$ — the **prior**: what we believed before seeing evidence.
- $P(B \mid A)$ — the **likelihood**: how probable the evidence is if the hypothesis is true.
- $P(A \mid B)$ — the **posterior**: our updated belief after seeing the evidence.

✎ WORKED EXAMPLE — THE SURPRISING MEDICAL TEST

A disease affects **1%** of people. A test catches it **90%** of the time when present (sensitivity), but also gives a positive result to **5%** of healthy people (false positives). You test positive. What's the chance you have the disease?

Most people guess around 90%. Let's compute.

$P(D) = 0.01$, $P(+ \mid D) = 0.90$, $P(+ \mid \text{healthy}) = 0.05$.

First, the overall chance of a positive test (sick-and-positive plus healthy-and-positive):

$$P(+) = 0.90 \times 0.01 + 0.05 \times 0.99 = 0.009 + 0.0495 = 0.0585$$

Then Bayes:

$$P(D \mid +) = \frac{0.90 \times 0.01}{0.0585} \approx 0.154$$

Only about **15%**! Because the disease is rare, the false positives from the 99% of healthy people outnumber the true positives.

The same thing with counts (often easier): in 1,000 people, 10 are sick and 9 of them test positive. Of the 990 healthy people, about 49.5 test positive. So of roughly 58.5 positives, only 9 are sick: $9/58.5 \approx 15\%$.

? MAKE SENSE OF IT

The situation: You're a doctor explaining a positive screening result to a worried patient.

What the result tells you: 15% means: among people who test positive, about 15 in 100 actually have the disease. The rest are false alarms, because healthy people vastly outnumber sick ones.

What if…?

- **A more common disease (10%):** the chance becomes **67%**. The same test means something very different in a different population.
- **A better test** (false positives 1% instead of 5%): about **48%**.
- **A second, independent positive test:** start from 15% and apply Bayes again — about **77%**.

Why it matters in ML: The base rate controls what a positive prediction means. Fraud, disease and fault detectors work with rare events, so a model with high accuracy can still produce mostly false alarms — which is why precision is reported separately (Lesson 9).

STEP-BY-STEP DERIVATION **Derive Bayes' theorem in four lines**

1 $$P(A \mid B) = \frac{P(A \text{ and } B)}{P(B)}$$

The definition of conditional probability.

2 $$P(B \mid A) = \frac{P(A \text{ and } B)}{P(A)}$$

The same definition with the roles of A and B swapped.

3 $$P(A \text{ and } B) = P(B \mid A)\, P(A)$$

Multiply both sides of the previous line by $P(A)$.

4 $$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Substitute into the first line. That's Bayes' theorem.

5 $$P(B) = P(B \mid A)P(A) + P(B \mid \text{not } A)\, P(\text{not } A)$$

To compute the bottom, add up the two ways B can happen — exactly what we did in the medical-test example.

READ THE FORMULA **Bayes' theorem**

$$P(A \mid B) = \frac{P(B \mid A)\ P(A)}{P(B)}$$

- **P(A | B)** — posterior — what we believe after seeing evidence B
- **P(B | A)** — likelihood — how expected the evidence is if A is true
- **P(A)** — prior — what we believed before
- **P(B)** — evidence — how common B is overall (makes it sum to 1)

Hover over (or tap) a part to highlight it in the formula.

5. Try it: Bayes with 1,000 people

Each square is a person. Change the disease rate and the test's accuracy, and see how many of the people who test positive are actually sick.

Interactive demo — open *lesson-04.html* on the website to try it.

6. Random variables and expected value

A **random variable** is a quantity whose value depends on chance — the result of a die roll, tomorrow's temperature, a house's sale price. Its **expected value** (mean) is the average you'd get over many repetitions: each possible value weighted by its probability.

$$\mathbb{E}[X] = \sum_{x} x \cdot P(X = x)$$

✎ WORKED EXAMPLE — A DIE

$\mathbb{E}[X] = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + \cdots + 6 \cdot \frac{1}{6} = \frac{21}{6} = 3.5$. You can never roll 3.5, but it's the long-run average.

? MAKE SENSE OF IT

The situation: A game pays you in rupees the number you roll. What's it worth on average?

What the result tells you: 3.5 is the long-run average payout per roll. No single roll pays 3.5, but over 1,000 rolls you'd collect about 3,500.

What if…?

- **A loaded die** that shows 6 half the time (each other face 1/10): the expected value rises to **4.5**.
- **Pay to play:** if each roll costs 4, a fair die loses you 0.5 per roll on average.

Why it matters in ML: A model's true loss is an expected value over all possible data. We can't compute it exactly, so we estimate it with the average loss on a sample — the training or test set.

ADVANCED · OPTIONAL **A useful identity: Var(X) = E[X²] − (E[X])²** ▾ hide

Write $\mu = E[X]$ and expand the definition, using the fact that expectation is linear:

$$\mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - 2\mu E[X] + \mu^2 = E[X^2] - \mu^2.$$

Check with a fair die: $E[X] = 3.5$ and $E[X^2] = (1+4+9+16+25+36)/6 = 91/6$, so $\mathrm{Var}(X) = 91/6 - 12.25 = 35/12 \approx 2.917$. The same identity is how libraries compute variance in one pass over the data.

7. Describing data: mean, variance, standard deviation

PICTURE IT Two coffee shops both average 5 minutes per order. At one, every order takes 4–6 minutes; at the other, some take 1 minute and some 15. The average is identical — the experience isn't. To describe data you need a centre *and* a spread, and you need to know how one strange value can move them.

Given n numbers $x_1, \ldots, x_n$:

The **mean** (average) is the centre:

$$\bar{x} = \mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The **variance** measures spread — the average squared distance from the mean:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - \mu)^2$$

The **standard deviation** $\sigma = \sqrt{\sigma^2}$ is spread in the original units.

✎ WORKED EXAMPLE

Exam scores: 4, 6, 8, 10, 12.

Mean: $\mu = (4+6+8+10+12)/5 = 40/5 = 8$.

Distances from the mean: $-4, -2, 0, 2, 4$. Squared: $16, 4, 0, 4, 16$.

Variance: $\sigma^2 = 40/5 = 8$. Standard deviation: $\sigma = \sqrt{8} \approx 2.83$.

So a typical score is about 2.8 points away from the average of 8.

? MAKE SENSE OF IT

The situation: A teacher has five students' test scores: 4, 6, 8, 10 and 12. She wants one number for "how the class did" and one for "how spread out" the scores are.

What the result tells you: The mean, 8, is the balance point: the gaps below it (−4, −2) exactly cancel the gaps above it (+2, +4). The standard deviation, 2.83, says a typical student is about 3 points from that average.

What if…?

- **One outlier.** A data-entry slip turns 12 into 52. The mean jumps from 8 to **16** — now higher than four of the five students — while the **median stays 8**. The standard deviation leaps from 2.83 to **18.1**.
- **Add a sixth student who scores exactly 8:** the mean stays 8, but the standard deviation *falls* to 2.58, because a score at the centre makes the class look less spread out.
- **Give everyone 10 bonus points:** the mean becomes 18; the standard deviation stays **2.83**. Shifting doesn't change spread.
- **Double every score:** the mean becomes 16 and the standard deviation doubles to **5.66**.
- **Same mean, different class:** scores 0, 0, 8, 16, 16 also average 8, but their standard deviation is 7.16. And 8, 8, 8, 8, 8 has a standard deviation of 0. The mean alone can't tell these classes apart.

Why it matters in ML: The mean is pulled by every value, so a single extreme value can move it a lot; the median ignores how extreme the values are. That's why you check for outliers before averaging, fill missing values with the median (Lesson 20), and why MSE — which is an average — is sensitive to outliers. Standardising features uses exactly this mean and standard deviation.

POPULATION VS SAMPLE

Dividing by n gives the population variance. When estimating from a sample, statisticians often divide by $n-1$ instead (the "sample variance"), which corrects a small bias. NumPy's `np.var` divides by n by default; pandas' `.var()` divides by $n-1$. For large datasets the difference is negligible.

Also useful: the **median** (middle value when sorted) is robust to outliers. Incomes of 30k, 35k, 40k, 45k and 1M have a mean of 230k but a median of 40k — the median describes the "typical" person far better.

8. Standardisation (z-scores)

A **z-score** says how many standard deviations a value is from the mean:

$$z = \frac{x - \mu}{\sigma}$$

A score of 12 above: $z = (12 - 8)/2.83 \approx 1.41$. Converting every feature to z-scores — mean 0, standard deviation 1 — is called **standardisation**. It's one of the most common preprocessing steps in ML (scikit-learn's `StandardScaler`), because many algorithms work badly when features have wildly different scales (Lessons 11 and 20).

9. Probability distributions

A **distribution** describes all the values a random variable can take and how likely each is.

Bernoulli: a single yes/no

One trial with success probability p: $P(1) = p$, $P(0) = 1 - p$. A coin flip is Bernoulli with $p = 0.5$. A classifier's "is this spam?" output is a Bernoulli probability.

Normal (Gaussian): the bell curve

Many natural quantities — heights, measurement errors — follow the bell-shaped **normal distribution**, described by its mean μ (centre) and standard deviation σ (width). Its density is:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}\, e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

You don't need to memorise the formula — just its shape and this rule of thumb:

- about **68%** of values lie within 1 σ of the mean,
- about **95%** within 2 σ,
- about **99.7%** within 3 σ.

> ✎ WORKED EXAMPLE
>
> Adult heights in a population: $\mu = 170$ cm, $\sigma = 10$ cm. About 95% of people are between $170 - 20 = 150$ and $170 + 20 = 190$ cm. Someone 200 cm tall is 3 σ above the mean — rarer than 1 in 300 (only 0.3% fall outside $\pm 3\sigma$, half of them above).

? MAKE SENSE OF IT

The situation: A clothing company is planning garment sizes for adults whose average height is 170 cm and standard deviation 10 cm.

What the result tells you: Within one standard deviation (160–180 cm) are about 68% of people; within two (150–190 cm) about 95%. 200 cm is three standard deviations up: about 1 person in 740 is taller.

What if…?

- **Someone 185 cm tall** has $z = 1.5$: about 6.7% of people are taller.
- **A less varied population** ($\sigma = 5$ cm): 95% now fall between 160 and 180 cm, and 200 cm is six standard deviations up — about 1 in a billion.

Why it matters in ML: z-scores put different features on a common scale and flag unusual values: a value with $|z| > 3$ is worth checking as a possible error or anomaly. Many models also assume errors are roughly normal.

10. Try it: sampling from a normal distribution

Each click draws random samples from a normal distribution and adds them to the histogram. With few samples the shape is ragged; with many, it settles into the bell curve. Adjust μ and σ to see the centre move and the width change.

Interactive demo — open *lesson-04.html* on the website to try it.

11. How this connects to machine learning

- **Classifiers output probabilities**: logistic regression (Lesson 8) outputs $P(y = 1 \mid \mathbf{x})$.
- **Naive Bayes** (Lesson 12) is Bayes' theorem applied to features.
- **Noise**: we model data as "true pattern + random noise", often assumed normal. That's why the squared error loss is so natural.
- **Evaluation** (Lesson 9) reports means and standard deviations of scores across experiments.
- **Preprocessing** uses means and standard deviations to standardise features.

12. In Python

```
import numpy as np

scores = np.array([4, 6, 8, 10, 12])
print(scores.mean())           # 8.0
print(scores.var())            # 8.0   (divides by n)
print(scores.std())            # 2.828...
print(np.median([30, 35, 40, 45, 1000]))   # 40.0

z = (scores - scores.mean()) / scores.std()
print(z.round(2))              # [-1.41 -0.71  0.    0.71  1.41]

# Bayes: the medical test
p_d, sens, fpr = 0.01, 0.90, 0.05
p_pos = sens * p_d + fpr * (1 - p_d)
print(round(sens * p_d / p_pos, 3))        # 0.154

# Sample 100,000 heights and check the 68-95-99.7 rule
rng = np.random.default_rng(0)
h = rng.normal(170, 10, size=100_000)
print(np.mean(np.abs(h - 170) < 20))       # ≈ 0.95
```

13. Running example: Chennai Homes

The average price is 72.1 lakh with a standard deviation of 17.3, so house 12 (96 lakh) has a z-score of 1.38. Half of all houses sold fast, but 75% of Central ones did: $P(\text{fast} \mid \text{Central}) = 0.75$. Turned around, only half of the fast sellers were Central — remember $P(A \mid B) \neq P(B \mid A)$.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
p = df["price_lakh"]
print("mean", round(p.mean(), 2), "std", round(p.std(ddof=0), 2))
print("z-score of house 12:", round((96 - p.mean()) / p.std(ddof=0), 2))
print("P(sold fast) =", df["sold_fast"].mean())
central = df[df["area"] == "Central"]
print("P(sold fast | Central) =", central["sold_fast"].mean())
print("P(Central | sold fast) =", round((df[df["sold_fast"] == 1]["area"] ==
"Central").mean(), 3))
```

Output:

```
mean 72.08 std 17.29
z-score of house 12: 1.38
P(sold fast) = 0.5
P(sold fast | Central) = 0.75
P(Central | sold fast) = 0.5
```

⚙ WHERE THIS IS USED

- Medical test interpretation
- A/B testing of websites
- Risk scores and probability forecasts
- Anomaly detection (values many standard deviations from normal)

Choose it when: Whenever you need to reason about uncertainty or summarise data.

✕ MISCONCEPTIONS TO UNLEARN

MYTH A 95%-accurate test means a positive result is 95% likely to be right.

REALITY It depends on how common the condition is. With rare conditions, most positives can be false — as the medical example showed (15%).

MYTH Probabilities of 0 or 1 are fine for a model to output.

REALITY Being certain is dangerous: one confident mistake gives infinite log loss. Real models keep a little doubt.

RESPONSIBLE AI · BASE RATES AND SMALL GROUPS

The Bayes demo shows that even an accurate test produces mostly false alarms when the condition is rare. The same maths applies to systems that flag rare behaviour such as fraud or crime: most people flagged may be innocent. Estimates for small subgroups are also far noisier than for the majority, so a model can look accurate overall while being unreliable for minorities.

14. Questions beginners often ask

Q. Why does ML care about probability?

Because data is noisy and predictions are uncertain. Outputs like "92% likely spam" are more useful than a bare yes/no, and many models are built on probability rules.

Q. Mean or median?

Use the median when data is skewed or has outliers (incomes, house prices). Use the mean when data is roughly symmetric.

Q. Is the normal distribution always the right assumption?

No. It fits many measurements, but counts, waiting times, incomes and yes/no outcomes follow other distributions. Always look at your data.

! COMMON MISTAKES TO AVOID

- Confusing $P(A \mid B)$ with $P(B \mid A)$ — the base-rate trap in the medical test example.
- Multiplying probabilities of events that are *not* independent.
- Using the mean to describe skewed data such as incomes, where the median is far more representative.

! INTERVIEW CORNER

Don't confuse

- **Probability vs likelihood** — Probability predicts data; likelihood judges parameters.
- **Independent vs mutually exclusive** — Exclusive events tell you a lot about each other.
- **Correlation vs causation** — Look for the hidden third variable.
- **Uncorrelated vs independent** — Correlation only sees straight lines.
- **Standard deviation vs standard error** — SD is about the people; SE is about the estimate.

Hidden questions — answer in your head first, then open.

<details>
<summary>▼ What's the difference between probability and likelihood?</summary>

The trap: Using the words interchangeably.

Same formula, different viewpoint. Probability fixes the parameters and asks how probable different data are (it sums to 1 over data). Likelihood fixes the observed data and asks how well different parameters explain it (it needn't sum to 1). Training by maximum likelihood picks the parameters under which the data are most probable.
</details>

<details>
<summary>▼ If two variables have zero correlation, are they independent?</summary>

The trap: Yes, because nothing's going on.

No. Correlation only measures linear relationships. For $x = -2, -1, 0, 1, 2$ and $y = x^2$, the correlation is exactly 0, yet y is completely determined by x.
</details>

<details>
<summary>▼ Are mutually exclusive events independent?</summary>

The trap: They seem to have nothing to do with each other.

The opposite: if two events can't both happen (and each has non-zero probability), learning that one happened tells you the other didn't. That's strong dependence.
</details>

<details>
<summary>▼ Why divide by $n - 1$ instead of n for a sample variance?</summary>

The trap: "It's just the formula."

The sample mean is fitted to the same data, so deviations from it are slightly too small on average. Dividing by $n - 1$ (Bessel's correction) makes the estimate unbiased. For large n the difference is tiny; NumPy's `np.var` divides by n unless you set `ddof=1`.
</details>

15. Key takeaways

- Probabilities run from 0 to 1. Key rules: complement, addition, multiplication for independent events.
- **Conditional probability** $P(A \mid B)$ updates odds given information; $P(A \mid B) \neq P(B \mid A)$.
- **Bayes' theorem**: posterior ∝ likelihood × prior. Rare events stay fairly unlikely even after positive evidence.
- **Mean** = centre, **variance**/**standard deviation** = spread; **z-scores** standardise features.
- The **normal distribution**: 68% / 95% / 99.7% within 1 / 2 / 3 standard deviations.

16. Check your understanding

1. You flip a fair coin 3 times. What is the probability of 3 heads?

a. 1/3

b. 1/8

c. 1/2

2. Data: 2, 4, 6. What is the variance (dividing by n)?

a. 8/3 ≈ 2.67

b. 4

c. 2

3. In Bayes' theorem, $P(A)$ is called the...

a. posterior

b. likelihood

c. prior

4. A value has z-score −2. What does that mean?

a. It is 2 units below the mean

b. It is 2 standard deviations below the mean

c. It is an error in the data

5. A normal distribution has $\mu = 50$, $\sigma = 5$. Roughly what fraction of values fall between 45 and 55?

a. 68%

b. 95%

c. 99.7%

17. Going deeper: statistical inference for ML

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above describes data: probabilities, averages, spread and the normal curve. This section is about *learning from* data and judging how sure you can be: how to fit a distribution's parameters (maximum likelihood), how to combine data with prior knowledge (Bayesian estimation and MAP), why averages behave so predictably (the central limit theorem), how to put an error bar on almost anything (the bootstrap), and how to run an A/B test without fooling yourself. These ideas sit underneath many ML methods and are a favourite of data-science interviews.

You'll need: everything in this lesson, especially Bayes' theorem, the normal distribution and standard deviation.

A few more distributions worth knowing

Each distribution is a model of a particular kind of randomness. Recognising which one fits your data tells you which loss function and which model to use.

Distribution	Describes	Example	Where it appears in ML
Bernoulli(p)	One yes/no outcome	Does this visitor click?	Logistic regression's output; log loss (Lesson 8)
Binomial(n, p)	Number of yeses in n tries	Clicks out of 10 visitors	A/B tests, accuracy on a test set
Poisson(λ)	Counts of events in a fixed time	Support calls per hour	Count regression, demand forecasting
Exponential(λ)	Waiting time until the next event	Minutes until the next call	Survival and churn models
Normal(μ, σ^2)	Sums of many small effects	Measurement error	Squared-error loss (Lesson 6)

```
import numpy as np
from scipy import stats

print("Binomial: P(8 or more of 10 visitors click | p 0.5):", round(stats.binom.sf(7,
10, 0.5), 4))
print("Poisson:  P(no support calls in an hour | 3 per hour):",
round(stats.poisson.pmf(0, 3), 4))
print("Exponential: P(wait over 30 min | mean wait 20 min):", round(stats.expon.sf(30,
scale=20), 4))
```

Output:

```
Binomial: P(8 or more of 10 visitors click | p 0.5): 0.0547
Poisson:  P(no support calls in an hour | 3 per hour): 0.0498
Exponential: P(wait over 30 min | mean wait 20 min): 0.2231
```

For example, if visitors click with probability 0.5, seeing 8 or more clicks out of 10 has probability 0.0547. If calls arrive at 3 per hour on average, a whole hour with no calls has probability $e^{-3} \approx 0.0498$. And if the average wait is 20 minutes, a wait of over 30 minutes has probability $e^{-30/20} \approx 0.2231$.

Maximum likelihood: the parameters that make the data least surprising

Suppose a coin lands heads 7 times in 10 flips. For any guess p of the heads probability, the **likelihood** is the probability of exactly this data under that guess. We work with its log, because a

product turns into a sum:

$$\log L(p) = 7 \log p + 3 \log(1 - p)$$

Maximum likelihood estimation (MLE) picks the p that makes this as large as possible. Set the derivative to zero:

$$\frac{7}{p} - \frac{3}{1 - p} = 0 \;\Rightarrow\; 7(1 - p) = 3p \;\Rightarrow\; p = 0.7$$

The answer is the obvious one, the observed fraction. But the same recipe works for every model: least squares is the MLE for normal noise (Lesson 6's "Going deeper"), and log loss is the MLE for yes/no outcomes (Lesson 8).

```
# Maximum likelihood for a coin: 7 heads in 10 flips
p_grid = np.linspace(0.01, 0.99, 99)
loglik = 7 * np.log(p_grid) + 3 * np.log(1 - p_grid)
print("p with the highest likelihood:", round(p_grid[np.argmax(loglik)], 2))

# MLE for a normal distribution divides by n, which underestimates the variance
rng = np.random.default_rng(0)
samples = rng.normal(0, 2, size=(100000, 5))           # 100,000 small samples, true
variance 4
print("average variance, divide by n:    ", round(samples.var(axis=1, ddof=0).mean(),
3))
print("average variance, divide by n - 1:", round(samples.var(axis=1, ddof=1).mean(),
3))
```

Output:

```
p with the highest likelihood: 0.7
average variance, divide by n:     3.21
average variance, divide by n - 1: 4.012
```

The MLE isn't always unbiased. For a normal distribution, the MLE of the variance divides by n. Across 100,000 samples of size 5 from a distribution with true variance 4, it averages 3.21, close to the theoretical $\frac{n-1}{n}\sigma^2 = \frac{4}{5} \times 4 = 3.2$. Dividing by $n - 1$ instead fixes the bias and averages 4.012. That's why sample variance uses $n - 1$, as this lesson mentioned.

Bayesian estimation, priors and MAP

MLE uses only the data. With just 10 flips, should we really believe $p = 0.7$? **Bayesian estimation** uses Bayes' theorem to combine a **prior** belief with the likelihood, giving a **posterior** distribution over p:

$$\text{posterior}(p) \propto \text{likelihood}(\text{data} \mid p) \times \text{prior}(p)$$

For a probability, a convenient prior is the **Beta**(a, b) distribution, which you can read as "a imaginary heads and b imaginary tails seen before". Beta$(2, 2)$ is a mild belief that the coin is probably near fair. After seeing 7 heads and 3 tails, the posterior is simply Beta$(2 + 7,\ 2 + 3)$ = Beta$(9, 5)$. When the prior and posterior come from the same family like this, the prior is called **conjugate**.

```
# Bayesian updating: prior Beta(2, 2), then observe 7 heads and 3 tails
a, b = 2 + 7, 2 + 3
post = stats.beta(a, b)
print(f"posterior: Beta({a}, {b})")
print("posterior mean:", round(post.mean(), 3), " MAP (most likely value):", round((a -
1) / (a + b - 2), 3))
print("95% credible interval:", np.round(post.ppf([0.025, 0.975]), 3))
```

Output:

```
posterior: Beta(9, 5)
posterior mean: 0.643  MAP (most likely value): 0.667
95% credible interval: [0.386 0.861]
```

Three summaries of the posterior: its mean is $9/14 \approx 0.643$, pulled from 0.7 towards 0.5 by the prior. Its peak, the **MAP estimate** (maximum a posteriori), is $\frac{9-1}{9+5-2} \approx 0.667$. And a **95% credible interval** runs from 0.386 to 0.861. That wide interval is honest: 10 flips really don't tell you much. With 1,000 flips the data would overwhelm the prior.

> **MAP IS REGULARIZATION IN DISGUISE**
>
> Taking logs, MAP maximises $\log(\text{likelihood}) + \log(\text{prior})$. For linear regression with a normal prior on the weights, $\log(\text{prior}) = -\lambda\|\mathbf{w}\|^2 + \text{const}$. That's exactly **ridge regression**. A Laplace (double-exponential) prior gives $-\lambda\|\mathbf{w}\|_1$, which is **lasso**. So the regularization in Lesson 10 is the same thing as believing, before seeing data, that weights are probably small.

The central limit theorem and the standard error

Take any variable with mean μ and standard deviation σ (it doesn't need to be normal), and average n independent values of it. The **central limit theorem** (CLT) says that for large n the average is approximately normal, with mean μ and standard deviation

$$\text{SE} = \frac{\sigma}{\sqrt{n}}$$

That's the **standard error** of the mean. Below, we average values from an exponential distribution, which is strongly skewed (a long tail to the right) with $\sigma = 1$:

```
# Central limit theorem: averages of a skewed variable become bell-shaped
rng = np.random.default_rng(1)
for n in [1, 5, 30, 200]:
    means = rng.exponential(scale=1.0, size=(50000, n)).mean(axis=1)
    print(f"n {n:3d}: skewness {stats.skew(means):5.2f}   spread of the means
{means.std():.3f}   1/sqrt(n) {1 / np.sqrt(n):.3f}")
```

Output:

```
n   1: skewness  1.98   spread of the means 0.997   1/sqrt(n) 1.000
n   5: skewness  0.90   spread of the means 0.445   1/sqrt(n) 0.447
n  30: skewness  0.37   spread of the means 0.182   1/sqrt(n) 0.183
n 200: skewness  0.15   spread of the means 0.071   1/sqrt(n) 0.071
```

Single values have skewness 1.98, far from the bell curve's 0. Averages of 30 have skewness 0.37, and averages of 200 have just 0.15. The spread of the averages matches $1/\sqrt{n}$ almost exactly. Notice the square root: to halve the error bar you need *four times* as much data. This is why test-set accuracy, cross-validation scores and A/B test results have predictable error bars.

Confidence intervals and the bootstrap

A 95% **confidence interval** comes from a method that, used on many datasets, captures the true value 95% of the time. For a mean, the CLT gives one directly: $\bar{x} \pm 1.96\,\mathrm{SE}$. But what about a median, a correlation, or a model's F1 score, where there's no simple formula?

The **bootstrap** answers with brute force. Treat your sample as a stand-in for the population: draw a new sample of the same size from it *with replacement*, compute the statistic, and repeat thousands of times. The spread of those results shows how much the statistic would vary between samples. The middle 95% of them is a confidence interval.

```
# Bootstrap: a confidence interval for the median house price (simulated, lakh)
rng = np.random.default_rng(2)
prices = rng.lognormal(mean=4.2, sigma=0.5, size=60)
boot = [np.median(rng.choice(prices, size=len(prices), replace=True)) for _ in
range(10000)]
print("sample median:", round(np.median(prices), 1))
print("95% bootstrap interval:", np.round(np.percentile(boot, [2.5, 97.5]), 1))
```

Output:

```
sample median: 72.6
95% bootstrap interval: [58.  78.7]
```

For 60 simulated house prices, the median is 72.6 lakh, with a 95% bootstrap interval of 58.0 to 78.7. The interval isn't symmetric around the median, because prices are skewed; the bootstrap captures that automatically. Bagging and random forests (Lesson 14) use exactly this resampling idea to build many different trees.

Hypothesis tests and A/B testing

A company tests a new checkout page: 2,400 visitors see the old page (A) and 2,400 see the new one (B). B converts at 6.5% against A's 5.0%. Is B really better, or was it luck? A hypothesis test makes this precise:

1. State the **null hypothesis** H_0: the pages convert equally well.
2. Compute a **test statistic** that measures the difference in standard-error units. For two proportions, using the pooled rate $\hat{p}$:

$$z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$$

3. The **p-value** is the probability of a difference at least this large *if H_0 were true*. If it's below a threshold α chosen in advance (usually 0.05), reject H_0.

```
# A/B test: did the new checkout page raise the conversion rate?
conv = np.array([120, 156]); n = np.array([2400, 2400])
p1, p2 = conv / n
p_pool = conv.sum() / n.sum()
se = np.sqrt(p_pool * (1 - p_pool) * (1 / n[0] + 1 / n[1]))
z = (p2 - p1) / se
print(f"conversion: A {p1:.3f}, B {p2:.3f}   z {z:.2f}   two-sided p-value {2 *
stats.norm.sf(abs(z)):.4f}")
```

Output:

```
conversion: A 0.050, B 0.065   z 2.23   two-sided p-value 0.0256
```

$z = 2.23$ and $p = 0.0256$: a difference this big would happen by luck less than 3% of the time if the pages were equal, so at $\alpha = 0.05$ we conclude B is better. Two kinds of mistake are possible. A **type I error** (false positive) means declaring a difference that isn't real; its rate is α. A **type II error** (false negative) means missing a real difference. The **power** of a test, $1 - P(\text{type II})$, is the chance of detecting a real effect. It grows with sample size, which is why A/B tests are planned

with a minimum number of visitors *before* they start, and must not be stopped early as soon as p dips below 0.05.

Multiple testing: why "significant" findings often vanish

Test 20 features that have no real effect, each at $\alpha = 0.05$, and the chance that *at least one* looks significant is $1 - 0.95^{20} \approx 0.64$:

```
# Multiple testing: 20 features with no real effect, each tested at the 5% level, in
5,000 experiments
rng = np.random.default_rng(4)
groups = rng.normal(size=(5000, 20, 2, 30))              # two groups of 30, no true
difference
p = stats.ttest_ind(groups[:, :, 0], groups[:, :, 1], axis=2).pvalue
print("share of single tests below 0.05:", round((p < 0.05).mean(), 3))
print("share of experiments with at least one 'significant' feature:", round((p <
0.05).any(axis=1).mean(), 3))
print("theory, 1 - 0.95^20:", round(1 - 0.95**20, 3))
print("with Bonferroni (p < 0.05/20):", round((p < 0.05 / 20).any(axis=1).mean(), 3))
```

Output:

```
share of single tests below 0.05: 0.05
share of experiments with at least one 'significant' feature: 0.643
theory, 1 - 0.95^20: 0.642
with Bonferroni (p < 0.05/20): 0.04
```

Each single test is fine: 5% false positives, as promised. But 64% of experiments "find" something. Trying many features, metrics, user segments or model variants and reporting only the winner is called **p-hacking** or the "garden of forking paths". It's a big reason why published results fail to replicate. The simplest fix, the **Bonferroni correction**, uses α/m for each of m tests, bringing the overall false-alarm rate back under 5% (0.04 here). A less strict alternative that is popular when testing many features, the Benjamini–Hochberg procedure, controls the *false discovery rate* instead. In ML the same trap appears as **overfitting to the validation set** when you try many models on it, which is why a final untouched test set matters (Lesson 9).

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **What's the difference between maximum likelihood (MLE) and maximum a posteriori (MAP) estimation?**

The trap: Saying they always give the same answer.

MLE chooses the parameters that make the observed data most probable. MAP maximises likelihood × prior, so it also uses a belief about the parameters held before seeing the data. They agree with a flat prior or when there's lots of data. With little data, MAP is pulled towards the prior. MAP with a Gaussian prior is ridge regression, and with a Laplace prior it is lasso.

▼ **What does the central limit theorem actually say, and what doesn't it say?**

The trap: "With enough data, everything becomes normally distributed."

It's about *averages* (and sums) of many independent values with finite variance: their distribution approaches a normal with standard deviation $\sigma/\sqrt{n}$. It does *not* make the data itself normal: 10,000 incomes are still skewed. It also needs independence and finite variance, and convergence is slow for very skewed or heavy-tailed data. It's why error bars on averages, test accuracy and A/B results work.

▼ **How does the bootstrap work, and when would you use it?**

The trap: Describing it as "making more data".

Resample your dataset with replacement to the same size, recompute the statistic, and repeat thousands of times. The spread of the results estimates the statistic's sampling variability. The middle 95% gives a confidence interval. Use it when no simple formula exists: medians, percentiles, ratios, AUC or F1 on a test set. It doesn't create information. With a tiny or unrepresentative sample, the intervals are unreliable too.

▼ **Your A/B test reaches $p < 0.05$ on day 3 of a planned 14-day test. Can you stop and ship?**

The trap: "Yes, it's significant."

No. Checking repeatedly and stopping at the first significant result (peeking) inflates the false-positive rate far above 5%, because random fluctuations cross the threshold at some point quite often. Run for the planned sample size, which should come from a power calculation, or use a method designed for continuous monitoring (sequential

tests or Bayesian approaches). Also watch for weekday/weekend effects, which a 3-day window can't average out.

▼ **You tried 50 model variants and the best one beats the baseline with $p = 0.03$. How much do you trust it?**

The trap: Trusting the p-value as if it were a single planned test.

Not much. With 50 tries, a few will look significant at the 5% level by chance alone (expected about 2.5). Correct for the number of comparisons (Bonferroni: $0.05/50 = 0.001$), or better, confirm on a fresh test set that wasn't used for choosing. Choosing a model on a validation set and reporting that same score is the ML version of this mistake.

Check your understanding: going deeper

1. A coin shows 12 heads in 20 flips. What is the maximum-likelihood estimate of p?

a. 0.5

b. 0.6

c. 0.12

2. You start with a Beta(1, 1) prior and see 3 heads and 1 tail. What is the posterior?

a. Beta(4, 2)

b. Beta(3, 1)

c. Beta(2, 4)

3. To halve the standard error of an average, you need...

a. Twice as much data

b. Four times as much data

c. Half as much data

4. MAP estimation of linear-regression weights with a normal (Gaussian) prior is the same as...

a. Lasso

b. Ridge regression

c. Ordinary least squares

5. You test 10 independent features with no real effect, each at $\alpha = 0.05$. The chance of at least one false positive is about...

a. 5%

b. 40%

c. 50%

18. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Two fair dice are rolled. What is the probability their sum is 7?

WORKING

EXERCISE 2 By hand

For the data $2, 4, 4, 4, 5, 5, 7, 9$, compute the mean, the variance (divide by n) and the standard deviation.

WORKING

EXERCISE 3 By hand

20% of emails are spam. The word "winner" appears in 50% of spam and 5% of normal emails. An email contains "winner". What is $P(\text{spam} \mid \text{winner})$?

WORKING

EXERCISE 4 By hand

Test scores are normal with $\mu = 70$, $\sigma = 10$. What is the z-score of 85? Between which two scores do about 95% of students fall?

WORKING

EXERCISE 5 Intermediate

Calls to a help desk follow a Poisson distribution with an average of 2 per hour. What is the probability of exactly 4 calls in an hour? (Use $P(k) = e^{-\lambda}\lambda^k / k!$.)

WORKING

EXERCISE 6 Intermediate

With a Beta(2, 2) prior, you observe 3 successes and 7 failures. Give the posterior's parameters, its mean, and the MAP estimate.

WORKING

EXERCISE 7 Intermediate

A model is right on 850 of 1,000 test examples. Using the CLT, give its accuracy's standard error and an approximate 95% confidence interval.

WORKING

EXERCISE 8 Intermediate

An A/B test has 200 conversions out of 4,000 for A and 250 out of 4,000 for B. Compute the pooled rate and the z-statistic.

WORKING

EXERCISE 9 Code · intermediate

Change the bootstrap code to estimate a 95% interval for the *mean* of the 60 prices instead of the median, keeping the same random seed. Compare it with $\bar{x} \pm 1.96\, s/\sqrt{n}$.

WORKING

NOTES

PART 2 · LEARNING FROM LABELLED DATA · ~45 MIN

Lesson 5: Data, Features & Generalization

Datasets, train/validation/test splits, overfitting vs underfitting, and cross-validation.

BY THE END OF THIS LESSON YOU CAN

- Explain generalization and why training error can mislead
- Recognise underfitting and overfitting from training and test errors
- Split data correctly into training, validation and test sets
- Run and interpret k-fold cross-validation, and avoid data leakage

↺ BEFORE YOU START

This lesson builds on:

- Lesson 1: What Is Machine Learning? — A model maps features x to a prediction $\hat{y}$. Training adjusts its parameters to make a loss (such as MSE) small, and the real goal is doing well on new data.
- Lesson 4: Probability & Statistics — Probabilities run from 0 to 1. Bayes' theorem turns $P(B \mid A)$ into $P(A \mid B)$. Mean and standard deviation summarise data; the z-score $(x - \mu)/\sigma$ standardises it.

Where this fits: Before building models, you need a fair way to test them. Everything you learn here is reused in every later lesson whenever we evaluate a model.

1. The real goal: doing well on data you've never seen

In Lesson 1 we said learning means "find the parameters that make the loss small". That's true — but there's a catch. A model's job is not to describe the data it was trained on; we already *know* the answers for that data. Its job is to make good predictions on **new** data: next month's houses, tomorrow's emails, a patient who walks in next week.

This ability is called **generalization**, and it is the single most important idea in machine learning. This lesson explains how data is organised, why a model can fool you by memorising, and the techniques everyone uses to measure generalization honestly.

◎ ANALOGY: THE EXAM

A teacher gives students 50 practice questions. Student A memorises the 50 answers. Student B learns the methods behind them. On the practice questions both score 100%. On the real exam — new questions — Student A fails and Student B passes. A model that memorises its training data is Student A. We want Student B, and we need a fair exam to tell them apart.

2. What a dataset looks like

A typical supervised dataset is a table: one row per **example**, one column per **feature**, plus a **label** column. From Lesson 2, the features form a matrix X of shape $n \times d$ (n examples, d features) and the labels form a vector $\mathbf{y}$ of length n.

Kinds of features

- **Numerical** — measured quantities: size in m², age, temperature. Can be *continuous* (any value) or *discrete* (counts like number of bedrooms).
- **Categorical** — one of a fixed set of categories: colour (red/green/blue), city, product type. Computers need these converted to numbers (Lesson 20 shows how).
- **Ordinal** — categories with a natural order: small < medium < large; ratings 1–5 stars.
- **Binary** — yes/no, true/false, encoded as 1/0.
- **Text, images, audio** — unstructured data that must be turned into numbers first (e.g. word counts, pixel values).

Good data matters more than clever algorithms

A model can only learn patterns that are actually in the data. Common problems:

- **Missing values** — blanks in the table.
- **Errors and outliers** — a house listed at 5 m², a typo adding three zeros to a price.
- **Too few examples** — the model can't tell real patterns from coincidences.
- **Unrepresentative data** — training only on city flats, then predicting rural farmhouses.
- **Wrong labels** — mislabelled examples teach the wrong lesson.

THE I.I.D. ASSUMPTION

Most ML assumes examples are *independent and identically distributed* (i.i.d.): each one is drawn from the same underlying source, without depending on the others. In plain words: the future data will look like the past data. When that stops being true — fashions change, a new competitor arrives — models quietly get worse. This is called **distribution shift** or **drift**.

3. Underfitting, good fit and overfitting

Imagine data that follows a gentle curve, plus some random noise. We can fit models of different **complexity** (flexibility):

- **Underfitting** — the model is too simple to capture the pattern, e.g. a straight line through a curve. It's wrong on the training data *and* on new data. High error everywhere.
- **Good fit** — the model captures the real pattern and ignores the noise. Low error on training data, and similarly low on new data.
- **Overfitting** — the model is so flexible that it bends to pass through every training point, noise included. Training error is tiny, but on new data it makes wild mistakes.

DEFINITIONS

Training error: the loss measured on the data the model was trained on.
Test (generalization) error: the loss on new data the model has never seen.
Overfitting: training error ≪ test error. **Underfitting**: both are high.

Training error almost always keeps falling as complexity grows — a more flexible model can always fit the training points better. Test error, however, falls at first and then *rises again* once the model starts fitting noise. The sweet spot is the bottom of the test-error curve.

4. Try it: watch overfitting happen

The blue dots are training data and the orange dots are test data, both drawn from the same hidden curve plus noise. The green line is a polynomial of the chosen **degree** fitted to the training dots only. Increase the degree and watch: training error keeps shrinking, but past a point the curve starts wiggling to chase individual points, and the test error grows.

Interactive demo — open *lesson-05.html* on the website to try it.

A polynomial of degree k is $\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_k x^k$. With 12 training points, a degree-11 polynomial has 12 parameters and can pass through every point exactly — a perfect training score and a terrible model.

5. The train / validation / test split

The fix is simple and non-negotiable: **evaluate on data the model did not train on.** Before doing anything else, split your data:

Set	Used for	Typical share
Training set	Fitting the model's parameters.	~70%
Validation set	Comparing models and choosing settings (like the polynomial degree).	~15%
Test set	One final, honest estimate of performance on new data.	~15%

Google's Machine Learning Crash Course uses exactly this 70 / 15 / 15 illustration. The exact percentages vary; with very large datasets the validation and test sets can be a smaller share.

> **! WHY A SEPARATE VALIDATION SET?**
>
> If you try 20 different models and pick whichever scores best on the test set, you have used the test set to make decisions — you've tuned to it. Its score is now optimistic. The test set must stay locked away until the very end. Choices are made on the validation set; the test set gives the final verdict. Validation and test sets also "wear out": the more you reuse them, the less trustworthy they become.

Rules for a good split:

- **Shuffle randomly** before splitting (unless the data is a time series — then train on the past and test on the future).
- Make sure the test set is **representative** — the same mix of cases as real-world data.
- **No duplicates** between training and test sets, or you're testing on memorised examples.
- For classification, use a **stratified** split so each set has the same proportion of each class.

6. k-fold cross-validation

PICTURE IT Judging a restaurant by one meal is risky: maybe the chef was off that night. Eat there five times and you know it far better. Cross-validation tests a model five times on five different slices of data for the same reason.

With a small dataset, a single validation split is unreliable: the score depends heavily on *which* examples happened to land in it. **k-fold cross-validation** uses the data more efficiently:

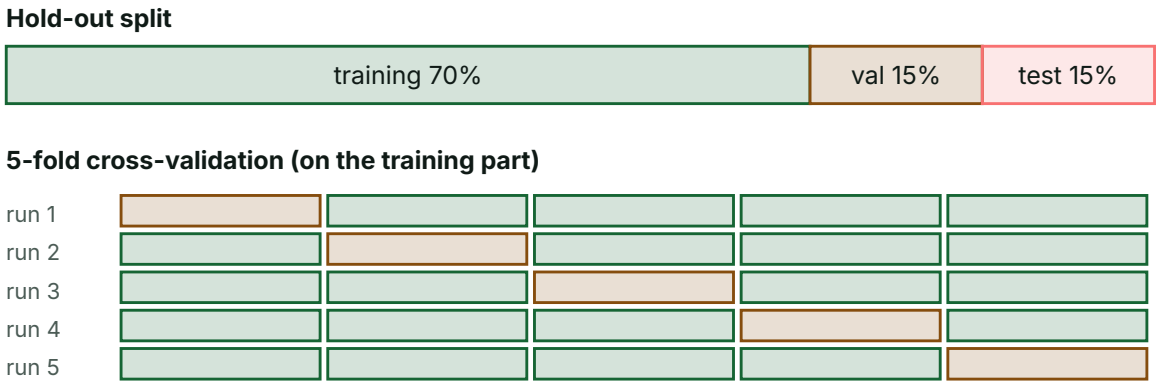


Figure 5. Top: the three-way split. Bottom: in k-fold cross-validation, each fold (orange) takes one turn as the validation set while the other folds train the model.

1. Split the training data into k equal parts called **folds** (commonly $k = 5$ or 10).
2. Train on $k - 1$ folds and evaluate on the remaining fold.
3. Repeat k times, so each fold is the validation fold exactly once.
4. Report the **mean** (and standard deviation) of the k scores.

$$\text{CV score} = \frac{1}{k} \sum_{j=1}^{k} \text{score}_j$$

✎ WORKED EXAMPLE — 5-FOLD CV

A model's accuracy on the 5 validation folds: $0.82,\ 0.85,\ 0.80,\ 0.84,\ 0.84$.

Mean: $(0.82 + 0.85 + 0.80 + 0.84 + 0.84)/5 = 4.15/5 = 0.83$.

Deviations from the mean: $-0.01,\ 0.02,\ -0.03,\ 0.01,\ 0.01$. Squared: $0.0001,\ 0.0004,\ 0.0009,\ 0.0001,\ 0.0001$. Variance $= 0.0016/5 = 0.00032$, standard deviation $= \sqrt{0.00032} \approx 0.018$.

We report **0.83 ± 0.02**. The ± tells you how much the score depends on the particular split — if two models differ by less than this, the difference may just be luck.

? MAKE SENSE OF IT

The situation: You're choosing between models for a client and want a score you can trust, not one that depends on a lucky split.

What the result tells you: 0.83 is the model's average accuracy across five different validation folds. The ± 0.02 says the score moves by about two points depending on which data it's tested on.

What if…?

- **One bad fold.** If the third fold scored 0.70 instead of 0.80, the mean drops to **0.81** and the standard deviation more than triples to **0.056**. A big spread is a warning that the model is unstable or one fold is unusual.
- **A rival model** scores 0.84, 0.86, 0.82, 0.85, 0.83: mean **0.84 ± 0.014**. It's ahead by 0.01, less than the spread of either model — not convincing evidence that it's better.

Why it matters in ML: Every score you compute is an estimate from a sample. Reporting the spread stops you from "improving" a model by chasing noise, and a high spread across folds is a clue to overfitting or data problems.

Every example is used for validation exactly once and for training $k - 1$ times. The cost: you train k models instead of one. You still keep a separate test set for the very end.

READ THE FORMULA **Cross-validation score**

$$\text{CV score} = \frac{1}{k} \sum_{j=1}^{k} \text{score}_j$$

- **1/k** — average over the k folds
- **Σ** — add up over every fold
- **score$_j$** — the score when fold j is the validation set

Hover over (or tap) a part to highlight it in the formula.

7. Data leakage: the silent killer

Data leakage happens when information from outside the training data sneaks into training, making evaluation look better than reality. Classic examples:

- **Preprocessing before splitting.** Computing the mean and standard deviation for standardisation (Lesson 4) on *all* data, including the test set. The model has indirectly "seen" the test data.
- **Features that contain the answer.** Predicting whether a customer will cancel, using a column "cancellation fee charged".
- **Time travel.** Using future information to predict the past, e.g. randomly splitting stock prices so the model trains on Wednesday to predict Tuesday.
- **Duplicates** across train and test.

> ! THE GOLDEN RULE
>
> Split first. Then learn everything — scaling statistics, feature choices, model parameters — from the training data only, and merely *apply* it to validation and test data. scikit-learn's documentation states it directly: never call `fit` on the test data.

8. Putting it together

1. Split off a **test set** and don't look at it.
2. On the training set, use **cross-validation** (or a validation set) to compare models and settings.
3. Pick the best, then **retrain** it on the full training set.
4. Evaluate **once** on the test set and report that number.

9. In Python

This experiment reproduces the demo: fit polynomials of increasing degree and compare training and test error.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
x = rng.uniform(-3, 3, 30)
y = np.sin(x) + rng.normal(0, 0.3, 30)          # hidden curve + noise
X = x.reshape(-1, 1)                             # sklearn wants a 2D matrix

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=1)

for degree in [1, 3, 9, 12]:
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    model.fit(X_train, y_train)
    train_mse = mean_squared_error(y_train, model.predict(X_train))
    test_mse = mean_squared_error(y_test, model.predict(X_test))
    print(f"degree {degree:2d}: train MSE {train_mse:.3f}   test MSE {test_mse:.3f}")
```

Output:

```
degree  1: train MSE 0.217   test MSE 0.188
degree  3: train MSE 0.041   test MSE 0.092
degree  9: train MSE 0.010   test MSE 0.250
degree 12: train MSE 0.006   test MSE 0.492
```

Degree 1 underfits: a straight line can't follow a sine wave, so both errors are high. Degree 3 fits well: both errors are low. As the degree rises to 9 and 12, training error keeps falling — but test error climbs to more than five times its best value. That's overfitting.

And 5-fold cross-validation in two lines:

```
from sklearn.model_selection import cross_val_score

model = make_pipeline(PolynomialFeatures(3), LinearRegression())
scores = cross_val_score(model, X_train, y_train, cv=5,
scoring="neg_mean_squared_error")
print(-scores.mean(), scores.std())     # mean MSE across the 5 folds, and its spread
```

(scikit-learn reports errors as negative numbers so that "higher is better" for every score; we flip the sign back.)

10. Running example: Chennai Homes

With only 12 houses, a single split leaves just 3 for testing — far too few to trust. Cross-validation uses every house for testing once: predicting price from size alone, the error per fold ranges from 8.9 to 20 lakh. That spread is a warning that such a small dataset gives noisy estimates.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
X, y = df[["size_m2"]], df["price_lakh"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=0)
print(len(X_train), "train /", len(X_test), "test houses")
scores = cross_val_score(LinearRegression(), X, y, cv=4,
scoring="neg_mean_absolute_error")
print("MAE per fold:", (-scores).round(1), " mean:", round(-scores.mean(), 1))
```

Output:

```
9 train / 3 test houses
MAE per fold: [14.4  8.9 14.5 20. ]  mean: 14.4
```

⚙ WHERE THIS IS USED

- Every serious ML project
- Model competitions keep a hidden test set for exactly this reason

Choose it when: Always.

Think twice when: Never skip it: a model evaluated on its own training data tells you almost nothing.

× MISCONCEPTIONS TO UNLEARN

MYTH A high training score proves the model is good.

REALITY Only performance on data the model hasn't seen is evidence of generalization.

MYTH Cross-validation replaces the test set.

REALITY Cross-validation is for choosing models and settings. You still need an untouched test set for the final, honest number.

RESPONSIBLE AI · BIAS STARTS IN THE DATA

A model can only be as fair as its data. Obermeyer et al. (*Science*, 2019) found that a widely used US healthcare algorithm under-referred Black patients because it predicted future healthcare *cost* as a proxy for health *need* — and less had historically been spent on Black patients with the same needs. Ask who is missing from the data, how labels were produced, and whether people consented to their data being used. "Datasheets for Datasets" (Gebru et al.) is a useful checklist.

11. Questions beginners often ask

Q. How big should the test set be?

Big enough that its score is stable — often 10–30% of the data. With millions of examples, 1% may be plenty.

Q. Can I ever look at the test set?

Only at the very end, once. If you change the model after seeing the test score, that score is no longer an honest estimate.

Q. What k should I use for cross-validation?

5 or 10 are common defaults. Larger k uses more data for training each time but costs more runs.

! COMMON MISTAKES TO AVOID

- Tuning on the test set — then its score is no longer an honest estimate.
- Preprocessing (scaling, imputing, selecting features) before splitting the data.
- Splitting time-ordered data randomly, so the model trains on the future.

! INTERVIEW CORNER

Don't confuse

- **Validation set vs test set** — Validate many times; test once.
- **Overfitting vs data leakage** — Overfitting shows up in validation; leakage hides from it.
- **Cross-validation vs a train/test split** — One opinion vs five.
- **Training error vs generalisation error** — Grading yourself vs being graded.

Hidden questions — answer in your head first, then open.

▼ **Why do you need both a validation set and a test set?**

The trap: "The test set is enough."

Every time you use a set to choose something (model, hyperparameters, threshold), its score becomes optimistically biased. The validation set absorbs that tuning; the untouched test set gives the final honest estimate. Use it once.

▼ **You select the 20 best features using all the data, then run 5-fold cross-validation. What's wrong?**

The trap: Thinking cross-validation protects you automatically.

Leakage: feature selection has already seen the validation folds' labels, so the CV score is optimistic — sometimes wildly (it can find "predictive" features in pure noise). Every learned step, including selection, scaling and imputation, must happen inside each fold — use a Pipeline.

▼ **Your training error is *higher* than your validation error. Is something broken?**

The trap: Assuming training error is always lower.

Not necessarily. Regularisation that's only active in training (dropout, data augmentation) makes training look worse; a small or easy validation set can score well by luck. It can also signal leakage or duplicates between the sets, so check.

▼ **Will collecting more data fix underfitting?**

The trap: "More data always helps."

Mostly no. More data reduces variance (overfitting). An underfitting model is too simple; it needs more features, a more flexible model or less regularisation.

12. Key takeaways

- The goal is **generalization**: good performance on new, unseen data.
- **Underfitting** = too simple (both errors high). **Overfitting** = memorising noise (training error low, test error high).
- Always split: **train** to fit, **validation** to choose, **test** once at the end.
- **k-fold cross-validation** gives a more reliable estimate: report mean ± standard deviation.
- Avoid **data leakage**: split first, fit everything on training data only.

13. Check your understanding

1. Training error 0.01, test error 2.5. What's happening?

a. Underfitting

b. A good fit

c. Overfitting

2. Training error 3.1, test error 3.3, and a simple model. What's the likely problem?

a. Underfitting

b. Overfitting

c. Data leakage

3. Which set should you use to choose between 10 candidate models?

a. The test set

b. The validation set (or cross-validation)

c. The training set

4. In 5-fold cross-validation, how many models are trained?

a. 1

b. 4

c. 5

5. You standardise all your data using its overall mean, then split into train and test. What's wrong?

a. Data leakage: test data influenced the preprocessing

b. Nothing — that's the correct order

c. Overfitting caused by too many features

14. Going deeper: validation that you can trust

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above gives you the core habits: split your data, cross-validate, avoid leakage. This section covers the situations where those habits quietly break: when tuning itself inflates your score, when your rows aren't independent (time series, repeated customers), when you need to know whether more data would help, and when tomorrow's data no longer looks like yesterday's. Each one is a common cause of models that look great in testing and disappoint in production.

You'll need: k-fold cross-validation and data leakage from this lesson.

Tuning inflates scores: nested cross-validation

When you try many settings and keep the one with the best cross-validated score, that best score is **optimistic**. Some settings win partly by luck on these particular folds, the same way the best of 20 coin-flippers looks skilled. Lesson 4's "Going deeper" called this multiple testing. The honest fix is **nested cross-validation**:

- An **inner** loop chooses the settings, using only the training part of each outer fold.
- An **outer** loop scores the whole procedure ("tune, then fit") on data the tuning never saw.

Below, a support-vector classifier (Lesson 15) is tuned over 25 combinations of settings on a small, noisy dataset:

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import GridSearchCV, cross_val_score, KFold
from sklearn.svm import SVC

# A small, noisy problem where tuning can easily "overfit the validation folds"
X, y = make_classification(n_samples=120, n_features=40, n_informative=4, flip_y=0.2,
random_state=0)
grid = {"C": [0.01, 0.1, 1, 10, 100], "gamma": [0.0001, 0.001, 0.01, 0.1, 1]}
inner, outer = KFold(5, shuffle=True, random_state=1), KFold(5, shuffle=True,
random_state=2)

search = GridSearchCV(SVC(), grid, cv=inner).fit(X, y)
print("best score inside the search (optimistic):", round(search.best_score_, 3))
nested = cross_val_score(GridSearchCV(SVC(), grid, cv=inner), X, y, cv=outer)
print("nested cross-validation (honest):        ", round(nested.mean(), 3))
```

Output:

```
best score inside the search (optimistic): 0.633
nested cross-validation (honest):         0.608
```

The search reports 0.633, but the nested estimate is 0.608. The gap grows with the number of settings you try and shrinks with more data. Nested CV is the right way to *report* a tuned model's performance. To build the final model, you still run the ordinary search on all the data. Nested CV tells you how well that recipe is likely to do.

When rows aren’t independent: time and groups

Ordinary k-fold assumes each row is an independent draw. Two very common cases break that:

- **Time series.** Neighbouring days look alike. A shuffled split puts Tuesday in training and Wednesday in validation, so the model just "remembers" the nearby level, which it won't be able to do for next month.
- **Groups.** Several rows per customer, patient or house. If one customer's rows land on both sides of the split, the model learns that person rather than the pattern.

Here's a daily sales series that drifts slowly over time, scored both ways with a random forest (Lesson 14):

```
from sklearn.model_selection import TimeSeriesSplit
from sklearn.ensemble import RandomForestRegressor

# Daily sales that drift slowly over time: neighbouring days look alike
rng = np.random.default_rng(0)
t = np.arange(600)
trend = np.cumsum(rng.normal(0, 1, 600))                  # a slow random walk
Xt = np.column_stack([t, np.sin(2 * np.pi * t / 7)])      # day number and weekday
pattern
yt = trend + 3 * Xt[:, 1] + rng.normal(0, 1, 600)

model = RandomForestRegressor(n_estimators=100, random_state=0)
shuffled = cross_val_score(model, Xt, yt, cv=KFold(5, shuffle=True, random_state=0),
scoring="r2")
forward = cross_val_score(model, Xt, yt, cv=TimeSeriesSplit(5), scoring="r2")
print("shuffled k-fold R²:   ", round(shuffled.mean(), 3))
print("time-ordered split R²:", round(forward.mean(), 3))
```

Output:

```
shuffled k-fold R²:    0.957
time-ordered split R²: -0.609
```

Shuffled k-fold says $R^2 = 0.957$: superb. A **time-ordered split** (`TimeSeriesSplit`, which always trains on the past and validates on the future) says -0.609: worse than predicting the average. The random forest can't extrapolate a drifting trend at all, and the shuffled split hid that completely. For grouped data, the same idea is `GroupKFold`: all of one group's rows go to the same side of the split. For time series with groups, combine them. A useful rule: **split the way the model will be used**.

Learning curves: would more data help?

Collecting data is expensive, so it's worth checking first whether it would help. A **learning curve** trains on growing subsets of the data and plots training and validation scores against training-set size:

```
from sklearn.model_selection import learning_curve
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

Xb, yb = load_breast_cancer(return_X_y=True)
sizes, tr, va = learning_curve(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000)), Xb, yb,
                               train_sizes=[0.05, 0.1, 0.25, 0.5, 1.0], cv=5,
shuffle=True, random_state=0)
for s, a, b in zip(sizes, tr.mean(axis=1), va.mean(axis=1)):
    print(f"{s:4d} training examples: train accuracy {a:.3f}   validation accuracy
{b:.3f}")
```

Output:

```
  22 training examples: train accuracy 1.000   validation accuracy 0.916
  45 training examples: train accuracy 0.991   validation accuracy 0.968
 113 training examples: train accuracy 0.995   validation accuracy 0.968
 227 training examples: train accuracy 0.989   validation accuracy 0.970
 455 training examples: train accuracy 0.989   validation accuracy 0.981
```

How to read it:

- **Big gap, validation still rising** (high variance): more data is likely to help. Here, validation accuracy climbs from 0.916 with 22 examples to 0.981 with 455, and the gap to training (0.989) is now small.
- **Both curves low and close together** (high bias): more data won't help much. You need a more flexible model or better features.
- **Both flat and close, both high**: you're near the limit for this model and these features.

This is the bias–variance trade-off (Lesson 10) made visible. It answers a question that cross-validation alone can't.

When the world changes: distribution shift

All validation assumes future data comes from the same distribution as your training data. In practice it often drifts:

- **Covariate shift**: the inputs change, but the relationship stays (a new city with bigger homes).
- **Label shift**: the class balance changes (fraud rises during a holiday season).
- **Concept drift**: the relationship itself changes (after a rule change, the same features mean something else).

A neat trick to detect the first kind is **adversarial validation**. Label training rows 0 and new rows 1, and train a classifier to tell them apart. If it can't (AUC near 0.5), the two look alike. If it can, the shift is real, and its feature importances show *where* it is.

```
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import cross_val_predict, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

# Adversarial validation: can a model tell training rows from "production" rows?
rng = np.random.default_rng(1)
train_rows = rng.normal(0, 1, size=(500, 3))
prod_same = rng.normal(0, 1, size=(500, 3))
prod_shift = rng.normal(0, 1, size=(500, 3)); prod_shift[:, 0] += 0.8   # one feature
has drifted

cv = StratifiedKFold(5, shuffle=True, random_state=0)
for name, prod in [("no shift", prod_same), ("feature 0 drifted", prod_shift)]:
    Xa = np.vstack([train_rows, prod]); ya = np.r_[np.zeros(500), np.ones(500)]
    clf = RandomForestClassifier(n_estimators=200, min_samples_leaf=10, random_state=0)
    p = cross_val_predict(clf, Xa, ya, cv=cv, method="predict_proba")[:, 1]
    imp = clf.fit(Xa, ya).feature_importances_
    print(f"{name:18}: AUC {roc_auc_score(ya, p):.3f}   feature importances
{imp.round(2)}")
```

Output:

```
no shift          : AUC 0.540   feature importances [0.34 0.34 0.32]
feature 0 drifted : AUC 0.695   feature importances [0.59 0.21 0.2 ]
```

With no shift, the AUC is 0.540, close to chance, and the importances are evenly spread. When feature 0 drifts by 0.8 standard deviations, the AUC jumps to 0.695 and feature 0's importance rises to 0.59, pointing straight at the culprit. In production, teams monitor this kind of drift continuously and retrain when it appears (Lesson 20).

A validation checklist

Situation	Use
Reporting a tuned model's score	Nested CV, or a test set used once at the very end
Data ordered in time	Time-ordered splits (`TimeSeriesSplit`); never shuffle
Several rows per person, item or place	`GroupKFold`
Rare classes	Stratified splits (`StratifiedKFold`)
Deciding whether to collect more data	A learning curve
New data might differ from training data	Adversarial validation and drift monitoring

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **What's nested cross-validation, and when do you need it?**

The trap: Thinking the best grid-search score is an unbiased estimate.

Two loops: an inner CV chooses hyperparameters using only the training part of each outer fold, and the outer CV scores the whole "tune then fit" procedure on data the tuning never saw. You need it when you want an honest performance estimate for a tuned model and don't have a separate test set. The best inner score is biased upwards, more so with many settings and little data.

▼ **Your model scores $R^2 = 0.95$ in cross-validation but fails badly in production a month later. List likely causes.**

The trap: Blaming bad luck.

Leakage (a feature that won't exist at prediction time, or preprocessing fitted on all data). A split that doesn't match use, such as shuffled folds on time-ordered data or the same customer on both sides. Distribution shift, where production inputs differ from training (check with adversarial validation). Overfitting to the validation set through many tuning rounds. Fix: time-based or group splits, pipelines, a final untouched test set and drift monitoring.

▼ **How do you decide whether collecting more data is worth it?**

The trap: "More data always helps."

Plot a learning curve: train and validation scores against training-set size. If the validation score is still rising and there's a big gap to training (high variance), more data should help. If both are low and close together (high bias), more data won't, and you need better features or a more flexible model. You can extrapolate the curve's trend to estimate the gain.

▼ **What's the difference between covariate shift, label shift and concept drift?**

The trap: Using "data drift" for all three.

Covariate shift: the distribution of inputs $P(x)$ changes but $P(y \mid x)$ stays the same. Label shift: the class balance $P(y)$ changes but $P(x \mid y)$ stays. Concept drift: the relationship $P(y \mid x)$ itself changes. Covariate shift can be detected without labels

(adversarial validation, statistical tests on features); concept drift needs fresh labels to detect, and usually retraining to fix.

Check your understanding: going deeper

1. You try 200 hyperparameter settings and report the best cross-validated score. That score is probably...

a. Slightly too optimistic

b. Exactly right

c. Too pessimistic

2. Which split suits forecasting next month's sales?

a. Shuffled k-fold

b. A time-ordered split: train on the past, validate on later dates

c. Leave-one-out

3. A learning curve shows training accuracy 0.99 and validation accuracy 0.80, with validation still rising as data grows. What's likely to help?

a. More data or more regularization

b. A more complex model

c. Nothing will help

4. In adversarial validation, a classifier separates training rows from new rows with AUC 0.95. What does that mean?

a. Your model is excellent

b. The new data differs a lot from the training data

c. There is label leakage

5. Your dataset has 10 rows per patient. Which cross-validation should you use?

a. KFold with shuffle

b. GroupKFold with patient as the group

c. StratifiedKFold

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 Concepts

Diagnose each model: (a) training error 0.02, test error 0.40; (b) training 0.35, test 0.37; (c) training 0.10, test 0.12.

WORKING

EXERCISE 2 By hand

5-fold cross-validation accuracies: $0.70, 0.74, 0.72, 0.76, 0.73$. Report mean ± standard deviation.

WORKING

EXERCISE 3 Concepts

Spot the leakage: a team (a) removes outliers using the whole dataset before splitting; (b) predicts loan default using the column "collections_agency_contacted"; (c) shuffles stock prices randomly before splitting.

WORKING

EXERCISE 4 By hand

You have 10,000 examples. How many go to training, validation and test with a 70/15/15 split?

WORKING

EXERCISE 5 Intermediate

A grid search tries 3 values of `C` and 4 values of `gamma`, with 5-fold inner CV inside a 5-fold outer CV. How many models are trained in total, not counting refits?

WORKING

EXERCISE 6 Intermediate

`TimeSeriesSplit(n_splits=4)` on 100 rows uses test blocks of 20 rows each. How many rows are in the training set of the first split, and of the last?

WORKING

EXERCISE 7 Intermediate

On a learning curve, the training score is 0.72 and the validation score 0.70 at every training size from 1,000 to 50,000 rows. Is the model suffering from high bias or high variance, and would 10 times more data help?

WORKING

EXERCISE 8 Code · intermediate

In the time-series code, change the forest to `LinearRegression()` and re-run both splits. Which split changes more, and why?

WORKING

NOTES

PART 2 · LEARNING FROM LABELLED DATA · ~55 MIN

Lesson 6: Linear Regression

Fit a line (and a plane) to data, derive the least-squares solution, and interpret the weights.

BY THE END OF THIS LESSON YOU CAN

- Write the linear regression model for one or many features
- Derive the least-squares formulas for the slope and intercept
- Fit a line by hand and with the normal equation
- Measure fit with R^2 and interpret the weights carefully

↺ BEFORE YOU START

This lesson builds on:

- Lesson 2: Vectors & Matrices — A vector is a list of numbers; a dataset X is a matrix with one row per example. The dot product $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models predict, and $\|\mathbf{a} - \mathbf{b}\|$ is the distance between two examples.
- Lesson 3: Derivatives & Gradients — A derivative is a slope; the gradient ∇L collects all partial derivatives and points uphill. Gradient descent steps the other way: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$. The chain rule differentiates functions inside functions.
- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.

Where this fits: Your first complete model. Its pattern — weights × features + bias, trained by minimising a loss — reappears in almost every later model.

1. The idea: draw the best straight line

Linear regression is the "hello world" of machine learning — and still one of the most used models in industry. It predicts a number by assuming the label changes in a straight-line way with each feature. You met it in Lesson 1 when you moved sliders; now you'll learn exactly how the best line is computed, and why.

It's worth learning thoroughly: logistic regression (Lesson 8), regularized models (Lesson 10), support vector machines (Lesson 15) and every neuron in a neural network (Lesson 18) are built on the same pattern of *weights × features + bias*.

2. The model

With a single feature x, the model is a straight line:

$$\hat{y} = wx + b$$

- w — the **weight** (slope): how much $\hat{y}$ changes when x goes up by 1.
- b — the **bias** (intercept): the prediction when $x = 0$.

With d features $x_1, \ldots, x_d$, each gets its own weight:

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \mathbf{w} \cdot \mathbf{x} + b$$

That's the dot product from Lesson 2. With two features the model is a flat *plane*; with more, a "hyperplane" — the same idea in more dimensions.

3. What does ”best” mean? Least squares

For each example i, the **residual** is the vertical gap between the true value and the line: $r_i = y_i - \hat{y}_i$. The best line is the one that makes the residuals as small as possible overall. Linear regression uses the **mean squared error**:

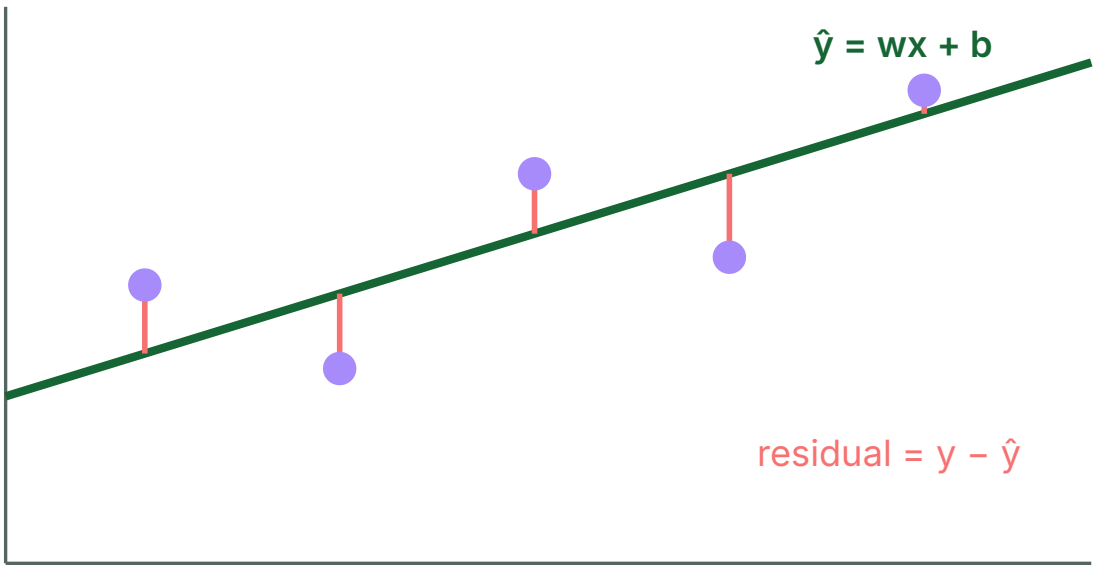


Figure 6. Each red segment is a residual — the gap between a data point and the line. Least squares minimises the sum of their squares.

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} \big(y_i - (w x_i + b)\big)^2$$

Minimising this is called **ordinary least squares (OLS)**. Why squares?

- Squaring makes every error positive, so errors above and below the line don't cancel out.
- Big errors are punished much more than small ones (an error of 4 costs 16; an error of 1 costs 1).
- The squared loss is a smooth bowl with exactly one bottom, so we can find the minimum with calculus.

> ◎ PICTURE IT
>
> Each residual squared is literally the *area of a square* whose side is the residual. Least squares finds the line that makes the total area of all those squares as small as possible. The demo below draws them.

4. Finding the best line with calculus

At the bottom of a bowl the slope is zero in every direction. So we take the partial derivatives of L (Lesson 3), set them to zero, and solve.

Step 1 — derivative with respect to b. By the chain rule, the derivative of $(y_i - w x_i - b)^2$ with respect to b is $2(y_i - w x_i - b) \cdot (-1)$:

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_{i=1}^{n} (y_i - w x_i - b) = 0$$

Divide by -2, and split the sum: $\frac{1}{n}\sum y_i - w\frac{1}{n}\sum x_i - b = 0$. Using the means $\bar{x}$ and $\bar{y}$:

$$b = \bar{y} - w\bar{x}$$

So the best line always passes through the point $(\bar{x}, \bar{y})$ — the centre of the data.

Step 2 — derivative with respect to w. The inner derivative is now $-x_i$:

$$\frac{\partial L}{\partial w} = -\frac{2}{n} \sum_{i=1}^{n} (y_i - w x_i - b)\, x_i = 0$$

Substitute $b = \bar{y} - w\bar{x}$ and rearrange (a few lines of algebra) to get:

$$w = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$$

WHAT THE FORMULA MEANS

The top measures how x and y move *together* (if x is above its average when y is above its average, the products are positive). The bottom measures how much x varies. The slope is "how much y moves with x, per unit of x's own spread". In statistics these are the *covariance* and *variance*: $w = \mathrm{cov}(x, y)/\mathrm{var}(x)$.

STEP-BY-STEP DERIVATION **Fill in the "few lines of algebra" for the slope**

1 $$\sum_i x_i(y_i - wx_i - b) = 0$$

Setting $\partial L/\partial w = 0$ and dropping the constant factor $-2/n$.

2 $$\sum_i x_i y_i - w\sum_i x_i^2 - b\sum_i x_i = 0$$

Multiply out the bracket and split the sum.

3 $$\sum_i x_i y_i - w\sum_i x_i^2 - (\bar{y} - w\bar{x})\, n\bar{x} = 0$$

Substitute $b = \bar{y} - w\bar{x}$ and use $\sum_i x_i = n\bar{x}$.

4 $$\sum_i x_i y_i - n\bar{x}\bar{y} = w\Big(\sum_i x_i^2 - n\bar{x}^2\Big)$$

Collect the terms with w on the right.

5 $$\sum_i (x_i - \bar{x})(y_i - \bar{y}) = \sum_i x_i y_i - n\bar{x}\bar{y}, \quad \sum_i (x_i - \bar{x})^2 = \sum_i x_i^2 - n\bar{x}^2$$

Two standard identities (expand the brackets to check them).

6 $$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

Divide — the slope formula.

5. Worked example by hand

✎ WORKED EXAMPLE — FIT A LINE TO 5 POINTS

Data: $x = 1, 2, 3, 4, 5$ and $y = 2, 4, 5, 4, 5$.

Means: $\bar{x} = 15/5 = 3$, $\bar{y} = 20/5 = 4$.

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	product	$(x_i - \bar{x})^2$
1	2	−2	−2	4	4
2	4	−1	0	0	1
3	5	0	1	0	0
4	4	1	0	0	1
5	5	2	1	2	4
Sum				**6**	**10**

Slope: $w = 6/10 = 0.6$. **Intercept:** $b = 4 - 0.6 \times 3 = 2.2$.

Best line: $\hat{y} = 0.6x + 2.2$.

Check the fit: predictions $2.8, 3.4, 4.0, 4.6, 5.2$; residuals $-0.8, 0.6, 1.0, -0.6, -0.2$; squared $0.64, 0.36, 1.00, 0.36, 0.04$, which sum to 2.4. So $\text{MSE} = 2.4/5 = 0.48$. No other straight line gets a smaller MSE on these five points.

Use it: for $x = 6$, predict $\hat{y} = 0.6 \times 6 + 2.2 = 5.8$.

? MAKE SENSE OF IT

The situation: Five students: hours of revision x and test score y out of 10. A teacher wants a simple rule for how much each hour helps.

What the result tells you: Slope 0.6: on average each extra hour goes with 0.6 more marks. Intercept 2.2: the line's prediction for zero hours. MSE 0.48 means a typical miss of about $\sqrt{0.48} \approx 0.69$ marks.

What if…?

- **One outlier.** If the last student had scored 15 instead of 5 (a typo), the slope jumps from 0.6 to **2.6** and the intercept to **−1.8**. Change the *first* student from 2 to 12 instead, and the slope flips to **−1.4**: "more revision, lower score". One point at the edge can tilt the whole line.
- **Everyone gets 10 more marks:** the intercept becomes 12.2; the slope stays 0.6.
- **Measure time in tenths of an hour** ($x \times 10$): the slope becomes 0.06 — the same relationship, in different units. A weight's size depends on its feature's units.
- **Add a sixth point exactly on the line** (6, 5.8): the line doesn't move at all, and the MSE falls to 0.40.

Why it matters in ML: Least squares squares the residuals, so points far from the line — especially at the ends of the x range — pull hard on it. Plot your data and look for outliers before trusting a fitted slope.

6. Try it: least squares, live

Click on the chart to add points (click a point to remove it). The line is recalculated with the formulas above every time. The shaded squares are the squared residuals — the line always positions itself to make their total area as small as possible. Notice how a single far-away point (an outlier) drags the whole line towards it.

Interactive demo — open *lesson-06.html* on the website to try it.

7. How good is the fit? R^2

MSE depends on the units of y, so on its own it's hard to judge. The **coefficient of determination**, R^2, compares the model with the simplest possible baseline — always predicting the mean $\bar{y}$:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\mathrm{SS}_{\mathrm{res}}}{\mathrm{SS}_{\mathrm{tot}}}$$

- $R^2 = 1$: perfect predictions.
- $R^2 = 0$: no better than predicting the average every time.

- $R^2 < 0$: worse than the average (possible on test data with a bad model).

✎ WORKED EXAMPLE

From before: $\text{SS}_{\text{res}} = 2.4$. And $\text{SS}_{\text{tot}} = (-2)^2 + 0^2 + 1^2 + 0^2 + 1^2 = 6$.

$R^2 = 1 - 2.4/6 = 0.6$. The line explains 60% of the variation in y.

? MAKE SENSE OF IT

The situation: Your manager asks: "How much of the difference in scores does revision time actually explain?"

What the result tells you: $R^2 = 0.6$: the line removes 60% of the squared variation around the average score. The other 40% comes from things the model doesn't know about — sleep, ability, luck.

What if...?

- **Points exactly on a line** ($y = x$): $R^2 = 1$.
- **Predict the average score for everyone:** $R^2 = 0$ — no better than knowing nothing.
- **A model worse than the average** has a *negative* R^2 (Lesson 9 has an example with −1.53).

Why it matters in ML: R^2 compares your model with the simplest possible baseline, the mean. It's unitless, so it's easy to communicate — but always report an error in real units (RMSE, MAE) too.

READ THE FORMULA **R^2 (coefficient of determination)**

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- **$\Sigma\,(y_i - \hat{y}_i)^2$** — our model's total squared error
- **$\Sigma\,(y_i - \bar{y})^2$** — the error of always guessing the mean (the baseline)

Hover over (or tap) a part to highlight it in the formula.

8. Many features and the normal equation

With many features, writing separate formulas gets messy. Matrices (Lesson 2) make it clean. A neat trick: add a column of 1s to the data matrix X, so the bias becomes just another weight. Then all predictions are:

$$\hat{\mathbf{y}} = X\mathbf{w}$$

and the loss is $L(\mathbf{w}) = \frac{1}{n}\|X\mathbf{w} - \mathbf{y}\|^2$. Setting the gradient to zero, $\nabla L = \frac{2}{n}X^\top(X\mathbf{w} - \mathbf{y}) = \mathbf{0}$, gives the **normal equation**:

$$X^\top X\,\mathbf{w} = X^\top \mathbf{y} \quad \Longrightarrow \quad \mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$$

Here $(X^\top X)^{-1}$ is the **inverse** matrix — the matrix version of dividing.

✎ WORKED EXAMPLE — THE SAME 5 POINTS, WITH MATRICES

Each row of X is $[1,\ x_i]$ (the 1 is for the bias):

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}, \quad X^\top X = \begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix}, \quad X^\top \mathbf{y} = \begin{bmatrix} 20 \\ 66 \end{bmatrix}$$

(5 = number of points, $15 = \sum x_i$, $55 = \sum x_i^2$, $20 = \sum y_i$, $66 = \sum x_i y_i$.)

The inverse of a 2×2 matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ is $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$. Here $ad - bc = 275 - 225 = 50$:

$$\mathbf{w} = \frac{1}{50}\begin{bmatrix} 55 & -15 \\ -15 & 5 \end{bmatrix}\begin{bmatrix} 20 \\ 66 \end{bmatrix} = \frac{1}{50}\begin{bmatrix} 110 \\ 30 \end{bmatrix} = \begin{bmatrix} 2.2 \\ 0.6 \end{bmatrix}$$

Bias 2.2, slope 0.6 — exactly what we found before.

? MAKE SENSE OF IT

The situation: The same calculation as the worked example, but written so it works unchanged for 2 features or 2,000.

What the result tells you: Solving $X^\top X\,\mathbf{w} = X^\top \mathbf{y}$ gives bias 2.2 and slope 0.6 — the same answer as the formula, which is a good check.

What if…?

- **All students revised 3 hours:** $X^\top X = \begin{bmatrix} 5 & 15 \\ 15 & 45 \end{bmatrix}$ and $ad - bc = 225 - 225 = 0$. No inverse, no unique line — you can't learn the effect of revision if it never varies.
- **Two identical feature columns** (say, hours and hours again) cause the same problem: the model can't tell how to split the credit between them.

Why it matters in ML: When features are perfectly or nearly duplicates (*multicollinearity*), the solution becomes unstable or impossible. Regularisation (Lesson 10) fixes this by adding λ to the diagonal so the matrix always has an inverse.

! WHEN THE NORMAL EQUATION STRUGGLES

Inverting $X^\top X$ costs roughly d^3 operations, so with hundreds of thousands of features it becomes slow. It also breaks when features are perfectly correlated (e.g. a price in dollars *and* the same price in euros), because $X^\top X$ has no inverse. Libraries use numerically safer methods, and for huge problems we use **gradient descent** instead — the subject of Lesson 7.

STEP-BY-STEP DERIVATION **Derive the normal equation**

1. $L(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|^2 = (X\mathbf{w} - \mathbf{y})^\top (X\mathbf{w} - \mathbf{y})$

   A squared length is a vector dotted with itself. (The $\frac{1}{n}$ factor doesn't change where the minimum is, so we drop it.)

2. $L(\mathbf{w}) = \mathbf{w}^\top X^\top X\mathbf{w} - 2\,\mathbf{w}^\top X^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$

   Multiply out, like $(a-b)^2 = a^2 - 2ab + b^2$.

3. $\nabla L(\mathbf{w}) = 2X^\top X\mathbf{w} - 2X^\top \mathbf{y}$

   Differentiate: the matrix versions of $\frac{d}{dw}(aw^2) = 2aw$ and $\frac{d}{dw}(cw) = c$.

4. $X^\top X\,\mathbf{w} = X^\top \mathbf{y}$

   At the minimum the gradient is zero.

5. $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$

   Multiply both sides by the inverse (if it exists).

ADVANCED · OPTIONAL **Deriving the normal equation, and what it means geometrically**

▾ hide

With all features in a matrix X (a column of 1s for the intercept) the loss is $L(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 = \mathbf{y}^\top \mathbf{y} - 2\mathbf{w}^\top X^\top \mathbf{y} + \mathbf{w}^\top X^\top X\mathbf{w}$. Its gradient is

$$\nabla L = -2X^\top \mathbf{y} + 2X^\top X\mathbf{w} = -2X^\top(\mathbf{y} - X\mathbf{w}).$$

Setting it to zero gives the **normal equation** $X^\top X\,\mathbf{w} = X^\top \mathbf{y}$, so $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$ when $X^\top X$ is invertible (no feature is an exact combination of the others). The Hessian $2X^\top X$ is positive semi-definite, so this is a minimum.

Geometry. $X^\top(\mathbf{y} - X\mathbf{w}) = \mathbf{0}$ says the residual vector is perpendicular to every column of X : the prediction $X\mathbf{w}$ is the *orthogonal projection* of $\mathbf{y}$ onto the space spanned by the features — the closest point to $\mathbf{y}$ that a linear model can reach.

9. Interpreting the weights

A big advantage of linear regression is that you can read it. If a house-price model learns

$$\widehat{\text{price}} = 2.5 \cdot \text{size} + 12 \cdot \text{bedrooms} - 4 \cdot \text{distance} + 40$$

(price in thousands, size in m², distance in km), then, holding the other features fixed:

- each extra m² adds about 2.5k to the price;
- each extra bedroom adds about 12k;
- each extra km from the city *subtracts* about 4k.

> **! TWO CAUTIONS**
>
> **Units matter.** A weight's size depends on its feature's units (m² vs km). To compare which feature "matters most", standardise features first (z-scores, Lesson 4) so all weights are per standard deviation.
>
> **Correlation is not causation.** A weight shows association in this data, not that changing the feature would cause the change. Adding a bedroom by splitting a room won't necessarily add 12k.

10. Limitations and how to extend it

- **Linearity.** If the true relationship curves, a straight line underfits. Fix: create new features such as x^2, x^3 (*polynomial features*, as in Lesson 5's demo) — the model is still "linear" in its weights, so the same maths works.
- **Outliers.** Squared error lets a single extreme point pull the line a long way (try it in the demo).
- **Correlated features** make individual weights unstable and hard to interpret; regularization (Lesson 10) helps.
- **Residuals** should look like random noise with no pattern. A curved pattern in the residuals means the model is missing something.

11. In Python: from scratch, then with scikit-learn

From scratch with NumPy, using the formulas above:

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 5, 4, 5], dtype=float)

# Simple formulas
w = np.sum((x - x.mean()) * (y - y.mean())) / np.sum((x - x.mean()) ** 2)
b = y.mean() - w * x.mean()
print(w, b)                          # 0.6 2.2

# Normal equation with a column of ones for the bias
X = np.column_stack([np.ones_like(x), x])
w_vec = np.linalg.solve(X.T @ X, X.T @ y)   # solves (XᵀX) w = Xᵀy
print(w_vec)                         # [2.2 0.6]

y_hat = X @ w_vec
r2 = 1 - np.sum((y - y_hat) ** 2) / np.sum((y - y.mean()) ** 2)
print(round(r2, 3))                  # 0.6
```

On a real dataset with scikit-learn — the built-in diabetes dataset (442 patients, 10 features, predicting disease progression one year later):

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = LinearRegression().fit(X_train, y_train)
print(model.coef_.round(1))          # one weight per feature
print(round(model.intercept_, 1))
print(round(r2_score(y_test, model.predict(X_test)), 3))   # R² on unseen data
```

Output:

```
[  37.9 -242.   542.4  347.7 -931.5  518.1  163.4  275.3  736.2   48.7]
151.3
0.453
```

(The weights look large because this dataset's features were already rescaled to small values by its creators.) An R^2 of about 0.45 on the test set: the model explains roughly 45% of the variation

in disease progression for patients it has never seen — useful, but far from perfect, which is typical for medical data.

12. Running example: Chennai Homes

Size alone explains little ($R^2 = 0.19$): big houses far from the centre are cheap. Adding distance and age lifts R^2 to 0.91. Read the weights: each extra m² adds about 0.72 lakh, each km from the centre subtracts about 2 lakh, and each year of age about 1 lakh — holding the others fixed.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.linear_model import LinearRegression
m1 = LinearRegression().fit(df[["size_m2"]], df["price_lakh"])
print("price = %.3f * size + %.2f" % (m1.coef_[0], m1.intercept_), " R2 =",
round(m1.score(df[["size_m2"]], df["price_lakh"]), 3))
cols = ["size_m2", "dist_km", "age_yrs"]
m3 = LinearRegression().fit(df[cols], df["price_lakh"])
print({c: round(float(v), 3) for c, v in zip(cols, m3.coef_)},
round(m3.intercept_, 2), " R2 =", round(m3.score(df[cols], df["price_lakh"]), 3))
```

Output:

```
price = 0.290 * size + 41.11  R2 = 0.187
{'size_m2': 0.715, 'dist_km': -1.985, 'age_yrs': -0.988} 29.0  R2 = 0.913
```

⚙ WHERE THIS IS USED

- Forecasting sales, demand and prices
- Estimating how much each factor contributes, e.g. in economics
- A fast, explainable baseline

Choose it when: The relationship is roughly linear, you need interpretable weights, or as a first baseline.

Think twice when: The relationship is strongly curved or full of interactions (add features, or use trees or neural networks).

× MISCONCEPTIONS TO UNLEARN

MYTH A large weight means a feature is important.

REALITY Weights depend on units. A feature in grams gets a weight 1000× bigger than the same feature in kilograms. Standardise before comparing.

MYTH Linear regression can only fit straight lines.

REALITY It's linear in its weights, not its inputs: add x^2, $\log x$ or products of features and it fits curves.

RESPONSIBLE AI · COEFFICIENTS ARE NOT CAUSES

A regression weight shows an association in this data, holding the other features fixed — not what would happen if you intervened. Leaving out a relevant variable (omitted-variable bias) can flip a coefficient's sign. Never base a policy decision on a coefficient alone without a causal argument or an experiment.

13. Questions beginners often ask

Q. Why square the errors instead of using absolute values?

Squared error is smooth, gives a closed-form solution and punishes large errors more. Absolute error (the basis of MAE) is more robust to outliers but harder to optimise; both are valid choices.

Q. Can linear regression fit curves?

Yes, by adding features such as x^2 or x^3. The model stays linear in its weights, so the same maths works.

Q. What if two features are almost the same?

Their weights become unstable and hard to interpret. Remove one, combine them, or use regularization (Lesson 10).

! COMMON MISTAKES TO AVOID

- Comparing raw weights across features measured in different units.
- Extrapolating far outside the range of the training data.
- Ignoring outliers, which can pull a least-squares line a long way.

! INTERVIEW CORNER

Don't confuse

- **Linear in features vs linear in parameters** — Linear refers to the weights, not the shape.
- **R^2 vs correlation** — r has a sign; R^2 is a score.
- **Residual vs error** — Residuals are the error's visible shadow.

Hidden questions — answer in your head first, then open.

▼ **Is $\hat{y} = w_1 x + w_2 x^2$ a linear regression?**

The trap: No, because it's a curve.

Yes. "Linear" means linear in the parameters w_1, w_2. You add x^2 as a feature and use the same least-squares solution.

▼ **You add a column of random noise to a linear regression. What happens to the training R^2?**

The trap: Expecting it to fall.

It can only stay the same or rise: least squares can always set the new weight to zero, and usually finds a tiny improvement by fitting noise (0.8476 → 0.8478 in a quick test). That's why adjusted R^2, or held-out R^2, is used to compare models with different numbers of features.

▼ **Can R^2 be negative?**

The trap: "It's a squared number."

On training data with an intercept, no. On test data, yes: $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$ goes below 0 whenever the model is worse than predicting the mean.

▼ **A feature's coefficient is positive on its own but turns negative when you add another feature. Is that a bug?**

The trap: Assuming coefficients are fixed properties of a feature.

No. A coefficient is the effect *holding the other features constant*. With two highly correlated features, the credit is split between them and signs can flip: in a quick test, a feature's slope was +1.97 alone and −1.04 alongside a correlated partner. Interpret coefficients with care, and use regularisation or drop redundant features.

14. Key takeaways

- Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$.
- **Least squares** chooses the weights that minimise the mean squared error.
- One feature: $w = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$, $b = \bar{y} - w\bar{x}$. Many features: $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$.
- R^2 measures the fraction of variation explained, compared with always predicting the mean.
- Weights are interpretable — but watch units, outliers and the difference between correlation and causation.

15. Check your understanding

1. A model is $\hat{y} = 3x - 2$. What does it predict for $x = 4$?

a. 12

b. 10

c. 14

2. The least-squares line always passes through which point?

a. $(\bar{x}, \bar{y})$, the mean of the data

b. The origin $(0, 0)$

c. The first data point

3. A model's R^2 on test data is 0. What does that mean?

a. It's perfect

b. It predicts the opposite of the truth

c. It's no better than always predicting the average

4. For data with $\sum(x_i - \bar{x})(y_i - \bar{y}) = 12$ and $\sum(x_i - \bar{x})^2 = 4$, what is the slope?

a. 0.33

b. 3

c. 48

5. Why might you prefer gradient descent over the normal equation?

a. With very many features, inverting $X^\top X$ is too slow

b. Gradient descent always gives a more accurate answer

c. The normal equation only works for one feature

16. Going deeper: the statistics behind linear regression

INTERMEDIATE · SKIP ON A FIRST READ

Everything above is enough to *build* a good linear regression model. This section is for when you need to *trust and explain* one: which weights are real and which could be chance, what to do when features overlap, and how to spot data points that bend the line. These questions come up constantly in data-science interviews and in any job where a model's weights are reported to people.

You'll need: Lesson 4 (standard deviation, the normal distribution) and the normal equation from section 8.

The four assumptions, and what each one buys you

Least squares always gives you *a* line. Whether you can trust the line's weights, standard errors and p-values depends on four assumptions about the errors $\varepsilon_i = y_i - (\mathbf{w} \cdot \mathbf{x}_i + b)$. An easy way to remember them is **LINE**:

Assumption	In plain words	How to check	If it fails
Linearity	The average of y really is a straight-line function of the features.	Residuals vs predictions show no curve.	Add features such as x^2 or $\log x$ (section 10), or use a non-linear model.
Independence	One error tells you nothing about the next one.	Think about how the data was collected; for data in time order, plot residuals over time.	Standard errors come out too small. Common with time series and repeated measurements of the same person.
Normality	Errors follow a bell curve.	Histogram of residuals.	Matters only for exact p-values in *small* datasets; with a few hundred points it hardly matters.
Equal spread	Errors are about as big for large predictions as for small ones (statisticians call this *homoscedasticity*).	Residuals fan out as predictions grow.	Weights are still fine on average, but standard errors are wrong. Fixes below.

Notice what is *not* on the list: the features don't need to be normally distributed, and neither does y itself. Only the errors matter. And if all you want is good predictions on new data, checked on a test set, none of these is strictly required. They matter when you start saying things like "bmi has a real effect".

Why squared error? Two classic answers

1. It is the maximum-likelihood answer for bell-curve noise. Suppose $y_i = \mathbf{w} \cdot \mathbf{x}_i + b + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. The probability (likelihood) of seeing all the data is a product of normal densities. Taking the log turns the product into a sum:

$$\log L = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2$$

The first term doesn't depend on the weights, so making $\log L$ as large as possible is exactly the same as making $\sum(y_i - \hat{y}_i)^2$ as small as possible. Least squares *is* maximum likelihood under normal noise. (With Laplace noise, which has heavier tails than the bell curve, the same argument

gives absolute error instead, which leads to median regression. Lesson 8 uses the same reasoning to derive log loss.)

2. The Gauss–Markov theorem. If the errors have mean zero, are uncorrelated and have equal spread (the I and E above; normality is not needed), then among all estimators that are *linear in y and unbiased*, least squares has the smallest variance. It is the "best linear unbiased estimator", or **BLUE**. The fine print matters: a *biased* estimator can do better overall. Ridge regression (Lesson 10) accepts a little bias for a big drop in variance, and often predicts better.

How sure are we about each weight? Standard errors, p-values and intervals

Fit the same model on a different sample of patients and you'd get slightly different weights. The **standard error** (SE) of a weight measures how much it would typically wobble from sample to sample. For least squares there's an exact formula:

$$\mathrm{Var}(\hat{\mathbf{w}}) = \sigma^2 \, (X^\top X)^{-1}, \qquad \hat{\sigma}^2 = \frac{\mathrm{SS}_{\mathrm{res}}}{n - d - 1}$$

Here X includes the column of ones, d is the number of features, and we divide by $n - d - 1$ (not n) because $d + 1$ weights were fitted from the same data. Each weight's SE is the square root of the matching diagonal entry. From the SE you get three standard results:

- The **t-statistic** $t = \hat{w} / \mathrm{SE}$: how many standard errors the weight is from zero.
- The **p-value**: if the feature truly had no effect (its weight is really 0), the chance of getting a $|t|$ at least this big by luck. It comes from a t-distribution with $n - d - 1$ degrees of freedom.
- A **95% confidence interval** $\hat{w} \pm t^* \, \mathrm{SE}$, where $t^* \approx 1.97$ here (it approaches 1.96 for large n).

Here it is from scratch on the diabetes data from section 11 (442 patients, 10 features). Statistics packages print the same table: with the `statsmodels` library it's `sm.OLS(y, sm.add_constant(X)).fit().summary()`.

```
import numpy as np
from scipy import stats
from sklearn.datasets import load_diabetes

def ols(X, y):
    """Least squares plus standard errors, t, p-values and 95% intervals."""
    n, d = X.shape
    A = np.column_stack([np.ones(n), X])           # add the bias column
    w = np.linalg.solve(A.T @ A, A.T @ y)          # the usual weights
    resid = y - A @ w
    dof = n - d - 1                                # data points minus parameters
    sigma2 = resid @ resid / dof                   # estimated noise variance
    se = np.sqrt(np.diag(sigma2 * np.linalg.inv(A.T @ A)))
    t = w / se                                     # how many SEs away from zero
    p = 2 * stats.t.sf(np.abs(t), dof)             # two-sided p-values
    tc = stats.t.ppf(0.975, dof)                   # 95% critical value
    return w, se, t, p, w - tc * se, w + tc * se

data = load_diabetes()
X, y, names = data.data, data.target, data.feature_names
w, se, t, p, lo, hi = ols(X, y)
print(f"{'':6}{'weight':>8}{'SE':>7}{'t':>7}{'p':>7}   95% interval")
for row in zip(["bias"] + names, w, se, t, p, lo, hi):
    print("{:6}{:8.1f}{:7.1f}{:7.2f}{:7.3f}   [{:7.1f}, {:7.1f}]".format(*row))
```

Output:

```
        weight     SE      t      p   95% interval
bias     152.1    2.6  59.06  0.000   [  147.1,   157.2]
age      -10.0   59.7  -0.17  0.867   [ -127.4,   107.4]
sex     -239.8   61.2  -3.92  0.000   [ -360.1,  -119.5]
bmi      519.8   66.5   7.81  0.000   [  389.1,   650.6]
bp       324.4   65.4   4.96  0.000   [  195.8,   453.0]
s1      -792.2  416.7  -1.90  0.058   [-1611.2,    26.8]
s2       476.7  339.0   1.41  0.160   [ -189.6,  1143.1]
s3       101.0  212.5   0.48  0.635   [ -316.7,   518.8]
s4       177.1  161.5   1.10  0.273   [ -140.3,   494.4]
s5       751.3  171.9   4.37  0.000   [  413.4,  1089.1]
s6        67.6   66.0   1.02  0.306   [  -62.1,   197.3]
```

How to read it: **bmi** is 7.81 standard errors from zero, with an interval from 389 to 651 that is nowhere near zero, so its effect is clearly real. **age** has $p = 0.867$: its weight of -10 is tiny compared with its uncertainty, and the interval runs from -127 to $+107$, so the data can't even tell the direction. **s1** is the interesting one: a large weight (-792) and yet $p = 0.058$, because its SE (416.7) is more than six times bigger than those of age, sex, bmi and bp. The next subsection explains why.

! THREE THINGS A P-VALUE DOES NOT MEAN

- $p = 0.058$ is not "a 5.8% chance the feature is useless". It's the chance of data this extreme *if* the feature were useless.
- A large p-value does not prove there's no effect. Often there just isn't enough data, or the feature overlaps with another one.
- A tiny p-value does not mean the effect is large, useful or causal. With a million rows, trivial effects get $p < 0.001$. Look at the size of the weight and the interval too.

When features overlap: multicollinearity and the VIF

In the diabetes data, **s1** (total cholesterol) and **s2** (LDL, the main part of total cholesterol) move together: their correlation is about 0.9. The model can predict well using either one, so it struggles to decide how to split the credit between them. Small changes in the data swing the weight from one to the other, and that shows up as huge standard errors.

The **variance inflation factor** measures this for each feature j. Predict feature j from all the other features, note that regression's R_j^2, and compute

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2}$$

If the other features explain none of feature j, $R_j^2 = 0$ and $\mathrm{VIF} = 1$ (no inflation). If they explain 90% of it, $\mathrm{VIF} = 10$. The name comes from what it does: the *variance* of $\hat{w}_j$ is multiplied by the VIF, so its SE is multiplied by $\sqrt{\mathrm{VIF}}$. A common rule of thumb is to investigate anything above 5–10.

```
from sklearn.linear_model import LinearRegression

# VIF of feature j = 1 / (1 - R²) when feature j is predicted from all the other
features
for j, name in enumerate(names):
    others = np.delete(X, j, axis=1)
    r2_j = LinearRegression().fit(others, X[:, j]).score(others, X[:, j])
    print(f"{name:4} VIF {1 / (1 - r2_j):5.1f}")

# Drop s2 and refit: s1's standard error shrinks, the fit barely changes
keep = [j for j, name in enumerate(names) if name ≠ "s2"]
w2, se2, *_ = ols(X[:, keep], y)
print("s1 with s2:    weight", round(w[5], 1), " SE", round(se[5], 1))
print("s1 without s2: weight", round(w2[5], 1), " SE", round(se2[5], 1))
print("R² with s2:   ", round(LinearRegression().fit(X, y).score(X, y), 4))
print("R² without s2:", round(LinearRegression().fit(X[:, keep], y).score(X[:, keep],
y), 4))
```

Output:

```
age  VIF   1.2
sex  VIF   1.3
bmi  VIF   1.5
bp   VIF   1.5
s1   VIF  59.2
s2   VIF  39.2
s3   VIF  15.4
s4   VIF   8.9
s5   VIF  10.1
s6   VIF   1.5
s1 with s2:    weight -792.2  SE 416.7
s1 without s2: weight -228.5  SE 114.0
R² with s2:    0.5177
R² without s2: 0.5155
```

s1's VIF of 59.2 means its standard error is $\sqrt{59.2} \approx 7.7$ times larger than it would be if s1 were unrelated to the other features. Drop s2 and s1's SE falls from 416.7 to 114.0. Yet the model's R^2 barely moves: 0.5177 with s2, 0.5155 without.

> **THE KEY DISTINCTION**
>
> Multicollinearity hurts **interpretation**, not **prediction**. If you only need accurate predictions, correlated features are mostly harmless. If you need to say what each feature does, deal with it: drop or merge overlapping features, collect more varied data, or use ridge regression (Lesson 10), which shares the credit smoothly instead of swinging it.

Adjusted R^2, and why cross-validation is better still

Training R^2 can never go down when you add a feature, even a column of pure random noise. The least-squares fit can always set the new weight to zero, and usually it finds a tiny bit of accidental pattern to exploit. **Adjusted** R^2 adds a penalty for each feature:

$$R^2_{\text{adj}} = 1 - (1 - R^2)\,\frac{n-1}{n-d-1}$$

Each extra feature makes $\frac{n-1}{n-d-1}$ bigger, so a feature has to improve R^2 by more than chance alone would to raise adjusted R^2. Watch what happens when we add columns of random noise to the diabetes features:

```
from sklearn.model_selection import cross_val_score

n = len(y)
rng = np.random.default_rng(0)
print("noise cols  train R²  adjusted R²  5-fold CV R²")
for extra in [0, 10, 50, 100]:
    Xe = np.column_stack([X, rng.normal(size=(n, extra))])     # add columns of pure
noise
    d = Xe.shape[1]
    r2 = LinearRegression().fit(Xe, y).score(Xe, y)
    adj = 1 - (1 - r2) * (n - 1) / (n - d - 1)
    cv = cross_val_score(LinearRegression(), Xe, y, cv=5, scoring="r2").mean()
    print(f"{extra:10d}  {r2:8.3f}  {adj:11.3f}  {cv:12.3f}")
```

Output:

```
noise cols  train R²  adjusted R²  5-fold CV R²
         0     0.518        0.507         0.482
        10     0.530        0.508         0.469
        50     0.582        0.516         0.398
       100     0.633        0.511         0.277
```

Training R^2 climbs from 0.518 to 0.633 on pure noise. Adjusted R^2 stays roughly flat around 0.51, correctly reporting that nothing was gained. The cross-validated R^2 tells the whole truth: performance on unseen patients actually *falls*, from 0.482 to 0.277, because the model is fitting noise. Adjusted R^2 is a quick check from the training data alone; in ML practice, cross-validation (Lesson 9) is the more reliable test.

When errors fan out: unequal spread and the log transform

Many money-like targets (salaries, prices, sales) vary in proportion to their size. A ₹4-lakh junior salary might be off by ₹1–2 lakh; a ₹20-lakh senior salary might be off by ₹5 lakh. The residuals fan out, breaking the "equal spread" assumption. Here's simulated salary data built that way:

```
# Salaries whose spread grows with experience (simulated, in lakh per year)
rng = np.random.default_rng(2)
years = rng.uniform(0, 30, size=300)
salary = 4 * np.exp(0.06 * years) * rng.lognormal(0, 0.25, size=300)

def spread(x, target):
    """Fit a line, then compare residual spread for the lowest and highest third of
x."""
    m = LinearRegression().fit(x[:, None], target)
    r = target - m.predict(x[:, None])
    low, high = x < np.quantile(x, 1/3), x > np.quantile(x, 2/3)
    return m, r[low].std(), r[high].std()

_, a, b = spread(years, salary)
print(f"raw salary: spread {a:.2f} (junior) vs {b:.2f} (senior)")
m_log, a, b = spread(years, np.log(salary))
print(f"log salary: spread {a:.3f} (junior) vs {b:.3f} (senior)")
k = m_log.coef_[0]
print(f"weight {k:.4f} → each extra year multiplies salary by {np.exp(k):.3f}")
```

Output:

```
raw salary: spread 1.53 (junior) vs 5.20 (senior)
log salary: spread 0.220 (junior) vs 0.249 (senior)
weight 0.0591 → each extra year multiplies salary by 1.061
```

On the raw scale the residuals of senior staff are 3.4 times more spread out than those of juniors. After taking $\log(\text{salary})$, the spread is nearly equal. The log turns "plus or minus 25%" into a constant "plus or minus 0.25".

Reading the weights of a log model. If $\log y = wx + b$, then increasing x by 1 multiplies y by e^w. Here $e^{0.0591} = 1.061$: each extra year of experience is linked to a salary about **6.1% higher**. For small w, $e^w \approx 1 + w$, so you can read $100w$ as a percentage straight away.

Two practical notes. First, converting predictions back with $e^{\hat{y}}$ gives roughly the *median*, which slightly underestimates the average. Second, if you can't transform the target, the other standard fixes are **robust standard errors** (in statsmodels, `fit(cov_type="HC3")`), which keep the weights but correct their SEs, and **weighted least squares**, which gives noisier points less weight.

Points that bend the line: leverage, Cook's distance and robust regression

Not every unusual point is dangerous. Two different things can make a point stand out:

- **Outlier**: its y is far from the line (a large residual).

- **High leverage**: its x is far from the other x values, so the line has to pass close to it. Leverage h_{ii} is the i-th diagonal entry of the *hat matrix* $H = X(X^\top X)^{-1}X^\top$, the matrix that turns $\mathbf{y}$ into predictions: $\hat{\mathbf{y}} = H\mathbf{y}$. The leverages always average $(d+1)/n$.

A point is **influential** when it is both: far out in x *and* off the line. **Cook's distance** combines the two into one number, which measures how much all the predictions would move if that point were deleted:

$$D_i = \frac{e_i^2}{(d+1)\,\hat{\sigma}^2} \cdot \frac{h_{ii}}{(1-h_{ii})^2}$$

Common rules of thumb flag $D_i > 1$, or $D_i > 4/n$ for a more sensitive check. In the example below, 20 points follow $y = 2x + 5$ closely, and one extra point sits far to the right, well below the line:

```
from sklearn.linear_model import HuberRegressor

rng = np.random.default_rng(1)
x = np.arange(1, 21, dtype=float)
y_ = 2 * x + 5 + rng.normal(0, 2, size=20)         # true line: y = 2x + 5
x = np.append(x, 40.0); y_ = np.append(y_, 20.0)   # one unusual point far to the right

A = np.column_stack([np.ones_like(x), x])
lev = np.diag(A @ np.linalg.inv(A.T @ A) @ A.T)    # leverage = diagonal of the hat
matrix
w_ = np.linalg.solve(A.T @ A, A.T @ y_)
e = y_ - A @ w_
p_ = A.shape[1]
s2 = e @ e / (len(y_) - p_)
cooks = e**2 / (p_ * s2) * lev / (1 - lev) ** 2     # Cook's distance

print("slope, least squares with the point:   ", round(w_[1], 2))
print("slope, least squares without the point:", round(LinearRegression().fit(x[:-1,
None], y_[:-1]).coef_[0], 2))
print("slope, Huber with the point:           ", round(HuberRegressor().fit(x[:, None],
y_).coef_[0], 2))
print("leverage: median", round(np.median(lev), 3), "| unusual point", round(lev[-1],
3))
print("Cook's D: largest other", round(cooks[:-1].max(), 3), "| unusual point",
round(cooks[-1], 2))
```

Output:

```
slope, least squares with the point:    0.75
slope, least squares without the point: 1.93
slope, Huber with the point:            1.85
leverage: median 0.065 | unusual point 0.576
Cook's D: largest other 0.099 | unusual point 12.73
```

That single point drags the slope from 1.93 down to 0.75. Its leverage (0.576) is nearly nine times the median, and its Cook's distance (12.73) is far beyond 1, while no other point exceeds 0.1. The diagnostics find it straight away.

What to do with a flagged point: first check it's not a data-entry mistake (a typo in units is the classic cause). If it's genuine, report results with and without it. Or use **robust regression**. The **Huber loss** treats small residuals with squared error and large ones with absolute error, so a single wild point can't dominate. Huber's slope (1.85) stays close to the true 2 even with the point included. Scikit-learn's `HuberRegressor` switches from squared to absolute error at 1.35 times its estimate of the noise scale by default.

Categorical features and the dummy-variable trap

To use a category such as the area of a Chennai home (Central, Suburb or Outskirts), you turn it into 0/1 **dummy** columns (one-hot encoding, Lesson 20). But with a bias term, you must leave **one** category out. Every row has exactly one of the three dummies equal to 1, so the three dummy columns always add up to the column of ones. One column is then an exact combination of the others, $X^\top X$ can't be inverted, and there's no unique answer. This is the **dummy-variable trap**.

```
import pandas as pd

df = pd.read_csv("houses.csv")
all3 = pd.get_dummies(df["area"], dtype=float)          # Central, Outskirts, Suburb
A = np.column_stack([np.ones(len(df)), df["size_m2"], all3])
print("columns:", A.shape[1], " independent columns (rank):", np.linalg.matrix_rank(A))

Xd = pd.get_dummies(df[["size_m2", "area"]], columns=["area"], drop_first=True,
dtype=float)
m = LinearRegression().fit(Xd, df["price_lakh"])
for name, wt in zip(Xd.columns, m.coef_):
    print(f"{name:15}{wt:7.2f}")
print(f"{'bias':15}{m.intercept_:7.2f}")
```

Output:

```
columns: 5  independent columns (rank): 4
size_m2           0.61
area_Outskirts  -41.13
area_Suburb     -14.79
bias             22.98
```

With all three dummies there are 5 columns but only 4 independent ones (the rank), so the normal equation has no unique solution. Dropping one category with `drop_first=True` fixes it. The dropped category, Central, becomes the **baseline**, and every dummy weight is a difference from it. For homes of the same size, the model says an Outskirts home sells for about 41.1 lakh less than a Central one, and a Suburb home about 14.8 lakh less. (Only 12 houses, so treat these as an illustration, not a market report.)

Scikit-learn's `LinearRegression` won't crash if you keep all the dummies, because it uses a solver that copes with dependent columns. But the weights are then not unique and can't be interpreted. With ridge or lasso the penalty makes the answer unique, so keeping all categories is normal practice there.

A checklist before you report a linear model

Check	Warning sign	Usual fix
Residuals vs predictions	A curve	Add features (x^2, $\log x$, interactions)
Residual spread	Fans out	Log the target, or use robust SEs
VIF of each feature	Above 5–10	Drop or merge features, or use ridge
Cook's distance	Any $D_i > 1$	Check the data; compare with and without; Huber
Weights and intervals	Interval crosses zero	Don't claim a direction for that feature
Train vs cross-validated R^2	Big gap	Fewer features or regularization (Lesson 10)

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **What are the assumptions of linear regression, and which of them do you need just to get good predictions?**

The trap: Reciting the list without saying what each one is for.

Linearity, independent errors, normally distributed errors and equal error spread (LINE). For predictions checked on a test set, none is strictly required, although a badly wrong linearity assumption will show up as poor test error. The assumptions matter for *inference*: trusting standard errors, p-values and confidence intervals. Independence and equal spread affect the SEs most; normality matters only in small samples.

▼ **A feature's p-value is 0.4. Should you remove it from the model?**

The trap: "Yes — it isn't significant."

Not automatically. A large p-value means the data can't pin the effect down, not that the effect is zero. It may overlap with another feature (check its VIF), the sample may be small, or domain knowledge may say it belongs. For prediction, decide with cross-validation; for explanation, report its confidence interval honestly instead of deleting it quietly.

▼ **Two features are 95% correlated. What happens to the model's predictions, and to its weights?**

The trap: Saying the model becomes inaccurate.

Predictions are usually fine: the model only needs the shared information, and either feature carries it. The individual weights become unstable, with huge standard errors (high VIF), and can even flip sign between samples, so you can't interpret them. Fixes: drop or merge one feature, or use ridge regression, which splits the credit smoothly.

▼ **Why do we minimise squared error rather than, say, absolute error?**

The trap: "Because squares are easier to differentiate."

That's part of it (it gives a closed-form solution), but the deeper reasons are statistical. If the noise is normal, least squares is the maximum-likelihood estimate. By the Gauss–Markov theorem, it is also the best linear unbiased estimator whenever the errors are uncorrelated with equal spread. Absolute error is the maximum-likelihood choice for heavier-tailed (Laplace) noise and is more robust to outliers; Huber loss is a compromise.

▼ **What's the difference between an outlier, a high-leverage point and an influential point?**

The trap: Using the three words as if they meant the same thing.

An outlier has an unusual y (large residual). A high-leverage point has unusual x values (large diagonal entry of the hat matrix), so the line is forced to pass near it. An influential point actually changes the fit a lot when removed. That usually takes both: high leverage *and* off the line. Cook's distance measures influence directly.

▼ **You one-hot encode a 3-level category, keep all 3 columns, and fit ordinary least squares with an intercept. What goes wrong?**

The trap: Expecting an error message every time.

The 3 dummies always sum to the intercept column, so $X^\top X$ is singular: the dummy-variable trap. The weights have no unique solution. Some solvers (including scikit-learn's) still return an answer, but the individual weights mean nothing. Drop one level (it becomes the baseline), or use regularization, which makes the solution unique.

Check your understanding: going deeper

1. A feature has $\mathrm{VIF} = 10$. Compared with a feature unrelated to the others, its standard error is multiplied by about...

a. $\sqrt{10} \approx 3.2$

b. 10

c. 100

2. Which of these is not an assumption of linear regression?

a. The errors have about equal spread

b. The features are normally distributed

c. The errors are independent

3. A model predicts $\log(\text{price})$, and the weight on bedrooms is 0.10. One extra bedroom is linked to a price that is...

a. ₹0.10 lakh higher

b. about 10.5% higher

c. 10 times higher

4. A point has very high leverage but lies exactly on the line through the other points. Its Cook's distance is...

a. Large: high leverage is always dangerous

b. Close to zero: its residual is tiny

c. Undefined

5. A category has 4 values and the model has a bias term. How many dummy columns should ordinary least squares get?

a. 4

b. 3

c. 1

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Fit a least-squares line to $x = 1, 2, 3$ and $y = 2, 3, 5$. Predict y at $x = 4$.

WORKING

EXERCISE 2 By hand

For the line from exercise 1, compute R^2.

WORKING

EXERCISE 3 Concepts

A model predicts salary (in thousands) as $12 + 3.5 \times \text{years of experience}$. Interpret both numbers. Is it safe to say "experience causes salary"?

WORKING

EXERCISE 4 Code

Solve exercise 1 with the normal equation in NumPy.

WORKING

EXERCISE 5 Intermediate

A model with $d = 4$ features is fitted to $n = 50$ points and has training $R^2 = 0.80$. Compute the adjusted R^2.

WORKING

EXERCISE 6 Intermediate

Feature j can be predicted from the other features with $R_j^2 = 0.9$. What is its VIF, and by what factor is its standard error inflated?

WORKING

EXERCISE 7 Intermediate

A weight is $\hat{w} = 4.0$ with standard error 1.5, from $n = 200$ points and $d = 3$ features. Compute its t-statistic and its 95% confidence interval, using $t^* = 1.97$.

WORKING

EXERCISE 8 Intermediate

A model predicts $\log(\text{price})$ and the weight on *number of bathrooms* is 0.05. By what factor does one extra bathroom multiply the predicted price, and what percentage increase is that?

WORKING

EXERCISE 9 Code · intermediate

Using the diabetes model from the "Going deeper" code, compute Cook's distance for all 442 patients. What is the largest value, and how many patients are above the sensitive threshold $4/n$?

WORKING

NOTES

NOTES

PART 2 · LEARNING FROM LABELLED DATA · ~50 MIN

Lesson 7: Gradient Descent

Train any model by walking downhill: learning rates, batch vs stochastic, and convergence.

BY THE END OF THIS LESSON YOU CAN

- Explain the gradient descent update rule and why it works
- Derive the gradient of the MSE loss and run steps by hand
- Choose a learning rate and recognise divergence
- Compare batch, stochastic and mini-batch gradient descent

↺ BEFORE YOU START

This lesson builds on:

- Lesson 3: Derivatives & Gradients — A derivative is a slope; the gradient ∇L collects all partial derivatives and points uphill. Gradient descent steps the other way: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$. The chain rule differentiates functions inside functions.
- Lesson 6: Linear Regression — Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ and chooses the weights that minimise the mean squared error (least squares). R^2 measures the share of variance explained.

Where this fits: Lesson 6 had a formula for the best weights; most models don't. Gradient descent is the general training method used from here to neural networks.

1. Why we need gradient descent

In Lesson 6, linear regression had a formula for the best weights: set the derivatives to zero and solve. That's a lucky special case. For almost every other model — logistic regression, neural networks, most of modern ML — no such formula exists. And even for linear regression, the formula becomes too slow with many features.

Gradient descent is the general-purpose answer. It doesn't need a formula for the minimum; it only needs to know which way is downhill. It is the optimisation method behind almost every model you'll meet from here on, including the largest neural networks.

READ THE FORMULA **The gradient descent update**

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$$

- **w (left)** — the new parameters
- **w (right)** — the current parameters
- **η** — learning rate — how big a step
- **∇L(w)** — gradient — points uphill, so we subtract it

Hover over (or tap) a part to highlight it in the formula.

2. The idea: a hiker in the fog

◎ ANALOGY

You're on a mountain in thick fog and want to reach the valley. You can't see the bottom, but you can feel the slope under your feet. Strategy: feel which way is steepest downhill, take a step that way, and repeat. Eventually you reach the bottom.

In ML: your position is the model's **parameters**, your altitude is the **loss**, and "feeling the slope" is computing the **gradient** (Lesson 3).

Recall that the gradient ∇L is the vector of partial derivatives and points *uphill* — the direction in which the loss grows fastest. So we step the opposite way:

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})}$$

- $\mathbf{w}$ — all the parameters (weights and bias).
- $\nabla L(\mathbf{w})$ — the gradient of the loss at the current parameters.
- η ("eta") — the **learning rate**, a small positive number controlling the step size.
- $\leftarrow$ — "replace with". We repeat this update many times.

3. The algorithm, step by step

1. **Initialise** the parameters, e.g. all zeros or small random numbers.
2. **Predict** with the current parameters and compute the loss.
3. **Compute the gradient** of the loss with respect to every parameter.

4. **Update** every parameter: subtract learning rate × its partial derivative.
5. **Repeat** steps 2–4 until the loss stops improving (or for a fixed number of steps).

4. The gradient for linear regression

For the model $\hat{y}_i = w x_i + b$ and loss $L = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$, apply the chain rule to each term. The derivative of $(\hat{y}_i - y_i)^2$ is $2(\hat{y}_i - y_i)$ times the derivative of $\hat{y}_i$, which is x_i (for w) or 1 (for b):

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_{i=1}^{n} (\hat{y}_i - y_i)\, x_i \qquad\qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n} (\hat{y}_i - y_i)$$

Read it in words: *each example votes on how to move the parameter, in proportion to its error.* If predictions are too high on average ($\hat{y} > y$), the gradient for b is positive and the update lowers b. Examples with larger x_i get more say in how w moves.

With many features, in matrix form (using a column of ones for the bias):

$$\nabla L(\mathbf{w}) = \frac{2}{n} X^\top (X\mathbf{w} - \mathbf{y})$$

STEP-BY-STEP DERIVATION **Derive the MSE gradient step by step**

1 $$L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2, \qquad \hat{y}_i = wx_i + b$$

The loss and the model.

2 $$\frac{\partial L}{\partial w} = \frac{1}{n}\sum_{i=1}^{n}\frac{\partial}{\partial w}(\hat{y}_i - y_i)^2$$

The derivative of a sum is the sum of the derivatives.

3 $$\frac{\partial}{\partial w}(\hat{y}_i - y_i)^2 = 2(\hat{y}_i - y_i)\cdot\frac{\partial \hat{y}_i}{\partial w}$$

Chain rule: outside u^2, inside $u = \hat{y}_i - y_i$.

4 $$\frac{\partial \hat{y}_i}{\partial w} = x_i, \qquad \frac{\partial \hat{y}_i}{\partial b} = 1$$

Differentiate $wx_i + b$ with respect to each parameter.

5 $$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\,x_i, \qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)$$

Put the pieces together.

5. Worked example: three steps by hand

✎ WORKED EXAMPLE — SAME DATA AS LESSON 6

Data: $x = 1, 2, 3, 4, 5$, $y = 2, 4, 5, 4, 5$. We know the answer should be $w = 0.6$, $b = 2.2$. Start from $w = 0$, $b = 0$ with learning rate $\eta = 0.05$.

Step 1. Predictions are all 0, so the errors $\hat{y}_i - y_i$ are $-2, -4, -5, -4, -5$. Loss $= (4 + 16 + 25 + 16 + 25)/5 = 17.2$.

$$\frac{\partial L}{\partial w} = \frac{2}{5}\big((-2)(1) + (-4)(2) + (-5)(3) + (-4)(4) + (-5)(5)\big) = \frac{2}{5}(-66) = -26.4$$

$$\frac{\partial L}{\partial b} = \frac{2}{5}(-2 - 4 - 5 - 4 - 5) = \frac{2}{5}(-20) = -8$$

Update: $w = 0 - 0.05 \times (-26.4) = 1.32$, $b = 0 - 0.05 \times (-8) = 0.4$.

Step 2. New predictions $1.72, 3.04, 4.36, 5.68, 7.00$. Loss drops to 1.646. Gradients: $\frac{\partial L}{\partial w} = 5.04$, $\frac{\partial L}{\partial b} = 0.72$ (now positive: we overshot a little). Update: $w = 1.32 - 0.252 = 1.068$, $b = 0.4 - 0.036 = 0.364$.

Step 3. Loss 1.105. And so on.

Steps	*w*	*b*	Loss (MSE)
0	0	0	17.2
1	1.32	0.40	1.646
10	1.041	0.607	0.942
100	0.695	1.857	0.501
1000	0.600	2.200	0.480

The loss falls fast at first, then slowly creeps to the minimum of 0.48 — exactly the least-squares answer from Lesson 6.

? MAKE SENSE OF IT

The situation: Same five students as Lesson 6, but now the computer finds the line by trial and improvement instead of by formula.

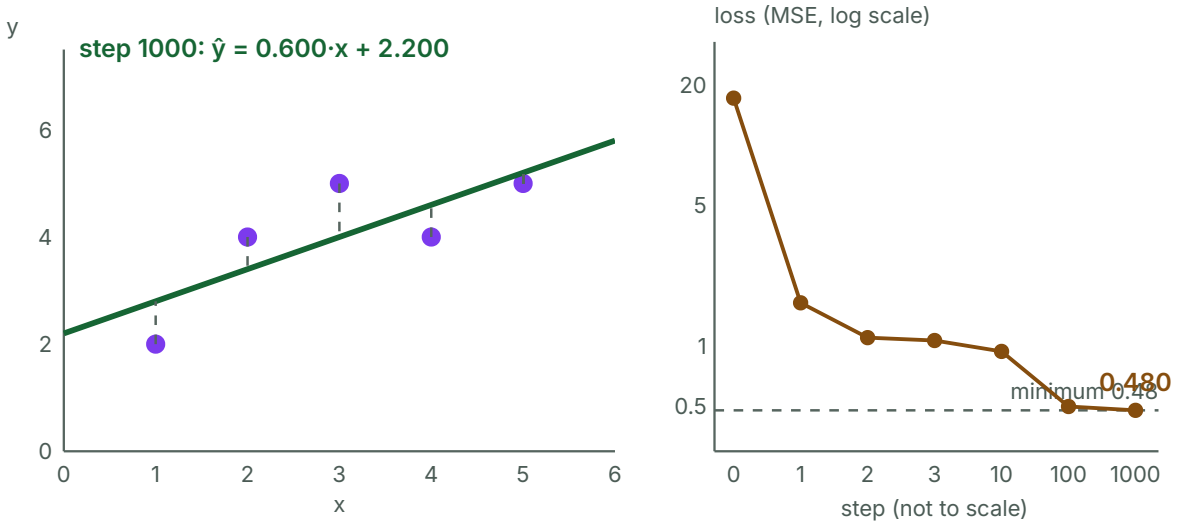
What the result tells you: Each step moves w and b a little in the direction that lowers the loss. After 1,000 steps it lands on $w = 0.6,\ b = 2.2$ — the exact least-squares answer.

What if...?

- **Smaller learning rate** ($\eta = 0.01$): after 1,000 steps it's still at $w \approx 0.618,\ b \approx 2.136$; it needs about 10,000 steps. Safe but slow.
- **Larger learning rate** ($\eta = 0.1$): it *diverges* — after 10 steps the loss is over 8,000 and still growing. On this data any η above about 0.085 blows up.
- **Unscaled features** ($x \times 10$) with the original $\eta = 0.05$: the loss explodes within 5 steps. Bigger feature values mean steeper slopes, so the same step size overshoots.

Why it matters in ML: The learning rate is the most important setting in training. Too small wastes time; too big never converges. Scaling features (Lesson 20) makes one learning rate work for every weight.

WALKTHROUGH (FINAL STEP SHOWN) **Gradient descent, step by step**



1. Start with $w = 0$, $b = 0$: the line lies flat on the x-axis and the loss is huge (17.2).
2. One step: both gradients are negative, so both parameters increase: $w = 1.32$, $b = 0.4$. Loss 1.646 — a giant improvement, but the slope is now too steep.
3. Step 2: the gradient for w is now positive (we overshot), so w comes back down to 1.068. Loss 1.105.
4. Step 3: loss 1.067. The big gains are already over; from here progress is slow and steady.
5. After 10 steps the slope is close-ish ($w = 1.041$) but the intercept is still far too small ($b = 0.607$ vs 2.2). Loss 0.942.
6. After 100 steps: $w = 0.695$, $b = 1.857$, loss 0.501. The intercept creeps up slowly — the two parameters want very different step sizes.
7. After 1000 steps: $w = 0.600$, $b = 2.200$, loss 0.48 — exactly the least-squares answer from Lesson 6.

6. Try it: gradient descent on a loss landscape

The left panel is a **contour map** of the loss: each ring joins parameter values (w, b) with the same loss, like height lines on a hiking map. The star marks the minimum. The right panel shows the current line through the data. Press *Step* or *Run* and watch the parameters walk downhill.

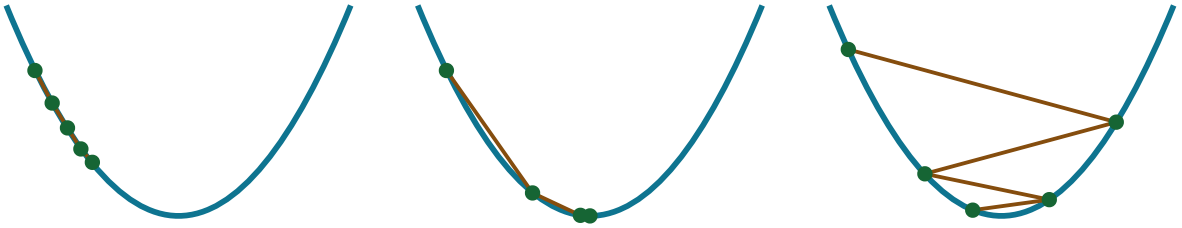
Things to try: (1) set the learning rate above about 0.085 and watch it **diverge**; (2) set it very low and watch it crawl; (3) tick *standardise x* and see the long, narrow valley become round — and descent become much faster.

Interactive demo — open *lesson-07.html* on the website to try it.

7. Choosing the learning rate

PICTURE IT Walking down a dark hill, you can take tiny shuffles (safe, slow) or giant leaps (fast, but you might jump straight over the bottom onto the opposite slope). The learning rate is your stride length.

The learning rate η is the most important setting in gradient descent.



too small: slow | just right | too large: diverges

Figure 7. The same loss bowl with three learning rates: tiny steps crawl, good steps converge, and steps that are too big overshoot further each time.

- **Too small:** each step is tiny, so training takes forever.
- **Too large:** steps overshoot the valley, bouncing from side to side. If it's large enough, each bounce goes higher than the last and the loss **explodes** (divergence).
- **Just right:** fast, steady progress to the minimum.

✎ WORKED EXAMPLE — OVERSHOOTING ON $L(W) = W^2$

The gradient is $2w$, so the update is $w \leftarrow w - \eta \cdot 2w = (1 - 2\eta)\, w$. Start at $w = 1$:

- $\eta = 0.1$: multiply by 0.8 each step → $0.8, 0.64, 0.51, \ldots$ smooth convergence.
- $\eta = 0.9$: multiply by -0.8 → $-0.8, 0.64, -0.51, \ldots$ bouncing across the valley, but still shrinking.
- $\eta = 1.1$: multiply by -1.2 → $-1.2, 1.44, -1.73, \ldots$ growing: **divergence**.

The stable limit here is $\eta < 1$. Steeper bowls have smaller limits.

? MAKE SENSE OF IT

The situation: A ball rolling into a valley. Each step you move it by the learning rate times the slope.

What the result tells you: With $\eta = 0.1$ you keep 80% of the distance each step; with $\eta = 0.9$ you jump past the bottom but still get closer; with $\eta = 1.1$ each jump is bigger than the last.

What if…?

- **A steeper valley** $L = 5w^2$ (gradient $10w$, update factor $1 - 10\eta$): now the limit is $\eta < 0.2$. With $\eta = 0.15$ the values go $-0.5, 0.25, -0.125$ (bouncing but shrinking); with $\eta = 0.25$ they go $-1.5, 2.25, -3.375$ (diverging).
- **The lucky case** $\eta = 0.1$ on $5w^2$ gives factor 0: it lands on the minimum in one step.

Why it matters in ML: Real loss surfaces are steep in some directions and flat in others. A learning rate small enough for the steep direction crawls in the flat one — which is why adaptive optimisers like Adam use a different step size for each weight.

| PRACTICAL RECIPE

Try values spaced by factors of 10 (0.001, 0.01, 0.1, …) and plot loss vs step for each. Pick the largest one where the loss falls smoothly. Many libraries also *decrease* the learning rate during training — a **learning-rate schedule** — taking big steps early and small, careful steps near the minimum.

ADVANCED · OPTIONAL **When does gradient descent converge? The stability limit** ▾ hide

Take the simplest bowl, $L(w) = aw^2$ with $a > 0$. Its gradient is $2aw$, so one step is

$$w \leftarrow w - \eta\, 2aw = (1 - 2\eta a)\, w.$$

After t steps $w_t = (1 - 2\eta a)^t w_0$, which shrinks to 0 exactly when $|1 - 2\eta a| < 1$, i.e. $0 < \eta < 1/a$. At $\eta = 1/a$ it bounces forever; above, it diverges. In general the curvature is the second derivative ($2a$ here), and the rule is $\eta < 2/\text{curvature}$. For several weights, the curvature in the steepest direction is the largest eigenvalue $\lambda_{\max}$ of the Hessian, so $\eta < 2/\lambda_{\max}$. For MSE linear regression the Hessian is $\frac{2}{n}X^\top X$: features on very different scales make $\lambda_{\max}$ huge and force a tiny learning rate — the maths behind feature scaling.

8. Why feature scaling matters

If one feature ranges from 0 to 1 and another from 0 to 100,000, the loss landscape becomes a long, narrow valley: very steep in one direction, almost flat in the other. A learning rate small enough to avoid bouncing in the steep direction is far too small for the flat direction, so gradient descent zigzags slowly along the valley floor.

Standardising features (subtract the mean, divide by the standard deviation, Lesson 4) makes the valley rounder, so the gradient points more directly at the minimum. In the demo, our x values 1–5 aren't centred, which already creates a stretched valley; at $\eta = 0.05$, standardising x cuts the number of steps needed to get within 0.001 of the minimum loss from 190 to 47.

9. When to stop

- **Fixed budget** — run a set number of steps (or *epochs*: full passes over the data).
- **Tolerance** — stop when the loss improves by less than a tiny amount, or the gradient is nearly zero.
- **Early stopping** — watch the loss on a *validation* set (Lesson 5) and stop when it starts rising, which signals overfitting.

> ! LOCAL MINIMA
>
> For linear regression, the loss is a single bowl (it's **convex**): every downhill path leads to the same, global minimum. For neural networks the landscape has many hills and valleys, so gradient descent may settle in a **local minimum**. In practice, for large networks, these local minima are usually good enough — but it's why the starting point and learning rate matter.

10. Batch, stochastic and mini-batch

Computing the exact gradient uses every training example. With millions of examples that's expensive for a single step. Three variants trade accuracy of each step for speed:

Variant	Examples per update	Pros	Cons
Batch GD	All n	Exact gradient, smooth path	Slow per step on big data
Stochastic GD (SGD)	1 (random)	Very cheap steps; noise can help escape bad spots	Noisy, zigzag path
Mini-batch GD	A small batch, e.g. 32–256	Best of both; efficient on GPUs	One more setting (batch size)

The update has the same form; only the gradient is estimated from fewer examples. Mini-batch is the standard choice for neural networks. An **epoch** means one full pass over the training data, however many updates that takes.

◎ WHY CAN NOISY STEPS STILL WORK?

The gradient from a random sample points in the right direction *on average*. Like asking a few random people for directions instead of polling the whole town: each answer is a bit off, but you move much sooner, and the errors average out over many steps.

11. In Python

Gradient descent from scratch, exactly as in the worked example:

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2, 4, 5, 4, 5], dtype=float)

w, b = 0.0, 0.0
lr = 0.05
for step in range(1000):
    y_hat = w * x + b
    error = y_hat - y
    grad_w = 2 * np.mean(error * x)     # dL/dw
    grad_b = 2 * np.mean(error)         # dL/db
    w -= lr * grad_w
    b -= lr * grad_b
    if step in (0, 9, 99, 999):
        print(step + 1, round(w, 3), round(b, 3), round(np.mean((w * x + b - y) ** 2),
3))
```

Output (step, w, b, loss after the update):

```
1 1.32 0.4 1.646
10 1.041 0.607 0.942
100 0.695 1.857 0.501
1000 0.6 2.2 0.48
```

Mini-batch SGD on many features, with scikit-learn. `SGDRegressor` is sensitive to feature scales, so we standardise inside a pipeline:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

model = make_pipeline(StandardScaler(),
                      SGDRegressor(learning_rate="invscaling", eta0=0.01, max_iter=1000,
random_state=0))
model.fit(X_train, y_train)
print(round(model.score(X_test, y_test), 3))     # R² on the test set
```

Output: `0.456` — essentially the same R^2 as the exact solution in Lesson 6 (0.453), reached by walking downhill instead of solving equations.

12. Running example: Chennai Homes

Gradient descent on standardised size reaches the same line as the formula in Lesson 6 — slope 0.29 and intercept 41.11 — after converting back to m². Two methods, one answer.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
x = df["size_m2"].to_numpy(); y = df["price_lakh"].to_numpy()
xs = (x - x.mean()) / x.std()            # standardise so one learning rate suits
both parameters
w, b, lr = 0.0, 0.0, 0.1
for step in range(200):
    err = w * xs + b - y
    w -= lr * 2 * np.mean(err * xs)
    b -= lr * 2 * np.mean(err)
print("in standardised units: w =", round(w, 3), " b =", round(b, 3))
print("back in m2: slope =", round(w / x.std(), 3), " intercept =", round(b - w *
x.mean() / x.std(), 2))
```

Output:

```
in standardised units: w = 7.482  b = 72.083
back in m2: slope = 0.29  intercept = 41.11
```

⚙ WHERE THIS IS USED

- Training neural networks, logistic regression and linear models at scale
- Online learning: updating a model as new data streams in

Choose it when: Many parameters or large data, or no closed-form solution.

Think twice when: Small problems with an exact formula (e.g. small linear regressions).

✕ MISCONCEPTIONS TO UNLEARN

MYTH A bigger learning rate always trains faster.

REALITY Past a limit, steps overshoot and the loss grows (divergence).

MYTH Gradient descent finds the exact best answer.

REALITY It gets very close after enough steps, and for non-convex problems it may stop in a local minimum.

RESPONSIBLE AI · THE COST OF TRAINING

Training is computation, and computation uses energy. Strubell, Ganesh and McCallum (2019) estimated that training and tuning some large language models emitted as much CO_2 as several cars over their lifetimes. For small models the cost is trivial, but it's worth asking whether a bigger model or another hyperparameter search is really needed. Fixing random seeds also makes results reproducible and auditable.

13. Questions beginners often ask

Q. How do I choose the learning rate?

Try values spaced by factors of 10 (0.001, 0.01, 0.1…) and plot the loss curve. Pick the largest one that decreases smoothly.

Q. How do I know when to stop?

When the loss stops improving, or — better — when the validation loss starts rising (early stopping).

Q. Can gradient descent get stuck?

For convex losses like linear and logistic regression, no. For neural networks it can reach local minima or flat regions; in practice the results are usually still good.

! COMMON MISTAKES TO AVOID

- Forgetting to scale features, which forces a tiny learning rate and slow zigzagging.
- Concluding a model can't learn when the learning rate is simply too small or too large — plot the loss curve.
- Assuming gradient descent always finds the global minimum; that's only guaranteed for convex losses.

! INTERVIEW CORNER

Don't confuse

- **Epoch vs batch vs iteration** — Epoch = whole dataset; iteration = one batch.
- **Batch vs mini-batch vs stochastic gradient descent** — How many examples per step.
- **Learning rate vs momentum** — Step size vs inertia.

Hidden questions — answer in your head first, then open.

▼ **What's the difference between an epoch, a batch and an iteration?**

The trap: Using epoch and iteration interchangeably.

A batch is the set of examples for one update; an iteration is one update; an epoch is one pass over all training data. 1,000 examples with batch size 100 = 10 iterations per epoch.

▼ **Does gradient descent always find the global minimum?**

The trap: "Yes, if you run it long enough."

Only for convex losses (linear and logistic regression), with a suitable learning rate. Neural-network losses are non-convex, so it finds a good-enough minimum depending on initialisation — which works surprisingly well in practice.

▼ **Your loss becomes NaN after a few hundred steps. Name three likely causes.**

The trap: Blaming the data only.

A learning rate that's too high (divergence), exploding gradients (try gradient clipping), and numerical problems such as $\log(0)$ or dividing by zero (use the framework's built-in, numerically stable losses). Also check for NaNs in the inputs.

▼ **Why does feature scaling speed up gradient descent?**

The trap: "Scaling is only for distance-based models."

Unscaled features make the loss surface a long, narrow valley; a learning rate small enough for the steep direction crawls along the flat one. In Lesson 7, multiplying x by 10 made the original learning rate diverge within 5 steps.

14. Key takeaways

- Gradient descent repeats $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$: step against the gradient.
- For MSE: $\frac{\partial L}{\partial w} = \frac{2}{n}\sum(\hat{y}_i - y_i)x_i$ and $\frac{\partial L}{\partial b} = \frac{2}{n}\sum(\hat{y}_i - y_i)$.
- The **learning rate** must be small enough not to diverge, large enough to make progress.
- **Standardise features** so the loss landscape is round and descent is fast.
- **Mini-batch SGD** estimates the gradient from small random batches — the standard for large data and neural networks.

15. Check your understanding

1. Current $w = 2$, gradient $\partial L/\partial w = 4$, learning rate 0.1. What is the new w?

a. 2.4

b. 1.6

c. -2

2. Your loss gets bigger every step and eventually becomes "nan". The most likely cause?

a. Learning rate too small

b. Too much training data

c. Learning rate too large

3. Which variant uses a single random example per update?

a. Stochastic gradient descent

b. Batch gradient descent

c. The normal equation

4. Why does standardising features usually speed up gradient descent?

a. It reduces the number of parameters

b. It makes the loss landscape rounder, so the gradient points more directly at the minimum

c. It removes the need for a learning rate

5. All predictions are too high. What sign does $\partial L/\partial b$ have, and what happens to b?

a. Negative; b increases

b. Zero; b stays the same

c. Positive; b decreases

16. Going deeper: modern optimisers and learning-rate schedules

INTERMEDIATE · SKIP ON A FIRST READ

Plain gradient descent, as in this lesson, is rarely used as-is on large models. This section explains the improvements that nearly every deep-learning project uses: **momentum**, adaptive methods such as **Adam**, **learning-rate schedules**, and **gradient clipping**. You'll see when each helps, and when it doesn't.

You'll need: this lesson, and the Hessian and condition number from the "Going deeper" sections of Lessons 2 and 3.

Why plain gradient descent is slow: the condition number again

On a loss whose curvature differs a lot between directions, the learning rate must be small enough for the *steepest* direction (below $2/\lambda_{\max}$, the stability limit from this lesson), but then progress along the *gentlest* direction is slow. Near a minimum, each step shrinks the error in the slowest direction by roughly a factor of

$$1 - \frac{\lambda_{\min}}{\lambda_{\max}} = 1 - \frac{1}{\kappa}$$

where κ is the condition number. With $\kappa = 1000$ you need thousands of steps. Feature scaling (this lesson) attacks the cause by making the curvatures similar; the optimisers below work around it.

Momentum: a ball rolling downhill

Momentum keeps a running **velocity**, a decaying sum of past gradients, and steps along it:

$$\mathbf{v} \leftarrow \beta\,\mathbf{v} + \nabla L, \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta\,\mathbf{v}$$

In the steep direction, gradients flip sign every step and largely cancel, which damps the zig-zag. In the gentle direction they point the same way every time and add up, so the ball speeds up along the valley floor. Typical values are $\beta = 0.9$ for deep learning. **Nesterov momentum** is a small refinement that computes the gradient at the point where the velocity is about to take you.

Adaptive learning rates: RMSProp and Adam

Instead of one learning rate for all weights, **RMSProp** divides each weight's step by a running average of that weight's recent gradient sizes. Weights with large, frequent gradients take smaller steps; rarely updated ones take bigger steps. **Adam** ("adaptive moment estimation") combines this with momentum:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1-\beta_1)\,\mathbf{g}, \quad \mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1-\beta_2)\,\mathbf{g}^2, \quad \mathbf{w} \leftarrow \mathbf{w} - \eta\,\frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{s}}} + \epsilon}$$

The squares and the division are done per weight. The hats are a **bias correction**, $\hat{\mathbf{m}} = \mathbf{m}/(1-\beta_1^k)$ at step k, because $\mathbf{m}$ and $\mathbf{s}$ start at zero and would otherwise be too small at first. Defaults that work surprisingly often: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Here are all three methods on two valleys, one narrow and one very narrow, each stopping when within 0.001 of the minimum:

```
import numpy as np

def run(update, grad, start, steps=20000, tol=1e-3):
    """Steps until w is within tol of the minimum at (0, 0)."""
    w, state = np.array(start, dtype=float), {}
    for k in range(1, steps + 1):
        w = update(w, grad(w), state, k)
        if np.linalg.norm(w) < tol: return k
    return None

def plain(lr):
    return lambda w, g, s, k: w - lr * g

def momentum(lr, beta):
    def step(w, g, s, k):
        s["v"] = beta * s.get("v", 0) + g                 # a running "velocity"
        return w - lr * s["v"]
    return step

def adam(lr, b1=0.9, b2=0.999, eps=1e-8):
    def step(w, g, s, k):
        s["m"] = b1 * s.get("m", 0) + (1 - b1) * g         # average gradient
        s["v"] = b2 * s.get("v", 0) + (1 - b2) * g**2      # average squared gradient
        m_hat, v_hat = s["m"] / (1 - b1**k), s["v"] / (1 - b2**k)   # bias correction
        return w - lr * m_hat / (np.sqrt(v_hat) + eps)
    return step

valleys = {"narrow (curvatures 2 and 20)":     (lambda w: np.array([2 * w[0], 20 *
w[1]]), 20),
           "very narrow (curvatures 2 and 2000)": (lambda w: np.array([2 * w[0], 2000 *
w[1]]), 2000)}
for name, (g, top) in valleys.items():
    lr = 1.8 / top                                        # just under the stability
limit 2 / top
    print(name)
    print(f"  plain gradient descent: {run(plain(lr), g, [10, 1])} steps")
    print(f"  momentum (beta 0.3):    {run(momentum(lr * 1.2, 0.3), g, [10, 1])} steps")
    print(f"  Adam (lr 0.1):          {run(adam(0.1), g, [10, 1])} steps")
```

Output:

```
narrow (curvatures 2 and 20)
  plain gradient descent: 47 steps
  momentum (beta 0.3):    17 steps
  Adam (lr 0.1):          310 steps
very narrow (curvatures 2 and 2000)
  plain gradient descent: 5113 steps
  momentum (beta 0.3):    2977 steps
  Adam (lr 0.1):          310 steps
```

Two lessons in one table. On the mildly narrow valley, **momentum** is fastest (17 steps against 47), and Adam is slowest (310). Adam takes steps of roughly fixed size η, so it crawls the final stretch unless its learning rate is decayed. But when the valley is 1,000 times steeper one way than the other, plain gradient descent needs 5,113 steps and momentum 2,977, while Adam still needs 310. Because it rescales every direction by its own gradient size, Adam hardly notices bad conditioning. That robustness, across very differently scaled weights, is why Adam is the default for deep networks. For convex problems with well-scaled features, simpler methods (or L-BFGS) are often better.

> **ADAM VS ADAMW**
>
> Adding an L2 penalty to the loss interacts badly with Adam's per-weight scaling: heavily updated weights end up barely regularised. **AdamW** applies weight decay directly to the weights instead, $\mathbf{w} \leftarrow \mathbf{w} - \eta\lambda\mathbf{w}$, separately from the adaptive step. It is the standard choice for training transformers.

Learning-rate schedules

With **stochastic** gradients, a constant learning rate never settles: each mini-batch's gradient is noisy, so the weights keep wobbling around the minimum. The classic theory (Robbins–Monro) says SGD converges if the learning rates add up to infinity (so you can travel anywhere) but their squares add up to a finite number (so the noise dies down). A rate like $\eta_t = \eta_0/(1 + t/T)$ does both.

```
from sklearn.datasets import load_diabetes

# Mini-batch SGD on the diabetes data: constant learning rate vs a decaying schedule
Xd, yd = load_diabetes(return_X_y=True)
Xd = (Xd - Xd.mean(0)) / Xd.std(0); yd = (yd - yd.mean()) / yd.std()
w_best = np.linalg.lstsq(Xd, yd, rcond=None)[0]
best = np.mean((Xd @ w_best - yd) ** 2)

def sgd(schedule, epochs=60, batch=16, seed=0):
    rng = np.random.default_rng(seed); w = np.zeros(10); step = 0; tail = []
    for ep in range(epochs):
        for idx in np.array_split(rng.permutation(len(yd)), len(yd) // batch):
            g = 2 * Xd[idx].T @ (Xd[idx] @ w - yd[idx]) / len(idx)
            w -= schedule(step) * g; step += 1
        if ep ⩾ epochs - 10: tail.append(np.mean((Xd @ w - yd) ** 2))
    return np.mean(tail) - best, np.std(tail)

for name, sch in [("constant 0.05", lambda t: 0.05),
                  ("decay 0.05/(1 + t/200)", lambda t: 0.05 / (1 + t / 200))]:
    gap, wobble = sgd(sch)
    print(f"{name:24}: extra loss above the best {gap:.4f}, wobble between epochs
{wobble:.4f}")
```

Output:

```
constant 0.05           : extra loss above the best 0.0074, wobble between epochs 0.0047
decay 0.05/(1 + t/200)  : extra loss above the best 0.0019, wobble between epochs 0.0001
```

With a constant rate, the loss stays 0.0074 above the best possible value and wobbles by 0.0047 from epoch to epoch. With the decaying schedule it ends only 0.0019 above, and the wobble almost vanishes (0.0001). Schedules used in practice:

- **Step decay**: divide the rate by 10 every so many epochs.
- **Cosine decay**: lower the rate smoothly along a cosine curve to near zero.
- **Warm-up**: start tiny and increase over the first few hundred or thousand steps, then decay. Adam's early statistics are unreliable and large models are fragile at the start; warm-up is standard for transformers.
- **Reduce on plateau**: cut the rate when the validation loss stops improving.

Gradient clipping

Occasionally a gradient is enormous: a steep cliff in the loss surface, a bad batch, or a recurrent network (Lesson 19) multiplying many large numbers. One such step can throw the weights far away and produce NaNs. **Gradient clipping by norm** rescales the gradient whenever its length exceeds a threshold, keeping its direction:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1,\ \frac{c}{\|\mathbf{g}\|}\right)$$

```
# Gradient clipping: cap the gradient's length before stepping
def clip(g, max_norm=1.0):
    n = np.linalg.norm(g)
    return g * (max_norm / n) if n > max_norm else g

huge = np.array([300.0, -400.0])                # an exploding gradient, length 500
print("before clipping:", huge, " length", np.linalg.norm(huge))
print("after clipping: ", clip(huge), " length", np.linalg.norm(clip(huge)))
```

Output:

```
before clipping: [ 300. -400.]  length 500.0
after clipping:  [ 0.6 -0.8]  length 1.0
```

The gradient of length 500 keeps its direction but is shrunk to length 1. In PyTorch this is one line, `torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`, and it's standard for recurrent networks and transformers.

Choosing an optimiser

Situation	Good default
Small or medium convex problem (linear or logistic regression)	Exact or quasi-Newton solvers (L-BFGS), as scikit-learn uses
Huge dataset, simple model	Mini-batch SGD with momentum and a decaying schedule
Deep network, first attempt	Adam or AdamW, learning rate about 10^{-3}, with warm-up and decay
Image models tuned for the best final accuracy	SGD with momentum 0.9 and cosine decay often generalises slightly better
Loss suddenly becomes NaN	Lower the learning rate, add gradient clipping, check the inputs

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Explain Adam in two sentences. Why is it so popular?**

The trap: Saying "it adapts the learning rate" without saying how.

Adam keeps two running averages per weight: of the gradient (momentum) and of the squared gradient. It steps along the first divided by the square root of the second, with a bias correction for the early steps. Because each weight's step is scaled by its own gradient size, it copes with badly scaled problems and needs little tuning, which suits deep networks with millions of very different weights.

▼ **If Adam is so good, why do people still train some models with SGD and momentum?**

The trap: "SGD is just older."

On many image tasks, well-tuned SGD with momentum and a decaying schedule reaches slightly better test accuracy than Adam (a generalisation gap that is still debated). SGD also uses less memory (Adam stores two extra numbers per weight) and has fewer moving parts. Adam's strengths are speed, robustness to scaling and less tuning, which is why it dominates for transformers, usually as AdamW.

▼ **What's the difference between L2 regularization and weight decay, and why does it matter for Adam?**

The trap: "They're the same thing."

For plain SGD they are mathematically equivalent. For Adam they aren't: an L2 term in the loss gets divided by Adam's per-weight scaling, so weights with large gradients are barely regularized. Decoupled weight decay (AdamW) shrinks the weights directly, $\mathbf{w} \leftarrow \mathbf{w} - \eta\lambda\mathbf{w}$, separately from the adaptive step, which restores the intended effect and usually generalises better.

▼ **Why use a learning-rate schedule instead of one well-chosen constant rate?**

The trap: "To train faster."

Early on, large steps make fast progress and the noise helps explore; near the end, mini-batch noise keeps the weights bouncing around the minimum unless the step size shrinks. Decaying the rate gets both. Warm-up at the very start avoids instability while

optimiser statistics and early layers settle. Theory agrees: SGD needs decaying rates to converge.

Check your understanding: going deeper

1. Why does momentum help in a long, narrow valley?

a. Zig-zag gradients across the valley cancel, while gradients along it add up

b. It computes the Hessian

c. It uses a bigger learning rate for every weight

2. What does Adam divide each weight's step by?

a. The number of steps so far

b. The square root of a running average of that weight's squared gradients

c. The loss

3. Why does SGD with a constant learning rate keep wobbling near the minimum?

a. Mini-batch gradients are noisy, so each step adds noise that never shrinks

b. The loss isn't convex

c. Floating-point errors

4. What does gradient clipping by norm do to a gradient longer than the threshold?

a. Sets it to zero

b. Rescales it to the threshold length, keeping its direction

c. Clips each entry separately to ±threshold

5. What is learning-rate warm-up?

a. Starting with a small learning rate and increasing it over the first steps

b. Training a smaller model first

c. Running a few epochs without updates

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Model $\hat{y} = wx$, data $(1, 2)$ and $(2, 4)$, MSE loss, $w = 0$, $\eta = 0.1$. Do two steps of gradient descent.

WORKING

EXERCISE 2 By hand

On $L(w) = w^2$, start at $w = 1$ with $\eta = 1.5$. What happens over three steps, and why?

WORKING

EXERCISE 3 By hand

10,000 training examples, mini-batch size 100. How many parameter updates happen in 5 epochs?

WORKING

EXERCISE 4 Code

In the Lesson 7 from-scratch code, change `lr = 0.05` to `lr = 0.1` and run 20 steps. What happens?

WORKING

EXERCISE 5 Intermediate

A loss has curvatures 4 and 400 in its two main directions. What is the largest stable learning rate for plain gradient descent, and roughly by what factor is the error in the gentle direction reduced per step at that rate?

WORKING

EXERCISE 6 Intermediate

Run one step of momentum by hand: $\beta = 0.9$, $\eta = 0.1$, starting from $\mathbf{v} = (1, 0)$, $\mathbf{w} = (2, 3)$, with gradient $\mathbf{g} = (0.5, -1)$. Give the new $\mathbf{w}$.

WORKING

EXERCISE 7 Intermediate

Adam's first step ($k = 1$), with $\mathbf{m}$ and $\mathbf{s}$ starting at 0, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon \approx 0$ and a single weight with gradient $g = 4$. What is $\hat{m}/\sqrt{\hat{s}}$, and how big is the step with $\eta = 0.001$?

WORKING

EXERCISE 8 Intermediate

Clip the gradient $(6, 8)$ to maximum norm 2.

WORKING

NOTES

PART 2 · LEARNING FROM LABELLED DATA · ~55 MIN

Lesson 8: Logistic Regression

Classification with the sigmoid, probabilities, log loss and decision boundaries.

BY THE END OF THIS LESSON YOU CAN

- Explain why classification needs the sigmoid function
- Turn model scores into probabilities, decisions and odds
- Compute log loss and explain why it punishes confident mistakes
- Derive the gradient and train a classifier with gradient descent

↺ BEFORE YOU START

This lesson builds on:

- Lesson 4: Probability & Statistics — Probabilities run from 0 to 1. Bayes' theorem turns $P(B \mid A)$ into $P(A \mid B)$. Mean and standard deviation summarise data; the z-score $(x - \mu)/\sigma$ standardises it.
- Lesson 6: Linear Regression — Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ and chooses the weights that minimise the mean squared error (least squares). R^2 measures the share of variance explained.
- Lesson 7: Gradient Descent — Gradient descent trains almost any model: compute the gradient of the loss, step against it, repeat. The learning rate controls the step size; too large diverges. Mini-batches make it fast on big data.

Where this fits: Moves from predicting numbers to predicting categories. A logistic regression is also exactly what a single neuron in a neural network computes.

1. From numbers to categories

So far we've predicted numbers. Many of the most useful questions are yes/no: *Is this email spam? Will this customer cancel? Is this tumour malignant? Will this student pass?* These are **binary classification** problems: the label y is either 1 ("yes", the *positive class*) or 0 ("no", the *negative class*).

Logistic regression is the standard first model for this. Despite its name it is a *classifier*. It doesn't just say yes or no; it outputs a **probability** — "83% likely to pass" — which is far more useful for real decisions.

Running example: predict whether a student passes an exam ($y = 1$) from the hours they studied (x).

Hours x	0.5	1	1.5	2	2.5	3	3.5	4	4.5	5
Passed y	0	0	0	1	0	1	0	1	1	1

More study generally helps, but it isn't a guarantee — exactly the kind of uncertainty probabilities describe.

2. Why not just use linear regression?

We could fit a straight line to the 0s and 1s and call anything above 0.5 a "pass". It works poorly:

- A line goes on forever, so it predicts values like 1.4 or -0.3 — meaningless as probabilities.
- One student who studied 40 hours would drag the whole line flatter, shifting the decision point for everyone.
- The squared-error loss isn't the right way to measure the quality of yes/no predictions.

We need a function that takes any number and squashes it into the range 0 to 1.

3. The sigmoid function

The **sigmoid** (or logistic) function does exactly that:

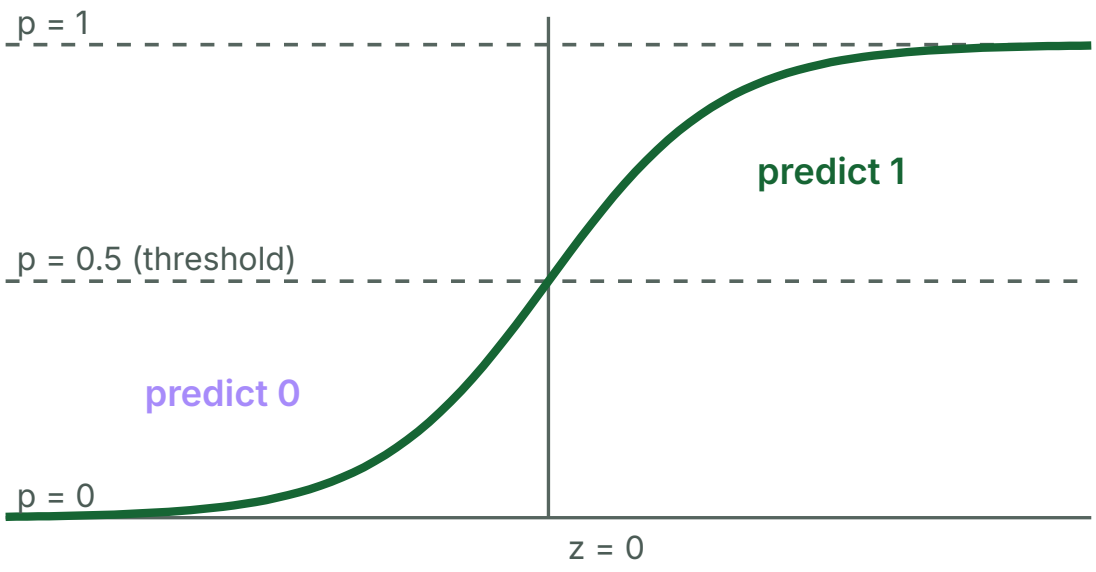


Figure 8. The sigmoid turns any score z into a probability. With threshold 0.5, the decision boundary is exactly where z = 0.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $e \approx 2.718$. Some values:

z	−4	−2	−1	0	1	2	4
$\sigma(z)$	0.018	0.119	0.269	0.5	0.731	0.881	0.982

- Large positive z → close to 1. Large negative z → close to 0.
- $\sigma(0) = 0.5$ exactly: the point of maximum uncertainty.
- Symmetry: $\sigma(-z) = 1 - \sigma(z)$.
- Its derivative has a neat form: $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$. We'll use this shortly.

> ◎ PICTURE IT
>
> The sigmoid is an S-shaped curve: flat near 0 on the far left, a smooth climb through 0.5 in the middle, flat near 1 on the far right. It turns an unlimited "score" into a "confidence".

STEP-BY-STEP DERIVATION **Derive the sigmoid's derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$**

1. $\sigma(z) = (1 + e^{-z})^{-1}$

   Write the sigmoid as a power.

2. $\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \dfrac{e^{-z}}{(1 + e^{-z})^2}$

   Chain rule: outside u^{-1}, inside $u = 1 + e^{-z}$, whose derivative is $-e^{-z}$.

3. $= \dfrac{1}{1 + e^{-z}} \cdot \dfrac{e^{-z}}{1 + e^{-z}}$

   Split into two fractions.

4. $\dfrac{e^{-z}}{1 + e^{-z}} = \dfrac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \sigma(z)$

   Rewrite the second fraction.

5. $\sigma'(z) = \sigma(z)\left(1 - \sigma(z)\right)$

   The first fraction is $\sigma(z)$. This tidy result is why the log-loss gradient simplifies to $p - y$.

4. The logistic regression model

Take the linear model from Lesson 6 and pass its output through the sigmoid:

$$z = \mathbf{w} \cdot \mathbf{x} + b \qquad\qquad p = \hat{y} = \sigma(z) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

z is a score (any number); p is the predicted probability that the example belongs to class 1: $p = P(y = 1 \mid \mathbf{x})$.

✎ WORKED EXAMPLE — USING A TRAINED MODEL

Trained on the study data (we'll see how below), the model is $z = 1.354\,x - 3.723$.

A student who studied **2 hours**: $z = 1.354 \times 2 - 3.723 = -1.015$, so $p = \sigma(-1.015) = \frac{1}{1+e^{1.015}} \approx 0.27$. A 27% chance of passing.

A student who studied **4 hours**: $z = 1.354 \times 4 - 3.723 = 1.693$, so $p \approx 0.84$. An 84% chance.

? MAKE SENSE OF IT

The situation: A tutor uses past results to tell each student their chance of passing from how many hours they studied.

What the result tells you: 27% and 84% are probabilities, not grades: among many students who study 2 hours, about 27 in 100 pass.

What if...?

- **No study at all:** $p = \sigma(-3.723) \approx$ **2%**.
- **The 50% point** is where $z = 0$: $x = 3.723/1.354 \approx$ **2.75 hours**. That's the decision boundary at a 0.5 threshold.
- **6 and 8 hours:** about **99%** and **99.9%**. The curve flattens — extra hours add less and less probability.

Why it matters in ML: The sigmoid turns any score into a probability between 0 and 1. The threshold that turns it into a yes/no decision is a separate choice, set by the cost of each kind of mistake (Lesson 9).

READ THE FORMULA **Logistic regression**

$$p = \sigma\left(\mathbf{w} \cdot \mathbf{x} + b\right)$$

- **p** — predicted probability of class 1
- **σ** — sigmoid — squashes any score into (0, 1)
- **w · x + b** — a linear score, exactly as in linear regression

Hover over (or tap) a part to highlight it in the formula.

5. From probability to a decision: the decision boundary

To output a class, pick a **threshold**, usually 0.5: predict 1 if $p \geq 0.5$, else 0. Since $\sigma(z) \geq 0.5$ exactly when $z \geq 0$, the rule is simply:

$$\text{predict } 1 \iff \mathbf{w} \cdot \mathbf{x} + b \geq 0$$

The set of points where $\mathbf{w} \cdot \mathbf{x} + b = 0$ is the **decision boundary**. For the study data: $1.354x - 3.723 = 0 \Rightarrow x \approx 2.75$ hours. Below that we predict "fail", above it "pass".

With two features the boundary is a straight line; with three, a plane. Logistic regression is therefore a **linear classifier** — it separates classes with a straight boundary. (Curved boundaries

need extra features, or models like those in Lessons 11–15 and 18.)

> **THE THRESHOLD IS A CHOICE**
>
> For cancer screening you might flag anyone with $p \geq 0.1$, accepting more false alarms to miss fewer real cases. For a spam filter you might require $p \geq 0.9$ so real emails are rarely lost. Lesson 9 shows how to measure this trade-off.

6. Interpreting the weights: odds

The **odds** of an event are $\frac{p}{1-p}$: a probability of 0.8 is odds of $0.8/0.2 = 4$ ("4 to 1"). Solving the model for z gives:

$$\ln\left(\frac{p}{1-p}\right) = \mathbf{w} \cdot \mathbf{x} + b$$

So logistic regression is a linear model for the **log-odds** (also called the *logit*). That gives the weights a clean meaning: increasing feature x_j by 1 *multiplies the odds* by e^{w_j}.

> **✎ WORKED EXAMPLE**
>
> With $w = 1.354$: $e^{1.354} \approx 3.87$. Each extra hour of study multiplies a student's odds of passing by about 3.9. A negative weight would shrink the odds instead.

> **? MAKE SENSE OF IT**
>
> **The situation:** Odds are how bookmakers talk: 3 to 1 means three times as likely to happen as not.
>
> **What the result tells you:** A weight of 1.354 means each extra hour multiplies the odds of passing by $e^{1.354} \approx 3.87$. The *probability* doesn't go up by a fixed amount, but the odds do go up by a fixed factor.
>
> **What if…?**
>
> - **From 2 to 3 hours:** odds go from 0.36 (p = 0.27) to $0.36 \times 3.87 \approx 1.40$ (p = 0.58).
> - **A weight of 0** gives a factor of $e^0 = 1$: the feature doesn't change the odds at all.
> - **A weight of −0.5** gives a factor of 0.61: each unit *reduces* the odds by about 39%.
>
> **Why it matters in ML:** This is how logistic-regression results are reported in medicine and credit scoring: "each extra year of age multiplies the odds of default by 1.05". Remember it's an association in the data, not proof of cause.

7. The loss: log loss (cross-entropy)

How do we measure how wrong a probability is? If the truth is $y = 1$, we want p close to 1; if $y = 0$, we want p close to 0. The standard loss for one example is **log loss**, also called **binary cross-entropy**:

$$\ell(p, y) = -\big[\, y \ln p + (1 - y) \ln(1 - p) \,\big]$$

Only one of the two terms is active at a time:

- If $y = 1$: $\ell = -\ln p$.
- If $y = 0$: $\ell = -\ln(1 - p)$.

✎ WORKED EXAMPLE — HOW LOG LOSS PUNISHES CONFIDENT MISTAKES

True label $y = 1$:

Predicted p	Loss $-\ln p$	Meaning
0.9	0.105	confident and right — tiny loss
0.5	0.693	unsure — moderate loss
0.1	2.303	confident and wrong — big loss
0.01	4.605	very confident and wrong — huge loss

As $p \to 0$ the loss goes to infinity. Log loss strongly discourages being confidently wrong.

? MAKE SENSE OF IT

The situation: A weather app says "1% chance of rain" and it pours. How embarrassed should it be? Log loss puts a number on it.

What the result tells you: The loss is small when the model gives high probability to what actually happened, and grows without limit as that probability approaches zero.

What if...?

- **p = 0.001 on something that happens:** loss **6.91**.
- **One disaster among many successes:** 99 predictions at 0.9 (loss 0.105 each) plus one at 0.001 average to **0.173** — that single confident miss raises the average loss by about 65%.

Why it matters in ML: Log loss rewards honest probabilities, not just correct labels. Training with it pushes models away from overconfidence — and it's the same cross-entropy loss used to train neural networks and language models.

The training loss is the average over all n examples:

$$L(\mathbf{w}, b) = -\frac{1}{n}\sum_{i=1}^{n}\left[\, y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\,\right]$$

| WHERE DOES THIS FORMULA COME FROM?

From probability (Lesson 4). The model says example i has label 1 with probability p_i. The probability of observing its actual label is $p_i^{y_i}(1-p_i)^{1-y_i}$ (this equals p_i if $y_i = 1$ and $1 - p_i$ if $y_i = 0$). Multiply over all examples to get the **likelihood** of the data. Taking the log turns the product into a sum, and minimising the negative log-likelihood is exactly minimising log loss. So we're choosing the weights that make the observed labels *most probable* — a principle called **maximum likelihood**. Unlike MSE with a sigmoid, this loss is convex: one bowl, one minimum.

ADVANCED · OPTIONAL **Where log loss comes from, and why its gradient is so simple**

▾ hide

Treat each label as a coin flip with $P(y = 1) = p$. The likelihood of one observation is $p^y(1-p)^{1-y}$, so the negative log-likelihood of the data is

$$-\sum_i \left[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\right],$$

which is exactly the log loss: minimising it is **maximum likelihood estimation**. For the gradient, use $\sigma'(z) = \sigma(z)(1-\sigma(z))$ and the chain rule on one example with $p = \sigma(z)$, $z = \mathbf{w} \cdot \mathbf{x}$:

$$\frac{\partial \ell}{\partial z} = -\frac{y}{p}\,p(1-p) + \frac{1-y}{1-p}\,p(1-p) = p - y, \qquad \frac{\partial \ell}{\partial \mathbf{w}} = (p-y)\,\mathbf{x}.$$

The Hessian is $\sum_i p_i(1-p_i)\,\mathbf{x}_i\mathbf{x}_i^\top$, a sum of positive semi-definite matrices, so log loss is **convex**: gradient descent cannot get stuck in a bad local minimum.

8. The gradient — surprisingly simple

There's no closed-form solution here, so we use gradient descent (Lesson 7). Let's derive the gradient for one example with the chain rule. The loss depends on p, which depends on z, which depends on w_j:

$$\frac{\partial \ell}{\partial w_j} = \frac{\partial \ell}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

1. $\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}$ (derivative of $\ln$ is $1/\cdot$).
2. $\frac{\partial p}{\partial z} = p(1-p)$ (the sigmoid's derivative).
3. $\frac{\partial z}{\partial w_j} = x_j$.
4. Multiply the first two: $\left(-\frac{y}{p} + \frac{1-y}{1-p}\right) p(1-p) = -y(1-p) + (1-y)p = p - y$.

So everything collapses to:

$$\frac{\partial L}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\,x_{ij} \qquad \frac{\partial L}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)$$

"Prediction minus truth, times the input" — the same shape as linear regression's gradient. The update is $w_j \leftarrow w_j - \eta\,\frac{\partial L}{\partial w_j}$, as always.

✎ WORKED EXAMPLE — ONE GRADIENT STEP

One student: $x = 2$ hours, passed ($y = 1$). Start with $w = 0$, $b = 0$, learning rate $\eta = 0.5$.

$z = 0$, $p = \sigma(0) = 0.5$. Loss $= -\ln 0.5 = 0.693$.

Gradients: $\frac{\partial \ell}{\partial w} = (0.5 - 1) \times 2 = -1$, $\frac{\partial \ell}{\partial b} = 0.5 - 1 = -0.5$.

Update: $w = 0 - 0.5 \times (-1) = 0.5$, $b = 0 - 0.5 \times (-0.5) = 0.25$.

Now $z = 0.5 \times 2 + 0.25 = 1.25$, $p = \sigma(1.25) \approx 0.777$, loss $= -\ln 0.777 \approx 0.252$. The model became more confident in the right answer.

? MAKE SENSE OF IT

The situation: The model sees one student who studied 2 hours and passed, and adjusts itself.

What the result tells you: Its prediction was 0.5 (no idea). The error $p - y = -0.5$ says "too low", so both w and b move up and the new prediction is 0.777.

What if…?

- **A second step** on the same student: p rises to about **0.859** and the loss falls to 0.152. Each step helps a little less, because the error $p - y$ is shrinking.
- **If the student had failed** ($y = 0$): the error is $+0.5$, the weights move *down* to $w = -0.5$, $b = -0.25$, and p falls to 0.223.

Why it matters in ML: The gradient for logistic regression is just (prediction − truth) × input. Big mistakes cause big updates; correct, confident predictions barely change anything.

9. Try it: train a classifier

Click to add points: choose the class with the buttons, then click on the chart. Press *Train* to run gradient descent. The background colour shows the predicted probability (stronger colour = more confident) and the white line is the decision boundary where $p = 0.5$. Try making the two groups overlap, or arranging one class in a ring around the other — a straight boundary can't separate that.

Interactive demo — open *lesson-08.html* on the website to try it.

10. More than two classes

For K classes (e.g. digits 0–9) there are two common approaches:

- **One-vs-rest:** train K binary classifiers ("is it a 3, or not?") and pick the most confident.
- **Softmax (multinomial) regression:** compute a score $z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$ for each class, then turn the scores into probabilities that sum to 1:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

✎ WORKED EXAMPLE — SOFTMAX

Scores for three classes: $z = (2, 1, 0)$. Exponentials: $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$; total ≈ 11.11.

Probabilities: $7.39/11.11 \approx 0.665$, $2.72/11.11 \approx 0.245$, $1/11.11 \approx 0.090$. They sum to 1, and the highest score gets the highest probability.

? MAKE SENSE OF IT

The situation: An image model scores a photo as cat 2, dog 1, rabbit 0 and has to turn the scores into probabilities.

What the result tells you: Softmax gives 66.5% cat, 24.5% dog, 9% rabbit. The probabilities always add to 1 and keep the order of the scores.

What if...?

- **Bigger gaps** (4, 2, 0): **86.7%**, 11.7%, 1.6% — more confident.
- **Add 10 to every score** (12, 11, 10): exactly the same probabilities. Only differences between scores matter.
- **Equal scores** (2, 2, 2): one-third each.

Why it matters in ML: Softmax is the last layer of nearly every classifier and of every language model, where the "classes" are possible next words. Making scores more spread out (or using a lower "temperature") makes the model more decisive.

Softmax with cross-entropy loss is also the standard output layer of neural network classifiers (Lesson 18).

11. In Python

From scratch on the study-hours data:

```
import numpy as np

x = np.array([0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5])
y = np.array([0, 0, 0, 1, 0, 1, 0, 1, 1, 1])

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

w, b, lr = 0.0, 0.0, 0.1
for step in range(5000):
    p = sigmoid(w * x + b)
    w -= lr * np.mean((p - y) * x)      # dL/dw = mean((p - y) x)
    b -= lr * np.mean(p - y)            # dL/db = mean(p - y)

p = sigmoid(w * x + b)
loss = -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
print(round(w, 3), round(b, 3), round(loss, 3))   # 1.353 -3.721 0.434
print(round(-b / w, 2))                           # decision boundary: 2.75 hours
```

On a real medical dataset with scikit-learn — 569 breast tumours, 30 measurements each, labelled malignant or benign:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)

print(round(model.score(X_test, y_test), 3))      # accuracy on unseen tumours
print(model.predict_proba(X_test[:3]).round(3))   # [P(class 0), P(class 1)] for 3
tumours
```

Output:

```
0.958
[[0.004 0.996]
 [1.    0.   ]
 [1.    0.   ]]
```

About 96% of unseen tumours are classified correctly, and for each one we get a probability, not just a label. (In this dataset class 1 means benign.) Note that scikit-learn's `LogisticRegression` applies L2 regularization by default — you'll learn what that means in Lesson 10.

12. Running example: Chennai Homes

Can we predict which houses sell fast? Both weights are negative (on standardised features): farther and older houses are less likely to sell fast. A new 5-year-old house 3 km from the centre gets an 88% chance; a 25-year-old one 20 km out, just 3%.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
cols = ["dist_km", "age_yrs"]
clf = make_pipeline(StandardScaler(), LogisticRegression()).fit(df[cols],
df["sold_fast"])
print({c: round(float(v), 2) for c, v in zip(cols, clf[-1].coef_[0])})
new = pd.DataFrame({"dist_km": [3, 20], "age_yrs": [5, 25]})
print(clf.predict_proba(new)[:, 1].round(2))      # P(sells fast) for two new
houses
```

Output:

```
{'dist_km': -1.04, 'age_yrs': -1.27}
[0.88 0.03]
```

⚙ WHERE THIS IS USED

- Credit scoring and loan default risk
- Medical risk prediction
- Click-through prediction for ads
- A strong, explainable baseline classifier

Choose it when: You need probabilities, interpretability and speed; classes are roughly linearly separable.

Think twice when: Boundaries are highly non-linear and you can't engineer suitable features.

× MISCONCEPTIONS TO UNLEARN

MYTH Logistic regression is a regression model.

REALITY It's a classifier — the name comes from regressing the log-odds.

MYTH A probability of 0.7 means the model is right 70% of the time.

REALITY Only if the model is well calibrated. Always check predicted probabilities against actual outcomes on held-out data.

RESPONSIBLE AI · DECISIONS ABOUT PEOPLE NEED EXPLANATIONS

Logistic regression is common in credit scoring, where regulations in many countries (for example the EU's GDPR and the US Equal Credit Opportunity Act) give people rights to information about automated decisions that affect them. Its weights are relatively easy to explain. Remember too that the decision threshold is a policy choice: moving it changes who is approved, and it may affect different groups differently.

13. Questions beginners often ask

Q. Why is it called regression if it classifies?

Historical naming: it regresses (fits) the log-odds of the class with a linear function. It's used as a classifier.

Q. Are its probabilities trustworthy?

Often reasonably well calibrated, especially compared with many other models — but always check on held-out data.

Q. What if classes aren't linearly separable?

Add features (squares, products) or use a non-linear model: trees, kNN, SVM kernels or neural networks.

! COMMON MISTAKES TO AVOID

- Using MSE instead of log loss for classification.
- Reading the weight as a change in probability — it changes the log-odds.
- Assuming 0.5 is always the right threshold; the costs of each mistake should decide it.

! INTERVIEW CORNER

Don't confuse

- **Probability vs odds vs log-odds (logit)** — 0.5 probability = odds 1 = log-odds 0.
- **Sigmoid vs softmax** — Sigmoid for two classes, softmax for many.
- **Log loss vs cross-entropy** — Two names, one formula.
- **Multi-class vs multi-label** — Pick one vs tick all that apply.

Hidden questions — answer in your head first, then open.

▼ **Why is it called logistic *regression* if it's used for classification?**

The trap: Thinking it's a misnomer.

It is a regression on the *log-odds*: $\ln \frac{p}{1-p} = \mathbf{w} \cdot \mathbf{x} + b$. The classification comes from thresholding the predicted probability.

▼ **Why use log loss rather than MSE for logistic regression?**

The trap: "MSE works for everything."

MSE through a sigmoid gives a non-convex loss with tiny gradients when the model is confidently wrong, so learning stalls exactly when it should be fastest. Log loss is convex for logistic regression and punishes confident mistakes heavily.

▼ **Is the decision boundary of logistic regression linear?**

The trap: "No, the sigmoid is curved."

Yes. The boundary is where $p = 0.5$, i.e. $\mathbf{w} \cdot \mathbf{x} + b = 0$ — a straight line or flat plane. The sigmoid only bends the probabilities, not the boundary. Curved boundaries need extra features or another model.

▼ **What happens to logistic regression on perfectly separable data without regularisation?**

The trap: Expecting a clean, finite answer.

The weights grow without limit: making them larger always makes the probabilities more confident and the log loss smaller, approaching but never reaching 0. Regularisation (sklearn's default C = 1.0) keeps them finite.

14. Key takeaways

- Logistic regression predicts $p = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, the probability of class 1; $\sigma(z) = 1/(1 + e^{-z})$.
- Predict class 1 when $p \geq$ threshold (often 0.5), i.e. when $\mathbf{w} \cdot \mathbf{x} + b \geq 0$: a **linear decision boundary**.
- Each weight multiplies the **odds** by e^{w_j} per unit increase of its feature.
- Trained by minimising **log loss** (maximum likelihood), which heavily punishes confident mistakes.
- Gradient: $\frac{1}{n} \sum (p_i - y_i)\, \mathbf{x}_i$. Multi-class uses **softmax**.

15. Check your understanding

1. What is $\sigma(0)$?

a. 0

b. 0.5

c. 1

2. A model has $z = 2x - 6$. Where is the decision boundary (threshold 0.5)?

a. $x = 6$

b. $x = 0.5$

c. $x = 3$

3. The true label is 1. Which prediction gives the largest log loss?

a. $p = 0.05$

b. $p = 0.5$

c. $p = 0.95$

4. A feature has weight $w = \ln 2 \approx 0.693$. Increasing that feature by 1...

a. adds 0.693 to the probability

b. doubles the odds

c. doubles the probability

5. For one example, $p = 0.8$, $y = 0$, $x = 3$. What is $\partial \ell / \partial w$?

a. -0.6

b. 0.8

c. 2.4

16. Going deeper: logistic regression as a probability model

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above trains logistic regression with gradient descent and reads its weights as odds. This section looks at it the way statisticians and library authors do: why its loss is convex, how scikit-learn actually solves it in a handful of steps, how regularization enters, whether its probabilities can be trusted (**calibration**), and what to do when one class is rare.

You'll need: log loss from this lesson, maximum likelihood from Lesson 4's "Going deeper", and the Hessian and Newton's method from Lesson 3's.

Why the loss is convex

Log loss is the negative log-likelihood of a Bernoulli model (Lesson 4's "Going deeper"). Its gradient, from this lesson, is $X^\top(\mathbf{p} - \mathbf{y})$. Differentiating once more gives the Hessian:

$$H = X^\top S\, X, \qquad S = \operatorname{diag}\big(p_i(1-p_i)\big)$$

Each $p_i(1-p_i)$ is positive, so for any $\mathbf{v}$, $\mathbf{v}^\top H \mathbf{v} = \sum_i p_i(1-p_i)\,(\mathbf{x}_i \cdot \mathbf{v})^2 \geq 0$. The Hessian is positive semi-definite everywhere, so the loss is **convex**: one valley, no bad local minima. Notice that $p_i(1-p_i)$ is largest (0.25) for points near the boundary ($p \approx 0.5$) and tiny for points the model is already sure about. Confident points hardly affect the curvature.

How libraries solve it: Newton's method (IRLS)

Because the Hessian is cheap for a moderate number of features, we can use Newton's method from Lesson 3's "Going deeper": $\mathbf{w} \leftarrow \mathbf{w} - H^{-1}\nabla L$. For logistic regression each step is a *weighted least-squares* problem with weights $p_i(1-p_i)$, so statisticians call it **iteratively reweighted least squares** (IRLS). Here it is from scratch with an L2 penalty $\frac{\lambda}{2}\|\mathbf{w}\|^2$, which adds $\lambda\mathbf{w}$ to the gradient and λI to the Hessian, on 5 features of the breast-cancer data:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)
X = StandardScaler().fit_transform(X[:, :5])          # 5 features keep the example
small
A = np.column_stack([np.ones(len(y)), X])
lam = 1.0                                             # L2 penalty, same as sklearn's C
= 1

w = np.zeros(A.shape[1])
for it in range(1, 11):                               # Newton's method (IRLS)
    p = 1 / (1 + np.exp(-A @ w))
    grad = A.T @ (p - y) + lam * np.r_[0, w[1:]]      # don't penalise the bias
    S = p * (1 - p)                                   # each point's weight
    H = A.T @ (A * S[:, None]) + lam * np.diag(np.r_[0, np.ones(5)])
    step = np.linalg.solve(H, grad)
    w -= step
    print(f"iteration {it}: step size {np.linalg.norm(step):.2e}")
    if np.linalg.norm(step) < 1e-8: break

sk = LogisticRegression(C=1.0, tol=1e-10, max_iter=1000).fit(X, y)   # tight tolerance
print("ours:   ", w.round(4))
print("sklearn:", np.r_[sk.intercept_, sk.coef_[0]].round(4))
```

Output:

```
iteration 1: step size 1.78e+00
iteration 2: step size 8.46e-01
iteration 3: step size 1.09e+00
iteration 4: step size 7.33e-01
iteration 5: step size 3.00e-01
iteration 6: step size 4.06e-02
iteration 7: step size 6.29e-04
iteration 8: step size 1.48e-07
iteration 9: step size 7.89e-15
ours:    [ 0.7707 -1.1797 -1.405  -1.8599 -1.5528 -1.7102]
sklearn: [ 0.7707 -1.1797 -1.405  -1.8599 -1.5528 -1.7102]
```

The steps shrink slowly at first and then collapse: 6×10^{-4}, then 1.5×10^{-7}, then 8×10^{-15}. That's Newton's **quadratic convergence**: near the answer, the number of correct digits roughly doubles each step. Nine iterations give the same weights as scikit-learn to 4 decimal places. (Scikit-learn's default solver, L-BFGS, approximates this Hessian instead of computing it, so it scales to more features.)

The penalty matters more than it looks. Scikit-learn always regularizes logistic regression unless told not to: `C` is $1/\lambda$, so **smaller `C` means stronger regularization**. Without a penalty, perfectly separable data sends the weights to infinity, because the loss keeps falling as the sigmoid gets steeper.

Can you trust the probabilities? Calibration

A classifier is **well calibrated** if, among all cases where it says 0.8, about 80% really are positive. Accuracy and AUC don't measure this at all: they only care about the order of the scores. Two ways to check:

- A **reliability table** (or plot): group predictions into bins and compare each bin's average prediction with the actual share of positives.
- The **Brier score**, the mean of $(p_i - y_i)^2$: 0 is perfect, and lower is better. It rewards both calibration and sharpness.

Here are logistic regression and Gaussian naive Bayes (Lesson 12) on the breast-cancer test set:

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import brier_score_loss

Xa, ya = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(Xa, ya, test_size=0.5, random_state=0,
stratify=ya)
sc = StandardScaler().fit(Xtr)
models = {"logistic regression":
LogisticRegression(max_iter=1000).fit(sc.transform(Xtr), ytr),
          "naive Bayes": GaussianNB().fit(sc.transform(Xtr), ytr)}
bins = [0, 0.2, 0.4, 0.6, 0.8, 1.0]
for name, m in models.items():
    p = m.predict_proba(sc.transform(Xte))[:, 1]
    print(f"{name}: Brier score {brier_score_loss(yte, p):.3f}")
    idx = np.digitize(p, bins[1:-1])
    for b in range(5):
        sel = idx == b
        if sel.sum(): print(f"   predicted {bins[b]:.1f}–{bins[b+1]:.1f}: {sel.sum():3d}
cases, average prediction {p[sel].mean():.2f}, actually positive {yte[sel].mean():.2f}")
```

Output:

```
logistic regression: Brier score 0.018
   predicted 0.0–0.2:  98 cases, average prediction 0.02, actually positive 0.00
   predicted 0.2–0.4:   6 cases, average prediction 0.32, actually positive 0.33
   predicted 0.4–0.6:   6 cases, average prediction 0.49, actually positive 0.67
   predicted 0.6–0.8:   7 cases, average prediction 0.72, actually positive 1.00
   predicted 0.8–1.0: 168 cases, average prediction 0.98, actually positive 0.99
naive Bayes: Brier score 0.071
   predicted 0.0–0.2: 107 cases, average prediction 0.00, actually positive 0.10
   predicted 0.6–0.8:   1 cases, average prediction 0.69, actually positive 1.00
   predicted 0.8–1.0: 177 cases, average prediction 1.00, actually positive 0.94
```

Logistic regression's bins line up fairly well: it predicts 0.02 where 0% are positive and 0.98 where 99% are (the middle bins hold only 6–7 cases each, so they're noisy). Its Brier score is 0.018. Naive Bayes is badly **over-confident**: it puts 107 cases at an average of 0.00, but 10% of them are actually positive. Its Brier score of 0.071 is four times worse. That's typical: naive Bayes assumes the features are independent, so it counts correlated evidence several times over. Logistic regression is usually well calibrated because it directly maximises the likelihood of the labels. When a model isn't, `CalibratedClassifierCV` can fix it on held-out data, with **Platt scaling** (a small logistic regression on the scores) or **isotonic regression** (a flexible increasing curve).

When one class is rare

With 5% positives, a model can reach 95% accuracy by never predicting the positive class. Logistic regression won't go quite that far, but with the default threshold of 0.5 it may catch very few positives:

```
from sklearn.metrics import recall_score, precision_score

# Imbalanced problem: 5% positives
from sklearn.datasets import make_classification
Xi, yi = make_classification(n_samples=4000, n_features=8, weights=[0.95],
class_sep=0.8, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(Xi, yi, test_size=0.5, random_state=0,
stratify=yi)
for cw in [None, "balanced"]:
    m = LogisticRegression(class_weight=cw).fit(Xtr, ytr)
    pr = m.predict(Xte)
    print(f"class_weight {str(cw):9}: recall {recall_score(yte, pr):.3f}  precision
{precision_score(yte, pr):.3f}  average predicted probability {m.predict_proba(Xte)[:,
1].mean():.3f}")
print("true share of positives in the test set:", round(yte.mean(), 3))
```

Output:

```
class_weight None     : recall 0.075  precision 0.348  average predicted probability
0.055
class_weight balanced : recall 0.822  precision 0.187  average predicted probability
0.314
true share of positives in the test set: 0.054
```

The default model catches only 7.5% of positives. Its probabilities are honest, averaging 0.055 against a true rate of 0.054, but few cases ever pass 0.5. With `class_weight="balanced"`, each positive counts about 19 times more in the loss, and recall jumps to 82.2%, at the cost of precision (0.187) and of calibration: the average prediction is now 0.314, nowhere near the true 5%. Two equivalent ways to think about it:

- **Reweighting** changes which errors the training loss cares about.
- **Moving the threshold** (Lesson 9) keeps the honest probabilities and simply flags anything above, say, 0.1.

If you need trustworthy probabilities (for pricing, expected costs or ranking by risk), prefer an unweighted model and choose the threshold from costs. If you only need decisions, either approach works. Oversampling the rare class (or methods like SMOTE) has a similar effect to reweighting, and must be done only inside the training folds, or it leaks.

More than two classes: the softmax gradient

Multinomial logistic regression (this lesson's softmax) has weights W with one column per class, probabilities $\mathbf{p} = \mathrm{softmax}(W^\top \mathbf{x})$, and log loss $-\log p_{\text{true}}$. Its gradient has the same simple form as the binary case:

$$\nabla_W L = \mathbf{x}\,(\mathbf{p} - \mathbf{y})^\top$$

where $\mathbf{y}$ is the one-hot true label. "Prediction minus truth, times the input" appears again and again: linear regression, logistic regression, softmax and the output layer of a neural network (Lesson 18) all share it. That's no coincidence. All of them are **generalised linear models** trained by maximum likelihood with their natural ("canonical") link function.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How does scikit-learn solve logistic regression, and why doesn't it use plain gradient descent?**

The trap: Assuming everything is trained with gradient descent.

The loss is smooth and convex, so second-order methods converge in a few dozen iterations. The default solver, L-BFGS, builds a low-memory approximation of the Hessian; Newton-CG and IRLS use the exact Hessian $X^\top S X$. Coordinate-descent solvers (liblinear, saga) handle L1 penalties and very large sparse data. Plain gradient descent would need careful learning rates and far more steps.

▼ **What is calibration, how do you measure it, and how do you fix a badly calibrated model?**

The trap: Confusing calibration with accuracy or AUC.

A model is calibrated if predictions of p come true about a fraction p of the time. Measure it with a reliability diagram (binned predictions vs observed rates), the Brier score, or expected calibration error. Fix it on held-out data with Platt scaling (a sigmoid fitted to the scores) or isotonic regression. Random forests, boosted trees, SVM scores and naive Bayes often need it; logistic regression usually doesn't, unless it was class-weighted or heavily regularized.

▼ **You have 1% fraud. Would you rather resample the data, reweight the classes, or change the threshold?**

The trap: Saying resampling is always required.

All three shift the precision–recall trade-off. Reweighting or resampling changes training, which can help the model focus on rare cases but distorts its probabilities. Threshold moving keeps a calibrated model and picks the cut-off from the cost of each error type, which is usually the simplest and most transparent. Whatever you choose, evaluate with precision–recall metrics, not accuracy, and resample only inside training folds.

▼ **Why does logistic regression need regularization on perfectly separable data?**

The trap: "To prevent overfitting" with no mechanism.

If a line separates the classes perfectly, making the weights larger always reduces log loss, because the sigmoid becomes steeper and every probability moves closer to 0 or

1. The optimum is at infinity, so training never converges and the probabilities become extreme. An L2 penalty gives a finite, unique answer.

Check your understanding: going deeper

1. In scikit-learn's `LogisticRegression`, what does a smaller `C` do?

a. Weakens regularization

b. Strengthens regularization

c. Changes the threshold

2. A model says 0.9 for 100 cases, and 60 of them are actually positive. The model is...

a. Over-confident (badly calibrated)

b. Well calibrated

c. Under-confident

3. Why does logistic regression's Hessian guarantee a single minimum?

a. It's $X^\top SX$ with positive weights, so it is positive semi-definite and the loss is convex

b. It's always diagonal

c. It's always the identity

4. `class_weight="balanced"` on data with 5% positives usually...

a. Raises recall and pushes the average predicted probability well above 5%

b. Leaves predictions unchanged

c. Makes the probabilities better calibrated

5. Which metric measures calibration and sharpness together?

a. Accuracy

b. ROC-AUC

c. Brier score

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Compute $\sigma(1.5)$ and $\sigma(-3)$.

WORKING

EXERCISE 2 By hand

A model has $z = 0.8x - 4$. Where is the decision boundary? What probability does it give at $x = 7$?

WORKING

EXERCISE 3 By hand

Compute the log loss for $p = 0.2$ when the true label is $y = 0$, and when it is $y = 1$.

WORKING

EXERCISE 4 Concepts

A feature's weight is 0.405. By what factor do the odds change per unit increase? If the probability was 0.5, what does it become?

WORKING

EXERCISE 5 Intermediate

Three training points have predicted probabilities 0.5, 0.9 and 0.99. Compute each point's Hessian weight $p(1-p)$. Which one shapes the curvature most?

WORKING

EXERCISE 6 Intermediate

Compute the Brier score for predictions $(0.9, 0.2, 0.6, 0.4)$ with true labels $(1, 0, 0, 1)$.

WORKING

EXERCISE 7 Intermediate

Scikit-learn's `class_weight="balanced"` gives class c the weight $n/(k\,n_c)$ for k classes. With 1,000 examples, 950 negative and 50 positive, what are the two weights?

WORKING

EXERCISE 8 Intermediate

Softmax gives $\mathbf{p} = (0.7, 0.2, 0.1)$ for an example whose true class is the second, with input $\mathbf{x} = (1, 2)$. What is $\mathbf{p} - \mathbf{y}$, and what is the gradient for the second class's weight column?

WORKING

NOTES

PART 2 · LEARNING FROM LABELLED DATA · ~50 MIN

Lesson 9: Evaluating Models

Confusion matrices, precision, recall, F1, ROC curves and regression metrics.

BY THE END OF THIS LESSON YOU CAN

- Build and read a confusion matrix
- Compute accuracy, precision, recall, specificity and F1 by hand
- Explain the threshold trade-off, ROC curves and AUC
- Choose suitable metrics for classification and regression problems

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 8: Logistic Regression — Logistic regression outputs a probability $p = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, is trained with log loss, and classifies with a threshold (often 0.5), giving a straight decision boundary.

Where this fits: Now that you can build classifiers, you need to judge them properly. These metrics are used in every remaining lesson and in real projects.

1. Why "accuracy" is not enough

You've trained a classifier. Is it good? The obvious answer is **accuracy**: the fraction of predictions that are correct. But accuracy can be badly misleading.

✎ THE ACCURACY TRAP

A disease affects 1 in 100 people. A "model" that simply says *"healthy"* for everyone is **99% accurate** — and completely useless, because it never finds a single sick person.

? MAKE SENSE OF IT

The situation: A hospital vendor boasts that its screening model is "99% accurate".

What the result tells you: On a disease affecting 1 in 100 people, a model that never finds anyone sick is also 99% accurate. Accuracy is dominated by the large healthy group.

What if...?

- **A model that flags everyone** finds every sick person but is only **1%** accurate.
- **If half the population were sick,** "always healthy" would be just **50%** accurate. How good an accuracy is depends on the base rate.

Why it matters in ML: Always compare a model with the trivial baseline that predicts the most common class, and use recall, precision or AUC when one class is rare.

Different mistakes have different costs. Missing a cancer is far worse than an unnecessary follow-up test; flagging a real email as spam is worse than letting one spam through. This lesson gives you the tools to see exactly *which* mistakes a model makes, and to measure what actually matters for your problem.

2. The confusion matrix

For a binary classifier, every prediction falls into one of four boxes:

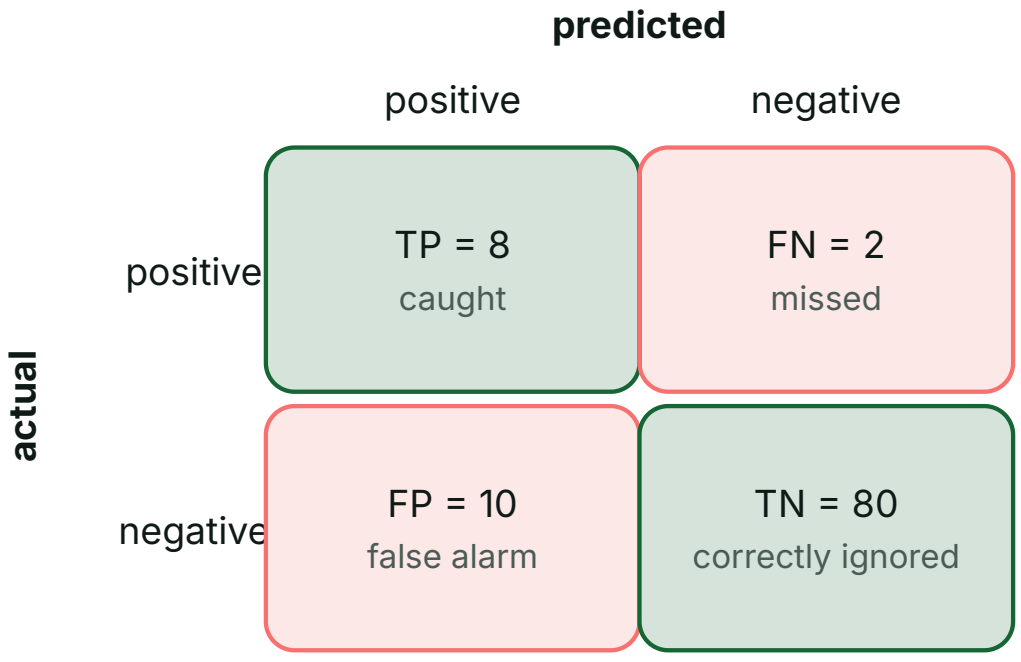


Figure 9. The screening-test example from this lesson. Green cells are correct; red cells are the two kinds of mistakes.

	Predicted positive	Predicted negative
Actually positive	**TP** — true positive correctly caught	**FN** — false negative missed ("Type II error")
Actually negative	**FP** — false positive false alarm ("Type I error")	**TN** — true negative correctly ignored

The word after "true/false" is what the model *predicted*; "true" means that prediction was right. So a false positive is a predicted positive that was wrong.

✎ WORKED EXAMPLE — A SCREENING TEST ON 100 PATIENTS

10 patients are actually sick, 90 are healthy. The model flags 18 patients: 8 of them are really sick.

	Predicted sick	Predicted healthy	Total
Actually sick	TP = 8	FN = 2	10
Actually healthy	FP = 10	TN = 80	90

We'll compute every metric below from these four numbers.

? MAKE SENSE OF IT

The situation: A clinic screens 100 patients; 10 are really sick. The model flags 18 for a follow-up test.

What the result tells you: It catches 8 of the 10 sick patients (recall 0.8), but only 8 of the 18 it flags are really sick (precision 0.44). Overall accuracy is 0.88.

What if…?

- **Flag more people (30)** and catch all 10: recall **1.0**, but precision falls to **0.33** and accuracy to 0.80. More follow-up tests, no missed patients.
- **Flag fewer (5)**, of whom 4 are sick: precision **0.80**, recall only **0.40** — yet accuracy *rises* to 0.93, because missing sick people barely dents accuracy.
- **Flag everyone:** recall 1.0, precision 0.10, accuracy 0.10.

Why it matters in ML: Precision and recall pull against each other as you move the threshold. Which matters more depends on costs: missing a cancer (low recall) is worse than an extra test (low precision); for spam filters it's the other way round.

3. Accuracy, precision, recall, specificity

PICTURE IT An airport security scanner can be tuned to beep at everything (it never misses a threat, but delays thousands of travellers) or almost nothing (smooth queues, real danger). Precision and recall put numbers on that trade-off.

Accuracy — what fraction of all predictions were right?

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{8 + 80}{100} = 0.88$$

Precision — of the ones we flagged, how many were really positive? ("When the model says yes, can I trust it?")

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{8}{8 + 10} \approx 0.44$$

Recall (also *sensitivity* or *true positive rate*) — of the real positives, how many did we catch? ("Does the model find what it should?")

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{8}{8 + 2} = 0.80$$

Specificity (true negative rate) — of the real negatives, how many did we correctly leave alone?

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{80}{90} \approx 0.89$$

◎ REMEMBER THEM WITH A FISHING NET

You cast a net to catch a particular kind of fish. **Recall**: what fraction of those fish in the lake ended up in your net? **Precision**: what fraction of the things in your net are those fish (and not old boots)? A huge net has great recall but poor precision; a tiny careful net has great precision but poor recall.

So our screening test catches 80% of sick patients (good recall), but only 44% of its alarms are real (low precision). Whether that's acceptable depends on the situation: for an initial screening followed by a more accurate test, it may be fine.

READ THE FORMULA **Precision and recall**

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

- **TP** — true positives — real positives we caught
- **FP** — false alarms — only precision is hurt by these
- **FN** — misses — only recall is hurt by these

Hover over (or tap) a part to highlight it in the formula.

4. The F1 score

To summarise precision and recall in one number, use their **harmonic mean**, the F1 score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,TP}{2\,TP + FP + FN}$$

For our example: $F_1 = \frac{2 \cdot 8}{2 \cdot 8 + 10 + 2} = \frac{16}{28} \approx 0.57$.

Why the harmonic mean rather than the ordinary average? It's dragged down by whichever value is smaller, so a model can't hide terrible precision behind great recall. A model with precision 1.0 and recall 0.01 has an ordinary average of about 0.5, but an F1 of only about 0.02.

WEIGHTING PRECISION VS RECALL

The F_β score lets you weight recall β times as much as precision: $F_\beta = \frac{(1+\beta^2)\,TP}{(1+\beta^2)\,TP + \beta^2 FN + FP}$. F_2 favours recall (useful for medical screening); $F_{0.5}$ favours precision (useful for spam filtering).

STEP-BY-STEP DERIVATION **Show that F1 equals $\frac{2TP}{2TP+FP+FN}$**

1 $F_1 = \frac{2PR}{P+R}, \qquad P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}$

Start from the harmonic mean of precision and recall.

2 $2PR = \frac{2\,TP^2}{(TP+FP)(TP+FN)}$

Multiply the two fractions.

3 $P + R = \frac{TP(TP+FN) + TP(TP+FP)}{(TP+FP)(TP+FN)} = \frac{TP\,(2TP+FP+FN)}{(TP+FP)(TP+FN)}$

Add them over a common denominator.

4 $F_1 = \frac{2\,TP^2}{TP\,(2TP+FP+FN)} = \frac{2\,TP}{2\,TP+FP+FN}$

Divide: the common denominators cancel, then one TP cancels.

5. The threshold trade-off

Most classifiers output a score or probability (Lesson 8). Turning it into a yes/no decision needs a **threshold**. Moving the threshold trades one kind of mistake for the other:

- **Lower threshold** → flag more cases → more TPs *and* more FPs → recall ↑, precision ↓.
- **Higher threshold** → flag fewer cases → fewer FPs *but* more FNs → precision ↑, recall ↓.

There is no threshold that is "correct" in general; you choose it based on the cost of each type of mistake.

6. Try it: move the threshold

The model gives each example a score from 0 to 1. Purple bars show the score distribution of real negatives, green bars of real positives. They overlap — as in real life — so no threshold is perfect. Drag the threshold and watch the confusion matrix, the metrics, and the point on the ROC curve (right) change.

Interactive demo — open *lesson-09.html* on the website to try it.

7. ROC curves and AUC

Instead of picking one threshold, we can look at *all* of them at once. For each threshold, compute:

$$\text{TPR} = \text{Recall} = \frac{TP}{TP + FN} \qquad \text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

Plotting TPR (up) against FPR (across) for every threshold gives the **ROC curve** (receiver operating characteristic, a name from radar engineering).

- The bottom-left corner $(0, 0)$ is threshold = 1: flag nothing.
- The top-right corner $(1, 1)$ is threshold = 0: flag everything.
- A perfect model reaches the top-left corner $(0, 1)$: all positives caught, no false alarms.
- Random guessing gives the diagonal line.

The **area under the ROC curve (AUC)** summarises the whole curve in one number from 0.5 (random) to 1.0 (perfect). It has a lovely interpretation:

≡ WHAT AUC MEANS

AUC is the probability that the model gives a randomly chosen positive example a *higher score* than a randomly chosen negative example.

✎ WORKED EXAMPLE — AUC BY COUNTING PAIRS

Scores for 3 positives: $0.9, 0.7, 0.4$. Scores for 3 negatives: $0.8, 0.3, 0.2$.

There are $3 \times 3 = 9$ positive–negative pairs. Count the pairs where the positive scores higher:

- 0.9 beats $0.8, 0.3, 0.2$ → 3
- 0.7 beats $0.3, 0.2$ (not 0.8) → 2
- 0.4 beats $0.3, 0.2$ → 2

AUC $= 7/9 \approx 0.78$. The model ranks a positive above a negative 78% of the time.

? MAKE SENSE OF IT

The situation: A fraud team only has time to check the riskiest transactions first. What matters is whether the model *ranks* fraud above normal transactions.

What the result tells you: AUC 0.78: pick one fraud and one normal transaction at random, and 78% of the time the model gives the fraud the higher score.

What if...?

- **The weakest positive improves** (0.4 → 0.85): it now beats the 0.8 negative too, and AUC rises to **8/9 ≈ 0.89**.
- **A negative scores 0.95** (instead of 0.2): AUC falls to **4/9 ≈ 0.44** — worse than guessing.
- **Add 0.05 to every score,** or square them all: AUC doesn't change, because the order is the same.

Why it matters in ML: AUC measures ranking quality across all thresholds, so it's good for comparing models before you've chosen a threshold. It says nothing about whether the scores are well-calibrated probabilities.

AUC is useful because it doesn't depend on any particular threshold. But when positives are very rare, the ROC curve can look flattering, because FPR divides by the huge number of negatives.

ADVANCED · OPTIONAL **Why AUC equals a ranking probability** ▾ hide

Sweep the threshold from high to low. Each time it passes a positive, the ROC curve steps *up* by $1/P$; each time it passes a negative, it steps *right* by $1/N$. The area added by a negative's right-step is $\frac{1}{N} \times$(fraction of positives already passed), i.e. $\frac{1}{N} \times \frac{\#\{\text{positives scored above it}\}}{P}$. Summing over all negatives:

$$\text{AUC} = \frac{1}{PN} \sum_{\text{neg } j} \sum_{\text{pos } i} 1[s_i > s_j] = P(\text{random positive outscores random negative})$$

(ties count ½). This is the Mann–Whitney U statistic divided by PN — so AUC depends only on the *ranking* of scores, not their values, which is why it ignores calibration.

8. Precision–recall curves

When positives are rare (fraud, disease, defects), a **precision–recall curve** — precision (up) against recall (across) for every threshold — is usually more informative. Its summary is **average precision (AP)**, a weighted average of precision across recall levels. A random model's AP equals the fraction of positives (e.g. 0.01 for 1% fraud), so the baseline is much lower than ROC's 0.5, making differences between models easier to see.

9. More than two classes

With K classes the confusion matrix is $K \times K$: row = true class, column = predicted class, and the diagonal holds the correct predictions. Off-diagonal cells show which classes get confused (e.g. "3" mistaken for "8"). Precision, recall and F1 are computed per class, then averaged:

- **Macro average**: plain average over classes — every class counts equally.
- **Weighted average**: weighted by how many examples each class has.

10. Metrics for regression

For predicting numbers, we measure the size of the errors $e_i = \hat{y}_i - y_i$:

Metric	Formula	Notes
MAE — mean absolute error	$\frac{1}{n}\sum \lvert e_i \rvert$	In the target's units; robust to outliers
MSE — mean squared error	$\frac{1}{n}\sum e_i^2$	Punishes large errors; squared units
RMSE — root MSE	$\sqrt{\text{MSE}}$	Back in the target's units
R^2	$1 - \frac{\sum e_i^2}{\sum (y_i - \bar{y})^2}$	Fraction of variance explained (Lesson 6)

✎ WORKED EXAMPLE

True $y = 3, 5, 2, 7$; predicted $\hat{y} = 2.5, 5, 4, 8$. Errors: $-0.5, 0, 2, 1$.

MAE $= (0.5 + 0 + 2 + 1)/4 = 0.875$.
MSE $= (0.25 + 0 + 4 + 1)/4 = 1.3125$, RMSE $= \sqrt{1.3125} \approx 1.15$.
$\bar{y} = 4.25$, $\sum (y_i - \bar{y})^2 = 14.75$, so $R^2 = 1 - 5.25/14.75 \approx 0.64$.

Notice RMSE (1.15) > MAE (0.875): the single error of 2 counts extra in RMSE. If you care a lot about avoiding big misses, use RMSE; if every unit of error costs the same, use MAE.

? MAKE SENSE OF IT

The situation: A delivery app predicts arrival times in minutes for four orders.

What the result tells you: MAE 0.875: on average the estimate is off by under a minute. RMSE 1.15 is larger because the one 2-minute miss counts extra.

What if...?

- **One bad miss** (predict 8 for the order that took 2): MAE becomes 1.875, RMSE **3.05**, and R^2 drops to **−1.53** — worse than predicting the average for every order.
- **Predict the average (4.25) for everyone:** $R^2 = 0$, MAE 1.75.

Why it matters in ML: MAE treats every minute of error the same; RMSE punishes big misses. A large gap between them tells you a few predictions are badly wrong — worth investigating.

11. Choosing the right metric

Situation	Focus on
Balanced classes, mistakes cost the same	Accuracy
Missing a positive is very costly (disease, fraud)	Recall (with acceptable precision), F_2
False alarms are very costly (spam filter, legal flags)	Precision, $F_{0.5}$
Need one balanced number for imbalanced data	F1, average precision
Comparing models independent of threshold	ROC-AUC (or AP if positives are rare)
Regression, all errors equally bad	MAE
Regression, big errors especially bad	RMSE

! ALWAYS COMPARE TO A BASELINE

Before celebrating a score, compute what a trivial model gets: always predicting the most common class, or always predicting the mean. A 95% accurate model is unimpressive if the baseline is 94%. And always measure on data the model hasn't trained on (Lesson 5).

12. In Python

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
y = 1 - y        # make "malignant" the positive class (1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_train,
y_train)
proba = model.predict_proba(X_test)[:, 1]          # P(malignant)

for threshold in [0.5, 0.2]:
    pred = (proba ⩾ threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
    print(f"threshold {threshold}: TP={tp} FN={fn} FP={fp} TN={tn}  "
          f"precision={tp/(tp+fp):.3f} recall={tp/(tp+fn):.3f}")

print("ROC-AUC:", round(roc_auc_score(y_test, proba), 3))
print(classification_report(y_test, (proba ⩾ 0.5).astype(int), target_names=["benign",
"malignant"]))
```

Output:

```
threshold 0.5: TP=51 FN=2 FP=0 TN=90  precision=1.000 recall=0.962
threshold 0.2: TP=52 FN=1 FP=9 TN=81  precision=0.852 recall=0.981
ROC-AUC: 0.996
              precision    recall  f1-score   support

      benign       0.98      1.00      0.99        90
   malignant       1.00      0.96      0.98        53

    accuracy                           0.99       143
   macro avg       0.99      0.98      0.98       143
weighted avg       0.99      0.99      0.99       143
```

At the default threshold of 0.5 the model missed 2 malignant tumours. Lowering the threshold to 0.2 missed only 1 (recall 0.96 → 0.98) but raised 9 false alarms (precision 1.00 → 0.85) — exactly the trade-off from the demo. For cancer screening, many doctors would accept those extra follow-up tests. (Splitting with different labels gives a slightly different test set than Lesson 8, so the numbers differ a little from there.)

13. Running example: Chennai Homes

To evaluate honestly with so little data, we use cross-validated predictions: every house is predicted by a model that didn't see it. The model caught all 6 fast sellers (recall 1.0) and raised one false alarm (precision 0.86).

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import confusion_matrix, precision_score, recall_score
clf = make_pipeline(StandardScaler(), LogisticRegression())
pred = cross_val_predict(clf, df[["dist_km", "age_yrs"]], df["sold_fast"], cv=4)
# honest predictions
tn, fp, fn, tp = confusion_matrix(df["sold_fast"], pred).ravel()
print("TP", tp, "FP", fp, "FN", fn, "TN", tn)
print("precision", round(precision_score(df["sold_fast"], pred), 2), "recall",
round(recall_score(df["sold_fast"], pred), 2))
```

Output:

```
TP 6 FP 1 FN 0 TN 5
precision 0.86 recall 1.0
```

⚙ WHERE THIS IS USED

- Choosing alert thresholds for fraud and disease screening
- Comparing models before deployment
- Monitoring models in production

Choose it when: Always — choose the metric that reflects the real cost of each mistake.

× MISCONCEPTIONS TO UNLEARN

MYTH Accuracy is the most important metric.

REALITY On imbalanced data a useless model can reach 99% accuracy. Choose the metric that matches the real cost of mistakes.

MYTH There is one correct threshold (0.5).

REALITY The threshold is a business decision that trades precision against recall.

RESPONSIBLE AI · EVALUATE FOR EVERY GROUP, NOT JUST ON AVERAGE

An overall accuracy can hide large differences between groups. Report precision and recall for each relevant subgroup. Fairness has several competing definitions — equal false-positive rates, equal true-positive rates, calibrated scores within each group — and research by Chouldechova (2017) and by Kleinberg, Mullainathan and Raghavan (2016) proved that when base rates differ between groups, these criteria cannot all hold at once. Choosing which one matters is a decision for people, not the algorithm.

14. Questions beginners often ask

Q. Which single metric should I report?

The one that matches the real cost of mistakes. If unsure, report several: precision, recall, F1 and ROC-AUC — plus a baseline.

Q. Why is my accuracy high but the model useless?

Probably class imbalance: predicting the majority class already gives high accuracy. Look at recall and precision for the minority class.

Q. ROC-AUC or precision–recall?

ROC-AUC for balanced classes; precision–recall (average precision) when positives are rare.

! COMMON MISTAKES TO AVOID

- Reporting only accuracy on imbalanced data.
- Choosing the threshold on the test set — that's tuning, so use validation data.
- Forgetting that scikit-learn's error scorers are negated (e.g. `neg_mean_squared_error`).

! INTERVIEW CORNER

Don't confuse

- **Precision vs recall** — Precision = purity of the flags; recall = coverage of the positives.
- **Sensitivity vs specificity** — Sensitive tests catch the sick; specific tests clear the healthy.
- **Type I vs Type II error** — Type I: the boy who cried wolf. Type II: the wolf nobody noticed.
- **ROC curve vs precision–recall curve** — Rare positives? Read the PR curve.
- **Micro vs macro averaging** — Macro is fair to small classes.

Hidden questions — answer in your head first, then open.

▼ **For cancer screening, would you optimise precision or recall? For a spam filter?**

The trap: Answering "accuracy".

Cancer screening: recall — missing a cancer is far worse than an extra follow-up test. Spam filter: precision — losing a real email in the spam folder is worse than letting some spam through.

▼ **Your fraud model has ROC-AUC 0.95 but the fraud team says most alerts are false. How can both be true?**

The trap: Trusting ROC-AUC on imbalanced data.

FPR is divided by the huge number of normal transactions, so it looks tiny. With 10 frauds in 1,000 transactions, catching 9 at a 5% FPR still means about 49.5 false alarms: precision is about 15%. Use precision–recall curves and PR-AUC for rare positives.

▼ **A model has ROC-AUC 0.3. Is it useless?**

The trap: Saying yes.

It's worse than random, which means it ranks consistently in the wrong direction. Flip its scores and AUC becomes 0.7. Often a sign of inverted labels somewhere in the pipeline.

▼ **Does changing the threshold change ROC-AUC?**

The trap: Thinking AUC depends on the operating point.

No. AUC summarises all thresholds at once and depends only on how the scores rank positives against negatives. Precision, recall, F1 and accuracy do change with the

threshold.

15. Key takeaways

- The **confusion matrix** (TP, FP, FN, TN) shows exactly which mistakes a classifier makes.
- **Precision** $= \frac{TP}{TP+FP}$: trust in positive predictions. **Recall** $= \frac{TP}{TP+FN}$: share of positives found. **F1** balances them.
- Accuracy misleads on imbalanced data — always compare with a trivial baseline.
- The **threshold** trades precision against recall; **ROC-AUC** and **average precision** summarise all thresholds.
- Regression: **MAE**, **RMSE** (punishes big errors) and R^2.

16. Check your understanding

1. TP = 30, FP = 10, FN = 20. What is the precision?

a. 0.60

b. 0.75

c. 0.50

2. A fraud detector must miss as few frauds as possible. Which metric matters most?

a. Precision

b. Specificity

c. Recall

3. You lower the classification threshold from 0.5 to 0.3. What usually happens?

a. Recall goes up, precision goes down

b. Precision goes up, recall goes down

c. Both go up

4. A model has ROC-AUC = 0.5. What does that suggest?

a. It's perfect

b. It ranks no better than random

c. It's right half of the time on each class

5. Errors are 1, −1, 1, −5. Which is larger, MAE or RMSE?

a. MAE

b. They are equal

c. RMSE

17. Going deeper: comparing models honestly

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above teaches what each metric means. This section is about using them for decisions: how sure you can be about a test-set score, how to tell whether one model is *really* better than another, how to pick a threshold from the actual cost of mistakes, and why ROC-AUC can look reassuring while a rare-class model is struggling.

You'll need: this lesson, plus the bootstrap and hypothesis tests from Lesson 4's "Going deeper".

Every score needs an error bar

A test set is a sample, so a score computed on it is an estimate. With 171 test cases, a single unusual case moves accuracy by about 0.6 percentage points. For accuracy there's a formula (Lesson 4's "Going deeper": $\sqrt{p(1-p)/n}$); for AUC, F1 or precision, use the **bootstrap**: resample the test cases with replacement, recompute, repeat. When comparing two models, bootstrap the *difference* on the same resampled cases, because the two models' scores go up and down together.

Here logistic regression and a depth-3 decision tree (Lesson 13) are trained on the breast-cancer data and compared on the same 171 test cases:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(Xtr,
ytr)
tree = DecisionTreeClassifier(max_depth=3, random_state=0).fit(Xtr, ytr)
p_lr, p_tr = logreg.predict_proba(Xte)[:, 1], tree.predict_proba(Xte)[:, 1]

rng = np.random.default_rng(0)
boot = []
for _ in range(2000):
    i = rng.integers(0, len(yte), len(yte))            # resample test cases with
replacement
    if len(np.unique(yte[i])) == 2:
        boot.append(roc_auc_score(yte[i], p_lr[i]) - roc_auc_score(yte[i], p_tr[i]))
print("AUC: logistic", round(roc_auc_score(yte, p_lr), 3), " tree",
round(roc_auc_score(yte, p_tr), 3))
print("95% bootstrap interval for the difference:", np.round(np.percentile(boot, [2.5,
97.5]), 3))
```

Output:

```
AUC: logistic 0.996  tree 0.904
95% bootstrap interval for the difference: [0.046 0.15 ]
```

Logistic regression's AUC is 0.996 and the tree's is 0.904. The 95% interval for the difference, 0.046 to 0.15, doesn't include 0, so the gap is unlikely to be a fluke of this test set. Had the interval crossed 0, the honest conclusion would be "no clear winner".

Paired tests: McNemar’s test

For accuracy on a shared test set, the standard test looks only at the cases where the two models **disagree**. Cases both get right, or both get wrong, say nothing about which is better. If the models were equally good, each disagreement would be a coin flip, so **McNemar's test** checks whether the split of disagreements is suspiciously lopsided:

```
from scipy import stats

# McNemar's test: do the two models make different numbers of mistakes on the SAME
cases?
right_lr, right_tr = logreg.predict(Xte) == yte, tree.predict(Xte) == yte
b = np.sum(right_lr & ~right_tr)          # logistic right, tree wrong
c = np.sum(~right_lr & right_tr)          # tree right, logistic wrong
print("accuracy: logistic", round(right_lr.mean(), 3), " tree", round(right_tr.mean(),
3))
print("cases only logistic got right:", b, "  only the tree got right:", c)
print("McNemar exact p-value:", round(stats.binomtest(b, b + c, 0.5).pvalue, 4))
```

Output:

```
accuracy: logistic 0.959  tree 0.901
cases only logistic got right: 11   only the tree got right: 1
McNemar exact p-value: 0.0063
```

Logistic regression got 11 cases right that the tree missed; the tree got only 1 right that logistic regression missed. If they were equally good, a split of 11 to 1 or more lopsided would happen with probability 0.0063, so we can be confident logistic regression is more accurate here. Two models with accuracies of 0.959 and 0.901 might or might not differ significantly on a *different* test set: it depends on how many cases they disagree on. When you compare models with cross-validation instead, the folds overlap, which makes ordinary t-tests over-confident; the corrected resampled t-test or the 5×2 cross-validation test are designed for that.

Choosing the threshold from costs

The threshold is a business decision, not a statistics one. If a false negative costs c_{FN} and a false positive costs c_{FP}, flagging a case with probability p costs $c_{FP}(1-p)$ on average and not flagging it costs $c_{FN}\,p$. Flag whenever the second is bigger:

$$p > \frac{c_{FP}}{c_{FP} + c_{FN}}$$

This only works if the probabilities are **calibrated** (Lesson 8's "Going deeper"). Suppose missing a malignant tumour costs 20 and a false alarm (an unnecessary follow-up test) costs 1. Then the threshold should be $1/21 \approx 0.048$, nowhere near 0.5:

```
# Choosing the threshold from costs: a missed cancer (false negative) costs 20, a false
alarm costs 1
y_bad = 1 - yte                            # in this dataset 0 = malignant, so flip: 1 =
malignant
p_bad = 1 - p_lr
for t in [0.5, 0.2, 0.1, 0.05]:
    flag = p_bad ⩾ t
    fn, fp = np.sum(~flag & (y_bad == 1)), np.sum(flag & (y_bad == 0))
    print(f"threshold {t:4.2f}: missed {fn:2d}, false alarms {fp:2d}, total cost {20 *
fn + fp}")
print("theory: flag when p is above cost_FP / (cost_FP + cost_FN), which is", round(1 /
21, 3))
```

Output:

```
threshold 0.50: missed  3, false alarms  4, total cost 64
threshold 0.20: missed  2, false alarms  7, total cost 47
threshold 0.10: missed  1, false alarms 11, total cost 31
threshold 0.05: missed  0, false alarms 18, total cost 18
theory: flag when p is above cost_FP / (cost_FP + cost_FN), which is 0.048
```

At the default 0.5, three malignant cases are missed, for a total cost of 64. Lowering the threshold to 0.05, close to the theoretical 0.048, catches all of them at the price of 18 false alarms: total cost 18, less than a third. In a real project you'd choose the threshold on validation data and confirm it on the test set, rather than tuning on the test set as this small example does.

ROC-AUC vs average precision when positives are rare

ROC curves plot the true-positive rate against the false-positive rate. Both are rates *within* a class, so the ROC curve doesn't change when one class becomes rarer. Precision does change: with few positives, even a small false-positive rate produces many false alarms compared with true hits. Below, the model's score distributions are identical in every row; only the share of positives changes:

```
from sklearn.metrics import average_precision_score

# Same model quality, rarer positives: scores of positives ~ N(1.5, 1), negatives ~ N(0,
1)
rng = np.random.default_rng(1)
for n_pos in [10000, 1000, 100]:
    s = np.r_[rng.normal(1.5, 1, n_pos), rng.normal(0, 1, 10000)]
    t = np.r_[np.ones(n_pos), np.zeros(10000)]
    print(f"positives {t.mean():6.1%}: ROC-AUC {roc_auc_score(t, s):.3f}   average
precision {average_precision_score(t, s):.3f}   (random model's AP {t.mean():.3f})")
```

Output:

```
positives  50.0%: ROC-AUC 0.857   average precision 0.855   (random model's AP 0.500)
positives   9.1%: ROC-AUC 0.860   average precision 0.451   (random model's AP 0.091)
positives   1.0%: ROC-AUC 0.867   average precision 0.136   (random model's AP 0.010)
```

ROC-AUC stays about 0.86 in every row. Average precision (the area under the precision–recall curve) falls from 0.855 to 0.136 as positives go from 50% to 1%. Neither number is wrong; they answer different questions. ROC-AUC asks "how well does the model rank a random positive above a random negative?" Average precision asks "when the model raises an alert, how often is it right?" For rare-event problems such as fraud, defects or rare diseases, the second is usually what the people using the alerts experience. Always compare average precision with its baseline, the share of positives (a random model's AP).

A reporting checklist

Report	Why
The metric chosen *before* looking at results	Avoids picking whichever metric looks best
A simple baseline (majority class, last value, linear model)	Shows whether the complex model earns its keep
A confidence interval	A test set is a sample
A paired comparison between models	Overlapping intervals don't prove "no difference"; paired tests are more sensitive
The threshold and the reason for it	Precision and recall depend on it
Results by subgroup	An average can hide groups the model fails (Lesson 21)

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Model A scores 0.851 accuracy and model B 0.846 on the same test set. Which do you ship?**

The trap: Picking A because the number is bigger.

First check whether the difference is real: use a paired test on the same cases (McNemar, or a paired bootstrap of the difference). With a few thousand test cases, 0.5 points is often within noise. If there's no clear difference, choose on other grounds: simplicity, speed, cost of errors on the business metric, calibration, fairness across groups, or how easy each is to maintain and explain.

▼ **How do you choose a classification threshold?**

The trap: "Use 0.5" or "maximise F1" by default.

Ideally from costs: with calibrated probabilities, flag when $p > c_{FP}/(c_{FP} + c_{FN})$. Without clear costs, use an operating constraint, such as recall of at least 95% or at most 100 alerts per day, and choose the threshold on validation data, then confirm on the test set. Revisit it when the base rate changes, since precision depends on it.

▼ **When would you report average precision (PR-AUC) instead of ROC-AUC?**

The trap: Saying ROC-AUC is "wrong" for imbalanced data.

When positives are rare and what matters is the quality of the alerts. ROC-AUC is unaffected by class balance, so it can look high while most alerts are false; average precision reflects that directly. Report AP together with its baseline (the positive rate). ROC-AUC is still useful for ranking quality and for comparing across datasets with different balance.

▼ **Why can't you use an ordinary t-test on the 10 fold scores of 10-fold cross-validation to compare two models?**

The trap: Thinking the 10 scores are independent samples.

The training sets of different folds overlap by about 89%, so the fold scores are correlated and their variance is underestimated. A standard t-test then claims significance far too often. Use the corrected resampled t-test (Nadeau–Bengio), the 5×2 cross-validation test, or repeated CV with a correction, or compare on a single large held-out set with a paired test.

Check your understanding: going deeper

1. Two models disagree on 30 test cases: model A is right on 16 of them and model B on 14. What does McNemar's test suggest?

a. A is clearly better

b. No real evidence of a difference

c. B is clearly better

2. A false negative costs 9 and a false positive costs 1. With calibrated probabilities, flag cases with p above...

a. 0.5

b. 0.9

c. 0.1

3. Positives become 10 times rarer, but the model's score distributions for each class stay the same. ROC-AUC will...

a. Stay about the same

b. Fall sharply

c. Rise

4. On a dataset with 2% positives, a model's average precision is 0.10. Compared with random guessing, that is...

a. Worse

b. About 5 times better

c. Exactly random

5. To compare two models' AUCs with the bootstrap, you should...

a. Bootstrap each model separately and see if the intervals overlap

b. Resample the same cases for both models and bootstrap the difference

c. Average the two AUCs

18. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

TP = 40, FP = 10, FN = 20, TN = 130. Compute accuracy, precision, recall, F1 and specificity.

WORKING

EXERCISE 2 Concepts

Which metric matters most? (a) An airport scanner for dangerous items; (b) a system that automatically blocks bank accounts; (c) comparing models on a balanced dataset.

WORKING

EXERCISE 3 By hand

Positive scores $0.8, 0.6$; negative scores $0.7, 0.2$. Compute the AUC by counting pairs.

WORKING

EXERCISE 4 By hand

Compare MAE and RMSE for errors $2, -2, 2, -2$ and for errors $0, 0, 0, 8$.

WORKING

EXERCISE 5 Intermediate

Two models are compared on 500 test cases. Model A alone is right on 25 of them and model B alone on 10. Under the null hypothesis each disagreement is a coin flip. What is the expected count for each model, and the McNemar statistic $(b-c)^2/(b+c)$?

WORKING

EXERCISE 6 Intermediate

A missed fraud costs ₹5,000 and checking a false alert costs ₹200. What threshold minimises expected cost for a calibrated model?

WORKING

EXERCISE 7 Intermediate

A model flags 200 cases. 50 of the 200 are truly positive, and there are 80 positives in total. Compute precision and recall.

WORKING

EXERCISE 8 Code · intermediate

Using the bootstrap code, compute a 95% interval for the logistic model's *accuracy* alone. Compare its width with the formula $\pm 1.96\sqrt{p(1-p)/n}$ for $n = 171$.

WORKING

NOTES

PART 2 · LEARNING FROM LABELLED DATA · ~50 MIN

Lesson 10: Overfitting, Regularization & Bias–Variance

Why models overfit and how L1/L2 penalties and the bias–variance trade-off help.

BY THE END OF THIS LESSON YOU CAN

- Explain bias, variance and the trade-off between them
- Diagnose underfitting vs overfitting and pick a fix
- Write and compute ridge (L2) and lasso (L1) penalties
- Choose the regularization strength with cross-validation

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 6: Linear Regression — Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ and chooses the weights that minimise the mean squared error (least squares). R^2 measures the share of variance explained.
- Lesson 7: Gradient Descent — Gradient descent trains almost any model: compute the gradient of the loss, step against it, repeat. The learning rate controls the step size; too large diverges. Mini-batches make it fast on big data.

Where this fits: Explains why models overfit and gives the main cure. This closes Part 2: you can now build, train, evaluate and regularise a model.

1. The problem: flexible models overfit

In Lesson 5 you watched a degree-11 polynomial thread through every training point and fail on new data. More flexible models can fit more patterns — including patterns that are just noise. But we don't want to give up flexibility, because real relationships are often complicated.

This lesson answers two questions. **Why** does overfitting happen? (The bias–variance trade-off.) And **how** can we keep a flexible model from overfitting? (Regularization — one of the most widely used ideas in all of ML.)

2. Bias and variance

PICTURE IT Two students prepare for an exam. One memorises last year's paper word for word (high variance: lost if the questions change). The other learns only one rule of thumb for everything (high bias: wrong in a predictable way). Good learning sits between them.

Imagine you could collect many different training sets from the same source, and train the same kind of model on each. You'd get a slightly different model each time. Two things can go wrong:

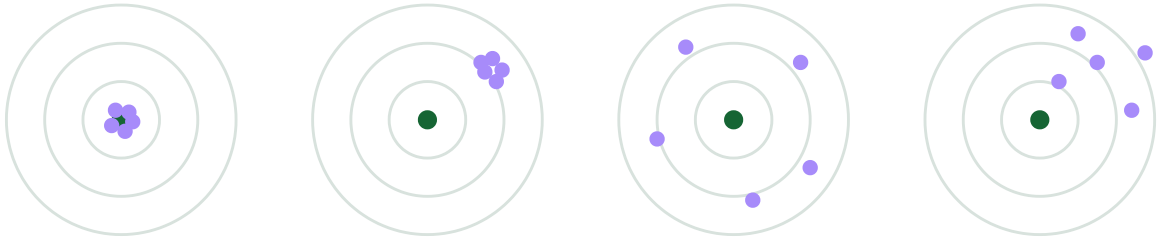


Figure 10. Each dot is a model trained on a different training set; the centre is the truth. Bias is being off-centre on average; variance is the scatter.

- **Bias** — error from wrong assumptions. Even averaged over all those training sets, the model misses the true pattern. A straight line fitted to a curve has high bias. High bias → **underfitting**.
- **Variance** — error from sensitivity to the particular training set. The model changes a lot depending on which examples it happened to see, because it's fitting their noise. A degree-11 polynomial has high variance. High variance → **overfitting**.

◎ ANALOGY: THROWING DARTS

Each training set gives you one dart throw; the bullseye is the true pattern.

- **Low bias, low variance**: darts tightly grouped on the bullseye. The goal.
- **High bias, low variance**: tightly grouped, but off-centre. Consistently wrong.
- **Low bias, high variance**: centred on the bullseye on average, but scattered all over.
- **High bias, high variance**: scattered and off-centre.

For squared-error loss, the expected error on a new example splits exactly into three parts:

$$\mathbb{E}\left[(y-\hat{y})^2\right] = \underbrace{\text{Bias}^2}_{\text{wrong assumptions}} + \underbrace{\text{Variance}}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

The noise σ^2 is randomness in the data itself; no model can remove it. The other two we can trade against each other.

ADVANCED · OPTIONAL **Proof of the bias–variance decomposition** ▾ hide

Assume $y = f(x) + \varepsilon$ with noise $E[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, independent of the training set used to fit $\hat{f}$. At a fixed x, write $\bar{f} = E[\hat{f}(x)]$ (averaging over training sets). Then

$$E\big[(y - \hat{f})^2\big] = E\big[(f + \varepsilon - \bar{f} + \bar{f} - \hat{f})^2\big].$$

Expand the square. The cross terms vanish: $E[\varepsilon] = 0$ and ε is independent of $\hat{f}$, and $E[\bar{f} - \hat{f}] = 0$ by definition. What remains is

$$E\big[(y - \hat{f})^2\big] = \underbrace{\sigma^2}_{\text{noise}} + \underbrace{(f - \bar{f})^2}_{\text{bias}^2} + \underbrace{E[(\hat{f} - \bar{f})^2]}_{\text{variance}}.$$

No model can remove the noise term; regularisation trades a little more bias for less variance.

3. The trade-off

As model complexity grows:

- bias **falls** (the model can represent more patterns), but
- variance **rises** (it can also fit more noise).

Total error is lowest somewhere in between — at the bottom of the U-shaped test-error curve you saw in Lesson 5's demo.

Symptom	Diagnosis	What helps
Training error high, test error high and similar	High bias (underfitting)	More flexible model, better features, less regularization
Training error low, test error much higher	High variance (overfitting)	More data, simpler model, **more regularization**, fewer features

MORE DATA REDUCES VARIANCE

With more training examples, random noise averages out, so a flexible model has less room to chase it. That's why large datasets let us use very complex models. When getting more data isn't possible, regularization is the next best tool.

4. Regularization: penalise complexity

Overfitted models usually have **large weights**: to bend sharply through individual noisy points, a polynomial needs huge coefficients that cancel each other out. Regularization adds a penalty for large weights to the loss:

$$\text{Loss}_{\text{regularized}} = \underbrace{\text{data loss (e.g. MSE)}}_{\text{fit the data}} + \underbrace{\lambda \cdot \text{penalty}(\mathbf{w})}_{\text{keep weights small}}$$

λ ("lambda", called `alpha` in scikit-learn) controls the balance:

- $\lambda = 0$: no penalty — ordinary least squares.
- Small λ: a gentle push towards smaller weights.
- Large λ: weights forced towards zero — the model becomes very simple and may underfit.

Regularization deliberately adds a little bias in exchange for a big reduction in variance. The bias term b is normally *not* penalised — it only shifts predictions up or down and doesn't make the model wiggly.

5. Ridge regression (L2)

Ridge penalises the sum of squared weights — the squared L2 norm from Lesson 2:

$$L(\mathbf{w}) = \sum_{i=1}^{n} (y_i - \mathbf{w} \cdot \mathbf{x}_i - b)^2 + \lambda \sum_{j=1}^{d} w_j^2$$

Setting the gradient to zero gives a closed-form solution, just like the normal equation but with λ added on the diagonal:

$$\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

Adding λI has a bonus: the matrix always has an inverse, so ridge also fixes the problem of perfectly correlated features from Lesson 6.

✎ WORKED EXAMPLE — WATCH THE WEIGHT SHRINK

One feature, no intercept: $x = 1, 2, 3$ and $y = 2, 4, 7$. With one weight, the ridge formula becomes

$$w = \frac{\sum x_i y_i}{\sum x_i^2 + \lambda} = \frac{1 \cdot 2 + 2 \cdot 4 + 3 \cdot 7}{1 + 4 + 9 + \lambda} = \frac{31}{14 + \lambda}$$

λ	0	1	10	100
w	2.214	2.067	1.292	0.272

As λ grows, the weight is pulled towards zero. It never reaches exactly zero — that's a property of L2.

? MAKE SENSE OF IT

The situation: With only three data points, a model can chase noise. Ridge adds a cost for large weights, a bit like telling the model "be cautious unless the data insists".

What the result tells you: Without a penalty the weight is 2.214. As λ grows, the weight shrinks — to 1.292 at $\lambda = 10$ and 0.272 at $\lambda = 100$.

What if…?

- **Ten times more data** (each point repeated 10 times): at $\lambda = 10$ the weight is **2.067** instead of 1.292. With more evidence, the same penalty matters less.
- **Measure x in units ten times larger** ($x = 0.1, 0.2, 0.3$): the unpenalised weight becomes 22.1 and $\sum x^2$ only 0.14, so $\lambda = 1$ now shrinks the weight to **2.72** — just 12% of its value, compared with 93% before. The penalty's strength depends on feature scale — standardise first.

Why it matters in ML: Regularisation trades a little bias for less variance. Choose λ by cross-validation, and always scale features first so the penalty treats them fairly.

With gradient descent, the penalty adds $2\lambda w_j$ to each weight's gradient. Every update then shrinks the weight slightly before applying the data gradient — which is why L2 regularization is also called **weight decay**.

STEP-BY-STEP DERIVATION

Derive the one-feature ridge solution

1 $L(w) = \sum_i (y_i - w x_i)^2 + \lambda w^2$

Ridge loss with one feature and no intercept.

2 $\frac{dL}{dw} = -2 \sum_i x_i (y_i - w x_i) + 2\lambda w$

Chain rule on each squared term, plus the derivative of λw^2.

3 $-\sum_i x_i y_i + w \sum_i x_i^2 + \lambda w = 0$

Set to zero and divide by 2.

4 $w\left(\sum_i x_i^2 + \lambda\right) = \sum_i x_i y_i$

Collect the w terms.

5 $w = \frac{\sum_i x_i y_i}{\sum_i x_i^2 + \lambda}$

Divide. With $\lambda = 0$ it's ordinary least squares; a larger λ makes the denominator bigger and w smaller.

READ THE FORMULA

Ridge regression loss

$$\sum_i (y_i - \mathbf{w} \cdot \mathbf{x}_i - b)^2 + \lambda \sum_j w_j^2$$

- **Σ (yᵢ − …)²** — how badly we fit the data
- **λ** — how much we care about simplicity
- **Σ wⱼ²** — how big the weights are — the penalty

Hover over (or tap) a part to highlight it in the formula.

6. Lasso regression (L1)

Lasso penalises the sum of *absolute* weights — the L1 norm:

$$L(\mathbf{w}) = \frac{1}{2n}\sum_{i=1}^{n}(y_i - \mathbf{w}\cdot\mathbf{x}_i - b)^2 + \lambda\sum_{j=1}^{d}|w_j|$$

(This is the scaling scikit-learn uses.) The key difference from ridge: lasso drives many weights to **exactly zero**. Those features are effectively removed, so lasso performs automatic **feature selection** and gives simpler, more interpretable models.

> ◎ WHY DOES L1 GIVE EXACT ZEROS?
>
> Look at the penalty's slope. For L2, the penalty w^2 has slope $2w$, which gets tiny as w approaches 0 — the push towards zero fades away, so weights get small but rarely reach 0. For L1, the penalty $|w|$ has slope ± 1 everywhere: a constant-strength push all the way to zero. If a feature isn't helping enough to resist that constant push, its weight lands on exactly 0.
>
> Geometrically: L2's "budget" region $w_1^2 + w_2^2 \le t$ is a circle; L1's $|w_1| + |w_2| \le t$ is a diamond with sharp corners on the axes. The best solution tends to touch the diamond at a corner — where one weight is zero.

ElasticNet mixes both penalties (scikit-learn's `l1_ratio` sets the mix). It's useful when you want sparsity but have groups of correlated features, where pure lasso tends to pick one arbitrarily.

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda\sum w_j^2$	$\lambda\sum \lvert w_j\rvert$
Effect on weights	Shrinks all smoothly	Sets many exactly to 0
Feature selection	No	Yes
Closed-form solution	Yes	No (iterative solvers)
Good when	Many features each matter a bit	Only a few features really matter

7. Try it: regularization tames a wild model

A degree-9 polynomial is fitted to 12 noisy training points using ridge regression. Slide λ from almost zero (wild overfitting) to very large (a nearly flat line that underfits). Watch the training and

test errors, and the size of the weights. Somewhere in between is the sweet spot.

Interactive demo — open *lesson-10.html* on the website to try it.

8. Always scale features before regularizing

The penalty treats all weights equally. But a weight's size depends on its feature's units: a feature measured in millimetres needs a weight 1000 times smaller than the same feature in metres. Without scaling, regularization would unfairly punish some features and barely touch others. **Standardise features first** (Lesson 4) — in a pipeline, so it's fitted on training data only (Lesson 5).

9. Choosing λ

λ is a **hyperparameter**: a setting you choose, rather than a parameter learned from the data. Pick it with cross-validation (Lesson 5):

1. Try a range of values spaced by factors, e.g. $0.01, 0.1, 1, 10, 100$.
2. For each, compute the cross-validation score on the training data.
3. Pick the λ with the best CV score, then retrain on all training data.

scikit-learn's `RidgeCV` and `LassoCV` do this automatically.

> **LOGISTIC REGRESSION IS REGULARIZED TOO**
>
> scikit-learn's `LogisticRegression` uses L2 regularization by default, controlled by `C`, which is the *inverse* strength: $C = 1/\lambda$. Smaller `C` means *stronger* regularization. Set `penalty="l1"` (with a compatible solver such as `"liblinear"` or `"saga"`) for lasso-style feature selection.

10. Other ways to fight overfitting

- **More training data** — the most reliable fix when available.
- **Simpler model** — fewer features, lower polynomial degree, shallower trees (Lesson 13).
- **Early stopping** — stop gradient descent when validation error starts rising (Lesson 7).
- **Ensembles** — averaging many models reduces variance (Lesson 14).
- **Dropout and data augmentation** — for neural networks (Lesson 19).

11. In Python

Using the diabetes dataset with all pairwise feature products added (65 features from 10), a model with lots of room to overfit:

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, RidgeCV
from sklearn.pipeline import make_pipeline

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

def build(reg):
    return make_pipeline(PolynomialFeatures(2, include_bias=False), StandardScaler(),
reg)

models = [("no regularization", LinearRegression())] + \
         [(f"Ridge alpha={a}", Ridge(alpha=a)) for a in [1, 10, 100]]
for name, reg in models:
    m = build(reg).fit(X_train, y_train)
    print(f"{name:20s} train R2 {m.score(X_train, y_train):.3f}  test R2
{m.score(X_test, y_test):.3f}")

for a in [0.1, 1, 5]:
    m = build(Lasso(alpha=a, max_iter=50000)).fit(X_train, y_train)
    coef = m[-1].coef_
    print(f"Lasso alpha={a}: test R2 {m.score(X_test, y_test):.3f}, non-zero weights
{np.sum(coef != 0)} of {coef.size}")

m = build(RidgeCV(alphas=np.logspace(-2, 3, 30))).fit(X_train, y_train)
print("best alpha", round(m[-1].alpha_, 2), "test R2", round(m.score(X_test, y_test),
3))
```

Output:

```
no regularization    train R2 0.606  test R2 0.416
Ridge alpha=1        train R2 0.601  test R2 0.456
Ridge alpha=10       train R2 0.590  test R2 0.503
Ridge alpha=100      train R2 0.561  test R2 0.498
Lasso alpha=0.1: test R2 0.479, non-zero weights 54 of 65
Lasso alpha=1: test R2 0.516, non-zero weights 36 of 65
Lasso alpha=5: test R2 0.488, non-zero weights 12 of 65
best alpha 92.37 test R2 0.5
```

Read it carefully. As regularization increases, the *training* score drops slightly (the model fits the training data less tightly) while the *test* score rises from 0.416 to about 0.50 — less overfitting. Lasso reaches the best test score while using only 36 of the 65 features, and with $\alpha = 5$ it keeps just 12 features and still beats the unregularized model. Too much regularization would eventually hurt again.

12. Running example: Chennai Homes

With all four standardised features, plain least squares gives large weights. Ridge shrinks them — strongly at $\alpha = 10$. Notice that size and bedrooms (which move together) share the credit more evenly once regularised.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
cols = ["size_m2", "bedrooms", "dist_km", "age_yrs"]
for name, reg in [("OLS", LinearRegression()), ("Ridge a=1", Ridge(alpha=1)),
("Ridge a=10", Ridge(alpha=10))]:
    m = make_pipeline(StandardScaler(), reg).fit(df[cols], df["price_lakh"])
    print(f"{name:11s}", m[-1].coef_.round(2))
```

Output:

```
OLS         [ 11.06   7.01 -14.42  -6.92]
Ridge a=1   [  7.72   8.32 -12.62  -5.6 ]
Ridge a=10  [ 3.79  4.93 -5.94 -2.63]
```

⚙ WHERE THIS IS USED

- Almost every production linear model, neural network and boosted model uses some regularization
- Lasso for picking a few important variables (e.g. genes)

Choose it when: Many features, little data, or a gap between training and test scores.

Think twice when: The model already underfits — then regularization makes it worse.

× MISCONCEPTIONS TO UNLEARN

MYTH Regularization always improves a model.

REALITY It reduces variance but adds bias. If a model underfits, regularization makes it worse.

MYTH Lasso tells you which features truly cause the outcome.

REALITY It picks features useful for prediction. With correlated features it may pick one arbitrarily, and prediction isn't causation.

RESPONSIBLE AI · OVERFITTING LEAKS PRIVATE DATA

A model that memorises its training data can reveal it. Shokri et al. (2017) showed **membership inference attacks**: by probing a model, an attacker can often tell whether a specific person's record was in the training set — sensitive if the dataset is, say, patients with a disease. Regularisation and good generalisation reduce this risk; techniques such as differential privacy give formal guarantees.

13. Questions beginners often ask

Q. How do I know if I have high bias or high variance?

Compare training and validation error. Both high and close → bias. Training much lower than validation → variance.

Q. Does regularization always help?

No. If the model underfits, regularization makes it worse. That's why its strength is tuned with cross-validation.

Q. Ridge or lasso?

Ridge when most features matter a bit; lasso when you expect only a few to matter or want automatic feature selection. ElasticNet mixes both.

! COMMON MISTAKES TO AVOID

- Regularizing without scaling the features first.
- Penalising the bias term.
- Choosing λ by looking at test-set results instead of cross-validation.

! INTERVIEW CORNER

Don't confuse

- **L1 (lasso) vs L2 (ridge) regularisation** — Lasso lassoes features out; ridge flattens them.
- **Bias (statistics) vs bias term vs bias (fairness)** — Ask which bias someone means.
- **Regularisation vs normalisation** — One limits the model, the other rescales the numbers.

Hidden questions — answer in your head first, then open.

▼ **Why does L1 regularisation produce exactly-zero weights, but L2 doesn't?**

The trap: "L1 is stronger."

L1's penalty pulls each weight towards zero with a constant force, however small the weight, so small weights get pushed all the way to 0. L2's pull is proportional to the weight, so it fades as the weight shrinks and never quite reaches zero. Geometrically, the L1 constraint region has corners on the axes.

▼ **Should you regularise the bias (intercept)?**

The trap: Penalising every parameter.

Usually not. The intercept just shifts predictions to the right average level; shrinking it towards 0 biases every prediction and makes results depend on where the target's zero happens to be. Most libraries leave it unpenalised.

▼ **Why must you scale features before ridge or lasso?**

The trap: Thinking regularisation is scale-free.

The penalty acts on weight sizes, and a weight's size depends on its feature's units. Unscaled, a feature in small units needs a large weight and gets penalised more. In Lesson 10, measuring x in units ten times larger changed how much of the weight survived $\lambda = 1$ from 93% to 12%.

▼ **Does more training data reduce bias or variance?**

The trap: "Both."

Variance. More data makes a flexible model's fit more stable. Bias comes from the model being too simple, and data doesn't change that.

14. Key takeaways

- Error = **bias²** (wrong assumptions) + **variance** (sensitivity to the training set) + irreducible noise.
- More complexity lowers bias but raises variance; the best model balances them.
- **Regularization** adds λ·penalty to the loss to keep weights small and reduce variance.
- **Ridge (L2)** shrinks all weights; **Lasso (L1)** sets many to exactly zero (feature selection).
- Scale features first, and choose λ by cross-validation.

15. Check your understanding

1. Your model's training and test errors are both high and close together. What should you try?

a. A more flexible model or less regularization

b. Stronger regularization

c. Removing features

2. What happens as $\lambda \to \infty$ in ridge regression?

a. The weights grow without limit

b. Nothing changes

c. All weights approach 0, so the model predicts a constant

3. You have 500 features but suspect only about 10 matter. Which regularization fits best?

a. Ridge (L2)

b. Lasso (L1)

c. No regularization

4. One feature, no intercept: $\sum x_i y_i = 20$, $\sum x_i^2 = 8$. What is the ridge weight with $\lambda = 2$?

a. 2.5

b. 2.0

c. 10

5. In scikit-learn's `LogisticRegression`, you change `C` from 1 to 0.01. Regularization becomes...

a. much stronger

b. much weaker

c. switched off

16. Going deeper: regularization in practice

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above explains bias, variance, ridge and lasso. This section measures bias and variance directly, shows the one-line rule behind lasso's zeros, chooses the penalty automatically with cross-validation, explains why lasso is unstable with correlated features (and how elastic net fixes it), and covers two "hidden" regularizers: early stopping and the surprising double-descent curve.

You'll need: this lesson, norms from Lesson 2's "Going deeper", and MAP estimation from Lesson 4's.

Measuring bias and variance

The decomposition in this lesson says expected test error $=$ bias² $+$ variance $+$ noise. We can measure each part by simulation: draw many fresh training sets from the same source, fit the model to each, and look at the predictions at fixed test points. Their *average* compared with the truth gives bias²; their *spread* gives the variance. Below, a degree-9 polynomial is fitted with ridge to 20 noisy points from a sine curve (noise variance $0.3^2 = 0.09$), 300 times for each λ:

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline

# Measure bias² and variance directly: refit on 300 fresh training sets
rng = np.random.default_rng(0)
f = lambda x: np.sin(2 * np.pi * x)                  # the true curve
x_test = np.linspace(0, 1, 50)
for lam in [1e-6, 1e-3, 1e-1, 10]:
    preds = []
    for _ in range(300):
        x = rng.uniform(0, 1, 20); y = f(x) + rng.normal(0, 0.3, 20)
        m = make_pipeline(PolynomialFeatures(9), Ridge(alpha=lam)).fit(x[:, None], y)
        preds.append(m.predict(x_test[:, None]))
    preds = np.array(preds)
    bias2 = np.mean((preds.mean(axis=0) - f(x_test)) ** 2)
    var = np.mean(preds.var(axis=0))
    print(f"lambda {lam:7.0e}: bias² {bias2:.3f}   variance {var:.3f}   expected error
{bias2 + var + 0.09:.3f}")
```

Output:

```
lambda   1e-06: bias² 0.001   variance 1.133   expected error 1.224
lambda   1e-03: bias² 0.028   variance 0.153   expected error 0.270
lambda   1e-01: bias² 0.100   variance 0.050   expected error 0.240
lambda   1e+01: bias² 0.335   variance 0.025   expected error 0.450
```

With almost no penalty, bias² is tiny (0.001) but the variance is huge (1.133): each training set gives a wildly different curve. A strong penalty ($\lambda = 10$) flattens every curve, so variance falls to 0.025 but bias² rises to 0.335. The best total, 0.240, is in between at $\lambda = 0.1$. This is the U-shaped trade-off from this lesson, now with real numbers.

Why lasso gives exact zeros: soft-thresholding

Consider one weight with the others held fixed, and suppose least squares alone would set it to z (with standardised features). Adding the penalties gives closed-form answers:

$$\text{ridge: } w = \frac{z}{1+\lambda}, \qquad \text{lasso: } w = \operatorname{sign}(z)\,\max\left(|z| - \lambda,\, 0\right)$$

Ridge *scales* every weight down by the same factor; it never reaches zero. Lasso *subtracts* a fixed amount and stops at zero: any weight whose least-squares value is smaller than λ becomes exactly 0. This is called **soft-thresholding**.

```
# Lasso's key step: soft-thresholding, used by coordinate descent
soft = lambda z, t: np.sign(z) * np.maximum(np.abs(z) - t, 0)
for z in [3.0, 0.8, -0.3, -2.5]:
    print(f"least-squares value {z:5.1f} → lasso value {soft(z, 1.0) + 0.0:5.1f}
ridge value {z / (1 + 1.0):5.2f}")
```

Output:

```
least-squares value   3.0 → lasso value   2.0    ridge value  1.50
least-squares value   0.8 → lasso value   0.0    ridge value  0.40
least-squares value  -0.3 → lasso value   0.0    ridge value -0.15
least-squares value  -2.5 → lasso value  -1.5    ridge value -1.25
```

With $\lambda = 1$, lasso turns 3.0 into 2.0, but 0.8 and −0.3 become exactly 0, while ridge halves everything. Lasso solvers such as scikit-learn's use **coordinate descent**: cycle through the weights, apply this rule to each in turn (using the current residuals), and repeat until nothing changes. It's fast, and it's why the lasso can be fitted for many values of λ in one go (the "regularization path").

Choosing the penalty automatically

`RidgeCV`, `LassoCV` and `ElasticNetCV` fit a whole grid of penalties and pick the best by cross-validation, reusing work along the path so it costs little more than a single fit. **Elastic net** mixes the two penalties: $\lambda\big(\rho\|\mathbf{w}\|_1 + \frac{1-\rho}{2}\|\mathbf{w}\|_2^2\big)$, where ρ is `l1_ratio`.

```
from sklearn.linear_model import LassoCV, ElasticNetCV
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler

X, y = load_diabetes(return_X_y=True)
X = StandardScaler().fit_transform(X)
lasso = LassoCV(cv=5, random_state=0).fit(X, y)
enet = ElasticNetCV(l1_ratio=[0.1, 0.5, 0.9], cv=5, random_state=0).fit(X, y)
print("lasso: best alpha", round(lasso.alpha_, 3), " non-zero weights",
np.sum(lasso.coef_ ≠ 0), "of 10")
print("elastic net: best alpha", round(enet.alpha_, 3), " l1_ratio", enet.l1_ratio_, "
non-zero", np.sum(enet.coef_ ≠ 0))
```

Output:

```
lasso: best alpha 0.079  non-zero weights 9 of 10
elastic net: best alpha 0.217  l1_ratio 0.9  non-zero 8
```

On the standardised diabetes data, cross-validation picks a fairly light lasso ($\alpha \approx 0.079$, keeping 9 of the 10 weights), and an elastic net that is mostly lasso ($\rho = 0.9$, keeping 8). That makes sense: with only 10 features and 442 patients, little regularization is needed. Remember from Lesson 5's "Going deeper" that the CV score at the chosen penalty is slightly optimistic; nested CV gives the honest estimate.

Lasso and correlated features: why elastic net exists

When two features carry almost the same information, lasso tends to keep one and drop the other, and *which* one it keeps can flip with small changes in the data. Here, `a` and `b` are two noisy measurements of the same quantity, equally useful. We refit lasso on 200 bootstrap samples:

```
# Lasso with two almost identical features, on bootstrap samples: which one does it
keep?
rng = np.random.default_rng(1)
from sklearn.linear_model import Lasso
n = 200
z = rng.normal(size=n)                                  # one underlying quantity…
a = z + rng.normal(0, 0.05, n); b = z + rng.normal(0, 0.05, n)   # …measured twice
yy = 3 * z + rng.normal(0, 1, n)
picks = {"only a": 0, "only b": 0, "both": 0}
for _ in range(200):
    i = rng.integers(0, n, n)
    w = Lasso(alpha=1.0).fit(np.column_stack([a, b])[i], yy[i]).coef_
    key = "both" if (w ≠ 0).all() else ("only a" if w[0] ≠ 0 else "only b")
    picks[key] += 1
print("lasso, 200 bootstrap samples:", picks)
from sklearn.linear_model import ElasticNet
w = ElasticNet(alpha=1.0, l1_ratio=0.5).fit(np.column_stack([a, b]), yy).coef_
print("elastic net weights on the full data:", w.round(2))
```

Output:

```
lasso, 200 bootstrap samples: {'only a': 47, 'only b': 63, 'both': 90}
elastic net weights on the full data: [0.89 0.89]
```

Lasso kept only `a` 47 times, only `b` 63 times, and both 90 times. If you reported "lasso selected feature a" from one fit, the next dataset might well say b. The L2 part of **elastic net** makes the penalty strictly convex, which spreads the weight evenly over correlated features (0.89 each here) and keeps groups of related features together. So: use lasso when you believe only a few features matter and they aren't strongly correlated; use elastic net when features come in correlated groups (genes, pixels, words, survey questions). And never treat a single lasso fit's feature selection as stable truth: check it on bootstrap samples, as above.

Early stopping is regularization too

When a model is trained by gradient descent starting from zero weights, the weights grow gradually. Stopping early, when the validation loss stops improving, keeps them small, much like an L2 penalty. For linear regression the link is exact in a simple case: k gradient-descent steps with learning rate η behave approximately like ridge with $\lambda \approx 1/(k\eta)$. Fewer steps means stronger regularization. This is why early stopping is standard for neural networks (Lesson 19) and gradient boosting (Lesson 14), where the number of trees plays the role of the number of steps.

Double descent: when bigger models get better again

The classical U-curve says test error rises once a model is complex enough to fit the noise. Modern experiments found something extra: if you keep increasing the number of parameters past the point where the model can fit the training data *exactly* (the "interpolation threshold", roughly where parameters equal data points), the test error peaks there and then **falls again**. This is **double descent**.

The explanation: right at the threshold there's exactly one way to fit all the points, and it's usually a wild one. With many more parameters than needed, there are infinitely many perfect fits, and gradient descent (or the minimum-norm solution, the pseudo-inverse from Lesson 2's "Going deeper") picks the *smoothest* one: the one with the smallest weights. That's an implicit regularizer. It helps explain why huge neural networks generalise despite having far more parameters than examples. Explicit regularization (a well-tuned ridge penalty) largely removes the peak.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Describe how coordinate descent solves the lasso.**

The trap: Saying it uses gradient descent (the L1 penalty isn't differentiable at 0).

Cycle through the weights one at a time. For weight j, compute the least-squares value it would take given the current residuals (with the other weights fixed), then apply soft-thresholding: shrink it towards 0 by λ and set it to exactly 0 if it would cross. Update the residuals and move on. Repeat until the weights stop changing. Each step is closed-form, so it's fast, and warm starts make fitting a whole path of λ values cheap.

▼ **How would you check whether the features lasso selected are trustworthy?**

The trap: Trusting a single fit's non-zero weights.

Refit on many bootstrap samples (or CV folds) and record how often each feature is selected. This is the idea of stability selection. Features chosen in, say, over 80% of fits are robust; ones that flip are usually members of a correlated group. Check correlations or VIFs, and consider elastic net or reporting the group rather than one member.

▼ **Can you show that bias and variance trade off, rather than just asserting it?**

The trap: Only drawing the U-curve.

Simulate: draw many training sets from the same distribution, fit the model to each, and evaluate at fixed test points. Bias² is the squared gap between the average prediction and the truth; variance is the spread of the predictions across fits. Sweeping model complexity or λ shows bias² falling and variance rising as the model becomes more flexible, with the total error minimised in between.

▼ **Is early stopping really a form of regularization? Explain.**

The trap: "It just saves time."

Yes. Starting from small weights, gradient descent first fits the strongest, smoothest patterns (large-eigenvalue directions) and only later the weak, noisy ones. Stopping early leaves the weights small along those weak directions, which is the same effect ridge has. For linear least squares the equivalence is approximately $\lambda \approx 1/(k\eta)$ for k steps. The number of epochs (or of boosting rounds) is therefore a regularization hyperparameter, chosen on validation data.

Check your understanding: going deeper

1. With $\lambda = 2$, what does lasso's soft-thresholding turn a least-squares value of 1.5 into?

a. 0

b. 0.5

c. 0.75

2. As λ grows, what happens to bias² and variance?

a. Both fall

b. Bias² rises and variance falls

c. Bias² falls and variance rises

3. Why prefer elastic net over lasso when features come in correlated groups?

a. It's faster

b. Its L2 part spreads weight across correlated features instead of picking one arbitrarily

c. It never sets weights to zero

4. How is early stopping related to ridge regression?

a. It isn't

b. Fewer gradient steps from zero keep weights small, much like a stronger L2 penalty

c. It's the same as lasso

5. In double descent, where does test error peak?

a. Near the point where the model can just fit the training data exactly

b. At the simplest model

c. At the largest model

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 Concepts

Your model has training $R^2 = 0.98$ and test $R^2 = 0.55$. Name the problem and three things to try.

WORKING

EXERCISE 2 By hand

One feature, no intercept: $\sum x_i y_i = 50$, $\sum x_i^2 = 20$. Compute the ridge weight for $\lambda = 0, 5, 30$.

WORKING

EXERCISE 3 Concepts

Lasso or ridge? (a) Genetic data with 20,000 features where only a few genes matter; (b) house prices with 15 features that all plausibly matter.

WORKING

EXERCISE 4 By hand

`LogisticRegression(C=0.01)`: what is λ, and is regularization strong or weak?

WORKING

EXERCISE 5 Intermediate

With $\lambda = 0.5$ and standardised features, apply lasso soft-thresholding and ridge shrinkage $z/(1+\lambda)$ to the least-squares values $z = 2$, 0.4 and -1. Give the lasso values.

WORKING

EXERCISE 6 Intermediate

In a simulation, the average prediction at a test point is 4.2, the true value is 5.0, and predictions across training sets have standard deviation 0.6. The noise variance is 0.25. Compute bias², variance and the expected squared error at that point.

WORKING

EXERCISE 7 Intermediate

Elastic net with $\lambda = 2$ and `l1_ratio` $\rho = 0.25$: what are the effective L1 and L2 penalty strengths, using $\lambda\rho\|\mathbf{w}\|_1 + \lambda\frac{1-\rho}{2}\|\mathbf{w}\|_2^2$?

WORKING

EXERCISE 8 Code · intermediate

In the correlated-features code, change `Lasso(alpha=1.0)` to `ElasticNet(alpha=1.0, l1_ratio=0.5)` inside the bootstrap loop. How often is each pattern chosen now?

WORKING

NOTES

PART 3 · CLASSIC ALGORITHMS · ~40 MIN

Lesson 11: k-Nearest Neighbours

Classify by asking the neighbours: distances, choosing k, scaling and the curse of dimensionality.

BY THE END OF THIS LESSON YOU CAN

- Classify and regress with the k-nearest neighbours algorithm by hand
- Compute Euclidean and Manhattan distances
- Explain how k controls the bias–variance trade-off
- Explain why feature scaling and the curse of dimensionality matter

↺ BEFORE YOU START

This lesson builds on:

- Lesson 2: Vectors & Matrices — A vector is a list of numbers; a dataset X is a matrix with one row per example. The dot product $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models predict, and $\|\mathbf{a} - \mathbf{b}\|$ is the distance between two examples.
- Lesson 4: Probability & Statistics — Probabilities run from 0 to 1. Bayes' theorem turns $P(B \mid A)$ into $P(A \mid B)$. Mean and standard deviation summarise data; the z-score $(x - \mu)/\sigma$ standardises it.
- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.

Where this fits: Starts Part 3, the classic algorithm toolbox. kNN is the simplest one — no training at all — and highlights why scaling matters.

1. The simplest idea in machine learning

PICTURE IT New to a city and wondering whether a restaurant is any good? Ask the three people who live closest to it. That's k-nearest neighbours: no formula, just "you're probably like the examples you're near".

"Tell me who your neighbours are, and I'll tell you who you are." That proverb *is* the **k-nearest neighbours** algorithm (kNN). To classify a new example, find the k training examples most similar to it and let them vote.

kNN has no equations to fit and no weights to learn — and yet it can draw curved, complex decision boundaries that linear models can't. It's a great way to understand the idea of *similarity*, which runs through all of ML, and it teaches two crucial lessons: the importance of **feature scaling** and the **curse of dimensionality**.

> ◎ ANALOGY: ESTIMATING A HOUSE PRICE
>
> An estate agent valuing a house doesn't solve equations. They look at the 3–5 most similar houses that sold recently nearby (similar size, area, condition) and take something like their average price. That's kNN regression.

2. The algorithm

Training: just store all the training examples. That's it. (kNN is called a *lazy learner*, because all the work happens at prediction time.)

k = 3 neighbours: 2 green, 1 purple → predict green

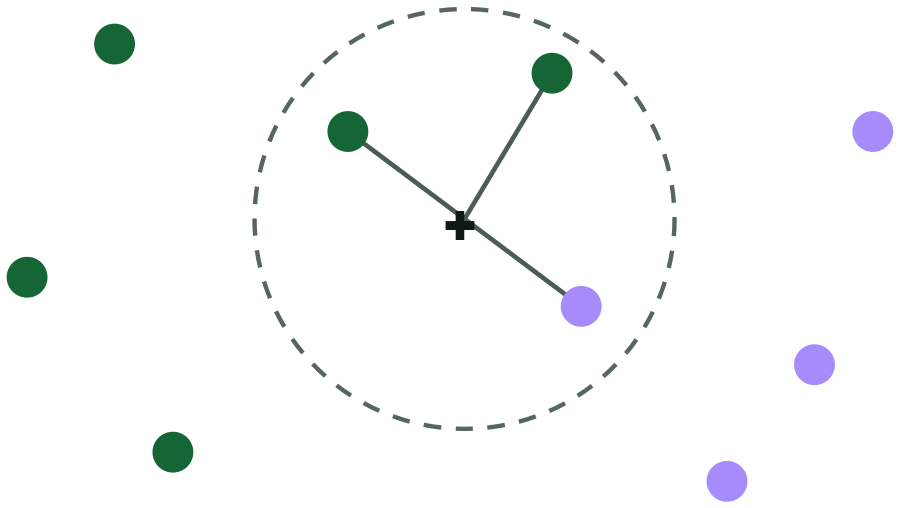


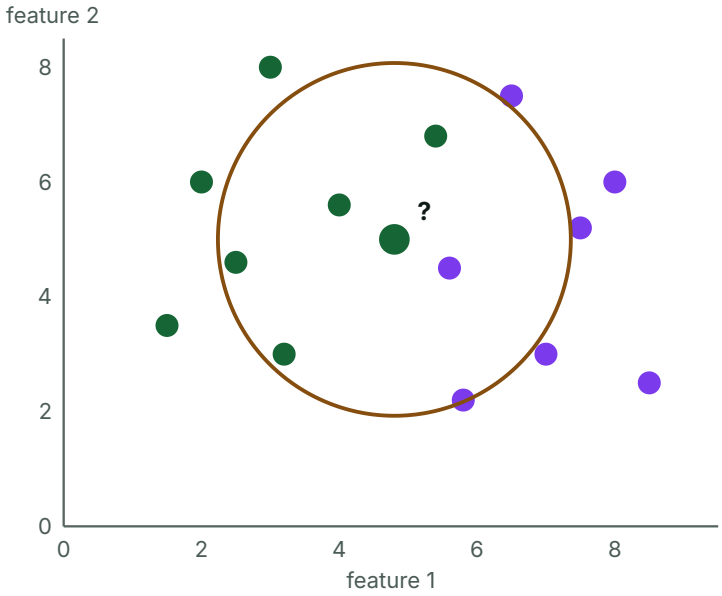
Figure 11. kNN with k = 3: the query (✚) takes a vote of its three nearest training points; the dashed circle reaches the third neighbour.

Predicting for a new point $\mathbf{q}$:

1. Compute the distance from $\mathbf{q}$ to every training example.
2. Pick the k examples with the smallest distances — the nearest neighbours.
3. **Classification:** predict the most common class among those k (majority vote). **Regression:** predict the average of their y values.

kNN is **non-parametric**: it doesn't summarise the data in a fixed set of weights. The training data itself is the model.

WALKTHROUGH (FINAL STEP SHOWN) **kNN classifies a new point**



Nearest neighbours (sorted)

- #1 at (5.6, 4.5) distance 0.94
- #2 at (4, 5.6) distance 1.00
- #3 at (5.4, 6.8) distance 1.90
- #4 at (2.5, 4.6) distance 2.33
- #5 at (3.2, 3) distance 2.56

Vote: 4 green vs 1 violet → green

1. Training data: two classes (green and violet). kNN does no training at all — it simply stores these points.
2. A new point arrives at $(4.8, 5)$. Which class is it? kNN measures its distance to *every* stored point.
3. Neighbour 1 is violet, at distance 0.94. With $k = 1$ we would stop here and predict **violet**.
4. Neighbour 2 is green, at distance 1.00.
5. Neighbour 3 is green, at distance 1.90.
6. Neighbour 4 is green, at distance 2.33.
7. Neighbour 5 is green, at distance 2.56.
8. With $k = 5$ the vote is 4 green to 1 violet, so the prediction is **green**. The choice of k changed the answer — that's why k is tuned with cross-validation.

3. Measuring distance

"Nearest" needs a distance. The most common is the **Euclidean distance** (straight-line distance, from Pythagoras — Lesson 2):

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{j=1}^{d} (a_j - b_j)^2}$$

Alternatives:

- **Manhattan distance** $\sum_j |a_j - b_j|$ — like walking along city blocks; less dominated by one big difference.
- **Minkowski distance** $\left(\sum_j |a_j - b_j|^p\right)^{1/p}$ — the general family: $p = 1$ is Manhattan, $p = 2$ is Euclidean. scikit-learn's default is Minkowski with $p = 2$.
- **Cosine distance** — based on the angle between vectors (Lesson 2), popular for text.

✎ WORKED EXAMPLE — CLASSIFY A POINT BY HAND

Training points: two red, three blue.

Point	Coordinates	Class	Distance to $\mathbf{q} = (2, 2)$
A	(1, 1)	red	$\sqrt{1^2 + 1^2} = 1.414$
B	(2, 1)	red	$\sqrt{0^2 + 1^2} = 1.000$
C	(4, 3)	blue	$\sqrt{2^2 + 1^2} = 2.236$
D	(5, 4)	blue	$\sqrt{3^2 + 2^2} = 3.606$
E	(3, 3.5)	blue	$\sqrt{1^2 + 1.5^2} = 1.803$

Sorted by distance: B (1.000), A (1.414), E (1.803), C (2.236), D (3.606).

- $k = 1$: nearest is B → **red**.
- $k = 3$: B, A, E → 2 red, 1 blue → **red**.
- $k = 5$: all five → 2 red, 3 blue → **blue**.

The answer depends on k. With $k = 5$ we're just counting every point, so the larger class wins regardless of where $\mathbf{q}$ is.

? MAKE SENSE OF IT

The situation: A new customer, $\mathbf{q} = (2, 2)$, arrives. The two red points are customers who left; the three blue ones stayed. Will this one leave?

What the result tells you: kNN copies its nearest neighbours. With $k = 1$ or 3 the closest customers are mostly red, so it predicts red. With $k = 5$ it asks everyone, and the majority (blue) wins wherever $\mathbf{q}$ is.

What if…?

- **Move the customer to (3.5, 3):** the three nearest are all blue, so every k from 1 to 5 says **blue**.
- **Move to (3, 2):** the nearest red and the nearest blue are both exactly 1.414 away — a tie that the order of the data has to break.
- **Weight votes by 1/distance** with $k = 5$: red scores $1/1 + 1/1.414 = 1.71$, blue scores 1.28, so close neighbours win and the answer is **red** again.

Why it matters in ML: k controls how local the model is: small k follows every quirk of the data (overfitting); large k smooths everything towards the majority (underfitting). Pick it by cross-validation.

READ THE FORMULA **Euclidean distance**

$$d(\mathbf{a}, \mathbf{b}) = \left(\sum_{j=1}^{d} (a_j - b_j)^2 \right)^{1/2}$$

- **power ½** — the square root brings the units back (Pythagoras)
- **Σ** — add up over every feature
- **$(a_j - b_j)^2$** — the squared difference on feature j — big-scale features dominate unless you scale them

Hover over (or tap) a part to highlight it in the formula.

4. Choosing k: bias vs variance again

- **Small** k (e.g. 1): the boundary follows every single training point, including noisy or mislabelled ones. Jagged boundary, little islands of the wrong class. **High variance → overfitting.** Note that with $k = 1$, training accuracy is always 100% — each point is its own nearest neighbour — which tells you nothing.

- **Large k**: votes average over a big neighbourhood, so the boundary is smooth. Too large and it blurs real structure, eventually just predicting the majority class everywhere. **High bias → underfitting.**

Choose k by cross-validation (Lesson 5). For two classes, odd values avoid tied votes.

> **DISTANCE-WEIGHTED VOTING**
>
> Instead of one vote each, neighbours can vote with weight $1/d$, so closer ones count more. In scikit-learn: `weights="distance"`. This makes the choice of k less sensitive.

5. Try it: kNN decision regions

Three classes of points. The background colour shows what kNN would predict at every location. Change k and watch the boundaries go from jagged (small k) to smooth (large k). Click anywhere to place a query point and see its k nearest neighbours and their vote.

Interactive demo — open *lesson-11.html* on the website to try it.

6. Feature scaling is essential

Distances add up differences across all features. If features are on different scales, the large-scale feature completely dominates.

> **✎ WORKED EXAMPLE — AGE VS INCOME**
>
> Person P is 30 years old earning 50,000. Q is 60 earning 51,000. R is 31 earning 70,000.
>
> $d(P, Q) = \sqrt{30^2 + 1000^2} \approx 1000.4$
> $d(P, R) = \sqrt{1^2 + 20000^2} \approx 20000.0$
>
> So Q is "much closer" to P than R is — even though Q is twice P's age — because the income differences (thousands) swamp the age differences (tens). Age is effectively ignored.
>
> After standardising each feature to mean 0 and standard deviation 1 (Lesson 4), a 30-year age gap and a 20,000 income gap are each measured relative to how much that feature normally varies, so both count.

? MAKE SENSE OF IT

The situation: A dating app matches people by age and income. P is 30 earning 50,000; Q is 60 earning 51,000; R is 31 earning 70,000. Who is P's closest match?

What the result tells you: On raw numbers Q looks 20 times closer than R, purely because income is measured in thousands and age in tens. The 30-year age gap barely registers.

What if…?

- **Standardise both features** (using these three people): P–Q becomes **2.16** and P–R **2.17**. Now the age gap and the income gap count about equally — a very different picture.
- **Income in thousands** (50, 51, 70): P–Q $= \sqrt{30^2 + 1^2} \approx 30.0$ and P–R $= \sqrt{1 + 20^2} \approx 20.0$, so R becomes the closer one. The "nearest" person depends entirely on the units.

Why it matters in ML: Every distance-based method (kNN, k-means, SVMs with RBF kernels) is sensitive to scale. Standardising is not optional for them.

The code below shows the real effect on a dataset: accuracy jumps from 67% to 93% just by scaling.

7. kNN for regression

For a numeric target, predict the average of the neighbours' values:

$$\hat{y}(\mathbf{q}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{q})} y_i$$

where $N_k(\mathbf{q})$ is the set of the k nearest neighbours. With distance weighting, it's a weighted average. Example: the three nearest houses sold for 300k, 320k and 340k, so predict 320k.

8. The curse of dimensionality

kNN relies on nearby points being similar. In high dimensions, "nearby" stops meaning much — this is the **curse of dimensionality**.

✎ WORKED EXAMPLE — HOW BIG IS A "NEIGHBOURHOOD"?

Suppose data is spread evenly inside a unit cube, and we want a small cube-shaped neighbourhood that contains 10% of the data. Its volume must be 0.1, so each side has length $0.1^{1/d}$:

Dimensions d	1	2	3	10	100
Side length needed	0.10	0.32	0.46	0.79	0.98

In 10 dimensions, to capture just 10% of the data, the "neighbourhood" must span 79% of each feature's range. In 100 dimensions, 98%. The "nearest" neighbours aren't near at all — they're scattered almost across the whole space.

? MAKE SENSE OF IT

The situation: You want the 10% of houses most like yours. With one feature (size) that's a narrow band. Add more features and see what happens to "nearby".

What the result tells you: In 10 dimensions, a box holding 10% of the data must stretch 79% of the way along every feature. Your "neighbours" differ from you on almost everything.

What if...?

- **Ask for only 1% of the data** as neighbours: the side length is still **0.63** in 10 dimensions and **0.955** in 100.
- **To keep a side of 0.1** (truly local) in 10 dimensions, the box holds only 0.1^{10} — one ten-billionth of the data. You'd need billions of examples to find a single neighbour.

Why it matters in ML: This is the curse of dimensionality: with many features, data becomes sparse and distances lose their meaning. Remove irrelevant features or reduce dimensions (Lesson 17) before using kNN.

Consequences: in many dimensions, all points end up roughly the same distance from each other, so nearest neighbours become nearly meaningless, and you'd need exponentially more data to fill the space. Remedies: remove irrelevant features, or reduce dimensions first with PCA (Lesson 17).

9. Strengths and weaknesses

Strengths	Weaknesses
Simple to understand and explain	Slow predictions: compares to every training point
No training time	Must store the whole training set
Flexible, curved decision boundaries	Needs feature scaling
Works for classification and regression	Struggles in high dimensions and with irrelevant features

To speed up the search, libraries use tree-shaped indexes such as **KD-trees** and **ball trees**, which skip whole regions of space that can't contain a nearest neighbour. They work well in low dimensions; scikit-learn chooses automatically (`algorithm="auto"`).

10. In Python

The wine dataset: 178 wines, 13 chemical measurements with very different scales (one ranges up to 1,680, others are below 1), and 3 grape varieties.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)

raw = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
print("without scaling:", round(raw.score(X_test, y_test), 3))

scaled = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=5)).fit(X_train, y_train)
print("with scaling:   ", round(scaled.score(X_test, y_test), 3))

for k in [1, 5, 15, 51, 81, 101]:
    s = cross_val_score(make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=k)),
                        X_train, y_train, cv=5)
    print(f"k={k:3d}  CV accuracy {s.mean():.3f} ± {s.std():.3f}")
```

Output:

```
without scaling: 0.667
with scaling:    0.933
k=  1  CV accuracy 0.918 ± 0.054
k=  5  CV accuracy 0.940 ± 0.038
k= 15  CV accuracy 0.932 ± 0.037
k= 51  CV accuracy 0.962 ± 0.023
k= 81  CV accuracy 0.593 ± 0.065
k=101  CV accuracy 0.399 ± 0.019
```

Two lessons in one output. Scaling lifts accuracy from 67% to 93%. And k matters: small k is noisier (look at the larger ± for $k = 1$), moderate values work well, and once k approaches the size of the training folds (about 106 examples), the vote is dominated by whole classes and accuracy collapses — extreme underfitting.

11. Running example: Chennai Homes

> A new 100 m², 5-year-old house 6 km from the centre: after scaling, its 3 nearest neighbours are houses 5, 4 and 2 (88, 55 and 82 lakh), so kNN predicts their average, 75 lakh.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
cols = ["size_m2", "dist_km", "age_yrs"]
knn = make_pipeline(StandardScaler(),
KNeighborsRegressor(n_neighbors=3)).fit(df[cols], df["price_lakh"])
new = pd.DataFrame({"size_m2": [100], "dist_km": [6], "age_yrs": [5]})
dist, idx = knn[-1].kneighbors(knn[0].transform(new))
print("nearest houses:", (df["id"].iloc[idx[0]]).tolist(), "prices:",
df["price_lakh"].iloc[idx[0]].tolist())
print("predicted price:", round(knn.predict(new)[0], 1), "lakh")
```

Output:

```
nearest houses: [5, 4, 2] prices: [88, 55, 82]
predicted price: 75.0 lakh
```

⚙ WHERE THIS IS USED

- Simple recommenders ("customers similar to you bought...")
- Filling in missing values from similar rows (KNNImputer)
- A quick baseline for small datasets

Choose it when: Small to medium data, few features, and a meaningful notion of similarity.

Think twice when: Very large datasets (slow predictions) or many features (curse of dimensionality).

× MISCONCEPTIONS TO UNLEARN

MYTH kNN learns a model from the data.

REALITY It just stores the data and compares at prediction time.

MYTH Larger k is always safer.

REALITY Too large k blurs real boundaries and, in the extreme, predicts the majority class everywhere.

RESPONSIBLE AI · A MODEL THAT STORES ITS DATA

kNN keeps every training example and uses it at prediction time, so deploying it means deploying the raw data — with all its privacy obligations, including deleting a person's record if they ask. Distances also treat every feature as meaningful: if a sensitive attribute (or a proxy) is among the features, "similar people" will be judged alike for the wrong reasons.

12. Questions beginners often ask

Q. How do I choose k?

Try a range (e.g. 1–50) with cross-validation and pick the best. Odd numbers avoid ties for two classes.

Q. Why is kNN slow to predict?

It compares each new point with every training example. Tree indexes (KD-tree, ball tree) speed this up in low dimensions.

Q. Does kNN learn anything?

Not in the usual sense — it stores the data. All the work happens at prediction time, which is why it's called a lazy learner.

! COMMON MISTAKES TO AVOID

- Running kNN on unscaled features.
- Judging $k = 1$ by its training accuracy (always 100%).
- Keeping many irrelevant features, which dilute the meaning of distance.

! INTERVIEW CORNER

Don't confuse

- **kNN vs k-means** — kNN looks at neighbours; k-means moves centres.
- **Lazy vs eager learning** — Pay now or pay later.

Hidden questions — answer in your head first, then open.

▼ **What does a kNN model learn during training?**

The trap: Describing weights or a boundary.

Nothing beyond storing the training data (and perhaps building a search index). It's a "lazy" learner: all the work happens at prediction time, which is why predictions get slow on large datasets and why deploying it means deploying the data.

▼ **With $k = 1$, what's the training accuracy, and what does it tell you?**

The trap: Reporting it as evidence the model is good.

Essentially 100%: each training point's nearest neighbour is itself (unless identical points have different labels). It tells you nothing about generalisation — always evaluate kNN on held-out data.

▼ **Why is an odd k often recommended for two-class kNN?**

The trap: Not noticing ties.

With an even k the vote can tie (2 vs 2). An odd k avoids ties for two classes; with more classes ties can still happen, so implementations break them by distance or order.

▼ **Your kNN accuracy jumps after you standardise the features. Why?**

The trap: "Scaling just helps."

Distances are dominated by features with large numeric ranges. In Lesson 11, income in units of thousands drowned out age entirely. Standardising lets every feature contribute in proportion to how much it normally varies.

13. Key takeaways

- kNN predicts from the k most similar training examples: majority vote (classification) or average (regression).
- It needs a distance — usually Euclidean $\sqrt{\sum (a_j - b_j)^2}$.
- Small k → jagged boundary, overfitting; large k → smooth boundary, underfitting. Choose k by cross-validation.
- **Always scale features** — otherwise large-scale features dominate the distance.
- The **curse of dimensionality**: in many dimensions, neighbourhoods stop being local.

14. Check your understanding

1. What is the Euclidean distance between $(1, 2)$ and $(4, 6)$?

a. 7

b. 5

c. 25

2. With $k = 1$, what is kNN's accuracy on its own training data (no duplicate points)?

a. 100%

b. Around 50%

c. It depends on the data

3. Your kNN boundary is jagged with many small islands. What should you change?

a. Decrease k

b. Remove feature scaling

c. Increase k

4. The 3 nearest neighbours of a house sold for 200k, 260k and 230k. What does kNN regression ($k = 3$, unweighted) predict?

a. 260k

b. 230k

c. 200k

5. Why does kNN struggle with hundreds of features?

a. In high dimensions all points are roughly equally far apart, so "nearest" loses meaning

b. It can only use two features

c. The vote always ties

15. Going deeper: distances, dimensions and fast neighbour search

INTERMEDIATE · SKIP ON A FIRST READ

kNN is simple to describe but has three practical questions that the lesson above only touches on: why it breaks down with many features (the curse of dimensionality, measured), which distance and weighting to use, and how to find neighbours quickly in millions of points. The last one matters far beyond kNN: modern search engines, recommendation systems and the "retrieval" step of LLM applications all rely on fast neighbour search over embeddings.

You'll need: this lesson, and norms from Lesson 2's "Going deeper".

The curse of dimensionality, measured

kNN assumes that "nearest" means "similar". In high dimensions that assumption weakens, because distances **concentrate**: all points end up almost the same distance away. Here are 1,000 random points in a unit cube and one query point:

```
import numpy as np

# Distance concentration: in high dimensions the nearest and farthest points are almost
equally far
rng = np.random.default_rng(0)
for d in [2, 10, 100, 1000]:
    pts = rng.uniform(size=(1000, d)); q = rng.uniform(size=d)
    dist = np.linalg.norm(pts - q, axis=1)
    print(f"{d:4d} dimensions: nearest {dist.min():6.2f}   farthest {dist.max():6.2f}
ratio {dist.max() / dist.min():5.2f}")
```

Output:

```
   2 dimensions: nearest   0.01   farthest   1.33   ratio 135.62
  10 dimensions: nearest   0.47   farthest   2.11   ratio  4.48
 100 dimensions: nearest   3.35   farthest   4.65   ratio  1.39
1000 dimensions: nearest  12.17   farthest  13.55   ratio  1.11
```

In 2 dimensions the farthest point is 136 times farther than the nearest. In 1,000 dimensions the ratio is only 1.11: the "nearest" neighbour is barely nearer than the farthest, so it carries little information. The intuition: each dimension adds a little independent random difference, and sums of many random differences are all close to their average (the central limit theorem again, Lesson 4's "Going deeper").

Real data is rescued by the fact that it rarely fills all its dimensions. Images, text embeddings and sensor readings usually lie near a much lower-dimensional surface (the **manifold hypothesis**), so their *intrinsic* dimension is small. That's why reducing dimensions first (PCA, Lesson 17) or using learned embeddings often makes kNN work well again.

Choosing the distance and the weighting

Three common distances:

- **Euclidean** (L2): the default for measurements on comparable, standardised scales.
- **Manhattan** (L1): less dominated by a single large difference; sometimes better with many features or outliers.
- **Cosine distance**, $1 - \cos\theta$: compares directions and ignores length. The standard for text and embeddings, where a document twice as long shouldn't count as "farther".

Distance weighting lets closer neighbours count more (weight $1/d$) instead of every one of the k getting an equal vote. It makes the result less sensitive to the exact choice of k. On the standardised breast-cancer data with $k = 15$:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
for metric in ["euclidean", "manhattan", "cosine"]:
    for weights in ["uniform", "distance"]:
        m = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=15,
metric=metric, weights=weights))
        print(f"{metric:9} {weights:8}: accuracy {cross_val_score(m, X, y,
cv=10).mean():.3f}")
```

Output:

```
euclidean uniform : accuracy 0.961
euclidean distance: accuracy 0.963
manhattan uniform : accuracy 0.954
manhattan distance: accuracy 0.956
cosine    uniform : accuracy 0.963
cosine    distance: accuracy 0.963
```

All six combinations land between 0.954 and 0.963. On this dataset the choice hardly matters, which is common once features are standardised; the gaps are within cross-validation noise (Lesson 9's "Going deeper"). The things that matter most for kNN are, in order: scaling the features, choosing k by cross-validation, and using meaningful features. The metric is worth tuning mainly for text, images or mixed data types.

Finding neighbours fast

Brute-force kNN compares a query with all n stored points: $O(nd)$ work per query, too slow for millions of points answered in milliseconds. Two families of solutions:

- **Space-partitioning trees** (k-d trees and ball trees) split space into nested boxes or balls. A search walks down to the query's box and skips whole regions that can't contain anything closer than the best point found so far. This is *exact* and very fast in low dimensions (roughly $O(\log n)$ per query), but the benefit fades above about 20 dimensions, again because of distance concentration.
- **Approximate nearest-neighbour** (ANN) methods trade a little accuracy for huge speed-ups in high dimensions. Graph-based methods such as HNSW hop through a "small world" graph linking each point to its near neighbours. Others use clustering or quantization (as in the FAISS library). They typically find over 95% of the true neighbours, thousands of times faster. Vector databases are built on them.

```
import time
from sklearn.neighbors import NearestNeighbors

data = rng.normal(size=(200000, 3)); queries = rng.normal(size=(2000, 3))
found, times = {}, {}
for algo in ["brute", "kd_tree"]:
    nn = NearestNeighbors(n_neighbors=5, algorithm=algo).fit(data)
    t = time.perf_counter(); found[algo] = nn.kneighbors(queries)[1]; times[algo] =
time.perf_counter() - t
print("same neighbours found by both:", np.array_equal(np.sort(found["brute"]),
np.sort(found["kd_tree"])))
print("k-d tree faster:", times["kd_tree"] < times["brute"])   # exact timings vary from
computer to computer
```

Output:

```
same neighbours found by both: True
k-d tree faster: True
```

On 200,000 points in 3 dimensions, the k-d tree finds exactly the same neighbours as brute force, much faster. (Exact timings depend on your computer, so the code only prints whether it was faster; try printing `times` to see them.) Scikit-learn's default, `algorithm="auto"`, picks brute force, a k-d tree or a ball tree based on the data's size and dimension.

How good can kNN get?

A classic result (Cover and Hart, 1967): as the amount of data grows, the error of 1-nearest-neighbour is at most **twice the Bayes error**, the error of the best possible classifier for that problem. With k growing slowly as n grows (so $k \to \infty$ but $k/n \to 0$), kNN's error approaches the Bayes error itself. kNN is therefore *consistent*: with enough data it can learn any decision boundary. The catch, as above, is how much data "enough" is when there are many dimensions.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How would you serve nearest-neighbour search over 100 million embeddings with low latency?**

The trap: Proposing brute force or a k-d tree.

Use an approximate nearest-neighbour index: graph-based (HNSW) or inverted-file with product quantization (IVF-PQ, as in FAISS), often in a vector database. Normalise the embeddings if using cosine similarity. Tune the index (graph degree, number of probed clusters) for the recall/latency trade-off, measure recall against brute force on a sample, and shard across machines when the index doesn't fit in memory.

▼ **Why do distances concentrate in high dimensions, and why does kNN still work on images and text?**

The trap: Saying kNN never works in high dimensions.

Each dimension adds an independent random difference, so squared distances are sums of many terms, whose relative spread shrinks like $1/\sqrt{d}$. Nearest and farthest become similar. Real data, though, lies near a much lower-dimensional manifold, so its intrinsic dimension is small. Good representations (PCA, learned embeddings) expose that structure, and kNN works in that space.

▼ **When would you choose cosine distance over Euclidean?**

The trap: "Cosine is always better for text." without saying why.

When the length of a vector reflects something irrelevant, such as document length or overall activity, and only the pattern matters. TF-IDF vectors, word and sentence embeddings, and user-preference vectors are typical. On unit-length vectors, cosine and Euclidean rank neighbours identically, because $\|\mathbf{a}-\mathbf{b}\|^2 = 2 - 2\cos\theta$.

▼ **What's the theoretical guarantee for 1-nearest-neighbour as data grows?**

The trap: Claiming it reaches perfect accuracy.

Cover and Hart (1967): as $n \to \infty$, the 1-NN error is at most twice the Bayes error (the best achievable). With $k \to \infty$ and $k/n \to 0$, kNN's error approaches the Bayes error itself. The data needed to get close grows exponentially with dimension, which is the practical catch.

Check your understanding: going deeper

1. In 1,000 dimensions, the farthest of 1,000 random points is only about 1.1 times farther than the nearest. What does that mean for kNN?

a. Neighbours are very informative

b. "Nearest" carries little information, so kNN struggles

c. kNN becomes faster

2. Which distance ignores vector length and compares only direction?

a. Euclidean

b. Manhattan

c. Cosine distance

3. What does `weights="distance"` do in kNN?

a. Closer neighbours get bigger votes

b. It scales the features

c. It picks k automatically

4. When do k-d trees stop helping much?

a. With more than about 20 dimensions

b. With more than 1,000 points

c. With categorical labels

5. Approximate nearest-neighbour methods such as HNSW trade...

a. A little accuracy for a huge speed-up

b. Speed for perfect accuracy

c. Memory for nothing

16. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Training points: A $(0, 0)$ red, B $(1, 1)$ red, C $(4, 4)$ blue, D $(5, 4)$ blue, E $(1, 4)$ blue. Classify $\mathbf{q} = (2, 1)$ with $k = 1, 3, 5$.

WORKING

EXERCISE 2 By hand

Compute the Euclidean and Manhattan distances between $(1, 2)$ and $(4, 6)$.

WORKING

EXERCISE 3 Concepts

A dataset has "age" (20–70) and "salary" (20,000–200,000). Without scaling, which feature will dominate kNN distances? What should you do?

WORKING

EXERCISE 4 By hand

In 5 dimensions, what side length must a cube-shaped neighbourhood have to contain 10% of uniformly spread data?

WORKING

EXERCISE 5 Intermediate

Show that for unit-length vectors, $\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\cos\theta$. If the cosine similarity is 0.8, what is the Euclidean distance?

WORKING

EXERCISE 6 Intermediate

A query has 3 neighbours at distances 1, 2 and 4, with classes A, B and B. Which class wins with uniform votes, and which with distance weights $1/d$?

WORKING

EXERCISE 7 Intermediate

Brute-force kNN on $n = 10^6$ points with $d = 100$ features: roughly how many multiply-adds does one query need?

WORKING

EXERCISE 8 Code · intermediate

Change the concentration code to use 1,000 points from a normal distribution (`rng.normal`) instead of the cube. Does the farthest/nearest ratio still shrink towards 1 as dimension grows?

WORKING

NOTES

NOTES

PART 3 · CLASSIC ALGORITHMS · ~40 MIN

Lesson 12: Naive Bayes

Bayes' theorem as a classifier — the classic spam filter, built step by step.

BY THE END OF THIS LESSON YOU CAN

- Turn Bayes' theorem into a classifier
- Explain the naive independence assumption and its limits
- Build a spam filter by hand with Laplace smoothing
- Choose between Multinomial, Bernoulli and Gaussian Naive Bayes

↺ BEFORE YOU START

This lesson builds on:

- Lesson 4: Probability & Statistics — Probabilities run from 0 to 1. Bayes' theorem turns $P(B \mid A)$ into $P(A \mid B)$. Mean and standard deviation summarise data; the z-score $(x - \mu)/\sigma$ standardises it.
- Lesson 9: Evaluating Models — Accuracy can mislead. Precision $= TP/(TP + FP)$, recall $= TP/(TP + FN)$, and F1 balances them. Moving the threshold trades one for the other; ROC-AUC summarises all thresholds.

Where this fits: A second classic algorithm, built directly on Bayes' theorem. Fast and a great baseline, especially for text.

1. A classifier built from Bayes' theorem

In Lesson 4 you used Bayes' theorem to update a belief given evidence: a positive test changed the probability of disease. **Naive Bayes** turns that idea into a classifier. The "evidence" is the example's features; the "hypothesis" is the class.

It's one of the oldest and simplest ML methods — the classic spam filters of the early 2000s were Naive Bayes — and it's still useful today: it trains almost instantly, needs little data, and works surprisingly well on text. In this lesson you'll build a spam filter completely by hand.

2. Bayes' theorem for classification

We want the probability of each class y given the features $\mathbf{x} = (x_1, \ldots, x_d)$:

$$P(y \mid \mathbf{x}) = \frac{P(y)\ P(\mathbf{x} \mid y)}{P(\mathbf{x})}$$

- $P(y)$ — the **prior**: how common the class is overall (e.g. 40% of emails are spam).
- $P(\mathbf{x} \mid y)$ — the **likelihood**: how probable these features are for this class.
- $P(\mathbf{x})$ — the evidence: the same for every class, so for *comparing* classes we can ignore it.

So the prediction is the class with the largest $P(y)\,P(\mathbf{x} \mid y)$. We can always divide by the sum over classes at the end to get proper probabilities.

3. The "naive" assumption

The hard part is $P(\mathbf{x} \mid y)$: the probability of a whole *combination* of features. An email has thousands of possible words, and we'll never see most combinations even once, so we can't count them.

Naive Bayes makes a bold simplification: **assume the features are independent of each other, given the class.** Then the joint probability is just a product (the multiplication rule from Lesson 4):

$$P(x_1, x_2, \ldots, x_d \mid y) = P(x_1 \mid y)\,P(x_2 \mid y) \cdots P(x_d \mid y) = \prod_{j=1}^{d} P(x_j \mid y)$$

which gives the Naive Bayes classifier:

$$\hat{y} = \arg\max_{y}\ P(y) \prod_{j=1}^{d} P(x_j \mid y)$$

($\arg\max_y$ means "the value of y that makes this largest"; $\prod$ means "multiply together", like $\sum$ means "add up".)

> ◎ WHY "NAIVE"?
>
> The assumption is usually false. In real emails, "free" and "money" tend to appear *together* — they're not independent. Naive Bayes ignores that. Yet it often classifies well anyway, because it only needs to get the *ranking* of classes right, not the exact probabilities. Its probability estimates, however, tend to be over-confident (too close to 0 or 1).

STEP-BY-STEP DERIVATION **From Bayes' theorem to the Naive Bayes rule**

1 $$P(y \mid \mathbf{x}) = \frac{P(y)\,P(\mathbf{x} \mid y)}{P(\mathbf{x})}$$

Bayes' theorem for a class y and features $\mathbf{x}$.

2 $$\hat{y} = \arg\max_{y}\; P(y)\,P(\mathbf{x} \mid y)$$

$P(\mathbf{x})$ is the same for every class, so it can't change which class wins.

3 $$P(\mathbf{x} \mid y) = \prod_{j} P(x_j \mid y)$$

The naive assumption: features independent given the class.

4 $$\hat{y} = \arg\max_{y}\; P(y) \prod_{j} P(x_j \mid y)$$

Substitute.

5 $$\hat{y} = \arg\max_{y}\; \left[\ln P(y) + \sum_{j} \ln P(x_j \mid y)\right]$$

Take logs: the logarithm only ever increases, so the winner is the same — and sums don't underflow.

READ THE FORMULA **The Naive Bayes rule**

$$\hat{y} = \arg\max_{y}\; P(y) \prod_{j=1}^{d} P(x_j \mid y)$$

- **argmax** — pick the class with the highest score
- **P(y)** — prior — how common the class is
- **Π P(x$_j$ | y)** — multiply the evidence from each feature (the naive independence assumption)

Hover over (or tap) a part to highlight it in the formula.

4. Worked example: a spam filter by hand

Our tiny training set: 10 emails, 4 spam and 6 not spam ("ham").

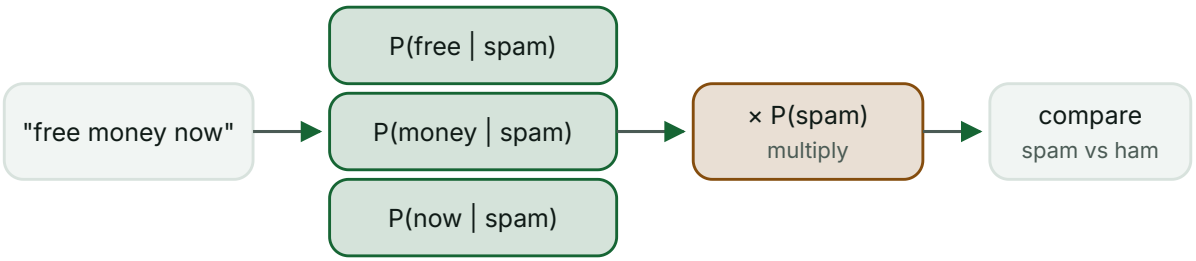


Figure 12. Naive Bayes multiplies the prior with one probability per word for each class (shown for spam; ham is computed the same way), then picks the larger.

Spam (4)	Ham (6)
win money now	meeting at noon
free money offer	lunch with the team now
win a free prize now	project meeting notes
claim your free prize	see you at lunch
	your project report is ready
	notes from the team meeting

Task: is the new email *"free money now"* spam?

Step 1 — priors

$$P(\text{spam}) = 4/10 = 0.4, \quad P(\text{ham}) = 6/10 = 0.6.$$

Step 2 — word probabilities (likelihoods)

We treat each email as a "bag of words" and count how often each word appears in each class. The spam emails contain 15 words in total; the ham emails contain 25. There are 24 different words in the vocabulary overall.

The probability of a word given a class is its count divided by the total word count of that class. But there's a trap: "free" never appears in ham, so $P(\text{free} \mid \text{ham}) = 0/25 = 0$ — and since we

multiply, a single zero would make the whole ham probability zero, no matter what the other words say. One unseen word shouldn't be able to veto everything.

> **≡ LAPLACE (ADD-ONE) SMOOTHING**
>
> Pretend every vocabulary word was seen once more than it really was:
>
> $$P(w \mid y) = \frac{\text{count}(w, y) + 1}{\text{total words in } y + |V|}$$
>
> where $|V|$ is the vocabulary size (24). This guarantees no probability is ever zero. (In scikit-learn this "+1" is the parameter `alpha=1.0`.)

Word	Count in spam	$P(w \mid \text{spam})$	Count in ham	$P(w \mid \text{ham})$
free	3	$\frac{3+1}{15+24} = \frac{4}{39} = 0.1026$	0	$\frac{0+1}{25+24} = \frac{1}{49} = 0.0204$
money	2	$\frac{3}{39} = 0.0769$	0	$\frac{1}{49} = 0.0204$
now	2	$\frac{3}{39} = 0.0769$	1	$\frac{2}{49} = 0.0408$

Step 3 — multiply

$$\text{spam score} = 0.4 \times 0.1026 \times 0.0769 \times 0.0769 \approx 0.000243$$

$$\text{ham score} = 0.6 \times 0.0204 \times 0.0204 \times 0.0408 \approx 0.0000102$$

Step 4 — normalise into probabilities

$$P(\text{spam} \mid \text{"free money now"}) = \frac{0.000243}{0.000243 + 0.0000102} \approx 0.96$$

96% spam. Notice how each word contributed: "free" is about 5× more likely in spam than ham, "money" about 3.8×, and "now" about 1.9× — together they overwhelm the prior, which favoured ham.

5. Working with logs

A real email has hundreds of words. Multiplying hundreds of numbers like 0.001 gives results so small that computers round them to exactly 0 (*underflow*). The fix: take logarithms. Since $\ln(ab) = \ln a + \ln b$, products become sums:

$$\ln\Big(P(y)\prod_j P(x_j \mid y)\Big) = \ln P(y) + \sum_j \ln P(x_j \mid y)$$

The logarithm keeps the ordering (bigger stays bigger), so we can compare the log-scores directly. Every Naive Bayes implementation works this way.

✎ WORKED EXAMPLE — THE SAME EMAIL IN LOG SPACE

Spam: $\ln 0.4 + \ln 0.1026 + \ln 0.0769 + \ln 0.0769 \approx -0.916 - 2.277 - 2.565 - 2.565 = -8.32$.

Ham: $\ln 0.6 + \ln 0.0204 + \ln 0.0204 + \ln 0.0408 \approx -0.511 - 3.892 - 3.892 - 3.199 = -11.49$.

Spam's log-score is higher (less negative) → predict spam. The difference, $-8.32 - (-11.49) = 3.17$, means spam is $e^{3.17} \approx 24$ times more likely, i.e. $24/25 = 0.96$. Same answer.

? MAKE SENSE OF IT

The situation: A real spam filter multiplies hundreds of small probabilities. Computers can't store numbers that tiny, so they add logarithms instead.

What the result tells you: The log scores −8.32 (spam) and −11.49 (ham) differ by 3.17, so spam is $e^{3.17} \approx 24$ times more likely: a 96% chance.

What if…?

- **The email also says "meeting"** (3 times in ham, never in spam): the chance of spam falls to **88%**. One ham-like word pulls it back.
- **"free" appears twice:** its evidence counts twice, and the chance rises to **99.2%**.
- **Spam is rarer** (prior 0.1 instead of 0.4): the same words now give **80%**. The prior matters.
- **No smoothing:** "free" has probability 0 in ham, the ham score is 0, and the email is 100% spam whatever else it says.

Why it matters in ML: Each word adds its own log-evidence for or against a class, so the model is easy to explain: you can list the words that pushed the decision. The "naive" independence assumption makes repeated or correlated words over-count.

ADVANCED · OPTIONAL **Normalising log-scores safely: the log-sum-exp trick** ▾ hide

To turn class log-scores a_k into probabilities you need $P(k) = e^{a_k} / \sum_j e^{a_j}$, but e^{a_k} underflows to 0 when a_k is around −800. Subtract the largest score $m = \max_j a_j$ first:

$$\log \sum_j e^{a_j} = m + \log \sum_j e^{a_j - m}, \qquad P(k) = e^{\,a_k - m} \Big/ \sum_j e^{a_j - m}.$$

Every exponent is now ≤ 0 and the largest is exactly $e^0 = 1$, so nothing overflows and the sum is at least 1. Example: scores -1000 and -1001 would both underflow, but after subtracting $m = -1000$ they become 0 and -1, giving probabilities $1/(1+e^{-1}) \approx 0.731$ and 0.269. The same trick sits inside every softmax in Lessons 18–19.

6. Try it: your own spam filter

This filter is trained on exactly the 10 emails above. Type any message and see the score each word contributes. Green bars push towards spam, purple bars towards ham; the bar length is $\ln \frac{P(w|\text{spam})}{P(w|\text{ham})}$. Words never seen in training are ignored. Try "win a free lunch", "team meeting now", or "claim your prize".

Interactive demo — open *lesson-12.html* on the website to try it.

7. Variants for different kinds of features

The formula $P(y)\prod_j P(x_j \mid y)$ is always the same; what changes is how $P(x_j \mid y)$ is modelled.

Variant	Features	$P(x_j \mid y)$ comes from	Typical use
Multinomial NB	Counts	Smoothed relative frequencies (as above)	Text classification
Bernoulli NB	Binary (present / absent)	Fraction of examples where the feature is 1	Short texts, yes/no features
Gaussian NB	Continuous numbers	A normal distribution per class and feature	Measurements
Complement NB	Counts	Statistics from all *other* classes	Imbalanced text data
Categorical NB	Categories	Smoothed frequency of each category	Categorical data

Gaussian Naive Bayes

For continuous features, assume each feature follows a normal distribution (Lesson 4) within each class, with its own mean μ_y and standard deviation σ_y estimated from the training data:

$$P(x_j \mid y) = \frac{1}{\sigma_y\sqrt{2\pi}} \exp\left(-\frac{(x_j - \mu_y)^2}{2\sigma_y^2}\right)$$

✎ WORKED EXAMPLE — APPLE OR ORANGE?

From training data: apples weigh on average $\mu = 150$ g ($\sigma = 20$); oranges $\mu = 200$ g ($\sigma = 25$). Equal priors (0.5 each). A fruit weighs **170 g**.

Apple: $z = (170 - 150)/20 = 1$. Density $= \frac{1}{20\sqrt{2\pi}} e^{-1^2/2} \approx 0.01995 \times 0.607 \approx 0.0121$.

Orange: $z = (170 - 200)/25 = -1.2$. Density $= \frac{1}{25\sqrt{2\pi}} e^{-1.44/2} \approx 0.01596 \times 0.487 \approx 0.0078$.

$P(\text{apple} \mid 170\text{ g}) = \frac{0.5 \times 0.0121}{0.5 \times 0.0121 + 0.5 \times 0.0078} \approx 0.61$.

A 61% chance it's an apple — 170 g is closer to the apple average, measured in standard deviations. With more features (colour, diameter…) we'd multiply in more densities.

? MAKE SENSE OF IT

The situation: A fruit-sorting machine on a conveyor belt weighs each fruit and must decide: apple or orange?

What the result tells you: 170 g is 1 standard deviation above the apple average but 1.2 below the orange average, so it's a bit more apple-like: 61%.

What if…?

- **160 g:** 80% apple. **175 g:** 49% — almost a coin toss. **185 g:** 25%. **200 g:** 5%.
- **80% of the fruit are apples** (prior 0.8): a 170 g fruit becomes 86% apple, and even 185 g is 56% apple. A strong prior shifts the boundary.

Why it matters in ML: Gaussian Naive Bayes turns each feature into a likelihood using its mean and standard deviation, then multiplies. It trains instantly and gives probabilities — a good first baseline.

8. Strengths and weaknesses

Strengths	Weaknesses
Extremely fast to train: just counting	The independence assumption is usually false
Works with little data and many features (like text)	Probabilities are often over-confident
Handles many classes naturally	Can't learn interactions between features
Easy to explain: each feature adds evidence	Gaussian NB assumes bell-shaped features

Naive Bayes is an excellent **baseline**: quick to try, hard to break. If a more complex model can't beat it, the complexity isn't earning its keep.

9. In Python

The spam filter from this lesson, with scikit-learn. `CountVectorizer` turns each text into word counts:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

spam = ["win money now", "free money offer", "win a free prize now", "claim your free
prize"]
ham = ["meeting at noon", "lunch with the team now", "project meeting notes",
       "see you at lunch", "your project report is ready", "notes from the team
meeting"]

vec = CountVectorizer(token_pattern=r"(?u)\b\w+\b")   # keep 1-letter words like "a"
X = vec.fit_transform(spam + ham)
y = [1] * 4 + [0] * 6                                  # 1 = spam, 0 = ham

nb = MultinomialNB(alpha=1.0).fit(X, y)                # alpha=1 is Laplace smoothing
tests = ["free money now", "meeting notes for the team", "win lunch now"]
print(nb.predict_proba(vec.transform(tests)).round(3))  # columns: [P(ham), P(spam)]
```

Output:

```
[[0.04  0.96 ]
 [0.985 0.015]
 [0.335 0.665]]
```

The first row matches our hand calculation exactly: 96% spam. "win lunch now" is mixed — "win" points to spam, "lunch" to ham — giving 67% spam.

Gaussian NB on the wine dataset from Lesson 11:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)
print(round(GaussianNB().fit(X_train, y_train).score(X_test, y_test), 3))   # 0.956
```

95.6% accuracy with no scaling and no tuning, trained in milliseconds — better than kNN's 93.3% on the same split.

10. Running example: Chennai Homes

Naive Bayes on the listing text alone: words like "new" and "metro" push towards selling fast, "old" and "renovation" away. "New flat near metro" scores 0.96; "old villa needs renovation" just 0.02; "quiet flat with garden" is mixed (0.59).

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
vec = CountVectorizer()
Xw = vec.fit_transform(df["description"])
nb = MultinomialNB(alpha=1.0).fit(Xw, df["sold_fast"])
for text in ["new flat near metro", "old villa needs renovation", "quiet flat
with garden"]:
    print(f"{text:28s} P(sells fast) = {nb.predict_proba(vec.transform([text]))
[0, 1]:.2f}")
```

Output:

```
new flat near metro          P(sells fast) = 0.96
old villa needs renovation   P(sells fast) = 0.02
quiet flat with garden       P(sells fast) = 0.59
```

⚙ WHERE THIS IS USED

- Spam and sentiment classification
- Document categorisation
- A very fast baseline for text

Choose it when: Text or many features, little data, and when speed matters.

Think twice when: Features strongly interact, or you need well-calibrated probabilities.

✕ MISCONCEPTIONS TO UNLEARN

MYTH Naive Bayes needs the features to really be independent.

REALITY It often works well even when they aren't — it only needs the right class to score highest.

MYTH Naive Bayes probabilities are reliable confidence levels.

REALITY They tend to be pushed towards 0 and 1. Use them for ranking, not as exact confidence.

RESPONSIBLE AI · LANGUAGE MODELS OF WHO IS "NORMAL"

Text classifiers learn which words go with which label. Sap et al. (2019) found that tweets written in African-American English were far more likely to be labelled offensive by widely used toxicity datasets and the models trained on them. A spam or moderation filter can silence some groups more than others; test it on varied dialects and writing styles before trusting it.

11. Questions beginners often ask

Q. If the independence assumption is wrong, why does it work?

Because classification only needs the correct class to score highest. Errors in the exact probabilities often don't change which class wins.

Q. Which variant should I use?

Multinomial for word counts, Bernoulli for present/absent features, Gaussian for continuous measurements.

Q. Is Naive Bayes still used today?

Yes — as a fast baseline, for text classification with little data, and wherever training must be nearly instant.

! COMMON MISTAKES TO AVOID

- Forgetting smoothing, so one unseen word sets a class's probability to zero.
- Multiplying many raw probabilities instead of adding log-probabilities (underflow).
- Trusting Naive Bayes probabilities as well-calibrated confidence levels.

! INTERVIEW CORNER

Don't confuse

- **Generative vs discriminative models** — Describe each class vs draw the line between them.
- **Prior vs likelihood vs posterior** — Before, evidence, after.

Hidden questions — answer in your head first, then open.

▼ **Why is Naive Bayes called "naive", and why does it still work?**

The trap: Thinking the assumption must hold for it to work.

It assumes features are independent given the class, which is rarely true (words like "free" and "prize" go together). It still classifies well because it only needs the *right class to score highest*, not accurate probabilities — the ranking often survives the wrong assumption.

▼ **Can you trust Naive Bayes probabilities?**

The trap: Reporting them as calibrated.

Usually not. Counting correlated evidence several times pushes probabilities towards 0 or 1 — in Lesson 12, repeating "free" lifted the spam probability from 96% to 99.2%. Use it for ranking or decisions, and calibrate if you need probabilities.

▼ **A word never appeared in any ham email during training. What happens without smoothing?**

The trap: Thinking one word can't matter much.

Its estimated probability in ham is 0, so the whole ham score is 0 and the email is classified as spam with certainty, whatever its other words say. Laplace (add-one) smoothing prevents this.

▼ **Why do implementations add log-probabilities instead of multiplying probabilities?**

The trap: "It's just a convention."

Multiplying hundreds of small numbers underflows to 0 in floating point. Logs turn the product into a sum of moderate numbers, and the class ranking is unchanged because log is increasing.

12. Key takeaways

- Naive Bayes picks the class maximising $P(y) \prod_j P(x_j \mid y)$ — Bayes' theorem plus an independence assumption.
- The "naive" independence assumption is usually wrong, but the classifier often works well anyway.
- **Laplace smoothing** (+1 counts) prevents a single unseen word from zeroing out a class.
- Implementations add **log-probabilities** instead of multiplying, to avoid underflow.
- Multinomial NB for word counts, Bernoulli for yes/no features, Gaussian for continuous measurements.

13. Check your understanding

1. What does the "naive" assumption say?

a. All classes are equally likely

b. Features follow a normal distribution

c. Features are independent of each other given the class

2. Why is Laplace smoothing needed?

a. To make training faster

b. So a word never seen in a class doesn't make that class's probability exactly zero

c. To remove common words

3. A class has 20 words in total, the vocabulary has 30 words, and "deal" appears 4 times in the class. With add-one smoothing, $P(\text{deal} \mid \text{class})$ is...

a. 5/50 = 0.1

b. 4/20 = 0.2

c. 5/20 = 0.25

4. Why do implementations add log-probabilities instead of multiplying probabilities?

a. Logs are more accurate in theory

b. It changes which class wins

c. Multiplying many small numbers underflows to zero

5. Which variant suits features like height and weight?

a. Multinomial NB

b. Gaussian NB

c. Bernoulli NB

14. Going deeper: smoothing, text pipelines and generative models

INTERMEDIATE · SKIP ON A FIRST READ

Naive Bayes is still a strong, fast baseline for text. This section explains where its smoothing rule really comes from, how to read what it has learned, how a real text pipeline is built (TF-IDF and word pairs), and the big idea it represents: **generative** models, which learn how each class produces data, compared with **discriminative** models such as logistic regression, which learn the boundary directly.

You'll need: this lesson, and Bayesian estimation with priors from Lesson 4's "Going deeper".

Where smoothing comes from

Without smoothing, a word never seen in one class gets probability 0 for that class, and a single such word wipes out all the other evidence. In this tiny spam example, both new messages contain words that appeared in only one class:

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["win cash prize now", "cheap meds win big", "claim your free prize", "free cash
offer now",
         "meeting moved to monday", "lunch on monday?", "project report attached", "see
you at the meeting"]
labels = np.array([1, 1, 1, 1, 0, 0, 0, 0])               # 1 = spam
vec = CountVectorizer().fit(texts)
Xc = vec.transform(texts)
for alpha in [1e-10, 1.0]:
    nb = MultinomialNB(alpha=alpha).fit(Xc, labels)
    p = nb.predict_proba(vec.transform(["free lunch on monday", "win a free meeting"]))
[:, 1]
    print(f"alpha {alpha:g}: P(spam) for the two new messages {p.round(3)}")
```

Output:

```
alpha 1e-10: P(spam) for the two new messages [0. 1.]
alpha 1: P(spam) for the two new messages [0.184 0.735]
```

With essentially no smoothing ($\alpha = 10^{-10}$) the answers are absurdly certain, exactly 0 and 1, decided by one unseen word each. With $\alpha = 1$ (Laplace smoothing) they become sensible: 0.184 and 0.735. The smoothing formula,

$$P(\text{word } j \mid c) = \frac{\text{count}_{j,c} + \alpha}{\text{total}_c + \alpha V}$$

where V is the vocabulary size, is not an arbitrary fix. It is the **Bayesian posterior mean** when the word probabilities get a symmetric **Dirichlet** prior, which is the many-category version of the Beta prior in Lesson 4's "Going deeper". Adding α to every count is like having seen every word α extra times before training. α is a regularization strength: tune it by cross-validation (values between 0.01 and 1 are common for text).

Reading what the model learned

Naive Bayes is easy to interpret. For each word, the log-ratio $\log P(\text{word} \mid \text{spam}) - \log P(\text{word} \mid \text{ham})$ is exactly how much that word adds to the spam score each time it appears:

```
# Which words push towards spam? The log-ratio log P(word | spam) - log P(word | ham)
nb = MultinomialNB(alpha=1.0).fit(Xc, labels)
ratio = nb.feature_log_prob_[1] - nb.feature_log_prob_[0]
words = vec.get_feature_names_out()
order = np.argsort(ratio)
print("most spam-like:", [(str(words[i]), float(round(ratio[i], 2))) for i in
order[::-1][:4]])
print("most ham-like: ", [(str(words[i]), float(round(ratio[i], 2))) for i in
order[:4]])
```

Output:

```
most spam-like: [('win', 1.07), ('cash', 1.07), ('now', 1.07), ('prize', 1.07)]
most ham-like:  [('monday', -1.12), ('meeting', -1.12), ('at', -0.72), ('attached',
-0.72)]
```

"win", "cash", "now" and "prize" each add 1.07 to the log-odds of spam; "monday" and "meeting" subtract 1.12. Notice the classifier is **linear in the word counts**: the log-odds is a weighted sum of counts plus a constant (the log prior ratio). Multinomial naive Bayes is a linear classifier, just like logistic regression, but with its weights set by counting rather than by optimisation.

A realistic text pipeline

Real text classifiers usually add three things:

- **Word pairs (bigrams)**, so "not good" isn't treated as "not" plus "good".
- **TF-IDF weighting**: term frequency × inverse document frequency. Words that appear in almost every document ("the", "to") are down-weighted, and distinctive words count more.
- **Complement naive Bayes**: estimates each class's word statistics from all the *other* classes. It's more stable when classes are imbalanced, and often beats standard multinomial NB on text.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.naive_bayes import ComplementNB

# A text pipeline: words and word pairs, TF-IDF weighting, then naive Bayes
pipe = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)),
ComplementNB(alpha=0.5)).fit(texts, labels)
for msg in ["claim your cash now", "report for the monday meeting"]:
    print(f"{msg!r:34} → {'spam' if pipe.predict([msg])[0] else 'ham'}")
print("number of features (words and word pairs):",
len(pipe[0].get_feature_names_out()))
```

Output:

```
'claim your cash now'              → spam
'report for the monday meeting'    → ham
number of features (words and word pairs): 47
```

Even on 8 training messages the pipeline classifies both new ones correctly, using 47 features (words plus word pairs). On real datasets with thousands of documents, this kind of pipeline trains in seconds and is a hard baseline to beat without much more expensive models. Try a linear SVM or logistic regression on the same TF-IDF features too; on text, they're often slightly more accurate. Modern alternatives replace TF-IDF with learned embeddings or a fine-tuned transformer (Lesson 19).

Generative vs discriminative models

Naive Bayes is a **generative** model: it learns $P(\mathbf{x} \mid y)$ and $P(y)$, a story of how each class *produces* data, and then uses Bayes' theorem to get $P(y \mid \mathbf{x})$. Logistic regression is **discriminative**: it models $P(y \mid \mathbf{x})$ directly and never describes $\mathbf{x}$ itself. The two can be exact counterparts. Gaussian naive Bayes with a shared variance per feature produces a log-odds that is linear in $\mathbf{x}$, the same form as logistic regression.

	Generative (naive Bayes, LDA, mixture models)	**Discriminative (logistic regression, SVM, neural nets)**
Learns	How each class generates features	The boundary between classes
With little data	Often reaches its best accuracy sooner	Needs more data
With lots of data	Limited by wrong assumptions	Usually more accurate
Missing features	Handled naturally (leave them out of the product)	Need imputation
Can generate new examples	Yes	No
Probabilities	Often badly calibrated (Lesson 8's "Going deeper")	Usually better calibrated

A well-known analysis (Ng and Jordan, 2001) showed that naive Bayes approaches its *own* best error with far fewer examples (roughly logarithmic in the number of features), while logistic

regression needs more (roughly linear) but ends lower when the independence assumption is wrong. In practice: with very few labelled examples, or as a quick baseline, try naive Bayes; with plenty of data, a discriminative model usually wins. Today's generative AI (Lesson 19) is the generative idea at enormous scale: models of $P(\mathbf{x})$ that can produce new text or images.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Naive Bayes vs logistic regression for text: how do you choose?**

The trap: Saying one is always better.

Naive Bayes trains by counting, needs very little data to reach its best accuracy, and handles huge vocabularies cheaply, but its independence assumption double-counts correlated words, so its probabilities are over-confident and its final accuracy is often lower. Logistic regression (or a linear SVM) on the same TF-IDF features usually wins with enough data and gives better-calibrated probabilities. Try both: they're cheap. NB is great as a baseline, for tiny datasets, or for streaming updates.

▼ **What is TF-IDF and why use it instead of raw counts?**

The trap: Only giving the acronym.

Term frequency × inverse document frequency: a word's count in a document, multiplied by $\log(N/\mathrm{df})$, where df is the number of documents containing it (implementations add smoothing). Very common words get weights near 0; distinctive words get more. Vectors are usually normalised to unit length, so long documents don't dominate. It gives linear models and kNN a much better representation than raw counts.

▼ **Why is the smoothing parameter α a regularization hyperparameter?**

The trap: Thinking it only avoids division by zero.

It pulls every class's word distribution towards uniform, like a prior. A large α means estimates trust the prior more, giving simpler, less confident models (more bias, less variance); a tiny α trusts sparse counts, giving extreme, overfit probabilities. So it's tuned by cross-validation like any regularization strength.

Check your understanding: going deeper

1. Laplace smoothing in naive Bayes is the same as...

a. A Bayesian prior that adds α imaginary counts of every word

b. Removing rare words

c. Scaling the features

2. What does TF-IDF do to a word that appears in almost every document?

a. Gives it a large weight

b. Gives it a small weight

c. Removes the document

3. Multinomial naive Bayes's decision rule is...

a. Linear in the word counts

b. Quadratic in the word counts

c. A decision tree

4. Which model is generative?

a. Logistic regression

b. Gaussian naive Bayes

c. A linear SVM

5. With only 30 labelled examples and 1,000 word features, a quick strong baseline is...

a. A deep neural network

b. Naive Bayes

c. kNN with Euclidean distance

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

With add-one smoothing, a word never seen in the ham class, ham total word count 50, vocabulary size 100. What is $P(\text{word} \mid \text{ham})$?

WORKING

EXERCISE 2 By hand

Priors: spam 0.5, ham 0.5. $P(\text{cheap} \mid \text{spam}) = 0.2$, $P(\text{cheap} \mid \text{ham}) = 0.02$; $P(\text{meeting} \mid \text{spam}) = 0.01$, $P(\text{meeting} \mid \text{ham}) = 0.1$. Classify "cheap meeting".

WORKING

EXERCISE 3 By hand

Gaussian NB, one feature. Class A: $\mu = 10$, $\sigma = 2$. Class B: $\mu = 14$, $\sigma = 2$. Equal priors. Classify $x = 11.5$.

WORKING

EXERCISE 4 Concepts

Why does Naive Bayes often classify well even though its probabilities are over-confident?

WORKING

EXERCISE 5 Intermediate

A word appears 3 times in spam emails, which contain 200 words in total. The vocabulary has 1,000 words. With $\alpha = 1$, what is $P(\text{word} \mid \text{spam})$? What would it be for a word never seen in spam?

WORKING

EXERCISE 6 Intermediate

A word appears in 10 of 1,000 documents. Using $\text{idf} = \ln(N/\text{df})$, compute its IDF. A word in all 1,000 documents gets what IDF?

WORKING

EXERCISE 7 Intermediate

The spam log-odds starts at the log prior ratio $\ln(0.4/0.6)$. A message contains one word with log-ratio $+1.5$ and two copies of a word with log-ratio -0.3. What are the final log-odds and $P(\text{spam})$?

WORKING

NOTES

PART 3 · CLASSIC ALGORITHMS · ~50 MIN

Lesson 13: Decision Trees

Twenty questions for data: Gini impurity, entropy, information gain and pruning.

BY THE END OF THIS LESSON YOU CAN

- Read and follow a decision tree to make a prediction
- Compute Gini impurity, entropy and information gain by hand
- Explain how the CART algorithm chooses splits
- Control overfitting with depth limits and pruning

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 9: Evaluating Models — Accuracy can mislead. Precision $= TP/(TP + FP)$, recall $= TP/(TP + FN)$, and F1 balances them. Moving the threshold trades one for the other; ROC-AUC summarises all thresholds.

Where this fits: A completely different kind of model — questions instead of equations. Trees are the building blocks of the ensembles in Lesson 14.

1. Twenty questions with data

Remember the game "twenty questions"? You find an object by asking yes/no questions — "Is it alive?", "Is it bigger than a cat?" — each one narrowing down the possibilities. A **decision tree** classifies data the same way: it asks a sequence of simple questions about the features, like "Is petal width ≤ 0.8 cm?", until it reaches an answer.

Trees are one of the most popular model families in practice. A single tree is easy to read and explain; combined into forests and boosted ensembles (Lesson 14), trees are often the strongest models available for tabular data — spreadsheets and databases.

2. Anatomy of a tree

- The **root node** at the top holds all the training data and asks the first question.

- Each **internal node** tests one feature against a threshold: "$x_j \leq t$?". Yes goes left, no goes right.
- Each **leaf** at the bottom gives a prediction: the majority class of the training examples that ended up there (or their average value, for regression).
- The **depth** is the number of questions on the longest path from root to leaf.

Here is a real depth-2 tree trained by scikit-learn on the Iris flower dataset (3 species, 4 measurements):

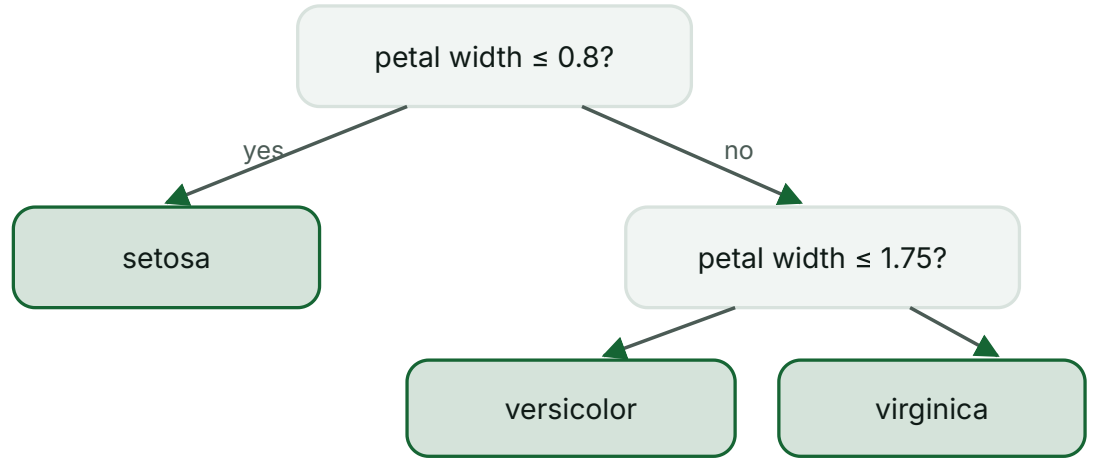


Figure 13. The depth-2 iris tree from this lesson. Internal nodes ask questions; leaves (green) give the prediction.

```
|--- petal width (cm) <= 0.80
|   |--- class: 0              (setosa)
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1          (versicolor)
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2          (virginica)
```

To classify a flower with petal width 1.2 cm: is $1.2 \leq 0.8$? No → go right. Is $1.2 \leq 1.75$? Yes → **versicolor**. Anyone can follow that — a big advantage when decisions must be explained.

◎ WHAT THE TREE DOES TO THE FEATURE SPACE

Each question cuts the space with a straight line parallel to an axis. The tree chops the space into rectangles (boxes, in more dimensions), and each box gets one prediction. So decision boundaries look like staircases made of horizontal and vertical segments. You'll see this in the demo.

3. How does a tree choose its questions? Impurity

PICTURE IT Sorting laundry: a pile of only white shirts is "pure" — you know exactly what to do with it. A pile of mixed colours is "impure". A decision tree keeps asking questions until its piles are as pure as possible.

A good question separates the classes: after splitting, each side should be as "pure" as possible — ideally containing just one class. We need a number that measures impurity. Let p_k be the fraction of examples in a node that belong to class k.

Gini impurity — the probability of mislabelling a random example if you labelled it randomly according to the node's class mix:

$$G = \sum_k p_k(1 - p_k) = 1 - \sum_k p_k^2$$

Entropy — from information theory, the average number of yes/no questions (bits) needed to identify the class:

$$H = -\sum_k p_k \log_2 p_k$$

Both are 0 for a pure node and largest when classes are evenly mixed. For two classes the maximum Gini is 0.5 and the maximum entropy is 1 bit.

✎ WORKED EXAMPLE — IMPURITY OF A NODE

A node has 10 examples: 6 "yes", 4 "no". So $p_{\text{yes}} = 0.6$, $p_{\text{no}} = 0.4$.

Gini: $1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 0.48$.

Entropy: $-(0.6 \log_2 0.6 + 0.4 \log_2 0.4) = -(0.6 \times (-0.737) + 0.4 \times (-1.322)) \approx 0.971$ bits.

A pure node (10 yes, 0 no): Gini $= 1 - 1^2 = 0$, entropy $= -1 \cdot \log_2 1 = 0$.

? MAKE SENSE OF IT

The situation: A bank sorts loan applicants into groups. A group where everyone repaid is easy to act on; a 50/50 group isn't. Impurity measures that mix.

What the result tells you: Gini 0.48 means: pick two people at random from this node and there's a 48% chance they have different labels. Entropy 0.971 bits says you'd need almost one yes/no question to learn the label.

What if…?

- **5 yes, 5 no:** Gini **0.5**, entropy **1 bit** — the worst possible mix for two classes.
- **9 yes, 1 no:** Gini 0.18, entropy 0.47 — nearly pure.
- **Three classes, equally mixed:** Gini 0.667, entropy 1.585 bits — more classes, more uncertainty.

Why it matters in ML: A tree's whole job is to make its leaves pure. Impurity is the score it uses to choose every question it asks.

READ THE FORMULA **Gini impurity**

$$G = \sum_k p_k(1 - p_k)$$

- **Σ_k** — add up over every class
- **p_k** — chance a random example in the node is class k
- **$1 - p_k$** — chance a random label would then be wrong

Hover over (or tap) a part to highlight it in the formula.

4. Scoring a split: information gain

A split sends n_L examples left and n_R right. Its quality is the **weighted impurity** of the two children, weighted by how many examples each gets:

$$G_{\text{split}} = \frac{n_L}{n} G(\text{left}) + \frac{n_R}{n} G(\text{right})$$

The improvement over the parent is the **impurity decrease** — called **information gain** when using entropy:

$$\text{Gain} = G(\text{parent}) - G_{\text{split}}$$

✎ WORKED EXAMPLE — COMPARE TWO CANDIDATE SPLITS

Parent: 6 yes / 4 no, Gini 0.48 (from above).

Split A sends 4 examples left (4 yes, 0 no) and 6 right (2 yes, 4 no).

- Left Gini $= 0$ (pure).
- Right Gini $= 1 - \left((2/6)^2 + (4/6)^2\right) = 1 - (0.111 + 0.444) \approx 0.444$.
- Weighted: $\frac{4}{10} \times 0 + \frac{6}{10} \times 0.444 \approx 0.267$. Gain $= 0.48 - 0.267 = 0.213$.

Split B sends 5 left (3 yes, 2 no) and 5 right (3 yes, 2 no).

- Both children have the same 60/40 mix as the parent: Gini 0.48 each. Weighted: 0.48. Gain $= 0$.

Split B is useless — it doesn't separate anything. Split A wins. (With entropy, split A's information gain is $0.971 - 0.6 \times 0.918 \approx 0.420$ bits; the ranking is the same.)

? MAKE SENSE OF IT

The situation: The tree has to pick its first question. Each candidate split divides the 10 applicants differently.

What the result tells you: Split A creates one perfectly pure group and cuts impurity from 0.48 to 0.267 — a gain of 0.213. Split B leaves both groups as mixed as before, so it's worthless.

What if...?

- **A perfect split** (6 yes on one side, 4 no on the other): both children are pure and the gain is the full **0.48**.
- **Peeling off one example** (1 yes alone, 5 yes + 4 no together): a pure leaf, but the gain is only **0.036**. Tiny pure leaves look tempting and help little — that's how deep trees overfit.

Why it matters in ML: Trees choose splits greedily, one at a time, by the biggest gain. Limiting depth or requiring a minimum number of samples per leaf stops them chasing tiny pure leaves.

ADVANCED · OPTIONAL **Why a split never increases weighted impurity** ▾ hide

Gini impurity $G(\mathbf{p}) = 1 - \sum_k p_k^2$ and entropy $H(\mathbf{p}) = -\sum_k p_k \log_2 p_k$ are **concave** functions of the class proportions $\mathbf{p}$. A split sends a fraction w_L of a node's examples left and $w_R =$

$1 - w_L$ right, and the parent's proportions are the weighted average $\mathbf{p} = w_L \mathbf{p}_L + w_R \mathbf{p}_R$. Concavity (Jensen's inequality) gives

$$G(w_L \mathbf{p}_L + w_R \mathbf{p}_R) \;\geq\; w_L G(\mathbf{p}_L) + w_R G(\mathbf{p}_R),$$

so the information gain is always ≥ 0, and it is 0 only when both children have the same class mix as the parent. That's why a tree keeps finding "useful" splits all the way down to pure leaves — and why it must be stopped deliberately (Lesson 13's overfitting section).

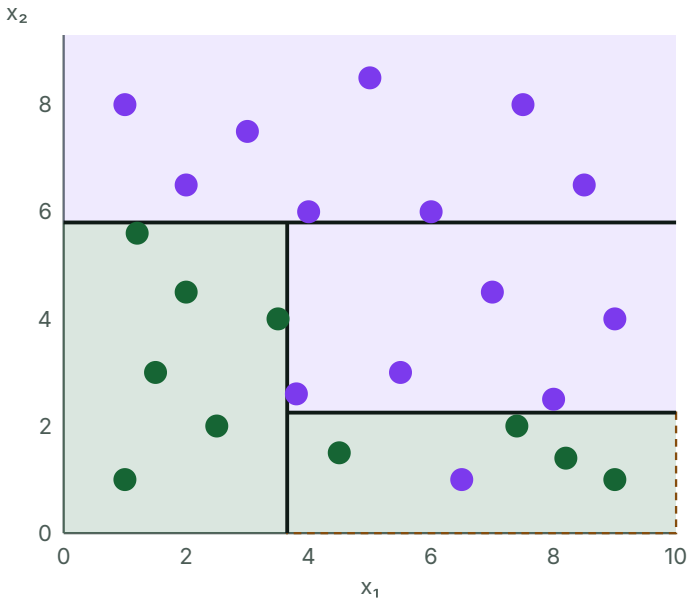
5. Growing the tree: the CART algorithm

scikit-learn builds trees with an optimised version of **CART** (Classification And Regression Trees). It's *greedy* and *recursive*:

1. At the current node, try every feature, and for each feature every possible threshold (the midpoints between sorted values).
2. Pick the feature and threshold with the largest impurity decrease.
3. Split the data into left and right children.
4. Repeat on each child, until a stopping rule is met (node is pure, maximum depth reached, too few examples to split…).

"Greedy" means it picks the best split *right now*, without looking ahead. That's not guaranteed to find the best possible tree (that problem is far too hard to solve exactly), but it works very well in practice.

WALKTHROUGH (FINAL STEP SHOWN) **A decision tree grows, split by split**



Depth 3 reached: 4 leaves

training accuracy 96%

1. Start: one region holding all 24 points (10 green, 14 violet). Gini impurity 0.486. The tree tries every possible threshold on every feature and keeps the one that lowers impurity the most.
2. Split 1: the best question for this region is "$x_2 \le 5.8$?". It divides 24 points into 16 and 8, lowering the weighted Gini from 0.486 to 0.312.
3. Split 2: the best question for this region is "$x_1 \le 3.65$?". It divides 16 points into 6 and 10, lowering the weighted Gini from 0.469 to 0.300.
4. Split 3: the best question for this region is "$x_2 \le 2.25$?". It divides 10 points into 5 and 5, lowering the weighted Gini from 0.480 to 0.160.
5. The maximum depth (3) is reached. Each of the 4 rectangles is a leaf; a new point gets the majority class of the rectangle it falls in. Training accuracy: 96%. Shaded rectangles still containing both colours are impure.

6. Try it: watch a tree carve up the space

The tree below is grown on the points using Gini impurity. Increase the maximum depth and watch it add axis-parallel cuts. At low depth it underfits; at high depth it carves tiny boxes around individual points — overfitting. The first few splits are listed underneath.

Interactive demo — open *lesson-13.html* on the website to try it.

7. Trees for regression

For predicting numbers, a leaf predicts the **average** y of its training examples, and impurity is measured by the **variance** (equivalently the MSE) within the node. A split is good if it creates

groups whose y values are tightly clustered. The result is a step function: flat predictions within each box, with jumps between boxes.

8. Trees overfit easily — and how to stop them

Left alone, a tree keeps splitting until every leaf is pure — it can memorise the training set perfectly, noise included. Ways to control it (all are hyperparameters, chosen by cross-validation):

- `max_depth` — limit the number of questions.
- `min_samples_split` / `min_samples_leaf` — don't split tiny groups; every leaf must contain at least this many examples.
- `max_leaf_nodes` — cap the total number of leaves.
- **Cost-complexity pruning** (`ccp_alpha`) — grow a full tree, then cut back branches that don't earn their keep. It minimises

$$R_\alpha(T) = R(T) + \alpha\,|T|$$

where $R(T)$ is the tree's training error (impurity), $|T|$ is its number of leaves, and α charges a price per leaf — the same "fit + penalty" idea as regularization in Lesson 10.

9. Feature importance

Each split reduces impurity. Add up those reductions for each feature (weighted by the number of examples reaching the node) and normalise so they sum to 1: that's the tree's **feature importance**. It tells you which features the tree relied on most.

> **! READ IMPORTANCES WITH CARE**
>
> Impurity-based importance favours features with many distinct values (like IDs or continuous measurements), and when two features are correlated, the tree may use just one, making the other look unimportant. It describes the model, not causation.

10. Strengths and weaknesses

Strengths	Weaknesses
Easy to read and explain	Overfit easily without limits
No feature scaling needed — splits compare a feature with a threshold, so units don't matter	**Unstable**: a small change in data can produce a very different tree (high variance)
Handles numeric and categorical features, non-linear patterns and interactions	Boundaries are axis-parallel staircases; a diagonal boundary needs many steps
Fast to train and predict	A single tree is rarely the most accurate model

The instability problem has a beautiful fix: train *many* trees and combine them. That's Lesson 14.

11. In Python

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, stratify=data.target, random_state=0)

for depth in [1, 2, 3, 5, None]:          # None = grow until leaves are pure
    tree = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_train, y_train)
    print(f"max_depth={str(depth):4s} depth={tree.get_depth()} leaves=
{tree.get_n_leaves():2d} "
          f"train={tree.score(X_train, y_train):.3f} test={tree.score(X_test,
y_test):.3f}")

tree = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_train, y_train)
top = sorted(zip(tree.feature_importances_, data.feature_names), reverse=True)[:3]
print([(str(name), round(float(imp), 3)) for imp, name in top])
```

Output:

```
max_depth=1    depth=1 leaves= 2 train=0.930 test=0.888
max_depth=2    depth=2 leaves= 4 train=0.932 test=0.888
max_depth=3    depth=3 leaves= 8 train=0.977 test=0.916
max_depth=5    depth=5 leaves=18 train=1.000 test=0.902
max_depth=None depth=5 leaves=18 train=1.000 test=0.902
[('worst perimeter', 0.791), ('worst concave points', 0.115), ('worst radius', 0.04)]
```

Depth 3 does best on unseen data (91.6%). Deeper trees reach 100% training accuracy but do *worse* on the test set — overfitting again. The depth-3 tree leans heavily on one feature, "worst perimeter" (79% of the importance). Notice that a single tree (91.6%) is below logistic regression's ~96% on this data (Lesson 8) — Lesson 14 closes that gap.

Print any tree as text with `sklearn.tree.export_text(tree, feature_names=...)`, or draw it with `sklearn.tree.plot_tree`.

12. Running example: Chennai Homes

A depth-2 tree finds a simple rule: houses within 11.5 km and no older than 16 years sold fast. It fits all 12 training houses perfectly — which, with this little data, is more likely luck than proof (Lesson 5).

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.tree import DecisionTreeClassifier, export_text
cols = ["size_m2", "dist_km", "age_yrs"]
tree = DecisionTreeClassifier(max_depth=2, random_state=0).fit(df[cols],
df["sold_fast"])
print(export_text(tree, feature_names=cols))
print("training accuracy:", tree.score(df[cols], df["sold_fast"]))
```

Output:

```
|--- dist_km ⩽ 11.50
|   |--- age_yrs ⩽ 16.00
|   |   |--- class: 1
|   |--- age_yrs >  16.00
|   |   |--- class: 0
|--- dist_km >  11.50
|   |--- class: 0

training accuracy: 1.0
```

⚙ WHERE THIS IS USED

- Business rules that must be explainable (e.g. eligibility decisions)
- Quick exploration of which features matter
- The building block of random forests and boosting

Choose it when: You need a readable model, or features are a mix of numbers and categories.

Think twice when: You need top accuracy from a single model — use an ensemble instead.

✕ MISCONCEPTIONS TO UNLEARN

MYTH Feature importance shows which features cause the outcome.

REALITY It shows what the tree used for splitting — biased towards features with many values, and silent about correlated alternatives.

MYTH A deeper tree is a better tree.

REALITY Deeper trees fit training data better but overfit; depth must be tuned.

RESPONSIBLE AI · TRANSPARENT, BUT NOT AUTOMATICALLY FAIR

A small tree's rules can be read and audited, which is a real advantage for accountability. But readable is not the same as fair: a split on a proxy feature (such as a postcode) can encode discrimination plainly in the rules. Audit the splits, not just the accuracy.

13. Questions beginners often ask

Q. Gini or entropy?

They usually give very similar trees. Gini is scikit-learn's default and slightly faster to compute.

Q. Why don't trees need scaling?

Each split compares one feature with a threshold, so changing a feature's units just changes the threshold.

Q. How deep should a tree be?

Choose max_depth (or min_samples_leaf) by cross-validation; shallow trees for explanation, deeper ones inside ensembles.

! COMMON MISTAKES TO AVOID

- Letting trees grow without limits and trusting their 100% training accuracy.
- Scaling features for trees (harmless, but unnecessary).
- Reading impurity-based feature importance as causal, or as fair between correlated features.

! INTERVIEW CORNER

Don't confuse

- **Gini impurity vs entropy** — Two rulers for the same mess.
- **Pre-pruning vs post-pruning** — Stop early vs trim afterwards.
- **Impurity importance vs permutation importance** — Credit claimed in training vs damage done when removed.

Hidden questions — answer in your head first, then open.

▼ **Do decision trees need feature scaling?**

The trap: Scaling everything by habit.

No. A split asks "is $x \leq t$?", and rescaling x just rescales t; the tree is identical. The same goes for random forests and gradient-boosted trees.

▼ **A regression tree is trained on $x = 0, \ldots, 9$ with $y = 2x$. What does it predict for $x = 100$?**

The trap: Expecting 200.

18 — the value of the last leaf. Trees predict constants inside regions, so they can't extrapolate beyond the training range. Random forests and boosted trees share this limitation; linear models don't.

▼ **Why do fully grown decision trees overfit?**

The trap: "Trees are just bad."

Without limits a tree keeps splitting until every leaf is pure, often isolating single noisy examples. Limit depth or leaf size, prune, or average many trees (a random forest).

▼ **Gini or entropy — does the choice matter much?**

The trap: Spending a long time tuning it.

Rarely. They rank splits almost identically (in Lesson 13's example both preferred split A). Gini is slightly faster to compute. Depth and leaf-size limits matter far more.

14. Key takeaways

- A decision tree asks a sequence of "feature ≤ threshold?" questions; leaves hold predictions.
- Splits are chosen to reduce impurity: **Gini** $1 - \sum p_k^2$ or **entropy** $-\sum p_k \log_2 p_k$.
- The CART algorithm greedily picks the split with the largest impurity decrease, then repeats on each child.
- Unrestricted trees overfit; control them with `max_depth`, `min_samples_leaf` or pruning (`ccp_alpha`).
- Trees are readable and need no scaling, but are unstable — which ensembles fix.

15. Check your understanding

1. A node has 5 examples of class A and 5 of class B. What is its Gini impurity?

a. 0

b. 0.5

c. 1

2. Which split is better: one that creates two pure children, or one whose children have the same class mix as the parent?

a. Two pure children

b. Same mix as the parent

c. They're equally good

3. Why don't decision trees need feature scaling?

a. They standardise features internally

b. They only work with categorical features

c. Each split compares one feature with a threshold, so the feature's units don't matter

4. A tree has 100% training accuracy and 85% test accuracy. What would most likely help?

a. Allow deeper trees

b. Limit max_depth or increase min_samples_leaf

c. Use entropy instead of Gini

5. What shape are a decision tree's decision boundaries?

a. Axis-parallel staircases (boxes)

b. Single straight diagonal lines

c. Smooth curves

16. Going deeper: pruning, honest importances and instability

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above grows a tree and limits its depth. This section covers the principled way to shrink a tree (**cost-complexity pruning**, chosen by cross-validation), why the feature importances printed by tree libraries can mislead you (and the honest alternative), and the trees' biggest weakness, **instability**, which is exactly what random forests and boosting (Lesson 14) are designed to fix.

You'll need: this lesson, cross-validation from Lesson 5 and the bootstrap from Lesson 4's "Going deeper".

Cost-complexity pruning

Limits like `max_depth` stop growth early, but a split that looks useless can enable a very useful split below it. **Pruning** grows the full tree first, then cuts back. Cost-complexity pruning scores a tree T by its training error plus a price per leaf:

$$R_\alpha(T) = R(T) + \alpha\,|\text{leaves}(T)|$$

For each α there's a best subtree. As α grows, branches whose improvement isn't worth their extra leaves are removed one at a time ("weakest-link" pruning), giving a nested sequence of smaller and smaller trees. Scikit-learn computes the whole sequence:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
path = DecisionTreeClassifier(random_state=0).cost_complexity_pruning_path(Xtr, ytr)
print(" ccp_alpha   leaves   train acc   test acc")
for a in path.ccp_alphas[::3]:
    t = DecisionTreeClassifier(random_state=0, ccp_alpha=a).fit(Xtr, ytr)
    print(f"{a:9.4f}   {t.get_n_leaves():6d}   {t.score(Xtr, ytr):9.3f}   {t.score(Xte,
yte):8.3f}")
```

Output:

```
 ccp_alpha   leaves   train acc   test acc
   0.0000       16       1.000      0.906
   0.0042       11       0.992      0.912
   0.0088        6       0.977      0.918
   0.0259        3       0.932      0.889
```

The full tree (16 leaves) fits the training data perfectly but scores 0.906 on the test set. Pruning to 6 leaves gives up a little training accuracy and *improves* the test score to 0.918; pruning to 3 leaves goes too far (0.889). It's the bias–variance trade-off again (Lesson 10), with α as the knob. Choose α by cross-validation on the training set only:

```
from sklearn.model_selection import GridSearchCV
search = GridSearchCV(DecisionTreeClassifier(random_state=0), {"ccp_alpha":
path.ccp_alphas}, cv=5).fit(Xtr, ytr)
best = search.best_estimator_
print("alpha chosen by CV:", round(search.best_params_["ccp_alpha"], 4), " leaves:",
best.get_n_leaves(), " test accuracy:", round(best.score(Xte, yte), 3))
```

Output:

```
alpha chosen by CV: 0.0088  leaves: 6  test accuracy: 0.918
```

Cross-validation picks $\alpha \approx 0.0088$, the 6-leaf tree, which scores 0.918 on the untouched test set.

Feature importance: impurity vs permutation

The default `feature_importances_` adds up how much each feature reduced impurity across all splits (the "mean decrease in impurity"). Two problems:

- It's measured on the **training** data, so splits that only memorise noise still earn credit.
- It's biased towards features with **many distinct values** (continuous numbers, IDs), because they offer many possible split points and some will fit noise well by chance.

Permutation importance fixes both. On held-out data, shuffle one feature's column, breaking its link to the target, and measure how much the score drops. A feature the model truly relies on causes a big drop; a useless one causes none. Here we add a column of pure noise with a different value on every row, like a customer ID:

```
from sklearn.inspection import permutation_importance

# Add a column of pure noise with many distinct values (like an ID number)
rng = np.random.default_rng(0)
Xn_tr = np.column_stack([Xtr, rng.permutation(len(ytr))])
Xn_te = np.column_stack([Xte, rng.permutation(len(yte))])
tree = DecisionTreeClassifier(random_state=0).fit(Xn_tr, ytr)
imp = tree.feature_importances_
perm = permutation_importance(tree, Xn_te, yte, n_repeats=30,
random_state=0).importances_mean
print("noise column: impurity importance", round(imp[-1], 3), " rank", int((imp >
imp[-1]).sum()) + 1, "of 31")
print("noise column: permutation importance on test data", round(perm[-1], 3))
```

Output:

```
noise column: impurity importance 0.011  rank 6 of 31
noise column: permutation importance on test data 0.0
```

Impurity importance ranks the meaningless noise column **6th of 31**, ahead of 25 real medical measurements. Permutation importance on the test data gives it exactly 0. Use permutation importance (or SHAP values) when you report which features matter. Two further cautions: when features are correlated, shuffling one lets the model lean on its partner, so both can look unimportant; and importance means "the model uses it", not "it causes the outcome".

Trees are unstable

Because each split is chosen greedily and every later split depends on it, a small change in the data can change the top split, and with it the whole tree. Here we grow a tree on 100 bootstrap samples of the same training data and record the feature used at the root:

```
# Instability: which feature does the root split use on bootstrap samples?
names = load_breast_cancer().feature_names
roots = {}
for _ in range(100):
    i = rng.integers(0, len(ytr), len(ytr))
    r = names[DecisionTreeClassifier(random_state=0).fit(Xtr[i],
ytr[i]).tree_.feature[0]]
    roots[str(r)] = roots.get(str(r), 0) + 1
print(sorted(roots.items(), key=lambda kv: -kv[1]))
```

Output:

```
[('worst perimeter', 52), ('worst concave points', 30), ('worst radius', 8), ('mean
concave points', 7), ('worst area', 3)]
```

Five different features end up at the root: "worst perimeter" 52 times, "worst concave points" 30 times, and three others. Each tree tells a different story, so explaining a decision by reading a single tree's structure is fragile. This high variance is the weakness that **bagging** and **random forests** (Lesson 14) exploit: average many unstable trees and the variance cancels, while keeping their low bias.

Practical details worth knowing

- **Regression trees** choose splits that most reduce the squared error, and predict the average of each leaf. They can't extrapolate beyond the training range (Lesson 5's "Going deeper" showed this with a forest).
- **Categorical features.** Scikit-learn trees need numbers; one-hot encoding works, though very many categories make splits weak. LightGBM and CatBoost can split categories directly.
- **Missing values.** Recent scikit-learn trees can learn which side missing values should go to; gradient-boosting libraries have done this for years.
- **Finding the best tree is hard.** Finding the smallest accurate tree is NP-hard, which is why every library grows trees greedily, one split at a time.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How would you prune a decision tree, and how do you choose how much?**

The trap: Only mentioning max_depth.

Grow the full tree, then apply cost-complexity (weakest-link) pruning: minimise training error + α × number of leaves. Increasing α gives a nested sequence of subtrees. Pick α by cross-validation (often the simplest tree within one standard error of the best score). Pre-pruning limits (max_depth, min_samples_leaf) are cheaper alternatives, but they can stop before a useful deeper split.

▼ **Your stakeholder asks which features drive the model. What do you report, and what caveats do you give?**

The trap: Pasting the default feature_importances_ bar chart.

Use permutation importance on held-out data (or SHAP values for per-prediction explanations), with error bars from repeats. Caveats: correlated features share or hide importance, so group them; importance describes what the model uses, not what causes the outcome; and results depend on the model, so a different good model might rank features differently.

▼ **Why do people say random forests fix the weaknesses of decision trees?**

The trap: "Because more trees is better."

A deep tree has low bias but high variance: small data changes give very different trees. Averaging many trees trained on bootstrap samples (bagging), with random feature subsets at each split to make them less correlated, cancels much of that variance while keeping low bias. The price is interpretability and some speed.

Check your understanding: going deeper

1. In cost-complexity pruning, what does a larger α do?

a. Grows a bigger tree

b. Charges more per leaf, so the tree becomes smaller

c. Changes the impurity measure

2. Why can impurity-based importance rank a random ID column highly?

a. IDs are always useful

b. Features with many distinct values offer many split points, some of which fit training noise

c. It's computed on test data

3. Permutation importance measures...

a. How much the held-out score drops when one feature's values are shuffled

b. How many splits use the feature

c. The feature's correlation with the target

4. Two strongly correlated features are both useful. What can permutation importance show for each?

a. Both look very important

b. Both can look unimportant, because the model falls back on the partner

c. It always picks the right one

5. Why are single decision trees unstable?

a. Greedy top splits change with small data changes, and everything below depends on them

b. They use random numbers

c. They need feature scaling

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

A node has 3 examples of class A and 1 of class B. Compute its Gini impurity and entropy.

WORKING

EXERCISE 2 By hand

Parent: 5 A / 5 B. A split gives left 4 A / 1 B and right 1 A / 4 B. Compute the Gini impurity decrease.

WORKING

EXERCISE 3 By hand

Using the iris tree in this lesson, classify a flower with petal width 2.1 cm, and one with 0.4 cm.

WORKING

EXERCISE 4 Code

Train `DecisionTreeClassifier(max_depth=2)` on iris and print it with `export_text`. Does it match the tree in this lesson?

WORKING

EXERCISE 5 Intermediate

A tree with 10 leaves has training error 0.05. Pruning one branch gives 7 leaves with training error 0.08. For which values of α is the pruned tree preferred?

WORKING

EXERCISE 6 Intermediate

A node holds 40 positives and 60 negatives. A split sends 30 positives and 10 negatives left, and 10 positives and 50 negatives right. Compute the Gini impurity before the split and the weighted Gini after it.

WORKING

EXERCISE 7 Intermediate

A model scores 0.90 on test data. After shuffling feature A it scores 0.84, and after shuffling feature B, 0.90. What are the two permutation importances, and what might B's value mean?

WORKING

NOTES

PART 3 · CLASSIC ALGORITHMS · ~50 MIN

Lesson 14: Random Forests & Boosting

Combine many trees: bagging, random forests, AdaBoost and gradient boosting.

BY THE END OF THIS LESSON YOU CAN

- Explain mathematically why combining diverse models reduces error
- Describe bagging, bootstrap samples and out-of-bag evaluation
- Explain how random forests decorrelate trees
- Run AdaBoost and gradient boosting rounds by hand

↺ BEFORE YOU START

This lesson builds on:

- Lesson 10: Overfitting, Regularization & Bias–Variance — Error = bias² + variance + noise. Regularization adds a penalty on weight size (L2 ridge shrinks weights; L1 lasso zeros some) to reduce variance; its strength is chosen by cross-validation.
- Lesson 13: Decision Trees — A decision tree asks a sequence of 'feature ≤ threshold?' questions chosen to reduce impurity (Gini or entropy). Deep trees overfit and are unstable.

Where this fits: Combines many trees into some of the most accurate models available for tabular data — often the first thing professionals try.

1. The wisdom of crowds

In 1906, at a country fair, about 800 people guessed the weight of an ox. Individual guesses were all over the place — but their median was within 1% of the true weight. Many independent, imperfect opinions, combined, beat almost any single expert.

Ensemble methods apply this to models: train many models and combine their predictions. Lesson 13 showed that a single decision tree is unstable — small changes in data produce a different tree. Ensembles turn that weakness into a strength. Random forests and gradient boosting, the two families in this lesson, win a large share of real-world competitions on tabular data.

2. Why combining models works: the maths

PICTURE IT At a village fair, nobody guesses the ox's weight exactly, but the average of hundreds of guesses is remarkably close. Many imperfect models whose errors differ can be better together than any one of them.

Suppose you have 25 classifiers, each correct 60% of the time, and — crucially — they make their mistakes **independently**. You take a majority vote. The vote is right whenever at least 13 of the 25 are right. Using the binomial distribution:

$$P(\text{majority correct}) = \sum_{k=13}^{25} \binom{25}{k} (0.6)^k (0.4)^{25-k} \approx 0.846$$

Twenty-five 60%-accurate models make an **85%-accurate** committee. With 101 such models it rises to about 98%.

($\binom{25}{k}$, "25 choose k", counts the ways to pick which k of the 25 models are right.)

! THE CATCH: INDEPENDENCE

This only works if the models make *different* mistakes. Twenty-five copies of the same model all fail on the same examples, so voting gains nothing. The art of ensembles is making models that are individually good *and* diverse. Bagging and random forests create diversity through randomness; boosting creates it by making each model focus on the previous ones' errors.

For regression the effect is easy to see too: averaging M predictions with independent errors divides the variance by M — exactly the "variance" part of the bias–variance trade-off from Lesson 10.

3. Bagging: bootstrap aggregating

We only have one training set. How do we train many different trees? **Bagging** creates a new training set for each tree by **bootstrap sampling**: draw n examples from the n originals *with replacement*. Some examples appear twice or more, some not at all.

✎ WORKED EXAMPLE — HOW MUCH OF THE DATA DOES EACH TREE SEE?

The chance a particular example is *not* picked in one draw is $1 - \frac{1}{n}$. Over n draws: $\left(1 - \frac{1}{n}\right)^n$. For large n this approaches $e^{-1} \approx 0.368$.

So each bootstrap sample contains about $1 - 0.368 = 63.2\%$ of the distinct original examples. (For $n = 1000$: $1 - 0.999^{1000} \approx 0.632$.)

? MAKE SENSE OF IT

The situation: A random forest trains each tree on a "bootstrap" sample: draw examples at random, with replacement, as many times as there are examples.

What the result tells you: Each tree sees about 63% of the distinct examples, some of them more than once. The other 37% are "out-of-bag" for that tree.

What if...?

- **Small data:** with 10 examples, 65% are picked; with 3, 70%. It settles at 63.2% as n grows.
- **The left-out 37%** can test each tree for free — the out-of-bag score, a built-in validation estimate.

Why it matters in ML: Different samples make the trees different, so their mistakes partly cancel when they vote. That's how bagging reduces variance without needing more data.

The algorithm:

1. For $m = 1, \ldots, M$: draw a bootstrap sample and train a deep (low-bias, high-variance) tree on it.
2. To predict: majority vote (classification) or average (regression) over the M trees.

Out-of-bag (OOB) evaluation is a free bonus: each example was left out of about 37% of the trees, so we can predict it using only those trees and get a validation score without a separate validation set.

4. Random forests

Bagged trees are still quite similar: if one feature is very strong, every tree splits on it first. A **random forest** (Breiman, 2001) adds a second source of randomness: **at each split, only a random subset of the features is considered**. A common default for classification is $\sqrt{d}$ features out of d (scikit-learn: `max_features="sqrt"`).

This forces trees to use different features, making them less correlated — and the less correlated the trees, the more averaging helps.

Setting	Effect
`n_estimators` (number of trees)	More is better up to a plateau; more trees never cause overfitting, they just cost time.
`max_features`	Smaller → more diverse trees but each is weaker.
`max_depth`, `min_samples_leaf`	Trees are usually grown deep; limiting them adds bias but can help on noisy data.

WHY RANDOM FORESTS ARE SO POPULAR

They work well "out of the box" with little tuning, need no feature scaling, handle many features, give feature importances, and are hard to badly overfit. They're an excellent default for tabular data.

5. Boosting: learning from mistakes

Bagging trains models *in parallel*, independently. **Boosting** trains them *in sequence*: each new model focuses on what the ensemble so far gets wrong. It typically uses weak, shallow trees (even depth-1 "stumps") and combines them into a strong model. Bagging mainly reduces variance; boosting mainly reduces bias.

◎ ANALOGY

A student preparing for an exam does a practice test, then spends the next study session on the questions they got wrong, then retests, and repeats. Each session adds a little knowledge targeted at the remaining weaknesses.

6. AdaBoost

AdaBoost (adaptive boosting) gives every training example a weight, starting equal. Each round:

1. Train a weak learner on the weighted data. Compute its weighted error rate ε.
2. Give it a vote of strength $\alpha = \frac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}$ — accurate learners get louder voices.
3. Increase the weights of misclassified examples (multiply by e^{α}) and decrease the correct ones (multiply by $e^{-\alpha}$), then renormalise so weights sum to 1.

Final prediction: the α-weighted vote of all learners.

✎ WORKED EXAMPLE — ONE ADABOOST ROUND

10 examples, each weight 0.1. A stump misclassifies 3 of them: $\varepsilon = 0.3$.

Vote strength: $\alpha = \frac{1}{2}\ln\frac{0.7}{0.3} = \frac{1}{2}\ln 2.333 \approx 0.424$.

Wrong examples: $0.1 \times e^{0.424} \approx 0.1 \times 1.528 = 0.153$. Correct ones: $0.1 \times e^{-0.424} \approx 0.065$.

Sum: $3 \times 0.153 + 7 \times 0.065 \approx 0.917$. After dividing by it: wrong examples weigh about 0.167 each, correct ones about 0.071.

The 3 mistakes now hold **50%** of the total weight (up from 30%). The next stump is forced to pay attention to them.

? MAKE SENSE OF IT

The situation: A teacher sets the next practice test using the questions the class got wrong last time. AdaBoost does the same with its training examples.

What the result tells you: The stump is 70% right, so it gets a vote of 0.424. Then the 3 mistakes are up-weighted until they hold exactly half of the total weight.

What if…?

- **A better stump** ($\varepsilon = 0.1$): its vote is **1.10**, and again its mistakes end up with exactly 50% of the weight. This always happens: after each round, the last stump is no better than a coin toss on the new weights.
- **A coin-flip stump** ($\varepsilon = 0.5$): its vote is **0** — it's ignored.
- **Worse than chance** ($\varepsilon = 0.6$): its vote is **negative** (−0.20), so its predictions are flipped.

Why it matters in ML: Boosting focuses each new model on what the ensemble still gets wrong. That's powerful, but it also means noisy or mislabelled examples get more and more weight — boosting is sensitive to label noise.

7. Gradient boosting

Gradient boosting (Friedman, 2001) is the most widely used form today. The idea for regression with squared error is wonderfully simple:

1. Start with a constant prediction $F_0(x) = \bar{y}$ (the mean).
2. Compute the **residuals** — what's still wrong: $r_i = y_i - F_{m-1}(x_i)$.
3. Fit a small tree h_m to predict those residuals.
4. Add it in, scaled by a **learning rate** η (e.g. 0.1): $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$.
5. Repeat for M rounds.

Why "gradient"? For squared loss $\frac{1}{2}(y - F)^2$, the negative gradient with respect to the prediction F is exactly the residual $y - F$. So each tree takes a step of gradient descent (Lesson 7) — not in the space of weights, but in the space of predictions. Swapping in a different loss (log loss for classification, absolute error for robustness) gives a different "residual" to fit; the algorithm stays the same.

✎ WORKED EXAMPLE — TWO ROUNDS BY HAND

Data: $x = 1, 2, 3$ with $y = 3, 5, 8$. Learning rate $\eta = 0.5$ (large, so the effect is visible).

Round 0: $F_0 = \bar{y} = 16/3 \approx 5.33$ for everyone. Residuals: $-2.33, -0.33, 2.67$. MSE ≈ 4.22.

Round 1: fit a stump to the residuals. The best split is $x \leq 2.5$: the left leaf predicts the mean residual of the first two, $(-2.33 - 0.33)/2 = -1.33$; the right leaf predicts 2.67.

Update: $F_1 = 5.33 + 0.5 \times (-1.33) = 4.67$ for $x = 1, 2$, and $5.33 + 0.5 \times 2.67 = 6.67$ for $x = 3$.

New residuals: $3 - 4.67 = -1.67$, $5 - 4.67 = 0.33$, $8 - 6.67 = 1.33$. MSE drops from 4.22 to ≈ 1.56.

Round 2 fits a new stump to these new residuals, and so on. Each round fixes part of what's left.

? MAKE SENSE OF IT

The situation: Predicting delivery times. The first guess is the average; each new small tree corrects part of what's still wrong.

What the result tells you: After one round the MSE falls from 4.22 to 1.56. Each tree predicts the *residuals* — the leftover errors — and only half its correction is applied.

What if…?

- **A third round** (η = 0.5): MSE **0.20**.
- **η = 1** (full corrections): MSE 0.67 after one round and 0.17 after two — faster, but with real noisy data this memorises the training set.
- **η = 0.1**: after 10 rounds the MSE is still 0.85. Small steps need many trees, but generalise better.

Why it matters in ML: The learning rate and the number of trees work together: smaller steps plus more trees usually give the best test accuracy. Libraries like XGBoost and LightGBM use early stopping on a validation set to choose when to stop.

The learning rate and number of trees trade off: a small η (0.01–0.1) with many trees generalises better than a large η with few. Unlike random forests, boosting *can* overfit with too many rounds,

so the number of trees is tuned — often with **early stopping** on a validation set.

STEP-BY-STEP DERIVATION **Why each tree fits the residuals**

1 $\ell = \frac{1}{2}(y - F)^2$

Squared loss for one example, where F is the ensemble's current prediction.

2 $\frac{\partial \ell}{\partial F} = -(y - F)$

Differentiate with respect to the prediction (chain rule; the $\frac{1}{2}$ cancels the 2).

3 $-\frac{\partial \ell}{\partial F} = y - F = r$

The negative gradient is exactly the residual.

4 $F \leftarrow F - \eta \frac{\partial \ell}{\partial F} = F + \eta\, r$

A gradient descent step on the prediction itself.

5 $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$

We can't adjust predictions point by point for unseen data, so a small tree h_m learns to approximate the residuals r as a function of x.

ADVANCED · OPTIONAL **Gradient boosting is gradient descent in function space** ▾ hide

Think of the predictions $F(x_1), \ldots, F(x_n)$ as the parameters. For the loss $L = \sum_i \frac{1}{2}(y_i - F(x_i))^2$ the gradient with respect to each prediction is

$$\frac{\partial L}{\partial F(x_i)} = -(y_i - F(x_i)),$$

so the residuals are exactly the *negative gradient*. Fitting a small tree h to the residuals and updating $F \leftarrow F + \eta h$ is one gradient-descent step, with the tree generalising the step to new points. For any other differentiable loss, boosting fits trees to the **pseudo-residuals** $-\partial L / \partial F(x_i)$: for log loss these are $y_i - p_i$, which is how gradient-boosted classifiers work. XGBoost goes one step further and also uses the second derivatives (a Newton step).

READ THE FORMULA **One gradient boosting round**

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

- **$F_m(x)$** — the ensemble after round m
- **$F_{m-1}(x)$** — the ensemble so far
- **η** — learning rate — take only part of each correction
- **$h_m(x)$** — a small tree trained on the residuals

Hover over (or tap) a part to highlight it in the formula.

8. Try it: gradient boosting, tree by tree

Gradient boosting with depth-1 trees (stumps) fits the blue points. Add trees one at a time and watch the green prediction become a better and better step-shaped fit. The bottom panel shows the residuals the *next* tree will try to fit. Try a large learning rate, then a small one with many trees.

Interactive demo — open *lesson-14.html* on the website to try it.

9. Modern boosting libraries

XGBoost, **LightGBM** and **CatBoost** are highly optimised gradient-boosting libraries. XGBoost, for example, adds regularization of the trees directly into its objective:

$$\text{obj} = \sum_i \ell(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_j w_j^2$$

where T is the number of leaves in a tree and w_j are its leaf values — penalising both big trees and extreme predictions. scikit-learn's `HistGradientBoostingClassifier` uses a similar fast approach: it bins each feature into at most 255 bins, handles missing values natively, and turns on early stopping automatically for datasets with more than 10,000 samples.

10. Bagging vs boosting at a glance

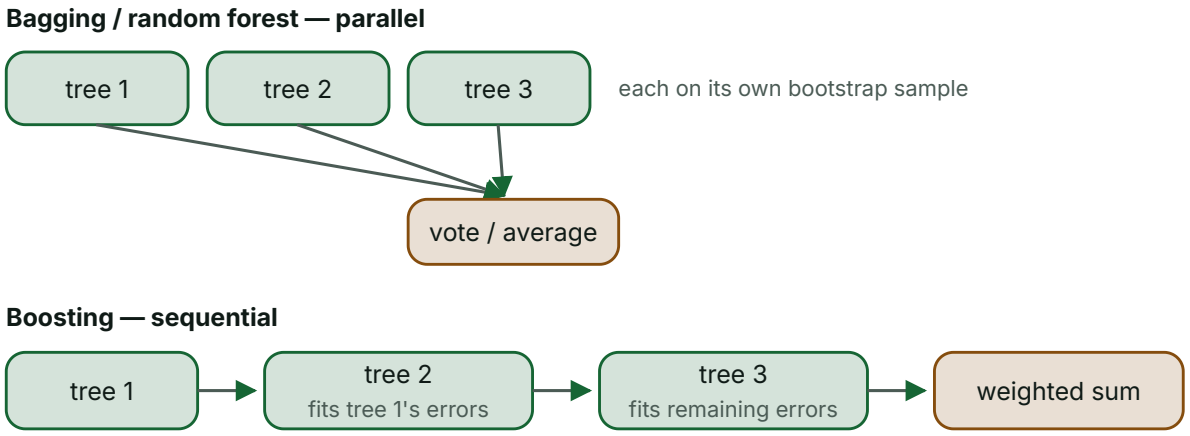


Figure 14. Bagging trains trees independently and averages them (cuts variance). Boosting trains each tree to fix what the previous ones got wrong (cuts bias).

	Random forest (bagging)	Gradient boosting
Trees trained	Independently, in parallel	Sequentially, each fixing the last
Individual trees	Deep (low bias, high variance)	Shallow (high bias, low variance)
Mainly reduces	Variance	Bias
Overfits with more trees?	No	Yes — tune the number of trees
Tuning effort	Low	Moderate (learning rate, trees, depth)
Typical accuracy on tabular data	Very good	Often the best

11. In Python

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, AdaBoostClassifier,
                              GradientBoostingClassifier,
HistGradientBoostingClassifier)

X, y = load_breast_cancer(return_X_y=True)
models = {
    "single tree": DecisionTreeClassifier(random_state=0),
    "random forest": RandomForestClassifier(n_estimators=300, random_state=0),
    "AdaBoost": AdaBoostClassifier(n_estimators=200, random_state=0),
    "gradient boosting": GradientBoostingClassifier(random_state=0),
    "hist. gradient boosting": HistGradientBoostingClassifier(random_state=0),
}
for name, model in models.items():
    s = cross_val_score(model, X, y, cv=5)
    print(f"{name:24s} {s.mean():.3f} ± {s.std():.3f}")

rf = RandomForestClassifier(n_estimators=300, oob_score=True, random_state=0).fit(X, y)
print("out-of-bag accuracy:", round(rf.oob_score_, 3))
```

Output:

```
single tree              0.917 ± 0.016
random forest            0.963 ± 0.022
AdaBoost                 0.977 ± 0.009
gradient boosting        0.963 ± 0.021
hist. gradient boosting  0.965 ± 0.018
out-of-bag accuracy: 0.965
```

Every ensemble beats the single tree by 5–6 percentage points, with default settings. Notice the out-of-bag accuracy (0.965) closely matches the cross-validated score (0.963) — a free and reliable estimate. On this particular dataset AdaBoost edges ahead; on another dataset the order could differ, which is why we compare with cross-validation.

12. Running example: Chennai Homes

With 12 houses, leave-one-out accuracy is very noisy: a single tree scores 0.50, a random forest 0.58. The forest spreads importance across age, distance and size rather than relying on one split.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, LeaveOneOut
cols = ["size_m2", "bedrooms", "dist_km", "age_yrs"]
for name, m in [("single tree", DecisionTreeClassifier(random_state=0)),
                ("random forest", RandomForestClassifier(n_estimators=300,
random_state=0))]:
    acc = cross_val_score(m, df[cols], df["sold_fast"], cv=LeaveOneOut()).mean()
    print(f"{name:14s} leave-one-out accuracy {acc:.2f}")
rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(df[cols],
df["sold_fast"])
print({c: round(float(v), 2) for c, v in zip(cols, rf.feature_importances_)})
```

Output:

```
single tree    leave-one-out accuracy 0.50
random forest  leave-one-out accuracy 0.58
{'size_m2': 0.27, 'bedrooms': 0.04, 'dist_km': 0.31, 'age_yrs': 0.37}
```

⚙ WHERE THIS IS USED

- Fraud detection, credit risk, insurance pricing
- Ranking search results
- A very common choice in data-science competitions on tabular data

Choose it when: Tabular (spreadsheet-like) data — usually the strongest choice.

Think twice when: Images, audio and raw text, where deep learning is usually better.

× MISCONCEPTIONS TO UNLEARN

MYTH More trees in a random forest can cause overfitting.

REALITY Adding trees to a random forest doesn't make it overfit — it only costs time. (Boosting is different: too many rounds can overfit.)

MYTH An ensemble of weak models can't beat one strong model.

REALITY Combining many diverse, decent models is one of the most reliable ways to get top accuracy on tabular data.

RESPONSIBLE AI · ACCURACY VERSUS EXPLAINABILITY

Forests and boosting are often more accurate than a single tree but much harder to explain. Post-hoc tools (feature importance, SHAP values, partial-dependence plots) help, but they describe the model approximately and can be misleading — permutation importance, for example, spreads credit unpredictably between correlated features. For high-stakes decisions, consider whether a slightly less accurate but interpretable model is the responsible choice.

13. Questions beginners often ask

Q. Random forest or gradient boosting?

Random forests work well with almost no tuning. Gradient boosting is often slightly more accurate but needs its learning rate and number of trees tuned.

Q. How many trees?

For random forests, hundreds is typical — more never overfits, it only costs time. For boosting, choose with early stopping or cross-validation.

Q. Can I still interpret an ensemble?

Less directly than one tree, but feature importances and tools such as permutation importance and partial dependence plots help.

! COMMON MISTAKES TO AVOID

- Using too many boosting rounds with a large learning rate, which overfits.
- Assuming more random-forest trees causes overfitting — it only costs time.
- Ensembling near-identical models, which barely helps.

! INTERVIEW CORNER

Don't confuse

- **Bagging vs boosting** — Committee in parallel vs relay team in sequence.
- **AdaBoost vs gradient boosting** — Re-weight the examples vs fit the leftovers.
- **Out-of-bag score vs validation score** — Free built-in check vs a dedicated exam.

Hidden questions — answer in your head first, then open.

▼ **Does bagging reduce bias or variance? And boosting?**

The trap: "Both reduce error" without saying how.

Bagging (random forests) mainly reduces *variance* by averaging many deep, overfit trees. Boosting mainly reduces *bias* by adding simple models that correct remaining errors.

▼ **Will a random forest overfit if you add more trees?**

The trap: Treating trees like boosting rounds.

No. More trees make the average more stable; the test error levels off, it doesn't rise. Gradient boosting is different: more rounds *can* overfit, so use early stopping.

▼ **Why does a random forest choose a random subset of features at each split?**

The trap: "For speed."

To decorrelate the trees. The variance of an average of B trees is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, where ρ is their correlation. With $\rho = 0.5$ and 100 trees it's 0.505σ²; with $\rho = 0.1$ it's 0.109σ². Bootstrap samples alone leave trees too similar, because they all split on the strongest feature.

▼ **What is the out-of-bag score?**

The trap: Not knowing about it.

Each tree skips about 37% of the examples. Predicting each example using only the trees that didn't see it gives a free validation estimate, without a separate hold-out set.

14. Key takeaways

- Combining many diverse models reduces error — but only if their mistakes are not all the same.
- **Bagging** trains models on bootstrap samples (~63% of the data each) and averages them, reducing variance.
- **Random forests** also pick a random subset of features at each split, making trees more diverse.
- **Boosting** trains models in sequence; **AdaBoost** re-weights mistakes, **gradient boosting** fits each new tree to the residuals (the negative gradient).
- Boosting needs the learning rate and number of trees tuned; random forests work well with little tuning.

15. Check your understanding

1. Why does a majority vote of many classifiers help?

a. Each model sees more data

b. It always reaches 100% accuracy

c. Independent errors tend to cancel out when combined

2. What fraction of distinct examples does a bootstrap sample contain, roughly?

a. 63%

b. 50%

c. 100%

3. What does a random forest add on top of bagging?

a. Sequential training

b. A random subset of features considered at each split

c. Example weights

4. In gradient boosting with squared error, each new tree is trained to predict...

a. the original labels y

b. random noise

c. the residuals $y - F(x)$ of the current ensemble

5. A weak learner has weighted error $\varepsilon = 0.5$. What is its AdaBoost vote α?

a. 0

b. 0.5

c. 1

16. Going deeper: tuning ensembles and how XGBoost works

INTERMEDIATE · SKIP ON A FIRST READ

Random forests and gradient boosting are the workhorses of tabular ML. This section explains the maths of *why* averaging works (and why it needs the trees to disagree), how to tune boosting with early stopping, what XGBoost, LightGBM and CatBoost add to plain gradient boosting, and when combining different models (stacking) is worth the effort.

You'll need: this lesson, tree instability from Lesson 13's "Going deeper", and early stopping from Lesson 10's.

Why averaging works: correlation is the limit

If B models each have variance σ^2 and every pair has correlation ρ, their average has variance

$$\mathrm{Var}(\text{average}) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

The second term vanishes as you add trees, but the first doesn't. **Correlation sets the floor**: averaging 1,000 identical trees ($\rho = 1$) is no better than one. That's why a random forest considers only a random subset of features at each split (`max_features`): it makes the trees disagree more. The trade-off is that each tree becomes individually weaker.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
# max_features controls how different the trees are
for mf in [1.0, 0.5, "sqrt", 1]:
    rf = RandomForestClassifier(n_estimators=300, max_features=mf, oob_score=True,
random_state=0, n_jobs=-1).fit(Xtr, ytr)
    preds = np.array([t.predict(Xte) for t in rf.estimators_])
    corr = np.corrcoef(preds)[np.triu_indices(len(preds), 1)].mean()
    print(f"max_features {str(mf):5}: tree-to-tree correlation {corr:.2f}   out-of-bag
accuracy {rf.oob_score_:.3f}   test {rf.score(Xte, yte):.3f}")
```

Output:

```
max_features 1.0  : tree-to-tree correlation 0.83   out-of-bag accuracy 0.960   test
0.930
max_features 0.5  : tree-to-tree correlation 0.82   out-of-bag accuracy 0.957   test
0.942
max_features sqrt : tree-to-tree correlation 0.80   out-of-bag accuracy 0.967   test
0.953
max_features 1    : tree-to-tree correlation 0.70   out-of-bag accuracy 0.965   test
0.936
```

Using all features at every split (1.0, which is plain bagging), trees agree with correlation 0.83 and the test accuracy is 0.930. With $\sqrt{30} \approx 5$ features per split, correlation drops to 0.80 and accuracy rises to 0.953. With just 1 feature per split, correlation falls to 0.70, but each tree is now so weak that accuracy dips again (0.936). The default `"sqrt"` is a good middle. The out-of-bag accuracy, computed on the rows each tree didn't see in its bootstrap sample, tracks the right ordering without needing a separate validation set.

Tuning gradient boosting: learning rate and early stopping

Each boosting round adds a tree multiplied by the learning rate (also called **shrinkage**). A smaller rate means each tree corrects less, so you need more trees, but the combined model usually generalises better. Rather than guess the number of trees, hold back part of the training data and **stop early** when the validation score stops improving (Lesson 10's "Going deeper"):

```
from sklearn.ensemble import HistGradientBoostingClassifier

for lr in [1.0, 0.3, 0.1, 0.03]:
    gb = HistGradientBoostingClassifier(learning_rate=lr, max_iter=2000,
early_stopping=True,
                                        validation_fraction=0.2, n_iter_no_change=30,
random_state=0).fit(Xtr, ytr)
    print(f"learning rate {lr:4}: trees used {gb.n_iter_:4d}   test accuracy
{gb.score(Xte, yte):.3f}")
```

Output:

```
learning rate  1.0: trees used   35   test accuracy 0.924
learning rate  0.3: trees used   49   test accuracy 0.942
learning rate  0.1: trees used   83   test accuracy 0.947
learning rate 0.03: trees used  203   test accuracy 0.936
```

With rate 1.0, boosting stops after 35 trees at 0.924; at 0.1 it uses 83 trees and reaches 0.947. Going smaller still (0.03, 203 trees) doesn't help on this small dataset; the differences here are within test-set noise (Lesson 9's "Going deeper"). The usual recipe: pick a modest learning rate (0.05–0.1), let early stopping choose the number of trees, then tune tree size (depth or number of leaves) and regularization. Halving the learning rate roughly doubles the trees needed.

What XGBoost, LightGBM and CatBoost add

Plain gradient boosting fits each new tree to the negative gradient of the loss (this lesson's "Advanced" box). **XGBoost** made three key improvements:

- **Second-order steps.** It uses each point's gradient g_i *and* curvature h_i (a Newton step, Lesson 3's "Going deeper"). The best value for a leaf is then a closed formula:

$$w^* = -\frac{\sum_{i \in \text{leaf}} g_i}{\sum_{i \in \text{leaf}} h_i + \lambda}$$

- **Regularization built into the objective.** A penalty $\gamma T + \frac{\lambda}{2}\sum_j w_j^2$ on the number of leaves T and on the leaf values. A split is made only if its gain exceeds γ, which prunes automatically.
- **Engineering:** it learns which way missing values should go at each split, computes in parallel, and supports sparse data.

LightGBM speeds this up by grouping each feature's values into histograms (about 255 bins) and growing trees *leaf-wise*, always splitting the leaf with the biggest gain. Scikit-learn's `HistGradientBoostingClassifier`, used above, follows the same design. **CatBoost** handles categorical features natively, using carefully ordered target statistics that avoid leakage, and builds symmetric trees that are fast at prediction time. On most tabular problems the three are close; differences in tuning usually matter more than the choice of library.

Stacking: combining different models

Stacking trains several different models, then a simple "blender" (often logistic regression) that learns how much to trust each one. The blender must be trained on *out-of-fold* predictions; otherwise it learns from predictions on rows the models memorised, which is a form of leakage (Lesson 5).

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

base = [("forest", RandomForestClassifier(n_estimators=300, random_state=0)),
        ("boost", HistGradientBoostingClassifier(random_state=0)),
        ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(15)))]
for name, m in base:
    print(f"{name:7}: test accuracy {m.fit(Xtr, ytr).score(Xte, yte):.3f}")
stack = StackingClassifier(base, final_estimator=LogisticRegression(max_iter=1000),
cv=5).fit(Xtr, ytr)
print(f"stacked: test accuracy {stack.score(Xte, yte):.3f}   blender weights
{stack.final_estimator_.coef_.round(2)}")
```

Output:

```
forest : test accuracy 0.953
boost  : test accuracy 0.942
knn    : test accuracy 0.942
stacked: test accuracy 0.942   blender weights [[2.84 2.28 3.05]]
```

Here, stacking a forest, boosting and kNN scores 0.942: no better than the forest alone (0.953). That's an honest and common result: stacking helps most when the base models are strong *and make different kinds of mistakes*, and with 171 test cases small gains are invisible anyway. It's popular in competitions, where every 0.1% matters. In production, its extra complexity, latency and maintenance often aren't worth it.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **What does XGBoost do differently from classic gradient boosting?**

The trap: "It's just faster."

It optimises a regularized objective with a second-order (Newton) approximation: each point contributes a gradient g_i and curvature h_i, leaf values have the closed form $-\sum g/(\sum h + \lambda)$, and a split's gain must exceed γ (a cost per leaf). It also learns default directions for missing values, uses column and row subsampling, and is engineered for parallel, sparse and out-of-core data.

▼ **How would you tune a gradient-boosted model in limited time?**

The trap: Grid-searching everything at once.

Fix a moderate learning rate (0.05–0.1) and use early stopping on a validation set to pick the number of trees. Then tune tree complexity (max_depth or num_leaves, min samples per leaf), then subsampling (rows and columns) and L2 regularization, using random or Bayesian search with cross-validation. Finally, lower the learning rate, raise the tree limit, and refit with early stopping for a small final gain.

▼ **Why does a random forest choose features randomly at each split, in terms of the variance formula?**

The trap: "To be random."

The variance of an average of B trees is $\rho\sigma^2 + (1-\rho)\sigma^2/B$. Adding trees only shrinks the second term; the first, set by the correlation ρ between trees, remains. Bootstrap samples alone leave trees highly correlated, because the strongest features dominate every tree's top splits. Random feature subsets lower ρ, and so the floor, at the cost of slightly weaker individual trees.

Check your understanding: going deeper

1. Averaging 1,000 trees that are perfectly correlated ($\rho = 1$) reduces variance by...

a. A factor of 1,000

b. Nothing

c. Half

2. In a random forest, lowering `max_features` ...

a. Makes trees more alike

b. Makes trees less correlated but each one weaker

c. Has no effect

3. If you halve gradient boosting's learning rate, you usually need about...

a. Half as many trees

b. Twice as many trees

c. The same number of trees

4. XGBoost's leaf value $w^* = -\sum g_i / (\sum h_i + \lambda)$ uses...

a. Only gradients

b. Gradients and second derivatives (curvature), plus an L2 penalty

c. Random numbers

5. Why must a stacking blender be trained on out-of-fold predictions?

a. It's faster

b. Predictions on training rows are over-optimistic, so the blender would learn from leaked information

c. Scikit-learn requires it

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Three independent classifiers, each 70% accurate, vote by majority. How accurate is the vote?

WORKING

EXERCISE 2 By hand

A bootstrap sample of size 5 is drawn from 5 examples. What's the chance a particular example is never picked?

WORKING

EXERCISE 3 By hand

An AdaBoost weak learner has weighted error 0.2. What is its vote weight α?

WORKING

EXERCISE 4 By hand

Gradient boosting: $y = 10, 20$; F_0 is the mean. A stump fits the residuals perfectly; learning rate 0.1. What are the predictions after one round?

WORKING

EXERCISE 5 Intermediate

Trees have variance $\sigma^2 = 1$ and pairwise correlation $\rho = 0.3$. What is the variance of the average of 10 trees, and the limit as the number of trees grows without bound?

WORKING

EXERCISE 6 Intermediate

A leaf contains points with gradients $g = (0.4, -0.2, 0.6)$ and curvatures $h = (0.24, 0.16, 0.24)$. With $\lambda = 1$, compute XGBoost's leaf value.

WORKING

EXERCISE 7 Intermediate

Gradient boosting with learning rate 0.1 stops early at 300 trees. Roughly how many trees would you expect at learning rate 0.05?

WORKING

NOTES

PART 3 · CLASSIC ALGORITHMS · ~55 MIN

Lesson 15: Support Vector Machines

Maximum margins, the hinge loss, and the kernel trick for curved boundaries.

BY THE END OF THIS LESSON YOU CAN

- Explain the maximum-margin idea and support vectors
- Compute a margin and hinge losses by hand
- Explain how C trades margin width against errors
- Use the kernel trick and tune C and gamma for an RBF SVM

↺ BEFORE YOU START

This lesson builds on:

- Lesson 2: Vectors & Matrices — A vector is a list of numbers; a dataset X is a matrix with one row per example. The dot product $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models predict, and $\|\mathbf{a} - \mathbf{b}\|$ is the distance between two examples.
- Lesson 8: Logistic Regression — Logistic regression outputs a probability $p = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, is trained with log loss, and classifies with a threshold (often 0.5), giving a straight decision boundary.
- Lesson 10: Overfitting, Regularization & Bias–Variance — Error = bias² + variance + noise. Regularization adds a penalty on weight size (L2 ridge shrinks weights; L1 lasso zeros some) to reduce variance; its strength is chosen by cross-validation.

Where this fits: A different idea of the "best" boundary (the widest margin), plus the kernel trick for curved boundaries. Completes the classic toolbox.

1. Which line is best?

When two classes can be separated by a straight line, there are usually *infinitely many* lines that do it. Logistic regression (Lesson 8) picks one by maximising likelihood. A **support vector machine (SVM)** has a different and very intuitive answer: choose the line that stays **as far as possible from both classes** — the one with the widest "safety corridor" between them.

◎ ANALOGY: THE WIDEST ROAD

Imagine two villages on either side of a field, and you must build a straight road between them. A road that grazes the houses on one side is risky — a slight error and it cuts through someone's garden. The safest road is the widest one you can fit, running right through the middle of the gap. SVMs build the widest possible road; the houses at its edges are the **support vectors**.

A wide margin means that new points can land a bit off from the training points and still be classified correctly — better generalization. Combined with the **kernel trick**, SVMs also draw curved boundaries, and for years they were among the best general-purpose classifiers.

2. Setting up: labels ±1 and a hyperplane

For SVMs it's convenient to label the classes $y = +1$ and $y = -1$ (instead of 1 and 0). As in Lesson 8, the boundary is the set of points where

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

and we predict $\text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$: $+1$ if positive, -1 if negative. The **score** $\mathbf{w} \cdot \mathbf{x} + b$ also measures how far a point is from the boundary: the actual distance is

$$\text{distance}(\mathbf{x}) = \frac{|\mathbf{w} \cdot \mathbf{x} + b|}{\|\mathbf{w}\|}$$

A handy trick: a point is correctly classified exactly when $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) > 0$ — the label and the score have the same sign.

3. The margin

PICTURE IT Parking between two cars, you aim for the middle of the gap, not just anywhere that fits — leaving the most room for error on both sides. An SVM chooses its boundary the same way.

The scale of $\mathbf{w}$ and b is arbitrary (doubling both gives the same line), so we fix it: require the points closest to the boundary to have score exactly ± 1. All points must then satisfy

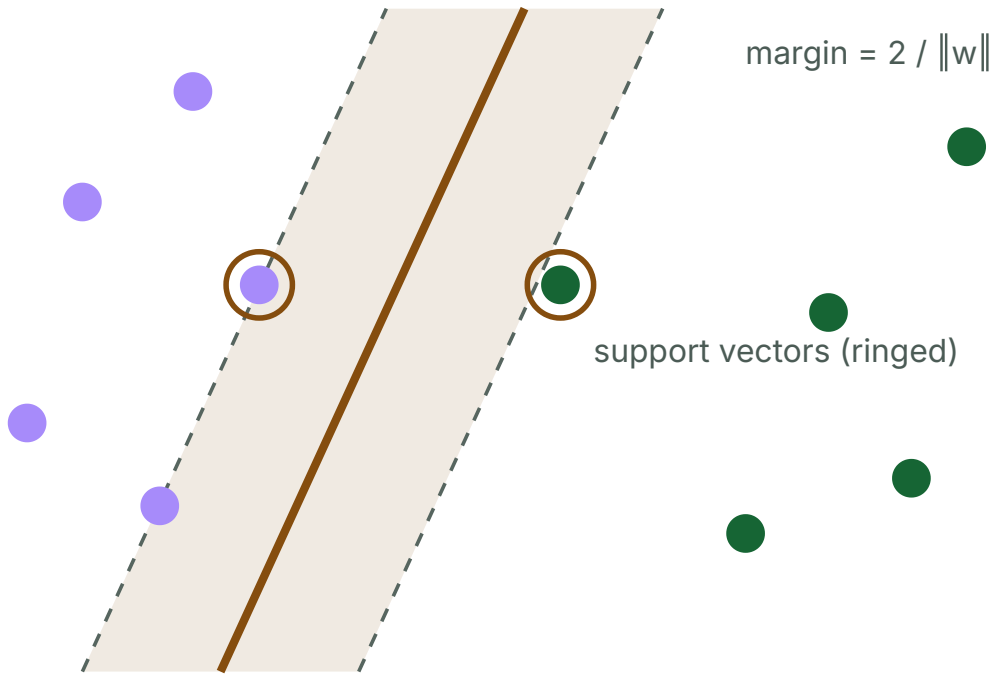


Figure 15. The SVM boundary (solid) runs down the middle of the widest possible road. Only the points on the road's edges — the support vectors — fix its position.

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \text{for every } i$$

The two "gutters" $\mathbf{w} \cdot \mathbf{x} + b = +1$ and $\mathbf{w} \cdot \mathbf{x} + b = -1$ are the edges of the road. Each is at distance $1/\|\mathbf{w}\|$ from the boundary, so the road's total width — the **margin** — is

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

✎ WORKED EXAMPLE — MEASURING A MARGIN

Suppose an SVM has $\mathbf{w} = (1, 1)$, $b = -3$. Then $\|\mathbf{w}\| = \sqrt{1^2 + 1^2} = \sqrt{2}$, so the margin is $2/\sqrt{2} \approx 1.41$.

Point $(1, 1)$: score $= 1 + 1 - 3 = -1$ → class -1, sitting exactly on the gutter — a support vector.

Point $(3, 2)$: score $= 3 + 2 - 3 = 2$ → class $+1$, distance from boundary $2/\sqrt{2} \approx 1.41$ — safely outside the road.

? MAKE SENSE OF IT

The situation: Picture a road between two neighbourhoods. The SVM wants the widest road that keeps every red house on one side and every blue house on the other.

What the result tells you: The road is 1.41 units wide. A point with score exactly ±1, like (1, 1), sits on the edge of the road: a support vector.

What if...?

- **Double the weights** to $\mathbf{w} = (2, 2),\ b = -6$: the boundary line is the same, but the road narrows to **0.71**. The margin is set by $\|\mathbf{w}\|$, which is why SVMs minimise it.
- **Halve them** to $(0.5, 0.5),\ b = -1.5$: road width **2.83** — but now points like (1, 1) are inside the road.

Why it matters in ML: A wider margin usually means the boundary is less sensitive to small changes in the data, so it generalises better.

STEP-BY-STEP DERIVATION **Why the margin is $2/\|\mathbf{w}\|$**

1. $$\text{distance}(\mathbf{x}) = \frac{|\mathbf{w} \cdot \mathbf{x} + b|}{\|\mathbf{w}\|}$$

   Distance from a point to the boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$.

2. $$\mathbf{w} \cdot \mathbf{x} + b = \pm 1$$

   Points on the gutters have score exactly ± 1 (that's how we fixed the scale).

3. $$\text{distance} = \frac{1}{\|\mathbf{w}\|}$$

   So each gutter is $1/\|\mathbf{w}\|$ from the boundary.

4. $$\text{margin} = \frac{2}{\|\mathbf{w}\|}$$

   Two gutters, one on each side.

5. $$\max \frac{2}{\|\mathbf{w}\|} \iff \min \|\mathbf{w}\| \iff \min \tfrac{1}{2}\|\mathbf{w}\|^2$$

   Maximising the margin means making $\mathbf{w}$ small; squaring gives the same answer with nicer maths.

4. The optimisation problem

Maximising the margin $2/\|\mathbf{w}\|$ is the same as minimising $\|\mathbf{w}\|$, and for convenient maths we minimise $\frac{1}{2}\|\mathbf{w}\|^2$. The **hard-margin SVM** is:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 \ \text{ for all } i$$

"Make the road as wide as possible, while keeping every point off the road and on its correct side." This is a convex problem with one best solution, solved by specialised algorithms.

≡ SUPPORT VECTORS

In the solution, only the points lying exactly on the gutters (or, with soft margins, inside the road or misclassified) affect where the boundary is. These are the **support vectors**. Delete any other training point and the boundary doesn't move at all. That's where the name comes from — they "support" the boundary.

5. Soft margins and the hinge loss

Real data overlaps and has outliers; often no line separates the classes perfectly, and the hard-margin problem has no solution. The **soft-margin SVM** lets points violate the margin, at a price. The cleanest way to see it is through the **hinge loss**:

$$\ell_{\text{hinge}} = \max\big(0,\ 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)\big)$$

✎ WORKED EXAMPLE — HINGE LOSS FOR A POSITIVE POINT ($Y = +1$)

Score $\mathbf{w}\cdot\mathbf{x}+b$	Hinge loss	Meaning
1.5	$\max(0, 1-1.5) = 0$	correct and outside the road — no penalty
0.4	$\max(0, 1-0.4) = 0.6$	correct side, but inside the road — small penalty
−0.5	$\max(0, 1+0.5) = 1.5$	wrong side — bigger penalty

Unlike log loss (Lesson 8), hinge loss is exactly zero once a point is safely classified. Those points stop mattering — which is why only the support vectors define the boundary.

? MAKE SENSE OF IT

The situation: A strict but fair examiner: full marks with no comment if you're clearly right, a small penalty if you're right but only just, a bigger one if you're wrong.

What the result tells you: Hinge loss is zero once a point is past the edge of the road (score ≥ 1), and grows in a straight line as the point moves into or across the road.

What if...?

- **Score exactly 1:** loss 0 — on the edge, no penalty.
- **Score 0** (on the boundary): loss 1.
- **A negative point** ($y = -1$) with score −1.5: loss $\max(0, 1 - (-1)(-1.5)) = 0$ — correctly classified and outside the road.

Why it matters in ML: Because confident, correct points have zero loss, they have no influence on the solution. Only points near or across the road shape the boundary — which makes SVMs robust to far-away points but sensitive to the ones near the boundary.

The soft-margin SVM balances margin width against total hinge loss:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \max\left(0,\ 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)\right)$$

Look closely: it's "regularization + loss", the same shape as ridge regression in Lesson 10. The parameter C sets the balance:

- **Small** C: margin violations are cheap → wide road, many support vectors, smoother boundary. More regularization (more bias).
- **Large** C: violations are expensive → narrow road that tries to classify every training point correctly. Less regularization (more variance).

READ THE FORMULA **Soft-margin SVM objective**

$$\tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i} \max\left(0,\ 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)\right)$$

- **½‖w‖²** — small w = wide margin
- **C** — how expensive margin violations are
- **Σ max(0, ...)** — hinge losses: zero for points safely outside the margin

Hover over (or tap) a part to highlight it in the formula.

6. Try it: margin and C

A linear soft-margin SVM is trained on the points by gradient descent on the objective above. The solid line is the boundary; the dashed lines are the gutters. Ringed points are support vectors (on or inside the margin, or misclassified). Change C and watch the road narrow and widen. Click to add points (switch class with the button) and see how only points near the road change it.

Interactive demo — open *lesson-15.html* on the website to try it.

7. The kernel trick: curved boundaries

What if the classes form a ring — one class in the middle, the other all around? No straight line works. The idea: **map the data into a higher-dimensional space where a straight boundary does work**.

✎ WORKED EXAMPLE — LIFTING DATA TO SEPARATE IT

One feature: class $+1$ at $x = -3, 3$ and class -1 at $x = -1, 0, 1$. On a number line, the -1 class is in the middle — no single threshold separates them.

Add a feature x^2, so each point becomes (x, x^2): the $+1$ points go to $(\pm 3, 9)$, the -1 points to $(-1, 1), (0, 0), (1, 1)$. Now the horizontal line $x^2 = 5$ separates them perfectly. A straight line in the new space is a curved boundary ($x = \pm\sqrt{5}$) in the original one.

? MAKE SENSE OF IT

The situation: Normal blood pressure is in the middle; both very low and very high are dangerous. No single threshold separates healthy from unhealthy.

What the result tells you: Adding x^2 as a feature turns "far from 0 in either direction" into "large x^2", which one threshold can separate.

What if…?

- **Bring the outer points in** to ±2: their $x^2 = 4$ is still above the inner points' 1, so a threshold between 1 and 4 still works.
- **Put a −1 point at ±3 too:** then no threshold on x^2 works, and you'd need yet another feature.

Why it matters in ML: Many non-linear problems become linear with the right extra features. Kernels add them implicitly; neural networks learn them.

The problem: with many features, the number of such combinations explodes. Here's the magic. The SVM's solution (in its "dual" form) only ever uses the training data through **dot products**

$\phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$ of mapped points. A **kernel** is a function that computes that dot product directly from the original vectors, without ever building $\phi(\mathbf{x})$:

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z})$$

✎ WORKED EXAMPLE — THE TRICK IN ACTION

For 2D vectors, take $\phi(\mathbf{x}) = (x_1^2,\ \sqrt{2}\,x_1 x_2,\ x_2^2)$. Let $\mathbf{x} = (1, 2)$, $\mathbf{z} = (3, 1)$.

The long way: $\phi(\mathbf{x}) = (1,\ 2\sqrt{2},\ 4)$, $\phi(\mathbf{z}) = (9,\ 3\sqrt{2},\ 1)$. Dot product: $9 + 2\sqrt{2} \cdot 3\sqrt{2} + 4 = 9 + 12 + 4 = 25$.

The kernel way: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z})^2 = (1 \cdot 3 + 2 \cdot 1)^2 = 5^2 = 25$.

Same answer, without computing the 3D vectors. For higher degrees and dimensions, the savings are enormous.

? MAKE SENSE OF IT

The situation: Comparing two documents by every pair of words they contain would mean building enormous vectors. The kernel gets the same similarity from the short vectors.

What the result tells you: Both ways give 25: the kernel $(\mathbf{x} \cdot \mathbf{z})^2$ equals a dot product in the 3-D feature space, without building it.

What if…?

- **Perpendicular inputs** $\mathbf{x} = (1, 2),\ \mathbf{z} = (2, -1)$: $\mathbf{x} \cdot \mathbf{z} = 0$, so $K = 0$.
- **100 features, degree 3:** the explicit feature space has **171,700** dimensions; the kernel needs one 100-term dot product and a cube.

Why it matters in ML: The kernel trick lets SVMs use huge (even infinite, for the RBF kernel) feature spaces cheaply. The cost moves elsewhere: kernel SVMs compare every pair of training points, so they slow down badly beyond tens of thousands of examples.

Common kernels

Kernel	*K*(x, z)	Boundary shape
Linear	$\mathbf{x} \cdot \mathbf{z}$	Straight line / plane
Polynomial	$(\gamma\,\mathbf{x} \cdot \mathbf{z} + r)^d$	Polynomial curves
RBF (Gaussian)	$\exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)$	Flexible, smooth, any shape

The **RBF kernel** (radial basis function) is the most popular. It's a similarity score: 1 when $\mathbf{x} = \mathbf{z}$, falling towards 0 as they move apart. Each support vector acts like a "bump" of influence, and the boundary is built from these bumps. For $\mathbf{x} = (1, 2)$ and $\mathbf{z} = (3, 1)$, $\|\mathbf{x} - \mathbf{z}\|^2 = 5$: with $\gamma = 0.1$, $K = e^{-0.5} \approx 0.61$ (fairly similar); with $\gamma = 1$, $K = e^{-5} \approx 0.007$ (barely related).

- **Small** γ: each point influences a wide area → smooth boundary (can underfit).
- **Large** γ: influence is very local → wiggly boundary around individual points (can overfit).

Tune C and γ together with a grid search over values spaced by factors of 10.

ADVANCED · OPTIONAL **The maths behind kernels** ▾ hide

An explicit example. For $\mathbf{x}, \mathbf{z} \in \mathbb{R}^2$, let $\phi(\mathbf{x}) = (x_1^2,\ \sqrt{2}\,x_1x_2,\ x_2^2)$. Then

$$\phi(\mathbf{x}) \cdot \phi(\mathbf{z}) = x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 = (\mathbf{x} \cdot \mathbf{z})^2.$$

So the kernel $(\mathbf{x} \cdot \mathbf{z})^2$ computes a dot product in a 3-dimensional feature space without ever building ϕ. With d features and degree p the explicit space has $\binom{d+p-1}{p}$ dimensions; the kernel still costs one dot product.

Why only dot products are needed. The SVM's optimal weights are a combination of training points, $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ (with $\alpha_i > 0$ only for support vectors). Substituting, the decision function becomes

$$f(\mathbf{x}) = \sum_i \alpha_i y_i\, K(\mathbf{x}_i, \mathbf{x}) + b,$$

and training (the *dual problem*) also uses the data only through $K(\mathbf{x}_i, \mathbf{x}_j)$. **Mercer's condition:** a function K corresponds to *some* feature map exactly when every kernel matrix $[K(\mathbf{x}_i, \mathbf{x}_j)]$ it produces is symmetric positive semi-definite. The RBF kernel $e^{-\gamma\|\mathbf{x}-\mathbf{z}\|^2}$ passes this test, and its feature space is infinite-dimensional.

8. SVMs in practice

- **Scale your features.** Distances and dot products depend on scale, just like kNN.
- **Size limits.** Kernel SVM training time grows roughly between n^2 and n^3 in the number of examples, so it suits small and medium datasets (up to tens of thousands). For large data, use `LinearSVC` or `SGDClassifier(loss="hinge")`.
- **Probabilities.** SVMs output scores, not probabilities. `SVC(probability=True)` adds a calibration step, at extra cost.
- **Regression.** `SVR` applies the same ideas to regression: errors smaller than ε are ignored (a "tube" around the prediction).

9. In Python

A ring-shaped dataset where one class surrounds the other:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = make_circles(n_samples=400, noise=0.2, factor=0.5, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=0)

for kernel in ["linear", "poly", "rbf"]:
    m = make_pipeline(StandardScaler(), SVC(kernel=kernel, degree=2)).fit(X_train,
y_train)
    print(f"{kernel:6s} test accuracy {m.score(X_test, y_test):.3f}   support vectors
{m[-1].n_support_.sum()}")

grid = {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]}
search = GridSearchCV(make_pipeline(StandardScaler(), SVC(kernel="rbf")), grid,
cv=5).fit(X_train, y_train)
print("best:", search.best_params_, "CV", round(search.best_score_, 3), "test",
round(search.score(X_test, y_test), 3))

m = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=100, gamma=10)).fit(X_train,
y_train)
print("C=100, gamma=10: train", round(m.score(X_train, y_train), 3), "test",
round(m.score(X_test, y_test), 3))
```

Output:

```
linear test accuracy 0.470   support vectors 297
poly   test accuracy 0.950   support vectors 97
rbf    test accuracy 0.930   support vectors 112
best: {'svc__C': 0.1, 'svc__gamma': 1} CV 0.893 test 0.9
C=100, gamma=10: train 0.983 test 0.86
```

The linear SVM is no better than guessing (47%) — a straight line can't separate a ring — and nearly every point becomes a support vector. A degree-2 polynomial kernel (which can draw circles) and the RBF kernel both succeed. The last line shows overfitting with large C and γ: 98% on training data but only 86% on the test set.

10. Running example: Chennai Homes

A linear SVM on distance and age separates fast and slow sellers. Only 5 houses — the support vectors — define the boundary; the others could be removed without changing it.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
cols = ["dist_km", "age_yrs"]
svm = make_pipeline(StandardScaler(), SVC(kernel="linear", C=1.0)).fit(df[cols],
df["sold_fast"])
w = svm[-1].coef_[0]
print("margin width (standardised units):", round(2 / np.linalg.norm(w), 2))
print("support vectors: houses", (df["id"].iloc[svm[-1].support_]).tolist())
```

Output:

```
margin width (standardised units): 1.18
support vectors: houses [3, 7, 8, 2, 4]
```

⚙ WHERE THIS IS USED

- Text classification with many features
- Small-to-medium datasets needing a strong classifier
- Historically: handwriting and image classification

Choose it when: Medium-sized data with clear margins or many features; when a kernel captures the shape well.

Think twice when: Very large datasets (slow kernel training) or when you need probabilities.

× MISCONCEPTIONS TO UNLEARN

MYTH Every training point shapes the SVM boundary.

REALITY Only the support vectors matter; the others could be deleted without changing it.

MYTH The kernel trick computes the high-dimensional features.

REALITY It computes dot products in that space directly, without ever building the features.

RESPONSIBLE AI · SCORES ARE NOT PROBABILITIES

An SVM's output is a distance from the boundary, not a probability, and a large margin is not a guarantee of being right. If a decision needs a probability ("85% chance of default"), calibrate the scores and check calibration for each group. And for consequential decisions, keep a human able to review and override the model.

11. Questions beginners often ask

Q. What does C actually control?

The trade-off between a wide margin and classifying every training point correctly. Small C = wider margin, more tolerance of errors (more regularization).

Q. Which kernel should I start with?

RBF is the usual default. Try linear first if you have very many features, such as text.

Q. Why are SVMs less popular than they were?

Gradient boosting tends to win on tabular data and deep learning on images and text; kernel SVMs also scale poorly to very large datasets. They remain a solid choice for medium-sized problems.

! COMMON MISTAKES TO AVOID

- Forgetting to scale features before an SVM.
- Tuning C and γ separately instead of together.
- Using kernel SVMs on hundreds of thousands of examples (use LinearSVC or SGD).

! INTERVIEW CORNER

Don't confuse

- **Hard margin vs soft margin** — No exceptions vs a fine for each exception.
- **C vs gamma in an RBF SVM** — C is strictness; gamma is reach.

Hidden questions — answer in your head first, then open.

▼ **Which training points determine an SVM's boundary?**

The trap: "All of them."

Only the support vectors: points on or inside the margin, or misclassified. Points safely beyond the margin have zero hinge loss and could be deleted without changing the model.

▼ **What happens as you make C very large in a soft-margin SVM?**

The trap: Thinking larger C means more regularisation.

The opposite: C is the penalty for margin violations, so a large C tolerates almost none — a narrow margin that fits the training data closely and can overfit. A small C allows more violations and a wider, simpler margin. (C is like $1/\lambda$.)

▼ **What does the RBF kernel's gamma control?**

The trap: Confusing it with C.

How far each training point's influence reaches. Large gamma: very local influence, wiggly boundaries, overfitting. Small gamma: smooth boundaries, possibly underfitting. Tune C and gamma together with cross-validation.

▼ **Does the kernel trick compute the high-dimensional features?**

The trap: Thinking it transforms the data explicitly.

No — that's the point. The kernel gives the dot product in the feature space directly from the original vectors. For 100 features and a degree-3 polynomial kernel, the feature space has 171,700 dimensions, yet each kernel value costs one 100-term dot product.

12. Key takeaways

- An SVM chooses the boundary with the largest **margin** $2/\|\mathbf{w}\|$ between the classes.
- Only the **support vectors** — points on or inside the margin — determine the boundary.
- Soft-margin SVM: $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum \text{hinge}$; small C = wider margin, more regularization.
- The **kernel trick** computes dot products in a high-dimensional space cheaply, giving curved boundaries.
- The **RBF kernel** is the usual default; tune C and γ together, and scale features first.

13. Check your understanding

1. An SVM has $\|\mathbf{w}\| = 4$. How wide is its margin?

a. 4

b. 0.5

c. 8

2. A point with $y = -1$ has score $\mathbf{w} \cdot \mathbf{x} + b = 0.5$. What is its hinge loss?

a. 1.5

b. 0.5

c. 0

3. You delete a training point that is far from the boundary and outside the margin. What happens to the SVM?

a. The boundary moves towards it

b. The margin doubles

c. Nothing — it isn't a support vector

4. Your RBF SVM gets 100% training accuracy and 70% test accuracy. What should you try?

a. Increase γ and C

b. Decrease γ and/or C

c. Switch off feature scaling

5. What does a kernel function compute?

a. A dot product in a transformed feature space, without building that space

b. The distance between two classes

c. The number of support vectors

14. Going deeper: the dual problem, kernels and hinge loss

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above describes the SVM as the widest-margin boundary, with kernels for curved boundaries. This section shows the mathematics underneath: the **dual problem** and why only support vectors matter, a numerical check that the kernel trick really computes a dot product in a bigger space, how `C` and `gamma` interact in practice, and how the SVM's hinge loss relates to logistic regression.

You'll need: this lesson, and Lagrange multipliers from Lesson 3's "Going deeper".

The dual problem and the KKT conditions

The soft-margin SVM solves: minimise $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$, subject to $y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$, with labels $y_i = \pm 1$. Introducing a Lagrange multiplier α_i for each margin constraint and eliminating $\mathbf{w}$ and b gives the equivalent **dual problem**:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, (\mathbf{x}_i \cdot \mathbf{x}_j) \quad \text{subject to } 0 \leq \alpha_i \leq C, \;\; \sum_i \alpha_i y_i = 0$$

and the weights are recovered as $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$. Two consequences follow from the **KKT conditions**, which extend Lagrange multipliers to inequality constraints:

- **Only support vectors matter.** Points safely outside the margin get $\alpha_i = 0$, so they don't appear in $\mathbf{w}$ at all. Points on the margin have $0 < \alpha_i < C$; points inside it or misclassified have $\alpha_i = C$.

- **The data appears only through dot products $\mathbf{x}_i \cdot \mathbf{x}_j$**, in training and in prediction, $f(\mathbf{x}) = \sum_i \alpha_i y_i \, (\mathbf{x}_i \cdot \mathbf{x}) + b$. Replacing that dot product with a kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ is the whole of the kernel trick.

The kernel trick, checked

The polynomial kernel $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + 1)^2$ in 2 dimensions equals an ordinary dot product of 6 explicit features: $\phi(\mathbf{x}) = \left(1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2\right)$. Expanding $(x_1z_1 + x_2z_2 + 1)^2$ term by term gives exactly $\phi(\mathbf{x}) \cdot \phi(\mathbf{z})$.

```
import numpy as np

# The kernel trick: (x·z + 1)² equals the dot product of explicit degree-2 features
def phi(v):
    a, b = v
    return np.array([1, np.sqrt(2) * a, np.sqrt(2) * b, a * a, b * b, np.sqrt(2) * a *
b])

x, z = np.array([1.0, 2.0]), np.array([3.0, -1.0])
print("kernel (x·z + 1)²:     ", (x @ z + 1) ** 2)
print("explicit φ(x)·φ(z):    ", round(phi(x) @ phi(z), 10))
```

Output:

```
kernel (x·z + 1)²:      4.0
explicit φ(x)·φ(z):     4.0
```

Both give 4.0, but the kernel needed one 2-D dot product while the explicit version built 6 features. For degree p in d dimensions the explicit space grows like d^p, and the RBF kernel corresponds to an *infinite* number of features. The kernel computes the dot product without ever constructing them. A function can serve as a kernel exactly when every matrix of its values $K(\mathbf{x}_i, \mathbf{x}_j)$ is positive semi-definite (**Mercer's condition**, linking back to Lesson 2's "Going deeper").

How C and gamma interact

With the RBF kernel $K = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$, `gamma` sets how far each point's influence reaches and `C` sets how much margin violations cost. They must be tuned *together*, usually on a logarithmic grid:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_moons(n_samples=400, noise=0.3, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.5, random_state=0)
print("    C   gamma   support vectors   train   test")
for C in [0.1, 1, 100]:
    for gamma in [0.1, 1, 30]:
        m = SVC(C=C, gamma=gamma).fit(Xtr, ytr)
        print(f"{C:5} {gamma:7}   {m.n_support_.sum():15d}   {m.score(Xtr, ytr):5.3f}
{m.score(Xte, yte):5.3f}")
```

Output:

```
    C   gamma   support vectors   train   test
  0.1     0.1              158   0.810   0.860
  0.1       1              127   0.885   0.895
  0.1      30              193   0.535   0.470
    1     0.1               95   0.860   0.870
    1       1               69   0.905   0.910
    1      30              158   0.915   0.900
  100     0.1               64   0.900   0.875
  100       1               52   0.905   0.885
  100      30              118   1.000   0.845
```

Three patterns stand out. Large γ with large C (30, 100) fits the training set perfectly (1.000) but drops to 0.845 on the test set: each point's influence is tiny and violations are expensive, so the boundary wraps around individual points. Large γ with small C (30, 0.1) collapses to 0.470: influence is so local and so cheaply ignored that the model predicts almost one class. The best region is moderate values of both (around $C = 1$, $\gamma = 1$: 0.910). The number of support vectors is a useful diagnostic: when nearly every training point is a support vector (193 of 200 here), the model hasn't found a simple boundary.

Hinge loss: the SVM as regularized loss minimisation

The soft-margin SVM can be rewritten without constraints as

$$\min_{\mathbf{w},b} \; \frac{1}{2C}\|\mathbf{w}\|^2 + \sum_i \max\left(0,\; 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)\right)$$

The second term is the **hinge loss**: zero for points beyond the margin, rising linearly for points inside it. Logistic regression minimises the same kind of objective with log loss $\log(1 + e^{-y\,f(\mathbf{x})})$ instead, which is a smooth version of the hinge that never quite reaches zero. So a linear SVM and L2-regularized logistic regression are close cousins:

```
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression

# Hinge loss and log loss give very similar linear boundaries
Xl, yl = make_moons(n_samples=400, noise=0.3, random_state=1)
svm = LinearSVC(C=1.0).fit(Xl, yl)
lr = LogisticRegression(C=1.0).fit(Xl, yl)
unit = lambda w: w / np.linalg.norm(w)
print("boundary direction, linear SVM:     ", unit(svm.coef_[0]).round(3))
print("boundary direction, logistic regr.: ", unit(lr.coef_[0]).round(3))
print("agreement on predictions:", np.mean(svm.predict(Xl) == lr.predict(Xl)))
```

Output:

```
boundary direction, linear SVM:      [ 0.384 -0.923]
boundary direction, logistic regr.:  [ 0.384 -0.923]
agreement on predictions: 1.0
```

On these 400 points the two boundaries point in the same direction to three decimal places, and the predictions agree on every point. The real differences: the hinge loss ignores points that are confidently correct, giving sparse support vectors and a pure maximum-margin answer, but no probabilities. Logistic regression uses every point and gives calibrated probabilities (Lesson 8's "Going deeper"). With kernels, the SVM's sparsity is what makes it practical, since predictions need only the support vectors.

Scaling up, and support vector regression

- **Cost.** Kernel SVM training needs the $n \times n$ kernel matrix: roughly $O(n^2)$ memory and between $O(n^2)$ and $O(n^3)$ time. Beyond about 100,000 points, use a linear SVM (`LinearSVC` or `SGDClassifier(loss="hinge")`), or approximate the kernel with random features (`RBFSampler`, `Nystroem`) and then a linear model.
- **Support vector regression** (`SVR`) uses an "ε-insensitive" loss: errors smaller than ε cost nothing, so predictions inside a tube around the data are free, and only points outside the tube become support vectors.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Explain the SVM dual problem and why it matters.**

The trap: Saying it's "just another way to write it".

Introducing Lagrange multipliers α_i for the margin constraints and eliminating $\mathbf{w}$ gives a problem over the α_i with box constraints $0 \leq \alpha_i \leq C$. It matters for two reasons: the KKT conditions make most α_i zero, so the solution depends only on the support vectors; and the data enters only through dot products, which can be replaced by any valid kernel, giving non-linear boundaries without computing high-dimensional features.

▼ **What makes a function a valid kernel?**

The trap: "Any similarity function works."

Mercer's condition: it must be symmetric, and for any set of points the matrix of its values must be positive semi-definite. Then it equals a dot product in some (possibly infinite-dimensional) feature space. Sums, positive multiples and products of valid kernels are valid. Arbitrary similarity scores can fail this, making the SVM optimisation ill-posed.

▼ **Linear SVM or logistic regression: how do you choose?**

The trap: Claiming one is always more accurate.

Accuracy is usually very close, because both are linear models with a regularized convex loss (hinge vs log loss). Choose logistic regression when you need calibrated probabilities or easy interpretation of odds; a linear SVM when you only need decisions and want a pure max-margin boundary, or in very high-dimensional sparse text where it's a strong default. Kernel SVMs are a different choice, for small to medium data with non-linear boundaries.

Check your understanding: going deeper

1. In the SVM dual, a training point safely outside the margin has...

a. $\alpha_i = 0$, so it doesn't affect the boundary

b. $\alpha_i = C$

c. The largest α_i

2. Why can the SVM use kernels?

a. In the dual, the data appears only through dot products

b. Kernels make training faster in every case

c. Because of the hinge loss

3. An RBF SVM has 100% training accuracy and much lower test accuracy. Which change is most likely to help?

a. Increase gamma

b. Decrease gamma and/or C

c. Remove feature scaling

4. The hinge loss for a point with $y\, f(\mathbf{x}) = 2$ is...

a. 0

b. 1

c. 2

5. Which statement about linear SVMs and logistic regression is true?

a. They usually give very similar boundaries; logistic regression also gives probabilities

b. They're unrelated

c. The SVM always gives probabilities

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

An SVM has $\mathbf{w} = (3, 4)$. How wide is its margin?

WORKING

EXERCISE 2 By hand

Hinge loss for a point with $y = -1$ and score -2; and with score 0.3.

WORKING

EXERCISE 3 By hand

For $\mathbf{x} = (2, 1)$, $\mathbf{z} = (1, 3)$, compute the polynomial kernel $(\mathbf{x} \cdot \mathbf{z})^2$ and the RBF kernel with $\gamma = 0.5$.

WORKING

EXERCISE 4 Concepts

An RBF SVM underfits: training 75%, test 74%, with a very smooth boundary. Which way should C and γ move?

WORKING

EXERCISE 5 Intermediate

Compute the polynomial kernel $(\mathbf{x} \cdot \mathbf{z} + 1)^2$ for $\mathbf{x} = (2, 0)$ and $\mathbf{z} = (1, 3)$, and check it with ϕ from the code.

WORKING

EXERCISE 6 Intermediate

Compute the hinge loss $\max(0, 1 - yf)$ for three points with $yf = 1.5$, 0.4 and -0.5.

WORKING

EXERCISE 7 Intermediate

An RBF kernel with $\gamma = 0.5$: compute K for two points at distance 2, and for two points at distance 0.5.

WORKING

NOTES

PART 4 · FINDING STRUCTURE WITHOUT LABELS · ~50 MIN

Lesson 16: Clustering

Group data without labels: k-means, choosing k, hierarchical clustering and DBSCAN.

BY THE END OF THIS LESSON YOU CAN

- Run the k-means algorithm by hand
- Choose k with the elbow method and silhouette score
- Explain hierarchical clustering and the different linkages
- Use DBSCAN to find clusters of any shape and flag outliers

↺ BEFORE YOU START

This lesson builds on:

- Lesson 2: Vectors & Matrices — A vector is a list of numbers; a dataset X is a matrix with one row per example. The dot product $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models predict, and $\|\mathbf{a} - \mathbf{b}\|$ is the distance between two examples.
- Lesson 11: k-Nearest Neighbours — kNN predicts from the k most similar training examples, so it depends on distances — features must be scaled, and it struggles in very high dimensions.

Where this fits: Starts Part 4: learning without labels. Instead of predicting answers, we discover groups in the data.

1. Learning without labels

PICTURE IT A shop owner looks at what people buy and notices groups: families buying in bulk, students buying snacks, office workers buying lunch. Nobody labelled them — the groups emerged from the data. That's clustering.

Every model so far learned from labelled examples: houses with prices, emails marked spam. But most of the world's data has no labels. A shop has millions of purchase records; nobody has tagged each customer as "bargain hunter" or "loyal regular". **Clustering** finds natural groups in data on its own — examples within a group are similar to each other and different from those in other groups.

Uses: customer segmentation, grouping news articles by topic, organising photos, finding unusual activity (points that fit no group), and compressing images by reducing them to a few colours.

◎ ANALOGY: SORTING LAUNDRY

Nobody tells you the categories, but looking at a pile of clothes you naturally form groups: whites, darks, colours. You're clustering by similarity. Different people might sort slightly differently — clustering has no single "right answer", which makes it both powerful and tricky to evaluate.

2. k-means: the classic algorithm

k-means splits the data into exactly k clusters (you choose k). Each cluster is represented by its **centroid** $\boldsymbol{\mu}_j$ — the average of its points. The goal is to make points close to their own centroid, measured by the **inertia** (within-cluster sum of squares):

$$\text{inertia} = \sum_{i=1}^{n} \min_{j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

"For each point, the squared distance to its nearest centroid; add them all up." Lower is better.

Lloyd's algorithm

1. **Initialise:** pick k starting centroids (e.g. k random data points).
2. **Assign:** put each point in the cluster of its nearest centroid.
3. **Update:** move each centroid to the mean of the points assigned to it.
4. **Repeat** steps 2–3 until the assignments stop changing.

Each step can only lower (or keep) the inertia, so the algorithm always converges — usually within a few iterations.

✎ WORKED EXAMPLE — K-MEANS BY HAND IN 1D

Points: $1, 2, 3, 8, 9, 10$. $k = 2$. A deliberately bad start: centroids $\mu_1 = 1$, $\mu_2 = 2$.

Iteration 1 — assign: point 1 is nearest μ_1; points 2, 3, 8, 9, 10 are nearest (or tied with) μ_2. Clusters: $\{1\}$ and $\{2, 3, 8, 9, 10\}$.
Update: $\mu_1 = 1$, $\mu_2 = (2 + 3 + 8 + 9 + 10)/5 = 6.4$.

Iteration 2 — assign: point 2: distance 1 to μ_1, 4.4 to μ_2 → cluster 1. Point 3: 2 vs 3.4 → cluster 1. Points 8, 9, 10 → cluster 2. Clusters: $\{1, 2, 3\}$ and $\{8, 9, 10\}$.
Update: $\mu_1 = 2$, $\mu_2 = 9$.

Iteration 3: assignments don't change → converged.

Inertia: $(1-2)^2 + 0 + (3-2)^2 + (8-9)^2 + 0 + (10-9)^2 = 4$.

? MAKE SENSE OF IT

The situation: A delivery company has customers along one road at kilometres 1, 2, 3, 8, 9 and 10, and wants two depots. Where should they go?

What the result tells you: k-means puts the depots at 2 and 9 — the middle of each group of customers. The inertia, 4, is the total squared distance from every customer to their depot.

What if…?

- **A sensible start** (depots at 1 and 10): it reaches the same answer in one round instead of two.
- **Three depots, started at 2, 8 and 10:** groups {1, 2, 3}, {8, 9}, {10}, and the inertia falls to **2.5**. More clusters always lower the inertia — six depots would make it 0 — so inertia alone can't choose k.
- **Three depots, started at 1, 5 and 10:** the middle depot attracts no customers and stays empty, and the inertia stays at 4. A bad start can waste a cluster — which is why k-means++ and several restarts are used.
- **A far-away customer at 50** with two depots: one depot moves out to serve that single customer, the other has to cover 1–10 from 5.5, and the inertia leaps to **77.5**. Outliers can grab a whole cluster.

Why it matters in ML: k-means minimises squared distances, so like the mean it is pulled by outliers, depends on its starting points and on feature scale, and always returns exactly k groups whether or not they really exist.

STEP-BY-STEP DERIVATION **Why the centroid update uses the mean**

1 $$J(\boldsymbol{\mu}) = \sum_{i \in \text{cluster}} \|\mathbf{x}_i - \boldsymbol{\mu}\|^2$$

The inertia contributed by one cluster, as a function of its centroid.

2 $$\nabla J = -2 \sum_{i \in \text{cluster}} (\mathbf{x}_i - \boldsymbol{\mu})$$

Differentiate (chain rule, as in Lesson 7).

3 $$\sum_i \mathbf{x}_i - m\,\boldsymbol{\mu} = \mathbf{0}$$

Set to zero; m is the number of points in the cluster.

4 $$\boldsymbol{\mu} = \frac{1}{m} \sum_i \mathbf{x}_i$$

The best centroid is exactly the mean — so the update step can only lower the inertia.

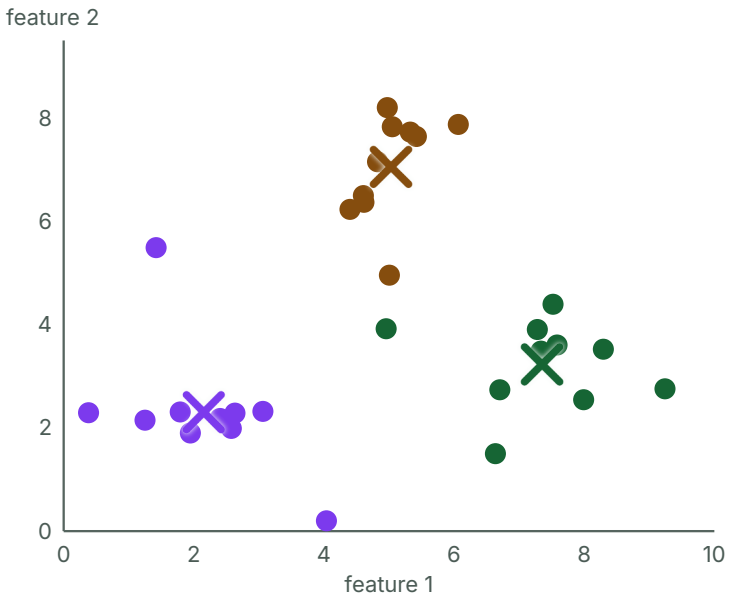
ADVANCED · OPTIONAL **Why k-means always stops** ▾ hide

Let $J = \sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|^2$ be the inertia for assignments c and centroids $\boldsymbol{\mu}$.

- **Assign step:** moving each point to its nearest centroid can only lower or keep its own term, so J does not increase.
- **Update step:** for a fixed cluster, $\sum_{i \in C} \|\mathbf{x}_i - \mathbf{m}\|^2$ is minimised by $\mathbf{m}$ = the mean (set its gradient $-2\sum_{i \in C}(\mathbf{x}_i - \mathbf{m})$ to zero). So J does not increase.

J never goes up, and there are only finitely many ways to assign n points to k clusters, so the algorithm cannot cycle and must stop. What's *not* guaranteed is the global minimum — finding that is NP-hard in general, which is why k-means++ starts and several restarts help.

WALKTHROUGH (FINAL STEP SHOWN) **k-means, iteration by iteration**



1. 30 unlabelled points and $k = 3$ starting centroids (×), picked at random from the data. Two of them landed in the same group — a poor start, on purpose.
2. Iteration 1 — **assign**: each point joins its nearest centroid. Inertia 232.0.
3. Iteration 1 — **update**: each centroid moves to the mean of its points. Inertia falls to 144.6.
4. Iteration 2 — **assign**: each point joins its nearest centroid (1 point switched cluster). Inertia 139.6.
5. Iteration 2 — **update**: each centroid moves to the mean of its points. Inertia falls to 134.7.
6. Iteration 3 — **assign**: each point joins its nearest centroid (2 points switched cluster). Inertia 131.9.
7. Iteration 3 — **update**: each centroid moves to the mean of its points. Inertia falls to 122.6.
8. Iteration 4 — **assign**: each point joins its nearest centroid (4 points switched cluster). Inertia 106.7.
9. Iteration 4 — **update**: each centroid moves to the mean of its points. Inertia falls to 83.4.
10. Iteration 5 — **assign**: each point joins its nearest centroid (3 points switched cluster). Inertia 64.9.
11. Iteration 5 — **update**: each centroid moves to the mean of its points. Inertia falls to 55.9.
12. Iteration 6 — **assign**: each point joins its nearest centroid (1 point switched cluster). Inertia 55.5.
13. Iteration 6 — **update**: each centroid moves to the mean of its points. Inertia falls to 53.9.
14. Iteration 7: assigning again changes nothing — the algorithm has **converged** after 6 updates. Final inertia 53.9.

READ THE FORMULA **k-means inertia**

$$\text{inertia} = \sum_{i=1}^{n} \min_{j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

- **Σ_i** — add up over every point
- **$\min_j$** — use the nearest centroid
- **$\|x_i - \mu_j\|^2$** — squared distance from the point to that centroid

Hover over (or tap) a part to highlight it in the formula.

3. Try it: k-means step by step

Press *Step* to alternate between the assign and update steps and watch the centroids (**+**) move. Try different k values and new random starts. Then switch to the *moons* dataset: k-means tries to make round clusters and cuts the two curved shapes in the wrong place.

Interactive demo — open *lesson-16.html* on the website to try it.

4. Initialisation matters: k-means++

Lloyd's algorithm always converges, but possibly to a **local minimum** — a poor clustering that a different start would beat. For example, two centroids might start inside the same natural group and split it, while another group gets merged with its neighbour.

Two standard fixes:

- **k-means++** initialisation: pick the first centroid at random, then pick each next one with probability proportional to its squared distance from the nearest centroid already chosen. Starting centroids end up spread out. It's scikit-learn's default.
- **Multiple restarts** (`n_init`): run k-means several times from different starts and keep the result with the lowest inertia.

5. Choosing k

Inertia always decreases as k grows — with $k = n$, every point is its own cluster and inertia is 0. So we can't just minimise it. Two common guides:

The elbow method

Plot inertia against k. It drops steeply while k is below the true number of groups, then flattens. The "elbow" where it bends is a good choice.

The silhouette score

For each point i:

- $a(i)$ = average distance to the other points in its **own** cluster (how tight its cluster is);
- $b(i)$ = average distance to the points of the **nearest other** cluster (how far the next cluster is).

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\ b(i)\big)}$$

s ranges from -1 to 1: near 1 means well inside its cluster and far from others; near 0 means on a border; negative means probably in the wrong cluster. The **silhouette score** is the average over all points; choose the k that maximises it.

✎ WORKED EXAMPLE

A point's average distance to its own cluster is $a = 1$, and to the nearest other cluster $b = 4$. Then $s = (4 - 1)/\max(1, 4) = 3/4 = 0.75$ — clearly well placed.

If instead $a = 3$ and $b = 2$: $s = (2 - 3)/3 \approx -0.33$ — it's closer to the other cluster, so it's probably misassigned.

? MAKE SENSE OF IT

The situation: Checking whether each customer really belongs with their depot or would be better served by the next one.

What the result tells you: 0.75 means the point is much closer to its own cluster than to the next one. A negative value means it's closer to another cluster — probably in the wrong one.

What if…?

- **a = 0.5, b = 4:** $s = 0.875$ — even better placed.
- **a = b:** $s = 0$ — right on the border between two clusters.

Why it matters in ML: The average silhouette over all points is a common way to compare different values of k: higher is better. Look at the individual values too — a cluster full of near-zero scores may not be a real group.

6. Limitations of k-means

- You must choose k in advance.
- It assumes clusters are roughly **round** (spherical) and similar in size — because it assigns by distance to a centre. It fails on elongated, curved or nested shapes.
- It's sensitive to **feature scales** (use standardisation) and to outliers (they pull centroids).
- Every point is forced into some cluster, even outliers that belong nowhere.

7. Hierarchical clustering

Agglomerative clustering builds clusters bottom-up:

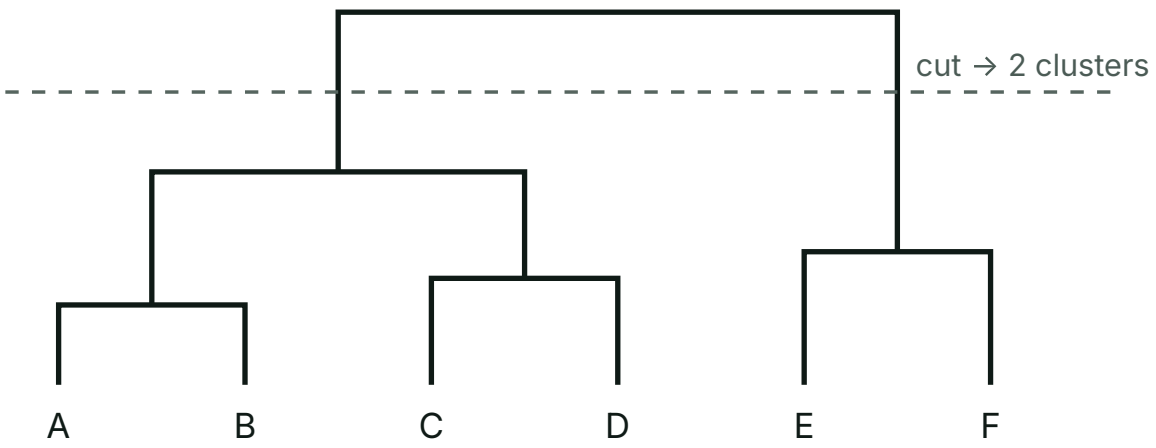


Figure 16. A dendrogram records every merge (lower = merged earlier, more similar). Cutting at the dashed line gives 2 clusters: {A, B, C, D} and {E, F}.

1. Start with every point as its own cluster.
2. Merge the two closest clusters.
3. Repeat until one cluster remains.

The full history of merges forms a tree called a **dendrogram**. Cutting it at a chosen height gives any number of clusters — you don't have to pick k before running it. "Closest clusters" needs a definition, called the **linkage**:

Linkage	Distance between clusters	Tends to give
Single	Closest pair of points	Long chains; follows curved shapes
Complete	Farthest pair of points	Compact clusters
Average	Average over all pairs	A compromise
Ward	Merge that increases total within-cluster variance the least	Similar-sized round clusters, like k-means (scikit-learn default)

The cost: it needs distances between all pairs of points, so it's slow for very large datasets.

8. DBSCAN: clusters as dense regions

DBSCAN (density-based spatial clustering of applications with noise) defines clusters as regions where points are packed closely together, separated by sparse regions. It has two settings:

- `eps` (ε) — the radius of a point's neighbourhood;
- `min_samples` — how many points must be within that radius for the region to count as dense.

Each point becomes one of three types:

- **Core point**: has at least `min_samples` points (counting itself, in scikit-learn) within distance ε.
- **Border point**: not a core point itself, but within ε of one.
- **Noise**: neither — an outlier, labelled -1.

Clusters grow by connecting core points that are within ε of each other, then adding their border points.

> **WHY DBSCAN IS USEFUL**
>
> It finds clusters of **any shape**, works out the number of clusters itself, and explicitly flags outliers instead of forcing them into a cluster. Its weakness: a single ε struggles when clusters have very different densities. `HDBSCAN` extends it to handle varying densities.

9. How do you know a clustering is good?

Without labels, there's no accuracy to compute. Options:

- **Internal scores** like silhouette — measure tightness and separation, but favour round clusters.
- **External scores** when some true labels exist for checking, such as the **adjusted Rand index (ARI)**: 1 = identical to the true groups, about 0 = no better than random.
- **Usefulness**: do the clusters make sense to domain experts and help the actual task?

10. In Python

Choosing k on data with 4 hidden groups:

```
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.preprocessing import StandardScaler

X, y_true = make_blobs(n_samples=500, centers=4, cluster_std=1.0, random_state=42)
for k in range(2, 8):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(f"k={k}  inertia={km.inertia_:8.1f}  silhouette={silhouette_score(X,
km.labels_):.3f}")
```

```
k=2  inertia= 15737.1  silhouette=0.596
k=3  inertia=  3426.3  silhouette=0.761
k=4  inertia=   948.9  silhouette=0.791
k=5  inertia=   855.9  silhouette=0.665
k=6  inertia=   768.7  silhouette=0.561
k=7  inertia=   681.0  silhouette=0.442
```

Inertia keeps falling, but its big drops stop after $k = 4$ (the elbow), and the silhouette score peaks at $k = 4$. Both point to the true number of groups.

Curved clusters — where k-means struggles:

```
Xm, ym = make_moons(n_samples=300, noise=0.06, random_state=0)
Xm = StandardScaler().fit_transform(Xm)

km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(Xm)
db = DBSCAN(eps=0.3, min_samples=5).fit(Xm)
ag = AgglomerativeClustering(n_clusters=2, linkage="single").fit(Xm)

print("k-means ARI:", round(adjusted_rand_score(ym, km.labels_), 3))
print("DBSCAN  ARI:", round(adjusted_rand_score(ym, db.labels_), 3),
      " clusters:", len(set(db.labels_) - {-1}), " noise points:",
list(db.labels_).count(-1))
print("single-linkage ARI:", round(adjusted_rand_score(ym, ag.labels_), 3))
```

```
k-means ARI: 0.47
DBSCAN  ARI: 0.993  clusters: 2  noise points: 1
single-linkage ARI: 1.0
```

k-means only half-recovers the two moons (ARI 0.47), because it cuts them with a straight line. DBSCAN finds both moons almost perfectly, works out by itself that there are 2 clusters, and flags 1 point as noise. Single-linkage hierarchical clustering follows the curves exactly.

11. Running example: Chennai Homes

k-means with $k = 3$ on size, distance and price, without seeing the area labels, finds: small central-ish homes (cluster 0), big pricey ones (cluster 1) and far, cheap ones (cluster 2). Cluster 2 is exactly the Outskirts houses 9 and 10 — the algorithm rediscovered geography from the numbers.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
cols = ["size_m2", "dist_km", "price_lakh"]
Z = StandardScaler().fit_transform(df[cols])
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Z)
df["cluster"] = km.labels_
print(df.groupby("cluster")[cols].mean().round(1))
print(pd.crosstab(df["cluster"], df["area"]))
```

Output:

```
         size_m2  dist_km  price_lakh
cluster
0           82.0      6.2        66.2
1          129.0     11.0        88.4
2          112.5     22.5        46.0
area     Central  Outskirts  Suburb
cluster
0              3          0       2
1              1          1       3
2              0          2       0
```

⚙ WHERE THIS IS USED

- Customer segmentation for marketing
- Grouping documents or images by similarity
- Finding unusual activity (points that fit no cluster)

Choose it when: You want to discover structure without labels.

Think twice when: You actually have labels for the task — use supervised learning.

× MISCONCEPTIONS TO UNLEARN

MYTH Clustering finds the true groups.

REALITY It finds groups according to its own definition of similarity, and it will return clusters even for random data.

MYTH k-means works for any cluster shape.

REALITY It assumes roughly round, similar-sized clusters; curved or nested shapes need DBSCAN or hierarchical clustering.

RESPONSIBLE AI · CLUSTERS ARE NOT NATURAL CATEGORIES

k-means will always return *k* groups, whether or not real groups exist. Giving clusters labels like "low-value customers" can become self-fulfilling if they decide who gets offers or attention. Treat clusters as a description of this dataset under these features, and be careful when they line up with sensitive attributes.

12. Questions beginners often ask

Q. How do I know the clusters are meaningful?

Check silhouette scores, look at examples from each cluster, and ask whether they're useful for your goal. Clustering always returns clusters, even in random data.

Q. k-means or DBSCAN?

k-means for roughly round clusters when you know (or can choose) k. DBSCAN for irregular shapes, unknown k and when you want outliers flagged.

Q. Why do I get different results each run?

k-means depends on its random starting centroids. Use k-means++ and several restarts (n_init) and fix random_state for reproducibility.

! COMMON MISTAKES TO AVOID

- Picking k by minimising inertia (it always prefers more clusters).
- Running k-means on unscaled features or on non-round cluster shapes.
- Treating cluster labels as ground truth without checking they're meaningful.

! INTERVIEW CORNER

Don't confuse

- **Clustering vs classification** — Known answer vs discovered groups.
- **Inertia vs silhouette score** — Inertia can't choose k; silhouette can help.
- **k-means vs DBSCAN** — Centres vs crowds.

Hidden questions — answer in your head first, then open.

▼ **What shapes of cluster does k-means find well, and which does it fail on?**

The trap: Thinking k-means finds any group.

It works best for compact, roughly spherical clusters of similar size, because it assigns each point to the nearest centre. It fails on elongated, ring-shaped or very unequal clusters — try DBSCAN, Gaussian mixtures or spectral clustering.

▼ **You run k-means twice and get different clusters. Is it broken?**

The trap: Expecting a deterministic answer.

No. It converges to a local optimum that depends on the starting centroids. Use k-means++ initialisation, several restarts (`n_init`), and keep the run with the lowest inertia. Fix the random seed for reproducibility.

▼ **Why can't you choose k by minimising inertia?**

The trap: Picking the k with the lowest inertia.

Inertia always falls as k grows and reaches 0 when every point is its own cluster. Look for an "elbow", use the silhouette score, or — best — ask which grouping is useful for the purpose.

▼ **How do you evaluate clustering when there are no labels?**

The trap: Saying "you can't".

Internal measures (silhouette, Davies–Bouldin), stability across resamples and random seeds, and above all usefulness: are the clusters interpretable and do they lead to different actions? If some labels exist, compare with them (adjusted Rand index).

13. Key takeaways

- Clustering finds groups in **unlabelled** data.
- **k-means** alternates "assign to nearest centroid" and "move centroid to mean", minimising inertia $\sum \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|^2$.
- Use **k-means++** and several restarts; choose k with the elbow method or the **silhouette score** $\frac{b-a}{\max(a,b)}$.
- k-means assumes round clusters; **hierarchical clustering** and **DBSCAN** handle other shapes, and DBSCAN flags outliers.
- Scale features first, and judge clusters by whether they're useful.

14. Check your understanding

1. In k-means, after assigning points, each centroid is moved to...

a. the point farthest from it

b. the mean of the points assigned to it

c. a random new location

2. Why can't you choose k by minimising inertia?

a. Inertia can't be computed without labels

b. Inertia increases with k

c. Inertia always decreases as k grows, reaching 0 when every point is its own cluster

3. A point has $a = 2$ and $b = 6$. What is its silhouette value?

a. 0.67

b. 0.33

c. 3

4. Your data has two interlocking crescent shapes and a few outliers. Which method fits best?

a. k-means with $k = 2$

b. DBSCAN

c. k-means with $k = 10$

5. In DBSCAN, what is a core point?

a. The centre of a cluster

b. A point labelled as noise

c. A point with at least `min_samples` points within distance ε

15. Going deeper: mixture models, density and hierarchy

INTERMEDIATE · SKIP ON A FIRST READ

k-means is one answer to "find groups", built on one assumption: clusters are round blobs of similar size. This section covers the main alternatives and when each is right: **Gaussian mixture models** (soft, stretched clusters, fitted by the EM algorithm), **DBSCAN** (clusters of any shape, plus outliers) and **hierarchical clustering** (a whole tree of groupings), along with principled ways to choose the number of clusters.

You'll need: this lesson, maximum likelihood from Lesson 4's "Going deeper", and covariance from Lesson 17.

Gaussian mixture models and the EM algorithm

A **Gaussian mixture model** (GMM) assumes each point was produced by first picking a cluster k with probability π_k, then drawing from a normal distribution with that cluster's mean $\boldsymbol{\mu}_k$ and covariance matrix Σ_k. The covariance lets each cluster be stretched and tilted, not just round. It's fitted by the **expectation–maximisation** (EM) algorithm, which alternates two steps much like k-means:

- **E-step:** for every point, compute the probability ("responsibility") that each cluster produced it, using the current parameters and Bayes' theorem.
- **M-step:** re-estimate each cluster's π_k, $\boldsymbol{\mu}_k$ and Σ_k as weighted averages, with each point weighted by its responsibility.

Each round never lowers the likelihood, so EM converges, though like k-means it can stop at a local optimum, and it's restarted from several starting points. k-means is in fact a limiting case of EM: round clusters with equal tiny variance, and "hard" responsibilities of 0 or 1.

```
import numpy as np
from sklearn.datasets import make_blobs, make_moons
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score, silhouette_score

# Stretched clusters: k-means assumes round blobs, a Gaussian mixture doesn't
Xb, yb = make_blobs(n_samples=600, centers=3, random_state=170)
Xs = Xb @ np.array([[0.6, -0.6], [-0.4, 0.8]])          # stretch and shear the blobs
km = KMeans(3, n_init=10, random_state=0).fit_predict(Xs)
gmm = GaussianMixture(3, covariance_type="full", random_state=0).fit(Xs)
print("stretched blobs: ARI k-means", round(adjusted_rand_score(yb, km), 3), "  ARI
Gaussian mixture", round(adjusted_rand_score(yb, gmm.predict(Xs)), 3))
p = gmm.predict_proba(Xs)
print("share of points the mixture is unsure about (max probability < 0.9):",
round(np.mean(p.max(axis=1) < 0.9), 3))
```

Output:

```
stretched blobs: ARI k-means 0.58   ARI Gaussian mixture 1.0
share of points the mixture is unsure about (max probability < 0.9): 0.002
```

On three stretched, tilted blobs, k-means gets an adjusted Rand index (ARI, 1 = perfect agreement with the true groups, 0 = chance) of only 0.58: its round-cluster assumption cuts the long clusters in the wrong places. The Gaussian mixture recovers the groups perfectly (ARI 1.0). As a bonus it gives **soft** assignments: here only 0.2% of points are uncertain (no cluster above 90%), but in overlapping data those probabilities show which points are genuinely ambiguous.

Clusters of any shape: DBSCAN and linkage

Some clusters aren't blobs at all. **DBSCAN** (this lesson) grows clusters through dense regions: a point with at least `min_samples` neighbours within distance `eps` is a "core" point, clusters are chains of connected core points, and anything not reachable is labelled noise. **Agglomerative (hierarchical) clustering** starts with every point alone and repeatedly merges the two closest groups. The **linkage** rule defines "closest": *single* linkage uses the closest pair of points (it can follow long, thin shapes), and *Ward* linkage merges the pair that least increases within-cluster variance (it prefers compact, k-means-like groups). Here are all five methods on two interlocking half-moons:

```
Xm, ym = make_moons(n_samples=500, noise=0.06, random_state=0)
results = {
    "k-means":            KMeans(2, n_init=10, random_state=0).fit_predict(Xm),
    "Gaussian mixture":   GaussianMixture(2, random_state=0).fit(Xm).predict(Xm),
    "DBSCAN":             DBSCAN(eps=0.2, min_samples=5).fit_predict(Xm),
    "agglomerative, single linkage": AgglomerativeClustering(2,
linkage="single").fit_predict(Xm),
    "agglomerative, Ward linkage":   AgglomerativeClustering(2,
linkage="ward").fit_predict(Xm),
}
for name, lab in results.items():
    print(f"{name:30}: ARI {adjusted_rand_score(ym, lab):.3f}")
```

Output:

```
k-means                       : ARI 0.253
Gaussian mixture              : ARI 0.495
DBSCAN                        : ARI 1.000
agglomerative, single linkage : ARI 1.000
agglomerative, Ward linkage   : ARI 0.559
```

DBSCAN and single linkage both recover the moons perfectly (ARI 1.000) because they follow connected, dense regions. k-means (0.253), the Gaussian mixture (0.495) and Ward linkage (0.559) all assume compact groups and cut each moon in half. The catch with single linkage is "chaining": a thin bridge of noise points can merge two real clusters. DBSCAN is sensitive to `eps`, and struggles when clusters have very different densities; **HDBSCAN** (in scikit-learn since version 1.3) fixes that by trying many densities at once.

Hierarchical clustering also produces a **dendrogram**, a tree showing every merge and the distance at which it happened. Cutting it at different heights gives different numbers of clusters, which is useful when the "right" level of detail isn't known in advance (for example customer segments, or a taxonomy of documents).

Choosing the number of clusters

Without labels, use criteria that reward tight, well-separated clusters without simply rewarding more clusters:

- **Silhouette score**: for each point, $s = (b - a)/\max(a, b)$, where a is its average distance to its own cluster and b to the nearest other cluster. Averaged over points, higher is better; values near 1 mean clear clusters.
- **BIC** for mixture models: $-2 \log L + p \log n$, where p counts parameters. It penalises extra clusters; lower is better.

```
# Choosing k without labels: silhouette score and the mixture's BIC
Xk, _ = make_blobs(n_samples=600, centers=4, cluster_std=1.0, random_state=7)
print(" k   silhouette   BIC")
for k in range(2, 8):
    lab = KMeans(k, n_init=10, random_state=0).fit_predict(Xk)
    bic = GaussianMixture(k, random_state=0).fit(Xk).bic(Xk)
    print(f"{k:2d}   {silhouette_score(Xk, lab):10.3f}   {bic:7.0f}")
```

Output:

```
 k   silhouette   BIC
 2        0.588      6194
 3        0.762      5490
 4        0.811      5133
 5        0.680      5168
 6        0.577      5193
 7        0.446      5227
```

The data has 4 true clusters. The silhouette peaks at $k = 4$ (0.811) and BIC is lowest at $k = 4$ (5133), so both point to the right answer. On real data the signals are often weaker or disagree. Treat them as guides, and choose k also by whether the clusters are *useful*: can someone describe and act on each segment?

Choosing a clustering method

Method	Finds	Needs	Watch out for
k-means	Round blobs of similar size	k	Scaling; elongated or uneven clusters
Gaussian mixture	Elliptical clusters, soft membership	k (choose by BIC)	Local optima; many parameters in high dimensions
DBSCAN / HDBSCAN	Any shape; marks noise	`eps`, `min_samples`	Very different densities (DBSCAN); high dimensions
Agglomerative	A hierarchy of groups	Linkage; where to cut	About $O(n^2)$ memory; chaining with single linkage

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Compare k-means and Gaussian mixture models.**

The trap: Saying they're the same thing.

Both alternate assignment and update steps. k-means makes hard assignments to the nearest centroid, assumes round clusters of similar size, and minimises within-cluster squared distance. A GMM makes soft probabilistic assignments, learns each cluster's covariance (stretched, tilted, different sizes) and maximises likelihood with EM. GMMs give uncertainty and a likelihood (so BIC can choose k), but have more parameters, can be unstable in high dimensions, and are slower.

▼ **When would you choose DBSCAN over k-means?**

The trap: Only saying "for non-round clusters".

When clusters have arbitrary shapes, when you don't know the number of clusters, and when the data contains noise or outliers that shouldn't be forced into a cluster (DBSCAN labels them -1). Avoid it when cluster densities differ a lot (use HDBSCAN) or in high dimensions where distances concentrate (reduce dimensions first).

▼ **How can you evaluate a clustering if you do have labels for a sample?**

The trap: Using accuracy directly.

Cluster numbers are arbitrary, so use label-invariant measures: the adjusted Rand index (pairs of points grouped consistently, corrected for chance), normalised mutual information, or homogeneity and completeness. A score near 0 means chance-level agreement. Remember the labels may reflect only one of several valid ways to group the data.

Check your understanding: going deeper

1. What does the E-step of EM compute for a Gaussian mixture?

a. New cluster means

b. Each point's probability of belonging to each cluster

c. The number of clusters

2. Which method can find two interlocking half-moon shapes?

a. k-means

b. DBSCAN

c. Ward linkage

3. A point's silhouette is near −1. What does that suggest?

a. It's well clustered

b. It's probably in the wrong cluster

c. It's exactly between two clusters

4. For Gaussian mixtures, which k does BIC prefer?

a. The one with the lowest BIC

b. The one with the highest BIC

c. Always the largest

5. k-means can be seen as a special case of...

a. EM for a Gaussian mixture with equal, round, tiny-variance clusters and hard assignments

b. DBSCAN

c. PCA

16. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Run k-means with $k = 2$ on $1, 2, 6, 7, 8$, starting from centroids 1 and 8. Give the final centroids and inertia.

WORKING

EXERCISE 2 By hand

A point has $a = 2$ (own cluster) and $b = 5$ (nearest other cluster). What is its silhouette value?

WORKING

EXERCISE 3 By hand

1D points $1, 2, 3, 6, 10$; DBSCAN with $\varepsilon = 1$ and `min_samples` $= 3$ (counting the point itself). Label each point core, border or noise.

WORKING

EXERCISE 4 Concepts

Which method fits? (a) Segment customers into exactly 5 groups for 5 marketing teams; (b) find unusual GPS traces among millions of normal ones; (c) see how species group at every level of similarity.

WORKING

EXERCISE 5 Intermediate

A point's average distance to its own cluster is $a = 2$ and to the nearest other cluster is $b = 5$. What is its silhouette?

WORKING

EXERCISE 6 Intermediate

Two models are fitted to $n = 1000$ points. Model A has log-likelihood -2400 with 11 parameters; model B has -2380 with 17. Using $\mathrm{BIC} = -2\log L + p\ln n$, which is preferred?

WORKING

EXERCISE 7 Intermediate

In the E-step, a point's weighted densities under three clusters are 0.02, 0.06 and 0.12. What are its responsibilities?

WORKING

NOTES

PART 4 · FINDING STRUCTURE WITHOUT LABELS · ~55 MIN

Lesson 17: Dimensionality Reduction & PCA

Compress many features into a few: variance, principal components and t-SNE.

BY THE END OF THIS LESSON YOU CAN

- Explain why reducing dimensions helps
- Compute a covariance matrix and its eigenvalues by hand for 2D data
- Find principal components, explained variance and projections
- Choose the number of components and read t-SNE plots safely

↺ BEFORE YOU START

This lesson builds on:

- Lesson 2: Vectors & Matrices — A vector is a list of numbers; a dataset X is a matrix with one row per example. The dot product $\mathbf{w} \cdot \mathbf{x} = \sum w_i x_i$ is how linear models predict, and $\|\mathbf{a} - \mathbf{b}\|$ is the distance between two examples.
- Lesson 4: Probability & Statistics — Probabilities run from 0 to 1. Bayes' theorem turns $P(B \mid A)$ into $P(A \mid B)$. Mean and standard deviation summarise data; the z-score $(x - \mu)/\sigma$ standardises it.
- Lesson 16: Clustering — Clustering groups unlabelled data. k-means alternates assigning points to the nearest centroid and moving each centroid to the mean of its points.

Where this fits: The second unsupervised tool: compressing many features into a few, for visualisation, speed and noise reduction.

1. Too many features

PICTURE IT A photo of a teapot from its best angle tells you almost everything about its shape; a photo straight down the spout tells you very little. PCA finds the "best angles" to look at your data from, so that a few of them show nearly everything.

A small 8×8 image of a handwritten digit is 64 numbers. A 1-megapixel photo is 3 million. A customer record might have hundreds of columns. Lots of features cause real problems:

- **The curse of dimensionality** (Lesson 11): in many dimensions, data is sparse and distances lose meaning.

- **Overfitting**: more features means more parameters to fit noise (Lesson 10).
- **Slowness** and memory use grow with the number of features.
- **We can't see it**: humans can plot 2 or 3 dimensions, not 64.

Fortunately, real features are often **redundant**. Height in cm and height in inches carry the same information; neighbouring pixels in an image are strongly correlated. **Dimensionality reduction** compresses many features into a few new ones while keeping most of the information. The most important method is **principal component analysis (PCA)**.

> ◎ ANALOGY: PHOTOGRAPHING A TEAPOT
>
> You have to describe a 3D teapot with a single 2D photo. Some angles (straight down from the top) lose almost everything — you just see a circle. The best angle, from the side, shows the spout, handle and lid: the photo keeps the most "spread-out" view of the object. PCA finds the best camera angle for your data: the directions in which the data varies the most.

2. The key idea: keep the directions with the most variance

Imagine points in 2D that lie roughly along a diagonal line. Two numbers (x_1, x_2) describe each point, but really the points differ mainly in one way: how far along the line they are. If we rotate our axes so the first axis runs along the line, then:

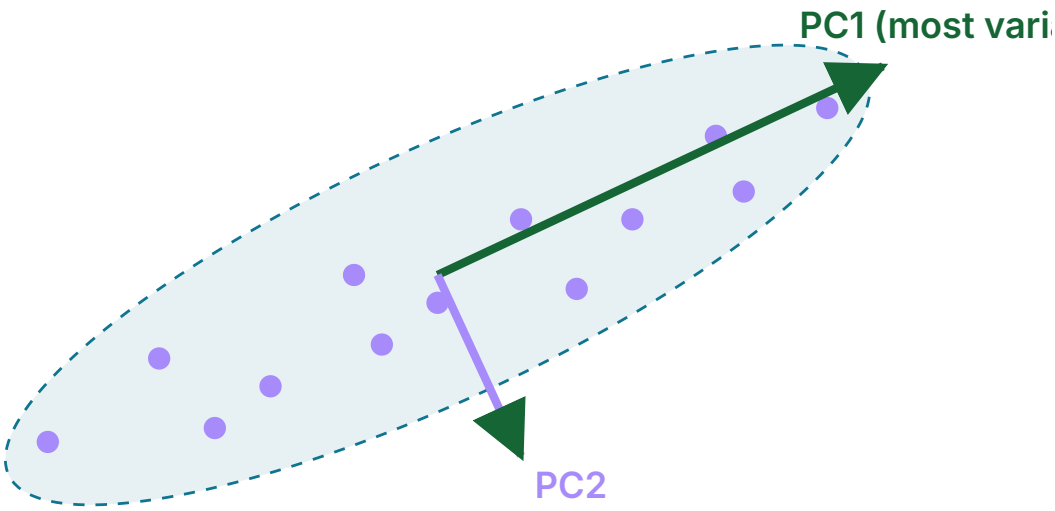


Figure 17. PCA rotates the axes: PC1 points along the data's longest spread; PC2 is perpendicular and captures what's left.

- the first new coordinate captures almost all the differences between points;
- the second new coordinate (perpendicular) is nearly the same for every point — mostly noise.

Dropping the second coordinate loses very little. That's PCA: **find new axes, ordered by how much of the data's variance they capture, and keep only the first few**.

> ≡ PRINCIPAL COMPONENTS
>
> The **first principal component** (PC1) is the direction along which the data has the largest variance. The second (PC2) is the direction of largest remaining variance that is perpendicular to PC1, and so on. With d features there are d principal components; we keep the first k.

3. Try it: find the best direction

Rotate the orange line. Each point is projected (dropped at a right angle) onto the line, and the readout shows the variance of those projected positions — how much "spread" this 1D view keeps. Find the angle that maximises it: that's PC1. Press *Snap to PC1* to check, and notice that the best direction also minimises the total squared length of the projection lines (the information lost).

Interactive demo — open *lesson-17.html* on the website to try it.

4. Measuring how features vary together: covariance

To find directions of maximum variance, we need to know how features vary *together*. The **covariance** of two features (after subtracting their means) is the average product of their deviations:

$$\operatorname{cov}(x_1, x_2) = \frac{1}{n-1}\sum_{i=1}^{n}(x_{i1} - \bar{x}_1)(x_{i2} - \bar{x}_2)$$

- Positive: they tend to be above average together (like height and weight).
- Negative: when one is above average, the other tends to be below.
- Near zero: no linear relationship.

The covariance of a feature with itself is its variance (Lesson 4). Putting all the pairs in a table gives the **covariance matrix** Σ ($d \times d$, symmetric). For centred data matrix X_c it's simply

$$\Sigma = \frac{1}{n-1} X_c^\top X_c$$

5. Eigenvectors: the directions PCA finds

Here's the central fact (proved in linear algebra courses): **the principal components are the eigenvectors of the covariance matrix, and the variance along each one is its eigenvalue.**

An **eigenvector** of a matrix Σ is a special direction $\mathbf{v}$ that the matrix only stretches, without turning:

$$\Sigma \mathbf{v} = \lambda \mathbf{v}$$

The stretch factor λ is the **eigenvalue**. For a covariance matrix, eigenvectors are perpendicular to each other and eigenvalues are never negative. Sorting the eigenvectors by eigenvalue, largest first, gives PC1, PC2, …

The fraction of the total variance captured by component j is the **explained variance ratio**:

$$\text{explained}_j = \frac{\lambda_j}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$$

> **FOR 2×2 MATRICES THERE'S A FORMULA**
>
> For $\Sigma = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$ the eigenvalues are $\lambda = \frac{a+c}{2} \pm \sqrt{\left(\frac{a-c}{2}\right)^2 + b^2}$. Bigger matrices are handled by software (scikit-learn computes PCA with the singular value decomposition, an equivalent and more numerically stable method).

STEP-BY-STEP DERIVATION

Why the best direction is the top eigenvector

1 $z_i = \mathbf{u}^\top \mathbf{x}_i, \qquad \|\mathbf{u}\| = 1$

Project each centred point onto a unit direction $\mathbf{u}$.

2 $\mathrm{Var}(z) = \frac{1}{n-1}\sum_i (\mathbf{u}^\top \mathbf{x}_i)^2 = \mathbf{u}^\top \Big(\frac{1}{n-1}\sum_i \mathbf{x}_i \mathbf{x}_i^\top\Big)\mathbf{u} = \mathbf{u}^\top \Sigma\, \mathbf{u}$

The variance along $\mathbf{u}$ is a simple expression in the covariance matrix.

3 $\mathbf{u} = \mathbf{v}_j \Rightarrow \mathbf{u}^\top \Sigma \mathbf{u} = \mathbf{v}_j^\top \lambda_j \mathbf{v}_j = \lambda_j$

Along an eigenvector, the variance is its eigenvalue.

4 $\mathbf{u} = \sum_j c_j \mathbf{v}_j \Rightarrow \mathbf{u}^\top \Sigma \mathbf{u} = \sum_j \lambda_j c_j^2, \quad \sum_j c_j^2 = 1$

Any unit direction is a mix of the (perpendicular) eigenvectors.

5 $\max_{\mathbf{u}} \mathbf{u}^\top \Sigma \mathbf{u} = \lambda_1 \;\text{ at }\; \mathbf{u} = \mathbf{v}_1$

A weighted average of eigenvalues is largest when all the weight is on the biggest one: PC1.

READ THE FORMULA

Explained variance ratio

$$\text{explained}_j = \frac{\lambda_j}{\lambda_1 + \lambda_2 + \cdots + \lambda_d}$$

- **λ_j** — variance along component j (its eigenvalue)
- **$\lambda_1 + \ldots + \lambda_d$** — total variance in the data

Hover over (or tap) a part to highlight it in the formula.

6. Worked example: PCA by hand

✎ WORKED EXAMPLE — FIVE POINTS IN 2D

Data: $(1,2),\ (2,3),\ (3,5),\ (4,4),\ (5,6)$.

Step 1 — centre. Means: $\bar{x}_1 = 3$, $\bar{x}_2 = 4$. Subtract them:

$$(-2,-2),\ (-1,-1),\ (0,1),\ (1,0),\ (2,2)$$

Step 2 — covariance matrix (dividing by $n-1=4$):

$\mathrm{var}(x_1) = \frac{4+1+0+1+4}{4} = 2.5,\ \mathrm{var}(x_2) = \frac{4+1+1+0+4}{4} = 2.5,$
$\mathrm{cov}(x_1,x_2) = \frac{4+1+0+0+4}{4} = 2.25.$

$$\Sigma = \begin{bmatrix} 2.5 & 2.25 \\ 2.25 & 2.5 \end{bmatrix}$$

Step 3 — eigenvalues: $\lambda = 2.5 \pm \sqrt{0 + 2.25^2} = 2.5 \pm 2.25$, so $\lambda_1 = 4.75$ and $\lambda_2 = 0.25$.

Step 4 — eigenvectors: PC1 $= \frac{1}{\sqrt{2}}(1,1)$, the diagonal. Check: $\Sigma\,(1,1) = (2.5+2.25,\ 2.25+2.5) = (4.75, 4.75) = 4.75\,(1,1)$ ✓. PC2 $= \frac{1}{\sqrt{2}}(1,-1)$, perpendicular.

Step 5 — explained variance: PC1 keeps $\frac{4.75}{4.75+0.25} = 95\%$ of the variance. One number per point instead of two, losing only 5%.

Step 6 — project: each point's new coordinate is the dot product of its centred vector with PC1. For $(-2,-2)$: $\frac{-2-2}{\sqrt{2}} \approx -2.83$. All five: $-2.83,\ -1.41,\ 0.71,\ 0.71,\ 2.83$.

? MAKE SENSE OF IT

The situation: Two exam scores per student — maths and physics — that rise and fall together. Could one number describe each student almost as well?

What the result tells you: PC1 is the diagonal direction "good at both". It holds 95% of the variation, so a single score per student keeps nearly all the information. PC2, "better at one than the other", holds only 5%.

What if…?

- **Points exactly on a line** (1, 1), (2, 2), …: the second eigenvalue is 0 and PC1 keeps **100%**. One number loses nothing.
- **Weakly related scores** (1, 4), (2, 2), (3, 6), (4, 4), (5, 4): PC1 keeps only **62%**. Dropping PC2 would throw away a lot.
- **Measure the second score in different units** (×10): PC1 now keeps **99.8%** and points almost entirely along that feature (0.996). Nothing about the students changed — PCA simply chases the feature with the biggest numbers.

Why it matters in ML: PCA finds the directions of greatest variance, so it works best when features are correlated — and it must be run on standardised features, or the units decide the answer. Check the explained-variance ratio before deciding how many components to keep.

7. PCA step by step

1. **Centre** the data (subtract each feature's mean). Usually also **standardise** (divide by the standard deviation) if features are in different units — otherwise a feature measured in large numbers dominates the variance.
2. Compute the **covariance matrix** (or use the SVD directly).
3. Find its **eigenvectors and eigenvalues**, sorted by eigenvalue.
4. Choose k and keep the top k eigenvectors as the columns of a matrix W_k ($d \times k$).
5. **Project**: $Z = X_c W_k$, giving $n \times k$ new features.

You can also approximately **reconstruct** the original data from the compressed version: $\hat{X} = ZW_k^\top + \bar{\mathbf{x}}$. The reconstruction error is exactly the variance you threw away.

ADVANCED · OPTIONAL **Deriving PCA — from maximum variance and from the SVD** ▾ hide

Maximum variance. For centred data with covariance C, the variance along a unit vector $\mathbf{u}$ is $\mathbf{u}^\top C\mathbf{u}$. Maximise it subject to $\mathbf{u}^\top\mathbf{u} = 1$ with a Lagrange multiplier: $\nabla_{\mathbf{u}}\left[\mathbf{u}^\top C\mathbf{u} - \lambda(\mathbf{u}^\top\mathbf{u} - 1)\right] = 2C\mathbf{u} - 2\lambda\mathbf{u} = 0$, so $C\mathbf{u} = \lambda\mathbf{u}$: $\mathbf{u}$ must be an eigenvector, and the variance it captures is $\mathbf{u}^\top C\mathbf{u} = \lambda$. The best direction is the eigenvector with the largest eigenvalue.

Via the SVD (how libraries actually do it). Write the centred $n \times d$ data matrix as $X = U\Sigma V^\top$, with U, V orthogonal and Σ diagonal with singular values $s_1 \geq s_2 \geq \ldots$. Then

$$C = \frac{X^\top X}{n-1} = \frac{V\Sigma U^\top U\Sigma V^\top}{n-1} = V\,\frac{\Sigma^2}{n-1}\,V^\top.$$

This is an eigendecomposition of C: the columns of V are the principal components and the eigenvalues are $\lambda_i = s_i^2/(n-1)$. The projected data is $XV = U\Sigma$. The SVD is preferred because it never forms $X^\top X$, which would square the condition number and lose numerical precision.

8. How many components?

- **Explained variance threshold:** keep enough components to explain, say, 95% of the variance. In scikit-learn: `PCA(n_components=0.95)`.
- **Scree plot:** plot eigenvalues in order and look for an elbow, as with k-means.
- **For a downstream model:** treat k as a hyperparameter and choose it by cross-validation.
- **For visualisation:** $k = 2$ or 3.

9. Uses and limitations

Good for	Watch out for
Visualising high-dimensional data in 2D	Components are mixtures of features — harder to interpret
Speeding up and regularizing other models	Only finds *linear* structure (straight directions)
Removing noise (drop low-variance components)	High variance ≠ useful for prediction; a low-variance direction may matter for the label
Compressing data	Sensitive to feature scale — standardise first

10. Beyond PCA: t-SNE and UMAP for visualisation

When structure is curved (like the moons in Lesson 16), a linear projection can squash different groups on top of each other. **t-SNE** is a non-linear method designed purely for visualisation: it places points in 2D so that points that were *neighbours* in the original space stay neighbours. Its main setting, **perplexity**, roughly controls how many neighbours each point pays attention to. **UMAP** is a newer, faster method with a similar goal.

! READING T-SNE PLOTS SAFELY

t-SNE pictures are easy to over-interpret. The authors of the guide "How to Use t-SNE Effectively" show that cluster *sizes* in a t-SNE plot mean nothing, distances *between* well-separated clusters may not mean anything either, and with a low perplexity even pure random noise can appear to form clusters. Different runs can give different pictures. Use t-SNE to explore, never as proof — and unlike PCA, it can't transform new data for a model.

11. In Python

The digits dataset: 1,797 images of handwritten digits, each 8×8 = 64 pixel features.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

X, y = load_digits(return_X_y=True)
pca = PCA().fit(X)
cum = np.cumsum(pca.explained_variance_ratio_)
for k in [2, 10, 20, 30]:
    print(f"{k:2d} components keep {cum[k-1]:.1%} of the variance")
print("components needed for 95%:", np.argmax(cum ⩾ 0.95) + 1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)
for k in [5, 10, 20, 40, 64]:
    model = make_pipeline(StandardScaler(), PCA(n_components=k),
LogisticRegression(max_iter=2000))
    model.fit(X_train, y_train)
    print(f"{k:2d} components: test accuracy {model.score(X_test, y_test):.3f}")
```

Output:

```
 2 components keep 28.5% of the variance
10 components keep 73.8% of the variance
20 components keep 89.4% of the variance
30 components keep 95.9% of the variance
components needed for 95%: 29
 5 components: test accuracy 0.820
10 components: test accuracy 0.867
20 components: test accuracy 0.956
40 components: test accuracy 0.971
64 components: test accuracy 0.969
```

29 of the 64 pixel features hold 95% of the variance. With 40 components the classifier is as accurate as with all 64 (slightly better, in fact), using under two-thirds of the features. Even 20 components give 95.6%. To plot the digits in 2D, use `PCA(n_components=2).fit_transform(X)` — or `sklearn.manifold.TSNE` for a non-linear picture.

12. Running example: Chennai Homes

PCA on the four standardised features: the first component captures 61.5% of the variance. Its weights are all positive — it measures "big, far and older" all at once — and four components can be squeezed into two while keeping about 82.5% of the information.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
cols = ["size_m2", "bedrooms", "dist_km", "age_yrs"]
pca = PCA().fit(StandardScaler().fit_transform(df[cols]))
print("explained variance ratio:", pca.explained_variance_ratio_.round(3))
print("PC1 weights:", {c: round(float(v), 2) for c, v in zip(cols,
pca.components_[0])})
```

Output:

```
explained variance ratio: [0.615 0.21  0.165 0.01 ]
PC1 weights: {'size_m2': 0.61, 'bedrooms': 0.57, 'dist_km': 0.42, 'age_yrs':
0.34}
```

⚙ WHERE THIS IS USED

- Visualising high-dimensional data in 2D
- Compressing features before another model
- Removing noise from signals and images

Choose it when: Many correlated features, or you need a 2D picture.

Think twice when: Interpretability of individual original features matters, or structure is strongly non-linear.

× MISCONCEPTIONS TO UNLEARN

MYTH PCA picks the most important original features.

REALITY Each component is a mixture of all the original features.

MYTH Distances in a t-SNE picture are meaningful.

REALITY Cluster sizes and gaps between clusters in t-SNE plots can be misleading; it's for exploration only.

RESPONSIBLE AI · REDUCING DIMENSIONS DOES NOT ANONYMISE

Compressing or transforming data does not make it anonymous. Narayanan and Shmatikov (2008) re-identified users in the "anonymised" Netflix Prize dataset by matching a handful of their movie ratings with public reviews. A few dimensions, or a few rare values, can be enough to single someone out; treat derived features of personal data as personal data.

13. Questions beginners often ask

Q. Does PCA throw away information?

Yes, the variance in the dropped components — by design, usually a small amount. The explained-variance ratio tells you how much you kept.

Q. Should I always use PCA before a model?

No. It helps with many correlated features, visualisation or speed. With few features, or when interpretability matters, it's often unnecessary.

Q. PCA or t-SNE?

PCA for compression and as input to models; t-SNE (or UMAP) only for exploring a 2D picture.

! COMMON MISTAKES TO AVOID

- Running PCA on unstandardised features measured in different units.
- Fitting PCA on all data before the train/test split.
- Reading cluster sizes or gaps in a t-SNE plot as real distances.

! INTERVIEW CORNER

Don't confuse

- **PCA vs t-SNE** — PCA to compress; t-SNE to look.
- **Feature selection vs feature extraction** — Choose columns vs make new ones.
- **Eigenvector vs eigenvalue** — Direction vs amount.

Hidden questions — answer in your head first, then open.

▼ **Should you standardise before PCA?**

The trap: Running PCA on raw features.

Almost always. PCA chases variance, so a feature in large units dominates. In Lesson 17, measuring one score ×10 made PC1 keep 99.8% of the variance and point almost entirely along that feature.

▼ **PCA keeps 95% of the variance. Can it still remove the feature that best predicts the target?**

The trap: Assuming high variance means useful.

Yes. PCA ignores the target. A low-variance direction can be the most predictive one, and PCA might discard it. If prediction is the goal, compare with supervised selection, or tune the number of components by cross-validated performance.

▼ **You fit PCA on the full dataset, then split into train and test. Problem?**

The trap: Thinking unsupervised steps can't leak.

Yes, a mild leak: the components were shaped by the test data. Fit PCA on the training data only (inside a Pipeline) and apply that transformation to the test data.

▼ **In a t-SNE plot, two clusters are far apart. Are they very different?**

The trap: Reading t-SNE like a map.

Not necessarily. t-SNE preserves local neighbourhoods, not global distances or cluster sizes, and its picture changes with the perplexity setting. Use it to explore, not to measure.

14. Key takeaways

- Dimensionality reduction compresses many correlated features into a few new ones.
- **PCA** finds perpendicular directions of maximum variance: the **eigenvectors** of the covariance matrix, ordered by **eigenvalue**.
- Explained variance ratio $= \lambda_j / \sum \lambda$; keep enough components for e.g. 95%.
- Centre (and usually standardise) first; project with $Z = X_c W_k$.
- PCA is linear; **t-SNE**/**UMAP** give non-linear pictures for exploration only.

15. Check your understanding

1. What is the first principal component?

a. The feature with the largest values

b. The direction along which the data varies the most

c. The average of all features

2. Eigenvalues are 6, 3 and 1. What fraction of the variance do the first two components explain?

a. 60%

b. 70%

c. 90%

3. Why standardise features before PCA?

a. Otherwise features with large units dominate the variance, and so the components

b. PCA only accepts values between 0 and 1

c. It removes the need for eigenvectors

4. $\Sigma\mathbf{v} = 3\mathbf{v}$. What is $\mathbf{v}$ and what is 3?

a. $\mathbf{v}$ is an eigenvalue, 3 is an eigenvector

b. $\mathbf{v}$ is an eigenvector, 3 is its eigenvalue

c. $\mathbf{v}$ is the mean, 3 is the variance

5. A t-SNE plot shows one big cluster and one small one. What can you conclude about their real sizes?

a. The big one has more spread in the original space

b. The big one has more points

c. Nothing reliable — cluster sizes in t-SNE plots don't carry meaning

16. Going deeper: choosing components, and non-linear maps

INTERMEDIATE · SKIP ON A FIRST READ

PCA finds the best *flat* summary of the data. This section shows how to decide how many components to keep, what reconstruction error tells you, and what to do when the structure is curved: **kernel PCA** for features, and **t-SNE** and **UMAP** for pictures, including how to read those pictures without being misled.

You'll need: this lesson, and eigenvectors and the SVD from Lesson 2's "Going deeper".

How many components?

Keeping k components has an exact cost: the **reconstruction error**, how far each point is from its projection back into the original space, equals the variance in the components you dropped. Projecting onto the top k principal components gives the smallest possible error of any k-dimensional flat subspace (the Eckart–Young result from Lesson 2's "Going deeper"). On the 8×8 handwritten digits (64 pixel features):

```
import numpy as np
from sklearn.datasets import load_digits, make_circles
from sklearn.decomposition import PCA, KernelPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

# How many components? Variance kept and reconstruction error on the digits (64 pixels)
X, y = load_digits(return_X_y=True)
pca = PCA().fit(X)
cum = np.cumsum(pca.explained_variance_ratio_)
for k in [2, 10, 20, 30, 40]:
    p = PCA(k).fit(X)
    rec = p.inverse_transform(p.transform(X))
    err = np.mean((X - rec) ** 2) / np.mean((X - X.mean(0)) ** 2)
    print(f"{k:2d} components: variance kept {cum[k - 1]:.3f}   relative reconstruction
error {err:.3f}")
print("components needed for 95% of the variance:", int(np.searchsorted(cum, 0.95) + 1))
```

Output:

```
 2 components: variance kept 0.285   relative reconstruction error 0.715
10 components: variance kept 0.738   relative reconstruction error 0.262
20 components: variance kept 0.894   relative reconstruction error 0.106
30 components: variance kept 0.959   relative reconstruction error 0.041
40 components: variance kept 0.988   relative reconstruction error 0.012
components needed for 95% of the variance: 29
```

Two components keep only 28.5% of the variance; 29 components are needed to keep 95%, and the relative reconstruction error is exactly 1 minus the variance kept (0.041 at 30 components, where 0.959 is kept). Common ways to choose k:

- A variance target, such as 90–99%. Scikit-learn accepts it directly: `PCA(0.95)`.
- An "elbow" in the plot of explained variance against component number.
- Best of all, when PCA feeds a model: treat k as a hyperparameter and choose it by cross-validation of the whole pipeline.

Reconstruction error has a useful side job: **anomaly detection**. Points that the main components can't reconstruct well don't follow the usual pattern.

Kernel PCA: principal components in a curved space

PCA can only find flat directions. When classes sit on concentric circles, no straight direction separates them. **Kernel PCA** applies the kernel trick (Lesson 15's "Going deeper") to PCA: it finds principal components in a high-dimensional feature space, computed entirely through the kernel matrix.

```
# Kernel PCA: a non-linear version that can "unroll" circles
Xc, yc = make_circles(n_samples=400, factor=0.3, noise=0.05, random_state=0)
for name, emb in [("ordinary PCA", PCA(1)), ("kernel PCA (RBF)", KernelPCA(1,
kernel="rbf", gamma=5))]:
    z = emb.fit_transform(Xc)
    acc = cross_val_score(LogisticRegression(), z, yc, cv=5).mean()
    print(f"{name:17}: a linear classifier on the 1 new feature scores {acc:.3f}")
```

Output:

```
ordinary PCA     : a linear classifier on the 1 new feature scores 0.450
kernel PCA (RBF) : a linear classifier on the 1 new feature scores 1.000
```

Ordinary PCA's single component is useless for separating the inner and outer circles (0.450, worse than guessing). With an RBF kernel, the first kernel principal component separates them perfectly (1.000): in the kernel's feature space, "distance from the centre" becomes a straight direction. Kernel PCA needs the $n \times n$ kernel matrix, so like kernel SVMs it doesn't scale to very large datasets, and its components are hard to interpret.

t-SNE and UMAP: maps for looking, not features

t-SNE builds a 2-D map that keeps *neighbourhoods*: points that are close in the original space should stay close on the map. It turns distances into probabilities of being neighbours (in the original space using a normal curve whose width is set by the **perplexity**, roughly "how many neighbours to care about"; on the map using a heavier-tailed t-distribution), then moves the map points to make the two sets of probabilities match. **UMAP** has a similar goal, is faster, and keeps more of the global layout.

```
from sklearn.manifold import TSNE
from sklearn.neighbors import KNeighborsClassifier

# Compare 2-D maps: how well do a point's map neighbours share its digit label?
maps = {"PCA, 2-D": PCA(2).fit_transform(X),
        "t-SNE, 2-D": TSNE(2, perplexity=30, random_state=0,
init="pca").fit_transform(X)}
for name, Z in maps.items():
    acc = cross_val_score(KNeighborsClassifier(5), Z, y, cv=5).mean()
    print(f"{name:10}: 5-nearest-neighbour accuracy in the map {acc:.3f}")
```

Output:

```
PCA, 2-D  : 5-nearest-neighbour accuracy in the map 0.603
t-SNE, 2-D: 5-nearest-neighbour accuracy in the map 0.976
```

How well does each 2-D map keep similar digits together? A point's 5 nearest map neighbours share its label 97.6% of the time for t-SNE, against 60.3% for PCA's flat 2-D projection. That's why t-SNE pictures of the digits show ten neat islands. But read such maps carefully:

- **Distances between clusters mean little.** t-SNE preserves local neighbourhoods, not global distances. Two islands far apart are not necessarily very different.
- **Cluster sizes mean little.** t-SNE expands dense clusters and shrinks sparse ones.
- **Perplexity and the random seed change the picture.** Try several values before trusting a pattern. Random noise can look clustered at low perplexity.
- **It's for looking, not for modelling.** t-SNE has no `transform` for new points, and its axes have no meaning. Use PCA (or UMAP, which can transform new data) when you need features for a model.

Practical PCA notes

- **Fit PCA inside the pipeline**, on training data only, so test data doesn't leak into the components (Lesson 5).
- **Standardise first** when features have different units; otherwise the component directions are dominated by the features with the biggest numbers.
- **Whitening** (`PCA(whiten=True)`) rescales each component to variance 1. It helps some models but also amplifies noise in the small components.
- **For huge or sparse data**, use `TruncatedSVD` (works on sparse matrices, like TF-IDF text, where it's called latent semantic analysis), `IncrementalPCA` (in batches) or randomized SVD.
- **Autoencoders** (Lesson 19) are a neural-network generalisation: an encoder squeezes the data through a small middle layer and a decoder reconstructs it. A linear autoencoder with squared error learns the same subspace as PCA.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How do you choose the number of principal components?**

The trap: "Always keep 95%."

Look at the cumulative explained-variance curve and the reconstruction error (they're the same information: error = variance dropped). A variance threshold (90–99%) or an elbow is a reasonable start. When PCA feeds a model, the best answer is to treat k as a hyperparameter and pick it by cross-validating the whole pipeline. For visualisation, 2 or 3 components, accepting that much variance is lost.

▼ **What are the main pitfalls of interpreting t-SNE plots?**

The trap: Reading cluster distances and sizes as meaningful.

t-SNE preserves local neighbourhoods, so between-cluster distances and cluster sizes are unreliable; results depend on perplexity and the random seed; noise can look clustered at low perplexity; and the axes mean nothing. Try several settings, colour by known labels, and confirm structure with other methods. It has no out-of-sample transform, so don't use it to create model features.

▼ **How can PCA be used for anomaly detection?**

The trap: Only mentioning dimensionality reduction.

Fit PCA on normal data and keep enough components to describe typical variation. For each new point, project and reconstruct it; the reconstruction error measures how badly it fits the usual pattern. A threshold on that error (chosen on validation data) flags anomalies. It works well when normal data really does lie near a lower-dimensional subspace.

Check your understanding: going deeper

1. PCA with k components keeps 93% of the variance. What is the relative reconstruction error?

a. 0.93

b. 0.07

c. It can't be known

2. Why can kernel PCA separate concentric circles when PCA can't?

a. It finds principal components in a kernel feature space, where the curved structure becomes straight

b. It uses more data

c. It standardises the features

3. In a t-SNE map, two clusters are far apart. You can conclude that...

a. They're very different in the original space

b. Not much: t-SNE preserves neighbourhoods, not distances between clusters

c. They have different sizes

4. What does t-SNE's perplexity roughly control?

a. The number of output dimensions

b. How many neighbours each point pays attention to

c. The learning rate

5. Which method should you use to create features for a downstream model?

a. t-SNE

b. PCA (inside a pipeline)

c. Neither

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Covariance matrix $\begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$. Find the eigenvalues and the share of variance on PC1.

WORKING

EXERCISE 2 By hand

Eigenvalues $8, 4, 2, 1, 1$. What share do the first two explain? How many components are needed for at least 90%?

WORKING

EXERCISE 3 By hand

A centred point $(3, 1)$ is projected onto PC1 $= \frac{1}{\sqrt{2}}(1, 1)$. What is its new coordinate?

WORKING

EXERCISE 4 Code

Standardise the wine dataset (13 features) and fit `PCA()`. How many components are needed for 90% of the variance?

WORKING

EXERCISE 5 Intermediate

Eigenvalues of a covariance matrix are $5, 3, 1.5, 0.3, 0.2$. What fraction of the variance do 2 components keep, and how many are needed for at least 95%?

WORKING

EXERCISE 6 Intermediate

With the eigenvalues above, what is the relative reconstruction error with 3 components?

WORKING

EXERCISE 7 Code · intermediate

Using the digits code, keep enough components for 90% of the variance with `PCA(0.90)`. How many components does it choose?

WORKING

NOTES

PART 5 · NEURAL NETWORKS & REAL PROJECTS · ~70 MIN

Lesson 18: Neural Networks from Scratch

Neurons, layers, activations and backpropagation — built by hand in NumPy.

BY THE END OF THIS LESSON YOU CAN

- Explain neurons, layers and why activation functions are needed
- Run a forward pass through a small network by hand
- Run backpropagation by hand and update the weights
- Implement and train a neural network from scratch in NumPy

↺ BEFORE YOU START

This lesson builds on:

- Lesson 3: Derivatives & Gradients — A derivative is a slope; the gradient ∇L collects all partial derivatives and points uphill. Gradient descent steps the other way: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$. The chain rule differentiates functions inside functions.
- Lesson 7: Gradient Descent — Gradient descent trains almost any model: compute the gradient of the loss, step against it, repeat. The learning rate controls the step size; too large diverges. Mini-batches make it fast on big data.
- Lesson 8: Logistic Regression — Logistic regression outputs a probability $p = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, is trained with log loss, and classifies with a threshold (often 0.5), giving a straight decision boundary.

Where this fits: Starts Part 5. Neural networks combine everything so far — dot products, activations like the sigmoid, losses, the chain rule and gradient descent.

1. Why neural networks?

Every model so far has a fixed shape: a straight line, a set of boxes, a kernel. **Neural networks** learn their own internal features. Stack enough simple units together and the network can represent extremely complicated relationships — recognising faces, translating languages, generating text. Modern AI is built on them.

The good news: you already know every ingredient. A neuron is logistic regression (Lesson 8). Training is gradient descent (Lesson 7). And the famous **backpropagation** algorithm is the chain rule (Lesson 3), applied carefully. In this lesson you'll compute a full forward and backward pass by hand, then write a working network in about 20 lines of NumPy.

2. The artificial neuron

PICTURE IT Deciding whether to go for a run, you weigh the weather, how tired you are and whether a friend is coming, then decide yes or no. A neuron does the same: weigh the inputs, add them up, and squash the total into a decision.

A neuron takes inputs $x_1, \ldots, x_d$, multiplies each by a weight, adds a bias, and passes the result through an **activation function** g:

$$z = \mathbf{w} \cdot \mathbf{x} + b, \qquad a = g(z)$$

With $g =$ sigmoid, a single neuron *is* logistic regression. It draws one straight boundary. To learn more complex shapes, we connect many neurons in layers.

◎ LOOSELY INSPIRED BY THE BRAIN

A biological neuron receives signals from other neurons, and "fires" if the combined signal is strong enough. The artificial version is a very simplified mathematical cartoon of that: weighted inputs, summed, then squashed. Don't take the analogy too far — what matters is the maths.

READ THE FORMULA **One neuron**

$$a = g\big(\mathbf{w} \cdot \mathbf{x} + b\big)$$

- **a** — the neuron's output (activation)
- **g** — activation function, e.g. ReLU — adds the non-linearity
- **w · x** — weighted sum of the inputs
- **b** — bias — shifts the threshold

Hover over (or tap) a part to highlight it in the formula.

3. Why one neuron isn't enough: XOR

The XOR ("exclusive or") function outputs 1 when exactly one input is 1:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Plot the four points: the 1s sit on one diagonal, the 0s on the other. No single straight line separates them, so no single neuron (and no logistic regression) can learn XOR. But two neurons can each draw one line, and a third neuron can combine their answers — "above line A *and* below line B". That's the power of **hidden layers**: they create new features that make the problem easy for the next layer.

4. Layers: the multilayer perceptron

A **multilayer perceptron (MLP)**, or feed-forward network, has:

- an **input layer** — the features (no computation);
- one or more **hidden layers** — neurons whose outputs feed the next layer;
- an **output layer** — the prediction.

Each layer does a matrix multiplication (Lesson 2) followed by an activation:

$$\mathbf{h} = g\big(W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}\big), \qquad \hat{y} = \sigma\big(\mathbf{v}\cdot\mathbf{h} + c\big)$$

Here $W^{[1]}$ is a matrix with one row of weights per hidden neuron. With d inputs and m hidden neurons it has shape $m \times d$, plus m biases.

> **! WHY ACTIVATION FUNCTIONS ARE ESSENTIAL**
>
> Without g, two layers collapse into one: $V(W\mathbf{x}) = (VW)\mathbf{x}$, still a linear model. The non-linear activation is what lets depth add power.

5. Activation functions

Name	Formula	Range	Typical use
ReLU	$\max(0, z)$	$[0, \infty)$	Default for hidden layers
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Output for binary classification
tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	Hidden layers (zero-centred)
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	probabilities summing to 1	Output for multi-class

Their derivatives, needed for training: ReLU′ is 1 if $z > 0$ else 0; $\sigma'(z) = \sigma(z)(1 - \sigma(z))$; $\tanh'(z) = 1 - \tanh^2(z)$. ReLU is popular because it's cheap and its gradient doesn't shrink for positive inputs, which helps deep networks train (Lesson 19).

6. Worked example: a forward pass

✎ WORKED EXAMPLE — A 2-2-1 NETWORK

Inputs $\mathbf{x} = (1, 0)$, true label $y = 1$. Two sigmoid hidden neurons, one sigmoid output, all biases 0.

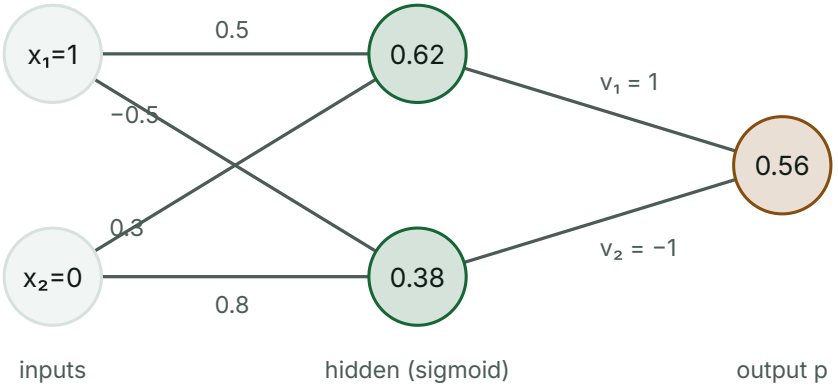


Figure 18. The 2-2-1 network from the worked example, with its weights and the values from the forward pass.

Hidden weights: neuron 1 has $(0.5,\ 0.3)$, neuron 2 has $(-0.5,\ 0.8)$. Output weights $\mathbf{v} = (1,\ -1)$.

Hidden layer:
$z_1 = 0.5 \cdot 1 + 0.3 \cdot 0 = 0.5 \Rightarrow h_1 = \sigma(0.5) \approx 0.6225$
$z_2 = -0.5 \cdot 1 + 0.8 \cdot 0 = -0.5 \Rightarrow h_2 = \sigma(-0.5) \approx 0.3775$

Output: $z = 1 \cdot 0.6225 - 1 \cdot 0.3775 = 0.2449 \Rightarrow p = \sigma(0.2449) \approx 0.5609$.

Loss (log loss, Lesson 8): $L = -\ln 0.5609 \approx 0.578$. The network gives only a 56% chance to the correct answer. Let's teach it.

? MAKE SENSE OF IT

The situation: A tiny network decides whether to approve something from two yes/no signals. Each hidden neuron is a small feature detector; the output neuron weighs their opinions.

What the result tells you: Neuron 1 likes input 1 (output 0.62); neuron 2 dislikes it (0.38). The output adds them with weights +1 and −1, giving 0.56 — barely leaning towards "yes".

What if…?

- **Other input** $\mathbf{x} = (0, 1)$: the hidden outputs become 0.57 and 0.69, and p drops to **0.47** — the network now leans "no" (loss 0.75 if the truth is 1).
- **ReLU hidden neurons** instead of sigmoid: $h = (0.5, 0)$ and $p = 0.62$ — neuron 2 is switched off entirely for this input.
- **All weights 0:** both hidden neurons output exactly 0.5 and $p = 0.5$. Worse, both neurons get identical gradients forever, so they never become different — which is why weights start random.

Why it matters in ML: A forward pass is just repeated dot products (Lesson 2) followed by a non-linear function. Stacking layers lets the network build complex features out of simple ones.

7. Backpropagation: the chain rule, backwards

To update every weight we need $\partial L / \partial(\text{weight})$. The loss depends on the weights through a chain: weight → hidden z → hidden h → output z → p → L. The chain rule multiplies the local derivatives along this path. **Backpropagation** organises it efficiently: compute an "error signal" δ at the output, then pass it backwards layer by layer, reusing each result.

1. **Output error.** For sigmoid + log loss, Lesson 8 showed the derivative w.r.t. the output's z simplifies to $\delta_{\text{out}} = p - y$.
2. **Output weights.** $\frac{\partial L}{\partial v_j} = \delta_{\text{out}} \cdot h_j$ and $\frac{\partial L}{\partial c} = \delta_{\text{out}}$.
3. **Pass the error back.** Each hidden neuron gets the output error times the weight connecting it, times its own activation derivative: $\delta_j = \delta_{\text{out}} \cdot v_j \cdot g'(z_j)$.
4. **Hidden weights.** $\frac{\partial L}{\partial W_{jk}} = \delta_j \cdot x_k$ and $\frac{\partial L}{\partial b_j} = \delta_j$.
5. **Update** all weights with gradient descent: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.

Every gradient has the same pattern: **(error signal arriving at a neuron) × (input that came into it)**.

✎ WORKED EXAMPLE — BACKWARD PASS AND ONE UPDATE

Continuing from the forward pass ($p = 0.5609$, $y = 1$, $h = (0.6225, 0.3775)$).

1. Output error: $\delta_{\text{out}} = 0.5609 - 1 = -0.4391$.

2. Output weights: $\frac{\partial L}{\partial v_1} = -0.4391 \times 0.6225 \approx -0.2733$, $\frac{\partial L}{\partial v_2} = -0.4391 \times 0.3775 \approx -0.1658$, $\frac{\partial L}{\partial c} = -0.4391$.

3. Hidden errors (sigmoid derivative $h(1-h) = 0.6225 \times 0.3775 \approx 0.2350$ for both):
$\delta_1 = -0.4391 \times (1) \times 0.2350 \approx -0.1032$
$\delta_2 = -0.4391 \times (-1) \times 0.2350 \approx +0.1032$

4. Hidden weights: multiply by the inputs $(1, 0)$: neuron 1's gradients are $(-0.1032,\ 0)$, neuron 2's are $(0.1032,\ 0)$. (Weights from $x_2 = 0$ get zero gradient — that input contributed nothing.)

5. Update with $\eta = 1$:
$v = (1 + 0.2733,\ -1 + 0.1658) = (1.2733,\ -0.8342)$, $c = 0.4391$
neuron 1: $(0.5 + 0.1032,\ 0.3) = (0.6032,\ 0.3)$; neuron 2: $(-0.5 - 0.1032,\ 0.8) = (-0.6032,\ 0.8)$

Check: running the forward pass again gives $p \approx 0.7245$ and loss ≈ 0.322 (down from 0.578). One step of backpropagation made the network noticeably more confident in the right answer.

? MAKE SENSE OF IT

The situation: The network gave 56% to the right answer. Backpropagation works out how much each weight was to blame, and nudges each one.

What the result tells you: The output error −0.44 says "too low". It flows back through each weight: weights that pushed the wrong way get the biggest corrections. After one step the network gives 72%.

What if…?

- **Learning rate 0.1:** p rises only to **0.579** (loss 0.547) — a small, safe step.
- **Learning rate 5:** p jumps to **0.98** in one step — great on this one example, but steps this big usually make training on many examples unstable.
- **If the truth were 0:** every gradient flips sign; one step ($\eta = 1$) lowers p to 0.34 and the loss from 0.82 to 0.41.

Why it matters in ML: Every deep-learning framework does exactly this, automatically, for millions of weights: forward pass, loss, backward pass (the chain rule, Lesson 3), update (gradient descent, Lesson 7).

WHY IT'S EFFICIENT

Backpropagation computes the gradients for *all* weights in roughly the same time as one forward pass, by reusing the δ values. That's what makes training networks with millions or billions of weights possible. It was popularised for neural networks by Rumelhart, Hinton and Williams in 1986.

STEP-BY-STEP DERIVATION — Derive the gradient for one hidden weight

1 $$\frac{\partial L}{\partial W_{jk}} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial W_{jk}}$$

Chain rule along the path $W_{jk} \to z_j \to h_j \to z \to L$.

2 $$\frac{\partial L}{\partial z} = p - y$$

Sigmoid output with log loss (Lesson 8).

3 $$\frac{\partial z}{\partial h_j} = v_j$$

Because $z = \sum_j v_j h_j + c$.

4 $$\frac{\partial h_j}{\partial z_j} = g'(z_j)$$

The hidden activation's derivative.

5 $$\frac{\partial z_j}{\partial W_{jk}} = x_k$$

Because $z_j = \sum_k W_{jk} x_k + b_j$.

6 $$\frac{\partial L}{\partial W_{jk}} = \underbrace{(p - y)\, v_j\, g'(z_j)}_{\delta_j} \cdot x_k$$

Multiply: error signal × input — exactly the backprop rule.

ADVANCED · OPTIONAL — The softmax + cross-entropy gradient

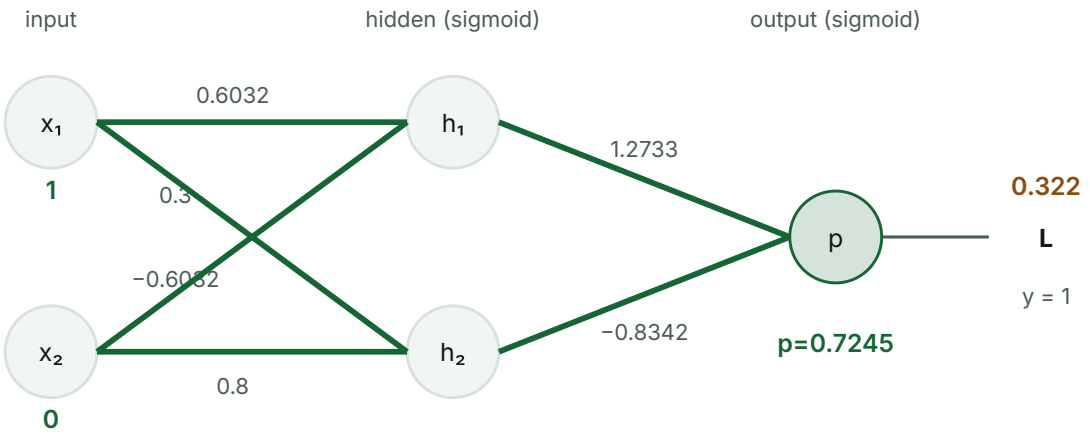
▾ hide

With K classes, the output layer computes $p_k = e^{z_k} / \sum_j e^{z_j}$ and the loss for a one-hot label $\mathbf{y}$ is $\ell = -\sum_k y_k \ln p_k$. The softmax derivative is $\partial p_k / \partial z_j = p_k(1[k = j] - p_j)$. Therefore

$$\frac{\partial \ell}{\partial z_j} = -\sum_k \frac{y_k}{p_k} p_k(1[k=j] - p_j) = -y_j + p_j \sum_k y_k = p_j - y_j.$$

Exactly the same "prediction minus target" error signal as the sigmoid + log loss in this lesson — which is why softmax and cross-entropy are always paired, and why frameworks fuse them into one numerically stable operation.

WALKTHROUGH (FINAL STEP SHOWN) **Forward pass, backward pass, update**



1. The 2-2-1 network from the worked example, with its starting weights on the edges. Input $\mathbf{x} = (1, 0)$, true label $y = 1$, all biases 0.
2. **Forward 1:** each hidden neuron computes $z = \mathbf{w} \cdot \mathbf{x}$ and squashes it: $h_1 = \sigma(0.5) = 0.6225$, $h_2 = \sigma(-0.5) = 0.3775$.
3. **Forward 2:** the output neuron combines them: $z = 1 \cdot 0.6225 - 1 \cdot 0.3775 = 0.2449$, so $p = \sigma(z) = 0.5609$.
4. **Loss:** $L = -\ln p = 0.578$. The network gives only a 56% chance to the right answer. Now the backward pass works out who to blame.
5. **Backward 1:** the error signal at the output is $\delta_{out} = p - y = -0.4391$ (negative: p should go *up*).
6. **Backward 2:** each output weight's gradient is (error arriving) × (input it carried): $\partial L/\partial v_1 = -0.4391 \times 0.6225 = -0.2733$, $\partial L/\partial v_2 = -0.1658$.
7. **Backward 3:** the error flows back through each output weight and the sigmoid's slope $h(1-h) = 0.2350$: $\delta_1 = -0.4391 \times 1 \times 0.2350 = -0.1032$, $\delta_2 = 0.1032$.
8. **Backward 4:** hidden-weight gradients are $\delta_j \times x_k$. Because $x_2 = 0$, the weights leaving x_2 get zero gradient — that input contributed nothing.
9. **Update** ($\eta = 1$): every weight moves against its gradient. $v = (1.2733, -0.8342)$, $c = 0.4391$, $w_{11} = 0.6032$, $w_{21} = -0.6032$.
10. **Forward again** with the new weights: $p = 0.7245$ and the loss drops from 0.578 to 0.322. Training repeats this loop thousands of times.

8. The same thing with matrices

For a batch of n examples (rows of X), with tanh hidden units and a sigmoid output:

$$H = \tanh(XW_1 + \mathbf{b}_1), \qquad P = \sigma(HW_2 + b_2)$$

Backward:

$$\Delta_2 = \tfrac{1}{n}(P - Y), \quad \frac{\partial L}{\partial W_2} = H^\top \Delta_2, \quad \Delta_1 = (\Delta_2 W_2^\top) \odot (1 - H^2), \quad \frac{\partial L}{\partial W_1} = X^\top \Delta_1$$

($\odot$ means element-wise multiplication; $1 - H^2$ is tanh's derivative.) These four lines are exactly what the NumPy code below does.

9. Try it: train a neural network

A small network with one hidden layer of tanh neurons learns to separate two classes. Pick a dataset — XOR, circle or spiral — and a number of hidden neurons, then press *Train*. The background shows the network's predicted probability. With 1 hidden neuron it can only draw a straight line; add neurons and watch it bend the boundary. The spiral needs the most neurons — and the most training: press *Train* again if it hasn't finished.

Interactive demo — open *lesson-18.html* on the website to try it.

10. Training in practice

- **Initialisation:** start weights as small random numbers, never all equal. If every weight is the same, every hidden neuron computes the same thing and gets the same gradient — they'd never become different.
- **Scale inputs** (standardise), as with any gradient-descent model.
- **Mini-batches and epochs** (Lesson 7): update on small random batches; one epoch is one pass over the data.
- **Watch the validation loss** and stop when it starts rising (early stopping); use L2 regularization (Lesson 10).
- **Size:** more neurons and layers give more capacity — and more risk of overfitting.

> **≡ UNIVERSAL APPROXIMATION**
>
> A famous result says that a network with just one hidden layer and enough neurons can approximate any reasonable continuous function as closely as you like. It doesn't say how many neurons you'd need, or that training will find the right weights — in practice, deeper networks with fewer neurons per layer usually learn much more efficiently.

11. In Python: a network from scratch

A 2-4-1 network learning XOR with NumPy — every line corresponds to an equation above:

```
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
Y = np.array([[0], [1], [1], [0]], dtype=float)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

rng = np.random.default_rng(1)
W1 = rng.normal(0, 1, (2, 4)); b1 = np.zeros(4)     # input → 4 hidden neurons
W2 = rng.normal(0, 1, (4, 1)); b2 = np.zeros(1)     # hidden → output
lr = 0.5

for step in range(5001):
    # forward pass
    H = np.tanh(X @ W1 + b1)
    P = sigmoid(H @ W2 + b2)
    loss = -np.mean(Y * np.log(P) + (1 - Y) * np.log(1 - P))
    # backward pass (backpropagation)
    dZ2 = (P - Y) / len(X)
    dW2 = H.T @ dZ2;  db2 = dZ2.sum(axis=0)
    dZ1 = (dZ2 @ W2.T) * (1 - H ** 2)          # tanh'(z) = 1 - tanh(z)^2
    dW1 = X.T @ dZ1;  db1 = dZ1.sum(axis=0)
    # gradient descent update
    W1 -= lr * dW1;  b1 -= lr * db1
    W2 -= lr * dW2;  b2 -= lr * db2
    if step in (0, 1000, 5000):
        print(step, round(loss, 4))

print(P.round(3).ravel())
```

Output:

```
0 0.6712
1000 0.0051
5000 0.0009
[0.    0.999 0.999 0.002]
```

The loss falls from 0.67 (guessing) to almost 0, and the predictions for $(0,0), (0,1), (1,0), (1,1)$ are essentially $0, 1, 1, 0$ — XOR, solved by a hidden layer.

With scikit-learn, on the 8×8 handwritten digits (64 inputs, 10 classes):

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y,
random_state=0)

mlp = make_pipeline(StandardScaler(),
                    MLPClassifier(hidden_layer_sizes=(64,), max_iter=500,
random_state=0))
mlp.fit(X_train, y_train)
print(round(mlp.score(X_train, y_train), 3), round(mlp.score(X_test, y_test), 3))
```

Output: `1.0 0.973`. One hidden layer of 64 ReLU neurons (scikit-learn's default activation) — $64 \times 64 + 64 + 64 \times 10 + 10 = 4{,}810$ parameters — reaches 97.3% on unseen digits, a little above logistic regression on the same split (96.9%). The 100% training score hints at mild overfitting; increasing `alpha` (L2) or using `early_stopping=True` are the usual remedies.

12. Running example: Chennai Homes

A tiny neural network (one hidden layer of 4 neurons) learns to predict fast sales from size, distance and age, getting every house right even in leave-one-out evaluation. On such small, simple data, though, the decision tree's rule works just as well and is far easier to explain.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score, LeaveOneOut
cols = ["size_m2", "dist_km", "age_yrs"]
mlp = make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(4,),
max_iter=5000, random_state=0))
print("training accuracy:", mlp.fit(df[cols], df["sold_fast"]).score(df[cols],
df["sold_fast"]))
print("leave-one-out accuracy:", round(cross_val_score(mlp, df[cols],
df["sold_fast"], cv=LeaveOneOut()).mean(), 2))
```

Output:

```
training accuracy: 1.0
leave-one-out accuracy: 1.0
```

⚙ WHERE THIS IS USED

- Handwriting and image recognition
- The foundation of every modern deep learning system

Choose it when: Large datasets with complex patterns, especially images, audio and text.

Think twice when: Small tabular datasets — gradient boosting or logistic regression are usually better and simpler.

✕ MISCONCEPTIONS TO UNLEARN

MYTH Neural networks work like the human brain.

REALITY They're loosely inspired by neurons, but they are mathematical functions trained by gradient descent.

MYTH Deeper networks are always better.

REALITY More depth needs more data and careful training; on small tabular data, simple models often win.

RESPONSIBLE AI · FRAGILE AND OPAQUE

Neural networks can be fooled by tiny, deliberately chosen changes to their input: Szegedy et al. (2014) and Goodfellow et al. (2015) showed images altered imperceptibly to humans that networks misclassify with high confidence. Their reasoning is also hard to inspect. For safety-critical uses, test robustness, monitor behaviour after deployment, and don't treat confidence as correctness.

13. Questions beginners often ask

Q. How many layers and neurons should I use?

Start small (one or two hidden layers), check for underfitting or overfitting, and grow or regularise from there.

Q. Why ReLU instead of sigmoid in hidden layers?

ReLU is cheap and its gradient doesn't shrink for positive inputs, so deep networks train much better.

Q. Is a neural network always better?

No. On small tabular datasets, gradient boosting or logistic regression is often more accurate and far simpler.

! COMMON MISTAKES TO AVOID

- Initialising all weights to the same value (usually zero).
- Forgetting non-linear activations between layers.
- Not scaling inputs, which slows or stalls training.

! INTERVIEW CORNER

Don't confuse

- **Activation function vs loss function** — Inside the network vs at the end.
- **Forward pass vs backward pass** — Predict, then assign blame.
- **Dropout vs batch normalisation** — Random silence vs steady scaling.

Hidden questions — answer in your head first, then open.

▼ **Why does a neural network need non-linear activation functions?**

The trap: "To make outputs between 0 and 1."

Without them, stacked layers collapse into a single linear layer: $W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$. However deep, the network could only draw straight boundaries. The non-linearity is what lets depth build complex functions.

▼ **What happens if you initialise all weights to zero?**

The trap: "Training just starts slowly."

Every neuron in a layer computes the same output and receives the same gradient, so they stay identical forever — the layer behaves like a single neuron. Random initialisation breaks this symmetry. (Biases can start at zero.)

▼ **Why did sigmoid hidden layers make deep networks hard to train?**

The trap: Blaming the data.

The sigmoid's slope is at most 0.25, and nearly 0 for large inputs. Backpropagation multiplies one such factor per layer, so gradients shrink exponentially in early layers: the vanishing-gradient problem. ReLU (slope 1 for positive inputs), careful initialisation, batch normalisation and residual connections fixed it.

▼ **What is a "dying ReLU"?**

The trap: Thinking ReLU has no downsides.

If a neuron's input becomes negative for every example (often after a large update), it outputs 0 and gets zero gradient, so it never recovers. Lower learning rates, careful initialisation or Leaky ReLU help.

14. Key takeaways

- A neuron computes $g(\mathbf{w} \cdot \mathbf{x} + b)$; layers of neurons are matrix multiplications plus activations.
- **Non-linear activations** (ReLU, tanh, sigmoid) are what make hidden layers useful — they let networks solve problems like XOR.
- **Backpropagation** is the chain rule: compute $\delta_{\text{out}} = p - y$, pass errors backwards ($\delta_j = \delta_{\text{out}} v_j g'(z_j)$), and each gradient = error × input.
- Training = forward pass, loss, backward pass, gradient-descent update, repeated over mini-batches.
- Initialise randomly, scale inputs, and watch validation loss to avoid overfitting.

15. Check your understanding

1. Why can't a single neuron learn XOR?

a. XOR needs more than two inputs

b. A single neuron draws one straight boundary, and XOR's classes aren't linearly separable

c. Sigmoid can't output 0

2. What happens if you remove all activation functions from a deep network?

a. It becomes equivalent to a single linear model

b. It trains faster and is more accurate

c. It can only output 0 or 1

3. What is ReLU(−3)?

a. −3

b. 3

c. 0

4. Output $p = 0.8$, label $y = 0$, hidden activation $h_1 = 0.5$. What is $\partial L / \partial v_1$ (sigmoid + log loss)?

a. −0.4

b. 0.4

c. 0.8

5. Why must weights be initialised randomly rather than all zeros?

a. Identical weights give identical neurons with identical gradients, so they never learn different features

b. Zero weights cause division by zero

c. Random weights guarantee the global minimum

16. Going deeper: making deep networks trainable

INTERMEDIATE · SKIP ON A FIRST READ

A network with one hidden layer, as in this lesson, trains easily. Stack 20 layers and it often won't train at all. This section shows why, with measurements: signals that shrink or explode as they pass through layers, gradients that vanish on the way back, and the standard fixes that made deep learning work: **careful initialisation**, **ReLU**, **batch normalisation** and **dropout**.

You'll need: backpropagation from this lesson, and variance from Lesson 4.

Initialisation: keeping the signal alive

Each layer computes $\mathbf{h}_{\text{new}} = \sigma(W\mathbf{h})$. If each of the n inputs to a unit has variance 1 and each weight has variance s^2, the sum $W\mathbf{h}$ has variance about ns^2. If ns^2 is bigger than 1 the signal grows layer after layer; if smaller, it shrinks. Two classic choices keep it steady:

- **Xavier (Glorot) initialisation**, weight variance $1/n$, for tanh-like activations.
- **He initialisation**, weight variance $2/n$, for ReLU, which sets half its inputs to zero and so needs twice the variance.

Here a batch passes through 20 layers of width 256 with different setups:

```
import numpy as np

# Push a batch through 20 layers of width 256 and watch the activations' spread
rng = np.random.default_rng(0)
x0 = rng.normal(size=(500, 256))
relu = lambda z: np.maximum(z, 0)
setups = {"tanh, weights std 1":          (np.tanh, 1.0),
          "tanh, weights std 0.01":       (np.tanh, 0.01),
          "tanh, Xavier std 1/sqrt(n)":   (np.tanh, 1 / np.sqrt(256)),
          "ReLU, Xavier std 1/sqrt(n)":   (relu, 1 / np.sqrt(256)),
          "ReLU, He std sqrt(2/n)":       (relu, np.sqrt(2 / 256))}
for name, (act, std) in setups.items():
    h = x0; spreads = []
    for layer in range(20):
        h = act(h @ rng.normal(0, std, size=(256, 256)))
        spreads.append(h.std())
    print(f"{name:27}: activation std after layers 1, 10, 20: {spreads[0]:.3f},
{spreads[9]:.3g}, {spreads[19]:.3g}")

# With weights of std 1, tanh is saturated: almost every unit sits near -1 or +1, where
its slope is ~0
h = x0
for _ in range(20): h = np.tanh(h @ rng.normal(0, 1.0, size=(256, 256)))
print("tanh with std 1: share of units with |activation| > 0.99:",
round(np.mean(np.abs(h) > 0.99), 3))
```

Output:

```
tanh, weights std 1        : activation std after layers 1, 10, 20: 0.975, 0.974, 0.974
tanh, weights std 0.01     : activation std after layers 1, 10, 20: 0.157, 1.06e-08,
1.12e-16
tanh, Xavier std 1/sqrt(n) : activation std after layers 1, 10, 20: 0.627, 0.221, 0.163
ReLU, Xavier std 1/sqrt(n) : activation std after layers 1, 10, 20: 0.581, 0.0212,
0.000893
ReLU, He std sqrt(2/n)     : activation std after layers 1, 10, 20: 0.829, 0.665, 0.674
tanh with std 1: share of units with |activation| > 0.99: 0.866
```

Weights that are too small (std 0.01) make the signal vanish: its spread falls to 10^{-16} by layer 20, so every input produces the same output. Weights that are too big (std 1) push tanh into saturation: the spread looks healthy (0.974), but 86.6% of units sit beyond ± 0.99, where tanh's slope is almost 0, so almost no gradient flows back. Xavier keeps tanh in a usable range. Xavier with ReLU slowly dies away (0.000893 by layer 20), while He initialisation keeps ReLU steady (0.674). Frameworks build in scaled defaults like these (PyTorch's layers scale their random starting weights by the number of inputs), so you rarely set them by hand, but they explain why defaults matter, and why you may want `torch.nn.init.kaiming_normal_` for a very deep ReLU network.

Vanishing gradients, measured

Backpropagation multiplies by each layer's activation slope on the way back. The sigmoid's slope is at most 0.25, so over 20 layers the gradient can shrink by up to $0.25^{20} \approx 10^{-12}$. ReLU's slope is exactly 1 wherever a unit is active:

```
# Backpropagate through 20 sigmoid layers vs 20 ReLU layers: how big is the gradient at
the first layer?
def grad_norms(act, dact, std):
    h, cache, Ws = x0[:100], [], []
    for _ in range(20):
        W = rng.normal(0, std, size=(256, 256)); z = h @ W
        cache.append(z); Ws.append(W); h = act(z)
    g = np.ones_like(h)                                  # pretend dLoss/dOutput is all
ones
    norms = []
    for z, W in zip(reversed(cache), reversed(Ws)):
        g = (g * dact(z)) @ W.T                          # chain rule, one layer back
        norms.append(np.linalg.norm(g))
    return norms[0], norms[-1]

sig = lambda z: 1 / (1 + np.exp(-z))
for name, act, dact, std in [("sigmoid", sig, lambda z: sig(z) * (1 - sig(z)), 1 /
np.sqrt(256)),
                             ("ReLU", relu, lambda z: (z > 0).astype(float), np.sqrt(2 /
256))]:
    last, first = grad_norms(act, dact, std)
    print(f"{name:7}: gradient size at the last layer {last:.3g}, at the first layer
{first:.3g}")
```

Output:

```
sigmoid: gradient size at the last layer 40.4, at the first layer 3.5e-11
ReLU   : gradient size at the last layer 186, at the first layer 658
```

With sigmoid, the gradient reaching the first layer is 4×10^{-11}, about a trillionth of its size at the last layer, so the early layers essentially never learn. With ReLU and He initialisation, it stays the same order of magnitude (197 at the end, 524 at the start). That single change, together with good initialisation, is a big part of why deep networks became trainable around 2010–2012. The other big fixes are below, and **residual connections** (Lesson 19), which add a shortcut $\mathbf{h} + f(\mathbf{h})$ so gradients can skip layers.

Batch normalisation

Batch normalisation standardises each unit's values across the current mini-batch (mean 0, variance 1), then rescales them with two learned numbers per unit, γ and β:

$$\hat{z} = \frac{z - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}, \qquad \text{output} = \gamma\,\hat{z} + \beta$$

It keeps every layer's inputs in a steady range however the earlier weights change, which allows larger learning rates and makes training much less sensitive to initialisation. At prediction time it uses running averages of μ and σ^2 collected during training, not the current batch. **Layer normalisation** standardises across the units of a single example instead; it doesn't depend on the batch, and it's the standard choice in transformers.

Dropout

Dropout randomly switches off each hidden unit with probability p at every training step, so the network can't rely on any single unit and has to spread knowledge out. It works like training a huge ensemble of thinned networks that share weights, and averaging them at the end. Modern implementations use **inverted dropout**: scale the surviving units by $1/(1-p)$ during training, so nothing needs to change at prediction time, when dropout is switched off.

```
# Inverted dropout: drop units with probability p during training and scale the rest by
1/(1-p)
p = 0.5
h = np.ones(100000)
mask = rng.random(h.shape) > p
train_out = h * mask / (1 - p)
print("share of units kept:", round(mask.mean(), 3), "  average activation, training:",
round(train_out.mean(), 3), "  at prediction time (no dropout):", h.mean())

# Batch normalisation: standardise each feature over the batch, then rescale with
learned gamma and beta
z = rng.normal(5, 3, size=(64, 4))
gamma, beta = np.array([1.0, 2.0, 1.0, 0.5]), np.array([0.0, 1.0, -1.0, 0.0])
bn = gamma * (z - z.mean(0)) / np.sqrt(z.var(0) + 1e-5) + beta
print("after batch norm, per-feature mean:", (bn.mean(0).round(3) + 0.0), " std:",
bn.std(0).round(3))
```

Output:

```
share of units kept: 0.499   average activation, training: 0.997   at prediction time
(no dropout): 1.0
after batch norm, per-feature mean: [ 0.  1. -1.  0.]  std: [1.  2.  1.  0.5]
```

With $p = 0.5$, about half the units survive (0.498), and thanks to the $1/(1-p)$ scaling the average activation stays about 1 (0.995), matching prediction time exactly. The batch-norm example shows each feature leaving with exactly the mean β and spread γ chosen for it. In PyTorch, `model.train()` and `model.eval()` switch both dropout and batch norm between their training and prediction behaviour. Forgetting `model.eval()` before predicting is a classic bug.

A training recipe for a new network

Choice	Sensible default
Activation	ReLU (or GELU in transformers)
Initialisation	The framework default (He or Xavier variants)
Normalisation	Batch norm for CNNs; layer norm for transformers
Optimiser	AdamW, learning rate about 10^{-3}, warm-up then decay (Lesson 7's "Going deeper")
Regularization	Weight decay, dropout 0.1–0.5, early stopping, data augmentation
First check	Overfit a tiny batch (say 10 examples) to near-zero loss. If you can't, there's a bug.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Explain vanishing and exploding gradients and three ways to fix them.**

The trap: Naming the problem without the mechanism.

Backprop multiplies, layer by layer, by each layer's weights and activation slopes. If those factors are typically below 1 the gradient shrinks exponentially with depth (vanishing); if above 1 it grows (exploding). Fixes: ReLU-family activations (slope 1 when active); variance-preserving initialisation (Xavier for tanh, He for ReLU); normalisation layers (batch or layer norm); residual connections that give gradients a shortcut; gradient clipping for explosions; and, for sequences, gated units (LSTM, GRU) or attention.

▼ **How does batch normalisation behave differently in training and inference, and what goes wrong if you forget?**

The trap: Not knowing there's a difference.

In training it standardises with the current mini-batch's mean and variance and updates running averages. In inference it uses those running averages, so predictions don't depend on the batch. Forgetting to switch to evaluation mode (`model.eval()` in PyTorch) makes predictions depend on batch composition and size, often badly for batch size 1. Very small training batches also make batch statistics noisy, which is one reason layer or group norm are used instead.

▼ **Why does dropout reduce overfitting, and why scale by $1/(1-p)$?**

The trap: "It removes neurons."

Randomly dropping units prevents co-adaptation (units relying on specific partners) and acts like training an exponential ensemble of thinned subnetworks that share weights; prediction with the full network approximates averaging them. Scaling survivors by $1/(1-p)$ during training keeps each unit's expected input the same as at test time, when nothing is dropped, so no adjustment is needed at prediction.

Check your understanding: going deeper

1. He initialisation uses weight variance $2/n$ rather than $1/n$ because...

a. ReLU zeroes about half its inputs, halving the variance passed on

b. It trains faster on GPUs

c. Tanh needs it

2. A deep sigmoid network's first layers barely learn. The most likely reason is...

a. Vanishing gradients: each sigmoid layer multiplies the gradient by at most 0.25

b. Too much data

c. The learning rate is too small for the last layer

3. At prediction time, batch normalisation uses...

a. The statistics of the current batch

b. Running averages of the mean and variance saved during training

c. No normalisation

4. With inverted dropout and $p = 0.2$, surviving units during training are multiplied by...

a. 0.8

b. 1.25

c. 5

5. Weights drawn with standard deviation 1 in a 256-wide tanh network mostly cause...

a. Saturation: units stuck near ±1 with almost zero slope

b. Activations that shrink to zero

c. Perfect training

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Inputs $\mathbf{x} = (2, 1)$. Hidden ReLU neurons with weights $(1, -1)$ and $(-1, 2)$, biases 0. Output sigmoid with weights $(2, 3)$, bias -1. Compute the output.

WORKING

EXERCISE 2 By hand

Continue exercise 1 with label $y = 0$. Compute $\partial L/\partial v_1$ and $\partial L/\partial v_2$ (sigmoid + log loss).

WORKING

EXERCISE 3 By hand

How many parameters has a network with 10 inputs, hidden layers of 32 and 16 neurons, and 3 outputs?

WORKING

EXERCISE 4 Code

In the XOR code, change the hidden size from 4 to 1 (`(2, 1)` and `(1, 1)`) and run it. Can it learn XOR? Why?

WORKING

EXERCISE 5 Intermediate

A layer has 512 inputs. What weight standard deviation does Xavier ($\mathrm{var} = 1/n$) give, and what does He ($\mathrm{var} = 2/n$) give?

WORKING

EXERCISE 6 Intermediate

Each layer of a 10-layer network multiplies the gradient by about 0.25. By what factor has it shrunk at the first layer? And if each factor is 1.3?

WORKING

EXERCISE 7 Intermediate

Batch norm receives values $(2, 4, 6, 8)$ for one unit, with $\gamma = 2$, $\beta = 1$ and $\epsilon \approx 0$. What are the outputs? (Use the population variance.)

WORKING

NOTES

PART 5 · NEURAL NETWORKS & REAL PROJECTS · ~65 MIN

Lesson 19: Deep Learning Architectures

Training deep nets, CNNs for images, RNNs for sequences, and Transformers.

BY THE END OF THIS LESSON YOU CAN

- Explain vanishing gradients and the techniques that fix them
- Compute a convolution, output sizes and max pooling by hand
- Describe how RNNs and LSTMs process sequences
- Compute scaled dot-product attention for a small example

↺ BEFORE YOU START

This lesson builds on:

- Lesson 10: Overfitting, Regularization & Bias–Variance — Error = bias² + variance + noise. Regularization adds a penalty on weight size (L2 ridge shrinks weights; L1 lasso zeros some) to reduce variance; its strength is chosen by cross-validation.
- Lesson 18: Neural Networks from Scratch — A neural network stacks layers of neurons $g(W\mathbf{x} + \mathbf{b})$. Backpropagation (the chain rule) gives every weight's gradient, and gradient descent updates them.

Where this fits: How the ideas of Lesson 18 scale up to the deep learning behind modern image recognition and language models.

1. What makes learning ”deep”?

Deep learning means neural networks with many layers. Each layer builds on the previous one: in an image network, early layers detect edges, middle layers combine edges into textures and shapes, and later layers recognise eyes, wheels or letters. The network learns this hierarchy of features by itself, from data.

Lesson 18's fully connected network treats every input independently. For images, sound and text that wastes structure we know is there — nearby pixels belong together; word order matters. Deep learning's big successes come from **architectures** built around that structure: **convolutional networks** for images, **recurrent networks** for sequences, and **Transformers**, which now power language models. This lesson explains how each works, plus the techniques that make deep networks trainable.

2. Making deep networks trainable

Vanishing gradients

Backpropagation multiplies derivatives layer by layer (Lesson 18). The sigmoid's derivative is at most 0.25. Through 10 sigmoid layers, a gradient can shrink by a factor of up to $0.25^{10} \approx 0.000001$ — the early layers barely learn. This **vanishing gradient** problem held deep learning back for years. Fixes:

- **ReLU activations**: derivative exactly 1 for positive inputs, so gradients pass through undiminished.
- **Careful initialisation**: scale random initial weights to the layer size (e.g. "He" or "Glorot" initialisation) so signals neither explode nor vanish.
- **Normalisation layers** (batch norm, layer norm): rescale each layer's activations to a stable range during training.
- **Residual (skip) connections**: compute $\mathbf{h} + f(\mathbf{h})$ instead of $f(\mathbf{h})$, giving gradients a shortcut path. These made networks with 100+ layers possible.

Better optimisers

Momentum keeps a running average of past gradients, so steps build speed in consistent directions and damp oscillations — like a ball rolling downhill. **Adam** (Kingma & Ba, ICLR 2015) combines momentum with a per-parameter step size, adapted from running averages of the gradient g and the squared gradient:

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \qquad s \leftarrow \beta_2 s + (1-\beta_2) g^2, \qquad w \leftarrow w - \eta \, \frac{\hat{m}}{\sqrt{\hat{s}} + \epsilon}$$

($\hat{m}, \hat{s}$ are bias-corrected versions of m, s; typical values are $\beta_1 = 0.9$, $\beta_2 = 0.999$.) Parameters with consistently large gradients take smaller steps, and rarely-updated ones take larger steps. Adam is the default optimiser in most deep learning work.

Fighting overfitting

- **Dropout**: during training, randomly switch off a fraction of neurons (e.g. 20–50%) at each step. No neuron can rely on any specific other neuron, which acts like training an ensemble of many thinner networks (Lesson 14).
- **Data augmentation**: create extra training examples by flipping, cropping or slightly rotating images — the label stays the same.
- **Early stopping** and **weight decay** (L2), as before.

3. Convolutional neural networks (CNNs)

A 224×224 colour image has $224 \times 224 \times 3 = 150{,}528$ inputs. A fully connected layer of just 1,000 neurons would need 150 million weights — and it would have to learn "cat ear" separately for every position in the image. CNNs fix this with two ideas:

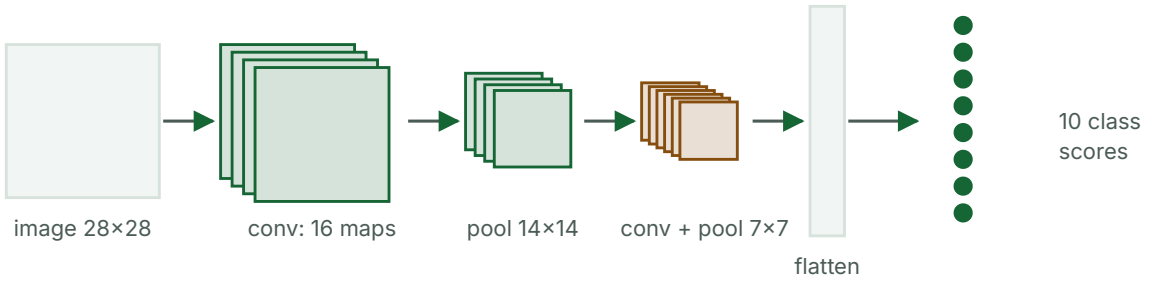


Figure 19. A typical small CNN: convolution and pooling layers shrink the image while increasing the number of feature maps, then a dense layer scores each class.

- **Local connections**: each neuron looks only at a small patch (e.g. 3×3 pixels).
- **Weight sharing**: the same small set of weights — a **filter** or **kernel** — is slid across the whole image. A feature detector learned in one place works everywhere.

The convolution operation

Slide the kernel over the image. At each position, multiply the overlapping numbers element by element and add them up; that sum is one pixel of the output, called a **feature map**.

$$\text{out}[i, j] = \sum_{u} \sum_{v} \text{image}[i+u,\ j+v] \cdot \text{kernel}[u, v]$$

✎ WORKED EXAMPLE — DETECTING A VERTICAL EDGE

A 4×6 image: dark (0) on the left, bright (9) on the right. A 3×3 vertical-edge kernel:

$$\text{image} = \begin{bmatrix} 0 & 0 & 0 & 9 & 9 & 9 \\ 0 & 0 & 0 & 9 & 9 & 9 \\ 0 & 0 & 0 & 9 & 9 & 9 \\ 0 & 0 & 0 & 9 & 9 & 9 \end{bmatrix} \qquad \text{kernel} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

The kernel computes (right column − left column) of each 3×3 window.

- Window over columns 1–3 (all 0s): 0.
- Window over columns 2–4: left column 0s, right column 9s: $3 \times 9 - 0 = 27$.
- Window over columns 3–5: left 0s, right 9s again: 27.
- Window over columns 4–6 (all 9s): $3 \times 9 - 3 \times 9 = 0$.

$$\text{feature map} = \begin{bmatrix} 0 & 27 & 27 & 0 \\ 0 & 27 & 27 & 0 \end{bmatrix}$$

The output "lights up" exactly where the edge is and is 0 in flat regions. A CNN *learns* its kernel values by backpropagation, discovering whatever detectors are useful.

? MAKE SENSE OF IT

The situation: A phone camera app needs to find the edge of a document in a photo. A 3×3 kernel slides across the image looking for a jump from dark to bright.

What the result tells you: The feature map is 27 where the window straddles the edge and 0 in flat areas. Big value = "edge here".

What if…?

- **Bright on the left, dark on the right:** the map becomes **−27**. The sign tells you the direction of the edge.
- **A faint edge** (0 to 1 instead of 0 to 9): the response is only **3**. The size tells you the strength.
- **A horizontal edge** (dark top, bright bottom): this vertical-edge kernel gives **0** everywhere. You need a different kernel — so CNNs learn many.

Why it matters in ML: The same small kernel is reused at every position, so a CNN needs few weights and finds a pattern wherever it appears. Early layers learn edges like this; later layers combine them into shapes and objects.

Output size, stride and padding

For input width W, kernel size F, padding P (zeros added around the border) and stride S (step size):

$$W_{\text{out}} = \frac{W - F + 2P}{S} + 1$$

Example above: $(6 - 3 + 0)/1 + 1 = 4$ ✓. With a 28×28 image, $F = 3$, $P = 1$, $S = 1$: $(28 - 3 + 2)/1 + 1 = 28$ — "same" padding keeps the size.

Channels and parameter counts

A colour image has 3 channels (red, green, blue), so a 3×3 kernel actually has $3 \times 3 \times 3 = 27$ weights plus a bias. A layer usually has many kernels, each producing its own feature map. 16 kernels: $(27 + 1) \times 16 = 448$ parameters. Compare a fully connected layer from a 32×32×3 image to just 16 neurons: $3072 \times 16 + 16 = 49{,}168$. Weight sharing makes CNNs over 100 times leaner here.

Pooling

Max pooling shrinks a feature map by keeping only the largest value in each small window, usually 2×2 with stride 2. It keeps "was the feature present around here?" while discarding the exact position, and cuts the size by 4×, which discards 75% of the activations.

✎ WORKED EXAMPLE — 2×2 MAX POOLING

$$\begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 1 \\ 0 & 2 & 5 & 7 \\ 1 & 1 & 3 & 2 \end{bmatrix} \xrightarrow{\text{max pool}} \begin{bmatrix} 6 & 2 \\ 2 & 7 \end{bmatrix}$$

Top-left window $\{1, 3, 4, 6\}$ → 6; top-right $\{2, 0, 1, 1\}$ → 2; bottom-left $\{0, 2, 1, 1\}$ → 2; bottom-right $\{5, 7, 3, 2\}$ → 7.

? MAKE SENSE OF IT

The situation: After finding edges, the network only needs to know *roughly* where each one is, not to the exact pixel.

What the result tells you: Each 2×2 window keeps only its strongest response, so the map shrinks to a quarter of its size while keeping the important signals.

What if...?

- **Move the 6 one pixel** within its window: the output is still 6. Small shifts don't change the result.
- **Average pooling** instead: 3.5, 1, 1, 4.25 — smoother, but weak signals dilute strong ones.

Why it matters in ML: Pooling makes CNNs cheaper and more tolerant of small shifts in position. Many modern networks use strided convolutions instead, but the idea is the same.

A typical CNN

[conv → ReLU → pool] repeated several times (feature maps get smaller but more numerous), then flatten, then one or two fully connected layers, then softmax over the classes.

STEP-BY-STEP DERIVATION **Where the output-size formula comes from**

1. $W + 2P$

   Padding adds P zeros on each side, so the padded width is $W + 2P$.

2. $0,\ S,\ 2S,\ \ldots,\ (W + 2P - F)$

   The kernel's left edge can start at these positions: it must still fit, so the last start is $W + 2P - F$.

3. $\frac{W + 2P - F}{S} + 1$

   Count the positions: the number of steps of size S, plus 1 for the starting position.

4. Try it: convolution explorer

Left: a real 8×8 handwritten digit from scikit-learn's digits dataset. Choose a kernel and see the feature map it produces (right). Green = strong positive response, red = strong negative. The

vertical-edge kernel lights up the left and right edges of the strokes; the horizontal one finds tops and bottoms. Hover over the output to see the exact sum for that position.

Interactive demo — open *lesson-19.html* on the website to try it.

5. Recurrent neural networks (RNNs)

Text, speech, sensor readings and prices are **sequences**: order matters, and lengths vary. An RNN reads a sequence one step at a time, carrying a **hidden state** $\mathbf{h}_t$ — a memory of what it has seen so far:

$$\mathbf{h}_t = \tanh\left(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b}\right)$$

The same weights are used at every step (weight sharing again, this time across time). The final state, or the state at each step, feeds an output layer — e.g. to classify a review as positive or negative, or to predict the next word.

The problem: training backpropagates through every time step, multiplying by W_h again and again, so gradients vanish (or explode) over long sequences. Plain RNNs struggle to remember things from more than about 10–20 steps back.

LSTMs

The **long short-term memory** network (Hochreiter & Schmidhuber, 1997) adds a separate **cell state** — a kind of conveyor belt that information can travel along unchanged — controlled by three **gates**. Each gate is a sigmoid layer outputting numbers between 0 and 1, multiplied element-wise with the information it controls:

- the **forget gate** decides what to erase from the cell state;
- the **input gate** decides what new information to write;
- the **output gate** decides what part of the cell state to output as $\mathbf{h}_t$.

Because the cell state is updated by addition rather than repeated multiplication, gradients survive over much longer sequences. LSTMs (and the simpler GRUs) dominated sequence tasks until Transformers arrived.

6. Transformers and attention

`PICTURE IT` At a busy party you hear your name across the room and instantly tune in to that conversation. Attention lets each word in a sentence tune in to the other words that matter for it, and mostly ignore the rest.

RNNs process words one at a time, which is slow and makes long-range connections hard. The **Transformer** (Vaswani et al., 2017, "Attention Is All You Need") is "based solely on attention

mechanisms, dispensing with recurrence and convolutions entirely". Every position looks at every other position directly, in one step.

Self-attention

Each word (token) is represented by a vector. From it, the model computes three vectors with learned weight matrices:

- a **query** $\mathbf{q}$ — "what am I looking for?",
- a **key** $\mathbf{k}$ — "what do I contain?",
- a **value** $\mathbf{v}$ — "what do I pass on if selected?".

Each token compares its query with every token's key (dot products — Lesson 2 — measure similarity), turns the scores into weights with softmax (Lesson 8), and takes the weighted average of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Dividing by $\sqrt{d_k}$ (the key length) keeps the scores from growing too large as vectors get longer.

✎ WORKED EXAMPLE — ATTENTION OVER THREE TOKENS

Three tokens with 2-dimensional vectors. For simplicity, queries = keys: $\mathbf{q}_1 = \mathbf{k}_1 = (1, 0)$, $\mathbf{q}_2 = \mathbf{k}_2 = (0, 1)$, $\mathbf{q}_3 = \mathbf{k}_3 = (1, 1)$. Values: $\mathbf{v}_1 = (1, 0)$, $\mathbf{v}_2 = (0, 1)$, $\mathbf{v}_3 = (0.5, 0.5)$. Here $d_k = 2$, so we divide by $\sqrt{2} \approx 1.414$.

Token 1's scores against keys 1, 2, 3: $\frac{(1,0)\cdot(1,0)}{\sqrt{2}} = 0.707$, $\frac{(1,0)\cdot(0,1)}{\sqrt{2}} = 0$, $\frac{(1,0)\cdot(1,1)}{\sqrt{2}} = 0.707$.

Softmax: $e^{0.707} \approx 2.03$, $e^0 = 1$, $e^{0.707} \approx 2.03$; total 5.06. Weights $\approx (0.401,\ 0.198,\ 0.401)$.

Output: $0.401\,(1, 0) + 0.198\,(0, 1) + 0.401\,(0.5, 0.5) \approx (0.602,\ 0.398)$.

Token 1 "attended" most to itself and to token 3, whose keys resemble its query, and its new representation mixes in their values. In a sentence like "The animal didn't cross the street because *it* was tired", attention lets "it" pull information from "animal".

? MAKE SENSE OF IT

The situation: In "The animal didn't cross the street because it was tired", the word "it" must find which earlier word it refers to. Attention lets each word ask the others "are you relevant to me?"

What the result tells you: Token 1's query matches keys 1 and 3 equally, so it takes 40% from each and 20% from token 2. Its new vector (0.60, 0.40) is a blend weighted by relevance.

What if...?

- **Without dividing by $\sqrt{d_k}$:** weights (0.42, 0.16, 0.42) — sharper. With 512-dimensional vectors, unscaled scores get huge and softmax becomes almost all-or-nothing, which makes training unstable; that's why the scaling is there.
- **A stronger query** (10, 0): weights (0.5, 0.0, 0.5) — it ignores token 2 completely.
- **Token 3's query** (1, 1) matches itself most: weights (0.25, 0.25, 0.50).

Why it matters in ML: Attention is dot products (similarity), softmax (turn scores into weights) and a weighted average. That's the core of every Transformer, including large language models.

A Transformer stacks many such layers, each with several attention "heads" running in parallel, plus small feed-forward networks, residual connections and layer normalisation. Since attention itself ignores order, a **positional encoding** is added to each token's vector. Because all positions are processed at once, Transformers train efficiently on GPUs — which is what made today's large language models possible.

| WHERE THINGS STAND NOW

Large language models, reasoning models, agents and diffusion image generators are all built from the ideas in this lesson. For a dated overview of recent developments — and how each connects back to this course — see Recent developments in LLMs and generative AI.

ADVANCED · OPTIONAL **Why attention divides by $\sqrt{d_k}$** ▾ hide

Suppose the entries of a query $\mathbf{q}$ and key $\mathbf{k}$ are independent with mean 0 and variance 1. Then $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ is a sum of d_k terms each with mean 0 and variance 1, so

$$\mathrm{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k, \qquad \mathrm{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = 1.$$

Without the scaling, with $d_k = 64$ the scores would have standard deviation 8; softmax of such spread-out numbers puts almost all the weight on one token, and its gradients become

tiny (the softmax *saturates*). Dividing by $\sqrt{d_k}$ keeps the scores in a range where attention can spread out and still learn — the reason given in the original Transformer paper (Vaswani et al., 2017).

READ THE FORMULA **Scaled dot-product attention**

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- **softmax** — turns scores into weights that sum to 1
- **QKᵀ** — similarity of every query with every key (dot products)
- **√dₖ** — keeps scores from growing too large
- **V** — the values that get averaged using those weights

Hover over (or tap) a part to highlight it in the formula.

7. Transfer learning: standing on giants' shoulders

Training a large network from scratch needs huge datasets and computing power. Instead, you can take a **pretrained** model — trained on millions of images or billions of words — and **fine-tune** it on your smaller dataset. Its early layers already detect general features (edges, word meanings), so you only need to adapt the later layers. This is how most practical deep learning is done today.

8. Tools

scikit-learn's `MLPClassifier` is fine for small fully connected networks (Lesson 18), but it has no convolutions, recurrence, attention or GPU support. For deep learning, practitioners use **PyTorch** or **TensorFlow/Keras**, which compute gradients automatically ("autograd") and run on GPUs.

9. In Python

Convolution, pooling and attention from scratch with NumPy — the same calculations as the worked examples:

```
import numpy as np

def conv2d(image, kernel):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1))
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            out[i, j] = np.sum(image[i:i+kh, j:j+kw] * kernel)
    return out

image = np.array([[0, 0, 0, 9, 9, 9]] * 4, dtype=float)
edge = np.array([[-1, 0, 1]] * 3, dtype=float)
print(conv2d(image, edge))

fmap = np.array([[1, 3, 2, 0], [4, 6, 1, 1], [0, 2, 5, 7], [1, 1, 3, 2]], dtype=float)
print(fmap.reshape(2, 2, 2, 2).max(axis=(1, 3)))      # 2x2 max pooling

def attention(Q, K, V):
    scores = Q @ K.T / np.sqrt(K.shape[1])
    weights = np.exp(scores) / np.exp(scores).sum(axis=1, keepdims=True)   # softmax per
row
    return weights, weights @ V

Q = K = np.array([[1, 0], [0, 1], [1, 1]], dtype=float)
V = np.array([[1, 0], [0, 1], [0.5, 0.5]])
weights, out = attention(Q, K, V)
print(weights.round(3))
print(out.round(3))
```

Output:

```
[[ 0. 27. 27.  0.]
 [ 0. 27. 27.  0.]]
[[6. 2.]
 [2. 7.]]
[[0.401 0.198 0.401]
 [0.198 0.401 0.401]
 [0.248 0.248 0.503]]
[[0.602 0.398]
 [0.398 0.602]
 [0.5   0.5  ]]
```

Every number matches the worked examples. For reference, this is what a small CNN for 28×28 grayscale digits looks like in PyTorch. *This snippet was not run as part of this course's checks — PyTorch must be installed separately (see pytorch.org).*

```
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),   # 28x28x1  → 28x28x16
    nn.ReLU(),
    nn.MaxPool2d(2),                              # → 14x14x16
    nn.Conv2d(16, 32, kernel_size=3, padding=1),  # → 14x14x32
    nn.ReLU(),
    nn.MaxPool2d(2),                              # → 7x7x32
    nn.Flatten(),                                 # → 1568 numbers
    nn.Dropout(0.25),
    nn.Linear(32 * 7 * 7, 10),                    # one score per digit
)
```

10. Running example: Chennai Homes

Deep learning would use the richer data we've ignored: a CNN could learn from photos of each house, and a Transformer could read the listings. Here's what a Transformer would see — a sequence of tokens; with 12 short listings the vocabulary is only 23 words, far too little to train one from scratch, so in practice you'd fine-tune a pretrained language model.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
# A Transformer would read the listing text word by word; here we just look at
what it would see.
for d in df["description"].head(3):
    tokens = d.split()
    print(len(tokens), "tokens:", tokens)
print("vocabulary size:", len({w for d in df["description"] for w in d.split()}))
```

Output:

```
5 tokens: ['bright', 'compact', 'flat', 'near', 'metro']
6 tokens: ['spacious', 'flat', 'near', 'metro', 'with', 'balcony']
5 tokens: ['large', 'family', 'flat', 'needs', 'renovation']
vocabulary size: 23
```

⚙ WHERE THIS IS USED

- Image recognition (CNNs)
- Speech recognition and translation
- Chatbots and large language models (Transformers)

Choose it when: Images, audio, video and text — especially starting from a pretrained model.

Think twice when: Small structured datasets, or when you need a cheap, explainable model.

× MISCONCEPTIONS TO UNLEARN

MYTH Deep learning always needs millions of examples.

REALITY Fine-tuning a pretrained model can work with a few hundred or thousand labelled examples.

MYTH Attention shows exactly why a model made its decision.

REALITY Attention weights are one internal signal, not a complete explanation of the model's reasoning.

RESPONSIBLE AI · GENERATIVE MODELS RAISE NEW QUESTIONS

Models that generate text and images inherit the biases of their training data, can produce fluent but false statements ("hallucinations"), and make convincing fake media easy to create. Their training data raises questions of consent and copyright that courts and regulators are still working through. See Recent developments for a dated overview.

11. Questions beginners often ask

Q. Do I need a GPU?

For training large networks on images or text, yes in practice. For small experiments and for fine-tuning small pretrained models, a CPU can be enough.

Q. What's the difference between a CNN and a Transformer?

CNNs look at local patches with shared filters — ideal for images. Transformers let every position attend to every other — ideal for sequences like text, and now also used for images.

Q. Should I train from scratch?

Rarely. Fine-tuning a pretrained model needs far less data and computing power.

! COMMON MISTAKES TO AVOID

- Training a large network from scratch when a pretrained model could be fine-tuned.
- Forgetting to switch dropout off when evaluating (in PyTorch: `model.eval()`).
- Using sigmoid in every layer of a deep network (vanishing gradients) — prefer ReLU in hidden layers.

! INTERVIEW CORNER

Don't confuse

- **CNN vs RNN vs Transformer** — Grids, chains, all-to-all.
- **Fine-tuning vs feature extraction** — Use it as-is vs adjust it.
- **Padding vs stride** — Padding adds space; stride skips.

Hidden questions — answer in your head first, then open.

▼ **Why do CNNs need far fewer parameters than dense networks on images?**

The trap: "Convolutions are just efficient."

Weight sharing: one small kernel is reused at every position. A 3×3 convolution from 3 to 64 channels has 1,792 parameters whatever the image size; a dense layer from a 224×224×3 image to just 64 units needs 9,633,856.

▼ **Why does attention divide by $\sqrt{d_k}$?**

The trap: "For normalisation", with no explanation.

Dot products of random d_k-dimensional vectors have a spread that grows like $\sqrt{d_k}$. Without scaling, large scores push softmax towards all-or-nothing, where its gradients vanish. Dividing keeps scores in a range where softmax is still sensitive.

▼ **Why do Transformers need positional encodings?**

The trap: Assuming attention knows word order.

Attention treats its inputs as a set: shuffle the tokens and each output is just shuffled too. "Dog bites man" and "man bites dog" would look the same. Positional encodings (fixed or learned) add order information.

▼ **Why did LSTMs replace simple RNNs, and why did Transformers largely replace LSTMs?**

The trap: Naming them without the reasons.

Simple RNNs suffer vanishing gradients over long sequences; LSTMs' gates and cell state carry information further. Transformers then removed the step-by-step recurrence: attention connects any two positions directly and trains in parallel on GPUs.

12. Key takeaways

- Deep networks learn a hierarchy of features. ReLU, good initialisation, normalisation and residual connections fight **vanishing gradients**; **Adam** and **dropout** help training and generalization.
- **CNNs** slide small shared kernels over images: output size $(W - F + 2P)/S + 1$; **pooling** downsamples.
- **RNNs** carry a hidden state through a sequence; **LSTMs** use gates and a cell state to remember longer.
- **Transformers** use self-attention, $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$, so every token can look at every other token at once.
- In practice, start from a **pretrained** model and fine-tune it.

13. Check your understanding

1. A 32×32 input, 5×5 kernel, padding 0, stride 1. What is the output width?

a. 32

b. 28

c. 27

2. Why do CNNs need far fewer parameters than fully connected networks on images?

a. They ignore colour

b. They use smaller images

c. Each small kernel is shared across all positions in the image

3. What problem do LSTMs solve compared with plain RNNs?

a. Gradients vanishing over long sequences, so the network can't remember far back

b. RNNs can't process text

c. RNNs need labelled data

4. In self-attention, how is the similarity between a query and a key measured?

a. Euclidean distance

b. A (scaled) dot product

c. A decision tree

5. 2×2 max pooling on $\begin{bmatrix} 1 & 5 \\ 3 & 2 \end{bmatrix}$ gives...

a. 2.75

b. 1

c. 5

14. Going deeper: attention and residual connections, from scratch

INTERMEDIATE · SKIP ON A FIRST READ

Two ideas power nearly every modern architecture, from image models to large language models: **attention**, which lets each part of the input decide which other parts matter, and **residual connections**, which make networks with hundreds of layers trainable. This section builds both in a few lines of NumPy, measures why attention divides by $\sqrt{d}$, and explains how pretrained models are adapted to new tasks.

You'll need: this lesson, the softmax from Lesson 8, and vanishing gradients from Lesson 18's "Going deeper".

Attention, step by step

Each token (a word piece, or an image patch) is a vector. Three learned matrices turn it into a **query** ("what am I looking for?"), a **key** ("what do I contain?") and a **value** ("what will I pass on?"). Each token's output is a weighted average of all the values, where the weights come from how well its query matches each key:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

```
import numpy as np

# Scaled dot-product attention from scratch, for a "sentence" of 4 tokens with 8-number
vectors
rng = np.random.default_rng(0)
X = rng.normal(size=(4, 8))                                    # one row per token
Wq, Wk, Wv = (rng.normal(0, 1 / np.sqrt(8), size=(8, 8)) for _ in range(3))
Q, K, V = X @ Wq, X @ Wk, X @ Wv

def softmax(s):
    e = np.exp(s - s.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)

A = softmax(Q @ K.T / np.sqrt(8))                              # attention weights, one
row per token
out = A @ V                                                    # each output mixes the
value vectors
print("attention weights (row i: how much token i looks at each token):")
print(A.round(2))
print("each row sums to:", A.sum(axis=1).round(3), "  output shape:", out.shape)
```

Output:

```
attention weights (row i: how much token i looks at each token):
[[0.34 0.13 0.33 0.2 ]
 [0.78 0.   0.05 0.17]
 [0.25 0.28 0.29 0.18]
 [0.22 0.28 0.21 0.29]]
each row sums to: [1. 1. 1. 1.]   output shape: (4, 8)
```

Row i of the weight matrix shows how much token i attends to each token, and each row adds up to 1. Token 2 (the second row) puts 78% of its attention on token 1, while token 3 spreads its attention almost evenly. Every token's output is computed in parallel by matrix products, which is why transformers train so efficiently on GPUs compared with RNNs, which must process one step at a time. **Multi-head attention** runs several of these in parallel with different matrices, so different heads can track different relationships (grammar, meaning, position), and concatenates their outputs.

Why divide by $\sqrt{d_k}$, measured

If query and key entries have variance 1, their dot product over d dimensions has variance d, so its spread grows like $\sqrt{d}$. Large scores push the softmax towards a one-hot choice, where its gradients are almost zero, and learning stalls. Dividing by $\sqrt{d}$ brings the spread back to about 1. Here each query is compared with 10 random keys:

```
# Why divide by sqrt(d)? Dot products of random vectors grow with d, making softmax
nearly one-hot
for d in [8, 64, 512]:
    q, k = rng.normal(size=(1000, d)), rng.normal(size=(1000, 10, d))
    raw = np.einsum("nd,nkd→nk", q, k)
    top_raw = softmax(raw).max(axis=1).mean()
    top_scaled = softmax(raw / np.sqrt(d)).max(axis=1).mean()
    print(f"d {d:3d}: spread of dot products {raw.std():6.1f}   average top weight,
unscaled {top_raw:.2f}, scaled {top_scaled:.2f}")
```

Output:

```
d   8: spread of dot products    2.9   average top weight, unscaled 0.62, scaled 0.31
d  64: spread of dot products    8.0   average top weight, unscaled 0.85, scaled 0.31
d 512: spread of dot products   22.5   average top weight, unscaled 0.95, scaled 0.32
```

Unscaled, the average largest weight climbs from 0.62 at $d = 8$ to 0.95 at $d = 512$: almost every query locks onto a single key. Scaled, it stays about 0.31 at every size, so attention stays soft and trainable. (This lesson's "Advanced" box derives the variance argument.)

Residual connections

A **residual block** computes $\mathbf{h} + f(\mathbf{h})$ instead of $f(\mathbf{h})$: the layer learns a *correction* to its input rather than a whole new representation. In backpropagation the derivative of $\mathbf{h} + f(\mathbf{h})$ is $I + J_f$, so there is always an identity path for the gradient, however small J_f becomes (Lesson 3's "Going deeper" described the chain rule as multiplying Jacobians). Here is a 50-layer tanh network with small weights, with and without shortcuts:

```
# Residual connections: gradient reaching the first of 50 layers, with and without a
shortcut h + f(h)
def first_layer_grad(residual, depth=50, width=64):
    W = [rng.normal(0, 0.5 / np.sqrt(width), size=(width, width)) for _ in range(depth)]
# small weights
    h = rng.normal(size=(32, width)); zs = []
    for Wl in W:
        z = h @ Wl; zs.append(z)
        h = h + np.tanh(z) if residual else np.tanh(z)
    g = np.ones_like(h)
    for Wl, z in zip(reversed(W), reversed(zs)):
        local = (g * (1 - np.tanh(z) ** 2)) @ Wl.T             # gradient through the
layer itself
        g = g + local if residual else local                   # the shortcut adds an
identity path
    return np.linalg.norm(g)

print("plain 50-layer network, gradient at layer 1:   ", f"
{first_layer_grad(False):.2e}")
print("residual 50-layer network, gradient at layer 1:", f"
{first_layer_grad(True):.2e}")
```

Output:

```
plain 50-layer network, gradient at layer 1:    2.04e-14
residual 50-layer network, gradient at layer 1: 4.02e+02
```

Without shortcuts, the gradient reaching the first layer is 2×10^{-14}: effectively zero, so the early layers never learn. With shortcuts it's about 400. ResNets (2015) used this to train image networks with over 100 layers, and every transformer block wraps both its attention and its feed-forward part in a residual connection, usually with layer normalisation (Lesson 18's "Going deeper").

Positions, and putting a transformer block together

Attention on its own treats the input as an unordered set: shuffle the tokens and each output just moves with its token. Word order is reintroduced with **positional encodings** added to the token vectors: the original transformer used sine and cosine waves of different frequencies, $PE_{(p,2i)} = \sin(p/10000^{2i/d})$, and many modern LLMs use rotary encodings (RoPE), which rotate queries and keys by an angle that depends on position. One transformer block is then:

1. Multi-head self-attention, plus a residual connection, then layer norm.
2. A small feed-forward network applied to each token separately (usually two layers with a GELU activation), plus a residual connection, then layer norm.

A language model stacks dozens of these blocks, uses a **causal mask** so each token can only attend to earlier tokens, and is trained to predict the next token.

Transfer learning and fine-tuning

Training a large model from scratch needs huge data and compute. Instead, start from a model **pretrained** on a broad task (images from ImageNet, or text from the web) and adapt it:

Approach	What's trained	When to use it
Feature extraction	Only a new output layer; the pretrained network is frozen	Very little data; a task close to the pretraining task
Fine-tuning	All or the top layers, with a small learning rate	A moderate amount of data; the best accuracy
Parameter-efficient fine-tuning (LoRA, adapters)	Small added matrices; the original weights are frozen	Large language models: much less memory and storage per task
Prompting / in-context learning	Nothing	LLMs, when instructions and a few examples are enough

LoRA uses an idea from Lesson 2's "Going deeper": it learns the change to a big weight matrix as a product of two thin matrices, $\Delta W = BA$ with rank r of perhaps 8 or 16, so a 4096×4096 layer needs about $2 \times 4096 \times 8 = 65{,}536$ new numbers instead of 16.8 million.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Walk me through what a single self-attention head computes.**

The trap: Describing attention as "focusing" without the computation.

Project each token's vector into a query, key and value with learned matrices. Compute scores $QK^\top$, scale by $1/\sqrt{d_k}$, apply a mask if needed (causal masking for language models), and softmax each row into weights. Each output is the weighted sum of the value vectors. Cost is quadratic in sequence length, which is why long contexts are expensive and why efficient-attention variants exist.

▼ **Why are residual connections so important in deep networks?**

The trap: "They add information from earlier layers."

They make each block learn a correction $f(\mathbf{h})$ to an identity mapping. In backprop the Jacobian becomes $I + J_f$, so gradients always have a direct path to early layers, avoiding vanishing gradients through many layers. They also make very deep networks easy to optimise, since a block can default to doing almost nothing. Both ResNets and transformers depend on them.

▼ **What's LoRA, and why is it popular for fine-tuning LLMs?**

The trap: Calling it a type of quantization.

Low-Rank Adaptation freezes the pretrained weights and learns, for chosen layers, a low-rank update $\Delta W = BA$ (rank 4–64). That trains and stores a tiny fraction of the parameters, needs much less GPU memory, and lets you keep many task-specific adapters for one base model. Quality is often close to full fine-tuning. QLoRA combines it with a quantized base model to save more memory.

Check your understanding: going deeper

1. In attention, what does each row of $\mathrm{softmax}(QK^\top/\sqrt{d})$ add up to?

a. 0

b. 1

c. d

2. Without the $\sqrt{d}$ scaling, large d makes the attention weights...

a. Almost one-hot, with tiny gradients

b. Exactly uniform

c. Negative

3. Why do residual connections help gradients flow?

a. The derivative of $\mathbf{h} + f(\mathbf{h})$ includes an identity term

b. They remove non-linearities

c. They make the network shallower

4. Without positional encodings, a transformer would treat a sentence as...

a. A sequence

b. An unordered set of tokens

c. A single token

5. LoRA fine-tunes a large model by...

a. Retraining every weight

b. Learning small low-rank matrices added to frozen weights

c. Deleting layers

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Output width: (a) input 32, kernel 5, padding 0, stride 1; (b) input 28, 2×2 pooling, stride 2.

WORKING

EXERCISE 2 By hand

Convolve $\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}$ with the 2×2 kernel $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ (no padding, stride 1).

WORKING

EXERCISE 3 By hand

A conv layer has 32 filters of size 5×5 on a 3-channel input. How many parameters?

WORKING

EXERCISE 4 By hand

A token's attention scores (already divided by $\sqrt{d_k}$) are 2 and 0. What are its attention weights?

WORKING

EXERCISE 5 Intermediate

Two tokens have attention scores $(2, 0)$ after scaling. What are the softmax weights? If their value vectors are $(1, 0)$ and $(0, 1)$, what is the output?

WORKING

EXERCISE 6 Intermediate

Queries and keys have $d_k = 64$ with entries of variance 1. What is the standard deviation of a raw dot product, and after dividing by $\sqrt{d_k}$?

WORKING

EXERCISE 7 Intermediate

A 1024×1024 weight matrix gets a LoRA update of rank 16. How many trainable numbers does LoRA add, and what fraction is that of the full matrix?

WORKING

NOTES

PART 5 · NEURAL NETWORKS & REAL PROJECTS · ~70 MIN

Lesson 20: Feature Engineering, Pipelines & Final Project

Clean and encode data, build leak-proof pipelines, and complete an end-to-end project.

BY THE END OF THIS LESSON YOU CAN

- Scale, transform and encode features correctly
- Handle missing values without leaking information
- Build leak-proof pipelines with ColumnTransformer
- Complete an end-to-end ML project from raw data to a saved model

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 9: Evaluating Models — Accuracy can mislead. Precision $= TP/(TP + FP)$, recall $= TP/(TP + FN)$, and F1 balances them. Moving the threshold trades one for the other; ROC-AUC summarises all thresholds.
- Lesson 10: Overfitting, Regularization & Bias–Variance — Error = bias² + variance + noise. Regularization adds a penalty on weight size (L2 ridge shrinks weights; L1 lasso zeros some) to reduce variance; its strength is chosen by cross-validation.

Where this fits: Brings everything together into a real project: preparing messy data, building leak-proof pipelines, choosing and evaluating a model, and deploying it.

1. Data beats algorithms

Every model in this course expects a neat table of numbers: no gaps, no text, sensible scales. Real data is never like that. It has missing values, categories like "Chennai" or "two-year contract", amounts that range from 5 to 5 million, and dates. Turning raw data into good model inputs is

called **feature engineering**, and in practice it often matters more than which algorithm you choose.

This final lesson covers the essential techniques, then shows how to package every step into a **pipeline** that is safe from data leakage. It ends with a complete project — from raw data to a saved, tested model — that puts the whole course together.

2. Numeric features

Scaling

From Lessons 7, 10, 11 and 15: distance-based and gradient-based models (kNN, SVMs, linear and logistic regression, neural networks) need features on similar scales. Tree models (Lessons 13–14) don't.

- **Standardisation** (`StandardScaler`): $z = (x - \mu)/\sigma$ — mean 0, standard deviation 1. The usual default.
- **Min–max scaling** (`MinMaxScaler`): $x' = (x - x_{\min})/(x_{\max} - x_{\min})$ — squeezes into $[0, 1]$.
- **Robust scaling** (`RobustScaler`): uses the median and interquartile range, so outliers don't distort it.

> ✎ WORKED EXAMPLE — STANDARDISING TENURE
>
> Three customers have been with the company 3, 12 and 24 months. Mean $\mu = 13$. Standard deviation $\sigma = \sqrt{\frac{(-10)^2+1^2+11^2}{3}} = \sqrt{74} \approx 8.60$.
>
> z-scores: $\frac{3-13}{8.60} \approx -1.16$, $\frac{12-13}{8.60} \approx -0.12$, $\frac{24-13}{8.60} \approx 1.28$.
>
> Crucially, μ and σ are computed on the **training set only**, then reused unchanged on the test set and on new data.

? MAKE SENSE OF IT

The situation: A telecom company's churn model uses tenure (months with the company). It must be put on a standard scale before training.

What the result tells you: The z-scores say how many standard deviations each customer is from the average: −1.16 is short tenure, +1.28 is long.

What if…?

- **A new customer with 60 months:** $z = (60 - 13)/8.60 = 5.46$ — far outside anything seen in training. Worth flagging: the model is extrapolating.
- **One long-standing customer (120 months) in the training data:** the mean becomes 39.75 and the standard deviation 46.9, so the original three get z-scores −0.78, −0.59 and −0.34. One outlier squashes everyone else together — robust scaling (median and IQR) avoids this.

Why it matters in ML: Compute the mean and standard deviation on the training data only and reuse them — otherwise information from the test set leaks into training and your scores are over-optimistic.

Transforming skewed values

Amounts like income, prices or total spending are often **skewed**: most values are small, a few are huge. A **log transform** compresses the range so the huge values don't dominate: 100, 1,000 and 10,000 become 2, 3 and 4 with $\log_{10}$. In code, `np.log1p(x)` computes $\ln(1 + x)$, which also handles zeros safely.

Binning

Sometimes it helps to turn a number into ranges (age 18–25, 26–40, …) — useful when the effect isn't smooth, or for readability. It throws away detail, so use it deliberately.

READ THE FORMULA **Standardising with training statistics**

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

- **x** — any value — training, test or a new customer
- **μ_train** — mean learned from the training set only
- **σ_train** — standard deviation learned from the training set only

Hover over (or tap) a part to highlight it in the formula.

3. Categorical features

PICTURE IT Giving cities numbers like house numbers on a street would imply Delhi lives "between" Chennai and Mumbai. One-hot encoding gives each city its own light switch instead: on or off.

Models need numbers, but you can't just number the categories (Chennai = 1, Mumbai = 2, Delhi = 3): the model would think Delhi is "three times" Chennai, and that Mumbai lies "between" them.

One-hot encoding

Create one 0/1 column per category; exactly one is 1 for each row.

✎ WORKED EXAMPLE

city	→	city_Chennai	city_Delhi	city_Mumbai
Chennai		1	0	0
Mumbai		0	0	1
Delhi		0	1	0

All categories are now equally different from each other. In scikit-learn: `OneHotEncoder(handle_unknown="ignore")` — the option means a category never seen in training (say "Kolkata") becomes all zeros instead of an error.

? MAKE SENSE OF IT

The situation: City is a category, not a number. Coding Chennai = 1, Delhi = 2, Mumbai = 3 would tell a linear model that Mumbai is "three times" Chennai.

What the result tells you: One-hot encoding gives each city its own 0/1 column, so no city is "bigger" or "between" others.

What if...?

- **A new city** (Kolkata) appears after deployment: with handle_unknown="ignore" it becomes all zeros — the model treats it as "none of the known cities".
- **1,000 different cities:** 1,000 new columns, most almost always 0. Group rare categories into "other", or use target or frequency encoding.

Why it matters in ML: Tree models can cope with simple integer codes; linear models, kNN and neural networks need one-hot (or learned embeddings, Lesson 19).

Ordinal encoding

When categories have a natural order (small < medium < large; month-to-month < one-year < two-year), mapping them to 0, 1, 2 keeps that order in one column (`OrdinalEncoder`). Fine for trees; for linear models it assumes equal steps between levels.

Many categories

A feature like "postcode" may have thousands of values, which would create thousands of columns. Options include grouping rare categories into "other", or target encoding (replacing each category by the average label for that category — computed carefully inside cross-validation, or it leaks).

4. Missing values

Most scikit-learn models refuse missing values (HistGradientBoosting is an exception). Choices:

- **Drop rows** — only if very few are missing and missingness is random. Otherwise you lose data and may bias the sample.
- **Impute** — fill in a value with `SimpleImputer`: the **median** (robust for numbers), **mean**, **most frequent** (for categories), or a constant like "unknown". `KNNImputer` fills values from similar rows.
- **Add a "was missing" indicator** (`add_indicator=True`). Missingness itself can carry information: a customer who didn't give a phone number may behave differently.

> **! IMPUTATION CAN LEAK TOO**
>
> The median used for filling gaps must be computed from the training data only — just like scaling statistics. Pipelines handle this automatically.

5. Creating new features

Often the most valuable features are ones you build from domain knowledge:

- **Ratios and differences**: price per m², debt-to-income, monthly charge per support call.
- **Dates**: from a timestamp, extract day of week, month, hour, "is weekend", or "days since last purchase". Raw timestamps are rarely useful directly.
- **Interactions**: products of features (e.g. `PolynomialFeatures`) let linear models capture combined effects.
- **Aggregates**: a customer's average order value, number of orders in the last 30 days.
- **Text**: word counts (`CountVectorizer`, Lesson 12) or TF-IDF weights, which down-weight words common to every document.

> **| THINK LIKE THE MODEL**
>
> Ask: "If I were predicting this by hand, what would I want to know?" A model predicting late payments benefits from "days until due date" far more than from two separate raw dates.

6. Feature selection

More features isn't always better (Lessons 10, 11, 17). Ways to keep only useful ones: remove near-constant features (`VarianceThreshold`), keep the best by a statistical test (`SelectKBest`), keep features a model rates as important (`SelectFromModel`, e.g. with Lasso), or let L1 regularization zero out the rest. Like all preprocessing, do it inside the pipeline.

7. Try it: a preprocessing pipeline, step by step

Edit the raw customer table (left) — try deleting a monthly charge or changing a city. On the right is the matrix the model actually receives: missing charge filled with the training median, numbers standardised, and the contract one-hot encoded. The statistics shown are "learned" from these rows, exactly what `fit` does on a training set.

Interactive demo — open *lesson-20.html* on the website to try it.

8. Pipelines: preprocessing done right

A `Pipeline` chains steps into a single model object. A `ColumnTransformer` applies different steps to different columns. Together they give you three huge benefits:

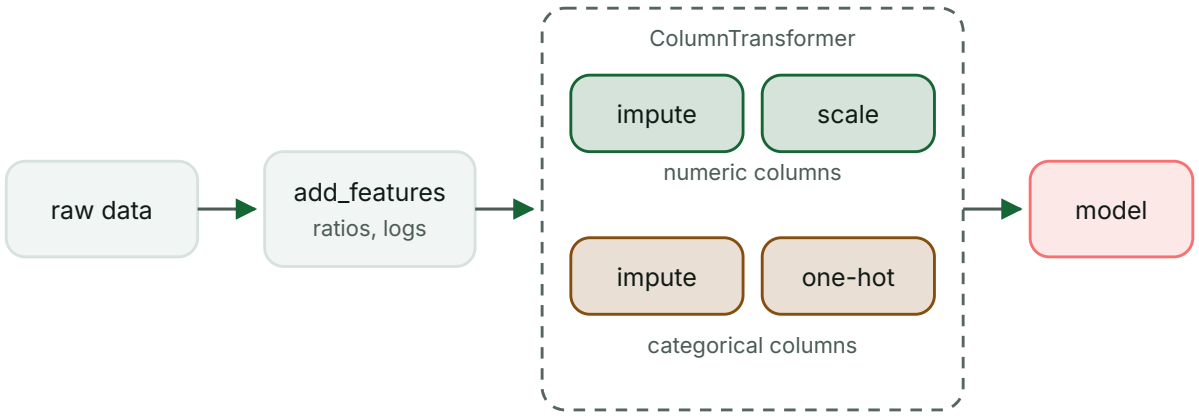


Figure 20. The final project's pipeline. Fitting it learns every step from the training data only; the same object then turns raw rows into predictions.

1. **No leakage.** Calling `fit` learns imputation values, scaling statistics and encodings from the training data only. During cross-validation, they're re-learned inside every fold.
2. **One object to tune.** Grid search can tune preprocessing and model settings together (e.g. `model__C`).
3. **One object to deploy.** Raw data in, prediction out — exactly the same transformations in training and production. Mismatches between the two ("training/serving skew") are a classic source of silent bugs.

9. Final project: predicting customer churn

The problem: a telecom company wants to know which customers are likely to cancel ("churn") so it can offer them a deal first. The table has numbers, categories and missing values — like real data. So that you can run everything yourself without downloading anything, the code first *generates* a realistic dataset of 2,000 customers. (It's synthetic: the patterns were built in, so they illustrate the workflow rather than real customer behaviour.)

Step 1 — Load and explore the data

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(42)
n = 2000
df = pd.DataFrame({
    "tenure_months": rng.integers(1, 73, n),
    "monthly_charge": rng.normal(65, 20, n).clip(15, 130).round(2),
    "support_calls": rng.poisson(1.5, n),
    "contract": rng.choice(["month-to-month", "one-year", "two-year"], n, p=[0.55, 0.25,
0.20]),
    "payment": rng.choice(["card", "bank", "check"], n, p=[0.4, 0.35, 0.25]),
    "city": rng.choice(["Chennai", "Mumbai", "Delhi", "Pune"], n),
})
df["total_spent"] = (df["tenure_months"] * df["monthly_charge"] * rng.normal(1, 0.05,
n)).round(2)
score = (-2.0 + 1.6 * (df["contract"] == "month-to-month") - 0.035 * df["tenure_months"]
         + 0.018 * df["monthly_charge"] + 0.45 * df["support_calls"] + 0.5 *
(df["payment"] == "check"))
df["churned"] = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
for col, frac in [("monthly_charge", 0.05), ("payment", 0.04)]:     # real data has gaps
    df.loc[rng.random(n) < frac, col] = np.nan

print(df.shape)
print(df.head(3).to_string())
print("churn rate:", round(df["churned"].mean(), 3))
print(df.isna().sum()[df.isna().sum() > 0].to_dict())
```

```
(2000, 8)
   tenure_months  monthly_charge  support_calls        contract payment     city
total_spent  churned
0              7           50.16              2        one-year    bank     Pune
316.79         0
1             56           83.49              0  month-to-month     NaN    Delhi
4578.37         1
2             48           65.69              2  month-to-month    card  Chennai
3111.67         1
churn rate: 0.403
{'monthly_charge': 105, 'payment': 86}
```

2,000 customers, 7 features plus the label. 40% churned, so the classes are only mildly imbalanced. Two columns have gaps (105 and 86 missing values), and three columns are text categories.

Step 2 — Split before anything else

```
from sklearn.model_selection import train_test_split

X = df.drop(columns="churned")
y = df["churned"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=0)
```

The test set (400 customers) is now locked away until Step 6 (Lesson 5).

Step 3 — Feature engineering and preprocessing

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer

def add_features(X):
    X = X.copy()
    X["charge_per_call"] = X["monthly_charge"] / (X["support_calls"] + 1)   # a ratio
feature
    X["log_total_spent"] = np.log1p(X["total_spent"])                       # tame the
skew
    return X

numeric = ["tenure_months", "monthly_charge", "support_calls", "total_spent",
           "charge_per_call", "log_total_spent"]
categorical = ["contract", "payment", "city"]

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
categorical),
])
```

Numbers: fill gaps with the median, then standardise. Categories: fill gaps with the most common value, then one-hot encode. Nothing is fitted yet — that happens inside cross-validation.

Step 4 — Start with a baseline, then compare models

```
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_score

candidates = {
    "baseline (majority)": DummyClassifier(strategy="most_frequent"),
    "logistic regression": LogisticRegression(max_iter=1000),
    "random forest": RandomForestClassifier(n_estimators=300, random_state=0),
    "gradient boosting": HistGradientBoostingClassifier(random_state=0),
}
for name, clf in candidates.items():
    pipe = Pipeline([("features", FunctionTransformer(add_features)),
                     ("prep", preprocess), ("model", clf)])
    s = cross_val_score(pipe, X_train, y_train, cv=5, scoring="roc_auc")
    print(f"{name:22s} ROC-AUC {s.mean():.3f} ± {s.std():.3f}")
```

```
baseline (majority)    ROC-AUC 0.500 ± 0.000
logistic regression    ROC-AUC 0.771 ± 0.008
random forest          ROC-AUC 0.756 ± 0.010
gradient boosting      ROC-AUC 0.739 ± 0.008
```

We use ROC-AUC (Lesson 9) because we'll want to rank customers by risk. The baseline scores 0.5 — pure guessing — so everything above it is real signal. Interesting: **logistic regression beats the ensembles** here. The relationships in this data are close to linear on the log-odds scale (Lesson 8), so the simpler model fits them best and the flexible models only add variance. Always compare — don't assume the fanciest model wins.

Step 5 — Tune the best model

```
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([("features", FunctionTransformer(add_features)), ("prep", preprocess),
                 ("model", LogisticRegression(max_iter=1000))])
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1, 10]}, cv=5, scoring="roc_auc")
search.fit(X_train, y_train)
print("best C:", search.best_params_["model__C"], " CV ROC-AUC:",
round(search.best_score_, 3))
```

```
best C: 0.1  CV ROC-AUC: 0.772
```

Slightly stronger regularization ($C = 0.1$, Lesson 10) helps a little.

Step 6 — Evaluate once on the test set

```
from sklearn.metrics import roc_auc_score, classification_report

best = search.best_estimator_
proba = best.predict_proba(X_test)[:, 1]
print("test ROC-AUC:", round(roc_auc_score(y_test, proba), 3))
print(classification_report(y_test, (proba ⩾ 0.5).astype(int), target_names=["stayed",
"churned"]))
```

```
test ROC-AUC: 0.765
              precision    recall  f1-score   support

      stayed       0.73      0.82      0.77       239
     churned       0.67      0.55      0.61       161

    accuracy                           0.71       400
   macro avg       0.70      0.68      0.69       400
weighted avg       0.71      0.71      0.70       400
```

Test ROC-AUC 0.765 is close to the cross-validated 0.772, so our estimate was honest. At the default threshold the model catches 55% of churners (recall) and is right 67% of the time when it flags someone (precision). If a retention offer is cheap, the company could lower the threshold to catch more churners at the cost of more false alarms (Lesson 9).

Step 7 — Interpret the model

```
names = best.named_steps["prep"].get_feature_names_out()
coefs = best.named_steps["model"].coef_[0]
for name, c in sorted(zip(names, coefs), key=lambda t: -abs(t[1]))[:5]:
    print(f"{name:32s} {c:+.2f}")
```

```
cat__contract_month-to-month     +0.85
num__tenure_months               -0.59
num__support_calls               +0.45
cat__contract_one-year           -0.43
cat__contract_two-year           -0.42
```

Because features were standardised, the coefficients are comparable (Lesson 6). Month-to-month contracts and more support calls push towards churn; longer tenure and annual contracts push away. These match the patterns built into the data — a good sanity check. Remember: these describe associations, not proven causes.

Step 8 — Save, reload and predict

```
import joblib

joblib.dump(best, "churn_model.joblib")
reloaded = joblib.load("churn_model.joblib")       # only ever load files you trust!

new_customer = pd.DataFrame([{"tenure_months": 3, "monthly_charge": 95.0,
"support_calls": 4,
                              "contract": "month-to-month", "payment": "check", "city":
"Pune",
                              "total_spent": 285.0}])
print("churn probability:", round(reloaded.predict_proba(new_customer)[0, 1], 3))
```

```
churn probability: 0.958
```

The whole pipeline — feature creation, imputation, scaling, encoding and model — is saved as one file. Raw customer data goes in, a probability comes out. A new, expensive, month-to-month customer with 4 support calls is flagged as very high risk.

ONE PRACTICAL DETAIL

The saved file stores a *reference* to the `add_features` function, not its code. To load the model in a different script or server, that function must be importable there too — for example, put it in a small module file (`features.py`) and import it in both places.

! SECURITY: LOADING MODEL FILES

scikit-learn's documentation warns that pickle-based formats (pickle, joblib) can execute arbitrary code when loaded. Never load a model file from a source you don't trust. Safer formats include `skops.io` (which lets you inspect untrusted types before loading) and ONNX. Pickle-based files also need the same library versions that were used to save them.

10. After deployment: monitoring

A model's performance can quietly decay because the world changes (Lesson 5's distribution shift). Google's *Rules of Machine Learning* warns to "watch for silent failures" — for example, a feature that used to be filled in for 90% of rows dropping to 60% because an upstream system changed. In practice:

- Log inputs and predictions; track the distribution of each feature and of the predicted probabilities over time.

- When true outcomes arrive (did the customer actually leave?), keep measuring real-world metrics.
- Check for training/serving skew: is production data processed exactly like training data?
- Retrain regularly on recent data, and evaluate on data from *later* than the training period.

11. The complete ML workflow — a checklist

1. Define the problem, the label and the metric that reflects real costs.
2. Collect and explore the data: types, gaps, imbalance, oddities.
3. Split off a test set immediately (by time, if the data is time-ordered).
4. Build a pipeline: feature engineering, imputation, encoding, scaling.
5. Score a trivial baseline.
6. Compare several model families with cross-validation.
7. Tune the best one or two.
8. Evaluate once on the test set; choose a decision threshold.
9. Interpret and sanity-check the model.
10. Save the whole pipeline, deploy, and monitor.

12. Congratulations — and where to go next

You've gone from "what is a model?" to building, evaluating and saving complete machine learning systems — and you understand the maths underneath: vectors, gradients, probability, loss functions, regularization, and backpropagation. Ideas to explore next:

- **Practice on real datasets**: the scikit-learn documentation's example gallery is full of complete, runnable examples.
- **Deep learning in depth**: install PyTorch and build the CNN from Lesson 19; the free *Deep Learning* book by Goodfellow, Bengio and Courville goes much further.
- **Stronger maths**: *Mathematics for Machine Learning* (Deisenroth, Faisal & Ong) is free online and builds directly on Lessons 2–4.
- **Revisit the demos**: re-reading a lesson after the later ones often makes things click.

13. Running example: Chennai Homes

> A complete pipeline: impute a missing age, scale the numbers, one-hot encode the area, then ridge regression. Adding the area feature didn't help here (MAE 6.9 → 7.2 lakh) — distance already carries most of that information, and with 12 houses every extra column adds noise.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_val_score
df.loc[3, "age_yrs"] = np.nan                    # pretend one value is missing
num = ["size_m2", "bedrooms", "dist_km", "age_yrs"]
numeric = Pipeline([("impute", SimpleImputer(strategy="median")), ("scale",
StandardScaler())])
with_area = ColumnTransformer([("num", numeric, num), ("cat",
OneHotEncoder(handle_unknown="ignore"), ["area"])])
numbers_only = ColumnTransformer([("num", numeric, num)])
for name, prep in [("numbers only", numbers_only), ("numbers + area",
with_area)]:
    model = Pipeline([("prep", prep), ("ridge", Ridge(alpha=1.0))])
    mae = -cross_val_score(model, df, df["price_lakh"], cv=4,
scoring="neg_mean_absolute_error").mean()
    print(f"{name:15s} cross-validated MAE: {mae:.1f} lakh")
```

Output:

```
numbers only    cross-validated MAE: 6.9 lakh
numbers + area  cross-validated MAE: 7.2 lakh
```

⚙ WHERE THIS IS USED

- Every real-world ML system: churn prediction, demand forecasting, risk scoring…

Choose it when: Always.

× MISCONCEPTIONS TO UNLEARN

MYTH Feature engineering is obsolete now that we have deep learning.

REALITY For tabular data, good features and careful preprocessing are still among the biggest drivers of performance.

MYTH Once a model is deployed, the work is done.

REALITY Data changes over time; models need monitoring and retraining.

RESPONSIBLE AI · RESPONSIBLE DEPLOYMENT

A model in production makes decisions at scale, so document it: **model cards** (Mitchell et al., 2019) record what the model is for, how it was evaluated, for which groups, and its known limits. Monitor for drift and for unequal error rates, watch for feedback loops (a model's decisions changing the data it later learns from), and make sure people affected can question a decision and reach a human.

14. Questions beginners often ask

Q. How much time goes into data preparation?

In real projects, usually more than into modelling. Clean, well-understood data matters more than the choice of algorithm.

Q. Should I always build new features?

Only if they add new information. Test each one with cross-validation — Exercise 4 in this lesson shows a feature that doesn't help.

Q. What should I learn next?

Practise on real datasets end to end, then go deeper into whichever area interests you: deep learning, time series, or the maths.

! COMMON MISTAKES TO AVOID

- Fitting any preprocessing step on the full dataset before splitting.
- Label-encoding unordered categories as 1, 2, 3 for linear or distance-based models.
- Deploying without monitoring, so silent data changes go unnoticed.

! INTERVIEW CORNER

Don't confuse

- **Normalisation vs standardisation** — Squeeze into a range vs centre and scale.
- **One-hot vs label vs ordinal encoding** — Switches, ID numbers, or ranks.
- **Data drift vs concept drift** — New questions vs new answers to old questions.

Hidden questions — answer in your head first, then open.

▼ **Where should the mean and standard deviation for scaling come from?**

The trap: "From the whole dataset."

From the training data only, then reused unchanged for validation, test and production data. Computing them on everything leaks information about the test set.

▼ **When you one-hot encode a category for linear regression, why drop one column?**

The trap: Keeping all columns by habit.

With an intercept, the one-hot columns always sum to 1, the same as the intercept column — perfect collinearity (the "dummy variable trap"). Dropping one column fixes it. Regularised models and trees don't need this.

▼ **Why is target encoding risky, and how is it done safely?**

The trap: Replacing each category with its average target on the full data.

Each row's own label contributes to its encoded value, so rare categories leak the answer and the model looks unrealistically good. Compute encodings out of fold (from other folds' data only) and add smoothing towards the overall mean.

▼ **You oversample the minority class (e.g. SMOTE), then split into train and test. What's wrong?**

The trap: Thinking resampling is just preprocessing.

Synthetic or duplicated minority examples end up on both sides, inflating the test score. Resample the *training* data only, inside cross-validation, and evaluate on untouched, naturally imbalanced data.

15. Key takeaways

- Good features often matter more than the choice of algorithm.
- **Scale** numbers for distance/gradient models, **log-transform** skewed amounts, **one-hot encode** categories, and **impute** missing values (optionally with an indicator).
- Create features from domain knowledge: ratios, date parts, aggregates.
- Put everything in a **Pipeline** + **ColumnTransformer** so it's fitted on training data only and deployed as one object.
- Workflow: split → baseline → compare with CV → tune → test once → interpret → save → monitor.

16. Check your understanding

1. Why not encode cities as Chennai = 1, Mumbai = 2, Delhi = 3 for a linear model?

a. Numbers are slower than text

b. The model would treat the codes as ordered quantities, e.g. Delhi = 3 × Chennai

c. Linear models can't use integers

2. Where should the median used to fill missing values come from?

a. The whole dataset

b. The test set

c. The training set only

3. A column holds account balances from 0 to 5,000,000, mostly under 10,000. A sensible transformation is...

a. A log transform, e.g. `np.log1p`

b. One-hot encoding

c. Deleting the column

4. What is the main advantage of putting preprocessing inside a Pipeline?

a. It makes models more accurate in every case

b. Preprocessing is re-fitted on training data inside each CV fold and travels with the model into production

c. It removes the need for a test set

5. Your new model scores ROC-AUC 0.52 and the baseline scores 0.50. What does that tell you?

a. The model is excellent

b. The baseline is broken

c. The model has learned almost nothing beyond guessing — revisit features and data

17. Going deeper: leak-proof pipelines and efficient tuning

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above builds features and a pipeline for a full project. This section shows, with numbers, how much a single leaky encoding can fake, how to handle numbers, categories and text in one leak-proof pipeline, why random search usually beats grid search, and how teams keep track of hundreds of experiments. (A later intermediate lesson covers hyperparameter tuning in full.)

You'll need: this lesson, leakage from Lesson 5 and nested cross-validation from Lesson 5's "Going deeper".

Target encoding: how much leakage can fake

Target encoding replaces each category with the average target for that category. It's powerful for categories with many levels (postcodes, product IDs), but if the averages are computed using the rows you then evaluate on, each row's own target leaks into its feature. Here a category with 2,000 levels has *no* real relationship with the target, so the honest accuracy is 50%:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import TargetEncoder

# A category with 2,000 levels (like a postcode) that has NO real link to the target
rng = np.random.default_rng(0)
n = 5000
code = rng.integers(0, 2000, n)
y = rng.integers(0, 2, n)
cv = KFold(5, shuffle=True, random_state=0)

# Wrong: encode with each level's average target computed on ALL the data, then cross-validate
means = pd.Series(y).groupby(code).mean()
leaky = means[code].to_numpy()[:, None]
print("naive target encoding, CV accuracy:", round(cross_val_score(LogisticRegression(), leaky, y, cv=cv).mean(), 3))

# Right: TargetEncoder inside the pipeline (it also uses internal cross-fitting when training)
pipe = make_pipeline(TargetEncoder(cv=KFold(5, shuffle=True, random_state=0)), LogisticRegression())
print("target encoding inside the pipeline, CV accuracy:", round(cross_val_score(pipe, code[:, None], y, cv=cv).mean(), 3))
```

Output:

```
naive target encoding, CV accuracy: 0.731
target encoding inside the pipeline, CV accuracy: 0.497
```

The naive encoding reports 73.1% accuracy on pure noise: with about 2.5 rows per level, each level's average is dominated by the rows' own labels. Inside the pipeline, scikit-learn's `TargetEncoder` is refitted on each training fold, and also uses **cross-fitting** during training (each training row is encoded using averages from the other internal folds), so the estimate drops to the honest 49.7%. Two further safeguards it applies: **smoothing** towards the overall average for rare levels, much like the Bayesian shrinkage in Lesson 4's "Going deeper", and handling of unseen levels.

One pipeline for mixed data

Real tables mix numbers, categories and text, each needing different preprocessing. `ColumnTransformer` applies the right steps to the right columns, and putting it inside a pipeline means every step is fitted on training folds only:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import Ridge

# One pipeline for mixed data: numbers, a category and free text (Chennai Homes)
df = pd.read_csv("houses.csv")
df.loc[2, "age_yrs"] = np.nan                                # pretend one value is
missing
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
["size_m2", "bedrooms", "dist_km", "age_yrs"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["area"]),
    ("txt", TfidfVectorizer(), "description"),
])
model = make_pipeline(prep, Ridge(alpha=1.0))
scores = cross_val_score(model, df, df["price_lakh"], cv=KFold(4, shuffle=True,
random_state=0), scoring="neg_mean_absolute_error")
print("columns after preprocessing:", prep.fit_transform(df).shape[1])
print("4-fold mean absolute error:", round(-scores.mean(), 1), "lakh")
```

Output:

```
columns after preprocessing: 30
4-fold mean absolute error: 7.0 lakh
```

Four numeric columns (one with a missing value, imputed with the training median and standardised), one-hot encoded areas and a TF-IDF of the descriptions give 30 columns, fed to ridge regression. Cross-validated on the 12 Chennai homes the mean error is about 7.0 lakh, which with so few homes is only an illustration. The important point is structural: the whole object can be cross-validated, tuned with `GridSearchCV` (parameter names such as `columntransformer__num__simpleimputer__strategy`) and saved as one file, so training and serving can't drift apart.

Grid search vs random search

A grid tries every combination of a few values per setting. **Random search** samples settings from ranges you specify. With the same budget it usually does as well or better, because typically only one or two settings really matter: a 3×3×2 grid tests just 3 distinct values of each, whereas 18 random draws test 18 different values of every setting (Bergstra and Bengio, 2012). Sample learning rates and penalties on a log scale (`loguniform`), since they matter by orders of magnitude.

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.datasets import load_breast_cancer
from scipy.stats import loguniform, randint

Xb, yb = load_breast_cancer(return_X_y=True)
gb = HistGradientBoostingClassifier(random_state=0)
grid = GridSearchCV(gb, {"learning_rate": [0.03, 0.1, 0.3], "max_leaf_nodes": [7, 15,
31],
                         "l2_regularization": [0.0, 1.0]}, cv=5).fit(Xb, yb)
rand = RandomizedSearchCV(gb, {"learning_rate": loguniform(0.01, 0.5), "max_leaf_nodes":
randint(4, 64),
                               "l2_regularization": loguniform(1e-3, 10)}, n_iter=18,
cv=5, random_state=0).fit(Xb, yb)
print(f"grid search:   {len(grid.cv_results_['params'])} settings, best CV accuracy
{grid.best_score_:.3f}")
print(f"random search: {len(rand.cv_results_['params'])} settings, best CV accuracy
{rand.best_score_:.3f}")
```

Output:

```
grid search:   18 settings, best CV accuracy 0.975
random search: 18 settings, best CV accuracy 0.972
```

Here, with 18 settings each, both reach about the same best cross-validated accuracy (0.975 and 0.972, a difference well within noise). The advantage of random search grows with the number of settings: the grid's size multiplies, the random budget doesn't. Beyond random search, **Bayesian optimisation** (for example Optuna's TPE sampler) uses past results to choose promising settings next, and **successive halving** (`HalvingRandomSearchCV`) gives every candidate a little data and only the best ones more.

Keeping track of experiments

After a few weeks a project has hundreds of runs. Without records, nobody can say which data, code and settings produced the model in production. Good practice:

- **Log every run:** parameters, metrics, the code version (a git commit), the data version, the random seed and the library versions. Tools such as MLflow and Weights & Biases do this with a few lines.
- **Version the data** as well as the code (for example with DVC, or by storing immutable dated snapshots).
- **Save the whole pipeline**, not just the model, together with a model card: intended use, training data, metrics by subgroup (Lesson 21) and known limitations.

- **Keep the test set out of the loop.** Every decision taken by looking at test results makes them optimistic (Lesson 4's "Going deeper" on multiple testing).

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How would you build a pipeline for a table with numeric, categorical and text columns, so it can't leak?**

The trap: Preprocessing the whole dataframe first, then splitting.

Use a `ColumnTransformer` inside a `Pipeline`: imputation and scaling for numbers, one-hot or target encoding (with cross-fitting) for categories, TF-IDF or embeddings for text, then the model. Cross-validate and tune the whole pipeline, so every statistic is learned from training folds only, and save it as one artifact so serving uses identical transformations.

▼ **Grid search, random search or Bayesian optimisation: which would you use?**

The trap: Always grid search.

Grid search only for one or two settings with few values. Random search as a strong default: with the same budget it explores far more values of the settings that matter, and it parallelises trivially. Bayesian optimisation (e.g. Optuna/TPE) when each run is expensive, since it uses earlier results to pick the next settings. Add early stopping or successive halving to drop bad candidates cheaply. Report the final score with nested CV or a held-out test set.

▼ **Six months later, you need to reproduce the model in production exactly. What should have been recorded?**

The trap: "The notebook."

The exact code version (commit), the data snapshot or version and how it was split, every parameter and random seed, library versions (requirements or lock file), the full preprocessing pipeline, and the evaluation metrics, ideally in an experiment tracker or model registry. Without the data version and the preprocessing, the same code can produce a different model.

Check your understanding: going deeper

1. Target-encoding a category using averages computed on all the data, then cross-validating, gives...

a. An honest score

b. An optimistic score, because each row's own target leaks into its feature

c. A pessimistic score

2. What does `ColumnTransformer` do?

a. Applies different preprocessing to different columns, inside a pipeline

b. Removes columns

c. Tunes hyperparameters

3. Why does random search often beat grid search with the same number of tries?

a. It tries many more distinct values of each setting, and usually only a few settings matter

b. It's exhaustive

c. It never overfits

4. How should a learning rate between 0.001 and 1 be sampled?

a. Uniformly

b. On a log scale

c. Only at 0.5

5. Which should be logged with every training run?

a. Only the final accuracy

b. Parameters, metrics, code and data versions, seed and library versions

c. Nothing, if the code is in git

18. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Training values of a feature are $10, 20, 30$. Standardise the new value 40 (divide by n).

WORKING

EXERCISE 2 By hand

One-hot encode the sizes small, large, medium, small (categories in alphabetical order: large, medium, small).

WORKING

EXERCISE 3 Concepts

What's wrong? `X = StandardScaler().fit_transform(X)`, then `train_test_split(X, y)`, then `cross_val_score(model, X_train, y_train)`.

WORKING

EXERCISE 4 Project

Extend the churn project: add a feature `is_new_customer` (tenure ≤ 6 months) inside `add_features`, add it to the numeric list, and re-run Steps 4–6. Did it help?

WORKING

EXERCISE 5 Intermediate

A category level appears in 4 training rows with targets $1, 1, 0, 1$. The overall average is 0.3. Using the smoothed encoding $\frac{n\,\bar{y}_{\text{level}} + m\,\bar{y}}{n+m}$ with $m = 6$, what is the level's encoding?

WORKING

EXERCISE 6 Intermediate

A grid search over 4 learning rates, 5 depths and 3 penalties, with 5-fold CV. How many model fits is that? How many different learning rates does it try?

WORKING

EXERCISE 7 Intermediate

Sampling learning rates log-uniformly between 0.001 and 1, what fraction of draws fall below 0.01?

WORKING

NOTES

PART 5 · NEURAL NETWORKS & REAL PROJECTS · ~60 MIN

Lesson 21: Ethics & Fairness in Machine Learning

Where bias comes from, how to measure fairness per group, why the criteria conflict, and what to do about it.

BY THE END OF THIS LESSON YOU CAN

- Name the main kinds of harm and the places where bias enters an ML system
- Compute selection rate, TPR, FPR and precision for each group from confusion-matrix counts
- Explain demographic parity, equal opportunity, equalised odds and predictive parity, and the four-fifths rule
- Explain why the fairness criteria conflict when base rates differ, and choose mitigations

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 8: Logistic Regression — Logistic regression outputs a probability $p = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, is trained with log loss, and classifies with a threshold (often 0.5), giving a straight decision boundary.
- Lesson 9: Evaluating Models — Accuracy can mislead. Precision $= TP/(TP + FP)$, recall $= TP/(TP + FN)$, and F1 balances them. Moving the threshold trades one for the other; ROC-AUC summarises all thresholds.

Where this fits: Steps back from building models to ask who they affect. It reuses the confusion matrix from Lesson 9 and the threshold idea from Lesson 8 to measure and reduce unfairness.

1. Why fairness is a machine-learning problem

Imagine a bank that replaces its loan officers with a model trained on ten years of past decisions. The model is 88% accurate, fast and consistent. Is it fair? Accuracy alone can't tell you. If past

officers turned down qualified applicants from one neighbourhood, the model has learned to do the same — automatically, at scale, and with an air of objectivity that makes it harder to challenge.

Machine learning finds patterns in data, and data records the world as it was, including its unfairness. A model doesn't need to be told anyone's ethnicity or gender to discriminate: other features — a postcode, a first name, a gap in employment — can act as **proxies**. This lesson gives you the vocabulary, the numbers and the habits to notice and reduce these problems.

> ◎ THE CORE IDEA
>
> A model can be accurate *on average* while making very different kinds of mistakes for different groups of people. Fairness work means measuring those differences, deciding which ones matter for the situation, and acting on them — a job for people, informed by numbers.

2. What can go wrong: kinds of harm

- **Allocation harms:** a resource or opportunity — a loan, a job interview, a medical referral — is withheld from some groups more than others.
- **Quality-of-service harms:** the system simply works worse for some people, like face recognition or speech recognition with higher error rates for some groups.
- **Representation harms:** outputs reinforce stereotypes or erase groups, such as image search or text generation that reproduces biased associations.
- **Privacy and autonomy harms:** people are profiled, re-identified or manipulated using data they never agreed to share for that purpose.

3. Where bias comes from

Bias can enter at every step of the workflow you learned in Lesson 5:

1. **Historical bias:** the world itself was unequal, so even a perfect record of it teaches the model those patterns (the Amazon hiring model, Lesson 1).
2. **Representation (sampling) bias:** some groups are under-represented in the training data, so the model is less accurate for them.
3. **Measurement and label bias:** the label is a *proxy* for what you care about. Using healthcare *cost* to stand for health *need* under-served Black patients (Obermeyer et al., 2019; Lesson 5).
4. **Aggregation bias:** one model for everyone ignores real differences between groups — for example, a medical threshold that suits one population but not another.
5. **Evaluation bias:** the test set or benchmark doesn't reflect the people the model will actually be used on.

6 **Deployment and feedback loops:** the model's decisions change the future data. Police sent where a model predicts crime record more crime there, which "confirms" the prediction.

4. Measuring fairness with group metrics

Split your test set by group and compute the confusion matrix (Lesson 9) *for each group separately*. From those counts come the standard fairness measures:

Measure	Formula (per group)	"Fair" means equal…	Name of the criterion
Selection rate	$\frac{TP+FP}{\text{all}}$	share of people approved	Demographic parity
True-positive rate (recall)	$\frac{TP}{TP+FN}$	chance a qualified person is approved	Equal opportunity
TPR *and* false-positive rate	$\frac{TP}{TP+FN}$ and $\frac{FP}{FP+TN}$	both kinds of error	Equalised odds
Precision (PPV)	$\frac{TP}{TP+FP}$	how reliable an approval is	Predictive parity

A common summary is the **demographic-parity ratio**: the lower selection rate divided by the higher one. In US employment law the "four-fifths rule" of thumb treats a ratio below 0.8 as evidence of possible adverse impact that needs explaining.

✎ WORKED EXAMPLE — ONE LOAN MODEL, TWO GROUPS

A bank tests its model on 100 applicants from each of two groups. In group A, 50 would repay; in group B, 20 would.

Group	TP	FP	FN	TN
A	40	5	10	45
B	12	4	8	76

Selection rate: A $= 45/100 = 0.45$; B $= 16/100 = 0.16$. Ratio $0.16/0.45 \approx 0.36$ — far below 0.8.

True-positive rate: A $= 40/50 = 0.80$; B $= 12/20 = 0.60$. A qualified applicant from B is approved less often.

False-positive rate: A $= 5/50 = 0.10$; B $= 4/80 = 0.05$.

Precision: A $= 40/45 \approx 0.89$; B $= 12/16 = 0.75$.

Accuracy: A $= 85/100 = 0.85$; B $= 88/100 = 0.88$ — the model is *more accurate* for group B even though it fails B's qualified applicants more often. Accuracy alone hides the problem.

? MAKE SENSE OF IT

The situation: A bank's compliance officer audits the loan model before it goes live, comparing two groups of applicants.

What the result tells you: Group B is approved far less often (16% vs 45%), and qualified B applicants are approved less often (60% vs 80%) — yet B's accuracy is higher. Each metric answers a different fairness question.

What if…?

- **Lower group B's threshold** so it approves 32 people (16 who would repay, 16 who wouldn't): selection 0.32 (ratio **0.71**), TPR **0.80** (now equal to A), but FPR doubles to 0.20 and precision falls to 0.50.
- **A smaller audit:** if group B had only 10 people, one applicant would move B's selection rate by 10 points. Small groups need confidence intervals before conclusions.

Why it matters in ML: Fairness metrics are the confusion matrix (Lesson 9) computed per group. Changing a threshold for one group moves all its metrics at once — the trade-offs are unavoidable, so decide which matters most with the people affected.

5. Try it: one threshold or two?

Two groups, A (half would repay) and B (one in five would repay), each get a score from the same model. Move the thresholds and watch the per-group rates. Try to make the selection rates equal; then try to make the true-positive rates equal; then notice what happens to precision and accuracy.

Interactive demo — open *lesson-21.html* on the website to try it.

6. You can't have everything: the impossibility result

Why not just make every rate equal? Because, when the groups' **base rates** differ (here 50% vs 20%), it is mathematically impossible except for a perfect classifier. For any group, the four quantities are tied together by an identity (Chouldechova, 2017):

$$\mathrm{FPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR},$$

where p is the group's base rate. If two groups have the same PPV and the same TPR but different p, their FPRs *must* differ.

✎ WORKED EXAMPLE — CHECKING THE IDENTITY

Group A: $p = 0.5$, PPV $= 40/45$, TPR $= 0.8$: $\frac{0.5}{0.5} \cdot \frac{5/45}{40/45} \cdot 0.8 = 1 \times 0.125 \times 0.8 = 0.10$ ✓ (the FPR we computed).

Group B: $p = 0.2$, PPV $= 0.75$, TPR $= 0.6$: $\frac{0.2}{0.8} \cdot \frac{0.25}{0.75} \cdot 0.6 = 0.25 \times 0.333 \times 0.6 = 0.05$ ✓.

Now suppose we tuned group B to match A's PPV (0.889) and TPR (0.8). The identity forces B's FPR to be $0.25 \times 0.125 \times 0.8 = 0.025$ — no longer equal to A's 0.10. Equal precision and equal opportunity together rule out equal false-positive rates.

? MAKE SENSE OF IT

The situation: A regulator asks: "Can't you just make every error rate equal for both groups?"

What the result tells you: The identity links FPR, precision, TPR and the base rate. Because the groups' base rates differ (50% vs 20%), fixing two of the rates forces the third apart.

What if...?

- **Insist B's FPR equals A's 0.10** while keeping TPR 0.8: B's precision must drop to **0.67** (vs A's 0.89).
- **If both groups had the same base rate,** the factor $\frac{p}{1-p}$ would be the same and all three rates could match.

Why it matters in ML: This is why fairness is a choice, not a setting. The impossibility comes from the data's base rates, not from a bad algorithm.

So **choosing a fairness criterion is a value judgement**. In lending, a bank may care most that qualified applicants are treated alike (equal opportunity). In criminal justice, critics of the COMPAS risk score (ProPublica, 2016) focused on unequal false-positive rates, while its makers pointed to equal predictive value — and both were right about their own measure. Kleinberg, Mullainathan and Raghavan (2016) proved a closely related result for calibrated scores.

7. What you can do about it

- **Fix the data (before training):** collect more data for under-represented groups, check that labels measure what you care about, remove or rethink proxy features, re-weight examples so groups count fairly.
- **Constrain the model (during training):** add a fairness penalty or constraint to the loss, just as regularisation adds a penalty (Lesson 10). Libraries such as Fairlearn and AIF360 implement several methods.
- **Adjust decisions (after training):** choose thresholds per group to equalise a chosen rate, as in the demo. This is effective but can be legally restricted, because it treats groups differently

explicitly — get legal advice.

- **Change the system, not just the model:** keep a human in the loop for consequential decisions, give people an explanation and a way to appeal, monitor outcomes after deployment, and be willing *not* to deploy.

> **! "FAIRNESS THROUGH UNAWARENESS" DOESN'T WORK**
>
> Deleting the protected attribute does not remove bias when other features are correlated with it — and it makes the bias harder to *measure*, because you can no longer compute group metrics. Most practitioners keep the attribute out of the model's inputs but use it for auditing.

8. Beyond fairness: privacy, transparency and accountability

- **Privacy:** collect only the data you need, with consent. Models can leak training data (membership inference, Lesson 10); *differential privacy* adds carefully calibrated random noise during training so that no single person's record changes the result much.
- **Transparency:** prefer interpretable models for high-stakes decisions, and explain outputs to the people affected. Document data with *datasheets* (Gebru et al.) and models with *model cards* (Mitchell et al., 2019).
- **Accountability:** someone must be responsible for a model's decisions, and there must be a route to contest them. Laws such as the EU AI Act (see Recent developments) now require risk management, documentation and human oversight for high-risk uses such as hiring and credit.
- **Wider impact:** energy use, effects on workers, and misuse (deepfakes, surveillance) are part of the responsibility too.

9. A practical checklist before you deploy

1. **Purpose:** who is affected, what decision does the model inform, and is ML appropriate at all?
2. **Data:** who is missing or under-represented? Is the label a fair proxy for the real goal?
3. **Metrics:** compute the confusion matrix, selection rate, TPR, FPR and PPV for every relevant group — and the uncertainty when groups are small.
4. **Choose:** decide with stakeholders which errors matter most here, and which fairness criterion you will hold the model to.
5. **Document:** write a model card: intended use, evaluation by group, known limitations.
6. **Monitor:** after deployment, track group metrics over time and watch for feedback loops; provide explanations and appeals.

10. In Python: a fairness report from scratch

Everything above is just the confusion matrix computed per group. Here is a small report on a synthetic loan dataset in which group B has a lower base rate:

```
import numpy as np
import pandas as pd

# A synthetic loan dataset: two groups with different base rates of repayment
rng = np.random.default_rng(0)
n = 2000
group = rng.choice(["A", "B"], size=n, p=[0.6, 0.4])
base = np.where(group == "A", 0.5, 0.2)              # share who would repay
repay = (rng.random(n) < base).astype(int)
score = np.clip(0.5 + 0.25 * (repay - 0.5) * 2 + rng.normal(0, 0.18, n), 0, 1)  #
model's score
df = pd.DataFrame({"group": group, "repay": repay, "score": score})

def group_report(df, threshold):
    d = df.assign(approve=(df["score"] ⩾ threshold).astype(int))
    rows = {}
    for g, s in d.groupby("group"):
        tp = ((s.approve == 1) & (s.repay == 1)).sum(); fp = ((s.approve == 1) &
(s.repay == 0)).sum()
        fn = ((s.approve == 0) & (s.repay == 1)).sum(); tn = ((s.approve == 0) &
(s.repay == 0)).sum()
        rows[g] = {"selection rate": (tp + fp) / len(s), "TPR": tp / (tp + fn), "FPR":
fp / (fp + tn), "PPV": tp / (tp + fp)}
    return pd.DataFrame(rows).T.round(3)

r = group_report(df, threshold=0.6)
print(r)
print("demographic-parity ratio:", round(r.loc["B", "selection rate"] / r.loc["A",
"selection rate"], 3))
```

```
   selection rate    TPR    FPR    PPV
A           0.430  0.799  0.039  0.956
B           0.178  0.801  0.025  0.887
demographic-parity ratio: 0.414
```

Read it like a fairness auditor: with one shared threshold, qualified applicants are treated alike — the true-positive rates are both 0.80 (**equal opportunity holds**). But group B is approved far less often (ratio 0.41, well below 0.8) and an approval is less reliable for B (PPV 0.89 vs 0.96) — both consequences of B's lower base rate, exactly as the impossibility result predicts.

11. Running example: Chennai Homes

Does the Lesson 8 model treat areas differently? It only sees distance and age, never the area name — yet distance is a near-perfect **proxy** for area. Every Central house is flagged as a fast seller (4 of 4), including the one that didn't sell fast (FPR 1.00); elsewhere only 3 of 8 are flagged and none wrongly. With 4 and 8 houses these rates are extremely noisy — one house moves a rate by 25 points — which is itself a fairness lesson: small groups need far more data before you trust their numbers.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_predict
cols = ["dist_km", "age_yrs"]                # the model never sees "area"
clf = make_pipeline(StandardScaler(), LogisticRegression())
df["pred"] = cross_val_predict(clf, df[cols], df["sold_fast"], cv=4)   # honest
out-of-fold predictions
df["group"] = np.where(df["area"] == "Central", "Central", "Not central")
for g, s in df.groupby("group"):
    tp = ((s.pred == 1) & (s.sold_fast == 1)).sum(); fp = ((s.pred == 1) &
(s.sold_fast == 0)).sum()
    pos, neg = (s.sold_fast == 1).sum(), (s.sold_fast == 0).sum()
    print(f"{g:12s} n={len(s):2d}  base rate {s.sold_fast.mean():.3f}  flagged
{s.pred.mean():.3f}  "
          f"TPR {tp / pos:.2f}  FPR {fp / neg if neg else float('nan'):.2f}")
```

Output:

```
Central      n= 4  base rate 0.750  flagged 1.000  TPR 1.00  FPR 1.00
Not central  n= 8  base rate 0.375  flagged 0.375  TPR 1.00  FPR 0.00
```

⚙ WHERE THIS IS USED

- Auditing credit, hiring, insurance and healthcare models before deployment
- Model cards and impact assessments required by regulators
- Monitoring deployed models for unequal error rates

Choose it when: Always, whenever a model's decisions affect people.

× MISCONCEPTIONS TO UNLEARN

MYTH Algorithms are objective because they are maths.

REALITY The maths is objective about the data it's given; the data, the label, the features and the threshold all carry human choices.

MYTH A model that's accurate overall is fair.

REALITY Overall accuracy can hide very different error rates for different groups, as the worked example shows (88% accuracy for B, yet B's qualified applicants are approved less often).

MYTH We can satisfy every fairness definition if we try hard enough.

REALITY When base rates differ, it is mathematically impossible (except for a perfect model). You must choose which criterion matters most.

RESPONSIBLE AI · FAIRNESS IS A PROCESS, NOT A NUMBER

No single metric certifies a model as fair. Fairness work means involving the people affected, documenting choices and their reasons, and keeping the ability to question, override and switch off the system. Metrics are evidence for that conversation, not a replacement for it.

12. Questions beginners often ask

Q. If I remove gender and ethnicity from the data, is my model fair?

Not necessarily. Other features (postcode, school, first name) can act as proxies, and without the attribute you can no longer measure group differences. Keep it for auditing.

Q. Which fairness metric is the right one?

There is no universal answer. Ask which mistake hurts people most in this situation: missing qualified people (compare TPR), wrongly flagging people (compare FPR), or unequal access overall (compare selection rates). Decide with the people affected.

Q. Isn't the model just reflecting reality?

It reflects the data, which records past decisions and unequal access as well as reality. A model used to make future decisions can lock those patterns in, so 'it's in the data' is a reason to look closer, not a defence.

Q. Are fairness rules the same everywhere?

No. Laws differ by country and by domain (lending, employment, housing). The four-fifths rule is a US guideline for employment, not a universal law. Get legal advice for real deployments.

! COMMON MISTAKES TO AVOID

- Reporting only overall accuracy and never splitting the metrics by group.
- Treating a tiny subgroup's rates as precise — with a handful of people, one person changes the rate a lot.
- Deleting the protected attribute and assuming the problem is solved.
- Choosing a fairness metric without involving the people the decisions affect.

! INTERVIEW CORNER

Don't confuse

- **Demographic parity vs equal opportunity vs equalised odds** — Equal approvals, equal chances for the qualified, or equal errors of both kinds.
- **Disparate treatment vs disparate impact** — Intent in the rule vs effect in the outcome.

Hidden questions — answer in your head first, then open.

▼ **If you remove gender and ethnicity from the data, is the model fair?**

The trap: "Yes, it can't see them."

Not necessarily. Other features (postcode, school, first name) act as proxies, and without the attribute you can't even measure group differences. Keep it out of the inputs if appropriate, but use it for auditing.

▼ **Can a model have equal precision, equal TPR and equal FPR across two groups?**

The trap: "Just tune the thresholds."

Not if the groups' base rates differ (unless the model is perfect). The identity $\mathrm{FPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR}$ forces at least one to differ. In Lesson 21's loan example, equalising FPR at 0.10 drops group B's precision to 0.67.

▼ **Group B's accuracy is higher than group A's. Does that show the model treats B fairly?**

The trap: Using accuracy as the fairness measure.

No. In Lesson 21's example B had higher accuracy (0.88 vs 0.85) yet qualified B applicants were approved less often (TPR 0.60 vs 0.80). Compare the specific error rates that matter for the decision.

▼ **Is the four-fifths rule a law that defines fairness?**

The trap: Treating it as universal.

It's a US employment-selection rule of thumb for spotting possible adverse impact, not a definition of fairness and not a law everywhere. Rules differ by country and domain.

13. Key takeaways

- Models learn the unfairness in their data; proxies let them discriminate without seeing protected attributes.
- Bias can enter through history, sampling, labels, aggregation, evaluation and feedback loops.
- Fairness is measured by comparing per-group rates: selection rate (demographic parity), TPR (equal opportunity), TPR and FPR (equalised odds), precision (predictive parity).
- When base rates differ, these criteria cannot all hold at once — choosing one is a value judgement to make with the people affected.
- Mitigation happens in the data, the training, the thresholds and, above all, the surrounding system: documentation, human oversight, appeal and monitoring.

> YOU'VE FINISHED THE COURSE
>
> That was the last lesson. To turn what you've learned into real skill, see What next: your learning plan — directions, a 12-week plan, project ideas and free resources.

14. Check your understanding

1. A model never receives applicants' ethnicity. Can it still discriminate by ethnicity?

a. No — it can't use information it doesn't have

b. Yes — other features can act as proxies for ethnicity

c. Only if it is a neural network

2. Group A's selection rate is 0.50 and group B's is 0.30. What is the demographic-parity ratio, and does it pass the four-fifths rule of thumb?

a. 0.6 — it does not pass

b. 0.6 — it passes

c. 0.2 — it passes

3. "Equal opportunity" asks which rate to be the same for every group?

a. The selection rate

b. The precision

c. The true-positive rate

4. Two groups have different base rates. A model has equal precision and equal TPR for both. What must be true?

a. Their false-positive rates are also equal

b. Their false-positive rates differ (unless the model is perfect)

c. Their accuracies are equal

5. A hospital model predicts healthcare *spending* to decide who gets extra care. What kind of bias is the biggest risk?

a. Label (measurement) bias — spending is a proxy for need

b. Too little regularisation

c. The learning rate is too high

15. Going deeper: the impossibility result, measured

INTERMEDIATE · SKIP ON A FIRST READ

The lesson above introduces the main fairness metrics and states that they can conflict. This section proves why with one line of algebra, shows it happening to a model that is perfectly calibrated for both groups, measures what a common fix (group-specific thresholds) gains and costs, and shows why small groups make fairness numbers unreliable unless you report their uncertainty.

You'll need: this lesson, the confusion matrix from Lesson 9, calibration from Lesson 8's "Going deeper", and the bootstrap from Lesson 4's.

The identity behind the conflict

For any classifier and any group with base rate p (the share of actual positives), the error rates are tied together by a simple identity, which follows from writing out the confusion matrix:

$$\mathrm{FPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR}$$

Now suppose two groups have *different* base rates p. If the model has equal precision (PPV) in both groups and equal TPR, the identity forces their FPRs to differ. Equal PPV, equal TPR and equal FPR can hold together only if the base rates are equal or the classifier is perfect. This is the result of Chouldechova (2017); Kleinberg, Mullainathan and Raghavan (2016) proved a similar one for calibration. It's not a flaw of a particular model: it is arithmetic.

A perfectly calibrated model can still have unequal error rates

Below, every person's score *is* their true probability of the outcome, so the model is calibrated in both groups by construction: among people scored 0.6–0.7, about 63–65% have the outcome in either group. Group A's base rate is 0.287 and group B's is 0.500. With one threshold of 0.5 for everyone:

```
import numpy as np

# Two groups with different base rates; a model whose scores are perfectly calibrated in
both groups
rng = np.random.default_rng(0)
def make_group(n, a, b):
    risk = rng.beta(a, b, n)                # each person's true probability
    return risk, (rng.random(n) < risk).astype(int)
groups = {"group A": make_group(20000, 2, 5), "group B": make_group(20000, 4, 4)}

print("group     base rate   calibration (pred 0.6-0.7: actual)   TPR     FPR     PPV")
for g, (score, y) in groups.items():
    flag = score ⩾ 0.5
    sel = (score ⩾ 0.6) & (score < 0.7)
    tpr = flag[y == 1].mean(); fpr = flag[y == 0].mean(); ppv = y[flag].mean()
    print(f"{g:8}   {y.mean():9.3f}   {y[sel].mean():33.3f}   {tpr:.3f}   {fpr:.3f}
{ppv:.3f}")
```

Output:

```
group     base rate   calibration (pred 0.6-0.7: actual)   TPR     FPR     PPV
group A        0.287                                0.629   0.232   0.060   0.607
group B        0.500                                0.654   0.631   0.362   0.636
```

Precision is similar (0.607 and 0.636), but group A's true-positive rate is 0.232 against group B's 0.631, and its false-positive rate 0.060 against 0.362. A person in group A who truly has the outcome is far less likely to be flagged; a person in group B who doesn't is far more likely to be wrongly flagged. Which of those matters more depends entirely on what being flagged means: a loan refused, an extra medical test, a police stop. That's a decision for people, not for the metric.

Group-specific thresholds: what they buy and what they cost

One common mitigation is to choose a separate threshold for each group so that a chosen metric matches, here the true-positive rate ("equal opportunity"):

```
# Equalise true-positive rates with group-specific thresholds, and see what it costs
def rates(score, y, t):
    flag = score ⩾ t
    return flag[y == 1].mean(), flag[y == 0].mean(), (flag == y).mean()

sA, yA = groups["group A"]; sB, yB = groups["group B"]
target_tpr = rates(sB, yB, 0.5)[0]
ts = np.linspace(0.05, 0.95, 181)
tA = ts[np.argmin([abs(rates(sA, yA, t)[0] - target_tpr) for t in ts])]
for name, t_A in [("one threshold 0.5 for both", 0.5), (f"threshold {tA:.3f} for group
A", tA)]:
    tprA, fprA, accA = rates(sA, yA, t_A); tprB, fprB, accB = rates(sB, yB, 0.5)
    print(f"{name:28}: TPR A {tprA:.3f} / B {tprB:.3f}   FPR A {fprA:.3f} / B {fprB:.3f}
accuracy A {accA:.3f}")
```

Output:

```
one threshold 0.5 for both  : TPR A 0.232 / B 0.631   FPR A 0.060 / B 0.362   accuracy A
0.736
threshold 0.305 for group A : TPR A 0.634 / B 0.631   FPR A 0.320 / B 0.362   accuracy A
0.667
```

Lowering group A's threshold to about 0.305 brings its TPR to 0.634, matching group B's 0.631. The FPRs come closer too (0.320 and 0.362). But group A's accuracy falls from 0.736 to 0.667, because many more of its negatives are now flagged, and precision equality is lost. Every option moves along this trade-off; none removes it. In some countries and domains, using a protected attribute to set different thresholds is also legally restricted, so check before deploying. Alternatives include improving the data for under-served groups, adding features that are informative for them, or changing the decision itself (for example offering a review rather than a refusal).

Small groups: put error bars on fairness metrics

Fairness audits often compare metrics for small groups, and small groups make noisy estimates. Here group A's TPR is measured on just 400 people (106 actual positives):

```
# Small groups make fairness metrics noisy: bootstrap interval for a TPR measured on
just 400 people
yS, sS = yA[:400], sA[:400]
pos = np.flatnonzero(yS == 1)
boot = [np.mean(sS[rng.choice(pos, len(pos))] >= 0.5) for _ in range(5000)]
print("positives in the small sample:", len(pos), "  TPR:", round(np.mean(sS[pos] >=
0.5), 3),
      "  95% interval:", np.round(np.percentile(boot, [2.5, 97.5]), 3))
```

Output:

```
positives in the small sample: 106   TPR: 0.226   95% interval: [0.151 0.311]
```

The TPR is 0.226, but the 95% bootstrap interval runs from 0.151 to 0.311, wider than many gaps people report as unfair (or as fixed). Report intervals for every group's metrics; check intersections (for example older women, not just "women" and "older people"), where groups are smaller still; and treat large gaps on small groups as a reason to collect more evidence, not as a final verdict.

An audit checklist

Step	Question
Decide what harm matters	Is a missed positive or a false alarm worse for the people affected?
Pick the metric before looking	Equal TPR, equal FPR, equal precision, calibration within groups, or demographic parity?
Measure every group and intersection	With bootstrap intervals, on held-out data
Check calibration within each group	Do scores mean the same thing for everyone?
Compare mitigations with their costs	Thresholds, reweighting, better data, a different decision process
Document and monitor	A model card, plus re-checks as data and base rates drift (Lesson 5's "Going deeper")

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Prove that equal precision, equal TPR and equal FPR can't all hold across groups with different base rates.**

The trap: Only citing the paper.

From the confusion matrix, $\mathrm{FP} = \mathrm{TP}(1-\mathrm{PPV})/\mathrm{PPV}$, and $\mathrm{FPR} = \mathrm{FP}/N$, $\mathrm{TPR} = \mathrm{TP}/P$. So $\mathrm{FPR} = \frac{P}{N} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR}$. If PPV and TPR are equal in both groups, FPR is proportional to $p/(1-p)$, which differs when base rates differ. Only a perfect classifier (PPV = 1, so FPR = 0) or equal base rates escape it.

▼ **Your audit shows a 6-point TPR gap between two groups. Is the model unfair?**

The trap: Answering yes or no from the point estimate.

First, is the gap real? Compute bootstrap intervals; for small groups a 6-point gap can be noise. Second, is it the gap that matters for this decision (would a missed positive or a false positive cause the harm)? Third, look at calibration within groups and at intersections. Then compare mitigations (better data, thresholds, reweighting, changing the decision) with their costs, and document the choice.

▼ **What are group-specific thresholds, and what are their downsides?**

The trap: Presenting them as a free fix.

Choosing a different decision threshold per group so that a target metric (for example TPR) matches. Downsides: other metrics (precision, FPR, accuracy) move, sometimes worse for the same group; it requires knowing the protected attribute at decision time; it may be legally restricted in some domains; and it treats the symptom, while the underlying data or feature gap remains.

Check your understanding: going deeper

1. Two groups have different base rates. A model has equal precision and equal TPR in both. Its FPRs must be...

a. Equal

b. Different (unless the model is perfect)

c. Zero

2. A model is perfectly calibrated in both groups. Does that guarantee equal true-positive rates?

a. Yes

b. No: with different base rates, TPR and FPR usually differ

c. Only with a 0.5 threshold

3. Lowering one group's threshold to equalise TPR typically...

a. Raises that group's false-positive rate

b. Changes nothing else

c. Raises its precision

4. A group's TPR is 0.23 measured on 100 positives. What should you report?

a. Just 0.23

b. 0.23 with a confidence interval, which is wide for so few cases

c. Nothing

5. Who should decide which fairness metric matters most?

a. The algorithm

b. People who understand the decision and its harms, including those affected

c. Whoever has the highest accuracy

16. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Group A: 60 of 200 applicants approved. Group B: 30 of 150 approved. Compute both selection rates and the demographic-parity ratio. Does it pass the four-fifths rule of thumb?

WORKING

EXERCISE 2 By hand

In group C, 40 people would repay a loan; the model approves 30 of them. Of the 60 who would not repay, it approves 6. Compute TPR, FPR and precision.

WORKING

EXERCISE 3 By hand

A group has base rate $p = 0.25$, precision 0.8 and TPR 0.6. Use the identity $\mathrm{FPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR}$ to find its FPR.

WORKING

EXERCISE 4 Concepts

A university admissions model is trained on past admissions decisions. List two ways bias could enter, and one check you would run before using it.

WORKING

EXERCISE 5 Project

Take the synthetic loan data from the Python section. Choose a separate threshold for group B so that its selection rate is within 0.8 of group A's. What happens to B's TPR, FPR and precision?

WORKING

EXERCISE 6 Intermediate

A group has base rate $p = 0.2$, precision 0.6 and TPR 0.75. Use the identity to find its FPR.

WORKING

EXERCISE 7 Intermediate

Another group has the same precision (0.6) and TPR (0.75) but base rate $p = 0.5$. What is its FPR?

WORKING

EXERCISE 8 Intermediate

A TPR of 0.25 is measured on 100 positives. Using the normal approximation $\pm 1.96\sqrt{p(1-p)/n}$, give the 95% interval.

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~60 MIN

Lesson 22: SQL & Data Wrangling for ML

Pull training data out of databases: SELECT, GROUP BY, joins, window functions, pandas equivalents and point-in-time features.

BY THE END OF THIS LESSON YOU CAN

- Write SELECT queries with WHERE, GROUP BY, HAVING and ORDER BY, and run them from Python
- Choose between INNER and LEFT joins and avoid the row-explosion trap
- Use window functions to add context features such as rank within a group
- Build a training table that uses only information available at prediction time

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 20: Feature Engineering, Pipelines & Final Project —

Where this fits: Starts Part 6 by stepping back to where data really comes from. It turns the leakage rules from Lesson 5 and the pipelines of Lesson 20 into habits for pulling data out of databases.

1. Why SQL belongs in an ML course

Every lesson so far started with a tidy table already loaded into Python. In a real job the data lives somewhere else: in a **database**, spread across many tables (customers, orders, payments, support tickets), often with millions of rows. Before any model can be trained, someone has to pull out the right rows, join the tables together, summarise events into features, and do it without accidentally peeking into the future.

That someone is usually you, and the language is almost always **SQL** (Structured Query Language). Data-science interviews almost always include a SQL round, and a large share of real ML work is data preparation. This lesson teaches the SQL and pandas you need for that work: selecting, filtering, grouping, joining, window functions, reshaping, and the one rule that keeps feature tables honest.

◎ THE CORE IDEA

A database stores each kind of thing in its own table and links tables with **keys**. Building a training set means joining those tables back together and summarising each entity's history into one row of features, using only information that was available *at the moment the prediction would have been made*.

2. Tables, rows and keys

A **relational database** is a set of tables. Each row is one record and each column one attribute, with a fixed type (number, text, date). Two kinds of column link tables together:

- A **primary key** uniquely identifies each row of a table, such as `houses.id`.
- A **foreign key** is a column that refers to another table's primary key, such as `viewings.house_id`, which says which house each viewing was for.

Storing viewings in their own table instead of as columns on each house means a house can have any number of viewings (zero, one or fifty) without empty columns or repeated house details. This design is called **normalisation**. Building ML features is mostly the reverse: *denormalising* many tables back into one wide row per entity.

Throughout this lesson we use the Chennai Homes table (12 houses) plus a second table of **viewings**: 11 visits by potential buyers, each with a house id and the day it happened. Python's built-in `sqlite3` module gives us a real SQL database with nothing to install.

3. SELECT, WHERE and ORDER BY

A query describes *what* you want, and the database decides how to find it. The basic shape is:

```
SELECT columns FROM table WHERE condition ORDER BY column
```

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")                # a database that lives in memory
pd.read_csv("houses.csv").to_sql("houses", con, index=False)
viewings = pd.DataFrame({"house_id": [1, 1, 2, 4, 4, 4, 5, 8, 11, 11, 12],
                         "day":      [3, 9, 2, 1, 4, 8, 6, 5, 2, 7, 1]})
viewings.to_sql("viewings", con, index=False)

q = """SELECT id, area, size_m2, price_lakh
       FROM houses
       WHERE price_lakh > 80
       ORDER BY price_lakh DESC"""
print(pd.read_sql(q, con))
```

Output:

```
   id       area  size_m2  price_lakh
0  12     Suburb      125          96
1   3    Central      120          95
2   5     Suburb      110          88
3   8  Outskirts      150          84
4   2    Central       90          82
```

Five houses cost more than 80 lakh, listed from the most expensive down. `pd.read_sql` runs the query and returns the result as a pandas DataFrame, which is how SQL and Python usually meet in practice. Useful extras: `LIMIT 10` keeps the first 10 rows, `DISTINCT` removes duplicate rows, and conditions combine with `AND`, `OR`, `IN (...)`, `BETWEEN` and `LIKE 'new%'` for text patterns. Missing values are `NULL` and need `IS NULL`, because `= NULL` is never true.

4. GROUP BY: from rows to summaries

`GROUP BY` collapses all rows that share a value into one row, and **aggregate functions** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) summarise each group. `WHERE` filters rows *before* grouping; `HAVING` filters groups *after*.

```
q = """SELECT area, COUNT(*) AS homes, ROUND(AVG(price_lakh), 1) AS avg_price,
              ROUND(AVG(price_lakh * 1.0 / size_m2), 3) AS lakh_per_m2
       FROM houses
       GROUP BY area
       HAVING COUNT(*) ⩾ 3
       ORDER BY avg_price DESC"""
print(pd.read_sql(q, con))
```

Output:

```
        area  homes  avg_price  lakh_per_m2
0    Central      4       76.8        0.889
1     Suburb      5       76.4        0.692
2  Outskirts      3       58.7        0.462
```

✎ WORKED EXAMPLE — CHECKING A GROUP BY BY HAND

The Outskirts homes are ids 8, 9 and 10, priced 84, 42 and 50 lakh, with sizes 150, 95 and 130 m².

COUNT(*): 3 rows, so `homes = 3`.

AVG(price): $(84 + 42 + 50)/3 = 176/3 \approx 58.7$ lakh.

AVG(price/size): $84/150 = 0.560$, $42/95 = 0.442$, $50/130 = 0.385$; their average is $1.387/3 \approx 0.462$ lakh per m².

Note that this is the average of the ratios, not total price divided by total size ($176/375 = 0.469$). Which one you want depends on the question, and mixing them up is a classic reporting error.

Central and Suburb homes have almost the same average price (76.8 and 76.4 lakh), but Central costs far more per square metre (0.889 against 0.692). Averages per group like these are exactly the kind of feature a model can use. The `* 1.0` forces decimal division, because some databases divide whole numbers as whole numbers.

5. Joins: combining tables

A **join** matches rows from two tables using a condition, usually "foreign key equals primary key". The two you'll use most:

- **INNER JOIN** keeps only rows that have a match in *both* tables.
- **LEFT JOIN** keeps *every* row of the left table; where there is no match, the right table's columns are `NULL`.

```
inner = """SELECT h.id, COUNT(v.day) AS n_viewings
           FROM houses h JOIN viewings v ON v.house_id = h.id
           GROUP BY h.id"""
left = """SELECT h.id, COUNT(v.day) AS n_viewings
          FROM houses h LEFT JOIN viewings v ON v.house_id = h.id
          GROUP BY h.id"""
print("inner join rows:", len(pd.read_sql(inner, con)), "  left join rows:",
len(pd.read_sql(left, con)))
print(pd.read_sql(left, con).T)
```

Output:

```
inner join rows: 7   left join rows: 12
            0   1   2   3   4   5   6   7   8   9   10  11
id           1   2   3   4   5   6   7   8   9  10  11  12
n_viewings   2   1   0   3   1   0   0   1   0   0   2   1
```

The inner join returns only the 7 houses that had viewings; the 5 houses with none simply vanish. The left join keeps all 12, with a count of 0 for the unviewed ones. For ML that difference is crucial: an inner join silently drops entities from your training set (often exactly the unusual ones), and "zero viewings" is a valuable feature in its own right. `COUNT(v.day)` counts only non-NULL values, which is why unmatched houses get 0 rather than 1.

> **! THE ROW-EXPLOSION TRAP**
>
> A join produces one row for *every* matching pair. House 4 had 3 viewings, so joining houses to viewings gives house 4 three rows. If you then compute `AVG(price)` over the joined table, house 4's price counts three times. Aggregate the "many" side first (count viewings per house), then join the one-row-per-house result. Always check the row count after a join.

6. Try it: INNER vs LEFT join

Switch between the two joins and turn grouping on and off. Watch which houses disappear, which get `NULL`, and how a house with several viewings is repeated until you group.

Interactive demo — open *lesson-22.html* on the website to try it.

7. Window functions: features from context

A **window function** computes something across a set of related rows *without* collapsing them, as `GROUP BY` would. The `OVER (PARTITION BY ... ORDER BY ...)` clause defines the window.

They're ideal for features such as "rank within its area" or "price compared with the area average":

```
q = """SELECT id, area, price_lakh,
              RANK() OVER (PARTITION BY area ORDER BY price_lakh DESC) AS rank_in_area,
              ROUND(price_lakh - AVG(price_lakh) OVER (PARTITION BY area), 1) AS
vs_area_avg
       FROM houses
       ORDER BY area, rank_in_area"""
print(pd.read_sql(q, con))
```

Output:

```
    id       area  price_lakh  rank_in_area  vs_area_avg
0    3    Central          95             1         18.3
1    2    Central          82             2          5.3
2   11    Central          72             3         -4.8
3    1    Central          58             4        -18.8
4    8  Outskirts          84             1         25.3
5   10  Outskirts          50             2         -8.7
6    9  Outskirts          42             3        -16.7
7   12     Suburb          96             1         19.6
8    5     Suburb          88             2         11.6
9    6     Suburb          79             3          2.6
10   7     Suburb          64             4        -12.4
11   4     Suburb          55             5        -21.4
```

Each house keeps its own row but gains two features: its price rank within its area, and how far its price is above or below its area's average (house 3 is 18.3 lakh above the Central average; house 4 is 21.4 below the Suburb average). Other window functions you'll meet: `ROW_NUMBER()`, `LAG(x)` (the previous row's value, perfect for "change since last month"), and running totals such as `SUM(x) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` for a 7-day rolling sum.

8. The same operations in pandas

Once data is in Python, pandas offers the same operations. Translating between them is a daily task:

SQL	pandas
`WHERE price > 80`	`df[df.price > 80]` or `df.query("price > 80")`
`GROUP BY area` + aggregates	`df.groupby("area").agg(...)`
`LEFT JOIN ... ON`	`left.merge(right, on=..., how="left")`
`AVG(x) OVER (PARTITION BY area)`	`df.groupby("area")["x"].transform("mean")`
`ORDER BY x DESC LIMIT 5`	`df.nlargest(5, "x")`

```
houses = pd.read_sql("SELECT * FROM houses", con)
n_view = viewings.groupby("house_id").size().rename("n_viewings")
merged = houses.merge(n_view, left_on="id", right_index=True,
how="left").fillna({"n_viewings": 0})
summary = merged.groupby("area").agg(homes=("id", "size"), avg_price=("price_lakh",
"mean"),
                                     avg_viewings=("n_viewings", "mean")).round(2)
print(summary)
```

Output:

```
           homes  avg_price  avg_viewings
area
Central        4      76.75          1.25
Outskirts      3      58.67          0.33
Suburb         5      76.40          1.00
```

The same left join (with missing counts filled as 0) and group summary, now in pandas: Central homes average 1.25 viewings, Outskirts homes only 0.33. As a rule of thumb, do heavy filtering and joining in the database, close to the data, and finish in pandas, where plotting and modelling live.

9. Reshaping and cleaning

A few more tools that come up constantly when preparing ML data:

- **Long vs wide.** Event tables are "long" (one row per house per day). Models need "wide" tables (one row per house, one column per feature). `pivot_table` goes from long to wide; `melt` goes back.

- **Types.** Check `df.dtypes`: numbers stored as text, or dates stored as strings, silently break calculations. Use `pd.to_numeric` and `pd.to_datetime`.
- **Duplicates.** Check that keys really are unique, for example with `df.id.is_unique`, before and after every join.
- **Missing values.** Decide deliberately: missing can mean "zero", "unknown" or "not applicable", and those need different handling (Lesson 20).

10. Point-in-time correctness: the rule that prevents leakage

Suppose we want to predict on day 5 whether a house will sell fast, using "number of viewings" as a feature. If we count *all* viewings, including those on days 6 to 9, the feature contains information from the future that the model won't have when it's used for real. That's **leakage** (Lesson 5), and with event data it hides inside innocent-looking joins.

The fix is to put the time condition *in the join*, so each feature uses only events before the prediction time:

```
# Point-in-time features: count only viewings that happened BEFORE the prediction day
predict_day = 5
q = """SELECT h.id, COUNT(v.day) AS viewings_so_far
       FROM houses h LEFT JOIN viewings v
            ON v.house_id = h.id AND v.day < ?
       GROUP BY h.id"""
past = pd.read_sql(q, con, params=(predict_day,))
allv = pd.read_sql(left, con)
print("viewings counted, all time:   ", int(allv.n_viewings.sum()))
print("viewings counted, before day 5:", int(past.viewings_so_far.sum()))
```

Output:

```
viewings counted, all time:    11
viewings counted, before day 5: 6
```

Counting all time gives 11 viewings, but only 6 had happened before day 5. A model trained with the all-time count would look better in testing than it ever could in practice. In real projects each training row has its own **cut-off time** (for example, the date of each loan application), and every feature must be computed from data before that row's cut-off. Feature stores and tools exist to enforce this, but the idea is just this join condition.

11. In Python: from database to training table

A complete pattern, using everything above: aggregate the events per entity with a point-in-time cut-off, left-join onto the entity table, fill true zeros, and hand the result to scikit-learn.

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")
pd.read_csv("houses.csv").to_sql("houses", con, index=False)
pd.DataFrame({"house_id": [1, 1, 2, 4, 4, 4, 5, 8, 11, 11, 12],
              "day": [3, 9, 2, 1, 4, 8, 6, 5, 2, 7, 1]}).to_sql("viewings", con,
index=False)

query = """
WITH v AS (                                -- step 1: summarise events before the cut-
off
    SELECT house_id, COUNT(*) AS n_viewings, MIN(day) AS first_view
    FROM viewings WHERE day < 5 GROUP BY house_id
)
SELECT h.*,                                -- step 2: one row per house, every house
kept
       COALESCE(v.n_viewings, 0) AS n_viewings,
       AVG(h.price_lakh) OVER (PARTITION BY h.area) AS area_avg_price
FROM houses h LEFT JOIN v ON v.house_id = h.id
"""
train = pd.read_sql(query, con)
print(train.shape)
print(train[["id", "area", "n_viewings", "area_avg_price", "sold_fast"]].head())
```

Output:

```
(12, 11)
   id       area  n_viewings  area_avg_price  sold_fast
0   1    Central           1       76.750000          1
1   2    Central           1       76.750000          1
2   3    Central           0       76.750000          0
3  11    Central           1       76.750000          1
4   8  Outskirts           0       58.666667          0
```

The `WITH` clause (a **common table expression**) names an intermediate result so the query reads top to bottom, and `COALESCE` turns missing counts into 0. With real databases (PostgreSQL, MySQL, BigQuery, Snowflake) only the connection line changes. Note one subtle leak left in this example: `area_avg_price` uses every house's price, including the house itself. Inside cross-validation, a group average should be computed from training folds only (Lesson 20's "Going deeper" on target encoding).

12. Running example: Chennai Homes

The Chennai Homes table becomes a database table. One GROUP BY gives the fast-sale rate per area: 3 of 4 Central homes sold fast (0.75), 3 of 5 Suburb homes (0.60) and none of the 3 Outskirts homes. A per-group rate like this is a useful feature, but compute it from training folds only, or it leaks the target (Lesson 20's "Going deeper").

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import sqlite3
import pandas as pd
df = pd.read_csv("houses.csv")   # download it from Lesson 1
con = sqlite3.connect(":memory:")
df.to_sql("houses", con, index=False)
q = """SELECT area, COUNT(*) AS homes, SUM(sold_fast) AS sold_fast,
              ROUND(AVG(sold_fast), 2) AS fast_rate, ROUND(AVG(price_lakh), 1) AS
avg_price
       FROM houses GROUP BY area ORDER BY fast_rate DESC"""
print(pd.read_sql(q, con))
```

Output:

```
        area  homes  sold_fast  fast_rate  avg_price
0    Central      4          3       0.75       76.8
1     Suburb      5          3       0.60       76.4
2  Outskirts      3          0       0.00       58.7
```

⚙ WHERE THIS IS USED

- Building customer, transaction and event features for churn, fraud and recommendation models
- Data-science and ML-engineering interviews, which nearly always include SQL
- Feature pipelines and feature stores in production systems

Choose it when: Whenever your data lives in a database or arrives as several related tables.

Think twice when: the data already arrives as a single clean table and is small enough to handle in pandas.

× MISCONCEPTIONS TO UNLEARN

MYTH Data preparation is a boring step before the real ML work.

REALITY It decides what the model can learn. Wrong joins, dropped rows and leaked future information cause more failed projects than model choice does.

MYTH An INNER JOIN is the default and is always safe.

REALITY It silently drops every entity without a match, which can remove exactly the unusual cases a model most needs to see.

MYTH If the feature exists in the database, the model can use it.

REALITY Only if it was known at prediction time. A value updated after the outcome is leakage, however innocent it looks.

RESPONSIBLE AI · DATA MINIMISATION

Pulling data is the moment to ask whether you need each column at all. Join and keep only what the model needs, avoid copying personal data into notebooks and spreadsheets, and respect the purpose it was collected for. Data-protection laws such as the GDPR and India's DPDP Act make purpose limitation and minimisation legal duties, not just good practice.

13. Questions beginners often ask

Q. SQL or pandas: which should I learn?

Both. SQL is how you get data out of databases, often after the database has done most of the work on millions of rows; pandas is where you finish, explore and model. The operations map one to one.

Q. Which database should I practise on?

SQLite comes with Python and needs no setup, so it's ideal for learning. The core SQL in this lesson works almost unchanged in PostgreSQL, MySQL, BigQuery and Snowflake; they differ mainly in dates and special functions.

Q. Why does my join have more rows than I expected?

A join returns one row for every matching pair. If the right table has several rows per key, each left row is repeated. Aggregate the many side first, or check that the key is unique with COUNT(DISTINCT ...).

Q. What's the difference between COUNT(*) and COUNT(column)?

COUNT(*) counts rows; COUNT(column) counts rows where that column isn't NULL. After a LEFT JOIN, count a column from the right table to get 0 for unmatched rows.

! COMMON MISTAKES TO AVOID

- Using an INNER JOIN and losing every customer who had no events.
- Averaging after a one-to-many join, so entities with many events count many times.
- Writing `WHERE x = NULL` instead of `WHERE x IS NULL`.
- Computing features from the whole history, including events after the prediction date.
- Not checking that keys are unique before joining.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **What's the difference between WHERE and HAVING?**

The trap: Saying they're interchangeable.

WHERE filters individual rows before grouping; HAVING filters groups after aggregation, so it can use aggregates such as `HAVING COUNT(*) > 5`. Filtering rows with WHERE first is also faster, because fewer rows are grouped.

▼ **When would you use a LEFT JOIN instead of an INNER JOIN when building features?**

The trap: Defaulting to INNER JOIN.

When every entity must stay in the training set even if it has no related events: customers with no purchases, houses with no viewings. An INNER JOIN silently drops them, which biases the data and loses a useful "zero activity" signal. After a LEFT JOIN, fill counts with 0 (COALESCE) but think about whether other missing values really mean zero.

▼ **After joining customers to orders, total revenue looks too high. What happened?**

The trap: Blaming the data source.

Probably a one-to-many (or many-to-many) join duplicated rows: each customer row is repeated once per order, and summing a customer-level column (or joining a second many-table) multiplies values. Aggregate the many side to one row per key before joining, and check row counts and key uniqueness before and after every join.

▼ **How do you make sure features built from event logs don't leak?**

The trap: "Use a random train/test split."

Give every training row a cut-off time (when the prediction would have been made) and compute each feature only from events strictly before it, by putting the time condition in the join. Split train and test by time as well. Watch for fields that are updated after the outcome (status columns, "last modified" values) and for aggregates computed over the whole table.

14. Key takeaways

- Real ML data lives in databases spread across tables linked by **primary and foreign keys**; building a training set means joining and summarising them.
- `SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY` : `WHERE` filters rows, `HAVING` filters groups.
- **LEFT JOIN** keeps every entity; **INNER JOIN** silently drops the unmatched ones. Check row counts after every join and aggregate the "many" side first.
- **Window functions** add context features (rank within group, difference from group average, previous value) without collapsing rows.
- pandas has an equivalent for each: boolean filtering, `groupby` , `merge` , `transform` .
- **Point-in-time correctness:** every feature must use only data from before each row's prediction time. Put the time condition in the join.

15. Check your understanding

1. Which clause filters *groups* after aggregation?

a. WHERE

b. HAVING

c. ORDER BY

2. 12 customers, 7 of whom placed orders. How many rows does `customers LEFT JOIN orders` followed by `GROUP BY customer` give?

a. 12

b. 7

c. As many as there are orders

3. You join houses to viewings (one row per viewing) and then take `AVG(price)`. What goes wrong?

a. Nothing

b. Houses without viewings count twice

c. Houses with many viewings count many times

4. Which window-function expression gives each house's price minus its area's average price?

a. `price - AVG(price)` with `GROUP BY area`

b. `price - AVG(price) OVER (PARTITION BY area)`

c. `RANK() OVER (ORDER BY price)`

5. A churn model uses "number of support calls" counted over the customer's whole history, including after the prediction date. What's the problem?

a. Leakage: the feature uses future information

b. The feature is categorical

c. The join is too slow

16. Going deeper: indexes, NULL logic and rolling features

`INTERMEDIATE · SKIP ON A FIRST READ`

Three things that separate working SQL from good SQL: making queries fast on big tables (**indexes** and query plans), understanding how `NULL` really behaves (it trips up even experienced analysts), and building **rolling time-window features** with window frames, which is how many production feature pipelines are written.

You'll need: this lesson.

Indexes and query plans

Without help, the database answers "find house 42's viewings" by reading every row: a **full scan**. An **index** is a sorted lookup structure (usually a B-tree) on one or more columns; with it the database can jump straight to the matching rows in about $\log n$ steps instead of n. `EXPLAIN QUERY PLAN` shows which strategy the database chose:

```
import sqlite3
import numpy as np
import pandas as pd

# A bigger events table: 200,000 viewings of 5,000 houses
rng = np.random.default_rng(0)
con = sqlite3.connect(":memory:")
pd.DataFrame({"house_id": rng.integers(1, 5001, 200000), "day": rng.integers(1, 366,
200000)}).to_sql("viewings", con, index=False)
q = "SELECT COUNT(*) FROM viewings WHERE house_id = 42"
print("plan without an index:", con.execute("EXPLAIN QUERY PLAN " + q).fetchall()[0]
[-1])
con.execute("CREATE INDEX idx_house ON viewings(house_id)")
print("plan with an index:   ", con.execute("EXPLAIN QUERY PLAN " + q).fetchall()[0]
[-1])
print("viewings of house 42:", con.execute(q).fetchone()[0])
```

Output:

```
plan without an index: SCAN viewings
plan with an index:    SEARCH viewings USING COVERING INDEX idx_house (house_id=?)
viewings of house 42: 37
```

On 200,000 viewings, the plan changes from `SCAN` (read everything) to `SEARCH ... USING COVERING INDEX` (jump to house 42's 37 rows). "Covering" means the index alone contains every column the query needs, so the table itself isn't read at all. Index the columns you filter and join on most. Indexes cost disk space and slow down inserts, so don't index everything. On analytical warehouses (BigQuery, Snowflake) the equivalents are partitioning and clustering, and you pay per data scanned, so filtering on the partition column (usually a date) saves real money.

NULL: three-valued logic

In SQL, `NULL` means "unknown", and a comparison with an unknown value is itself unknown, neither true nor false. `WHERE` keeps only rows where the condition is *true*:

```
# NULL is "unknown": comparisons with it are neither true nor false
t = pd.DataFrame({"id": [1, 2, 3, 4], "discount": [10, None, 0, None]})
t.to_sql("t", con, index=False)
conds = {"discount > 5": "discount > 5", "NOT (discount > 5)": "NOT (discount > 5)",
         "discount equals NULL": "discount = NULL", "discount IS NULL": "discount IS
NULL"}
for label, cond in conds.items():
    print(f"{label:21} → rows {[r[0] for r in con.execute(f'SELECT id FROM t WHERE
{cond}')]}")
print("COUNT(*), COUNT(discount), AVG(discount):", con.execute("SELECT COUNT(*),
COUNT(discount), AVG(discount) FROM t").fetchone())
```

Output:

```
discount > 5          → rows [1]
NOT (discount > 5)    → rows [3]
discount equals NULL  → rows []
discount IS NULL      → rows [2, 4]
COUNT(*), COUNT(discount), AVG(discount): (4, 2, 5.0)
```

Row 1 (discount 10) passes `discount > 5`. But `NOT (discount > 5)` returns only row 3, not rows 2, 3 and 4: for the NULL rows, "unknown" negated is still unknown. `discount = NULL` matches nothing at all; only `IS NULL` finds the missing values. Aggregates skip NULLs: `COUNT(discount)` is 2, not 4, and `AVG` averages only the two known values (5.0), which is different from treating missing as 0 (that would give 2.5). Decide explicitly what missing means, and use `COALESCE(discount, 0)` only when it truly means zero.

Rolling features with window frames

A window frame such as `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` limits the window to the current row and the six before it, giving a rolling 7-day total. `LAG` fetches the previous row's value. Together they produce standard time-series features (Lesson 26) directly in the database:

```
# Rolling, point-in-time features with a window frame: viewings in the 7 days up to and
including each day
daily = pd.DataFrame({"house_id": [1] * 10, "day": range(1, 11), "views": [0, 2, 1, 0,
3, 1, 0, 0, 4, 1]})
daily.to_sql("daily", con, index=False)
q = """SELECT day, views,
              SUM(views) OVER (PARTITION BY house_id ORDER BY day ROWS BETWEEN 6
PRECEDING AND CURRENT ROW) AS views_7d,
              LAG(views) OVER (PARTITION BY house_id ORDER BY day) AS views_yesterday
       FROM daily"""
print(pd.read_sql(q, con).to_string(index=False))
```

Output:

```
 day  views  views_7d  views_yesterday
   1      0         0              NaN
   2      2         2              0.0
   3      1         3              2.0
   4      0         3              1.0
   5      3         6              0.0
   6      1         7              3.0
   7      0         7              1.0
   8      0         7              0.0
   9      4         9              0.0
  10      1         9              4.0
```

On day 9, `views_7d` is 9: the views from days 3 to 9 ($1 + 0 + 3 + 1 + 0 + 0 + 4$). Day 1 has no previous day, so `views_yesterday` is NULL (shown as NaN in pandas). Two cautions for ML: if the prediction for day d is made at the *start* of day d, the frame must end at the previous row (`1 PRECEDING`), not `CURRENT ROW`, or the day's own views leak in. And `ROWS` counts rows, not days: with missing days, use `RANGE BETWEEN 6 PRECEDING AND CURRENT ROW` on a numeric day column, or fill in the missing days first.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **A query on a large table is slow. What do you check first?**

The trap: Jumping straight to buying a bigger machine.

Read the query plan (EXPLAIN): is it scanning the whole table? Check that filter and join columns are indexed (or, in a warehouse, that you filter on the partition column), that you select only the needed columns, and that joins aren't exploding rows. Filter early, aggregate before joining, and avoid functions on indexed columns in WHERE, which can stop the index being used.

▼ **Explain how NULL behaves in comparisons, NOT, COUNT and AVG.**

The trap: Treating NULL like zero or like false.

NULL is unknown, so any comparison with it (=, <, >) is unknown, and NOT unknown is still unknown; WHERE keeps only true rows, so NULL rows vanish from both a condition and its negation. Use IS NULL. COUNT(*) counts rows, COUNT(col) counts non-NULLs, and AVG/SUM ignore NULLs. NULL in an IN list or a NOT IN subquery can make NOT IN return no rows at all.

▼ **How would you compute "purchases in the previous 30 days" for every customer-day without leakage?**

The trap: Using a window that includes the current day.

Build a daily table per customer (filling days with no purchases), then use a window frame that ends before the current row, e.g. `SUM(purchases) OVER (PARTITION BY customer ORDER BY day ROWS BETWEEN 30 PRECEDING AND 1 PRECEDING)`, or a RANGE frame on dates. Validate on a time-based split. The same logic must run identically in training and in production, which is what feature stores provide.

Check your understanding: going deeper

1. What does an index on `viewings(house_id)` let the database do?

a. Jump straight to a house's rows instead of scanning the whole table

b. Store the table twice

c. Remove duplicate rows

2. A column has values 10, NULL, 0, NULL. How many rows does `WHERE NOT (x > 5)` return?

a. 3

b. 1

c. 2

3. What is `AVG(x)` for the values 10, NULL, 0, NULL?

a. 2.5

b. 5

c. NULL

4. A prediction for day d is made at the start of day d. Which frame avoids leakage in a 7-day sum?

a. ROWS BETWEEN 6 PRECEDING AND CURRENT ROW

b. ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING

c. ROWS BETWEEN CURRENT ROW AND 6 FOLLOWING

5. Why might `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` not give a 7-day window?

a. If some days have no row, it covers more than 7 calendar days

b. ROWS always means 7 days

c. It includes future rows

17. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

`EXERCISE 1` By hand

An orders table has 6 rows: customer A spent 20 and 30, customer B spent 50, and customer C spent 10, 15 and 5. What does `SELECT customer, COUNT(*), SUM(amount) FROM orders GROUP BY customer HAVING SUM(amount) > 40` return?

WORKING

EXERCISE 2 By hand

Customers: 10 rows. Orders: 25 rows, belonging to 6 of the customers. How many rows do `customers INNER JOIN orders` and `customers LEFT JOIN orders` return?

WORKING

EXERCISE 3 By hand

Prices in one area are 60, 80 and 100. What value does `price - AVG(price) OVER (PARTITION BY area)` give for each house?

WORKING

EXERCISE 4 Concepts

You're building a churn model. Each customer's prediction date is the first day of the month. Which of these features leak: (a) number of logins in the previous 30 days, (b) total refunds this year, (c) whether the customer's account was closed?

WORKING

EXERCISE 5 Code

In the lesson's database, write a query that returns each area with its number of houses that had at least one viewing.

WORKING

EXERCISE 6 Intermediate

Daily views are 2, 0, 5, 1, 3 for days 1–5. Using a frame that ends at the previous row and covers 3 rows (`ROWS BETWEEN 3 PRECEDING AND 1 PRECEDING`), what is the feature for day 5?

WORKING

EXERCISE 7 Intermediate

A column holds 4, NULL, 6, NULL, 8. What are `COUNT(*)`, `COUNT(x)` and `AVG(x)`?

WORKING

EXERCISE 8 Intermediate

Roughly how many steps does a balanced B-tree lookup need on 1,000,000 rows if each node has 100 children? Compare with a full scan.

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~55 MIN

Lesson 23: Hyperparameter Tuning & Model Selection

Validation curves, grid and random search, Bayesian optimisation, successive halving, nested CV and choosing the simplest good model.

BY THE END OF THIS LESSON YOU CAN

- Tell parameters from hyperparameters and read a validation curve
- Run grid and random search over a pipeline, and explain why random search usually wins
- Describe Bayesian optimisation and successive halving, and when to use them
- Report a tuned model's performance honestly and pick the simplest good model

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 10: Overfitting, Regularization & Bias–Variance — Error = bias² + variance + noise. Regularization adds a penalty on weight size (L2 ridge shrinks weights; L1 lasso zeros some) to reduce variance; its strength is chosen by cross-validation.
- Lesson 15: Support Vector Machines —

Where this fits: Turns the scattered advice on choosing k, λ, depth, C and γ from Lessons 10–15 into one systematic, honest procedure for any model.

1. Parameters vs hyperparameters

A model has two kinds of numbers. **Parameters** are learned from the data during training: the weights of a linear model, the split points of a tree. **Hyperparameters** are chosen *before* training and control how learning happens: the regularization strength λ (Lesson 10), the number of neighbours k (Lesson 11), a tree's maximum depth (Lesson 13), the SVM's C and γ (Lesson 15), the learning rate (Lesson 7).

Good hyperparameters often matter more than the choice of algorithm. But every time you try a setting and look at a score, you learn a little about the evaluation data, so careless tuning produces scores that are too good to be true. This lesson covers how to search well (grid, random, Bayesian and successive halving), how to report honestly (nested cross-validation), and how to choose between models.

> ◎ THE CORE IDEA
>
> Tuning is itself a learning process, so it needs its own unseen data. Choose hyperparameters using cross-validation on the training set only, search the space efficiently, and keep a final test set (or an outer cross-validation loop) that no tuning decision ever touched.

2. Seeing one hyperparameter at a time: validation curves

A **validation curve** shows training and cross-validated scores as one hyperparameter changes. It makes underfitting and overfitting visible. Here the RBF SVM's γ (how local each training point's influence is) varies on the breast-cancer data:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import validation_curve
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), SVC())
gammas = [0.0001, 0.001, 0.01, 0.1, 1]
train, val = validation_curve(model, X, y, param_name="svc__gamma", param_range=gammas,
cv=5)
for g, a, b in zip(gammas, train.mean(axis=1), val.mean(axis=1)):
    print(f"gamma {g:<6}: train accuracy {a:.3f}   cross-validated accuracy {b:.3f}")
```

Output:

```
gamma 0.0001: train accuracy 0.794   cross-validated accuracy 0.796
gamma 0.001 : train accuracy 0.951   cross-validated accuracy 0.947
gamma 0.01  : train accuracy 0.978   cross-validated accuracy 0.968
gamma 0.1   : train accuracy 0.990   cross-validated accuracy 0.960
gamma 1     : train accuracy 1.000   cross-validated accuracy 0.631
```

At $\gamma = 0.0001$ both scores are low (0.794 and 0.796): the boundary is too smooth, so the model underfits. At $\gamma = 1$ training accuracy is perfect but cross-validated accuracy collapses to 0.631:

each point's influence is tiny, so the model memorises. The best region is in between ($\gamma = 0.01$, 0.968). Values span orders of magnitude, so always try hyperparameters like this on a **log scale**.

3. Grid search

Grid search tries every combination of a list of values for each hyperparameter, scores each with cross-validation, and keeps the best. Scikit-learn's `GridSearchCV` then refits the best setting on all the training data. Hyperparameters of a step inside a pipeline are named `step__parameter`, as in `svc__C`.

```
from sklearn.model_selection import GridSearchCV, train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0,
stratify=y)
grid = {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.0001, 0.001, 0.01, 0.1]}
search = GridSearchCV(model, grid, cv=5).fit(X_tr, y_tr)
print("settings tried:", len(search.cv_results_["params"]))
print("best settings:", search.best_params_)
print("best cross-validated accuracy:", round(search.best_score_, 3))
print("test accuracy of the refitted best model:", round(search.score(X_te, y_te), 3))
```

Output:

```
settings tried: 16
best settings: {'svc__C': 100, 'svc__gamma': 0.001}
best cross-validated accuracy: 0.988
test accuracy of the refitted best model: 0.958
```

✎ WORKED EXAMPLE — HOW MUCH WORK IS A GRID SEARCH?

The grid above has 4 values of C and 4 values of γ: $4 \times 4 = 16$ settings.

Each is scored with 5-fold cross-validation, so $16 \times 5 = 80$ models are trained, plus 1 final refit of the best setting: **81 fits**.

Add a third hyperparameter with 5 values and it becomes $16 \times 5 \times 5 + 1 = 401$ fits. With 5 hyperparameters of 5 values each it's $5^5 \times 5 + 1 = 15{,}626$ fits. The grid's size multiplies with every hyperparameter you add: the curse of dimensionality again (Lesson 11).

The best cross-validated accuracy is 0.988, but the refitted model scores 0.958 on the untouched test set. Part of that gap is chance, because the test set has only 143 cases, and part is the optimism of picking the best of 16 settings (Lesson 5's "Going deeper").

4. Random search

Random search samples each hyperparameter from a distribution, typically `loguniform` for anything that spans orders of magnitude. With the same budget it usually matches or beats a grid, because in most problems only one or two hyperparameters really matter: a 4×4 grid tries just 4 values of each, while 16 random draws try 16 different values of every one (Bergstra and Bengio, 2012).

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

dists = {"svc__C": loguniform(0.01, 1000), "svc__gamma": loguniform(1e-5, 1)}
rs = RandomizedSearchCV(model, dists, n_iter=16, cv=5, random_state=0).fit(X_tr, y_tr)
print("best settings:", {k: round(float(v), 4) for k, v in rs.best_params_.items()})
print("best cross-validated accuracy:", round(rs.best_score_, 3), "  test accuracy:",
round(rs.score(X_te, y_te), 3))
```

Output:

```
best settings: {'svc__C': 1.313, 'svc__gamma': 0.017}
best cross-validated accuracy: 0.981   test accuracy: 0.951
```

Sixteen random draws find a setting with cross-validated accuracy 0.981 and test accuracy 0.951, about the same as the grid. On this problem both work; random search's advantage grows as you add hyperparameters, because its budget doesn't multiply.

5. Try it: grid vs random search

The landscape below has one important hyperparameter (left to right) and one that barely matters (bottom to top). The bright band is where validation scores are high. Change the budget and draw new random points: notice how few distinct values of the important hyperparameter the grid tests.

Interactive demo — open *lesson-23.html* on the website to try it.

6. Smarter searches: Bayesian optimisation and successive halving

Bayesian optimisation uses the results so far to choose what to try next. It fits a cheap *surrogate* model of "score as a function of hyperparameters" (often a Gaussian process or, in Optuna's TPE, density estimates of good and bad settings), then picks the next setting that best balances *exploitation* (near the current best) and *exploration* (where the surrogate is uncertain). It shines when each training run is expensive. This lesson's "Going deeper" builds a tiny version.

Successive halving saves time differently: start many candidates on a small budget (few rows, few epochs or few trees), keep the best third, give them three times the budget, and repeat:

```
from sklearn.experimental import enable_halving_search_cv  # noqa: F401  (switches the
feature on)
from sklearn.model_selection import HalvingRandomSearchCV

hs = HalvingRandomSearchCV(model, dists, n_candidates=48, factor=3, cv=5,
random_state=0).fit(X_tr, y_tr)
for it, (n_cand, n_res) in enumerate(zip(hs.n_candidates_, hs.n_resources_)):
    print(f"round {it + 1}: {n_cand:2d} candidates, each trained on {n_res} rows")
print("best cross-validated accuracy:", round(hs.best_score_, 3))
```

Output:

```
round 1: 48 candidates, each trained on 20 rows
round 2: 16 candidates, each trained on 60 rows
round 3:  6 candidates, each trained on 180 rows
best cross-validated accuracy: 0.978
```

48 candidates start on just 20 rows each; 16 survive to 60 rows; 6 reach 180 rows, and the best scores 0.978 in cross-validation. Most of the 48 cheap first-round fits used tiny data, so the whole search costs far less than training 48 candidates fully. **Hyperband** runs several halving schedules with different starting budgets, and early stopping in boosting and neural networks (Lesson 10's "Going deeper") applies the same idea within a single run.

7. Reporting honestly: nested cross-validation

The best score found *during* a search is optimistic, because it's the best of many tries on the same folds. To report how well "tune, then fit" works on new data, wrap the whole search inside an outer cross-validation loop (Lesson 5's "Going deeper"):

```
from sklearn.model_selection import cross_val_score, KFold

inner, outer = KFold(5, shuffle=True, random_state=1), KFold(5, shuffle=True,
random_state=2)
tuned = GridSearchCV(model, grid, cv=inner)
nested = cross_val_score(tuned, X, y, cv=outer)
print("nested CV accuracy:", round(nested.mean(), 3), "+/-", round(nested.std(), 3))
```

Output:

```
nested CV accuracy: 0.979 +/- 0.007
```

Nested CV estimates 0.979 accuracy (±0.007 across the outer folds) for the complete tuning procedure. Use it, or a single held-out test set used once, as the number you report. The model

you actually deploy is still produced by running the search once on all the training data.

8. Choosing between models: the simplest good one

Cross-validated scores are noisy, so the setting with the very highest mean is often not truly better than its neighbours. The **one-standard-error rule** picks the *simplest* model whose score is within one standard error of the best. Simpler models are cheaper, easier to explain, and usually more robust when the data shifts.

```
from sklearn.tree import DecisionTreeClassifier

# The one-standard-error rule: prefer the simplest model within 1 SE of the best
depths = list(range(1, 11))
res = GridSearchCV(DecisionTreeClassifier(random_state=0), {"max_depth": depths},
cv=10).fit(X_tr, y_tr).cv_results_
mean, se = res["mean_test_score"], res["std_test_score"] / np.sqrt(10)
best = int(np.argmax(mean))
simplest = next(i for i in range(len(depths)) if mean[i] ⩾ mean[best] - se[best])
print("best depth by score:", depths[best], " CV accuracy", round(mean[best], 3), " SE",
round(se[best], 3))
print("simplest depth within 1 SE:", depths[simplest], " CV accuracy",
round(mean[simplest], 3))
```

Output:

```
best depth by score: 5  CV accuracy 0.941  SE 0.012
simplest depth within 1 SE: 4  CV accuracy 0.929
```

Depth 5 scores highest (0.941), with a standard error of 0.012, but depth 4 scores 0.929, within one standard error, so the rule prefers the shallower tree. Other things to weigh when picking a final model: prediction speed and cost, calibration (Lesson 8's "Going deeper"), fairness across groups (Lesson 21), and how easy it will be to maintain and retrain.

9. In Python: a tuning recipe

A pattern that works for most projects: put all preprocessing in a pipeline, use random search with log-scale distributions and stratified folds, and report on a test set used once.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold,
train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from scipy.stats import loguniform

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0,
stratify=y)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
search = RandomizedSearchCV(pipe, {"logisticregression__C": loguniform(1e-3, 1e3)},
                            n_iter=30, cv=StratifiedKFold(5, shuffle=True,
random_state=0),
                            scoring="roc_auc", random_state=0).fit(X_tr, y_tr)
print("best C:", round(float(search.best_params_["logisticregression__C"]), 4))
print("CV ROC-AUC:", round(search.best_score_, 4), "  test ROC-AUC:",
round(search.score(X_te, y_te), 4))
```

Output:

```
best C: 0.3483
CV ROC-AUC: 0.9954   test ROC-AUC: 0.9943
```

`scoring="roc_auc"` makes the search optimise the metric you care about (Lesson 9); `search.score` then uses the same metric on the test set. Libraries such as **Optuna** add Bayesian optimisation, pruning of bad trials and dashboards, and follow the same pattern.

⚙ WHERE THIS IS USED

- Getting the most out of gradient boosting, SVMs and neural networks
- Fair comparisons between candidate models before deployment
- AutoML systems, which automate exactly this search

Choose it when: Whenever a model has settings that strongly affect its accuracy, which is almost always.

Think twice when: the dataset is tiny: heavy tuning on little data mostly fits noise, so prefer simple models with sensible defaults.

× MISCONCEPTIONS TO UNLEARN

MYTH The best cross-validated score from a search is what the model will achieve.

REALITY It's the best of many tries, so it's optimistic. Report nested CV or an untouched test set.

MYTH Grid search is more thorough than random search.

REALITY It's exhaustive only over the few values you listed. For the same budget random search tests many more values of each hyperparameter.

MYTH More tuning always helps.

REALITY On small datasets heavy tuning fits the noise in the validation folds. Gains below the CV standard error are usually not real.

RESPONSIBLE AI · TUNE FOR THE RIGHT OBJECTIVE

A search optimises exactly the metric you give it and nothing else. If the overall score improves while performance for a small group gets worse, the search won't notice. Include subgroup metrics and calibration in the final comparison (Lesson 21), and record what was tried, so results can be reproduced and checked.

10. Questions beginners often ask

Q. Which hyperparameters should I tune first?

The ones with the biggest effect: regularization strength (λ, C), learning rate and the number of iterations or trees (with early stopping), and model capacity (depth, number of leaves, k, γ). Library defaults for the rest are usually fine.

Q. How many random-search trials do I need?

A useful fact: with 60 random trials, there's a 95% chance at least one lands in the best 5% of the search space (because $1 - 0.95^{60} \approx 0.95$). Start with 20–60 and increase if results are still improving.

Q. Is it OK to tune on the test set if I'm careful?

No. Any decision based on test results makes them optimistic. Tune with cross-validation on training data and look at the test set once, at the end.

Q. Should I tune the preprocessing too?

Yes, if it matters: imputation strategy, the number of PCA components or polynomial degree can all go in the same search, using pipeline names like `pca__n_components`.

! COMMON MISTAKES TO AVOID

- Choosing hyperparameters by looking at test-set scores.
- Searching learning rates or penalties on a linear scale instead of a log scale.
- Scaling or selecting features outside the cross-validation loop, so tuning leaks.
- Reporting the best search score as the final performance.
- Picking a complex model for a gain smaller than the score's standard error.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **What's the difference between a parameter and a hyperparameter?**

The trap: Using the words interchangeably.

Parameters are learned from data during training (weights, split points). Hyperparameters are set before training and control the learning (λ, k, depth, learning rate, number of trees). Hyperparameters are chosen by validation, not by the training loss, since the training loss would always prefer the most flexible setting.

▼ **Why does random search often beat grid search?**

The trap: "Randomness explores better" with no mechanism.

Usually only a few hyperparameters really matter. A grid with m values per dimension tests only m distinct values of the important ones however large it gets, whereas n random trials test n distinct values of each. Random search also scales to many hyperparameters without the multiplicative blow-up, and any number of trials can be run.

▼ **Your tuned model scored 0.95 in grid search but 0.90 on the test set. Why might that be?**

The trap: Only saying "overfitting".

The best search score is optimistically biased: it's the maximum over many noisy estimates on the same folds. The test set is also a finite sample with its own noise. Other causes: leakage inside the search (preprocessing fitted outside the pipeline), or a test set from a different period or distribution. Use nested CV for an honest estimate and check the score's confidence interval.

▼ **How does Bayesian optimisation decide what to try next?**

The trap: Saying it's "like random search but smarter".

It fits a surrogate model of the validation score as a function of the hyperparameters (a Gaussian process, a tree ensemble or TPE's densities) and maximises an acquisition function, such as expected improvement, that rewards both a high predicted score and high uncertainty. Each new result updates the surrogate. It pays off when each trial is expensive.

11. Key takeaways

- **Hyperparameters** are set before training and control how a model learns; they're chosen with cross-validation on training data only.
- **Validation curves** show underfitting and overfitting as one hyperparameter changes. Search scale-type hyperparameters on a log scale.
- **Grid search** grows multiplicatively with each hyperparameter; **random search** covers important hyperparameters far better for the same budget.
- **Bayesian optimisation** chooses the next try from past results; **successive halving** gives many candidates a small budget and the best ones more.
- The best search score is optimistic: report **nested CV** or a test set used once.
- Prefer the **simplest** model within about one standard error of the best.

12. Check your understanding

1. Which of these is a hyperparameter?

a. A linear model's weights

b. The number of neighbours k in kNN

c. A tree's split thresholds

2. A grid has 3 values for each of 4 hyperparameters and uses 5-fold CV. How many models are trained, not counting the final refit?

a. 60

b. 81

c. 405

3. Why does random search usually beat grid search with the same budget?

a. It tries many more distinct values of the hyperparameters that matter

b. It always finds the global best

c. It needs no cross-validation

4. A grid search reports a best CV accuracy of 0.95. What is the honest estimate of the tuned model's accuracy?

a. Exactly 0.95

b. Usually a bit lower; measure it with nested CV or an untouched test set

c. Higher than 0.95

5. Depths 4 and 6 score 0.929 and 0.935, and the standard error at the best is 0.012. The one-standard-error rule picks...

a. Depth 4, the simpler model within one SE of the best

b. Depth 6, the highest score

c. Neither

13. Going deeper: Bayesian optimisation from scratch

INTERMEDIATE · SKIP ON A FIRST READ

This lesson described Bayesian optimisation in words. Here it is in about fifteen lines: a **Gaussian process** surrogate that predicts the validation score *and its uncertainty* for any setting, and the **expected improvement** rule that decides where to try next. Then we measure whether it actually beats random search.

You'll need: this lesson, the normal distribution from Lesson 4 and kernels from Lesson 15.

The surrogate: a Gaussian process

A **Gaussian process** (GP) is a probability distribution over smooth functions. Given a few observed points, it predicts a mean $\mu(x)$ and a standard deviation $\sigma(x)$ at every other point: close

to observed points it's confident, far from them it's uncertain. How smooth it assumes the function to be is set by a kernel, the same idea as the SVM's kernels (Lesson 15). Here we use the common Matérn kernel, and the GP learns its length scale from the data.

Choosing the next trial: expected improvement

Let f^* be the best score found so far. At a candidate x, the GP says the score is roughly normal with mean μ and standard deviation σ. The **expected improvement** is the average amount by which it would beat f^*, counting no improvement as zero:

$$\mathrm{EI}(x) = (\mu - f^*)\,\Phi(z) + \sigma\,\phi(z), \qquad z = \frac{\mu - f^*}{\sigma}$$

where Φ and ϕ are the standard normal CDF and density. The first term rewards a high predicted score (**exploitation**); the second rewards high uncertainty (**exploration**). Each round: fit the GP to all results so far, pick the x with the largest EI, evaluate it, and repeat.

Below, the "validation score" is a made-up curve over $\log_{10}(\text{learning rate})$ from -5 to 0 with a narrow peak. Both methods start fresh 20 times, and we report the average gap between the best score found and the true best:

```
import numpy as np
import warnings
warnings.filterwarnings("ignore")          # the GP optimiser warns about its bounds on
tiny data
from scipy.stats import norm
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern, ConstantKernel

# A made-up "validation score vs log10(learning rate)" curve with its peak near x = -2.3
f = lambda x: np.exp(-(x + 2.3) ** 2 / 0.3) * 0.3 + 0.65 + 0.02 * np.sin(5 * x)
grid = np.linspace(-5, 0, 501)[:, None]

def bayes_opt(n_iter, seed):
    rng = np.random.default_rng(seed)
    X = rng.uniform(-5, 0, size=(3, 1)); y = f(X[:, 0])               # 3 random
starting points
    kernel = ConstantKernel(1.0, (1e-3, 1e3)) * Matern(length_scale=0.5,
length_scale_bounds=(0.05, 5), nu=2.5)
    gp = GaussianProcessRegressor(kernel, alpha=1e-6, normalize_y=True,
n_restarts_optimizer=3, random_state=0)
    for _ in range(n_iter - 3):
        gp.fit(X, y)
        mu, sd = gp.predict(grid, return_std=True)
        z = (mu - y.max()) / np.maximum(sd, 1e-9)
        ei = (mu - y.max()) * norm.cdf(z) + sd * norm.pdf(z)             # expected
improvement
        x_next = grid[np.argmax(ei)]
        X = np.vstack([X, x_next]); y = np.append(y, f(x_next[0]))
    return y.max()

def random_search(n_iter, seed):
    return f(np.random.default_rng(seed).uniform(-5, 0, n_iter)).max()

best = f(grid[:, 0]).max()
for n in [5, 10, 15]:
    bo = np.mean([best - bayes_opt(n, s) for s in range(20)])
    rs = np.mean([best - random_search(n, s) for s in range(20)])
    print(f"{n:2d} trials: average gap to the best score, Bayesian {bo:.4f}   random
{rs:.4f}")
```

Output:

```
 5 trials: average gap to the best score, Bayesian 0.1297   random 0.1190
10 trials: average gap to the best score, Bayesian 0.0103   random 0.0510
15 trials: average gap to the best score, Bayesian 0.0000   random 0.0261
```

With only 5 trials, Bayesian optimisation is no better than random (0.130 against 0.119): 3 of those trials are random starting points, and two guided steps aren't enough. By 10 trials it is five times closer to the best (0.0103 against 0.0510), and by 15 it finds the peak every time, while random search is still 0.026 away. That's the pattern in practice: Bayesian optimisation helps most when trials are expensive and you can afford only tens of them, in a low-dimensional space.

In practice

- **Dimensions.** GPs work best with up to about 10–20 hyperparameters. Optuna's default, TPE (tree-structured Parzen estimator), copes better with many hyperparameters and with categorical and conditional ones (for example "number of layers", then a width per layer).
- **Noise.** Validation scores are noisy, so the surrogate should model noise (a white-noise kernel term, or `alpha`), or it will chase lucky trials.
- **Parallelism.** Random search parallelises trivially; Bayesian optimisation is sequential by nature, and needs tricks to propose several trials at once.
- **Pruning.** Combining a smart sampler with early stopping of bad trials (Optuna's pruners, or Hyperband) usually gives the biggest savings.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Explain expected improvement and the exploration–exploitation trade-off it encodes.**

The trap: Describing it only as "pick the best predicted point".

Given the surrogate's normal prediction (μ, σ) and the best score f^*, EI is the expected amount by which a trial beats f^*: $(\mu - f^*)\Phi(z) + \sigma\phi(z)$ with $z = (\mu - f^*)/\sigma$. The first term favours points predicted to be good (exploitation); the second favours uncertain points (exploration). Points with a low mean but high uncertainty can still be chosen, which prevents getting stuck near an early lucky result.

▼ **When would you choose Optuna's TPE over a Gaussian-process optimiser?**

The trap: "TPE is always better."

When there are many hyperparameters, categorical or conditional ones (a choice of model with its own settings), or many trials: TPE scales to these, while GP cost grows cubically with the number of trials and GPs struggle beyond about 20 dimensions. A GP can be more sample-efficient for a few continuous hyperparameters and very expensive trials.

Check your understanding: going deeper

1. What does a Gaussian process surrogate predict at an untried setting?

a. Only the score

b. A mean score and a standard deviation (its uncertainty)

c. The best learning rate

2. In expected improvement, the term $\sigma\,\phi(z)$ rewards...

a. Exploitation of high predicted scores

b. Exploration of uncertain regions

c. Fast training

3. When does Bayesian optimisation help most compared with random search?

a. Cheap trials, hundreds of hyperparameters

b. Expensive trials, a few hyperparameters, a small budget

c. Never

4. Why is basic Bayesian optimisation hard to run in parallel?

a. Each choice depends on the results of the previous trials

b. GPs can't use more than one CPU

c. It needs a GPU

5. If the best so far is $f^* = 0.90$ and a candidate has $\mu = 0.90$, $\sigma = 0.02$, its expected improvement is about...

a. 0

b. 0.008

c. 0.02

14. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

A grid has 5 values of C, 4 of γ and 2 kernels, with 10-fold cross-validation. How many fits does it run, including the final refit?

WORKING

EXERCISE 2 By hand

How many random trials do you need for a 90% chance that at least one lands in the best 5% of the space? (Solve $1 - 0.95^n \geq 0.9$.)

WORKING

EXERCISE 3 By hand

Successive halving starts 81 candidates with factor 3, each on 10 rows. How many candidates and rows per candidate are there in rounds 2, 3 and 4?

WORKING

EXERCISE 4 By hand

Cross-validated scores: depth 2: 0.90, depth 3: 0.93, depth 5: 0.94 (the best, standard error 0.015), depth 8: 0.935. Which depth does the one-standard-error rule pick?

WORKING

EXERCISE 5 Code

Change the lesson's tuning recipe to optimise accuracy instead of ROC-AUC (`scoring="accuracy"`). Does the chosen C change?

WORKING

EXERCISE 6 Intermediate

The best score so far is 0.80. A candidate has $\mu = 0.82$ and $\sigma = 0.02$. Compute z and the expected improvement, using $\Phi(1) = 0.8413$ and $\phi(1) = 0.2420$.

WORKING

EXERCISE 7 Intermediate

Two candidates: A has $\mu = 0.85$, $\sigma = 0.001$; B has $\mu = 0.83$, $\sigma = 0.05$. The best so far is 0.85. Which has the higher expected improvement? (Use $\phi(0) = 0.399$, $\Phi(-0.4) = 0.345$, $\phi(-0.4) = 0.368$.)

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~60 MIN

Lesson 24: Interpretability & Explainable ML

Explain models globally and locally: standardised weights, permutation importance, partial dependence and ICE, SHAP values from scratch, LIME, and the traps.

BY THE END OF THIS LESSON YOU CAN

- Tell global from local explanations and choose a method for each question
- Compute and read permutation importance and partial-dependence / ICE plots
- Compute Shapley values by hand for a small model and explain their fairness properties
- Spot misleading explanations: correlated features, unrealistic inputs and importance mistaken for causation

↺ BEFORE YOU START

This lesson builds on:

- Lesson 6: Linear Regression — Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ and chooses the weights that minimise the mean squared error (least squares). R^2 measures the share of variance explained.
- Lesson 13: Decision Trees — A decision tree asks a sequence of 'feature ≤ threshold?' questions chosen to reduce impurity (Gini or entropy). Deep trees overfit and are unstable.
- Lesson 14: Random Forests & Boosting —

Where this fits: Opens up the models of Parts 2–5. It builds on standardised weights (Lesson 6), tree importances and their flaws (Lessons 13–14) and turns them into trustworthy explanations.

1. Why explain a model?

A bank's model refuses a loan. The applicant asks why. The regulator asks how the model decides. The data scientist wants to know whether it learned something real or a shortcut, such as a hospital's scanner model that "detects" pneumonia by reading the hospital's name printed on the X-ray. Accuracy answers none of these questions.

Interpretability methods answer two kinds of question:

- **Global:** how does the model behave overall? Which features matter most, and how does the prediction change as a feature changes?

- **Local:** why did the model make *this* prediction for *this* person?

This lesson covers the methods practitioners use most: reading linear weights properly, permutation importance, partial-dependence and ICE plots, SHAP values (built from scratch, so there's no mystery), and LIME-style local surrogates, along with the traps that make explanations misleading.

> ◎ THE CORE IDEA
>
> An explanation is itself a simplified model of the model. Global methods summarise behaviour across all the data; local methods describe one prediction. Every method answers a specific, narrow question, and none of them says what *causes* the outcome in the real world: only what the *model* relies on.

2. Reading linear models properly

For linear and logistic regression the weights are the explanation, but only if the features are on comparable scales. A weight per millimetre and a weight per kilometre can't be compared. Standardise first (Lesson 10), and each weight becomes "the change in log-odds for a one-standard-deviation increase in the feature":

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = load_breast_cancer()
X, y, names = data.data, data.target, data.feature_names      # target 1 = benign
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0,
stratify=y)
lin = make_pipeline(StandardScaler(), LogisticRegression(C=0.1,
max_iter=5000)).fit(X_tr, y_tr)
coef = pd.Series(lin[-1].coef_[0], index=names)
print("largest standardised weights (positive pushes towards benign):")
print(coef.reindex(coef.abs().sort_values(ascending=False).index).head(5).round(3))
```

Output:

```
largest standardised weights (positive pushes towards benign):
worst texture          -0.543
worst radius           -0.529
worst perimeter        -0.496
worst concave points   -0.469
worst area             -0.463
dtype: float64
```

On the breast-cancer data (target 1 = benign), the five largest standardised weights are all negative: larger "worst" tumour measurements push the prediction away from benign. Three cautions from earlier lessons apply. Correlated features share and swap weight (Lesson 6's "Going deeper": radius, perimeter and area are almost the same measurement here). Regularization shrinks weights. And a weight describes association in this model, not cause.

3. Permutation importance

For any model, **permutation importance** asks: how much worse does the model get if this feature's information is destroyed? Shuffle one column of the *test* data, which breaks its link to the target but keeps its distribution, and measure the drop in score. Repeat with several shuffles and average. Lesson 13's "Going deeper" showed why this is more honest than a tree's built-in impurity importance.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance

rf = RandomForestClassifier(n_estimators=300, random_state=0).fit(X_tr, y_tr)
print("test accuracy:", round(rf.score(X_te, y_te), 3))
perm = permutation_importance(rf, X_te, y_te, n_repeats=30, random_state=0)
imp = pd.Series(perm.importances_mean, index=names).sort_values(ascending=False)
print("permutation importance on test data (drop in accuracy):")
print(imp.head(5).round(4))
```

Output:

```
test accuracy: 0.953
permutation importance on test data (drop in accuracy):
worst radius            0.0211
worst area              0.0177
worst concave points    0.0156
worst texture           0.0131
worst perimeter         0.0131
dtype: float64
```

The random forest's accuracy falls most when "worst radius" is shuffled (by 0.021, about 2 percentage points). The drops are small because the features are highly correlated: when one is shuffled, the forest can lean on a near-copy. That's the main weakness of permutation importance; the fix is to shuffle groups of correlated features together, or to read the results as "unique information this feature adds".

4. Partial dependence and ICE plots

Importance says *how much* a feature matters, not *how*. A **partial-dependence plot** (PDP) shows the average prediction as one feature is varied: set the feature to a value v for *every* row, predict, average, and repeat for a grid of values:

$$\mathrm{PD}(v) = \frac{1}{n}\sum_{i=1}^{n} f\big(v,\ \mathbf{x}_{i,\mathrm{other}}\big)$$

An **ICE plot** (individual conditional expectation) draws one such curve per row instead of their average, revealing whether different people respond differently.

```
from sklearn.datasets import load_diabetes
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.inspection import partial_dependence

d = load_diabetes()
Xd, yd = d.data, d.target
gbr = GradientBoostingRegressor(random_state=0).fit(Xd, yd)
bmi = list(d.feature_names).index("bmi")
pd_res = partial_dependence(gbr, Xd, [bmi], grid_resolution=6, kind="both")
print("bmi (scaled):        ", np.round(pd_res["grid_values"][0], 3))
print("average prediction:  ", np.round(pd_res["average"][0], 1))
ice = pd_res["individual"][0]
print("ICE spread at each bmi value (std across patients):", np.round(ice.std(axis=0),
1))
```

Output:

```
bmi (scaled):         [-0.067 -0.036 -0.005  0.025  0.056  0.087]
average prediction:   [130.9 129.1 141.7 176.4 177.  209.7]
ICE spread at each bmi value (std across patients): [45.2 40.6 49.  42.6 45.4 45.8]
```

For a gradient-boosting model of diabetes progression, the average prediction rises from about 131 to 210 as BMI goes from low to high: a clear, mostly increasing effect, with a jump in the middle of the range. The ICE curves are spread out by about 40–49 units at every BMI value, so patients' baseline predictions differ a lot even though the BMI effect is similar. One warning: a PDP sets BMI to every value for every patient, including combinations that don't occur in real data (say,

very high BMI with very low weight-related blood measures). With strongly correlated features, prefer **ALE plots** (accumulated local effects), which only use realistic nearby values.

5. SHAP values: fair credit for each feature

For one prediction, we'd like to split "prediction minus the average prediction" into a contribution from each feature. The fair way to do it comes from cooperative game theory: the **Shapley value**. Treat features as players and the prediction as the payout. A feature's Shapley value is its average extra contribution when it joins, averaged over every possible order in which the features could join:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \Big[v(S \cup \{j\}) - v(S) \Big]$$

Here $v(S)$ is the model's expected prediction when only the features in S are known; the unknown ones are filled in from a background sample. Shapley values are the only way of splitting the credit that is fair in a precise sense:

- **Efficiency:** the contributions add up exactly to prediction minus baseline.
- **Symmetry:** two features that always contribute equally get equal credit.
- **Dummy:** a feature that never changes the prediction gets 0.

✎ WORKED EXAMPLE — SHAPLEY VALUES BY HAND

A tiny model: $f = 2x_1 + 3x_2 + 4x_1x_3$, explained at $\mathbf{x} = (1, 0.5, 1)$ against the baseline $(0, 0, 0)$, where $f = 0$. So the total to share is $f(\mathbf{x}) = 2 + 1.5 + 4 = 7.5$.

x_2 enters the model on its own, so it adds $3 \times 0.5 = 1.5$ whenever it joins: $\phi_2 = 1.5$.

x_1 always adds its own 2. It also adds the interaction 4, but only if x_3 has already joined, which happens in half of the orderings. So $\phi_1 = 2 + 4 \times \frac{1}{2} = 4$.

x_3 has no effect of its own; it adds the interaction 4 only if x_1 is already in (half the orderings): $\phi_3 = 2$.

Check: $4 + 1.5 + 2 = 7.5$ ✓. The interaction's credit is split equally between the two features that create it.

The same formula, computed exactly for a gradient-boosting model on four diabetes features and one patient:

```
from itertools import combinations
from math import factorial

# Exact Shapley values for a model on 4 features, for one patient
feats = ["bmi", "bp", "s5", "age"]
idx = [list(d.feature_names).index(f) for f in feats]
X4 = Xd[:, idx]
model4 = GradientBoostingRegressor(random_state=0).fit(X4, yd)
background = X4[:100]                    # "absent" features are filled in from these
patients
x = X4[200]

def value(subset):
    """Average prediction when the features in `subset` take x's values and the others
vary as in the background."""
    Z = background.copy()
    Z[:, list(subset)] = x[list(subset)]
    return model4.predict(Z).mean()

p = len(feats); shap = np.zeros(p)
for j in range(p):
    others = [k for k in range(p) if k ≠ j]
    for size in range(p):
        for S in combinations(others, size):
            weight = factorial(size) * factorial(p - size - 1) / factorial(p)
            shap[j] += weight * (value(S + (j,)) - value(S))

base, pred = value(()), model4.predict(x[None, :])[0]
for f, v in zip(feats, shap):
    print(f"{f:4}: {v:+7.2f}")
print(f"baseline {base:.2f}, plus the contributions ({shap.sum():.2f}), gives {base +
shap.sum():.2f}")
print(f"the model's actual prediction: {pred:.2f}")
```

Output:

```
bmi :  -11.11
bp  :   +2.11
s5  :  -13.74
age :   +5.26
baseline 136.35, plus the contributions (-17.47), gives 118.87
the model's actual prediction: 118.87
```

The baseline (average prediction over the background patients) is 136.35. For this patient, low s5 (a blood-serum measure) lowers the prediction by 13.74 and low BMI by 11.11, while age adds 5.26 and blood pressure 2.11. The four contributions add to −17.47, giving 118.87, exactly the model's prediction: the efficiency property in action.

The exact formula needs 2^p subsets, so with 4 features it's 16 evaluations, but with 40 features it's about a trillion. The `shap` library makes it practical with clever approximations: **KernelSHAP** (a weighted regression over sampled subsets, for any model) and **TreeSHAP** (an exact, fast algorithm for tree ensembles). Plotting many patients' SHAP values together gives global views too: a "beeswarm" plot shows each feature's contributions across the whole dataset.

6. Try it: Shapley values for a tiny model

The model is $f = 2x_1 + 3x_2 + 4x_1x_3$, explained against the baseline $(0, 0, 0)$. Move the sliders and watch the contributions. Set $x_3 = 0$ and see x_1's share of the interaction disappear; set $x_1 = 0$ and x_3 gets nothing at all.

Interactive demo — open *lesson-24.html* on the website to try it.

7. Local surrogates (LIME)

LIME (local interpretable model-agnostic explanations) explains one prediction by fitting a *simple* model, usually linear, that imitates the black box *near that point*: create many perturbed copies of the example, ask the black box to predict them, weight each copy by how close it is, and fit a weighted linear regression. Its slopes are the explanation:

```
from sklearn.linear_model import Ridge

# A LIME-style local explanation: fit a weighted linear model to the black box near one
patient
rng = np.random.default_rng(0)
scale = X4.std(axis=0)
Z = x + rng.normal(0, 0.5, size=(2000, 4)) * scale          # perturbed copies of the
patient
w = np.exp(-np.sum(((Z - x) / scale) ** 2, axis=1) / 2)        # nearer copies count
more
local = Ridge(alpha=1e-3).fit((Z - x) / scale, model4.predict(Z), sample_weight=w)
print("local slopes (prediction change per +1 standard deviation):")
for f, c in zip(feats, local.coef_):
    print(f"{f:4}: {c:+7.2f}")
```

Output:

```
local slopes (prediction change per +1 standard deviation):
bmi :   +9.25
bp  :   +7.68
s5  :  +21.64
age :   +8.76
```

Near this patient, the model's prediction rises by about 22 units per standard deviation of s5 and by 7.7–9.3 units for the others. Notice these are *slopes* ("what if s5 increased a little?"), while the SHAP values above were *contributions* ("how much did this patient's s5 move the prediction from average?"). Both are valid; they answer different questions. LIME's results depend on choices such as the neighbourhood width and the sampling, and can change from run to run, which is why SHAP, with its guarantees, is more widely used.

8. How explanations mislead

- **Importance is not causation.** A model may lean on a feature because it's correlated with the true cause. Changing that feature in the real world won't change the outcome (Lesson 4: correlation vs causation).
- **Correlated features** share, split or hide credit. Methods that perturb one feature at a time also create unrealistic inputs.
- **Explanations describe the model, including its mistakes.** If the model learned a shortcut (a watermark, a hospital ID), a faithful explanation shows the shortcut. That's a feature, not a bug: it's how shortcuts are found.
- **Different methods answer different questions** (weights, drops in score, average curves, contributions, local slopes). Say which one you're reporting.
- **Simple, interpretable models** (a short tree, a sparse linear model, a scorecard) are sometimes nearly as accurate as a black box. When the stakes are high, consider them first rather than explaining a black box afterwards (Rudin, 2019).

9. In Python: an explanation workflow

A typical sequence for a tabular model: check global importance on test data, look at the effect shape of the top features, then explain individual predictions.

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.inspection import permutation_importance, partial_dependence

d = load_diabetes()
X_tr, X_te, y_tr, y_te = train_test_split(d.data, d.target, test_size=0.3,
random_state=0)
model = GradientBoostingRegressor(random_state=0).fit(X_tr, y_tr)

# 1. Global: which features does the model rely on? (on test data)
perm = permutation_importance(model, X_te, y_te, n_repeats=20, random_state=0,
scoring="r2")
top = np.argsort(perm.importances_mean)[::-1][:3]
print("top features:", [d.feature_names[i] for i in top],
np.round(perm.importances_mean[top], 3))

# 2. Shape: how does the prediction change with the most important feature?
pdr = partial_dependence(model, X_tr, [top[0]], grid_resolution=5, method="brute")
print("partial dependence:", np.round(pdr["average"][0], 1))
```

Output:

```
top features: ['s5', 'bmi', 'bp'] [0.221 0.181 0.043]
partial dependence: [103.4 129.  155.  186.5 194.5]
```

Shuffling s5 lowers the test R^2 the most (by 0.221), followed by BMI (0.181) and blood pressure (0.043), and the predicted progression rises steadily with s5, from about 103 to 195. For step 3, local explanations, install the `shap` library: `shap.TreeExplainer(model)(X_te)` gives exact SHAP values for tree models in milliseconds, and `shap.plots.waterfall` draws one prediction's contributions like the bars in the demo.

10. Running example: Chennai Homes

For a linear model with standardised features, each feature's SHAP value is simply its weight times its centred value, so explanations are exact and cheap. House 5 (a new Suburb villa) starts from the average price, then gains for its size, loses a little for its distance, and gains for being new; the contributions add up exactly to the model's prediction.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
cols = ["size_m2", "dist_km", "age_yrs"]
Xs = StandardScaler().fit_transform(df[cols])
lr = LinearRegression().fit(Xs, df["price_lakh"])
base = lr.intercept_                                  # average price (features
are centred)
contrib = Xs * lr.coef_                               # for a linear model, SHAP
= weight × centred value
h = 4                                                 # house id 5
print("baseline", round(base, 1), "| contributions:", {c: float(v) for c, v in
zip(cols, contrib[h].round(1))})
print("prediction", round(base + contrib[h].sum(), 1), "| actual",
df.price_lakh[h])
```

Output:

```
baseline 72.1 | contributions: {'size_m2': 2.4, 'dist_km': -0.2, 'age_yrs': 8.6}
prediction 82.9 | actual 88
```

⚙ WHERE THIS IS USED

- Explaining credit, insurance and medical decisions to customers and regulators
- Debugging models that learned shortcuts or leaked features
- Building stakeholder trust and choosing which features to collect

Choose it when: Whenever a model's predictions affect people or must be justified, and always while debugging.

Think twice when: you could use a simple model that is interpretable by design and nearly as accurate.

× MISCONCEPTIONS TO UNLEARN

MYTH A feature with high importance causes the outcome.

REALITY It's what the model relies on. The feature may just be correlated with the true cause.

MYTH SHAP values tell you what would happen if you changed a feature.

REALITY They split one prediction into contributions relative to a baseline. What-if questions need the model's curve (ICE) or, for the real world, causal analysis.

MYTH Deep models can't be explained, so they can't be trusted.

REALITY Many tools work for any model. But for high-stakes decisions, an interpretable model is often almost as accurate and is easier to trust.

RESPONSIBLE AI · EXPLANATIONS PEOPLE CAN ACT ON

Laws such as the EU's GDPR and AI Act, and lending rules in many countries, expect meaningful reasons for automated decisions. A useful explanation is one the affected person can understand and act on ("your debt-to-income ratio was the main factor"), not a list of 40 numbers. Check that explanations are faithful to the model, stable, and don't reveal other people's data.

11. Questions beginners often ask

Q. Should I use SHAP or permutation importance?

Both, for different questions. Permutation importance gives a quick, honest global ranking on test data. SHAP explains individual predictions and, aggregated, also gives global views. For tree models, TreeSHAP is fast and exact.

Q. Do I need the shap library?

For real projects, yes: it implements fast approximations and plots. This lesson computes exact Shapley values from scratch so you know what the library is doing, but the exact method needs 2^p evaluations and only works for a handful of features.

Q. Can explanations prove a model is fair?

No. They show what a model uses, which helps you find proxies and shortcuts (Lesson 21), but fairness is judged by outcomes and error rates per group.

Q. Why do different methods rank features differently?

They measure different things: weights, score drops on test data, impurity on training data, average contributions. Correlated features also share credit in method-dependent ways. Report which method you used.

! COMMON MISTAKES TO AVOID

- Comparing raw linear weights of features measured in different units.
- Computing permutation importance on training data, where overfitting inflates it.
- Reading importance as causation.
- Trusting a PDP when the feature is strongly correlated with others (use ALE or ICE).
- Reporting a single LIME explanation without checking that it is stable.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **What's the difference between permutation importance and a tree's built-in feature importance?**

The trap: Treating them as the same number.

Built-in (impurity) importance sums how much each feature reduced impurity during training: it's computed on training data and favours features with many distinct values, so noise can rank highly. Permutation importance measures the drop in a held-out score when the feature is shuffled: model-agnostic and honest, though correlated features can hide each other. Prefer permutation importance (or SHAP) for reporting.

▼ **Explain Shapley values and the properties that make them attractive.**

The trap: Calling SHAP "just feature importance".

A feature's Shapley value is its average marginal contribution to the prediction over all orders in which features could be revealed, with absent features filled in from a background. They satisfy efficiency (contributions sum to prediction minus baseline), symmetry, dummy (no effect, zero credit) and additivity, and are the unique attribution with these properties. Exact computation is exponential, so KernelSHAP samples and TreeSHAP exploits tree structure.

▼ **A PDP shows a strong effect of feature A, but A is highly correlated with B. What could go wrong?**

The trap: Trusting the curve as-is.

A PDP sets A to every value for every row while keeping B as it is, creating unrealistic combinations (high A with low B) that the model never saw, so the curve may reflect extrapolation. Credit may also be split arbitrarily between A and B. Use ALE plots or conditional methods, check ICE curves, and interpret A and B together.

▼ **Your model's top feature is the patient's hospital ID. What do you do?**

The trap: Celebrating the high accuracy.

Treat it as a red flag for a shortcut or leakage: the model may be learning which hospital treats sicker patients, or a data-collection artefact, which won't generalise to new hospitals. Remove or group the feature, validate on hospitals the model never saw

(group split), and look for other proxies. Explanations are most valuable exactly for finding this.

12. Key takeaways

- **Global** explanations describe overall behaviour; **local** ones explain one prediction.
- Linear weights are explanations only after **standardising**, and correlated features share their weight.
- **Permutation importance**: the drop in test score when one feature is shuffled. Model-agnostic, but correlated features hide each other.
- **PDP and ICE** show how the prediction changes with a feature, on average and per row; use ALE when features are strongly correlated.
- **SHAP values** split prediction minus baseline into fair contributions that add up exactly; TreeSHAP makes them fast for tree models.
- Explanations describe the **model**, not the world: importance is not causation.

13. Check your understanding

1. Permutation importance measures...

a. How often a feature is used in splits

b. How much the test score drops when a feature's values are shuffled

c. A feature's correlation with the target

2. A model's baseline prediction is 100 and it predicts 130 for Priya. Her SHAP values must add up to...

a. 30

b. 130

c. 1

3. In $f = 4x_1x_3$, explained at $x_1 = x_3 = 1$ against a baseline of 0, the SHAP value of x_1 is...

a. 4

b. 0

c. 2

4. A PDP shows predictions rising with "number of support calls". Can we conclude that more support calls cause customers to leave?

a. Yes

b. No: it shows what the model relies on, not what causes the outcome

c. Only if the model is accurate

5. ICE curves for a feature are parallel lines at different heights. What does that suggest?

a. The feature has the same effect for everyone, with no interaction with other features

b. The feature is useless

c. The model is overfitting

14. Going deeper: when features are correlated

INTERMEDIATE · SKIP ON A FIRST READ

Almost every real dataset has correlated features, and that's where explanation methods disagree most. This section puts two near-identical features into a model and compares how permutation importance, drop-column importance and SHAP share the credit, then shows the exact SHAP formula for linear models and how SHAP values become a global ranking.

You'll need: this lesson, and multicollinearity from Lesson 6's "Going deeper".

Three methods, three answers

Here the target depends on `a` (weight 3) and `c` (weight 1). Feature `b` is a near-copy of `a` that adds nothing new. **Drop-column importance** retrains the model without each feature and

measures the loss; it answers "what would we lose if we didn't have this feature at all?"

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance

# Two copies of the same signal: how do importance methods share the credit?
rng = np.random.default_rng(0)
n = 2000
a = rng.normal(size=n); b = a + rng.normal(0, 0.05, n)        # b is almost a copy of a
c = rng.normal(size=n)
X = np.column_stack([a, b, c]); y = 3 * a + 1 * c + rng.normal(0, 0.5, n)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)
rf = RandomForestRegressor(n_estimators=200, random_state=0).fit(X_tr, y_tr)
perm = permutation_importance(rf, X_te, y_te, n_repeats=10, random_state=0,
scoring="r2").importances_mean
print("permutation importance (a, b, c):", perm.round(3))

# Drop-column importance: retrain without each feature and measure the loss
full = rf.score(X_te, y_te)
drop = [full - RandomForestRegressor(n_estimators=200,
random_state=0).fit(np.delete(X_tr, j, 1), y_tr).score(np.delete(X_te, j, 1), y_te) for
j in range(3)]
print("drop-column importance (a, b, c):", np.round(drop, 3))
```

Output:

```
permutation importance (a, b, c): [1.045 0.129 0.172]
drop-column importance (a, b, c): [0.004 0.    0.117]
```

Permutation importance says `a` matters most (1.045) and gives `b` some credit (0.129), because the forest happens to split on both. Drop-column importance says *neither* `a` nor `b` matters much (0.004 and 0): remove one and the model simply uses the other. Only `c`, which carries unique information, loses a lot when dropped (0.117). Neither answer is wrong: permutation asks "does this model use the feature?", drop-column asks "is the information replaceable?". When features are correlated, report them as a group, or shuffle or drop the whole group together.

Exact SHAP for linear models, and global SHAP

For a linear model $f(\mathbf{x}) = b + \sum_j w_j x_j$ with independent features, the Shapley value has a closed form: $\phi_j = w_j\,(x_j - \bar{x}_j)$, the weight times how far this row's value is from the average. Averaging $|\phi_j|$ over many rows gives a **global** importance in the units of the prediction:

```
# For a linear model the exact SHAP value of feature j is w_j * (x_j - mean_j);
averaging |SHAP| gives a global ranking
lin = LinearRegression().fit(X_tr, y_tr)
shap_vals = lin.coef_ * (X_te - X_tr.mean(axis=0))
print("weights (a, b, c):            ", lin.coef_.round(2))
print("mean |SHAP| per feature:      ", np.abs(shap_vals).mean(axis=0).round(3))
print("check, first row: baseline + contributions", round(lin.predict(X_tr).mean() +
shap_vals[0].sum(), 4), "vs prediction", round(lin.predict(X_te[:1])[0], 4))
```

Output:

```
weights (a, b, c):            [2.75 0.24 1.02]
mean |SHAP| per feature:      [2.31  0.202 0.791]
check, first row: baseline + contributions 3.9815 vs prediction 3.9815
```

The linear model split the effect of the near-duplicates as 2.75 for `a` and 0.24 for `b`; with different random noise the split could easily come out differently (Lesson 6's "Going deeper" on unstable weights). The mean absolute SHAP values (2.31, 0.20, 0.79) inherit that arbitrary split. And the efficiency check holds: baseline plus contributions equals the prediction exactly (3.9815). Two ways to explain with correlated features: **interventional** SHAP (the default in most tools) treats features as independent and explains the model; **conditional** SHAP respects the correlations and explains the data-generating relationship more, but is harder to compute and can give credit to features the model never uses.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **Permutation importance, drop-column importance and SHAP rank your features differently. How do you explain that to a stakeholder?**

The trap: Picking whichever ranking looks most sensible.

They answer different questions. Permutation importance: how much does this trained model rely on the feature's information? Drop-column: how much would we lose if the feature didn't exist and we retrained (low for features with substitutes)? SHAP: how much does each feature move individual predictions from the baseline? Correlated features cause most of the disagreement, so report correlated groups together and say which question each number answers.

▼ **What's the closed-form SHAP value for a linear model, and what assumption does it rely on?**

The trap: Saying the SHAP value equals the weight.

$\phi_j = w_j(x_j - E[x_j])$: the weight times the row's deviation from the feature's mean. It assumes features are independent (interventional SHAP), so with correlated features it explains the model's arithmetic rather than how the features move together in reality.

Check your understanding: going deeper

1. Two near-identical features both have drop-column importance close to 0. The best conclusion is...

a. Both are useless

b. Each one is replaceable by the other; together they may matter a lot

c. The model is broken

2. For a linear model with weight 2 on feature x, mean of x 5 and a row with $x = 8$, the SHAP value of x is...

a. 16

b. 6

c. 2

3. What does drop-column importance require that permutation importance doesn't?

a. Retraining the model once per feature

b. Labels

c. A linear model

4. Global SHAP importance is usually computed as...

a. The sum of SHAP values

b. The mean of absolute SHAP values over many rows

c. The largest weight

5. Interventional SHAP treats features as...

a. Independent when filling in absent features

b. Causal

c. Always correlated

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

A model scores 0.90 accuracy on test data. Shuffling feature A gives 0.82, shuffling B gives 0.89 and shuffling C gives 0.90. Give the three permutation importances. Is C useless?

WORKING

EXERCISE 2 By hand

For $f = 3x_1 + 2x_2 + 6x_1x_2$ explained at $(1, 1)$ against the baseline $(0, 0)$, compute both Shapley values.

WORKING

EXERCISE 3 By hand

With 3 features, what weight does the Shapley formula give to a coalition S of size 1 when computing ϕ_j? (Use $|S|!(p-|S|-1)!/p!$.)

WORKING

EXERCISE 4 By hand

A model's baseline prediction is 50 lakh. For one house the SHAP values are size +12, distance −8 and age −3. What does the model predict?

WORKING

EXERCISE 5 Code

In the lesson's Shapley code, explain patient 0 instead of patient 200. Check that the contributions still add up exactly to prediction minus baseline.

WORKING

EXERCISE 6 Intermediate

A linear model is $f = 10 + 2x_1 - 3x_2$, with feature means $\bar{x}_1 = 4$, $\bar{x}_2 = 1$. For the row $(6, 2)$, compute both SHAP values and check that baseline + contributions = prediction.

WORKING

EXERCISE 7 Intermediate

A model scores $R^2 = 0.80$. Retrained without feature A it scores 0.79; without feature B, 0.60. What are the drop-column importances?

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~55 MIN

Lesson 25: Imbalanced Data & Anomaly Detection

Rare events: the right metrics, thresholds vs class weights vs resampling, SMOTE, and finding anomalies without labels with z-scores, Isolation Forest and LOF.

BY THE END OF THIS LESSON YOU CAN

- Explain why accuracy fails on rare classes and choose recall, precision, average precision or cost instead
- Compare threshold moving, class weights, oversampling and SMOTE, and apply them without leakage
- Detect anomalies without labels using z-scores, the IQR rule, Isolation Forest and Local Outlier Factor
- Evaluate a detector with precision in the top k and plan how it runs in practice

↺ BEFORE YOU START

This lesson builds on:

- Lesson 8: Logistic Regression — Logistic regression outputs a probability $p = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, is trained with log loss, and classifies with a threshold (often 0.5), giving a straight decision boundary.
- Lesson 9: Evaluating Models — Accuracy can mislead. Precision $= TP/(TP + FP)$, recall $= TP/(TP + FN)$, and F1 balances them. Moving the threshold trades one for the other; ROC-AUC summarises all thresholds.
- Lesson 16: Clustering — Clustering groups unlabelled data. k-means alternates assigning points to the nearest centroid and moving each centroid to the mean of its points.

Where this fits: Applies the metrics of Lesson 9 and the calibration and imbalance ideas of Lesson 8 to rare events, and uses the density ideas of clustering (Lesson 16) to find anomalies without labels.

1. When the thing you care about is rare

Fraud is perhaps 1 transaction in 1,000. A machine failure, one reading in 10,000. A rare disease, one patient in 500. In these problems the class you care about is a tiny minority, and almost

everything you learned about "accuracy" breaks: a model that never predicts the rare class is 99.9% accurate and completely useless.

This lesson covers the two situations you'll meet. When you **have labels** for some rare cases, it's an *imbalanced classification* problem: choose the right metrics, then fix the imbalance with thresholds, class weights or resampling. When you have **few or no labels**, it's *anomaly detection*: find points that don't look like the rest, with methods such as z-scores, Isolation Forest and Local Outlier Factor.

> ◎ THE CORE IDEA
>
> With rare classes, judge models with precision, recall and average precision, never accuracy. Most of the "fixes" for imbalance (reweighting, oversampling, SMOTE) mainly move the decision threshold; often the simplest and best fix is a calibrated model plus a threshold chosen from the cost of each mistake.

2. Why accuracy fails, in numbers

A dataset with 1% positives, split in half, with a logistic regression and a "model" that always says normal:

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score,
average_precision_score

# 1% positives, like fraud
X, y = make_classification(n_samples=20000, n_features=10, n_informative=5, weights=
[0.99],
                           class_sep=1.0, flip_y=0, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, random_state=0,
stratify=y)
print("positives in the test set:", int(y_te.sum()), "of", len(y_te))
for name, m in [("always 'normal'", DummyClassifier(strategy="most_frequent")),
("logistic regression", LogisticRegression(max_iter=1000))]:
    m.fit(X_tr, y_tr); pr = m.predict(X_te)
    print(f"{name:20}: accuracy {accuracy_score(y_te, pr):.3f}   recall
{recall_score(y_te, pr):.3f}")
```

Output:

```
positives in the test set: 100 of 10000
always 'normal'     : accuracy 0.990   recall 0.000
logistic regression : accuracy 0.995   recall 0.520
```

The do-nothing model is 99.0% accurate and catches none of the 100 positives. Logistic regression is 99.5% accurate, which sounds only slightly better, yet it catches 52% of them. Accuracy hides almost everything that matters here. Use (Lesson 9):

- **Recall**: the share of positives you catch.
- **Precision**: the share of your alerts that are real.
- **Average precision** (PR-AUC): the threshold-free summary of the precision–recall trade-off, compared with its baseline, the positive rate (0.01 here).
- Or a **cost** or **business metric**, such as rupees saved, or alerts per analyst per day.

3. Thresholds, class weights and resampling

Three families of fix, all aiming to make the model flag more of the rare class:

- **Move the threshold**: keep the model, flag anything with probability above, say, 0.1 instead of 0.5 (Lesson 9's "Going deeper" shows how to choose it from costs).
- **Class weights**: make each positive count more in the training loss (`class_weight="balanced"`, Lesson 8's "Going deeper").
- **Resampling**: train on a rebalanced copy of the training data, by *oversampling* positives (repeating them) or *undersampling* negatives (dropping some).

```
def report(name, scores, threshold=0.5):
    flag = scores >= threshold
    print(f"{name:32}: recall {recall_score(y_te, flag):.3f}  precision
{precision_score(y_te, flag, zero_division=0):.3f}  "
          f"average precision {average_precision_score(y_te, scores):.3f}")

plain = LogisticRegression(max_iter=1000).fit(X_tr, y_tr).predict_proba(X_te)[:, 1]
weighted = LogisticRegression(max_iter=1000, class_weight="balanced").fit(X_tr,
y_tr).predict_proba(X_te)[:, 1]
rng = np.random.default_rng(0)
pos = np.flatnonzero(y_tr == 1)
extra = rng.choice(pos, size=(y_tr == 0).sum() - len(pos), replace=True)        # random
oversampling
Xo, yo = np.vstack([X_tr, X_tr[extra]]), np.r_[y_tr, y_tr[extra]]
over = LogisticRegression(max_iter=1000).fit(Xo, yo).predict_proba(X_te)[:, 1]

report("plain, threshold 0.5", plain)
report("plain, threshold 0.1", plain, 0.1)
report("class_weight='balanced'", weighted)
report("random oversampling", over)
```

Output:

```
plain, threshold 0.5           : recall 0.520  precision 0.912  average precision 0.746
plain, threshold 0.1           : recall 0.760  precision 0.490  average precision 0.746
class_weight='balanced'        : recall 0.900  precision 0.146  average precision 0.471
random oversampling            : recall 0.900  precision 0.146  average precision 0.470
```

Read the four rows carefully. Lowering the threshold to 0.1 raises recall from 0.52 to 0.76 at a precision of 0.49, with the same average precision (0.746), because the model's ranking hasn't changed. Class weights and oversampling reach recall 0.90 but with precision 0.146 (most alerts false), and here they even *lower* average precision to about 0.47: training on a distorted class balance made the ranking worse, not just the threshold different. That isn't always so, but it's common enough that the order to try things in is: **first a good model and the right threshold; then weights or resampling if they improve average precision on validation data.**

✎ WORKED EXAMPLE — WHAT "BALANCED" WEIGHTS MEAN

The training half has 9,900 negatives and 100 positives ($n = 10{,}000$, $k = 2$ classes). Scikit-learn's balanced weight for class c is $n/(k\,n_c)$:

Negatives: $10{,}000/(2 \times 9{,}900) = 0.505$. Positives: $10{,}000/(2 \times 100) = 50$.

So each positive counts $50/0.505 = 99$ times as much as a negative: exactly the ratio of the class sizes. In total, positives and negatives now carry equal weight ($100 \times 50 = 5{,}000 \approx 9{,}900 \times 0.505$). Random oversampling to equal counts copies each positive about 99 times, which for logistic regression has almost the same effect, as the identical results above show.

4. SMOTE: synthetic minority examples

Copying the same positives many times can make a model memorise them. **SMOTE** (synthetic minority oversampling technique) creates *new* positives instead: pick a positive, pick one of its k nearest positive neighbours, and place a new point at a random position on the line between them.

```
# SMOTE from scratch: new minority points on the line between a point and one of its
minority neighbours
from sklearn.neighbors import NearestNeighbors

def smote(Xmin, n_new, k=5, seed=0):
    rng = np.random.default_rng(seed)
    nn = NearestNeighbors(n_neighbors=k + 1).fit(Xmin)
    _, nbrs = nn.kneighbors(Xmin)
    i = rng.integers(0, len(Xmin), n_new)
    j = nbrs[i, rng.integers(1, k + 1, n_new)]          # skip column 0 (the point
itself)
    gap = rng.random((n_new, 1))
    return Xmin[i] + gap * (Xmin[j] - Xmin[i])

Xmin = X_tr[y_tr == 1]
new = smote(Xmin, (y_tr == 0).sum() - len(Xmin))
Xs, ys = np.vstack([X_tr, new]), np.r_[y_tr, np.ones(len(new), int)]
report("SMOTE", LogisticRegression(max_iter=1000).fit(Xs, ys).predict_proba(X_te)[:, 1])
print("synthetic minority points created:", len(new))
```

Output:

```
SMOTE                          : recall 0.900  precision 0.169  average precision 0.515
synthetic minority points created: 9800
```

SMOTE created 9,800 synthetic positives and gives the same recall as oversampling (0.900) with slightly better precision (0.169) and average precision (0.515), still well below the plain model with a tuned threshold. SMOTE can help flexible models (trees, boosting, kNN) more than linear ones, and it can also hurt, by inventing positives in regions that really belong to negatives. Two firm rules: **resample only the training folds** (never the validation or test data, and never before splitting), and judge it by average precision or cost on untouched data.

5. Anomaly detection without labels

Often you have almost no labelled anomalies, or new kinds keep appearing. Then you model what's *normal* and flag what isn't. Common methods, from simplest:

- **z-scores**: flag values more than about 3 standard deviations from the mean. Simple, but assumes one bell-shaped cluster, and outliers inflate the very standard deviation used to find them (Lesson 4).
- **IQR rule**: flag values beyond 1.5 × the interquartile range outside the middle half. Robust to outliers, but one feature at a time.
- **Isolation Forest**: build random trees that split on random features at random values. Anomalies are few and different, so they get *isolated* in very few splits; the score is the average path length.
- **Local Outlier Factor (LOF)**: compare each point's local density with its neighbours' density. A point in a sparse spot next to a dense cluster stands out, even if another cluster elsewhere is sparse.
- **Reconstruction error**: fit PCA (Lesson 17's "Going deeper") or an autoencoder on normal data, and flag points it can't reconstruct well.

Below, 980 normal points form two clusters and 20 anomalies are scattered around them. Each method scores every point, and we check how many of its top 20 are real anomalies:

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import roc_auc_score as roc_auc

# Unsupervised anomaly detection: 980 normal points in two clusters, 20 anomalies
scattered around them
rng = np.random.default_rng(1)
normal = np.vstack([rng.normal([0, 0], 0.6, (490, 2)), rng.normal([4, 4], 0.6, (490,
2))])
anomalies = rng.uniform(-3, 7, (20, 2))
Xa = np.vstack([normal, anomalies]); truth = np.r_[np.zeros(980), np.ones(20)]

def precision_at_20(score):          # higher score = more anomalous
    return truth[np.argsort(score)[::-1][:20]].mean()

z = np.abs((Xa - Xa.mean(0)) / Xa.std(0)).max(axis=1)        # largest per-feature z-
score
iso = -IsolationForest(n_estimators=300, random_state=0).fit(Xa).score_samples(Xa)
lof = -LocalOutlierFactor(n_neighbors=20).fit(Xa).negative_outlier_factor_
for name, s in [("z-score", z), ("Isolation Forest", iso), ("Local Outlier Factor",
lof)]:
    print(f"{name:21}: share of true anomalies among the top 20 {precision_at_20(s):.2f}
ROC-AUC {roc_auc(truth, s):.3f}")
```

Output:

```
z-score              : share of true anomalies among the top 20 0.50   ROC-AUC 0.730
Isolation Forest     : share of true anomalies among the top 20 0.65   ROC-AUC 0.984
Local Outlier Factor : share of true anomalies among the top 20 0.80   ROC-AUC 0.991
```

The z-score catches only half (0.50 of its top 20; ROC-AUC 0.730): with two clusters, "far from the overall mean" isn't the same as "unusual", so points between the clusters look normal and the clusters' own edges look odd. Isolation Forest does much better (0.65, AUC 0.984), and LOF best (0.80, AUC 0.991), because it judges each point against its own neighbourhood. In practice anomaly scores are usually turned into a ranked list for people to review, so "precision in the top k " is often the metric that matters.

6. Try it: flag the most isolated points

Each point's score is its distance to its 5th nearest neighbour, a simple version of the idea behind LOF. Red points are the true anomalies. Increase the number of points flagged and watch recall rise while precision falls.

Interactive demo — open *lesson-25.html* on the website to try it.

7. Evaluating and running a detector

- **Get some labels.** Even a few hundred reviewed cases let you measure precision at the top of the list and compare methods.
- **Split by time.** Fraud and failures change; validate on later data (Lesson 5's "Going deeper").
- **Combine approaches.** A supervised model catches known patterns; an unsupervised detector watches for new ones. Analysts' decisions become labels for the next model.
- **Set the alert budget.** If a team can review 200 cases a day, flag the 200 highest scores; the threshold follows from capacity.
- **Watch for feedback.** You only learn the truth about cases you flagged and checked, so unflagged anomalies stay invisible. Randomly review a small sample of unflagged cases to estimate what you're missing.

8. In Python: a workflow for rare events

Train a calibrated model, choose the threshold on validation data to hit a target precision, and confirm on the test set.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import precision_recall_curve, precision_score, recall_score,
average_precision_score

X, y = make_classification(n_samples=30000, n_features=10, n_informative=5, weights=
[0.99],
                           class_sep=1.0, flip_y=0, random_state=0)
X_tr, X_rest, y_tr, y_rest = train_test_split(X, y, test_size=0.4, random_state=0,
stratify=y)
X_val, X_te, y_val, y_te = train_test_split(X_rest, y_rest, test_size=0.5,
random_state=0, stratify=y_rest)

model = HistGradientBoostingClassifier(random_state=0).fit(X_tr, y_tr)
p_val, p_te = model.predict_proba(X_val)[:, 1], model.predict_proba(X_te)[:, 1]
print("average precision, validation:", round(average_precision_score(y_val, p_val), 3))

prec, rec, thr = precision_recall_curve(y_val, p_val)
t = thr[np.argmax(prec[:-1] ⩾ 0.8)]                 # lowest threshold with precision
of at least 0.8
flag = p_te ⩾ t
print(f"threshold {t:.3f}: test precision {precision_score(y_te, flag):.3f}, recall
{recall_score(y_te, flag):.3f}")
```

Output:

```
average precision, validation: 0.691
threshold 0.458: test precision 0.682, recall 0.500
```

Gradient boosting reaches a validation average precision of 0.691. The threshold chosen to give 80% precision on validation data (0.458) gives only 68.2% precision on the test set, with recall 0.500. That gap is the honest lesson: the validation set holds only about 60 positives, so a threshold tuned on it is noisy. With rare events, use as much validation data as you can (or cross-validated predictions), report the test result with a confidence interval (Lesson 9's "Going deeper"), and re-check the threshold once the model is live.

9. Running example: Chennai Homes

Which Chennai homes look least like the others? An Isolation Forest on size, price, distance and age ranks every house by how easily it can be isolated. The three Outskirts homes come out on top (scores 0.578, 0.558 and 0.546): they are the only houses more than 20 km from the centre. With 12 houses the scores are rough, but a ranked list like this is where an analyst starts checking for data errors or genuinely unusual properties.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.ensemble import IsolationForest
cols = ["size_m2", "price_lakh", "dist_km", "age_yrs"]
iso = IsolationForest(n_estimators=500, random_state=0).fit(df[cols])
df["oddness"] = -iso.score_samples(df[cols])           # higher = more unusual
print(df.sort_values("oddness", ascending=False)[["id", "area", "description",
"oddness"]].head(3).round(3).to_string(index=False))
```

Output:

```
 id      area                description  oddness
 10 Outskirts  old house needs renovation    0.578
  9 Outskirts    quiet flat far from city    0.558
  8 Outskirts new large villa with garden    0.546
```

⚙ WHERE THIS IS USED

- Fraud and money-laundering detection
- Predictive maintenance and quality control (rare failures and defects)
- Rare-disease screening, intrusion detection and monitoring data pipelines for bad records

Choose it when: Whenever the class you care about is rare, or when you need to find unusual cases nobody has labelled.

Think twice when: the classes are roughly balanced: then ordinary methods and metrics work.

✕ MISCONCEPTIONS TO UNLEARN

MYTH You must balance the classes before training.

REALITY Many models train fine on imbalanced data; what must change is the metric and the threshold. Rebalancing is optional and can hurt calibration and ranking.

MYTH A high ROC-AUC means the alerts will be useful.

REALITY With very rare positives, even a small false-positive rate floods analysts with false alarms. Check precision and average precision (Lesson 9's "Going deeper").

MYTH Anomalies are just points far from the average.

REALITY With several clusters, unusual points can sit near the overall average. Judge points against their local neighbourhood.

RESPONSIBLE AI · WHO PAYS FOR A FALSE ALARM?

In fraud and safety systems, false alarms are not free: a blocked card, a frozen account or an unnecessary investigation lands on a real person, and often unevenly across groups. Measure false-positive rates per group (Lesson 21), give people a fast way to contest a flag, and budget alerts so humans can review them properly rather than rubber-stamping.

10. Questions beginners often ask

Q. How rare is "imbalanced"?

There's no hard line, but below about 10% positives accuracy starts to mislead, and below 1% the choice of metric and threshold dominates everything else.

Q. Should I always use SMOTE?

No. It sometimes helps flexible models, but often a calibrated model with a tuned threshold does as well or better, and class-imbalance corrections can distort risk estimates. Compare on average precision or cost with untouched validation data.

Q. Isolation Forest or LOF?

Isolation Forest is fast, scales to large data and works well in many dimensions. LOF judges each point against its local neighbourhood, so it's better when normal data has clusters of different densities, but it's slower on large datasets.

Q. What's the difference between outlier detection and novelty detection?

Outlier detection finds unusual points inside a dataset that may contain them. Novelty detection trains on clean, normal data and flags new points that don't fit, as a one-class model would.

! COMMON MISTAKES TO AVOID

- Reporting accuracy on a dataset with 1% positives.
- Oversampling or applying SMOTE before splitting, so copies leak into the test set.
- Using a class-weighted model's probabilities as if they were real risks.
- Tuning the threshold on the test set.
- Assuming unflagged cases are normal, without ever checking a sample of them.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **Why is accuracy a bad metric for fraud detection?**

The trap: Saying it's "too high".

With 0.1% fraud, predicting "not fraud" for everything is 99.9% accurate and catches nothing. Accuracy is dominated by the majority class. Use recall, precision, average precision (PR-AUC) against its baseline, or a cost-based metric, at the threshold you will actually use.

▼ **Class weights, oversampling or threshold moving: how do you choose for an imbalanced problem?**

The trap: Always reaching for SMOTE.

Start with a well-calibrated model and pick the threshold from costs or an alert budget: this keeps probabilities meaningful. Try class weights or resampling when they improve average precision or cost on validation data, which happens more for flexible models. Always resample inside training folds only, and recalibrate if you need real probabilities afterwards.

▼ **How does Isolation Forest work, and why does it suit anomaly detection?**

The trap: Describing it as a random forest classifier.

It builds many random trees: each split picks a random feature and a random threshold between its min and max. Anomalies, being few and different, are separated from the rest in fewer splits, so their average path length is short; the score normalises that length. It needs no labels or distance computations, scales linearly, and handles many features.

▼ **You deploy an anomaly detector and only review flagged cases. What problem does this create?**

The trap: Assuming recall can be measured from reviewed cases.

You never learn about anomalies the system missed, so recall is invisible and any retraining on reviewed cases only reinforces what the detector already finds (a feedback loop). Review a random sample of unflagged cases, track known incidents from other sources, and keep some exploration in what you flag.

11. Key takeaways

- With rare classes, **accuracy is misleading**: use recall, precision, average precision (against its baseline) or cost.
- Try a good model with a **tuned threshold** first; class weights and resampling mostly move the threshold, and can hurt the ranking.
- **SMOTE** creates synthetic positives between neighbours; resample **only the training folds**.
- Without labels, model normal behaviour: **z-scores and IQR** for simple cases, **Isolation Forest** and **LOF** for multi-dimensional, multi-cluster data.
- Evaluate detectors with **precision in the top k** on some reviewed cases, split by time, and sample unflagged cases to see what you miss.

12. Check your understanding

1. 0.5% of transactions are fraud. A model that never flags fraud has accuracy...

a. 50%

b. 0%

c. 99.5%

2. Lowering the threshold of a fixed model changes...

a. Precision and recall, but not average precision

b. Average precision only

c. Nothing

3. When should SMOTE or oversampling be applied?

a. To the whole dataset before splitting

b. To the training folds only

c. To the test set

4. Why does Isolation Forest give anomalies high scores?

a. They have large values

b. Random splits isolate them in very few steps

c. They are labelled

5. With two separate clusters of normal data, which detector is most likely to miss anomalies between the clusters?

a. Local Outlier Factor

b. Isolation Forest

c. A z-score from the overall mean

13. Going deeper: correcting probabilities and how isolation works

INTERMEDIATE · SKIP ON A FIRST READ

Two details that matter in real rare-event systems: how to turn a class-weighted (or resampled) model's distorted probabilities back into real risks with one formula, and why random splits isolate anomalies so quickly, measured directly.

You'll need: this lesson, calibration from Lesson 8's "Going deeper", and decision trees from Lesson 13.

Undoing class weights: the prior correction

Balanced class weights (or oversampling to 50/50) make the model behave as if positives were as common as negatives, so its probabilities are far too high. If the real positive rate is π, the odds can be corrected with Bayes' theorem by multiplying by the real prior odds:

$$\text{odds}_{\text{real}} = \frac{p}{1-p} \cdot \frac{\pi}{1-\pi}, \qquad p_{\text{real}} = \frac{\text{odds}_{\text{real}}}{1+\text{odds}_{\text{real}}}$$

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import brier_score_loss

X, y = make_classification(n_samples=40000, n_features=10, n_informative=5, weights=
[0.98], flip_y=0, random_state=1)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, random_state=0,
stratify=y)
pi = y_tr.mean()                                                  # true positive rate
in training data
p_bal = LogisticRegression(max_iter=1000, class_weight="balanced").fit(X_tr,
y_tr).predict_proba(X_te)[:, 1]

# Undo the reweighting: the model behaves as if positives were as common as negatives
odds = p_bal / (1 - p_bal) * (pi / (1 - pi))
p_fixed = odds / (1 + odds)
p_plain = LogisticRegression(max_iter=1000).fit(X_tr, y_tr).predict_proba(X_te)[:, 1]
for name, p in [("balanced weights", p_bal), ("balanced, corrected", p_fixed), ("plain
model", p_plain)]:
    print(f"{name:20}: average predicted risk {p.mean():.3f} (true rate
{y_te.mean():.3f})   Brier score {brier_score_loss(y_te, p):.4f}")
```

Output:

```
balanced weights    : average predicted risk 0.372 (true rate 0.020)   Brier score
0.1702
balanced, corrected : average predicted risk 0.017 (true rate 0.020)   Brier score
0.0178
plain model         : average predicted risk 0.020 (true rate 0.020)   Brier score
0.0169
```

The balanced model predicts an average risk of 0.372 when the true rate is 0.020, with a terrible Brier score of 0.1702. After the correction its average risk is 0.017 and its Brier score 0.0178, almost matching a model trained without weights (0.0169). So if you use weights or resampling to help a model learn the rare class, apply this correction (or recalibrate on held-out data with `CalibratedClassifierCV`) before using the outputs as probabilities: for pricing, ranking by expected loss, or telling a patient their risk.

Why random splits isolate anomalies

Isolation Forest grows each tree by choosing a random feature and a random split value between that feature's minimum and maximum, until every point sits alone. A point far from the crowd is very likely to be cut off by one of the first few random splits; a point deep inside a cluster needs

many. Here we measure the average number of splits for one far-away point and for typical points among 500:

```
# Isolation Forest's intuition: average number of random splits needed to isolate a
point
rng = np.random.default_rng(0)
pts = np.vstack([rng.normal(0, 1, (500, 2)), [[6, 6]]])            # 500 normal points
and one far away
def splits_to_isolate(i, trials=200):
    total = 0
    for _ in range(trials):
        idx = np.arange(len(pts)); depth = 0
        while len(idx) > 1:
            f = rng.integers(2); lo, hi = pts[idx, f].min(), pts[idx, f].max()
            t = rng.uniform(lo, hi)
            idx = idx[pts[idx, f] < t] if pts[i, f] < t else idx[pts[idx, f] ⩾ t]
            depth += 1
        total += depth
    return total / trials
print("average splits to isolate the far point:   ", round(splits_to_isolate(500), 1))
print("average splits to isolate a typical point: ", round(np.mean([splits_to_isolate(i,
40) for i in range(10)]), 1))
```

Output:

```
average splits to isolate the far point:    2.3
average splits to isolate a typical point:  14.6
```

The far point is isolated in 2.3 splits on average, typical points in 14.6. Isolation Forest turns that path length into a score between 0 and 1, normalised by the average path length expected for n points (about $2 \ln n$). Because it never computes distances, it scales linearly with the data and copes with many features; its main settings are the number of trees and the subsample size per tree (256 by default, which is enough because anomalies stand out even in small samples).

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **You trained on a 50/50 resampled dataset, but the real positive rate is 1%. How do you get calibrated probabilities?**

The trap: Using the raw scores as risks.

Apply the prior correction: multiply the model's odds by $\pi/(1-\pi)$ divided by the training odds (1 for 50/50), then convert back to a probability. Equivalently, add $\log \frac{\pi}{1-\pi} - \log \frac{\pi_{train}}{1-\pi_{train}}$ to the logit. Or recalibrate on a held-out set with the real class balance (Platt scaling or isotonic regression). Verify with a reliability table and the Brier score.

Check your understanding: going deeper

1. A model trained with balanced weights gives $p = 0.5$. The real positive rate is 2%. What is the corrected probability?

a. 0.5

b. About 0.02

c. About 0.25

2. Why do class weights inflate predicted probabilities?

a. The model learns as if positives were much more common than they are

b. They add noise

c. They change the features

3. In Isolation Forest, a short average path length means...

a. The point is typical

b. The point is easy to isolate, so it's likely an anomaly

c. The tree is too deep

4. Why can Isolation Forest use small subsamples (such as 256 points) per tree?

a. Anomalies stand out even in small samples, and smaller samples reduce masking by dense clusters

b. It can't handle more data

c. To save labels

5. After oversampling to 50/50, the simplest fix for probabilities is…

a. Ignore it

b. Apply the prior-odds correction or recalibrate on held-out data

c. Retrain with more trees

14. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

1,000 transactions include 10 frauds. A model flags 25, of which 8 are fraud. Compute accuracy, recall and precision.

WORKING

EXERCISE 2 By hand

A training set has 19,600 negatives and 400 positives. What balanced class weights does scikit-learn give each class, using $n/(k\,n_c)$?

WORKING

EXERCISE 3 By hand

SMOTE picks a positive at $(2, 4)$ and its neighbour $(6, 2)$, with random gap 0.25. Where is the new point?

WORKING

EXERCISE 4 By hand

Values: 10, 12, 11, 13, 12, 40. Using the quartiles $Q_1 = 11$ and $Q_3 = 13$, which values does the IQR rule (beyond $1.5\times$ IQR) flag?

WORKING

EXERCISE 5 Code

In the anomaly code, change LocalOutlierFactor's `n_neighbors` to 5 and to 50. How does its precision in the top 20 change?

WORKING

EXERCISE 6 Intermediate

A balanced-weight model outputs $p = 0.8$. The true positive rate is 5%. Compute the corrected probability.

WORKING

EXERCISE 7 Intermediate

An Isolation Forest uses subsamples of 256 points. Roughly what average path length is expected for a typical point, using $2 \ln n$?

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~55 MIN

Lesson 26: Time Series Forecasting

Trend and seasonality, baselines that are hard to beat, lag features, backtesting, MASE, and forecasting several steps ahead.

BY THE END OF THIS LESSON YOU CAN

- Describe a series by its trend, seasonality and noise, and compute naive, seasonal naive and moving-average baselines
- Turn a series into a supervised problem with past-only lag, rolling and calendar features
- Validate forecasts with time-ordered splits and backtesting, and report MAE and MASE
- Compare recursive and direct multi-step forecasting and report accuracy by horizon

↺ BEFORE YOU START

This lesson builds on:

- Lesson 5: Data, Features & Generalization — Always judge a model on data it hasn't seen: split into train / validation / test, or use k-fold cross-validation. Overfitting = low training error but high test error. Never let test data leak into training.
- Lesson 6: Linear Regression — Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$ and chooses the weights that minimise the mean squared error (least squares). R^2 measures the share of variance explained.
- Lesson 13: Decision Trees — A decision tree asks a sequence of 'feature ≤ threshold?' questions chosen to reduce impurity (Gini or entropy). Deep trees overfit and are unstable.

Where this fits: Applies regression (Lesson 6) and gradient boosting (Lesson 13) to ordered data, and replaces the shuffled cross-validation of Lesson 5 with time-ordered backtesting.

1. Why time series are different

How many umbrellas will the shop sell next week? How much electricity will the city need tomorrow evening? How many patients will arrive at A&E on Saturday night? These are **forecasting** problems: the data is a sequence of measurements over time, and we predict future values from past ones.

Time changes the rules you've learned. Rows aren't independent (today looks like yesterday), the future can differ from the past (trends, holidays, a pandemic), and a random train/test split is badly wrong because it lets the model train on the future (Lesson 5's "Going deeper" showed a shuffled split reporting $R^2 = 0.96$ for a model that couldn't forecast at all). This lesson covers the toolkit: components, baselines, lag features, time-based validation, error metrics for forecasts, and forecasting several days ahead.

> ◎ THE CORE IDEA
>
> Turn forecasting into ordinary supervised learning by building features only from the past (lags, rolling averages, calendar fields), always validate by training on the past and testing on the future, and always compare with simple baselines such as "same day last week", which are surprisingly hard to beat.

2. Trend, seasonality and noise

Most series combine three parts:

- **Trend**: the long-term direction (a growing shop).
- **Seasonality**: patterns that repeat with a fixed period: day of week, month of year, hour of day.
- **Noise**: what's left, the unpredictable part.

They can combine additively ($y = \text{trend} + \text{season} + \text{noise}$) or multiplicatively ($y = \text{trend} \times \text{season} \times \text{noise}$), where the seasonal swings grow as the level grows, which is typical of sales. Here is three years of simulated daily sales for a growing shop:

```
import numpy as np
import pandas as pd

# Three years of simulated daily sales for a shop: trend + weekly pattern + yearly
season + noise
rng = np.random.default_rng(0)
days = pd.date_range("2023-01-01", periods=3 * 365, freq="D")
t = np.arange(len(days))
weekly = np.array([0.9, 0.95, 1.0, 1.0, 1.1, 1.35, 1.25])[days.dayofweek]       # Mon …
Sun
yearly = 1 + 0.2 * np.sin(2 * np.pi * (days.dayofyear - 80) / 365)
sales = (200 + 0.15 * t) * weekly * yearly * rng.lognormal(0, 0.08, len(t))
s = pd.Series(sales.round(1), index=days, name="sales")
print(s.head(7).to_string())
print("average sales by weekday (Mon … Sun):",
s.groupby(s.index.dayofweek).mean().round(0).tolist())
print("average sales by year:", s.groupby(s.index.year).mean().round(0).tolist())
```

Output:

```
2023-01-01    203.1
2023-01-02    143.5
2023-01-03    161.4
2023-01-04    163.1
2023-01-05    155.2
2023-01-06    183.8
2023-01-07    243.8
Freq: D
average sales by weekday (Mon … Sun): [254.0, 267.0, 282.0, 281.0, 308.0, 383.0, 349.0]
average sales by year: [245.0, 304.0, 361.0]
```

The weekday averages show a strong weekly pattern (Saturday sells about 383, against about 254 on Monday), and the yearly averages a rising trend (245, then 304, then 361). A model must capture both. Taking logs turns a multiplicative series into an additive one, which suits linear models (Lesson 6's "Going deeper").

3. Baselines first

Before any model, compute simple forecasts. They set the bar and often turn out to be competitive:

- **Naive**: tomorrow equals today.
- **Seasonal naive**: tomorrow equals the same day last week (or last year).
- **Moving average**: tomorrow equals the average of the last k days.

```
# Forecast the last 8 weeks with simple baselines, one day ahead each day
test = s[-56:]
naive = s.shift(1)[-56:]                        # tomorrow = today
seasonal = s.shift(7)[-56:]                     # tomorrow = same weekday last week
moving = s.shift(1).rolling(28).mean()[-56:]    # tomorrow = average of the last 4
weeks
mae = lambda f: np.mean(np.abs(test - f))
for name, f in [("naive (yesterday)", naive), ("seasonal naive (last week)", seasonal),
("28-day average", moving)]:
    print(f"{name:27}: MAE {mae(f):6.1f}")
```

Output:

```
naive (yesterday)          : MAE   50.0
seasonal naive (last week) : MAE   32.6
28-day average             : MAE   45.3
```

Each forecast is made one day ahead, using only data up to the previous day, for the last 8 weeks. The seasonal naive forecast wins clearly (MAE 32.6), because it captures the weekly pattern for

free. "Yesterday" ignores the weekly cycle (50.0), and the 28-day average smooths it away (45.3). Any model you build must beat 32.6 to be worth its complexity.

4. Try it: four baselines

Switch between the four simple forecasts and compare their errors over the last 4 weeks. Notice how the weekly peaks are missed by the methods that ignore the weekday.

Interactive demo — open *lesson-26.html* on the website to try it.

5. Turning a series into features

To use the models you already know, build one row per day with features that only look backwards:

- **Lags**: the value 1, 7, 14 and 28 days earlier.
- **Rolling statistics**: the mean (or max, or standard deviation) of the previous 7 or 28 days. Shift before rolling, so the window excludes the day being predicted.
- **Calendar**: day of week (one-hot), month, holidays, and smooth yearly position as $\sin$ and $\cos$ of the day of year, so that 31 December and 1 January are close.
- **Known future inputs**: things known in advance, such as planned promotions or the weather forecast.

```
from sklearn.linear_model import Ridge
from sklearn.ensemble import HistGradientBoostingRegressor

# Features that only use the past: lags, rolling means and calendar fields
df = pd.DataFrame({"y": s})
for lag in [1, 7, 14, 28]:
    df[f"lag_{lag}"] = s.shift(lag)
df["roll_7"] = s.shift(1).rolling(7).mean()
df["roll_28"] = s.shift(1).rolling(28).mean()
df["dow"] = df.index.dayofweek
df["doy_sin"], df["doy_cos"] = np.sin(2 * np.pi * df.index.dayofyear / 365), np.cos(2 *
np.pi * df.index.dayofyear / 365)
df = df.dropna()
Xf = pd.get_dummies(df.drop(columns="y"), columns=["dow"], dtype=float)
start = s.index[-56]                                                    # the last 56
days are the test period
train, test_idx = Xf.index < start, Xf.index ⩾ start
yt = df.y[train].to_numpy()
scale = np.mean(np.abs(yt[7:] - yt[:-7]))                               # seasonal-naive
MAE on training data, for MASE
for name, m in [("ridge on lag features", Ridge(1.0)), ("gradient boosting",
HistGradientBoostingRegressor(random_state=0))]:
    pred = m.fit(Xf[train], df.y[train]).predict(Xf[test_idx])
    err = np.mean(np.abs(df.y[test_idx] - pred))
    print(f"{name:22}: MAE {err:6.1f}   MASE {err / scale:.2f}")
```

Output:

```
ridge on lag features : MAE   26.2   MASE 0.98
gradient boosting     : MAE   28.9   MASE 1.08
```

✎ WORKED EXAMPLE — MASE, THE SCALED ERROR

MAE is in the data's units, so "MAE 26" means nothing on its own. The **mean absolute scaled error** divides by the error of the seasonal naive forecast on the training data:

$$\text{MASE} = \frac{\text{MAE of the model}}{\text{MAE of seasonal naive on training data}}$$

Here the seasonal naive MAE on the training data is about 26.75. Ridge's test MAE is 26.2, so MASE $= 26.2/26.75 \approx 0.98$: just under 1, so a little better than the baseline. Gradient boosting's MAE of 28.9 gives MASE 1.08: *worse* than the baseline. A MASE below 1 means the model earns its keep; it works for any series and avoids MAPE's problems with values near zero.

Ridge on lag features reaches an MAE of 26.2 on the 8-week test period, beating the seasonal naive's 32.6 on the same days. Gradient boosting does slightly worse (28.9): tree models can't extrapolate the rising trend beyond the values seen in training (Lesson 13's "Going deeper"), a common surprise with growing series. Fixes include modelling the change from last week instead of the level, or detrending first.

6. Validating forecasts: backtesting

One test period can be lucky. **Backtesting** (rolling-origin evaluation) repeats "train on everything up to a date, forecast the next block" for several dates in a row. Scikit-learn's `TimeSeriesSplit` does this:

```
from sklearn.model_selection import TimeSeriesSplit

# Backtesting: repeat "train on the past, test on the next 8 weeks" several times
maes = []
for tr, te in TimeSeriesSplit(n_splits=5, test_size=56).split(Xf):
    pred = Ridge(1.0).fit(Xf.iloc[tr], df.y.iloc[tr]).predict(Xf.iloc[te])
    maes.append(np.mean(np.abs(df.y.iloc[te] - pred)))
print("MAE in each backtest window:", np.round(maes, 1))
print("average:", round(np.mean(maes), 1))
```

Output:

```
MAE in each backtest window: [21.9 28.3 28.6 19.8 26.2]
average: 25.0
```

Across five consecutive 8-week windows, the error ranges from 19.8 to 28.6: real variation from period to period that a single test window would hide. Report the average (25.0) and the spread. Two more rules: leave a **gap** between training and test if your features need recent data that won't be available in time, and re-run the backtest whenever you change a feature.

7. Forecasting several steps ahead

Businesses usually need more than tomorrow: a 14-day plan, a 12-month budget. Two strategies:

- **Recursive**: predict tomorrow, feed that prediction back in as a lag, predict the next day, and so on. One model, but errors can compound.
- **Direct**: train a separate model for each horizon ("7 days ahead" uses only features known 7 days earlier). More models, but no compounding.

```
# Forecasting 14 days ahead recursively: each prediction becomes the next day's lag
history = s[s.index < start].copy()
model = Ridge(1.0).fit(Xf[train], df.y[train])
cols = Xf.columns
for step in range(14):
    day = history.index[-1] + pd.Timedelta(days=1)
    row = {f"lag_{l}": history.iloc[-l] for l in [1, 7, 14, 28]}
    row.update(roll_7=history.iloc[-7:].mean(), roll_28=history.iloc[-28:].mean(),
               doy_sin=np.sin(2 * np.pi * day.dayofyear / 365), doy_cos=np.cos(2 * np.pi
* day.dayofyear / 365))
    row.update({f"dow_{d}": float(day.dayofweek == d) for d in range(7)})
    history.loc[day] = model.predict(pd.DataFrame([row])[cols])[0]
err = np.abs(history[-14:] - s[history.index[-14:]])
print("absolute error, days 1-7 ahead: ", round(err[:7].mean(), 1))
print("absolute error, days 8-14 ahead:", round(err[7:].mean(), 1))
```

Output:

```
absolute error, days 1-7 ahead:  28.1
absolute error, days 8-14 ahead: 24.8
```

Forecasting recursively from the start of the test period, the error is 28.1 for days 1–7 and 24.8 for days 8–14. On this smooth, strongly seasonal series it doesn't grow, because the 7-day lag mostly comes from real data and the pattern repeats. With shocks or weaker seasonality, recursive errors usually grow with the horizon, so always report accuracy *by horizon*.

8. The wider toolkit

Approach	Good for
Seasonal naive, moving averages	Baselines; surprisingly strong on stable series
Exponential smoothing (Holt–Winters, ETS)	Single series with trend and seasonality; fast and robust
ARIMA / SARIMA	Single series with autocorrelation; classical statistics with prediction intervals
Regression or gradient boosting on lag features	Many related series (thousands of products), extra inputs such as prices and promotions
Deep learning (sequence models, transformers)	Very many long series; usually needs lots of data to beat the above

The `statsmodels` library provides exponential smoothing and ARIMA; this lesson uses only scikit-learn and pandas. In forecasting competitions, simple methods and gradient boosting on well-engineered features have been very hard to beat (the M-competitions).

9. In Python: a compact forecasting pipeline

A reusable function that builds past-only features, trains, and scores against the seasonal naive baseline on a time-ordered test period:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge

rng = np.random.default_rng(1)
idx = pd.date_range("2024-01-01", periods=730, freq="D")
y = pd.Series((100 + 0.05 * np.arange(730)) * np.array([0.9, 1, 1, 1, 1.1, 1.3, 1.2])
[idx.dayofweek]
              * rng.lognormal(0, 0.1, 730), index=idx)

def make_features(y):
    X = pd.DataFrame({f"lag_{l}": y.shift(l) for l in [1, 7, 14]}, index=y.index)
    X["roll_7"] = y.shift(1).rolling(7).mean()
    return X.join(pd.get_dummies(pd.Series(y.index.dayofweek, index=y.index),
prefix="dow", dtype=float))

X = make_features(y).dropna(); target = y[X.index]
cut = X.index[-60]                                      # the last 60 days are the
test period
model = Ridge(1.0).fit(X[X.index < cut], target[target.index < cut])
pred = model.predict(X[X.index ⩾ cut])
actual, base = target[target.index ⩾ cut], y.shift(7)[target.index[target.index ⩾
cut]]
print("model MAE:", round(np.mean(np.abs(actual - pred)), 2), "  seasonal naive MAE:",
round(np.mean(np.abs(actual - base)), 2))
```

Output:

```
model MAE: 14.52   seasonal naive MAE: 20.01
```

On this second simulated series, the lag-feature model beats the seasonal naive baseline on the 60-day test period. The same structure (past-only features, a time cut-off, a baseline) scales to thousands of series, with an extra "series id" feature or one model per group.

10. Running example: Chennai Homes

The houses file has no dates, so this example simulates a monthly Chennai price index for 3 years, starting from the average price of the 12 homes, with gentle growth and a small yearly cycle. Over the last 6 months, "same as last month" has an MAE of 1.08 lakh, while "same month last year" has 4.77: here the trend dominates the weak seasonality, so the seasonal naive forecast lags a whole year of growth behind. Which baseline is best depends on the series, which is why you compute several.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
rng = np.random.default_rng(0)
months = pd.date_range("2023-01-01", periods=36, freq="MS")
level = df.price_lakh.mean() * (1 + 0.006 * np.arange(36)) * (1 + 0.03 * np.sin(2
* np.pi * months.month / 12))
index = pd.Series(level * rng.normal(1, 0.01, 36), index=months).round(1)   # a
simulated monthly price index
test = index.index ⩾ "2025-07-01"
naive, seasonal = index.shift(1)[test], index.shift(12)[test]
print("last 6 months:", index[test].tolist())
print("MAE, last month:", round((index[test] - naive).abs().mean(), 2), "  MAE,
same month last year:", round((index[test] - seasonal).abs().mean(), 2))
```

Output:

```
last 6 months: [82.9, 83.1, 83.2, 84.6, 85.7, 87.5]
MAE, last month: 1.08   MAE, same month last year: 4.77
```

⚙ WHERE THIS IS USED

- Demand and sales forecasting for stock and staffing
- Electricity load, traffic and call-centre volume planning
- Budgeting, capacity planning and early warnings when a metric drifts from its forecast

Choose it when: Whenever you need a number for a future date and you have a history of that number.

Think twice when: the future is driven by a one-off change the history never saw: then forecasts need scenarios and judgement, not just models.

× MISCONCEPTIONS TO UNLEARN

MYTH A complex model will always beat a simple forecast.

REALITY Seasonal naive and exponential smoothing often win on stable series; in this lesson, gradient boosting lost to the baseline.

MYTH Random cross-validation is fine if the model is regularised.

REALITY Shuffling lets the model see the future; validation must respect time order.

MYTH A low MAE means a good forecast.

REALITY MAE depends on the scale of the series; compare with a baseline using MASE.

RESPONSIBLE AI · FORECASTS BECOME DECISIONS

A demand forecast sets staffing rotas, stock in poorer neighbourhoods and hospital beds. A forecast that's systematically low for some stores or regions quietly under-serves them, and a forecast trained on a period of under-supply learns that low demand is normal. Check errors by segment, not just overall, and communicate uncertainty so planners don't treat a point forecast as a promise.

11. Questions beginners often ask

Q. How much history do I need?

At least two full seasonal cycles to learn a seasonal pattern (two years for a yearly pattern), and more if you want to backtest over several periods.

Q. Should I use ARIMA or a machine-learning model?

For one series, exponential smoothing or ARIMA are strong and quick. For many related series, or when you have extra inputs such as prices and promotions, gradient boosting on lag features usually wins. Backtest both against the seasonal naive baseline.

Q. How do I give prediction intervals?

Statistical models give them directly. For ML models, train quantile models (for example `loss="quantile"` in gradient boosting) or use the spread of backtest errors at each horizon.

Q. Can I use future values of other features?

Only if they'll really be known at prediction time, such as the calendar or planned promotions. A feature like "actual temperature tomorrow" must be replaced by the forecast you'd have had.

! COMMON MISTAKES TO AVOID

- Shuffling rows before splitting, so the model trains on the future.
- Computing a rolling mean that includes the day being predicted.
- Not comparing with a seasonal naive baseline.
- Reporting one test window instead of a backtest across several.
- Using a tree model on a trending level without detrending or modelling changes.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **Why can't you use ordinary k-fold cross-validation for a forecasting model?**

The trap: "You can, if you shuffle."

Shuffled folds train on days after the ones they test, and neighbouring days are similar, so the model effectively sees the future and the score is far too optimistic. Use time-ordered splits: train on the past, test on the next block, and repeat for several cut-off dates (backtesting).

▼ **What baseline would you compare a daily sales forecast with?**

The trap: Comparing only with a constant mean.

The seasonal naive forecast (same day last week), plus the naive forecast and a moving average. Seasonal naive is often hard to beat. Report MASE, the model's MAE divided by the seasonal naive MAE, so below 1 means the model adds value.

▼ **Your gradient boosting forecast is fine on old data but always too low recently, as sales grow. Why?**

The trap: Blaming not enough trees.

Trees predict averages of training targets, so they can't extrapolate a rising level beyond what they've seen. Model the change or ratio from a lag (for example y / y_{t−7}), detrend first, or combine with a linear model for the trend.

▼ **Recursive or direct multi-step forecasting: what's the trade-off?**

The trap: Not knowing the difference.

Recursive uses one one-step model and feeds its predictions back as lags: simple, but errors can compound. Direct trains one model per horizon on features available that far ahead: no compounding, but more models and less use of recent data. Report accuracy by horizon either way.

12. Key takeaways

- Time series have **trend**, **seasonality** and **noise**; rows are not independent.
- Always compute **baselines** (naive, seasonal naive, moving average); seasonal naive is often hard to beat.
- Build features **only from the past**: lags, shifted rolling statistics, calendar fields and known future inputs.
- Validate with **time-ordered splits and backtesting**, never shuffled folds.
- Report **MAE** and the scale-free **MASE** (below 1 beats seasonal naive), and accuracy by horizon.
- Tree models can't extrapolate trends; model changes or detrend first.

13. Check your understanding

1. For daily sales with a strong weekly pattern, the "seasonal naive" forecast for next Saturday is...

a. Next Friday's actual sales

b. Last Saturday's sales

c. The average of all days

2. Why is a shuffled k-fold split wrong for forecasting?

a. It's too slow

b. It uses too little data

c. It trains on days after the ones it tests, which leaks the future

3. A model has MASE 1.3. What does that mean?

a. Its error is 30% worse than the seasonal naive forecast's

b. It's 30% accurate

c. It beats the baseline by 30%

4. Which feature leaks when predicting day d?

a. Sales on day $d - 7$

b. A 7-day rolling mean that includes day d

c. The day of the week of d

5. Sales grow steadily every year. Why might a random forest on the raw sales level forecast poorly?

a. Forests can't use lags

b. Forests need scaled features

c. Trees can't predict values above the range seen in training

14. Going deeper: choosing lags and honest prediction intervals

INTERMEDIATE · SKIP ON A FIRST READ

Two practical skills: reading the autocorrelation of a series to decide which lags to use, and turning held-out forecast errors into prediction intervals, then checking whether they really cover what they claim.

You'll need: this lesson, correlation from Lesson 3, and quantiles from Lesson 2.

Autocorrelation: which lags matter?

The **autocorrelation** at lag k is the correlation between the series and itself shifted by k steps. Compute it on the *changes* (or after detrending), because a trend makes every lag look correlated. Peaks show the seasonal period, and so which lags to give a model:

```
import numpy as np
import pandas as pd
rng = np.random.default_rng(3)
t = np.arange(730)
y = pd.Series((200 + 0.15 * t) * np.array([0.85, 0.9, 0.95, 1.0, 1.05, 1.25, 1.1])[t %
7] * rng.lognormal(0, 0.08, 730))
change = y.diff(1)                                  # remove the trend first
acf = {lag: round(float(change.autocorr(lag)), 2) for lag in [1, 2, 3, 6, 7, 14, 21]}
print("autocorrelation of daily changes by lag:", acf)
```

Output:

```
autocorrelation of daily changes by lag: {1: -0.12, 2: -0.32, 3: -0.07, 6: 0.07, 7:
0.61, 14: 0.59, 21: 0.6}
```

On this daily series, the changes are strongly correlated at lags 7, 14 and 21 (about 0.6) and barely at lags 1 to 6: the series is dominated by its weekly cycle, so 7-day multiples are the lags to use. The negative value at lag 2 is a typical side effect of differencing. Plotting the autocorrelation for lags up to two seasons (the `plot_acf` function in statsmodels) is a standard first look at any new series.

Prediction intervals from held-out errors

A point forecast says nothing about how wrong it might be. A simple, model-agnostic way to get an interval: forecast a held-out calibration period, take the 5th and 95th percentiles of the errors, and add them to future forecasts. This is the idea behind split **conformal prediction**. Then check the **coverage** on a later test period:

```
from sklearn.linear_model import Ridge
X = pd.DataFrame({f"lag_{l}": y.shift(l) for l in [7, 14, 21]}).dropna(); target =
y[X.index]
cal, test = X.index[(X.index ⩾ 550) & (X.index < 640)], X.index[X.index ⩾ 640]
model = Ridge(1.0).fit(X[X.index < 550], target[target.index < 550])
resid = target[cal] - model.predict(X.loc[cal])
lo, hi = np.quantile(resid, [0.05, 0.95])            # 90% interval from held-out errors
pred = model.predict(X.loc[test])
inside = ((target[test] ⩾ pred + lo) & (target[test] ⩽ pred + hi)).mean()
print("interval offsets:", round(lo, 1), "to", round(hi, 1), "  coverage on the test
period:", round(inside, 3))
```

Output:

```
interval offsets: -40.3 to 50.2   coverage on the test period: 0.856
```

The interval was built to cover 90%, but on the test period it covers 85.6%. The reason is the trend: sales, and so the size of the errors, keep growing, so errors from an earlier period understate later ones. Fixes are to model the log of the series (so errors are relative), to use only recent errors, or to recompute the interval at each backtest step. Always report the coverage you actually achieve, by horizon, not just the nominal level.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How would you give prediction intervals for a gradient boosting forecast?**

The trap: "Gradient boosting can't give intervals."

Train quantile models (for example the 5th and 95th percentiles with a quantile loss), or use conformal prediction: compute errors on a held-out calibration period and add their quantiles to the point forecast. Either way, backtest the coverage by horizon, because errors usually grow with the horizon and with the series' level, and refresh the calibration regularly.

Check your understanding: going deeper

1. Why compute autocorrelation on the changes rather than the raw series?

a. A trend makes every lag look strongly correlated

b. Changes are smaller numbers

c. It's required by pandas

2. The autocorrelation of daily changes peaks at 7, 14 and 21. Which lags should a model use?

a. Lags 1 to 3

b. Multiples of 7

c. Lag 30

3. A 90% interval built from held-out errors covers 86% of test values. What does that mean?

a. The interval is too narrow for the test period

b. The model is unbiased

c. Everything is fine

4. Why might errors from an earlier period understate later errors?

a. The series' level, and so its errors, is growing

b. Errors are always constant

c. Because of the random seed

5. What does split conformal prediction need?

a. A held-out calibration set of errors

b. A Bayesian prior

c. A neural network

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

Daily sales for the last 8 days were 50, 52, 49, 70, 75, 51, 53, 54 (the last is yesterday). Give the naive forecast, the seasonal naive (7-day) forecast and the 3-day moving average for today.

WORKING

EXERCISE 2 By hand

A model's test MAE is 18. The seasonal naive forecast's MAE on the training data is 24. Compute the MASE and say whether the model beats the baseline.

WORKING

EXERCISE 3 By hand

Encode day 91 of a 365-day year as $\sin$ and $\cos$ of $2\pi \cdot 91/365$.

WORKING

EXERCISE 4 By hand

You have 730 days and use TimeSeriesSplit with 5 folds and a test size of 56 days. How many days does the first fold train on?

WORKING

EXERCISE 5 Code

In the lag-feature code, drop the 7-day and 14-day lags and keep only the 1-day lag and the rolling mean. What happens to the MAE, and why?

WORKING

EXERCISE 6 Intermediate

Held-out forecast errors (actual − forecast) have a 5th percentile of −12 and a 95th of 15. Tomorrow's forecast is 200. Give the 90% interval.

WORKING

EXERCISE 7 Intermediate

A 90% interval is checked on 60 test days, and 51 of them fall inside it. What's the coverage, and is the interval too narrow or too wide?

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~55 MIN

Lesson 27: Recommender Systems

Explicit and implicit feedback, bias baselines, neighbour-based collaborative filtering, matrix factorisation, ranking metrics, and cold start.

BY THE END OF THIS LESSON YOU CAN

- Represent explicit and implicit feedback as a sparse user–item matrix and build bias baselines
- Compute item–item collaborative filtering with cosine similarity
- Train a matrix factorisation model with stochastic gradient descent and explain its latent factors
- Evaluate recommendations with ranking metrics such as precision@k, and handle cold start with content features

↺ BEFORE YOU START

This lesson builds on:

- Lesson 7: Gradient Descent — Gradient descent trains almost any model: compute the gradient of the loss, step against it, repeat. The learning rate controls the step size; too large diverges. Mini-batches make it fast on big data.
- Lesson 11: k-Nearest Neighbours — kNN predicts from the k most similar training examples, so it depends on distances — features must be scaled, and it struggles in very high dimensions.
- Lesson 17: Dimensionality Reduction & PCA —

Where this fits: Uses the similarity and nearest-neighbour ideas of Lesson 11, the gradient descent of Lesson 7 and the low-rank thinking of PCA (Lesson 17) on the user–item matrix.

1. What recommender systems do

Every streaming service, online shop and social feed decides, for each person, which few items out of millions to show next. That's a **recommender system**, and it's one of the most commercially important uses of ML: a large share of what people watch and buy online comes from recommendations.

The data is usually a huge, mostly empty table of **users × items**. Most users have interacted with a tiny fraction of the items, and the task is to fill in the blanks: which unseen items would each

user like? This lesson builds the main approaches from scratch: popularity baselines, content-based filtering, collaborative filtering (neighbours) and matrix factorisation, then covers how to evaluate rankings and the practical traps of cold start and feedback loops.

> ◎ THE CORE IDEA
>
> People who agreed in the past tend to agree in the future. **Collaborative filtering** exploits this without knowing anything about the items themselves; **matrix factorisation** makes it scale by describing every user and item with a few hidden "taste" numbers learned from the ratings.

2. Explicit and implicit feedback

- **Explicit feedback**: users tell you what they think (star ratings, thumbs up/down). Clear but scarce.
- **Implicit feedback**: what users do (clicks, plays, purchases, watch time). Plentiful but noisy, and with no negatives: an unclicked item may be disliked, or simply never seen.

Most real systems run on implicit feedback. We'll use simulated star ratings so we can measure accuracy precisely: 300 users and 60 films, built from 3 hidden tastes per user and film, with each user having rated about 15% of the films.

```
import numpy as np
import pandas as pd

# 300 users, 60 films. Each user and film has 3 hidden "tastes"; users rate about 15% of
films
rng = np.random.default_rng(0)
n_users, n_items, k = 300, 60, 3
U, V = rng.normal(size=(n_users, k)), rng.normal(size=(n_items, k))
popularity = rng.normal(0, 0.5, n_items)
true = 3.2 + U @ V.T * 0.6 + popularity + rng.normal(0, 0.4, (n_users, n_items))
R = np.clip(np.round(true), 1, 5)
mask = rng.random((n_users, n_items)) < 0.15
ratings = pd.DataFrame([(u, i, R[u, i]) for u, i in zip(*np.nonzero(mask))], columns=
["user", "item", "rating"])
print(ratings.head(4).to_string(index=False))
print("ratings:", len(ratings), " share of the matrix filled:", round(mask.mean(), 3))
```

Output:

```
 user  item  rating
    0     0     3.0
    0     2     2.0
    0    10     4.0
    0    21     4.0
ratings: 2747  share of the matrix filled: 0.153
```

2,747 ratings fill just 15.3% of the 18,000 cells. Real systems are far sparser: often well under 1%.

3. Baselines: averages and biases

We hold out one rating per user to test on and measure the **root mean squared error** (RMSE) of the predicted ratings. The simplest predictors use averages:

```
# Hold out one rating per user (for evaluation) and train on the rest
test = ratings.groupby("user").sample(1, random_state=0)
train = ratings.drop(test.index)
mu = train.rating.mean()
item_mean = train.groupby("item").rating.mean()
user_bias = (train.rating - mu).groupby(train.user).mean()
rmse = lambda p: np.sqrt(np.mean((test.rating.to_numpy() - p) ** 2))
print("global mean      RMSE:", round(rmse(np.full(len(test), mu)), 3))
print("item average     RMSE:",
round(rmse(test.item.map(item_mean).fillna(mu).to_numpy()), 3))
print("item + user bias RMSE:", round(rmse((test.item.map(item_mean).fillna(mu) +
test.user.map(user_bias).fillna(0)).to_numpy()), 3))
```

Output:

```
global mean      RMSE: 1.077
item average     RMSE: 1.024
item + user bias RMSE: 1.057
```

Predicting the global average gives RMSE 1.077; each film's own average does better (1.024), because some films are simply better liked. Naively adding a "user bias" (how much each user rates above average) makes things *worse* here (1.057): with only about 9 ratings per user, that bias is noisy, and it double-counts the film effect (a user who happened to rate good films looks generous). Biases must be learned *jointly* and with regularization, which is exactly what matrix factorisation below does.

4. Collaborative filtering: neighbours

User-based collaborative filtering predicts your rating of a film from the ratings of users similar to you. **Item-based** filtering predicts it from your ratings of films similar to that film, where "similar

films" are ones the same users rated alike. Similarity is usually cosine or Pearson correlation computed on *centred* ratings (each user's ratings minus their own average), so a harsh rater and a generous rater with the same tastes look alike:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{j \in N(i)} \mathrm{sim}(i,j)\,(r_{uj} - \bar{r}_u)}{\sum_{j \in N(i)} |\mathrm{sim}(i,j)|}$$

```
# Item-item collaborative filtering: films are similar if the same users rate them alike
M = np.full((n_users, n_items), np.nan)
M[train.user, train.item] = train.rating
centred = M - np.nanmean(M, axis=1, keepdims=True)           # remove each user's
personal scale
C = np.nan_to_num(centred)
norms = np.linalg.norm(C, axis=0) + 1e-9
S = (C.T @ C) / np.outer(norms, norms)                        # cosine similarity
between films
np.fill_diagonal(S, 0)

def predict_item_item(u, i, k=10):
    rated = np.flatnonzero(~np.isnan(M[u]))
    top = rated[np.argsort(S[i, rated])[::-1][:k]]
    w = S[i, top]
    base = np.nanmean(M[u])
    return base + (w @ centred[u, top]) / (np.abs(w).sum() + 1e-9) if len(top) else mu

p_ii = np.array([predict_item_item(u, i) for u, i in zip(test.user, test.item)])
print("item-item CF     RMSE:", round(rmse(p_ii), 3))
```

Output:

```
item-item CF     RMSE: 1.012
```

Item-item collaborative filtering reaches RMSE 1.012: a little better than the baselines. With so few ratings per film (about 46), similarities between films are noisy; neighbour methods shine with much denser data. Item-based CF has been widely used in practice ("customers who bought this also bought…") because item similarities are stable and can be computed in advance.

5. Try it: recommendations from similar users

Rate a few films in the "You" row. The demo computes how similar your ratings are to each other user's, and predicts your rating of the films you haven't rated from the users who resemble you. Try rating the sci-fi films high and the romances low, then the reverse.

Interactive demo — open *lesson-27.html* on the website to try it.

6. Matrix factorisation

Instead of comparing users directly, **matrix factorisation** gives every user a vector $\mathbf{p}_u$ and every item a vector $\mathbf{q}_i$ of k hidden factors, plus a bias each, and predicts

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u \cdot \mathbf{q}_i$$

The factors are learned by minimising squared error on the *known* ratings, with L2 regularization (Lesson 10) to stop the many parameters from overfitting:

$$\min \sum_{(u,i)\ \text{known}} \left(r_{ui} - \hat{r}_{ui}\right)^2 + \lambda\left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\right)$$

It's the low-rank idea from the SVD (Lesson 2's "Going deeper"), adapted to a matrix with missing entries. The factors often end up meaning something ("action vs romance", "mainstream vs art-house") even though nobody labelled them. Below it's trained with stochastic gradient descent (Lesson 7), one rating at a time:

```
# Matrix factorisation with SGD: learn a vector for every user and film so that rating ≈
mu + b_u + b_i + p_u · q_i
def factorise(train, k=3, epochs=40, lr=0.02, reg=0.05, seed=0):
    g = np.random.default_rng(seed)
    P, Q = g.normal(0, 0.1, (n_users, k)), g.normal(0, 0.1, (n_items, k))
    bu, bi = np.zeros(n_users), np.zeros(n_items)
    data = train[["user", "item", "rating"]].to_numpy()
    for ep in range(epochs):
        for u, i, r in data[g.permutation(len(data))]:
            u, i = int(u), int(i)
            e = r - (mu + bu[u] + bi[i] + P[u] @ Q[i])
            bu[u] += lr * (e - reg * bu[u]); bi[i] += lr * (e - reg * bi[i])
            P[u], Q[i] = P[u] + lr * (e * Q[i] - reg * P[u]), Q[i] + lr * (e * P[u] -
reg * Q[i])
    return P, Q, bu, bi

P, Q, bu, bi = factorise(train)
p_mf = mu + bu[test.user] + bi[test.item] + np.sum(P[test.user] * Q[test.item], axis=1)
print("matrix factorisation RMSE:", round(rmse(p_mf), 3))
```

Output:

```
matrix factorisation RMSE: 0.715
```

✎ WORKED EXAMPLE — ONE SGD UPDATE

Suppose $\mu = 3.5$, $b_u = 0.2$, $b_i = -0.1$, $\mathbf{p}_u = (0.5, 0.1)$, $\mathbf{q}_i = (0.4, -0.2)$, and the true rating is $r = 4$.

Prediction: $3.5 + 0.2 - 0.1 + (0.5 \times 0.4 + 0.1 \times (-0.2)) = 3.6 + 0.18 = 3.78$. **Error:** $e = 4 - 3.78 = 0.22$.

Update with learning rate $\eta = 0.1$ and $\lambda = 0$: $b_u \leftarrow 0.2 + 0.1 \times 0.22 = 0.222$; $\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta e\, \mathbf{q}_i = (0.5 + 0.0088,\ 0.1 - 0.0044) = (0.5088, 0.0956)$. The user's vector moves slightly towards the film's, so their predicted rating rises.

Matrix factorisation cuts the RMSE to 0.715, far better than every method above, because the 3 learned factors match the hidden structure of the data. On real data the number of factors is a hyperparameter (often 20–200), tuned with the validation methods of Lesson 23. Libraries use **alternating least squares** (ALS: fix the items and solve for users exactly, then swap), which parallelises well; for implicit feedback, weighted ALS treats every unclicked item as a weak negative.

7. Evaluating recommendations: rankings, not ratings

Users see a short list, not predicted stars. What matters is whether the top few items are good, so ranking metrics are used:

- **Precision@k**: the share of the top k recommendations that are relevant.
- **Recall@k** and **hit rate@k**: the share of relevant items (or of users with at least one) that appear in the top k.
- **NDCG** and **MAP**: also reward putting relevant items higher in the list.

Because our data is simulated, we know every user's true rating for every film, so we can check the top 10 unseen films we'd recommend each user:

```
# Ranking quality: of the 10 unseen films we recommend, how many would the user really
like (true rating 4 or 5)?
# (We can check this only because the data is simulated and we know every true rating.)
liked = R ⩾ 4
def precision_at_10(score_fn):
    hits = []
    for u in range(n_users):
        scores = score_fn(u).astype(float).copy()
        scores[mask[u]] = -np.inf                                    # don't recommend
films they've rated
        top = np.argsort(scores)[::-1][:10]
        hits.append(liked[u, top].mean())
    return np.mean(hits)

pop = train.groupby("item").rating.mean().reindex(range(n_items),
fill_value=mu).to_numpy()
rand = np.random.default_rng(1)
print("precision@10, random films:          ", round(precision_at_10(lambda u:
rand.random(n_items)), 3))
print("precision@10, highest-rated films:   ", round(precision_at_10(lambda u: pop), 3))
print("precision@10, matrix factorisation:  ", round(precision_at_10(lambda u: bi + Q @
P[u]), 3))
```

Output:

```
precision@10, random films:          0.447
precision@10, highest-rated films:   0.671
precision@10, matrix factorisation:  0.912
```

Random films are liked 44.7% of the time (many films are decent). Recommending the highest-rated films to everyone, a popularity baseline, raises that to 67.1%. Matrix factorisation's *personalised* top 10 is liked 91.2% of the time. On real data you can't see true ratings for unseen items, so offline evaluation hides recent interactions (split by time, Lesson 5), and the final test is an online A/B test (Lesson 4's "Going deeper") on clicks, watch time or purchases.

8. Cold start, content and feedback loops

- **Cold start.** A new user or a new item has no interactions, so collaborative methods can't place it. Fall back on popularity, ask new users a few questions, or use **content-based** features: an item's genre, text description (TF-IDF or embeddings, Lessons 12 and 19), price; a user's stated preferences. Hybrid systems combine both.
- **Two-stage systems.** At scale, a fast *retrieval* stage picks a few hundred candidates (nearest neighbours of a user embedding, Lesson 11's "Going deeper"), then a *ranking* model (often gradient boosting or a neural network with many features) orders them.

- **Feedback loops and popularity bias.** Users can only click what they're shown, so the system's own choices shape its future training data. Popular items get more exposure and look even more popular. Mix in some exploration (Lesson 28's Thompson sampling) and measure diversity and novelty, not just accuracy.
- **What to optimise.** Clicks reward clickbait. Long-term satisfaction, return visits and explicit "not interested" signals are better guides.

9. In Python: content-based recommendations

When there are no interactions yet, item descriptions alone can drive recommendations. Here, Chennai Homes listings are recommended by the similarity of their descriptions (TF-IDF and cosine similarity), plus a small bonus for the same area:

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

df = pd.read_csv("houses.csv")   # download it from Lesson 1
T = TfidfVectorizer().fit_transform(df["description"])
sim = cosine_similarity(T) + 0.2 * (df["area"].to_numpy()[:, None] ==
df["area"].to_numpy()[None, :])
np.fill_diagonal(sim, -1)                          # never recommend the house itself
viewed = 5                                          # a buyer looked at house id 5
row = df.index[df.id == viewed][0]
top = np.argsort(sim[row])[::-1][:3]
print("viewed:", df.description[row])
print(df.loc[top, ["id", "area", "description"]].to_string(index=False))
```

Output:

```
viewed: new spacious villa with garden
 id      area                    description
 12    Suburb new spacious villa near school
  8 Outskirts    new large villa with garden
  6    Suburb     old villa needs renovation
```

A buyer who viewed house 5 ("new spacious villa with garden") is shown other new, spacious villas and houses with gardens first. This needs no ratings at all, so it works from the very first day, and it's easy to explain ("because you viewed..."). It can't discover surprising tastes, which is what collaborative methods add once interactions build up.

10. Running example: Chennai Homes

A buyer looks at a house; which others should the site show? Standardise size, bedrooms, distance, age and price, and recommend the homes with the highest cosine similarity. For the Central flat with id 2, the top matches are ids 11 and 1 (similarity 0.69 and 0.68), both nearby flats; for id 11 they are 1, 4 and 2. This is content-based recommendation: it works with no interaction history at all, which is how the cold-start problem is usually handled.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from sklearn.preprocessing import StandardScaler
from sklearn.metrics.pairwise import cosine_similarity
cols = ["size_m2", "bedrooms", "dist_km", "age_yrs", "price_lakh"]
Z = StandardScaler().fit_transform(df[cols])
sim = cosine_similarity(Z)
np.fill_diagonal(sim, -1)
for viewed in [2, 11]:
    row = df.index[df.id == viewed][0]
    top = np.argsort(sim[row])[::-1][:3]
    print("viewed", viewed, "→ similar:", df.id.iloc[top].tolist(), "
similarity", np.round(sim[row, top], 2).tolist())
```

Output:

```
viewed 2 → similar: [11, 1, 3]  similarity [0.69, 0.68, 0.33]
viewed 11 → similar: [1, 4, 2]  similarity [0.86, 0.74, 0.69]
```

⚙ WHERE THIS IS USED

- Products, films, music and news recommendations
- Job, property and course matching
- Ranking search results and choosing which content to show next

Choose it when: Whenever many users interact with many items and you need to choose a few items for each user.

Think twice when: there are few items or no interaction history: then simple rules, popularity or content similarity are enough.

× MISCONCEPTIONS TO UNLEARN

MYTH Lower rating error means better recommendations.

REALITY Users see a ranked top few; a model can have good RMSE but rank poorly. Evaluate rankings.

MYTH Missing ratings mean the user dislikes the item.

REALITY Mostly they haven't seen it. Treating every missing value as a zero biases the model.

MYTH More recommendations of what someone liked is always better.

REALITY Narrow feedback loops reduce diversity and discovery; good systems mix in exploration.

RESPONSIBLE AI · FEEDBACK LOOPS AND FILTER BUBBLES

A recommender learns from what it showed, so popular items get more popular and niche ones disappear, and users can be pushed towards ever more extreme content if that maximises clicks. Measure diversity, coverage and exposure for item creators, not just clicks; add exploration; and give users controls to see why something was shown and to change it.

11. Questions beginners often ask

Q. What if I only have clicks or purchases, not ratings?

That's implicit feedback: treat interactions as positive signals and unobserved pairs as weak negatives, use methods built for it (weighted matrix factorisation, ranking losses) and evaluate with ranking metrics.

Q. How many latent factors should I use?

Tune it on validation data: typically 10 to 200. More factors fit more detail but overfit sparse data, so tune regularisation at the same time.

Q. How are recommenders built at very large scale?

Usually in two stages: a fast candidate generator (approximate nearest neighbours on embeddings) picks hundreds of items, then a ranking model with many features orders them.

Q. How do I recommend to a brand-new user?

Use popularity, simple questions at sign-up, context (location, device) or content features, then switch to collaborative filtering as interactions arrive.

! COMMON MISTAKES TO AVOID

- Evaluating on a random split that lets the model see a user's future interactions.
- Treating unobserved user–item pairs as negative ratings.
- Reporting RMSE only, when users experience a ranked list.
- Forgetting to exclude items the user already has from their recommendations.
- Comparing only with random recommendations instead of a popularity baseline.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **What's the difference between collaborative filtering and content-based recommendation?**

The trap: Calling them the same thing.

Collaborative filtering uses only interaction patterns (users who liked this also liked that), so it needs history. Content-based uses item features (text, category, price) to find similar items, so it works for new items. Production systems usually combine both.

▼ **Why is RMSE on ratings a poor way to evaluate a recommender?**

The trap: Saying RMSE is fine because it's standard.

Users see a short ranked list, so what matters is whether the top few items are relevant: precision@k, recall@k, NDCG or hit rate on held-out interactions, split by time. RMSE weighs all items equally, including ones never shown, and ratings are missing not at random.

▼ **Explain matrix factorisation for recommendations.**

The trap: Describing it as PCA on a filled-in matrix.

Each user and item gets a vector of k latent factors; the predicted rating is the global mean plus user and item biases plus the dot product of the vectors. It's fitted only on observed entries, with L2 regularisation, by SGD or alternating least squares. The factors capture tastes such as genre without being told.

▼ **How would you handle cold start for a new item?**

The trap: "Wait until it has ratings."

Use its content features (text, category, price) to place it near similar items or to predict its latent factors, show it to a small exploration share of users, and update quickly as interactions arrive. For new users, use popularity, context and a few onboarding questions.

12. Key takeaways

- Recommenders fill in a huge, sparse **user × item** matrix; most data is **implicit** feedback with no true negatives.
- Start with **popularity and bias baselines**, which are strong and simple.
- **Collaborative filtering** uses similar users or items, on centred ratings; it needs no item information.
- **Matrix factorisation** learns low-dimensional user and item vectors with regularization; trained by SGD or ALS.
- Evaluate with **ranking metrics** (precision@k, recall@k, NDCG), time-based splits and, finally, online A/B tests.
- Handle **cold start** with content features, and watch for **feedback loops** and popularity bias.

13. Check your understanding

1. A user never clicked an item. In implicit feedback, that means...

a. They dislike it

b. Either they dislike it or they never saw it

c. They like it

2. Why are ratings centred (minus each user's average) before computing similarity?

a. So a harsh and a generous rater with the same tastes look alike

b. To make the matrix denser

c. To remove missing values

3. In matrix factorisation, a predicted rating is...

a. The item's average

b. The nearest user's rating

c. The global mean plus user and item biases plus the dot product of their factor vectors

4. A brand-new film has no ratings. Which method can still recommend it?

a. Item-item collaborative filtering

b. Content-based filtering using its genre and description

c. Matrix factorisation

5. Of the 10 films recommended to a user, 7 are relevant. What is precision@10?

a. 0.7

b. 7

c. It depends on how many relevant films exist

14. Going deeper: alternating least squares and popularity bias

INTERMEDIATE · SKIP ON A FIRST READ

Two topics behind production recommenders: **alternating least squares**, the other standard way to train matrix factorisation, and how to measure whether a recommender only pushes what's already popular.

You'll need: this lesson, ridge regression from Lesson 10, and matrix products from Lesson 17.

Alternating least squares

Matrix factorisation's loss is hard to minimise for P and Q together, but if you fix the item factors Q, finding each user's vector is just a small ridge regression with a closed-form answer, $\mathbf{p}_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top \mathbf{r}_u$, using only the items user u rated. Then fix P and solve for every item the same way. **Alternating least squares** (ALS) repeats these two steps:

```
import numpy as np
rng = np.random.default_rng(0)
n_users, n_items, k = 200, 40, 3
P_true, Q_true = rng.normal(0, 1, (n_users, k)), rng.normal(0, 1, (n_items, k))
R = P_true @ Q_true.T + rng.normal(0, 0.3, (n_users, n_items))
mask = rng.random((n_users, n_items)) < 0.2           # only 20% of ratings observed
test = (rng.random((n_users, n_items)) < 0.25) & mask
train = mask & ~test
P, Q, lam = rng.normal(0, 0.1, (n_users, k)), rng.normal(0, 0.1, (n_items, k)), 1.0
for sweep in range(1, 9):
    for u in range(n_users):                          # fix Q, solve a small ridge
problem per user
        Qi = Q[train[u]]
        P[u] = np.linalg.solve(Qi.T @ Qi + lam * np.eye(k), Qi.T @ R[u, train[u]])
    for i in range(n_items):                          # fix P, solve per item
        Pu = P[train[:, i]]
        Q[i] = np.linalg.solve(Pu.T @ Pu + lam * np.eye(k), Pu.T @ R[train[:, i], i])
    if sweep in (1, 2, 4, 8):
        rmse = np.sqrt(np.mean((R - P @ Q.T)[test] ** 2))
        print("sweep", sweep, " test RMSE", round(rmse, 3))
```

Output:

```
sweep 1  test RMSE 1.635
sweep 2  test RMSE 1.456
sweep 4  test RMSE 0.964
sweep 8  test RMSE 0.762
```

With 3 true factors and 20% of the ratings observed, the test RMSE falls from 1.635 after one sweep to 0.762 after eight. Every user's (and every item's) update is independent, so ALS parallelises very well, which is why Spark's recommender uses it. It also adapts neatly to implicit feedback, where every unobserved pair counts as a weak zero: the weighted version of Hu, Koren and Volinsky.

Measuring popularity bias

Accuracy metrics reward recommending popular items, because popular items are most likely to appear in anyone's held-out set. Two simple health checks: **catalogue coverage** (the share of items ever recommended to anyone) and the **average popularity** of recommended items:

```
popularity = train.sum(axis=0)                        # how many users interacted with
each item
scores_pop = np.tile(popularity, (n_users, 1)).astype(float)
scores_mf = P @ Q.T
for name, S in [("popular", scores_pop), ("matrix factorisation", scores_mf)]:
    S = np.where(train, -np.inf, S)                   # don't recommend items already
seen
    top = np.argsort(-S, axis=1)[:, :5]
    coverage = len(np.unique(top)) / n_items
    avg_pop = popularity[top].mean()
    print(f"{name:21s} catalogue coverage {coverage:.2f}   average popularity of
recommended items {avg_pop:.1f}")
print("average popularity across all items:", round(popularity.mean(), 1))
```

Output:

```
popular               catalogue coverage 0.30   average popularity of recommended items
38.7
matrix factorisation  catalogue coverage 0.85   average popularity of recommended items
30.4
average popularity across all items: 29.2
```

Recommending the most popular items reaches only 30% of the catalogue, and their average popularity (38.7 users) is well above the catalogue's 29.2. Matrix factorisation recommends 85% of the items, with popularity (30.4) close to average. Track these alongside precision@k: a change that raises precision slightly but halves coverage can make the product worse for users and for the people who supply the items.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **SGD or ALS for matrix factorisation: how do you choose?**

The trap: Saying one is always better.

SGD is simple, handles explicit ratings and extra terms (biases, side features) easily and is cheap per step, but needs a tuned learning rate. ALS has closed-form steps, needs no learning rate, parallelises across users and items, and handles implicit feedback with all unobserved pairs efficiently, which is why it's common on Spark. Both need regularisation tuned on validation data.

Check your understanding: going deeper

1. In ALS, with the item factors fixed, each user's vector is found by...

a. A small ridge regression on the items that user rated

b. Gradient boosting

c. Random search

2. Why does ALS parallelise well?

a. Each user's (and each item's) update is independent of the others in that step

b. It uses no matrices

c. It needs only one sweep

3. A recommender's catalogue coverage is 0.1. What does that mean?

a. Only 10% of items are ever recommended to anyone

b. Precision is 0.1

c. 10% of users get recommendations

4. Why do accuracy metrics tend to favour popular items?

a. Popular items are most likely to be in any user's held-out interactions

b. They have better ratings

c. They're cheaper

5. What's the ALS update for a user vector?

a. $(Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top \mathbf{r}_u$

b. $Q_u^\top \mathbf{r}_u$

c. $\mathbf{r}_u / \lambda$

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

The global mean rating is 3.5. An item's average is 4.1 and a user's average is 3.0. Using simple deviations, predict this user's rating of the item.

WORKING

EXERCISE 2 By hand

Two items have rating vectors (over the same three users) (5, 3, 0) and (4, 0, 2). Compute their cosine similarity.

WORKING

EXERCISE 3 By hand

A user's latent vector is (1.2, −0.5) and an item's is (0.8, 0.4). The global mean is 3.5 and both biases are 0. Predict the rating.

WORKING

EXERCISE 4 By hand

Of 10 recommended items, 4 are in the user's held-out set of 8 liked items. Compute precision@10 and recall@10.

WORKING

EXERCISE 5 Code

In the matrix factorisation code, change the number of factors from 5 to 1 and to 20. How does the test RMSE change?

WORKING

EXERCISE 6 Intermediate

A user rated two items whose factor vectors (1 factor) are 2 and 1, with ratings 4 and 3. With $\lambda = 1$, find the user's factor from the ALS formula.

WORKING

EXERCISE 7 Intermediate

A catalogue has 500 items. Across all users' top-10 lists, 120 distinct items appear. What's the catalogue coverage?

WORKING

NOTES

PART 6 · INTERMEDIATE TOPICS · ~55 MIN

Lesson 28: Bayesian & Probabilistic ML

Priors and posteriors, Bayesian A/B tests, Bayesian linear regression, Gaussian processes, MCMC, and Thompson sampling.

BY THE END OF THIS LESSON YOU CAN

- Update a Beta prior with conversion data and answer decision questions such as P(B > A) and expected loss
- Explain Bayesian linear regression's posterior and why its prediction intervals widen away from the data
- Describe Gaussian processes and when their uncertainty is useful
- Sample a posterior with the Metropolis algorithm and use posteriors to decide with Thompson sampling

↺ BEFORE YOU START

This lesson builds on:

- Lesson 4: Probability & Statistics — Probabilities run from 0 to 1. Bayes' theorem turns $P(B \mid A)$ into $P(A \mid B)$. Mean and standard deviation summarise data; the z-score $(x - \mu)/\sigma$ standardises it.
- Lesson 10: Overfitting, Regularization & Bias–Variance — Error = bias² + variance + noise. Regularization adds a penalty on weight size (L2 ridge shrinks weights; L1 lasso zeros some) to reduce variance; its strength is chosen by cross-validation.
- Lesson 15: Support Vector Machines —

Where this fits: Builds on Bayes' theorem (Lesson 4), shows that ridge (Lesson 10) is a Bayesian posterior mean, and reuses the kernels of Lesson 15 in Gaussian processes.

1. Why think in probabilities about the model itself?

Most models you've built give one answer: one set of weights, one predicted price. But with 10 data points the weights are very uncertain, and with 10 million they're not, and a single number hides that difference. **Bayesian machine learning** treats everything unknown, including the model's parameters, as uncertain, describes that uncertainty with probability distributions, and updates them as data arrives.

That gives you things ordinary models don't: honest error bars on predictions, sensible behaviour with little data, a principled way to include prior knowledge, and decisions that weigh risks ("the chance that B is worse, and how much we'd lose"). This lesson builds the core tools from scratch: conjugate updating for A/B tests, Bayesian linear regression, Gaussian processes, MCMC sampling, and Thompson sampling for learning while deciding.

◎ THE CORE IDEA

Start with a **prior** (what's plausible before the data), multiply by the **likelihood** (how well each possibility explains the data), and you get the **posterior**: an updated belief about every possibility. Predictions and decisions then average over the whole posterior instead of trusting a single best guess.

2. Bayes' theorem for parameters

Lesson 4 used Bayes' theorem for events. For a model's parameters θ and data D it reads:

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)} \;\propto\; \text{likelihood} \times \text{prior}$$

Lesson 4's "Going deeper" showed three useful facts: maximising the likelihood alone gives the MLE; maximising the posterior gives the MAP estimate, which equals ridge or lasso for normal or Laplace priors; and with a **conjugate** prior, such as a Beta prior for a probability, the posterior has a simple closed form. This lesson uses the *whole* posterior, not just its peak.

3. A Bayesian A/B test

Lesson 4's "Going deeper" ran a classical test on a checkout page: A converted 120 of 2,400 visitors and B 156 of 2,400, with $p = 0.026$. The Bayesian version gives each page's conversion rate a Beta(1, 1) prior (every rate equally plausible) and updates it with the data. Then it simply asks the questions a manager cares about, by drawing many samples from the two posteriors:

```
import numpy as np
from scipy import stats

# A/B test the Bayesian way: Beta(1, 1) priors, then the observed conversions
a_conv, a_n, b_conv, b_n = 120, 2400, 156, 2400
post_a = stats.beta(1 + a_conv, 1 + a_n - a_conv)
post_b = stats.beta(1 + b_conv, 1 + b_n - b_conv)
rng = np.random.default_rng(0)
da, db = post_a.rvs(200000, random_state=rng), post_b.rvs(200000, random_state=rng)
print("95% credible interval, A:", np.round(post_a.ppf([0.025, 0.975]), 4))
print("95% credible interval, B:", np.round(post_b.ppf([0.025, 0.975]), 4))
print("probability that B converts better than A:", round(np.mean(db > da), 4))
print("expected lift if we switch to B:", round(np.mean(db - da), 4), "  risk (expected
loss if B is actually worse):", round(np.mean(np.maximum(da - db, 0)), 5))
```

Output:

```
95% credible interval, A: [0.042  0.0595]
95% credible interval, B: [0.0558 0.0756]
probability that B converts better than A: 0.9869
expected lift if we switch to B: 0.015   risk (expected loss if B is actually worse):
3e-05
```

✎ WORKED EXAMPLE — THE POSTERIOR BY HAND

A Beta(a, b) prior plus s successes and f failures gives a Beta($a + s$, $b + f$) posterior. For page B: Beta($1 + 156$, $1 + 2{,}244$) = Beta(157, 2,245).

Its mean is $\frac{157}{157+2245} = \frac{157}{2402} \approx 0.0654$: very close to the observed $156/2400 = 0.065$, because 2,400 visitors overwhelm a weak prior.

Its standard deviation is $\sqrt{\frac{ab}{(a+b)^2(a+b+1)}} = \sqrt{\frac{157 \times 2245}{2402^2 \times 2403}} \approx 0.0050$, so the rate is about $6.5\% \pm 1.0\%$ (two standard deviations), matching the credible interval above.

There's a 98.7% probability that B really converts better, the expected gain from switching is 1.5 percentage points, and the expected loss if B is actually worse is tiny (0.003 percentage points). These are direct answers about the thing you care about, unlike a p-value (Lesson 4's "Going deeper"). The price is that you must choose a prior; with plenty of data, reasonable priors give almost the same answer.

4. Try it: updating a belief

The grey curve is the prior belief about a conversion rate, centred on 10%; the green curve is the posterior after the visitors you add. Make the prior stronger and see how much more data it takes

to move it. Add 100 visitors at a time and watch the curve narrow around 8%.

Interactive demo — open *lesson-28.html* on the website to try it.

5. Bayesian linear regression

Give the weights of a linear regression a normal prior, $\mathbf{w} \sim \mathcal{N}(0, \tau^2 I)$, and assume normal noise with variance σ^2. The posterior is again normal, with

$$\Sigma_{\text{post}} = \left(\tfrac{1}{\sigma^2} X^\top X + \tfrac{1}{\tau^2} I\right)^{-1}, \qquad \mathbf{m}_{\text{post}} = \tfrac{1}{\sigma^2}\, \Sigma_{\text{post}} X^\top \mathbf{y}$$

The posterior mean is exactly ridge regression with $\lambda = \sigma^2/\tau^2$ (Lesson 10). What's new is the covariance Σ_{post}: it says how uncertain each weight is, and it gives each prediction its own error bar, $\boldsymbol{\phi}^\top \Sigma_{\text{post}} \boldsymbol{\phi} + \sigma^2$, combining uncertainty about the line with the noise.

```
# Bayesian linear regression: a normal prior on the weights gives a full posterior, not
just one line
x = np.array([0.5, 1.0, 1.5, 3.0, 3.5])
y = np.array([1.1, 1.9, 3.2, 5.8, 7.1])
X = np.column_stack([np.ones_like(x), x])
sigma2, tau2 = 0.25, 10.0                                  # noise variance and prior
variance of each weight
S_post = np.linalg.inv(X.T @ X / sigma2 + np.eye(2) / tau2)  # posterior covariance
m_post = S_post @ X.T @ y / sigma2                         # posterior mean (= ridge
with lambda = sigma2/tau2)
print("posterior mean (intercept, slope):", m_post.round(3))
print("posterior std  (intercept, slope):", np.sqrt(np.diag(S_post)).round(3))
for x_new in [2.0, 6.0]:
    phi = np.array([1, x_new])
    mean, var = phi @ m_post, phi @ S_post @ phi + sigma2
    print(f"x {x_new}: predicted y {mean:.2f}, 95% prediction interval [{mean - 1.96 *
np.sqrt(var):.2f}, {mean + 1.96 * np.sqrt(var):.2f}]")
```

Output:

```
posterior mean (intercept, slope): [0.086 1.965]
posterior std  (intercept, slope): [0.425 0.192]
x 2.0: predicted y 4.02, 95% prediction interval [2.94, 5.09]
x 6.0: predicted y 11.88, 95% prediction interval [10.00, 13.75]
```

With five points, the slope is about 1.97 ± 0.19. The prediction interval at $x = 2$, inside the data, is about 2.1 units wide; at $x = 6$, beyond the data, it's about 3.8 units wide. The model knows it is extrapolating, which ordinary least squares can't tell you.

6. Gaussian processes

A **Gaussian process** (GP) goes further: instead of a distribution over a few weights, it puts a prior directly over *functions*, controlled by a kernel that says how similar nearby outputs are (the RBF kernel from Lesson 15). After seeing data, it predicts a mean and a standard deviation at every point. Lesson 23's "Going deeper" used one to drive Bayesian optimisation.

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel

# A Gaussian process: uncertainty grows away from the data
xs = np.array([1, 2, 3, 6, 7, 8.0])[:, None]
ys = np.sin(xs[:, 0]) + np.random.default_rng(2).normal(0, 0.1, 6)
gp = GaussianProcessRegressor(RBF(1.0) + WhiteKernel(0.01), random_state=0).fit(xs, ys)
for q in [2.0, 4.5, 11.0]:
    m, s = gp.predict([[q]], return_std=True)
    print(f"x {q:4}: mean {m[0]:+.2f}, std {s[0]:.2f}")
```

Output:

```
x  2.0: mean +0.86, std 0.00
x  4.5: mean -1.19, std 0.45
x 11.0: mean -0.01, std 0.99
```

At $x = 2$, where there's data, the GP is confident (standard deviation about 0). At $x = 4.5$, in the gap between the two groups of points, its uncertainty rises to 0.45. At $x = 11$, far beyond all data, it falls back towards the prior: mean near 0 and standard deviation about 1. GPs are excellent for small, smooth problems (calibration curves, sensor data, experiment design) and give honest uncertainty, but exact GPs cost $O(n^3)$, so they're limited to a few thousand points without approximations.

7. When there's no formula: MCMC

Most posteriors have no neat closed form. **Markov chain Monte Carlo** (MCMC) draws samples from them anyway. The simplest version, the **Metropolis algorithm**: propose a small random move; if the posterior is higher there, accept it; if lower, accept it with probability equal to the ratio. The chain then visits each region in proportion to its posterior probability, and only *ratios* are needed, so the awkward normalising constant $p(D)$ cancels.

Here we estimate the centre of seven measurements, one of which (8.5) looks like a mistake. A Cauchy likelihood, which has heavy tails, makes the model robust to it:

```
# MCMC from scratch: sample a posterior that has no neat formula with the Metropolis
algorithm
data = np.array([2.1, 2.9, 3.4, 2.2, 8.5, 3.0, 2.7])        # one suspicious value (8.5)
def log_post(mu):                                            # Cauchy likelihood (robust
to outliers) and normal prior
    return np.sum(stats.cauchy.logpdf(data, loc=mu, scale=0.5)) + stats.norm.logpdf(mu,
0, 10)
rng = np.random.default_rng(3)
mu, chain, accepted = 0.0, [], 0
for step in range(20000):
    prop = mu + rng.normal(0, 0.5)                           # propose a small random move
    if np.log(rng.random()) < log_post(prop) - log_post(mu):
        mu, accepted = prop, accepted + 1                    # accept with probability
min(1, ratio)
    chain.append(mu)
chain = np.array(chain[2000:])                               # drop the warm-up
print("acceptance rate:", round(accepted / 20000, 2))
print("posterior mean of the centre:", round(chain.mean(), 2), "  95% interval:",
np.round(np.percentile(chain, [2.5, 97.5]), 2))
print("ordinary mean of the data, for comparison:", round(data.mean(), 2))
```

Output:

```
acceptance rate: 0.52
posterior mean of the centre: 2.76   95% interval: [2.2  3.28]
ordinary mean of the data, for comparison: 3.54
```

About half of the proposals were accepted, a healthy rate. The posterior mean of the centre is 2.76 with a 95% interval of 2.2 to 3.28, while the ordinary mean (3.54) is dragged up by the outlier. Modern tools (Stan, PyMC, NumPyro) use far more efficient samplers such as Hamiltonian Monte Carlo and NUTS, but the idea is the same: write down the prior and likelihood, and let the sampler produce the posterior.

8. Deciding under uncertainty: Thompson sampling

A classical A/B test splits traffic evenly until it ends, so half the visitors keep seeing the worse page. A **multi-armed bandit** learns and earns at the same time. **Thompson sampling** is the Bayesian way: for each visitor, draw one plausible conversion rate for each design from its current posterior, show the design whose draw is highest, and update that design's posterior with the result. Uncertain designs sometimes win the draw (exploration); clearly good ones win most of the time (exploitation).

```
# Thompson sampling: a bandit that learns which of 3 page designs converts best while it
runs
true_rates = np.array([0.04, 0.05, 0.065])
wins, losses = np.ones(3), np.ones(3)                       # Beta(1, 1) prior for each
design
rng = np.random.default_rng(4)
shown = np.zeros(3, int)
for visitor in range(10000):
    choice = np.argmax(rng.beta(wins, losses))              # sample a plausible rate for
each, show the best
    shown[choice] += 1
    if rng.random() < true_rates[choice]: wins[choice] += 1
    else: losses[choice] += 1
print("visitors shown each design:", shown.tolist())
print("conversions:", int(wins.sum() - 3), "  versus a fixed even split (expected):",
round(10000 * true_rates.mean()))
```

Output:

```
visitors shown each design: [354, 458, 9188]
conversions: 652   versus a fixed even split (expected): 517
```

With true rates of 4%, 5% and 6.5%, Thompson sampling quickly shifted traffic: 9,188 of the 10,000 visitors saw the best design. It earned 652 conversions, against about 517 for an even three-way split. Bandits are used for headlines, recommendations (Lesson 27's feedback loops) and ad selection. The trade-off is that the weaker designs get too little traffic for a precise comparison, so use a classical or Bayesian A/B test when you need an exact measurement, and a bandit when you need results now.

9. In Python: Bayesian ridge with uncertainty

Scikit-learn's `BayesianRidge` learns the noise and prior variances from the data and returns a standard deviation with every prediction:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import BayesianRidge

df = pd.read_csv("houses.csv")   # download it from Lesson 1
X, y = df[["size_m2", "dist_km", "age_yrs"]], df["price_lakh"]
model = BayesianRidge().fit(X, y)
new = pd.DataFrame({"size_m2": [100, 200], "dist_km": [8, 30], "age_yrs": [5, 1]})
mean, std = model.predict(new, return_std=True)
for (_, h), m, s in zip(new.iterrows(), mean, std):
    print(f"{h.size_m2:.0f} m², {h.dist_km:.0f} km, {h.age_yrs:.0f} yrs: {m:.0f} lakh ± {1.96 * s:.0f} (95%)")
```

Output:

```
100 m², 8 km, 5 yrs: 79 lakh ± 12 (95%)
200 m², 30 km, 1 yrs: 111 lakh ± 20 (95%)
```

For a typical home (100 m², 8 km, 5 years), the model gives a prediction with a modest interval; for an unusual one (200 m², 30 km away), outside the range of the 12 training homes, the interval is much wider. Communicating that kind of uncertainty is often worth more than a slightly more accurate point prediction.

10. Running example: Chennai Homes

How sure can we be about which areas sell fast, with only 12 homes? Give each area's "sold fast" rate a Beta(1, 1) prior and update it with the area's homes. Central sold 3 of 4 fast (posterior mean 0.67) and the Suburb 3 of 5 (0.57), but their 90% credible intervals, about 0.34 to 0.92 and 0.27 to 0.85, overlap heavily. The Outskirts sold none of 3, yet the posterior still allows rates up to about 0.53. With this little data, the intervals are the honest answer.

Same 12 houses as Lesson 1. With so little data, treat the numbers as an illustration of the method, not as reliable estimates.

```
import pandas as pd
import numpy as np
df = pd.read_csv("houses.csv")   # download it from Lesson 1
from scipy import stats
for area, g in df.groupby("area"):
    a, b = 1 + g.sold_fast.sum(), 1 + (1 - g.sold_fast).sum()     # Beta(1, 1)
prior
    lo, hi = stats.beta(a, b).ppf([0.05, 0.95])
    print(f"{area:9s} sold fast {g.sold_fast.sum()} of {len(g)}  posterior mean
{a / (a + b):.2f}  90% interval [{lo:.2f}, {hi:.2f}]")
```

Output:

```
Central   sold fast 3 of 4  posterior mean 0.67  90% interval [0.34, 0.92]
Outskirts sold fast 0 of 3  posterior mean 0.20  90% interval [0.01, 0.53]
Suburb    sold fast 3 of 5  posterior mean 0.57  90% interval [0.27, 0.85]
```

⚙ WHERE THIS IS USED

- A/B tests that report the probability one version is better
- Small-data science and engineering models that need honest error bars
- Bandits for headlines and offers, and Bayesian optimisation of expensive experiments

Choose it when: Whenever uncertainty matters to the decision, data is limited, or you have real prior knowledge to include.

Think twice when: you have huge data and only need point predictions: then the posterior is narrow and ordinary models are simpler and faster.

× MISCONCEPTIONS TO UNLEARN

MYTH Bayesian methods need lots of computation.

REALITY Conjugate models such as the Beta A/B test are one line of arithmetic; MCMC is only needed when there's no formula.

MYTH The prior lets you get any answer you want.

REALITY With enough data, reasonable priors converge; a prior that changes the conclusion is a sign the data is weak, which is worth knowing.

MYTH A credible interval and a confidence interval are the same thing.

REALITY Only a credible interval is a direct probability statement about the parameter.

RESPONSIBLE AI · PRIORS AND UNCERTAINTY IN HIGH-STAKES DECISIONS

A prior encodes assumptions, and in decisions about people (credit, health, policing) a prior built from biased history carries that bias into every new case. State priors openly and test how much they move conclusions. Equally, uncertainty is a responsibility: when an interval is wide, say so, and route the decision to a person rather than presenting a guess as a fact.

11. Questions beginners often ask

Q. Isn't choosing a prior subjective?

Somewhat, but it's explicit and can be checked: try a few reasonable priors and see whether the conclusion changes. With plenty of data the likelihood dominates and they agree.

Q. What's the difference between a credible interval and a confidence interval?

A 95% credible interval contains the parameter with 95% probability given the data and prior. A 95% confidence interval comes from a procedure that covers the true value in 95% of repeated samples; it isn't a probability statement about this interval.

Q. Which tools do people use for Bayesian modelling?

PyMC, Stan and NumPyro for general models with MCMC or variational inference; scikit-learn's BayesianRidge and GaussianProcessRegressor for the common simple cases.

Q. Can neural networks be Bayesian?

Approximately: ensembles of networks, Monte Carlo dropout and variational methods give useful uncertainty estimates, though exact Bayesian deep learning is too expensive.

! COMMON MISTAKES TO AVOID

- Using a strong prior without checking how much it changes the result.
- Reporting only the posterior mean and throwing away the uncertainty.
- Running MCMC without checking the acceptance rate or whether the chain has settled.
- Using a bandit when you need an unbiased, precise comparison of every option.
- Using an exact Gaussian process on hundreds of thousands of points.

! INTERVIEW CORNER

Hidden questions — answer in your head first, then open.

▼ **What's the difference between a credible interval and a confidence interval?**

The trap: Saying they're the same.

A 95% credible interval contains the parameter with 95% probability given the data and the prior. A 95% confidence interval comes from a procedure that covers the true value in 95% of repeated samples; this particular interval either contains it or doesn't.

▼ **How would you run a Bayesian A/B test on conversion rates?**

The trap: Only computing a p-value.

Give each version's rate a Beta prior, update it to Beta(a + conversions, b + non-conversions), and draw samples from both posteriors to estimate P(B > A), the expected lift and the expected loss of choosing B. Decide when the expected loss falls below a threshold, and check that reasonable priors give the same decision.

▼ **Why is ridge regression a Bayesian method in disguise?**

The trap: Saying it isn't related.

With Gaussian noise of variance σ^2 and a Gaussian prior $N(0, \tau^2)$ on the weights, the posterior mean (and MAP) of the weights is the ridge solution with $\lambda = \sigma^2/\tau^2$. A Laplace prior gives lasso at the MAP. The full Bayesian treatment also gives the posterior covariance, and so uncertainty for each prediction.

▼ **What's Thompson sampling, and when would you use it instead of an A/B test?**

The trap: Calling it random allocation.

For each decision, sample a plausible value of each option's reward from its posterior and choose the best sample; update the chosen option's posterior with the result. It shifts traffic to better options while still exploring uncertain ones. Use it when you want to earn while learning (headlines, offers); use an A/B test when you need an unbiased, precise estimate of each option.

12. Key takeaways

- Bayesian ML treats unknown parameters as uncertain: **posterior ∝ likelihood × prior**.
- With **conjugate priors** (Beta for rates) the posterior is a simple update; a Bayesian A/B test reports $P(B > A)$ and expected loss directly.
- **Bayesian linear regression**'s mean is ridge; its covariance gives error bars that widen away from the data.
- **Gaussian processes** put priors on functions and give uncertainty everywhere, at $O(n^3)$ cost.
- **MCMC** (Metropolis, HMC) samples posteriors that have no formula, using only ratios.
- **Thompson sampling** uses posteriors to balance exploring and exploiting while deciding.

13. Check your understanding

1. A Beta(2, 8) prior is updated with 3 successes and 7 failures. The posterior is...

a. Beta(5, 15)

b. Beta(3, 7)

c. Beta(10, 20)

2. The posterior mean of Bayesian linear regression with a normal prior equals...

a. Ordinary least squares

b. Ridge regression

c. Lasso

3. Far from all training data, a Gaussian process's predictive standard deviation...

a. Shrinks to zero

b. Stays equal to the noise

c. Grows back towards the prior's

4. Why does the Metropolis algorithm not need the normalising constant $p(D)$?

a. It only uses ratios of posterior values, in which $p(D)$ cancels

b. It estimates $p(D)$ first

c. Because the prior is flat

5. In Thompson sampling, how is the design for each visitor chosen?

a. Always the one with the highest observed rate

b. Draw a plausible rate for each design from its posterior and pick the highest draw

c. At random, evenly

14. Going deeper: prior sensitivity and checking MCMC

INTERMEDIATE · SKIP ON A FIRST READ

Two habits of careful Bayesian work: checking how much the prior changes the answer, and checking that an MCMC sampler has really explored the posterior before trusting its samples.

You'll need: this lesson and variance from Lesson 2.

How much does the prior matter?

Here three analysts give a conversion rate different priors: flat Beta(1, 1); sceptical Beta(10, 190), centred on 5%; and optimistic Beta(30, 170), centred on 15%. All see the same 6% conversion rate, with growing numbers of visitors:

```
from scipy import stats
for visitors, conv in [(50, 3), (500, 30), (5000, 300)]:
    row = []
    for name, a, b in [("flat", 1, 1), ("sceptical", 10, 190), ("optimistic", 30, 170)]:
        post = stats.beta(a + conv, b + visitors - conv)
        row.append(f"{name} {post.mean():.3f}")
    print(f"{visitors:5d} visitors (6% converted):", "   ".join(row))
```

Output:

```
   50 visitors (6% converted): flat 0.077   sceptical 0.052   optimistic 0.132
  500 visitors (6% converted): flat 0.062   sceptical 0.057   optimistic 0.086
 5000 visitors (6% converted): flat 0.060   sceptical 0.060   optimistic 0.063
```

With 50 visitors, the posterior means range from 5.2% to 13.2%: the priors dominate. With 500 they range from 5.7% to 8.6%, and with 5,000 they all agree within a third of a percentage point. A Beta(a, b) prior acts like $a + b$ earlier visitors, so the optimistic prior is worth 200 of them. A **sensitivity analysis** like this belongs in any Bayesian report: if reasonable priors disagree, the data can't settle the question yet.

Has the sampler converged?

An MCMC chain starts somewhere arbitrary and needs time to reach the posterior; if its steps are too small it may never get there. The standard check is to run several chains from very different starting points and compare them with the $\hat{R}$ ("R-hat") statistic, which compares the spread between chains with the spread within them. Values near 1 mean the chains agree; above about 1.01 means keep sampling or fix the sampler. This version of the lesson's model uses a Cauchy likelihood with scale 1 and a wide normal prior:

```
import numpy as np
data = np.array([2.9, 3.1, 2.4, 3.3, 2.7, 8.5, 2.8])
def log_post(m):
    return -np.sum(np.log1p((data - m) ** 2)) - m ** 2 / 200       # Cauchy likelihood,
wide normal prior
def chain(start, step, n=4000, seed=0):
    rng, m, out = np.random.default_rng(seed), start, []
    for _ in range(n):
        prop = m + rng.normal(0, step)
        if np.log(rng.random()) < log_post(prop) - log_post(m):
            m = prop
        out.append(m)
    return np.array(out[n // 2:])                                    # drop the first
half as warm-up
def r_hat(chains):
    c = np.array(chains); n = c.shape[1]
    W, B = c.var(axis=1, ddof=1).mean(), n * c.mean(axis=1).var(ddof=1)
    return np.sqrt(((n - 1) / n * W + B / n) / W)
for step in [0.5, 0.005]:
    chains = [chain(s, step, seed=i) for i, s in enumerate([-10, 0, 10, 20])]
    print("step", step, " chain means", [round(float(c.mean()), 2) for c in chains], "
R-hat", round(float(r_hat(chains)), 2))
```

Output:

```
step 0.5  chain means [2.91, 2.91, 2.95, 2.9]  R-hat 1.0
step 0.005  chain means [-9.8, -0.06, 9.84, 20.19]  R-hat 122.7
```

With a step size of 0.5, four chains starting at −10, 0, 10 and 20 all settle at about 2.9, and $\hat{R}$ is 1.0. With a step size of 0.005 each chain barely moves from where it started, and $\hat{R}$ is enormous. Each chain alone would look perfectly smooth, which is why one chain is never enough. Stan and PyMC report $\hat{R}$ and the effective sample size automatically; check both before using the samples.

! INTERVIEW CORNER · INTERMEDIATE

Hidden questions — answer in your head first, then open.

▼ **How do you check that an MCMC run is trustworthy?**

The trap: "Look at the trace and it looks fine."

Run several chains from dispersed starting points; check that $\hat{R}$ is close to 1 (below about 1.01) and that the effective sample size is large enough for the quantities you report; inspect trace plots and, for HMC, divergent transitions. Then do posterior predictive checks: simulate data from the fitted model and compare with the real data.

Check your understanding: going deeper

1. A Beta(30, 170) prior is worth roughly how many earlier visitors?

a. 30

b. 200

c. 170

2. Three reasonable priors give clearly different posteriors. What should you conclude?

a. The data isn't yet strong enough to settle the question

b. Pick the prior you like

c. Bayesian methods don't work

3. What does $\hat{R}$ compare?

a. The spread between chains with the spread within chains

b. Prior and posterior means

c. Training and test error

4. Why is one smooth-looking chain not enough?

a. A stuck chain can look smooth while exploring only a tiny region

b. Chains are always wrong

c. It's too slow

5. $\hat{R}$ is 1.3. What do you do?

a. Run longer or improve the sampler before using the samples

b. Report the results

c. Lower the prior variance

15. Practice exercises

Work each exercise in the space provided, then compare with the solutions at the back of the workbook. The lesson's Jupyter notebook has two more coding challenges with automatic checks.

EXERCISE 1 By hand

A Beta(1, 1) prior is updated with 30 conversions from 200 visitors. Give the posterior and its mean.

WORKING

EXERCISE 2 By hand

A Beta(20, 180) prior (centred on 10%) is updated with 5 conversions from 100 visitors. What's the posterior mean, and how does it compare with the observed 5%?

WORKING

EXERCISE 3 By hand

In Bayesian linear regression the noise variance is 4 and the prior variance of the weights is 2. What ridge λ is the posterior mean equivalent to?

WORKING

EXERCISE 4 By hand

In a Metropolis step the current posterior density is 0.30 and the proposal's is 0.12. With what probability is the proposal accepted?

WORKING

EXERCISE 5 Code

In the Thompson sampling code, make the true rates closer (5%, 5.5%, 6%). How does the share of traffic for the best design change?

WORKING

EXERCISE 6 Intermediate

A Beta(10, 190) prior sees 30 conversions from 500 visitors. Compute the posterior mean.

WORKING

EXERCISE 7 Intermediate

Two chains have within-chain variances 1.0 and 1.2, so W = 1.1, with n = 1000 samples each, and B/n = 0.05. Compute $\hat{R} = \sqrt{((n-1)/n \cdot W + B/n)/W}$.

WORKING

NOTES

Answers

Lesson 0: Python, NumPy & pandas for Beginners

Quiz answers

1. **(b)** Positions start at 0, so position 1 is the second item, 20.
2. **(a)** A slice runs from the start position up to, but not including, the stop position: positions 1 and 2.
3. **(b)** NumPy applies arithmetic to every element. (With a plain Python list, `* 2` would repeat the list instead.)
4. **(c)** `axis=0` averages down the rows, giving one result per column — here, the average of each of the 5 features.
5. **(b)** The comparison makes a True/False column; putting it inside `df[...]` keeps the True rows. The first option only gives the True/False values.

Exercise solutions

1. (a) `a[-2]` counts from the end: the second-to-last item, **12**. (b) Positions 1, 2 and 3 (the stop position 4 is excluded): **[6, 9, 12]**.
2. Two rows and three columns: shape **(2, 3)**. `axis=0` adds down the rows, one total per column: **[5, 7, 9]**. `X[:, 1]` is every row, column 1 (the second column): **[2, 5]**.
3. ```
   df = pd.read_csv("houses.csv")
   print((df["price_lakh"] > 80).sum())                    # 5
   print(df[df["sold_fast"] == 1]["price_lakh"].mean())    # 75.1666...
   ```

   Summing a True/False column counts the Trues. **5** houses, and the fast sellers average about **75.17** lakh.
4. ```
   def celsius_to_f(c):
       return c * 9 / 5 + 32

   print(celsius_to_f(37))   # 98.6
   ```

   Body temperature, **98.6** °F.

Lesson 1: What Is Machine Learning?

Quiz answers

1. **(b)** The label is a number on a continuous scale, so it is **regression**. Classification would predict a category such as "hot / mild / cold".
2. **(c)** The label is the answer we want to predict. Words and sender are **features** — the inputs.
3. **(a)** Errors are -1 and 0; squared: 1 and 0; average $= (1 + 0)/2 = 0.5$.

4. **(b)** There are no labels — the algorithm discovers the groups itself. That is **unsupervised** learning (specifically clustering).
5. **(c)** It memorised the training examples instead of learning the general pattern: **overfitting**.

Exercise solutions

1. (a) regression — a number. (b) classification — yes/no. (c) clustering — no labels, discover groups. (d) regression — a number.
2. Errors $\hat{y} - y$: $1, -1, 0$. Squared: $1, 1, 0$. MSE $= 2/3 \approx 0.667$.
3. Line 1 predicts $3, 5, 7$; errors $0, 1, -1$; MSE $= 2/3 \approx 0.667$.
   Line 2 predicts $2.5, 4.5, 6.5$; errors $-0.5, 0.5, -1.5$; squared $0.25, 0.25, 2.25$; MSE $= 2.75/3 \approx 0.917$.
   $\hat{y} = 2x + 1$ **is better.**
4. Replace the last line with `print(model.predict([[3.5]]))`. It prints about `[7.015]` — roughly 701,500 USD, since the learned line is about $1.99x + 0.05$.

Lesson 2: Vectors & Matrices

Quiz answers

1. **(b)** $1 \cdot 4 + 2 \cdot 0 + 3 \cdot (-1) = 4 + 0 - 3 = 1$.
2. **(c)** $\sqrt{36 + 64} = \sqrt{100} = 10$. (14 is the L1 norm.)
3. **(a)** Rows = examples, columns = features: 500×20.
4. **(b)** $\cos 90^\circ = 0$, so non-zero vectors with dot product 0 are at a right angle (orthogonal).
5. **(c)** $(3 \times \mathbf{4})(\mathbf{4} \times 2)$: inner sizes match, result is outer sizes 3×2.
6. **(b)** L1 adds the sizes of the entries: $2 + 3 + 1 = 6$. ($\sqrt{14}$ is the L2 norm and 3 is the L∞ norm.)
7. **(a)** Rank counts independent columns. With one redundant column, ordinary least squares has no unique answer.
8. **(b)** That's the definition of an eigenvector with eigenvalue 3: same direction, three times as long.
9. **(b)** The values drop sharply after the second. The remaining ones are tiny and mostly describe noise.
10. **(b)** $(X^\top X + \lambda I)\mathbf{v} = (\mu + \lambda)\mathbf{v}$ for each eigenvector. The smallest eigenvalue rises, so the condition number falls.

Exercise solutions

1. $\mathbf{a} + \mathbf{b} = (4, 3, 2)$. $2\mathbf{a} - \mathbf{b} = (6 - 1, -2 - 4, 4 - 0) = (5, -6, 4)$. $\mathbf{a} \cdot \mathbf{b} = 3 - 4 + 0 = -1$ (slightly opposite directions).

2. $\sqrt{4+9+36} = \sqrt{49} = 7$.
   $\cos\theta = \frac{1\cdot 1 + 0\cdot 1}{1\cdot\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0.707$ (an angle of 45°).
3. Row by row: $2-2=0$, $6-4=2$, $10-6=4$, so $X\mathbf{w} = (0, 2, 4)$.
   AB is 2×4 (inner sizes 3 match; outer sizes remain).
4. `import numpy as np`
   `a, b = np.array([3, -1, 2]), np.array([1, 4, 0])`
   `print(a + b, 2*a - b, a @ b)` → `[4 3 2] [ 5 -6 4] -1`
   `print(np.linalg.norm([2, -3, 6]))` → `7.0`
   `print(np.array([[1,2],[3,4],[5,6]]) @ np.array([2,-1]))` → `[0 2 4]`
5. L1 $= 1+2+2 = 5$; L2 $= \sqrt{1+4+4} = 3$; L∞ $= 2$.
6. $\det(A - \lambda I) = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 2)(\lambda - 5)$, so $\lambda = 2$ and 5. Check: they add up to the trace $4+3=7$ and multiply to the determinant $12-2=10$.
7. $\sum \sigma_i^2 = 100 + 16 + 1 = 117$. Rank 1 keeps $100/117 \approx 0.855$ (85.5%). The leftover error is $\sqrt{16+1}/\sqrt{117} \approx 0.381$ in relative Frobenius norm.
8. Before: $200/0.5 = 400$. After: $(200+9.5)/(0.5+9.5) = 209.5/10 = 20.95$.
9. `(s[:3]**2).sum() / (s**2).sum()` gives about 0.975: the 3 hidden tastes explain 97.5% of the table's squared size. The rest is noise.

Lesson 3: Derivatives & Gradients

Quiz answers

1. **(c)** Power rule: $5 \cdot 4x^3 = 20x^3$.
2. **(a)** A negative derivative means the loss goes down as w goes up. Gradient descent does $w - \eta(-3) = w + 3\eta$: it increases w.
3. **(b)** Outer derivative $2(2x-5)$ times inner derivative 2: $4(2x-5)$.
4. **(b)** Treat b as a constant: $\frac{\partial}{\partial w}(3w) = 3$ and $\frac{\partial}{\partial w}(wb) = b$.
5. **(c)** It points uphill. That's why gradient descent subtracts it.
6. **(b)** One row per output (3) and one column per input (4).
7. **(c)** Curving up in one direction and down in another: a saddle.
8. **(a)** There is only one valley, so you can't get stuck in a worse one. You still need a sensible learning rate.
9. **(a)** A quadratic is exactly its own second-order Taylor approximation, so jumping to the bottom of that bowl lands on the true minimum.
10. **(b)** Correct gradients typically agree to about 10^{-7} or better. An error of 5×10^{-2} points to a mistake, such as a missing chain-rule factor.

Exercise solutions

1. $f'(x) = 6x - 5$. At $x = 2$: $12 - 5 = 7$ (going up, fairly steeply).
2. Outer: $3u^2$; inner $u = 2x + 1$ with derivative 2. So $f'(x) = 3(2x+1)^2 \cdot 2 = 6(2x+1)^2$, and $f'(0) = 6$.
3. $\frac{\partial f}{\partial w} = 2wb$, $\frac{\partial f}{\partial b} = w^2 + 3$. At $(2, 1)$: $\nabla f = (4, 7)$.
4. $L'(w) = 2(w - 4)$, so $w \leftarrow w - 0.5(w - 4)$.
   Step 1: $0 + 2 = 2$. Step 2: $2 + 1 = 3$. Step 3: $3 + 0.5 = 3.5$.
   Each step halves the distance to the minimum at $w = 4$.
5. $J = \begin{pmatrix} y & x \\ 1 & 2y \end{pmatrix}$, so at $(2, 3)$: $\begin{pmatrix} 3 & 2 \\ 1 & 6 \end{pmatrix}$.
6. $H = \begin{pmatrix} 6 & 2 \\ 2 & 2 \end{pmatrix}$. $\det(H - \lambda I) = \lambda^2 - 8\lambda + 8 = 0$, so $\lambda = 4 \pm 2\sqrt{2} \approx 1.172$ and 6.828.
   Both are positive, so H is positive definite and L is convex.
7. $L' = 4w^3 = 32$ and $L'' = 12w^2 = 48$ at $w = 2$. Step: $w \leftarrow 2 - 32/48 = 1.333$. (Each Newton step multiplies w by $2/3$ for this function, so it is slower here because the curvature vanishes at the minimum.)
8. $\nabla f = (1, 1) = \lambda(2x, 2y)$, so $x = y = 1/(2\lambda)$. The constraint gives $2x^2 = 8$, so $x = y = 2$ (the positive root gives the maximum) and $f = 4$.

Lesson 4: Probability & Statistics

Quiz answers

1. **(b)** Independent flips multiply: $0.5^3 = 1/8$.
2. **(a)** Mean is 4. Squared distances: 4, 0, 4. Average: $8/3 \approx 2.67$.
3. **(c)** The prior is your belief before seeing the evidence. The posterior $P(A \mid B)$ is after.
4. **(b)** $z = (x - \mu)/\sigma = -2$ means x is two standard deviations below the mean.
5. **(a)** 45–55 is within 1 standard deviation of the mean: about 68%.
6. **(b)** For yes/no data the MLE is the observed fraction: $12/20 = 0.6$.
7. **(a)** Add the heads to a and the tails to b: Beta($1 + 3$, $1 + 1$).
8. **(b)** SE $= \sigma/\sqrt{n}$, so halving it needs $\sqrt{n}$ to double, which means n times 4.
9. **(b)** The log of a normal prior is $-\lambda\|\mathbf{w}\|^2$, the ridge penalty. A Laplace prior gives lasso.
10. **(b)** $1 - 0.95^{10} \approx 0.401$.

Exercise solutions

1. 36 equally likely outcomes; 6 give 7: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1). Probability $6/36 = 1/6$.

2. Mean $= 40/8 = 5$. Squared deviations: $9, 1, 1, 1, 0, 0, 4, 16$ → sum 32. Variance $= 32/8 = 4$. Standard deviation $= 2$.
3. $P(\text{winner}) = 0.5 \times 0.2 + 0.05 \times 0.8 = 0.10 + 0.04 = 0.14$.
   $P(\text{spam} \mid \text{winner}) = 0.10/0.14 \approx 0.714$ — about 71%.
4. $z = (85 - 70)/10 = 1.5$. About 95% lie within 2σ: between $70 - 20 = 50$ and $70 + 20 = 90$.
5. $P(4) = e^{-2}\, 2^4/4! = 0.1353 \times 16/24 = 0.0902$.
6. Posterior Beta($2 + 3$, $2 + 7$) = Beta(5, 9). Mean $= 5/14 \approx 0.357$. MAP $= (5 - 1)/(5 + 9 - 2) = 4/12 \approx 0.333$. (The MLE would be $3/10 = 0.3$; the prior pulls both towards 0.5.)
7. $\hat{p} = 0.85$, $\text{SE} = \sqrt{0.85 \times 0.15/1000} = 0.0113$. Interval: $0.85 \pm 1.96 \times 0.0113 = 0.85 \pm 0.0221$, so about $[0.828,\ 0.872]$.
8. $\hat{p}_A = 0.05$, $\hat{p}_B = 0.0625$, pooled $\hat{p} = 450/8000 = 0.05625$. SE $= \sqrt{0.05625 \times 0.94375 \times (2/4000)} = 0.005152$. $z = 0.0125/0.005152 \approx 2.43$, so $p \approx 0.015$.
9. Replace `np.median` with `np.mean` in the resampling loop. The bootstrap gives about $[68.6,\ 88.3]$ and the formula about $[68.2,\ 88.4]$: very similar, because the CLT already applies at $n = 60$. The bootstrap's advantage shows for statistics like the median, where no simple formula exists.

Lesson 5: Data, Features & Generalization

Quiz answers

1. **(c)** A huge gap between low training error and high test error means the model memorised the training data: **overfitting**.
2. **(a)** Both errors are high and similar: the model is too simple to capture the pattern — **underfitting**. Try a more flexible model or better features.
3. **(b)** Choices are made on validation data. The test set is reserved for one final, unbiased evaluation; the training score is too optimistic.
4. **(c)** One per fold: each of the 5 folds takes a turn as the validation set.
5. **(a)** The mean was computed partly from test data. Split first, compute statistics on the training set only, then apply them to the test set.
6. **(a)** Picking the best of many includes some luck. Nested cross-validation (or an untouched test set) gives an honest estimate.
7. **(b)** Split the way the model will be used: it will always predict the future from the past.
8. **(a)** A large gap that is still closing is high variance: more data or a simpler (more regularized) model helps.
9. **(b)** If train and new rows were alike, telling them apart would be impossible (AUC ≈ 0.5).

10. **(b)** Keep each patient's rows together, so the model is scored on patients it has never seen.

Exercise solutions

1. (a) Overfitting — a big gap. (b) Underfitting — both high. (c) A good fit — both low and close.
2. Mean $= 3.65/5 = 0.73$. Deviations $-0.03, 0.01, -0.01, 0.03, 0$; squared sum 0.002; variance 0.0004; std 0.02. Report 0.73 ± 0.02.
3. (a) Leakage — decisions were made using test data. Split first. (b) Leakage — that column is only known *after* a default. (c) Leakage — the model trains on the future. Split by time instead.
4. 7,000 training, 1,500 validation, 1,500 test.
5. Each outer fold runs $3 \times 4 = 12$ settings × 5 inner folds $= 60$ fits. Five outer folds: $5 \times 60 = 300$ fits. (Scikit-learn also refits the best setting once per outer fold, making 305.)
6. The test blocks are rows 21–40, 41–60, 61–80 and 81–100. The first split trains on rows 1–20 (20 rows); the last trains on rows 1–80 (80 rows). The training window grows, and testing is always on later data.
7. High bias: both scores are low and the gap is tiny, and adding data hasn't moved them. Ten times more data is very unlikely to help. Try richer features or a more flexible model.
8. Shuffled k-fold drops to about $R^2 = 0.51$ and the time-ordered split collapses to about -8.26. A straight line in time extrapolates the wandering trend confidently and badly. Shuffled CV still flatters the model by a huge margin; only the time-ordered split shows that neither model can forecast this trend.

Lesson 6: Linear Regression

Quiz answers

1. **(b)** $3 \times 4 - 2 = 10$.
2. **(a)** From $\partial L/\partial b = 0$ we got $b = \bar{y} - w\bar{x}$, i.e. $\bar{y} = w\bar{x} + b$: the line goes through the mean point.
3. **(c)** $R^2 = 0$ means $\mathrm{SS}_{\mathrm{res}} = \mathrm{SS}_{\mathrm{tot}}$: the same error as the mean-only baseline.
4. **(b)** $w = 12/4 = 3$.
5. **(a)** The normal equation works for any number of features, but its cost grows roughly like d^3. Gradient descent scales to huge problems (Lesson 7).
6. **(a)** The VIF multiplies the *variance* of the weight; the standard error is its square root, so it grows by $\sqrt{10}$.
7. **(b)** Only the *errors* matter (LINE). Features can have any distribution: a 0/1 dummy is fine as a feature.
8. **(b)** Adding 1 to the feature multiplies the price by $e^{0.10} = 1.105$, about 10.5% more.

9. **(b)** Cook's distance multiplies the squared residual by a leverage factor. A point on the line has residual ≈ 0, so deleting it hardly changes anything. It's unusual, but not influential.
10. **(b)** Leave one out as the baseline. All 4 would always add up to the bias column (the dummy-variable trap).

Exercise solutions

1. $\bar{x} = 2$, $\bar{y} = 10/3 \approx 3.333$. $\sum(x-\bar{x})(y-\bar{y}) = (-1)(-1.333) + 0 + (1)(1.667) = 3$; $\sum(x-\bar{x})^2 = 2$.
   $w = 3/2 = 1.5$, $b = 3.333 - 1.5 \times 2 = 0.333$. At $x = 4$: $\hat{y} = 6.333$.
2. Predictions $1.833, 3.333, 4.833$; residuals $0.167, -0.333, 0.167$; $\mathrm{SS}_{\mathrm{res}} \approx 0.167$. $\mathrm{SS}_{\mathrm{tot}} = 1.778 + 0.111 + 2.778 = 4.667$.
   $R^2 = 1 - 0.167/4.667 \approx 0.964$.
3. The intercept 12 is the predicted salary at 0 years; each extra year adds about 3.5k on average. It's an association in this data — other factors (role, education) may drive both, so causation isn't established.
4. `X = np.column_stack([np.ones(3), [1, 2, 3]])`
   `y = np.array([2, 3, 5])`
   `print(np.linalg.solve(X.T @ X, X.T @ y))` → `[0.333... 1.5]` (bias, slope).
5. $R^2_{\mathrm{adj}} = 1 - (1 - 0.80) \cdot \frac{50-1}{50-4-1} = 1 - 0.20 \times \frac{49}{45} = 1 - 0.2178 = 0.782$.
6. $\mathrm{VIF} = 1/(1-0.9) = 10$. The variance is multiplied by 10, so the standard error is multiplied by $\sqrt{10} \approx 3.162$.
7. $t = 4.0/1.5 = 2.667$. Interval: $4.0 \pm 1.97 \times 1.5 = 4.0 \pm 2.955$, so $[1.045,\ 6.955]$. It doesn't include 0, so the effect is positive at the 5% level ($p \approx 0.008$).
8. Factor $e^{0.05} = 1.0513$, an increase of about 5.13%. (The shortcut $100 \times 0.05 = 5\%$ is close because w is small.)
9. Build `A` with a column of ones, compute `lev = np.diag(A @ np.linalg.inv(A.T @ A) @ A.T)`, the residuals `e`, `s2 = e @ e / (n - 11)` and `D = e**2 / (11 * s2) * lev / (1 - lev)**2`. The largest is $D \approx 0.0245$ (patient at index 382), and 22 patients exceed $4/442 \approx 0.009$. None is anywhere near 1: no single patient controls this model.

Lesson 7: Gradient Descent

Quiz answers

1. **(b)** $w - \eta \cdot \mathrm{grad} = 2 - 0.1 \times 4 = 1.6$.
2. **(c)** Steps overshoot the minimum by more each time — divergence. Reduce the learning rate (and check feature scaling).
3. **(a)** SGD uses one example at a time; batch uses all; mini-batch uses a small group.

4. **(b)** Different feature scales create long narrow valleys that make descent zigzag. Similar scales give a rounder bowl.
5. **(c)** $\frac{\partial L}{\partial b} = \frac{2}{n}\sum(\hat{y}_i - y_i) > 0$ when predictions are too high, so $b \leftarrow b - \eta(\text{positive})$ decreases — lowering every prediction.
6. **(a)** The velocity averages past gradients: the side-to-side parts cancel and the downhill part accumulates.
7. **(b)** That per-weight scaling is the RMSProp part; the running average of gradients is the momentum part.
8. **(a)** Decaying the learning rate shrinks the noisy steps so the weights can settle.
9. **(b)** Clipping by norm keeps the direction. (Clipping each value separately is a different method, which can change the direction.)
10. **(a)** It protects fragile early training, especially for Adam-trained transformers, before the usual decay.

Exercise solutions

1. Gradient $= \frac{2}{n}\sum(\hat{y} - y)x$.
   Step 1: errors $-2, -4$; gradient $= (-2 \cdot 1 - 4 \cdot 2) = -10$; $w = 0 + 1 = 1$.
   Step 2: errors $-1, -2$; gradient $= -1 - 4 = -5$; $w = 1 + 0.5 = 1.5$. (It's heading for $w = 2$.)
2. Update $w \leftarrow w - 1.5 \cdot 2w = -2w$: $1 \to -2 \to 4 \to -8$. The steps overshoot more each time — **divergence**; for this function you need $\eta < 1$.
3. 100 updates per epoch × 5 epochs = **500 updates**.
4. The loss explodes: after 20 steps $w \approx -575$ and the MSE is in the millions. 0.1 is above this problem's stable limit (about 0.085, as the Lesson 7 demo shows). Standardising x would allow larger rates.
5. Stability needs $\eta < 2/400 = 0.005$. In the gentle direction each step multiplies the error by $1 - \eta \cdot 4 = 1 - 0.02 = 0.98$. Only a 2% reduction per step, so hundreds of steps are needed.
6. $\mathbf{v} = 0.9(1, 0) + (0.5, -1) = (1.4, -1)$. $\mathbf{w} = (2, 3) - 0.1(1.4, -1) = (1.86, 3.1)$.
7. $m = 0.1 \times 4 = 0.4$, $\hat{m} = 0.4/0.1 = 4$. $s = 0.001 \times 16 = 0.016$, $\hat{s} = 0.016/0.001 = 16$. Ratio $= 4/\sqrt{16} = 1$, so the step is 0.001. Adam's first step has size η whatever the gradient's scale.
8. Its length is $\sqrt{36 + 64} = 10$, above 2, so multiply by $2/10 = 0.2$: $(1.2, 1.6)$.

Lesson 8: Logistic Regression

Quiz answers

1. **(b)** $\sigma(0) = 1/(1 + e^0) = 1/2$.

2. **(c)** The boundary is where $z = 0$: $2x - 6 = 0 \Rightarrow x = 3$.
3. **(a)** $-\ln 0.05 \approx 3.0$, much larger than $-\ln 0.5 \approx 0.69$ or $-\ln 0.95 \approx 0.05$. Confident and wrong is punished most.
4. **(b)** The odds are multiplied by $e^{w} = e^{\ln 2} = 2$. The probability itself changes by a varying amount.
5. **(c)** $(p - y)\,x = (0.8 - 0) \times 3 = 2.4$. Positive, so gradient descent decreases w, lowering p for this example.
6. **(b)** $C = 1/\lambda$: a smaller C means a larger penalty on the weights.
7. **(a)** Well calibrated would mean about 90 of the 100 were positive.
8. **(a)** A PSD Hessian everywhere means the loss is convex.
9. **(a)** It trades calibration for recall. Moving the threshold is the alternative that keeps honest probabilities.
10. **(c)** AUC and accuracy depend only on how the scores are ordered or thresholded, not on whether the probability values are right.

Exercise solutions

1. $\sigma(1.5) = 1/(1 + e^{-1.5}) \approx 0.818$. $\sigma(-3) = 1/(1 + e^{3}) \approx 0.047$.
2. Boundary where $z = 0$: $x = 5$. At $x = 7$: $z = 1.6$, $p = \sigma(1.6) \approx 0.832$.
3. $y = 0$: $-\ln(1 - 0.2) = -\ln 0.8 \approx 0.223$ (good prediction, small loss).
   $y = 1$: $-\ln 0.2 \approx 1.609$ (confident and wrong, big loss).
4. $e^{0.405} \approx 1.5$, so the odds are multiplied by 1.5. Odds 1 (probability 0.5) become 1.5, which is probability $1.5/2.5 = 0.6$.
5. $0.5 \times 0.5 = 0.25$; $0.9 \times 0.1 = 0.09$; $0.99 \times 0.01 = 0.0099$. The uncertain point ($p = 0.5$) has the biggest weight, about 25 times the confident one's.
6. Squared errors: 0.01, 0.04, 0.36, 0.36. Mean $= 0.77/4 = 0.1925$.
7. Negatives: $1000/(2 \times 950) \approx 0.526$. Positives: $1000/(2 \times 50) = 10$. Each positive counts 19 times as much as a negative.
8. $\mathbf{y} = (0, 1, 0)$, so $\mathbf{p} - \mathbf{y} = (0.7, -0.8, 0.1)$. The second column's gradient is $\mathbf{x} \times (-0.8) = (-0.8, -1.6)$; gradient descent moves those weights up, raising that class's score.

Lesson 9: Evaluating Models

Quiz answers

1. **(b)** Precision $= 30/(30 + 10) = 0.75$. (Recall would be $30/(30 + 20) = 0.60$.)
2. **(c)** Missing a fraud is a false negative; recall $= TP/(TP + FN)$ measures how many frauds are caught.

3. **(a)** More examples get flagged positive: more true positives are caught (recall ↑) but more false alarms too (precision ↓).
4. **(b)** AUC 0.5 means a random positive is scored above a random negative only half the time — like a coin flip.
5. **(c)** MAE $= 8/4 = 2$. MSE $= (1+1+1+25)/4 = 7$, RMSE ≈ 2.65. Squaring makes the big error count more.
6. **(b)** A 16–14 split is just what coin flips would give. Only the disagreements matter, and they are nearly balanced.
7. **(c)** $c_{FP}/(c_{FP}+c_{FN}) = 1/10 = 0.1$.
8. **(a)** TPR and FPR are each measured within one class, so class balance doesn't change the ROC curve. Precision and average precision do fall.
9. **(b)** A random model's AP equals the positive rate, 0.02, so 0.10 is five times that, though still low in absolute terms.
10. **(b)** Pairing removes the shared variation from the test set's difficulty, so the comparison is much more sensitive.

Exercise solutions

1. Accuracy $= 170/200 = 0.85$. Precision $= 40/50 = 0.80$. Recall $= 40/60 \approx 0.667$. F1 $= 80/(80+10+20) \approx 0.727$. Specificity $= 130/140 \approx 0.929$.
2. (a) Recall — missing a dangerous item is unacceptable. (b) Precision — wrongly blocking accounts is very costly. (c) Accuracy or ROC-AUC is fine.
3. 4 pairs. 0.8 beats 0.7 and 0.2 (2); 0.6 beats 0.2 only (1). AUC $= 3/4 = 0.75$.
4. First: MAE $= 2$, RMSE $= 2$. Second: MAE $= 2$, RMSE $= \sqrt{64/4} = 4$. Same MAE, but RMSE flags the single big miss.
5. There are 35 disagreements, so each would expect 17.5. The statistic is $(25-10)^2/35 = 225/35 \approx 6.43$. Compared with a χ^2 distribution with 1 degree of freedom (critical value 3.84), it is significant at the 5% level ($p \approx 0.011$).
6. $200/(200+5000) = 200/5200 \approx 0.0385$. Flag any transaction with predicted fraud probability above about 3.8%.
7. Precision $= 50/200 = 0.25$. Recall $= 50/80 = 0.625$.
8. Bootstrap `np.mean(logreg.predict(Xte[i]) == yte[i])` instead of the AUC difference. With $p = 0.959$ and $n = 171$, the formula gives $\pm 1.96 \times 0.0152 = \pm 0.030$, roughly $[0.929,\ 0.989]$. The bootstrap interval is similar. Even a 96% accuracy is uncertain by about ±3 points on 171 cases.

Lesson 10: Overfitting, Regularization & Bias–Variance

Quiz answers

1. **(a)** Both errors high and similar = high bias (underfitting). Make the model more flexible; more regularization would make it worse.
2. **(c)** An enormous penalty forces every weight towards zero; only the (unpenalised) bias remains, so predictions are roughly the mean of y.
3. **(b)** Lasso sets unhelpful weights exactly to zero, keeping a small set of useful features.
4. **(b)** $w = 20/(8+2) = 2$. Without regularization it would be $20/8 = 2.5$.
5. **(a)** $C = 1/\lambda$, so a smaller C means a larger λ — stronger regularization.
6. **(a)** $|1.5| - 2 < 0$, so the weight becomes exactly 0. (Ridge would give $1.5/3 = 0.5$.)
7. **(b)** A stronger penalty makes models simpler and more alike: more bias, less variance.
8. **(b)** Lasso's choice within a correlated group is unstable; elastic net tends to keep the group together.
9. **(b)** Roughly, k steps with learning rate η act like ridge with $\lambda \approx 1/(k\eta)$.
10. **(a)** At the interpolation threshold there's a single, usually wild, exact fit. With many more parameters, the smoothest exact fit is chosen.

Exercise solutions

1. High variance (overfitting). Try: stronger regularization (larger λ/alpha), a simpler model or fewer features, more training data (also early stopping for iterative models).
2. $w = 50/(20+\lambda)$: $\lambda = 0 \to 2.5$; $\lambda = 5 \to 2.0$; $\lambda = 30 \to 1.0$.
3. (a) Lasso — it sets most weights to exactly zero. (b) Ridge — it keeps every feature but shrinks them.
4. $\lambda = 1/C = 100$ — strong regularization.
5. Lasso: $2 - 0.5 = 1.5$; $0.4 < 0.5$ so 0; $-1 + 0.5 = -0.5$. Ridge: $2/1.5 = 1.333$, $0.4/1.5 = 0.267$, $-1/1.5 = -0.667$.
6. Bias² $= (4.2 - 5.0)^2 = 0.64$. Variance $= 0.6^2 = 0.36$. Expected error $= 0.64 + 0.36 + 0.25 = 1.25$.
7. L1: $2 \times 0.25 = 0.5$. L2: $2 \times 0.75/2 = 0.75$.
8. Elastic net keeps both features in essentially all 200 bootstrap samples, with similar weights each time. The L2 part removes the arbitrary choice between near-duplicates.

Lesson 11: k-Nearest Neighbours

Quiz answers

1. **(b)** $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$. (7 is the Manhattan distance.)
2. **(a)** Each training point's nearest neighbour is itself, so it's always "correct" — which says nothing about generalization.
3. **(c)** Jagged boundaries mean high variance. Averaging over more neighbours smooths the boundary.
4. **(b)** The average: $(200 + 260 + 230)/3 = 230$k.
5. **(a)** The curse of dimensionality: neighbourhoods must stretch across most of the space, so neighbours aren't really similar.
6. **(b)** When all distances are similar, the nearest neighbours are hardly more similar than random points.
7. **(c)** Cosine distance is $1 - \cos\theta$, which is why it's standard for text and embeddings.
8. **(a)** Each neighbour's vote is weighted by $1/d$.
9. **(a)** In high dimensions, distance concentration means the search can rarely skip regions.
10. **(a)** They usually find most true neighbours, far faster, which is what vector databases need.

Exercise solutions

1. Distances: B $= 1$, A $= \sqrt{5} \approx 2.236$, E $= \sqrt{10} \approx 3.162$, C $= \sqrt{13} \approx 3.606$, D $= \sqrt{18} \approx 4.243$.
   $k = 1$: B → **red**. $k = 3$: B, A, E → 2 red, 1 blue → **red**. $k = 5$: 2 red, 3 blue → **blue**.
2. Euclidean: $\sqrt{3^2 + 4^2} = 5$. Manhattan: $3 + 4 = 7$.
3. Salary — its differences are thousands of times larger. Standardise both features (fitted on training data) before kNN.
4. $0.1^{1/5} \approx 0.63$ — 63% of each feature's range. Hardly "local" any more.
5. $\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\mathbf{a} \cdot \mathbf{b} = 1 + 1 - 2\cos\theta$. With $\cos\theta = 0.8$: $\sqrt{2 - 1.6} = \sqrt{0.4} \approx 0.632$.
6. Uniform: B wins 2 to 1. Distance-weighted: A gets $1/1 = 1$; B gets $1/2 + 1/4 = 0.75$, so A wins.
7. About $n \times d = 10^6 \times 100 = 10^8$ multiply-adds per query. At 10^9 per second that's about 0.1 seconds per query: too slow for live serving, which is why indexes exist.
8. Yes. The ratio falls in the same way (large in 2 dimensions, close to 1 in 1,000), because concentration comes from adding many independent differences, not from the cube's shape.

Lesson 12: Naive Bayes

Quiz answers

1. **(c)** Conditional independence lets us multiply individual feature probabilities instead of modelling every combination.
2. **(b)** Without smoothing, one zero factor in the product wipes out all the other evidence.
3. **(a)** $(4+1)/(20+30) = 5/50 = 0.1$.
4. **(c)** Products of hundreds of small probabilities become too small to store. Logs turn them into sums of manageable numbers, without changing the ranking.
5. **(b)** Continuous measurements are modelled with a normal (Gaussian) distribution per class.
6. **(a)** It's the posterior mean under a symmetric Dirichlet prior.
7. **(b)** The inverse-document-frequency factor $\log(N/\mathrm{df})$ is near 0 for very common words.
8. **(a)** The log-odds is a constant plus a sum of (log-ratio × count) over words.
9. **(b)** It models $P(\mathbf{x} \mid y)$ for each class, then applies Bayes' theorem.
10. **(b)** Generative models like naive Bayes typically reach their best accuracy with far fewer examples.

Exercise solutions

1. $(0+1)/(50+100) = 1/150 \approx 0.0067$ — small, but not zero.
2. Spam: $0.5 \times 0.2 \times 0.01 = 0.001$. Ham: $0.5 \times 0.02 \times 0.1 = 0.001$. A perfect tie: $P(\text{spam}) = 0.5$. The two words' evidence (×10 for spam, ×10 for ham) cancels out.
3. Same σ, so compare $e^{-z^2/2}$. A: $z = 0.75$ → $e^{-0.281} \approx 0.755$. B: $z = -1.25$ → $e^{-0.781} \approx 0.458$.
   $P(A \mid x) = 0.755/(0.755 + 0.458) \approx 0.62$ → **class A**.
4. Classification only needs the *right class to score highest*. Violated independence distorts the size of the probabilities more than their ranking.
5. $(3+1)/(200+1000) = 4/1200 \approx 0.00333$. An unseen word: $1/1200 \approx 0.000833$, small but not zero.
6. $\ln(1000/10) = \ln 100 \approx 4.605$. In every document: $\ln 1 = 0$.
7. $\ln(0.4/0.6) = -0.405$. Total: $-0.405 + 1.5 - 0.6 = 0.495$. $P(\text{spam}) = 1/(1 + e^{-0.495}) \approx 0.621$.

Lesson 13: Decision Trees

Quiz answers

1. **(b)** $1 - (0.5^2 + 0.5^2) = 1 - 0.5 = 0.5$ — the maximum for two classes.

2. **(a)** Pure children have impurity 0 — the maximum possible gain. Keeping the parent's mix gives zero gain.
3. **(c)** Rescaling a feature just rescales its thresholds; the same examples go left and right.
4. **(b)** That gap is overfitting; restricting the tree's growth reduces variance.
5. **(a)** Each split is "one feature ≤ threshold", a cut parallel to an axis. Together they make rectangles.
6. **(b)** $R_\alpha(T) = R(T) + \alpha|\text{leaves}|$: a higher price per leaf favours smaller trees.
7. **(b)** It's measured on training data and favours high-cardinality features.
8. **(a)** Shuffling breaks the feature's link to the target; a useful feature's absence hurts the score.
9. **(b)** Shuffling one leaves the other, so the score barely drops. Group correlated features or use other methods.
10. **(a)** This high variance is exactly what bagging and random forests average away.

Exercise solutions

1. Gini $= 1 - (0.75^2 + 0.25^2) = 0.375$. Entropy $= -(0.75\log_2 0.75 + 0.25\log_2 0.25) \approx 0.811$ bits.
2. Parent Gini $= 0.5$. Each child: $1 - (0.8^2 + 0.2^2) = 0.32$. Weighted: 0.32. Decrease $= 0.5 - 0.32 = 0.18$.
3. 2.1: is $2.1 \le 0.8$? No → is $2.1 \le 1.75$? No → **virginica**. 0.4: $0.4 \le 0.8$ → **setosa**.
4. ```
   from sklearn.datasets import load_iris
   from sklearn.tree import DecisionTreeClassifier, export_text
   iris = load_iris()
   t = DecisionTreeClassifier(max_depth=2, random_state=0).fit(iris.data, iris.target)
   print(export_text(t, feature_names=iris.feature_names))
   ```
   Yes — splits at petal width 0.80 and 1.75.
5. Compare $0.05 + 10\alpha$ with $0.08 + 7\alpha$. The pruned tree wins when $0.08 + 7\alpha < 0.05 + 10\alpha$, i.e. $\alpha > 0.03/3 = 0.01$.
6. Before: $1 - (0.4^2 + 0.6^2) = 0.48$. Left (40 cases): $1 - (0.75^2 + 0.25^2) = 0.375$. Right (60 cases): $1 - ((1/6)^2 + (5/6)^2) = 0.2778$. Weighted: $0.4 \times 0.375 + 0.6 \times 0.2778 = 0.3167$. The decrease is 0.163.
7. A: $0.90 - 0.84 = 0.06$. B: 0. B may be useless, or it may be correlated with another feature the model can use instead.
   ```

Lesson 14: Random Forests & Boosting

Quiz answers

1. **(c)** If models err on different examples, the majority is usually right even when individuals are wrong.
2. **(a)** $1 - (1 - 1/n)^n \approx 1 - e^{-1} \approx 0.632$.
3. **(b)** Feature subsampling decorrelates the trees, making the average stronger.
4. **(c)** The residual is the negative gradient of the squared loss; fitting it is a gradient descent step in prediction space.
5. **(a)** $\alpha = \frac{1}{2}\ln\frac{0.5}{0.5} = \frac{1}{2}\ln 1 = 0$. A coin-flip learner gets no say.
6. **(b)** $\rho\sigma^2 + (1-\rho)\sigma^2/B = \sigma^2$ when $\rho = 1$. Identical trees give no benefit.
7. **(b)** Fewer candidate features per split means more disagreement between trees, at the cost of individual accuracy.
8. **(b)** Each tree corrects half as much. Let early stopping pick the exact number.
9. **(b)** It's a regularized Newton step for each leaf.
10. **(b)** Base models fit their own training rows too well; the blender needs predictions like those on new data.

Exercise solutions

1. Correct when at least 2 of 3 are right: $0.7^3 + 3 \times 0.7^2 \times 0.3 = 0.343 + 0.441 = 0.784$.
2. $(4/5)^5 \approx 0.328$ — about a third, close to the large-n value $e^{-1} \approx 0.368$.
3. $\alpha = \frac{1}{2}\ln\frac{0.8}{0.2} = \frac{1}{2}\ln 4 \approx 0.693$.
4. $F_0 = 15$; residuals $-5, +5$. $F_1 = 15 + 0.1 \times (-5) = 14.5$ and $15 + 0.1 \times 5 = 15.5$.
5. $0.3 + 0.7/10 = 0.37$. As $B \to \infty$: 0.3.
6. $\sum g = 0.8$, $\sum h = 0.64$. $w^* = -0.8/(0.64 + 1) = -0.488$.
7. About twice as many, around 600. Each tree's contribution is halved.

Lesson 15: Support Vector Machines

Quiz answers

1. **(b)** Margin $= 2/\|\mathbf{w}\| = 2/4 = 0.5$.
2. **(a)** $\max(0, 1 - y \cdot \text{score}) = \max(0, 1 - (-1)(0.5)) = 1.5$. It's on the wrong side.
3. **(c)** Only support vectors define the solution; points with zero hinge loss don't affect it.
4. **(b)** That's overfitting. Smaller γ smooths the boundary; smaller C allows more margin violations — both regularize.

5. **(a)** $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{z})$, computed directly from $\mathbf{x}$ and $\mathbf{z}$.
6. **(a)** Only support vectors (on or inside the margin) have non-zero α_i.
7. **(a)** Swap every $\mathbf{x}_i \cdot \mathbf{x}_j$ for $K(\mathbf{x}_i, \mathbf{x}_j)$.
8. **(b)** Large gamma and C give a wiggly, overfit boundary.
9. **(a)** $\max(0, 1 - 2) = 0$: confidently correct points cost nothing.
10. **(a)** Both minimise a regularized loss; log loss is a smooth version of the hinge.

Exercise solutions

1. $\|\mathbf{w}\| = 5$, so the margin is $2/5 = 0.4$.
2. Score -2: $\max(0, 1 - (-1)(-2)) = \max(0, -1) = 0$ — safely correct.
   Score 0.3: $\max(0, 1 + 0.3) = 1.3$ — wrong side.
3. $\mathbf{x} \cdot \mathbf{z} = 5 \to 25$. $\|\mathbf{x} - \mathbf{z}\|^2 = 1 + 4 = 5 \to e^{-0.5 \times 5} = e^{-2.5} \approx 0.082$.
4. Increase γ (more local, flexible boundary) and/or increase C (fewer margin violations allowed) — then check with cross-validation.
5. $\mathbf{x} \cdot \mathbf{z} = 2$, so $K = 3^2 = 9$. With $\phi(\mathbf{x}) = (1, 2\sqrt{2}, 0, 4, 0, 0)$ and $\phi(\mathbf{z}) = (1, \sqrt{2}, 3\sqrt{2}, 1, 9, 3\sqrt{2})$, the dot product is $1 + 4 + 0 + 4 + 0 + 0 = 9$.
6. 0; 0.6; 1.5. The first is beyond the margin (free), the second is inside it, and the third is misclassified.
7. $e^{-0.5 \times 4} = e^{-2} \approx 0.135$; $e^{-0.5 \times 0.25} = e^{-0.125} \approx 0.882$.

Lesson 16: Clustering

Quiz answers

1. **(b)** The update step sets each centroid to the average of its cluster, which minimises that cluster's sum of squared distances.
2. **(c)** More clusters always fit at least as tightly, so inertia alone favours too many clusters. Look for the elbow or use silhouette.
3. **(a)** $(6 - 2) / \max(2, 6) = 4/6 \approx 0.67$.
4. **(b)** DBSCAN follows dense regions of any shape and labels outliers as noise.
5. **(c)** Core points sit in dense regions; clusters grow outwards from them.
6. **(b)** The E-step computes responsibilities; the M-step then updates the means, covariances and weights.
7. **(b)** DBSCAN follows dense connected regions; k-means and Ward prefer compact blobs.
8. **(b)** Its own cluster is farther away on average than the nearest other cluster.
9. **(a)** BIC $= -2 \log L + p \log n$: fit quality plus a penalty for parameters. Lower is better.

10. **(a)** With those assumptions the E-step becomes "nearest centroid" and the M-step becomes "average of assigned points".

Exercise solutions

1. Assign: $\{1, 2\}$ and $\{6, 7, 8\}$ (6 is 5 from 1 but 2 from 8). Update: 1.5 and 7. Re-assigning changes nothing → converged.
   Inertia $= 0.25 + 0.25 + 1 + 0 + 1 = 2.5$.
2. $(5 - 2)/\max(2, 5) = 3/5 = 0.6$.
3. 2 has $\{1, 2, 3\}$ within 1 → **core**. 1 and 3 have only 2 points each, but are within 1 of point 2 → **border**. 6 and 10 → **noise**. One cluster: $\{1, 2, 3\}$.
4. (a) k-means with $k = 5$. (b) DBSCAN (its noise points are the unusual traces). (c) Hierarchical clustering with a dendrogram.
5. $s = (5 - 2)/\max(2, 5) = 3/5 = 0.6$.
6. A: $4800 + 11 \ln 1000 = 4800 + 75.99 = 4875.99$. B: $4760 + 17 \times 6.908 = 4760 + 117.43 = 4877.43$. A has the lower BIC, so it's preferred, narrowly: B's better fit doesn't quite justify 6 extra parameters.
7. Divide by the total 0.20: $(0.1, 0.3, 0.6)$.

Lesson 17: Dimensionality Reduction & PCA

Quiz answers

1. **(b)** PC1 is the direction of maximum variance — the eigenvector of the covariance matrix with the largest eigenvalue.
2. **(c)** $(6 + 3)/(6 + 3 + 1) = 9/10 = 90\%$.
3. **(a)** PCA maximises variance; a feature in millimetres has far more raw variance than the same feature in metres.
4. **(b)** The matrix only stretches $\mathbf{v}$ by the factor 3, so $\mathbf{v}$ is an eigenvector with eigenvalue 3.
5. **(c)** t-SNE expands dense clusters and contracts sparse ones, so visual sizes aren't trustworthy.
6. **(b)** The reconstruction error equals the variance in the dropped components: $1 - 0.93 = 0.07$.
7. **(a)** It's PCA plus the kernel trick.
8. **(b)** Global distances and cluster sizes on a t-SNE map are not reliable.
9. **(b)** Low perplexity focuses on very local structure; high perplexity on broader structure.
10. **(b)** PCA has a `transform` for new data and stable, meaningful axes; t-SNE is for visualisation.

Exercise solutions

1. $\lambda = 2.5 \pm \sqrt{1.5^2 + 2^2} = 2.5 \pm 2.5$: $\lambda_1 = 5$, $\lambda_2 = 0$. PC1 explains 100% — the data lies exactly on a line.
2. Total 16. First two: $12/16 = 75\%$. Three: $14/16 = 87.5\%$. Four: $15/16 \approx 93.8\%$ → **4 components**.
3. $(3 + 1)/\sqrt{2} = 4/\sqrt{2} \approx 2.83$.
4. `from sklearn.datasets import load_wine` … `PCA().fit(StandardScaler().fit_transform(X))`, then `np.argmax(np.cumsum(pca.explained_variance_ratio_) ⩾ 0.9) + 1` → **8**. (The first 2 explain about 55%.)
5. Total $= 10$. Two components: $8/10 = 0.8$. Three: $9.5/10 = 0.95$, so 3 components are needed.
6. The dropped variance: $(0.3 + 0.2)/10 = 0.05$.
7. `PCA(0.90).fit(X).n_components_` gives 21 components (from 64 pixels).

Lesson 18: Neural Networks from Scratch

Quiz answers

1. **(b)** The 1s and 0s lie on opposite diagonals; separating them needs at least two lines combined, i.e. a hidden layer.
2. **(a)** A chain of linear maps is still linear: $V(W\mathbf{x}) = (VW)\mathbf{x}$.
3. **(c)** $\max(0, -3) = 0$.
4. **(b)** $\delta_{\text{out}} = p - y = 0.8$; $\partial L/\partial v_1 = 0.8 \times 0.5 = 0.4$. Positive, so v_1 will decrease.
5. **(a)** Random initialisation "breaks the symmetry" so each hidden neuron can specialise.
6. **(a)** Doubling the variance compensates for the half of the signal ReLU throws away.
7. **(a)** Over 20 layers the product of slopes can be around 10^{-12}.
8. **(b)** So a single example gets the same output whatever else is in the batch.
9. **(b)** $1/(1 - p) = 1/0.8 = 1.25$, so the expected activation is unchanged.
10. **(a)** The pre-activations are huge (variance about 256), so tanh is flat almost everywhere it's evaluated.

Exercise solutions

1. $h_1 = \text{ReLU}(2 - 1) = 1$; $h_2 = \text{ReLU}(-2 + 2) = 0$. $z = 2 \cdot 1 + 3 \cdot 0 - 1 = 1$; $p = \sigma(1) \approx 0.731$.
2. $\delta_{\text{out}} = p - y = 0.731$. $\partial L/\partial v_1 = 0.731 \times h_1 = 0.731$; $\partial L/\partial v_2 = 0.731 \times 0 = 0$ (neuron 2 was inactive).

3. $(10 \cdot 32 + 32) + (32 \cdot 16 + 16) + (16 \cdot 3 + 3) = 352 + 528 + 51 = 931$.
4. No — the loss stays high and at least one point is misclassified. One hidden neuron draws one line, and XOR needs two.
5. Xavier: $\sqrt{1/512} \approx 0.0442$. He: $\sqrt{2/512} = 0.0625$.
6. $0.25^{10} \approx 9.5 \times 10^{-7}$ (vanishing). $1.3^{10} \approx 13.79$ (exploding, and much worse with more layers).
7. Mean 5; variance $(9 + 1 + 1 + 9)/4 = 5$; std ≈ 2.236. Standardised: $(-1.342, -0.447, 0.447, 1.342)$. Output $= 2\hat{z} + 1$: $(-1.683, 0.106, 1.894, 3.683)$.

Lesson 19: Deep Learning Architectures

Quiz answers

1. **(b)** $(32 - 5 + 0)/1 + 1 = 28$.
2. **(c)** Weight sharing: one 3×3 kernel's few weights are reused at every location.
3. **(a)** The gated cell state lets information and gradients flow across many time steps.
4. **(b)** Scores are $\mathbf{q} \cdot \mathbf{k}/\sqrt{d_k}$, then softmax turns them into weights.
5. **(c)** Max pooling keeps the largest value in the window: 5.
6. **(b)** Each row is a softmax: a set of weights for averaging the value vectors.
7. **(a)** Dot-product spread grows like $\sqrt{d}$, pushing the softmax towards a single winner.
8. **(a)** Even if f's Jacobian is tiny, the identity path carries the gradient back.
9. **(b)** Self-attention is permutation-equivariant: it has no built-in notion of order.
10. **(b)** $\Delta W = BA$ with a small rank, which is far fewer numbers to train and store.

Exercise solutions

1. (a) $(32 - 5)/1 + 1 = 28$. (b) $(28 - 2)/2 + 1 = 14$.
2. Each output = top-left − bottom-right of the window: $1 - 1 = 0$, $2 - 3 = -1$, $0 - 1 = -1$, $1 - 0 = 1$. Result $\begin{bmatrix} 0 & -1 \\ -1 & 1 \end{bmatrix}$.
3. $(5 \cdot 5 \cdot 3 + 1) \times 32 = 76 \times 32 = 2{,}432$.
4. Softmax: $e^2/(e^2 + 1) \approx 0.881$ and $1/(e^2 + 1) \approx 0.119$.
5. Weights: $e^2/(e^2 + 1) \approx 0.881$ and $1/(e^2 + 1) \approx 0.119$. Output: $0.881(1, 0) + 0.119(0, 1) = (0.881, 0.119)$.
6. Variance 64, so standard deviation 8. After scaling: $8/8 = 1$.
7. B is 1024×16 and A is 16×1024: $2 \times 16{,}384 = 32{,}768$. The full matrix has $1{,}048{,}576$, so the fraction is 0.03125 (3.1%).

Lesson 20: Feature Engineering, Pipelines & Final Project

Quiz answers

1. **(b)** Arbitrary numbers impose a fake order and spacing. One-hot encoding treats categories as equally different.
2. **(c)** Like all learned preprocessing statistics, it must come from training data to avoid leakage.
3. **(a)** The log compresses the huge range so a few extreme values don't dominate. `log1p` also handles zeros.
4. **(b)** Pipelines prevent leakage and guarantee identical processing in training and deployment.
5. **(c)** Always compare with a baseline; a tiny improvement over random ranking means little useful signal has been captured.
6. **(b)** On pure noise it reported 73% instead of 50%.
7. **(a)** Numbers, categories and text each get their own steps, all fitted on training folds only.
8. **(a)** A grid wastes tries repeating the same few values of the important settings.
9. **(b)** Each order of magnitude should get similar attention; uniform sampling would mostly try values above 0.1.
10. **(b)** Reproducing a model needs all of them.

Exercise solutions

1. $\mu = 20$, $\sigma = \sqrt{(100 + 0 + 100)/3} \approx 8.165$. $z = (40 - 20)/8.165 \approx 2.45$. Training statistics are reused — never recomputed on new data.
2. small → $(0, 0, 1)$; large → $(1, 0, 0)$; medium → $(0, 1, 0)$; small → $(0, 0, 1)$.
3. The scaler was fitted on all data (including test rows) — leakage. Split first and put the scaler inside a `Pipeline` so it's re-fitted on each fold.
4. Add `X["is_new_customer"] = (X["tenure_months"] ⩽ 6).astype(int)`. Compare the cross-validated ROC-AUC with the earlier 0.771. When we ran it, logistic regression stayed at exactly 0.771 — no gain. Tenure is already a feature, and the new column only repeats part of it. A useful lesson: a new feature must add *new* information. Any change smaller than the ± spread (about 0.01) isn't a real improvement.
5. $\bar{y}_{\text{level}} = 0.75$. Encoding $= (4 \times 0.75 + 6 \times 0.3)/(4 + 6) = (3 + 1.8)/10 = 0.48$. The rare level is pulled strongly towards the overall average.
6. $4 \times 5 \times 3 = 60$ settings $\times\ 5 = 300$ fits, but only 4 distinct learning rates. A random search with 60 settings would try 60.
7. The range covers 3 orders of magnitude, and 0.001 to 0.01 is one of them, so $1/3 \approx 0.333$. (Uniform sampling would give only 0.009.)

Lesson 21: Ethics & Fairness in Machine Learning

Quiz answers

1. **(b)** Features like postcode or school are often correlated with protected attributes, so a model can reproduce group differences without the attribute itself.
2. **(a)** $0.30/0.50 = 0.6$, which is below 0.8, so it would be flagged for possible adverse impact.
3. **(c)** Equal opportunity (Hardt, Price & Srebro, 2016) requires that qualified people — the true positives — are approved at the same rate in every group.
4. **(b)** By the identity $\mathrm{FPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR}$, different base rates p force different FPRs.
5. **(a)** This is the Obermeyer et al. (2019) case: groups with less access to care spend less at the same level of need, so the label understates their need.
6. **(b)** From $\mathrm{FPR} = \frac{p}{1-p} \cdot \frac{1-\mathrm{PPV}}{\mathrm{PPV}} \cdot \mathrm{TPR}$, different p forces different FPR.
7. **(b)** In the example, calibrated scores gave TPR 0.232 vs 0.631.
8. **(a)** More people are flagged, including more true negatives: in the example FPR rose from 0.060 to 0.320.
9. **(b)** With about 100 positives the 95% interval spans roughly ±0.08.
10. **(b)** The metrics encode different values; choosing between them is not a technical question.

Exercise solutions

1. A: $60/200 = 0.30$. B: $30/150 = 0.20$. Ratio $= 0.20/0.30 \approx 0.667$, below 0.8, so it would be flagged for possible adverse impact.
2. TPR $= 30/40 = 0.75$. FPR $= 6/60 = 0.10$. Precision $= 30/(30+6) = 30/36 \approx 0.833$.
3. $\frac{0.25}{0.75} \cdot \frac{0.2}{0.8} \cdot 0.6 = 0.3333 \times 0.25 \times 0.6 = 0.05$.
4. Historical bias: past decisions may have favoured some schools or groups, and the model copies them. Measurement bias: 'admitted' is a proxy for 'would succeed'. Representation bias: some groups have few past applicants. Check: compute selection rate and TPR (using later student success as the outcome) per group, with uncertainty for small groups.
5. Lowering B's threshold raises its selection rate; TPR and FPR both rise and precision falls. Equalising one rate moves the others — the trade-off from the impossibility result. The exact numbers depend on the threshold you pick.
6. $\mathrm{FPR} = \frac{0.2}{0.8} \cdot \frac{0.4}{0.6} \cdot 0.75 = 0.25 \times 0.6667 \times 0.75 = 0.125$.
7. $\frac{0.5}{0.5} \cdot \frac{0.4}{0.6} \cdot 0.75 = 1 \times 0.6667 \times 0.75 = 0.5$: four times the first group's, as the identity predicts.
8. $\sqrt{0.25 \times 0.75/100} = 0.0433$; $\pm 1.96 \times 0.0433 = \pm 0.0849$. Interval about $[0.165,\ 0.335]$.

Lesson 22: SQL & Data Wrangling for ML

Quiz answers

1. **(b)** `WHERE` filters rows before grouping; `HAVING` filters the grouped results, for example `HAVING COUNT(*) ⩾ 3`.
2. **(a)** The left join keeps every customer (those without orders get NULLs), and grouping gives one row per customer: 12.
3. **(c)** Each house appears once per matching viewing, so its price is repeated. Aggregate viewings first, then join.
4. **(b)** The window version computes the area average for every row without collapsing the rows.
5. **(a)** Features must be computed only from events before each row's cut-off time; otherwise test scores are optimistic.
6. **(a)** A B-tree index finds matching rows in about log n steps.
7. **(b)** Only the 0 row. For NULL rows, x > 5 is unknown and so is its negation, and WHERE keeps only true conditions.
8. **(b)** Aggregates skip NULLs: the average of 10 and 0 is 5.
9. **(b)** The window must end before the current day, whose data isn't known yet.
10. **(a)** ROWS counts rows. Fill in missing days, or use a RANGE frame on a numeric day column.

Exercise solutions

1. Totals: A = 50 (2 orders), B = 50 (1 order), C = 30 (3 orders). HAVING keeps sums above 40: A (2, 50) and B (1, 50). C is dropped.
2. Inner: one row per order that matches a customer, so 25. Left: the 25 matched rows plus one row for each of the 4 customers without orders, so 29.
3. The area average is 80, so the values are −20, 0 and +20.
4. (a) is fine if counted strictly before the prediction date. (b) leaks if "this year" includes days after the prediction date: count only up to the cut-off. (c) is essentially the outcome itself, so it's pure leakage.
5. `SELECT h.area, COUNT(DISTINCT v.house_id) FROM houses h JOIN viewings v ON v.house_id = h.id GROUP BY h.area` gives Central 3, Outskirts 1 and Suburb 3. `DISTINCT` stops houses with several viewings from counting more than once.
6. Days 2, 3 and 4: $0 + 5 + 1 = 6$.
7. COUNT(*) = 5, COUNT(x) = 3, AVG(x) = $(4 + 6 + 8)/3 = 6$.
8. About $\log_{100} 10^6 = 3$ levels, so 3 node reads, against 1,000,000 rows for a scan.

Lesson 23: Hyperparameter Tuning & Model Selection

Quiz answers

1. **(b)** k is chosen before training. Weights and split thresholds are parameters learned from the data.
2. **(c)** $3^4 = 81$ settings × 5 folds = 405 fits.
3. **(a)** Usually only a few hyperparameters matter; a grid wastes tries repeating the same few values of them.
4. **(b)** The best of many tries is optimistic; the selection used the same folds that produced the score.
5. **(a)** $0.935 - 0.012 = 0.923$, and depth 4's 0.929 is above that, so the simpler tree is preferred.
6. **(b)** The uncertainty is what lets Bayesian optimisation explore sensibly.
7. **(b)** It grows with the uncertainty σ, so unexplored regions get tried.
8. **(b)** Each trial's information matters most when trials are scarce.
9. **(a)** Random search's trials are independent; Bayesian optimisation is sequential by design.
10. **(b)** With $z = 0$: EI $= 0 + 0.02 \times \phi(0) = 0.02 \times 0.399 \approx 0.008$.

Exercise solutions

1. $5 \times 4 \times 2 = 40$ settings $\times 10 = 400$ fits, plus 1 refit: 401.
2. $0.95^n \le 0.1$ gives $n \ge \ln 0.1 / \ln 0.95 = 44.9$, so 45 trials.
3. Round 2: 27 candidates on 30 rows; round 3: 9 on 90; round 4: 3 on 270.
4. Threshold $0.94 - 0.015 = 0.925$. The simplest depth at or above it is depth 3 (0.93).
5. It can: different metrics reward different trade-offs, so the best C often moves. Always tune on the metric you'll actually report and use (Lesson 9).
6. $z = 0.02/0.02 = 1$. EI $= 0.02 \times 0.8413 + 0.02 \times 0.2420 = 0.01683 + 0.00484 = 0.02167$.
7. A: $z = 0$, EI $= 0 + 0.001 \times 0.399 = 0.0004$. B: $z = -0.02/0.05 = -0.4$, EI $= -0.02 \times 0.345 + 0.05 \times 0.368 = -0.0069 + 0.0184 = 0.0115$. B wins: its uncertainty makes it worth exploring.

Lesson 24: Interpretability & Explainable ML

Quiz answers

1. **(b)** Shuffling destroys the feature's information; a big drop means the model relied on it.
2. **(a)** Efficiency: contributions sum to prediction minus baseline.

3. **(c)** The interaction's credit is shared equally between the two features that create it.
4. **(b)** Unhappy customers call support and also leave; the calls may be a symptom, not a cause.
5. **(a)** Parallel curves mean the same shape of effect for each row; crossing or diverging curves reveal interactions.
6. **(b)** Dropping one lets the model use the other. Drop or shuffle them as a group to see their joint importance.
7. **(b)** $w(x - \bar{x}) = 2 \times 3 = 6$.
8. **(a)** That's why it's expensive, but it measures how replaceable each feature's information is.
9. **(b)** Absolute values stop positive and negative contributions cancelling out.
10. **(a)** It explains the model's behaviour; conditional SHAP respects correlations in the data instead.

Exercise solutions

1. A: 0.08, B: 0.01, C: 0. C adds no *unique* information to this model, but it may be correlated with A or B, so it isn't necessarily useless on its own.
2. x_1: own effect 3 plus half the interaction, 3, so 6. x_2: 2 plus 3, so 5. Check: $6 + 5 = 11 = f(1,1) - f(0,0)$.
3. $1! \times 1!/3! = 1/6 \approx 0.1667$. (Size 0 and size 2 coalitions each get $2/6 = 1/3$.)
4. $50 + 12 - 8 - 3 = 51$ lakh.
5. Change `x = X4[200]` to `x = X4[0]`. The printed baseline plus the sum of contributions equals the model's prediction to the second decimal, as the efficiency property guarantees.
6. $\phi_1 = 2(6 - 4) = 4$; $\phi_2 = -3(2 - 1) = -3$. Baseline $f(\bar{\mathbf{x}}) = 10 + 8 - 3 = 15$. $15 + 4 - 3 = 16$, and $f(6, 2) = 10 + 12 - 6 = 16$ ✓.
7. A: 0.01 (largely replaceable). B: 0.20 (unique, important information).

Lesson 25: Imbalanced Data & Anomaly Detection

Quiz answers

1. **(c)** It's right on every non-fraud case: 99.5%. That's why accuracy is the wrong metric here.
2. **(a)** Average precision summarises all thresholds, so it depends only on the ranking of the scores.
3. **(b)** Resampling before splitting puts copies (or near-copies) of the same positives on both sides, which leaks.
4. **(b)** Points that are few and different are separated from the rest quickly, so their average path length is short.

5. **(c)** Points between the clusters are close to the overall mean, so their z-scores look normal.
6. **(b)** Odds $1 \times 0.02/0.98 = 0.0204$, so $p \approx 0.020$.
7. **(a)** Weighting positives up is equivalent to changing the prior class balance.
8. **(b)** Few, different points get separated after few random splits.
9. **(a)** Subsampling keeps it fast and often improves detection.
10. **(b)** Either restores realistic risk estimates.

Exercise solutions

1. Correct: 8 true positives + (990 − 17) true negatives = 981, so accuracy is 0.981. Recall = 8/10 = 0.8. Precision = 8/25 = 0.32.
2. $n = 20{,}000$, $k = 2$. Negatives: $20{,}000/39{,}200 \approx 0.510$. Positives: $20{,}000/800 = 25$.
3. $(2, 4) + 0.25\,(4, -2) = (3, 3.5)$.
4. IQR $= 2$; the fences are $11 - 3 = 8$ and $13 + 3 = 16$. Only 40 is flagged.
5. Very small neighbourhoods make LOF noisy (a few close anomalies can look like a dense cluster); very large ones blur local structure towards a global view. Try both and compare with 20; the best value depends on cluster sizes and is worth tuning on labelled examples.
6. Model odds $= 0.8/0.2 = 4$. Prior odds $= 0.05/0.95 = 0.05263$. Corrected odds $= 0.2105$, so $p = 0.2105/1.2105 \approx 0.174$.
7. $2 \ln 256 = 2 \times 5.545 \approx 11.1$ splits (the exact normalising constant is a little smaller, about 10.2).

Lesson 26: Time Series Forecasting

Quiz answers

1. **(b)** It repeats the value from one season (one week) earlier.
2. **(c)** Neighbouring days are similar, so the model "remembers" nearby values it won't have in real use.
3. **(a)** MASE divides by the seasonal naive error; above 1 means worse than the baseline.
4. **(b)** Shift the series before rolling, so the window ends at day $d - 1$.
5. **(c)** A tree's prediction is an average of training targets, so it can't follow a trend beyond them. Model the change, or detrend.
6. **(a)** Removing the trend lets the seasonal peaks stand out.
7. **(b)** The peaks show a weekly cycle.
8. **(a)** Coverage below the nominal level means it understates uncertainty.
9. **(a)** Relative (log) errors or recent errors fix this.

10. **(a)** Its quantiles of errors give the interval, for any model.

Exercise solutions

1. Naive: 54 (yesterday). Seasonal naive: the value 7 days before today, which is the second value, 52. Moving average: (51 + 53 + 54)/3 = 52.67.
2. MASE = 18/24 = 0.75. Below 1, so it beats the seasonal naive forecast by about 25%.
3. Angle $= 2\pi \times 0.2493 \approx 1.566$ radians, so $\sin \approx 1.000$ and $\cos \approx 0.004$: a quarter of the way round the circle.
4. The five test blocks cover the last 5 × 56 = 280 days, so the first fold trains on 730 − 280 = 450 days.
5. The error rises sharply, towards the naive forecast's, because the weekly pattern is carried mainly by the 7- and 14-day lags (and the weekday dummies). Which lags matter depends on the series' seasonality, so check its autocorrelation.
6. [200 − 12, 200 + 15] = [188, 215].
7. 51/60 = 0.85, below 0.90, so the interval is too narrow.

Lesson 27: Recommender Systems

Quiz answers

1. **(b)** Missing interactions mix "not interested" with "not shown", which is the central difficulty of implicit data.
2. **(a)** Centring removes each user's personal rating scale.
3. **(c)** $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u \cdot \mathbf{q}_i$.
4. **(b)** Collaborative methods need interactions; content features solve cold start.
5. **(a)** Precision@k is relevant items in the top k divided by k. (Recall@k would depend on the total number relevant.)
6. **(a)** That's the closed-form step.
7. **(a)** All users can be solved at once, then all items.
8. **(a)** Coverage is the share of distinct items recommended.
9. **(a)** Recommending them is a safe bet for the metric.
10. **(a)** The ridge regression solution.

Exercise solutions

1. Item bias +0.6, user bias −0.5, so the prediction is 3.5 + 0.6 − 0.5 = 3.6.
2. Dot product 20. Lengths √34 ≈ 5.831 and √20 ≈ 4.472. Cosine = 20/26.08 ≈ 0.767.
3. Dot product 0.96 − 0.20 = 0.76, so the prediction is 3.5 + 0.76 = 4.26.

4. Precision@10 = 4/10 = 0.4. Recall@10 = 4/8 = 0.5.
5. With 1 factor the model underfits (RMSE rises towards the bias baseline); with 20, the training error keeps falling but the test RMSE gets worse unless you raise the regularisation. It's the usual bias–variance trade-off.
6. $Q^\top Q = 4 + 1 = 5$, plus λ gives 6. $Q^\top r = 8 + 3 = 11$. So $p = 11/6 \approx 1.833$.
7. 120/500 = 0.24.

Lesson 28: Bayesian & Probabilistic ML

Quiz answers

1. **(a)** Add successes to a and failures to b: Beta($2 + 3$, $8 + 7$).
2. **(b)** With $\lambda = \sigma^2/\tau^2$. A Laplace prior would give lasso at the MAP.
3. **(c)** With no nearby data, the GP has only its prior to go on.
4. **(a)** Accepting with probability min(1, posterior ratio) needs the posterior only up to a constant.
5. **(b)** Random draws let uncertain designs be tried sometimes, while clearly good ones are chosen most of the time.
6. **(b)** $a + b$ acts as a prior sample size.
7. **(a)** Collect more data, or report the range.
8. **(a)** Chains that agree have $\hat{R}$ near 1.
9. **(a)** Multiple starting points reveal it.
10. **(a)** Values above about 1.01 mean the chains haven't converged.

Exercise solutions

1. Beta(1 + 30, 1 + 170) = Beta(31, 171). Mean = 31/202 ≈ 0.153.
2. Beta(25, 275), mean = 25/300 ≈ 0.083: pulled from 5% towards the prior's 10%, because the prior is worth 200 visitors and the data only 100.
3. λ = σ²/τ² = 4/2 = 2.
4. min(1, 0.12/0.30) = 0.4.
5. It falls, and more traffic goes to the others for longer, because the posteriors overlap for longer and the algorithm keeps exploring. Close options are hard to separate, which is also when choosing the wrong one costs least.
6. Beta(40, 660): mean = 40/700 ≈ 0.0571.
7. $(0.999 \times 1.1 + 0.05)/1.1 = (1.0989 + 0.05)/1.1 = 1.0445$; square root ≈ 1.022: slightly too high.